

Прикладное машинное обучение и искусственный интеллект для инженеров

Решение задач, которые
невозможно решить
алгоритмически



Джеф Просиз

Applied Machine Learning and AI for Engineers

*Solve Business Problems That Can't
Be Solved Algorithmically*

Jeff Prosise

Foreword by Adam Prosise

Beijing • Boston • Farnham • Sebastopol • Tokyo

O'REILLY

Джеф Просиз

Прикладное машинное обучение и искусственный интеллект для инженеров

Решение задач, которые
невозможно решить
алгоритмически

Астана
«АЛИСТ»
2024

УДК 004.438
ББК 32.988.02-018
П82

Просиз Дж.

П82 Прикладное машинное обучение и искусственный интеллект
для инженеров: Пер. с англ. — Астана: АЛИСТ, 2024. — 432 с.: ил.

ISBN 978-601-09-5051-1

Книга рассказывает о применении искусственного интеллекта и машинного обучения в бизнесе и инженерной практике. Подробно описаны популярные алгоритмы машинного обучения и разъяснено, когда их целесообразно использовать. Приведены примеры построения моделей машинного обучения на языке Python с помощью библиотеки Scikit-Learn, а также создания нейронных сетей посредством библиотек Keras и TensorFlow. Изложены базовые принципы и способы оценки регрессионных моделей, моделей бинарной и многоклассовой классификации. Показаны примеры создания модели распознавания лиц и обнаружения объектов, языковых моделей, отвечающих на естественно-языковые вопросы и переводящих текст на другие языки. Рассмотрено использование набора облачных API Cognitive Services для внедрения ИИ в различные приложения.

Для инженеров и разработчиков программного обеспечения

УДК 004.438
ББК 32.988.02-018

© 2024 ALIST LLP

Authorized Russian translation of the English edition of *Applied Machine Learning and AI for Engineers*
ISBN 9781492098058 © 2022 Jeff Prosise.

This translation is published and sold by permission of O'Reilly Media, Inc., which owns or controls all rights to publish and sell the same.

Авторизованный перевод с английского языка на русский издания *Applied Machine Learning and AI for Engineers*
ISBN 9781492098058 © 2022 Jeff Prosise.

Перевод опубликован и продается с разрешения компании-правообладателя O'Reilly Media, Inc.

ISBN 978-1-492-09805-8 (англ.)
ISBN 978-601-09-5051-1 (каз.)

© Jeff Prosise, 2022
© Издание на русском языке. ТОО "АЛИСТ", 2024

Содержание

https://t.me/it_boooks/2

Отзывы о книге

"Прикладное машинное обучение и искусственный интеллект для инженеров"	11
--	----

Об авторе	17
-----------------	----

Об изображении на обложке	18
---------------------------------	----

Предисловие	19
-------------------	----

Введение	21
----------------	----

Кто должен прочитать эту книгу	21
--------------------------------------	----

Почему я написал эту книгу	22
----------------------------------	----

Использование примеров кода из книги	23
--	----

Как организована эта книга	24
----------------------------------	----

Условные обозначения и соглашения	25
---	----

Благодарности	25
---------------------	----

ЧАСТЬ I. МАШИННОЕ ОБУЧЕНИЕ С ПОМОЩЬЮ SCIKIT-LEARN	27
---	----

ГЛАВА 1. Машинное обучение	29
----------------------------------	----

Что такое машинное обучение?	30
------------------------------------	----

Машинное обучение в сравнении с искусственным интеллектом	33
---	----

Обучение с учителем и без учителя	35
---	----

Обучение без учителя с помощью кластеризации k средних	36
--	----

Применение кластеризации k средних к данным о клиентах	39
--	----

Сегментирование клиентов по более чем двум измерениям	43
---	----

Обучение с учителем	47
---------------------------	----

Метод k ближайших соседей	49
-----------------------------------	----

Использование k ближайших соседей для классификации цветов	51
--	----

Резюме	55
--------------	----

ГЛАВА 2. Регрессионные модели	57
-------------------------------------	----

Линейная регрессия	57
--------------------------	----

Деревья решений	61
-----------------------	----

Случайные леса	65
Градиентно-бустинговые машины	66
Метод опорных векторов	69
Меры точности для регрессионных моделей	70
Использование регрессии для прогнозирования тарифов на такси	74
Резюме	79
ГЛАВА 3. Классификационные модели	81
Логистическая регрессия	82
Меры точности для классификационных моделей	84
Категориальные данные	88
Бинарная классификация	90
Классификация пассажиров, плывших на "Титанике"	91
Обнаружение мошенничества с кредитными картами	95
Мультиклассовая классификация	100
Построение модели распознавания цифр	102
Резюме	105
ГЛАВА 4. Классификация текста	107
Подготовка текста к классификации	107
Анализ настроений	111
Наивный Байес	115
Фильтрация спама	118
Рекомендательные системы	122
Косинусное сходство	123
Построение системы рекомендаций фильмов	124
Резюме	126
ГЛАВА 5. Метод опорных векторов	128
Принцип работы метода опорных векторов	128
Ядра	131
Ядерные трюки	132
Настройка гиперпараметров	134
Нормализация данных	138
Конвейеризация	142
Использование SVM для распознавания лиц	143
Резюме	149
ГЛАВА 6. Анализ главных компонент	151
Что такое анализ главных компонент	152
Фильтрация шумов	159
Анонимизация данных	161
Визуализация высокоразмерных данных	163

Обнаружение аномалий.....	167
Использование PCA для обнаружения мошенничества с кредитными картами.....	168
Использование PCA для прогнозирования выхода из строя подшипников.....	172
Обнаружение многомерных аномалий	178
Резюме.....	178
ГЛАВА 7. Операционализация моделей машинного обучения	180
Обращение к Python-модели из Python-клиента	181
Версионирование файлов Pickle	184
Обращение к Python-модели из C#-клиента.....	185
Сохранение модели машинного обучения в контейнере	187
Использование ONNX для преодоления языковой лакуны	189
Построение моделей машинного обучения на C# с помощью ML.NET	193
Анализ настроек с помощью ML.NET	194
Сохранение и загрузка моделей ML.NET.....	197
Добавление возможностей машинного обучения в Excel	197
Резюме.....	201
Часть II. ГЛУБОКОЕ ОБУЧЕНИЕ С ПОМОЩЬЮ KERAS И TENSORFLOW.....	203
ГЛАВА 8. Глубокое обучение.....	205
Понимание нейронных сетей	206
Обучение нейронных сетей.....	211
Резюме.....	213
ГЛАВА 9. Нейронные сети	215
Построение нейронных сетей с помощью Keras и TensorFlow	216
Определение размеров нейронной сети.....	219
Использование нейронной сети для прогнозирования стоимости проезда в такси	221
Бинарная классификация с помощью нейронных сетей	225
Выполнение прогнозирования.....	227
Обучение нейронной сети для обнаружения мошенничества с кредитными картами.....	228
Мультиклассовая классификация с помощью нейронных сетей	232
Обучение нейронной сети для распознавания лиц	235
Отсев.....	238
Сохранение и загрузка моделей.....	240
Обратные вызовы Keras.....	241
Резюме.....	244

ГЛАВА 10. Классификация изображений с помощью сверточных нейронных сетей	246
Понимание работы сверточных нейронных сетей	247
Использование Keras и TensorFlow для построения CNN	250
Обучение CNN распознаванию арктических диких животных	255
Предварительно обученные CNN	260
Использование ResNet50V2 для классификации изображений	263
Трансферное обучение	265
Использование трансферного обучения для идентификации диких животных Арктики	268
Аугментация данных	272
Аугментация изображений с помощью <i>ImageDataGenerator</i>	272
Аугментация изображений с помощью аугментационных слоев	275
Применение аугментации для изображений дикой природы Арктики	276
Глобальный пулинг	279
Классификация аудио с помощью CNN	281
Резюме	288
ГЛАВА 11. Обнаружение и распознавание лиц	290
Обнаружение лиц	291
Обнаружение лиц с помощью алгоритма Виолы — Джонса	291
Использование реализации алгоритма Виолы — Джонса в OpenCV	294
Обнаружение лиц с помощью сверточных нейронных сетей	296
Извлечение лиц из фотографий	300
Распознавание лиц	302
Применение трансферного обучения для распознавания лиц	303
Совершенствование трансферного обучения с помощью весов для конкретной задачи	307
ArcFace	310
Собираем все вместе: обнаружение и распознавание лиц на фотографиях	311
Работа с неизвестными лицами: классификация по закрытым и открытым наборам	317
Резюме	319
ГЛАВА 12. Обнаружение объектов	320
R-CNN	321
Mask R-CNN	324
YOLO	330
YOLOv3 и Keras	332
Обнаружение нестандартных объектов	337
Обучение пользовательской модели обнаружения объектов с помощью службы Custom Vision	337
Использование экспортированной модели	345
Резюме	347

ГЛАВА 13. Обработка естественного языка	349
Подготовка текста	350
Эмбединги слов	353
Классификация текста	354
Автоматизация векторизации текста	357
Использование <i>TextVectorization</i> в модели анализа настроений	358
Учет порядка слов в прогнозах	361
Рекуррентные нейронные сети (RNN)	362
Использование предварительно обученных моделей для классификации текста	364
Нейронный машинный перевод	366
Кодировщик-декодировщик LSTM	366
Трансформерные кодировщики-декодировщики	369
Построение NMT-модели на основе трансформера	371
Использование предварительно обученных моделей для перевода текста	380
Двунаправленное представление кодировщика на основе трансформеров (BERT)	381
Построение системы ответов на вопросы на основе BERT	382
Тонкая настройка BERT для выполнения анализа настроений	386
Резюме	390
ГЛАВА 14. Azure Cognitive Services	392
Знакомство с Azure Cognitive Services	393
Ключи и конечные точки	395
Обращения к API-интерфейсам Azure Cognitive Services	398
Контейнеры Azure Cognitive Services	401
Сервис Computer Vision	402
Сервис Language	412
Сервис Translator	415
Сервис Speech	416
Собираем все вместе: Contoso Travel	418
Резюме	423
Предметный указатель	424

Отзывы о книге "Прикладное машинное обучение и искусственный интеллект для инженеров"

Эта книга — фантастическое руководство по машинному обучению и алгоритмам искусственного интеллекта. Она лаконичная и в то же время исчерпывающая, а конкретные примеры с рабочим кодом показывают, как применять теорию на практике.

— *Марк Руссинович,*
технический директор Azure и технический специалист Microsoft

Когда Джеф Просиз чем-то увлечен (будь то технологии, его домашний желтоногий амазон Хоукай или его хобби — постройка и полеты радиоуправляемых самолетов), вам обязательно захочется его послушать. Он сочетает эту страсть с ясностью изложения, что позволяет ему общаться и преподавать сложные темы лучше, чем кому-либо другому, кого я когда-либо знал. Он увлекает вас за собой в путешествие по пути познания и понимания. Теперь Джеф переносит эти навыки на происходящее и продолжающееся "технологическое цунами" (по его словам) машинного обучения и искусственного интеллекта. В этой новой книге он выстраивает ваше понимание с самых основ, всегда подчеркивая интуитивный подход и связь концепций и решений с реальным миром. Если вы хотите понять, как на самом деле работают искусственный интеллект и машинное обучение, как развивались и развиваются эти технологии, ПРОЧИТАЙТЕ ЭТУ КНИГУ.

— *Todd Файн,*
директор по стратегии компании Atmosera

Джеф превращает многолетние знания в области искусственного интеллекта и машинного обучения в практическое и понятное руководство для специалистов-практиков всех уровней.

— *Кен Мьюз,*
4x Azure MVP и старший архитектор DevOps компании GitHub

"Прикладное машинное обучение и искусственный интеллект для инженеров" — это книга, которая должна была быть у меня, когда я только начинал знакомиться с машинным обучением. Это фантастическое руководство для начинающих инженеров по машинному обучению и отличный справочник для более опытных. Теперь это мой основной ресурс для освежения и углубления понимания различных методов машинного обучения и их применимости. Мне нравится, что Джеф использует реальные примеры и наборы данных для демонстрации каждого набора инструментов и подхода. Как и в случае с нейронной сетью, я понятия не имею, как именно Джеф пришел к этому материалу, но в любом случае он чрезвычайно полезен.

— Брент Ректор,
главный менеджер технических программ, Amazon;
основатель компаний Wise Owl Consulting, LLC, Wise Owl Aviation Services,
LLC и Rector Aviation Law PC

Я знаю Джефа уже несколько десятилетий, и он всегда обладал способностью просто и понятно объяснять сложные концепции. И ему снова удалось это сделать в книге "Прикладное машинное обучение и искусственный интеллект для инженеров". Примеры, аналогии и цветные рисунки делают материал по-настоящему понятным как для начинающих, так и для более продвинутых читателей.

— Джефффри Рухтер,
архитектор программного обеспечения, Microsoft

Эта книга заполняет чрезвычайно важный пробел для инженеров и ученых, которые хотят использовать возможности искусственного интеллекта для решения своих самых сложных задач анализа данных. В ней достигнут правильный баланс между технической глубиной и практическим применением, представлены инструменты и множество практических примеров, позволяющих читателю стать эффективным практиком искусственного интеллекта в своей сфере работы (включая бизнес, науку и другие области, богатые данными).

— Шон С. Глисон,
доктор философии, директор отдела киберустойчивости и интеллекта
Национальной лаборатории Оук-Ридж

Книга "Прикладное машинное обучение и искусственный интеллект для инженеров" может стать основным пособием для энтузиастов в области машинного обучения и искусственного интеллекта. Отличительной чертой этой книги является то, насколько актуальны формулировки задач в условиях современного стремительно внедрения машинного обучения в мире технологий.

Обязательно к прочтению как новичкам, так и профессионалам!

— Липи Дипаакши Патнаик,
инженер по разработке программного обеспечения, Zeta Suite

Джефу всегда удавалось обернуть замечательные истории в глубокие технические концепции, которые заметно облегчают процесс обучения. Возможно, это его лучшая книга и, возможно, самая актуальная по тематике. Прочитав эту книгу, вы захотите пойти и разработать что-нибудь.

— Даг Тернур,
специалист по Azure, Microsoft

"Прикладное машинное обучение и искусственный интеллект для инженеров" — это практическое руководство по созданию полезных систем машинного обучения. В нем в доступной форме рассказывается о том, как применять передовые алгоритмы искусственного интеллекта для решения современных бизнес-задач.

— Брайан Спиринг,
преподаватель науки о данных, Metis

Эта книга — идеальный старт для инженеров и разработчиков программного обеспечения, желающих освоить машинное обучение. Она хорошо освещает основы машинного обучения и позволяет быстро приступить к решению широкого круга задач на основе данных, а не алгоритмов.

— Доктор Манджит Дахия,
*вице-президент и руководитель отдела искусственного интеллекта
и машинного обучения, Ecom Express*

Независимо от того, являетесь ли вы новичком в прикладном машинном обучении или опытным разработчиком, ищущим справочник, эта книга — обязательное, современное, исчерпывающее руководство по всем основным классам алгоритмов машинного обучения (с качественными реализациями кода, позволяющими по настоящему укрепить ваше понимание).

— Гоку Мохандас,
основатель компании Made With ML

Книга "Прикладное машинное обучение и искусственный интеллект для инженеров" — это блестящий учебник для начинающих и продолжающих изучать искусственный интеллект и машинное обучение. Книга построена по удобной схеме — от обычного машинного обучения к глубокому обучению с приятным бонусом в виде реализации искусственного интеллекта в облаке, что делает ее полным универсальным руководством для любого читателя-энтузиаста или профессионала-практика.

— Сатъярт Правин,
*инженер по вычислительной технике,
Национальная лаборатория Лоуренса Беркли*

Джеф Просиз — один из лучших преподавателей, с которыми я когда-либо сталкивался, и независимо от платформы (аудитория, блог, статья в журнале, вебинар, книга и т. д.) он обладает особым талантом брать сложные темы и делать их доступными для всех нас. В этой книге Джеф не просто четко объясняет основные концепции, лежащие в основе машинного обучения, но и приводит простые примеры, демонстрирующие эти концепции в реальных условиях. Он предлагает отличное введение/обзор широкого спектра тем машинного обучения и дает четкие рекомендации по использованию каждой из них. Джеф — один из немногих, кто смог взять эту сложную тему и изложить ее в такой форме, чтобы ее можно было легко усвоить и применить. Эта книга — **MUST READ** для инженеров и других специалистов по решению проблем, которые хотят использовать машинное обучение для расширения своего набора навыков.

— Ларри Клемент,
*доцент кафедры вычислительной техники, программного обеспечения
и наук о данных, бывший заведующий кафедрой Калифорнийского баптистского
университета. Ранее работал старшим инженером — научным сотрудником
корпорации Boeing по программе C-17*

В книге "Прикладное машинное обучение и искусственный интеллект для инженеров" использован легендарный стиль преподавания Джефа Просиза, который за годы работы помог тысячам разработчиков. Настоятельно рекомендуется инженерам, которые хотят включить концепции и навыки машинного обучения в свой арсенал.

— Вани Мандава,
*руководитель инженерного отдела Центра разработки
научного программного обеспечения Института электронной науки
Вашингтонского университета*

Всем моим прошлым и настоящим членам семьи Wintellect

Об авторе

Джеф Просиз (Jeff Prosis) — инженер, чья страсть — знакомить других инженеров и разработчиков программного обеспечения с чудесами искусственного интеллекта и машинного обучения. Соучредитель компании Wintellect, он написал девять книг и сотни статей в журналах, обучил тысячи разработчиков в Microsoft и выступал на крупнейших мировых конференциях по программному обеспечению. До этого Джеф занимался мощными лазерными системами и исследованиями в области термоядерной энергетики в Национальной лаборатории Оук-Ридж и Ливерморской национальной лаборатории имени Лоуренса. В свободное время он собирает и запускает большие радиоуправляемые самолеты, а также посещает лучшие в мире места для дайвинга. После продажи своей компании в 2021 году Джеф занимает должность директора по обучению в Atmosera, где он помогает клиентам внедрять искусственный интеллект в их продукты.

Об изображении на обложке

Птица на обложке книги — это праздничный попугай (*Amazona festiva*), известный как праздничный амазон. Попугаи этого вида обитают в тропических лесах, лесополосах и прибрежных мангровых зарослях нескольких стран Южной Америки, включая Бразилию, Колумбию, Эквадор, Перу и Боливию. Их редко можно встретить вдали от воды.

Праздничные попугаи — это ярко, можно даже сказать, празднично окрашенные птицы среднего размера. Их оперение преимущественно ярко-зеленое, слегка желтеющее к краям крыльев. Их головы украшает пестрая гамма цветов, включая красный, синий, а иногда желтый или оранжевый.

Праздничные попугаи — очень социальный вид, обычно их наблюдают парами или небольшими стайками. Большие группы птиц часто собираются по ночам на общих насестах или вокруг локального источника пищи и отличаются невероятным шумом. Они с удовольствием питаются фруктами, такими как манго и плоды персиковой пальмы, а также ягодами, орехами, семенами, цветами и листовыми почками, дополняющими их рацион.

Хотя праздничные попугаи все еще относительно распространены там, где их лесная среда обитания остается в основном нетронутой, они отнесены Международным союзом охраны природы (МСОП, IUCN) к находящейся под угрозой исчезновения категории фауны из-за продолжающейся вырубке лесов и прогнозируемого сокращения среды обитания. Многие из животных, представленных на обложках O'Reilly, находятся под угрозой исчезновения; все они важны для мира.

Иллюстрация обложки выполнена Карен Монтгомери (Karen Montgomery) на основе старинной линейной гравюры из "Иллюстрированной естественной истории" ("Illustrated Natural History") Вуда.

Предисловие

Когда твой отец проявляет неумное любопытство к тому, что ты изучаешь, то дом — это не тихая гавань.

В 2018 году я получил степень магистра в области аналитики. В ходе обучения в магистратуре моя группа училась использовать машинное обучение, искусственный интеллект и аналитику для повышения ценности бизнеса и разработки решений для реальных задач. У меня страсть к таким вещам — страсть, которую я разделяю с моим отцом, Джефом Просизом. На самом деле, я не могу сказать, сколько раз он спрашивал, может ли он присоединиться ко мне на занятиях (я не шучу), или обрушивал на кухне залп вопросов о том, что мы изучаем, когда я сбегал от рутинной работы в дом моих родителей.

Если вы когда-нибудь летали на самолете в условиях турбулентности, а сидящий рядом с вами человек заводил разговор о том, каким чудом инженерной мысли являются современные реактивные лайнеры, поскольку у них нет ни одной точки отказа, вы прекрасно поймете, что я чувствовал.

Наша любовь к данным и аналитике переросла в общую любовь к той ценности, которую они создают. Используя инструменты и методики, описанные в этой книге, можно обрести уверенность перед лицом неопределенности. Наука о данных позволяет найти скрытую правду — узнать, что происходит на самом деле и как это влияет на поведение и результаты. Способность заглянуть за кулисы, используя аналитику, а не интуицию, является ценным навыком в нашей информационной экономике и жизненно необходима для людей и организаций, ориентирующихся в современной неопределенности.

Не менее важно эффективно донести эти выводы до нетехнической аудитории, имея при этом глубокое техническое понимание того, что происходит "под капотом". Такой уровень общения невозможно подделать (я пару раз пытался это сделать во время кухонных разговоров с отцом).

Аналитика, искусственный интеллект и машинное обучение не имеют непреодолимых технических проблем или проблем с внедрением. Скорее, препятствием является сложность и доступность понимания того, что происходит и как это работает, поскольку это понимание часто окутано техническим жаргоном и отраслевыми

"шибболетами"¹. Эти барьеры выступают в качестве ограничивающего фактора и тормозят использование науки о данных для решения проблем и вопросов.

Именно это и призвана изменить данная книга: она снимает пелену с жаргона, в результате чего эти инструменты и ресурсы становятся доступными.

И, если быть до конца честным, мой отец очень хорошо умеет писать и преподавать пугающие технические темы так, что их изучение становится легким, и так было на протяжении всей моей жизни. Он сделал так, что все темы, начиная с DOS, стали понятными для воротил современного делового мира. Он сделал темы, которые, честно говоря, не укладываются у меня в голове, доступными для поколений программистов последних десятилетий. Проще говоря, он лучший.

Секрет заключается в следующем: его подход к преподаванию той или иной темы основан на принципе "Как бы я хотел, чтобы мне это объяснили, если бы я никогда об этом не слышал, но заинтересовался?". Учитывая уникальные задачи, которые ставит перед собой наука о данных, и огромное количество точек зрения специалистов в этой области, его подход обеспечивает такой уровень доступности, который не встречается во многих других источниках.

Послушайте меня: лучшего путеводителя по запутанным вопросам машинного обучения и искусственного интеллекта вам не найти. Если вы уже знакомы с ними, то эта книга поможет вам лучше понять их, как это произошло со мной. Если вы интересуетесь машинным обучением, искусственным интеллектом, аналитикой и тем, какую пользу они приносят человечеству, то вы попали по адресу.

Независимо от того, к какому лагерю вы относитесь, вы получите более глубокие знания по изложенным в книге темам, которые позволят вам использовать эти инструменты и затем рассказать о том, что вы нашли.

Обычно я заканчиваю подобную статью словами: "Я надеюсь, что книга вам понравится и вы чему-то научитесь", но в данном случае это было бы неправдой. Я не надеюсь — я знаю. Я знаю, что вы будете учиться у моего отца и рядом с ним, как это делал я.

В моей жизни не было человека, на которого я старался быть похожим больше, чем на Джефа Просиза, и мне не терпится поделиться с вами этой чертой. И, может быть, это зажжет в вас страсть к знаниям, как это произошло со мной.

Я оставляю вас с этой мыслью. Я скажу вам то, что он апокрифически сказал мне, когда держал меня на руках в день моего рождения: "Добро пожаловать на шоу, малыш".

— Адам Просиз,
специалист по процессам и инновациям, Delta Air Lines

¹ Шибболёт, шибболет (ивр. שִׁבּוּלֵת, "колос" или "течение") — библейское выражение, в переносном смысле обозначающее характерную речевую особенность, по которой можно опознать группу людей (в частности, этническую), своеобразный "речевой пароль", который неосознанно выдает человека, для которого язык — неродной. (См. Википедию.) — *Прим. пер.*

Введение

За свою жизнь я был свидетелем трех великих технических революций: сначала персональный компьютер, затем Интернет и, наконец, смартфон. Машинное обучение (machine learning, ML) и искусственный интеллект (artificial intelligence, AI) имеют такое же фундаментальное значение, как и все эти три революции, и окажут столь же глубокое влияние на нашу жизнь.

Впервые я заинтересовался машинным обучением в тот день, когда компания, обслуживающая мою кредитную карту, позвонила и уточнила, действительно ли я пытаюсь купить ожерелье стоимостью 700 долларов. Это было не так, но мне стало любопытно: как они узнали, что это не я? Я пользуюсь своей картой по всему миру и, между прочим, время от времени покупаю своей жене приятные вещи. Ни разу кредитная компания не отклонила законную покупку, но несколько раз она безошибочно фиксировала мошеннические покупки, причем до этого кто-то в Бразилии пытался использовать мою карту для покупки авиабилета. В этот раз все было иначе: ювелирный магазин находился в двух милях от моего дома. Я попытался представить себе алгоритм, который мог бы так надежно выявлять мошенничество с кредитными картами в местах продаж. Не сразу я понял, что здесь работает нечто более мощное, чем простой алгоритм.

Оказалось, что компания, выпускающая кредитные карты, прогоняет каждую транзакцию через сложную модель машинного обучения, которая невероятно хорошо справляется с выявлением мошенничества. Этот момент изменил мою жизнь. Это великолепный пример того, как ML и AI делают мир лучше. Более того, понимание, как машинное обучение может анализировать транзакции по кредитным картам в режиме реального времени и отсеивать плохие транзакции, позволяя при этом проводить законные платежи, стало для меня горой, на которую мне предстояло подняться.

Кто должен прочитать эту книгу

Недавно мне позвонил руководитель инженерного отдела одной производственной компании. Он начал разговор следующим образом: "До прошлой недели я не знал, что такое ML и AI. Теперь мой генеральный директор поручил мне выяснить, как они могут улучшить наш бизнес, и сделать это до того, как нас опередят конкуренты. Я начинаю с нуля. Можете ли вы помочь?"

Следующий звонок поступил от государственной подрядной организации, заинтересованной в использовании машинного обучения для выявления налогового мошенничества и отмывания денег. Сотрудники компании были достаточно хорошо знакомы с теорией машинного обучения, но задавались вопросом, как лучше построить необходимые им модели. Профессионалы повсюду осознают, что ML и AI представляют собой технологическое цунами, и пытаются успеть подняться на вершину волны, пока она не обрушилась на них. Эта книга для инженеров, разработчиков программного обеспечения, ИТ-менеджеров и других людей, чья цель — сформировать практическое понимание ML и AI и использовать эти знания для решения проблем, которые раньше были сложными или даже неразрешимыми. Цель курса — привить интуитивное понимание и прибегать к уравнениям только в случае необходимости. Несмотря на то что вы, возможно, слышали, нет необходимости быть экспертом в области исчислений или линейной алгебры, чтобы создавать системы, распознающие объекты на фотографиях, переводящие английский язык на французский или разоблачающие наркоторговцев и налоговых мошенников.

Почему я написал эту книгу

Внутри каждого автора живет маленький гномик, который говорит, что он может рассказать историю так, как никто другой. Свою первую книгу о компьютерах я написал более 30 лет назад, а последнюю — более 20 лет назад, и не собирался писать еще одну. Но теперь мне есть что рассказать. Это важная история, которую должен услышать каждый инженер и разработчик программного обеспечения. Меня не совсем устраивает то, как ее рассказывали другие, поэтому я написал книгу, которую хотел бы иметь, когда учился этому ремеслу. Она начинается с основ и поведет вас в путешествие к вершинам ML и AI. К концу книги вы поймете, как компании, обслуживающие кредитные карты, выявляют мошенничество, как авиационные компании используют машинное обучение для прогнозируемого обслуживания реактивных двигателей, как самодвижущиеся автомобили видят окружающий мир, как Google Translate переводит текст с одного языка на другой и как работают системы распознавания лиц. Более того, вы сможете сами создавать подобные системы или использовать существующие системы для внедрения искусственного интеллекта в приложения, которые вы пишете.

Наиболее продвинутые ML-модели сегодня обучаются на компьютерах, оснащенных графическими процессорами (graphics processing units, GPU) или тензорными процессорами (tensor processing units, TPU), что зачастую требует больших затрат времени и средств. В этой книге особое внимание уделяется примерам, которые могут быть построены на обычном персональном компьютере или ноутбуке без GPU. Когда мы перейдем к моделям компьютерного зрения, распознающим объекты на фотографиях, я расскажу, как работают такие модели и как они обучаются на миллионах изображений на кластерах GPU. Но затем я покажу, как с помощью техники, называемой *трансферным обучением*, перепрофилировать существующие модели для решения специфических задач и обучать их на обычном ноутбуке.

Эта книга в значительной степени основана на материалах занятий и семинаров, которые я провожу в компаниях и исследовательских институтах по всему миру. Я люблю преподавать, потому что мне нравится видеть, как загораются лампочки. Я часто начинаю занятия по ML и AI словами: "Я здесь не для того, чтобы учить, я здесь для того, чтобы изменить вашу жизнь". Надеюсь, что ваша жизнь станет немного другой и чуть лучше, чем до прочтения этой книги.

Использование примеров кода из книги

Инженеры лучше всего учатся *на практике*, а не посредством чтения. Данная книга содержит множество примеров кода, которые можно выполнить для закрепления концепций, представленных в каждой главе. Большинство из них написаны на языке Python и используют популярные библиотеки с открытым исходным кодом, такие как Scikit-Learn, Keras и TensorFlow. Все они доступны в публичном репозитории GitHub (<https://oreil.ly/applied-machine-learning-code>). Это единственный достоверный источник примеров кода, поскольку я могу обновлять его в любое время.

Существуют платформы машинного обучения, которые позволяют строить и обучать модели без кода. Но лучший способ понять, что и как делают эти платформы, — это написать код. Python — простой язык программирования. Его легко освоить. Сегодня инженеры должны уметь писать код. Вы можете изучить Python в процессе работы над примерами, приведенными в этой книге, и если вы уже умеете работать с Python (и программировать в целом), то будете впереди.

Для того чтобы запустить мои примеры на компьютере или ноутбуке, необходима 64-разрядная версия Python 3.7 или выше. Вы можете загрузить среду выполнения Python с сайта **Python.org** (<https://oreil.ly/kMjiJ>) или установить дистрибутив Python, например Anaconda (<https://oreil.ly/4NCqN>). Также необходимо убедиться, что установлены следующие пакеты и их зависимости:

- ◆ Scikit-Learn и TensorFlow для построения моделей машинного обучения;
- ◆ Pandas, Matplotlib и Seaborn для работы с данными и их визуализации;
- ◆ OpenCV и Pillow для работы с изображениями;
- ◆ Flask и Requests для вызова REST API и создания веб-сервисов;
- ◆ Sklearn-onnx и Onnxruntime для моделей Open Neural Network Exchange (ONNX);
- ◆ Librosa для генерации изображений спектрограмм из аудиофайлов;
- ◆ MTCNN и Keras-vggface для создания систем распознавания лиц;
- ◆ KerasNLP, Transformers, Datasets и PyTorch для построения моделей обработки естественного языка (NLP);
- ◆ Azure-cognitiveservices-vision-computervision, Azure-ai-textanalytics и Azure-cognitiveservices-speech для вызова Azure Cognitive Services.

Большинство из этих пакетов можно установить с помощью команды `pip install`. Если вы установили Anaconda, то многие из этих пакетов уже присутствуют, а остальные можно установить с помощью команд `conda install` или аналогичных.

Говоря об окружениях, излишним будет использовать виртуальные окружения Python, чтобы избежать конфликтов при установке пакетов. Если вы не знакомы с виртуальными окружениями, то можете прочитать о них на сайте **Python.org** (<https://oreil.ly/sm7ZO>). Если вы используете Anaconda, то виртуальные среды уже встроены в этот дистрибутив.

Большинство моих примеров кода были созданы для блокнотов Jupyter, которые представляют собой интерактивную платформу для написания и выполнения кода на языке Python. Блокноты невероятно популярны в сообществе специалистов по науке о данных для изучения данных и обучения моделей машинного обучения. Блокноты Jupyter можно запускать локально, установив такие пакеты, как Notebook (<https://oreil.ly/ZWQyG>) или JupyterLab (<https://oreil.ly/5A3Ia>), или использовать облачные среды, например Google Colab (<https://oreil.ly/RdRBa>). Одно из преимуществ Colab заключается в том, что вам не нужно ничего устанавливать на свой компьютер — даже Python. А в редких случаях, когда моим примерам требуется GPU, Colab предоставляет его вам.

Среды разработки на Python, как известно, очень сложны в настройке и сопровождении, особенно под Windows. Если вы предпочитаете не создавать такую среду, или если вы пытались, но не смогли заставить ее работать, помощь можно скачать. Я упаковал полную среду разработки, пригодную для запуска всех примеров из этой книги, в образ контейнера Docker (<https://oreil.ly/wzEbA>). Если на компьютере установлен движок Docker Engine (<https://oreil.ly/XO5GD>), то контейнер можно запустить следующей командой:

```
docker run -it -p 8888:8888 jeffpro/applied-machine-learning:latest
```

Перейдите в браузере по URL-адресу, который появится в выводе. Вы попадете в полноценную среду Jupyter со всеми моими примерами кода и всем необходимым для их выполнения. Они находятся в папке Applied-Machine-Learning, клонированной из одноименного репозитория на GitHub. Недостатком использования контейнера является то, что вносимые изменения по умолчанию не сохраняются. Один из способов исправить это — использовать ключ `-v` в команде `docker` для привязки к локальному каталогу. Более подробная информация приведена в разделе "Использование привязки монтирования" ("Use Bind Mounts", <https://oreil.ly/7wgda>) документации по Docker.

Как организована эта книга

Данная книга состоит из двух частей.

- ♦ В *части I* (главы 1–7) рассказывается об азбуке машинного обучения и представлены такие популярные алгоритмы обучения, как логистическая регрессия и градиентный бустинг.

- ♦ *Часть II (главы 8–14)* посвящена глубокому обучению, которое сегодня является синонимом искусственного интеллекта и использует глубокие нейронные сети для подбора математических моделей к данным.

Я настоятельно рекомендую вам выполнять упражнения по ходу чтения книги. Вы глубже поймете материал и, несомненно, придумаете, как модифицировать мои примеры для игры в "что если?" с кодом.

Файл с цветными иллюстрациями к этой книге можно загрузить по адресу: <https://files.alistbooks.kz/978-601-09-5051-1.zip>.

Условные обозначения и соглашения

В данной книге используются следующие типографские соглашения:

- ♦ *курсив* указывает на новые термины;
- ♦ **полужирный шрифт** указывает на URL, адреса электронной почты, элементы интерфейса программ и элементы веб-интерфейса;
- ♦ **моноширинный шрифт** используется в листингах программ, а также внутри абзацев для обозначения таких элементов программы, как имена переменных и функций, базы данных, типы данных, переменные среды, операторы и ключевые слова.
- ♦ **полужирный моноширинный шрифт** указывает команды или другой текст, который следует набрать пользователю буквально, или зарезервированные ключевые слова языка Python.



Данный элемент обозначает подсказку или совет.



Данный элемент обозначает общее замечание.

Благодарности

Написание и издание книги — это командная работа. Она начинается с автора, но прежде чем попасть на полки магазинов, она проходит через рецензентов, редакторов-разработчиков, редакторов-копировщиков, художников и производственный персонал.

Я хотел бы поблагодарить нескольких друзей и членов семьи за рецензирование глав по мере их написания и конструктивные замечания. Они инженеры, математики, аналитики данных и профессора и, как оказалось, одни из самых умных людей,

которых я знаю: Ларри Клемент (Larry Clement), Манджит Дахия (Manjeet Dahiya), Том Маршалл (Tom Marshall), Дон Мейер (Don Meyer), Гоку Мохандас (Goku Mohandas), Кен Мьюз (Ken Muse), Липи Дипаакши Патнаик (Lipi Deepaakshi Patnaik), Чарльз Петцольд (Charles Petzold), Эбби Просиз (Abby Prosise), Адам Просиз (Adam Prosise), Джеффри Рихтер (Jeffrey Richter), Брюс Шектер (Bruce Schecter), Вишвеш Рави Шримали (Vishwesh Ravi Shrimali), Брайан Спиринг (Brian Spiering) и Рон Сумида (Ron Sumida).

Огромная благодарность команде O'Reilly за то, что она превратила мои слова в прозу, а мои наброски — в искусство. Это Джилл Леонард (Jill Leonard), Одри Дойл (Audrey Doyle), Грегори Хайман (Gregory Hyman), Дэвид Футато (David Futato), Карен Монтгомери (Karen Montgomery) и Николь Баттерфилд (Nicole Butterfield). Особые слова благодарности Джону Хасселлу (Jon Hassell), который услышал мое видение этой книги и сказал: "Давай сделаем это". За годы работы мне посчастливилось сотрудничать с несколькими замечательными издательскими командами. Ни одна из них не была лучше этой.

И наконец, Лори — моей жене, спутнице и партнеру по "преступлению" на протяжении последних 40 лет. Без тебя я бы не справился. И я *обещаю*, что это будет последняя книга, которую я напишу!



- ЧАСТЬ I -

Машинное обучение с помощью Scikit-Learn

Машинное обучение

https://t.me/it_books/2

Машинное обучение (machine learning, ML) расширяет границы возможного, позволяя компьютерам решать задачи, которые еще несколько лет назад были неразрешимыми. Машинное обучение ежедневно оказывает влияние на нашу жизнь: от выявления мошенничества и медицинской диагностики до рекомендаций товаров и автомобилей с машинным зрением (которые "видят", что находится перед ними). Пока вы читаете эту главу, ученые используют машинное обучение для раскрытия секретов человеческого генома. Когда мы однажды найдем способ излечить больных от рака, мы будем благодарить машинное обучение за то, что оно сделало это возможным.

Машинное обучение является революционным, поскольку представляет собой альтернативу алгоритмическому решению задач. Имея рецепт, или *алгоритм*, несложно написать приложение, которое хеширует пароль или рассчитывает ежемесячный платеж по ипотеке. Вы разрабатываете алгоритм, подаете ему входные данные и получаете в ответ выходные. Совсем другое дело — создать код, определяющий, кто изображен на фотографии: кошка или собака. Вы можете попытаться реализовать идею алгоритмически, но как только вы начнете распознавать реальные образы, вы столкнетесь с фото кошки или собаки, которое нарушит алгоритм.

Машинное обучение использует другой подход к преобразованию входных данных в выходные. Вместо того чтобы полагаться на вас в реализации алгоритма, оно изучает набор входных и выходных данных и учится генерировать собственные выходные данные в процессе, называемом *обучением*. "Под капотом" специальные алгоритмы, называемые *алгоритмами обучения*, подстраивают математические модели под данные и устанавливают взаимосвязь между входными и выходными данными. После обучения модель может принимать новые входные данные и генерировать выходные данные, согласующиеся с теми, которые были получены в процессе обучения.

Для того чтобы использовать машинное обучение для различения кошек и собак, не нужно придумывать алгоритм "кошка против собаки". Вместо этого вы создаете модель машинного обучения на основе фотографий кошек и собак. Успех зависит от используемого алгоритма обучения, а также от качества и объема обучающих данных.

Частью работы инженера по машинному обучению является знакомство с различными алгоритмами обучения и развитие интуиции, позволяющей понять, когда

следует использовать тот или иной алгоритм. Эта интуиция приходит с опытом и пониманием того, как машинное обучение подстраивает математические модели под данные. Настоящая глава — первый шаг на этом пути. Она начинается с обзора машинного обучения и наиболее распространенных типов моделей машинного обучения, а завершается знакомством с двумя популярными алгоритмами обучения и их использованием для построения простых, но полнофункциональных моделей.

Что такое машинное обучение?

На самом деле *машинное обучение* (ML) — это поиск закономерностей в числах и использование этих закономерностей для построения прогнозов. Машинное обучение позволяет обучать модель, используя ряды или последовательности 1 и 0, и учиться на этих данных так, чтобы, получив новую последовательность, модель могла предсказать, каким будет результат. Обучение — это процесс, в ходе которого ML находит закономерности, возможные для использования с целью предсказания будущих результатов, и именно отсюда происходит слово "обучение" в термине "машинное обучение".

В качестве примера рассмотрим таблицу 1 и 0, изображенную на рис. 1.1. Каждое число в четвертом столбце каким-то образом основано на трех предшествующих ему числах в той же строке. Какого числа не хватает?

0	1	0	0
1	1	0	1
1	1	1	1
0	0	0	0
0	1	1	

Рис. 1.1. Простой набор данных, состоящий из 0 и 1

Одно из возможных решений заключается в том, что для данной строки, если в первых трех столбцах больше 0, чем 1, то четвертый столбец содержит 0. Если в первых трех столбцах больше 1, чем 0, то ответ равен 1. По этой логике пустой столбец должен содержать 1. Специалисты по исследованию данных называют столбец, содержащий ответы (красный столбец на рисунке), *столбцом меток*. Остальные столбцы — это *столбцы признаков*. Задача прогнозирующей модели — найти в строках столбцов признаков закономерности, позволяющие предсказать, какой будет метка.

Если бы все наборы данных были настолько простыми, то машинное обучение было бы не нужно. Однако в реальном мире наборы данных больше и сложнее. Что если набор данных содержит миллионы строк и тысячи столбцов (а это, как оказывается, часто встречается в машинном обучении)? А если бы набор данных был похож на тот, что показан на рис. 1.2?

[illegible]

Рис. 1.2. Более сложный набор данных

Человеку трудно изучить этот набор данных и выработать набор правил для предсказания того, что именно должно быть в красном поле (в правом нижнем углу): 0 или 1. (Нет-нет, это не просто подсчет единиц и нулей.) Представьте себе, насколько сложнее было бы, если бы набор данных действительно состоял из миллионов строк и тысяч столбцов.

Именно в этом и заключается суть машинного обучения: поиск закономерностей в огромных массивах данных с числами. Неважно, 100 строк или 1 000 000 строк. Во многих случаях лучше больше, поскольку ста строк может оказаться недостаточно для выявления закономерностей.

Не будет упрощением сказать, что машинное обучение решает проблемы путем математического моделирования закономерностей в наборах чисел. Практически любая проблема может быть сведена к набору чисел. Например, одним из распространенных применений ML сегодня является анализ настроений: рассматривается образец текста, например рецензия на фильм или комментарий, оставленный на сайте, и присваивается 0 баллов за негативные настроения (например, "Еда была безвкусной, а обслуживание — ужасным") или 1 балл за позитивные настроения ("Отличная еда и обслуживание. Не могу дождаться, чтобы посетить ресторан снова!"). Некоторые отзывы могут быть неоднозначными, например "Бургер был великолепен, но картофель фри был мокрым", поэтому в качестве оценки настроения мы используем *вероятность* того, что отзыв будет равен 1. Очень негативный отзыв может получить 0,1 балла, а очень позитивный — 0,9 балла, т. е. вероятность того, что он выражает позитивное настроение, составляет 90%.

Анализаторы настроения и другие модели, работающие с текстом, часто обучаются на наборах данных, подобных приведенному на рис. 1.3, в которых одна строка соответствует каждому образцу текста, а один столбец — каждому слову в массиве текста (все слова в наборе). Типичный набор данных, подобный этому, может содержать миллионы строк и 20 000 или более столбцов.

В каждой строке в столбце меток¹ стоит 0, если настроение негативное, или 1, если настроение позитивное. Внутри каждой строки приведены подсчеты слов — количество раз, когда данное слово встречается в отдельной выборке. Набор данных разрежен, т. е. в нем в основном 0, но иногда встречаются и ненулевые числа. Но машинному обучению не важен состав чисел. Если есть закономерности, которые можно использовать для определения того, выражает ли следующий образец позитивные или негативные настроения, ML их найдет. Спам-фильтры используют подобные наборы данных, в которых 1 и 0 в столбце меток обозначают спам и неспам. Такой подход позволяет современным спам-фильтрам достигать поразительной точности. Более того, со временем эти модели становятся все более "умными", поскольку обучаются на все большем количестве писем.

Рис. 1.3. Набор данных для анализа настроений

Анализ настроения — это пример задачи *классификации текста*: анализ текстового образца и его классификация как позитивного или негативного. Машинное обучение доказало свою эффективность и в *классификации изображений*. Простой пример классификации изображений — просмотр фото кошек и собак и классификация каждого из них как кошачьего (0) или собачьего (1). В реальной жизни классификация изображений может использоваться для выявления сходящих с конвейера бракованных деталей, идентификации объектов в поле зрения самостоятельно движущегося автомобиля и распознавания лиц на фотографиях.

Модели классификации изображений обучаются на наборах данных, подобных приведенному на рис. 1.4, в которых каждая строка представляет собой изображение, а каждый столбец — значение пиксела. Набор данных, содержащий 1 млн изображений шириной и высотой по 200 пикселей, содержит 1 млн строк и 40 000

¹ Крайний правый столбец на рис. 1.3. — Прим. ред.

столбцов. Всего это 40 млрд чисел, или 120 млрд, если изображения цветные, а не полутоновые. (В цветных изображениях значения пикселей состоят не из одного, а из трех чисел.) Столбец меток содержит число, обозначающее класс или категорию, к которой относится соответствующее изображение — в данном случае человек, чье лицо изображено на картинке: 0 — Герхард Шредер, 1 — Джордж Буш и т. д.

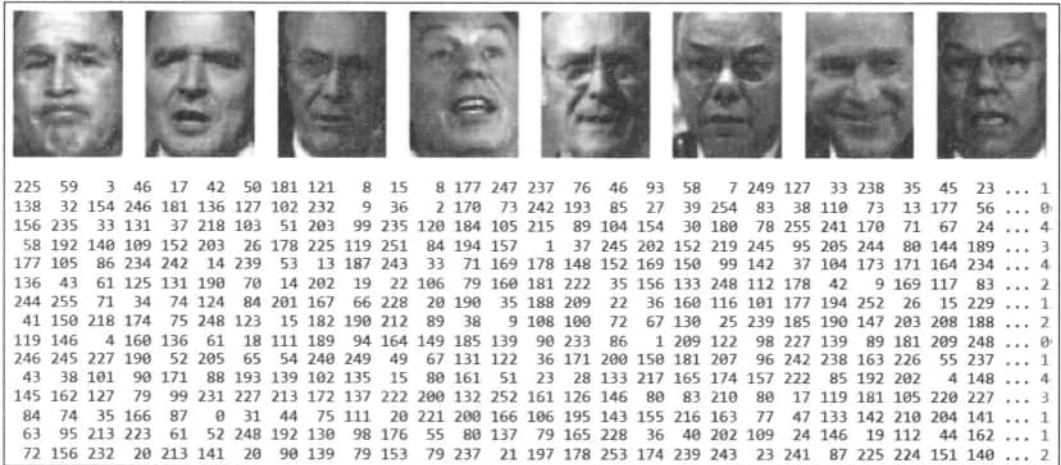


Рис. 1.4. Набор данных для классификации изображений

Приведенные изображения лиц взяты из известного общедоступного набора данных Labeled Faces in the Wild, или сокращенно LFW (<https://oreil.ly/YVIv2>). Это один из бесчисленных наборов данных с метками, которые публикуются в Интернете для всеобщего пользования. Машинное обучение не представляет собой ничего сложного, если у вас есть маркированные наборы данных, с которыми другие (чаще всего аспиранты) кропотливо работали часами, пометая их 1 и 0. В реальном мире инженеры иногда тратят большую часть своего времени на создание таких наборов данных. Одним из наиболее популярных хранилищ общедоступных наборов данных является сайт **Kaggle.com**, на котором размещено множество полезных наборов данных и проводятся конкурсы, позволяющие начинающим специалистам в области ML проверить свои навыки.

Машинное обучение в сравнении с искусственным интеллектом

Термины "*машинное обучение*" и "*искусственный интеллект*" (artificial intelligence, AI) сегодня используются практически как взаимозаменяемые, однако на самом деле каждый из них имеет собственное значение, как показано на рис. 1.5.

С технической точки зрения *машинное обучение* — это подмножество искусственного интеллекта, которое включает в себя не только модели машинного обучения, но и другие типы моделей, такие как *экспертные системы* (системы, принимающие решения на основе заданных вами правил) и *системы обучения с подкреплением*.

ем, которые учатся поведению, получая вознаграждение за положительные результаты и штрафы за отрицательные. Примером системы обучения с подкреплением является AlphaGo (<https://oreil.ly/uLwpd>), которая стала первой компьютерной программой, обыгравшей профессионального игрока в го. Она тренируется на уже сыгранных партиях и самостоятельно осваивает стратегии победы.

С практической точки зрения то, что большинство людей сегодня называют искусственным интеллектом, на самом деле является глубоким обучением, которое представляет собой подмножество машинного обучения. *Глубокое обучение* (deep learning, DL) — это машинное обучение с использованием нейронных сетей. (Существуют формы глубокого обучения без использования нейронных сетей — одним из примеров являются глубокие машины Больцмана, но в подавляющем большинстве случаев глубокое обучение сегодня осуществляется с применением нейронных сетей.) Таким образом, модели машинного обучения можно разделить на обычные модели, использующие алгоритмы обучения для моделирования закономерностей в данных, и модели глубокого обучения на основе нейронных сетей.



Рис. 1.5. Взаимосвязь между машинным обучением, глубоким обучением и искусственным интеллектом

Краткая история искусственного интеллекта

В последние годы популярность машинного обучения и искусственного интеллекта резко возросла. В 1980-х годах искусственный интеллект стал настоящей сенсацией, и было широко распространено мнение, что компьютеры вскоре смогут имитировать человеческий разум. Но затем ажиотаж спал, и в течение десятилетий — примерно до 2010 г. — искусственный интеллект редко попадал в новости. Затем произошла странная вещь.

Благодаря доступности графических процессоров (graphics processing units, GPU; <https://oreil.ly/tiYd0>) от таких компаний, как NVIDIA, у исследователей наконец-то появилась необходимая мощность для обучения продвинутых нейронных сетей. Это привело к улучшению состояния дел в данной области, что вызвало новый энтузиазм, привело к дополнительному финансированию, ускорившему дальнейшее развитие, и внезапно искусственный интеллект снова стал реальностью. Нейронные сети существуют (по крайней мере, в теории) с 1950-х годов, но исследователям не хватало вычислительных мощностей для их обучения на больших массивах данных. Сегодня каждый может купить графический процессор или создать кластер GPU в облаке. Искусственный интеллект сейчас развивается быстрее, чем когда-либо прежде, и с этим прогрессом появляется возможность создавать в программном обеспечении такие вещи, о которых еще десять лет назад инженеры могли только мечтать.

Со временем ученые, изучающие данные, разработали специальные типы нейронных сетей, которые отлично справляются с определенными задачами, включая задачи, связанные с компьютерным зрением (например, извлечение информации из изображений), и задачи, связанные с человеческими языками (например, перевод с английского на французский). В *главе 8* мы подробно рассмотрим нейронные сети, и вы узнаете, как именно глубокое обучение позволило поднять машинное обучение на новую высоту.

Обучение с учителем и без учителя

Большинство моделей машинного обучения делятся на две большие категории: модели обучения с учителем (контролируемого) и без учителя (неконтролируемого). Цель *моделей обучения с учителем* — делать предсказания. Вы обучаете их с помощью помеченных данных, чтобы они могли принимать последующие входные данные и предсказывать, какие метки будут получены. Большинство используемых сегодня моделей ML являются моделями контролируемого обучения. Отличным примером служит модель почтовой службы США для преобразования рукописных почтовых индексов в цифры, которые компьютер может распознать для сортировки почты. Другой пример — модель для авторизации покупок, которую использует выпускающая кредитные карты компания.

Модели обучения без учителя, напротив, не требуют маркированных данных. Их цель — дать представление о существующих данных или сгруппировать данные по категориям и соответствующим образом классифицировать будущие данные. Классическим примером неконтролируемого обучения является анализ записей о приобретенных в компании товарах и покупателях, которые их приобрели, для определения того, какие клиенты могут быть наиболее заинтересованы в новом запускаемом продукте, и последующего построения направленной на этих клиентов маркетинговой кампании.

Спам-фильтр — это модель контролируемого обучения. Для ее работы требуются помеченные данные. Модель, которая сегментирует клиентов на основе доходов,

кредитных баллов и истории покупок, является неконтролируемой моделью, и данные, которые она потребляет, не обязательно должны быть помечены. Для того чтобы наглядно показать разницу, в оставшейся части этой главы мы более подробно рассмотрим обучение с учителем и обучение без учителя.

Обучение без учителя с помощью кластеризации k средних

В обучении без учителя часто используется техника, называемая *кластеризацией*. Цель кластеризации — сгруппировать данные по сходству. Наиболее популярным алгоритмом кластеризации является *кластеризация по методу k средних* (k-means clustering, <https://oreil.ly/8duYJ>), при которой берут n образцов данных и группируют их в m кластеров, где m — заданное число.

Группировка осуществляется с помощью итерационного процесса, в ходе которого для каждого кластера вычисляется центроид, и образцы распределяются по кластерам в зависимости от их близости к центроидам кластеров. Если расстояние от конкретного образца до центроида первого кластера равно 2,0, а расстояние от того же образца до центра второго кластера равно 3,0, то образец относится к первому кластеру. На рис. 1.6 показано, как 200 образцов нечетко распределены по трем кластерам. На диаграмме слева представлены исходные, не сгруппированные образцы. На диаграмме справа — центроиды кластеров (красные точки²), а образцы раскрашены по кластерам.

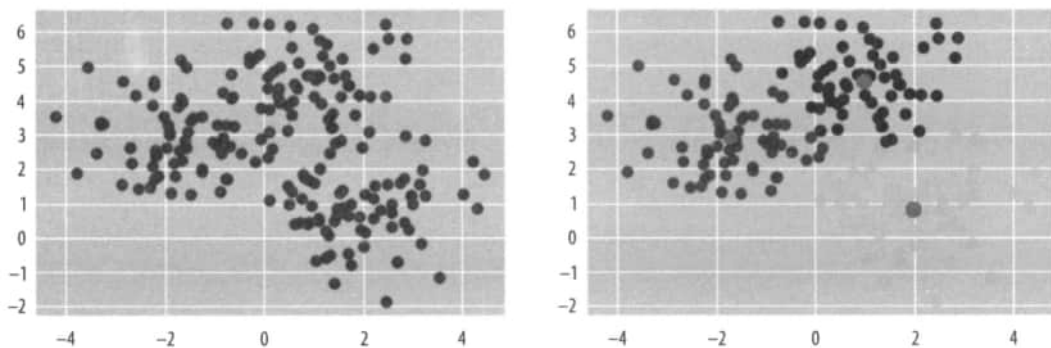


Рис. 1.6. Точки данных, сгруппированные с помощью кластеризации k средних

Как написать модель обучения без учителя с использованием кластеризации k средних? Проще всего это сделать с помощью самой популярной в мире библиотеки машинного обучения Scikit-Learn (<https://oreil.ly/bSQ2>). Она бесплатна, имеет открытый исходный код и написана на языке Python. Документация по ней великолепна, и если у вас возникнет вопрос, то, скорее всего, погуглив, вы найдете на него

² Цветные изображения можно скачать со страницы книги на сайте издательства bhv.ru. — Прим. ред.

ответ. Я буду использовать Scikit³ для большинства примеров в первой половине этой книги. Во *введении* к книге описано, как установить Scikit и настроить свой компьютер для запуска моих примеров (или использовать для этого контейнер Docker), так что если вы еще не сделали этого, то сейчас самое время настроить свою среду.

Для того чтобы освоить кластеризацию k средних, начните с создания нового блокнота Jupyter и вставьте в первую ячейку следующие команды:

```
%matplotlib inline
import matplotlib.pyplot as plt
import seaborn as sns
sns.set()
```

Запустите эту ячейку, а затем выполните следующий код в следующей ячейке для генерации полуслучайного набора пар координат x и y . Этот код использует функцию `make_blobs` из Scikit (<https://oreil.ly/h5sIB>) для создания пар координат и функцию `scatter` из Matplotlib (<https://oreil.ly/bnmNw>) для их построения:

```
from sklearn.datasets import make_blobs
points, cluster_indexes = make_blobs(n_samples=300, centers=4,
                                     cluster_std=0.8, random_state=0)

x = points[:, 0]
y = points[:, 1]
plt.scatter(x, y, s=50, alpha=0.7)
```

Ваш результат должен быть идентичен моему (рис. 1.7) благодаря параметру `random_state`, который инициализирует генератор случайных чисел, используемый `make_blobs`:

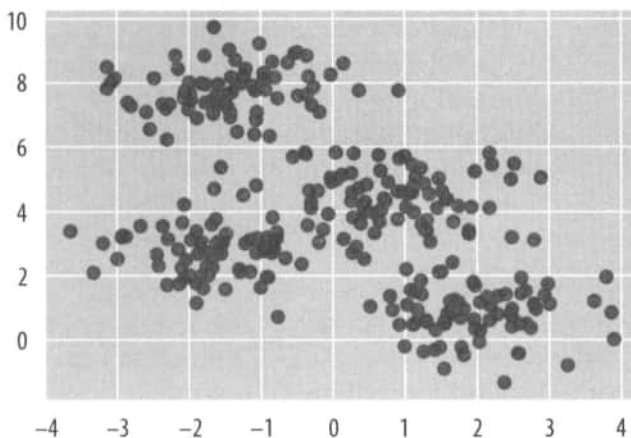


Рис. 1.7. Первый результат

³ Автор книги использует названия Scikit-Learn и Scikit как синонимы. Это одна и та же библиотека. — Прим. ред.

Далее с помощью кластеризации k средних разделите пары координат на четыре группы. Затем выделите центральные точки кластеров красным цветом и распределите точки данных по кластерам. Класс `KMeans` (<https://oreil.ly/wgm9z>) из Scikit выполняет основную работу, и после того, как ему предоставлены пары координат, вы можете получить местоположение центроидов из атрибута `cluster_centers_` класса `KMeans`:

```
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=4, random_state=0)
kmeans.fit(points)
predicted_cluster_indexes = kmeans.predict(points)

plt.scatter(x, y, c=predicted_cluster_indexes, s=50, alpha=0.7, cmap='viridis')

centers = kmeans.cluster_centers_
plt.scatter(centers[:, 0], centers[:, 1], c='red', s=100)
```

Результат приведен на рис. 1.8.

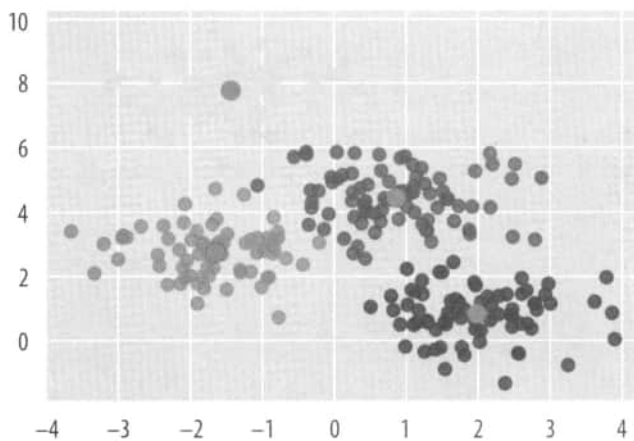


Рис. 1.8. Второй результат

Попробуйте задать для `n_clusters` другие значения, например 3 и 5, чтобы посмотреть, как группируются точки при разном количестве кластеров. В связи с этим возникает вопрос: как определить *правильное* количество кластеров? Ответ не всегда очевиден при взгляде на график, а если данные имеют более трех измерений, то построить график в любом случае не удастся.

Одним из способов выбора нужного числа кластеров является *метод локтя*, который показывает *инерцию* (сумму квадратов расстояний точек данных до ближайшего центра кластера), полученную из `KMeans.inertia_`, как функцию от количества кластеров.

Постройте инерции таким образом и найдите наиболее острый локоть кривой:

```
inertias = []

for i in range(1, 10):
    kmeans = KMeans(n_clusters=i, random_state=0)
    kmeans.fit(points)
    inertias.append(kmeans.inertia_)
```

```
plt.plot(range(1, 10), inertias)
plt.xlabel('Количество кластеров')
plt.ylabel('Итерация')
```

В данном примере, по-видимому, 4 кластера являются правильным выбором (рис. 1.9).

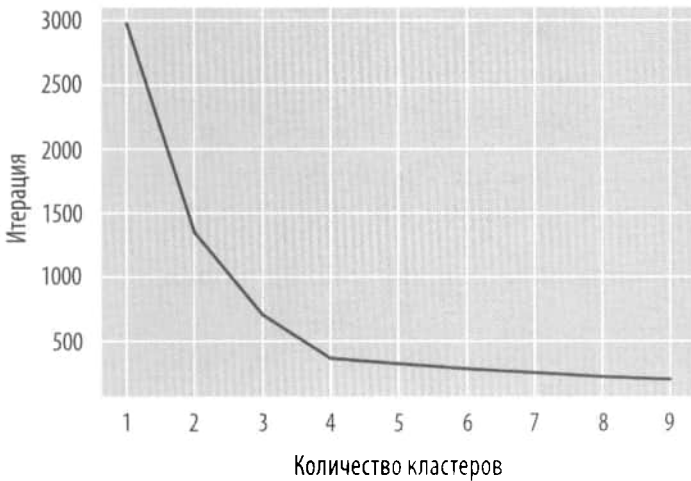


Рис. 1.9. Третий результат

В реальной жизни локоть может быть не таким четким. Это нормально, поскольку, кластеризуя данные различными способами, вы иногда получаете информацию, которую не смогли бы получить в противном случае.

Применение кластеризации k средних к данным о клиентах

Давайте применим кластеризацию k средних для решения реальной задачи: сегментирования клиентов с целью выявления тех, на кого следует направить рекламную акцию для повышения их покупательской активности. В качестве базы данных будет использован пример сегментации клиентов с именем `customers.csv`. Начните с создания подкаталога `Data` в папке, где находятся ваши блокноты, загрузите файл `customers.csv` (<https://oreil.ly/Md86Y>) и скопируйте его в подкаталог `Data`. Затем

используйте следующий код для загрузки набора данных в Pandas DataFrame и отображения первых пяти строк:

```
import pandas as pd
```

```
customers = pd.read_csv('Data/customers.csv')
customers.head()
```

Из полученного результата (рис. 1.10) вы видите, что набор данных содержит пять столбцов⁴, два из которых описывают годовой доход клиента и его рейтинг расходов. Последний представляет собой значение от 0 до 100. Чем больше число, тем существеннее клиент потратился на вашу компанию в прошлом.

CustomerID	Gender	Age	Annual Income (k\$)	Spending Score (1-100)	
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

Рис. 1.10. Первый результат
в задаче сегментирования клиентов

Теперь с помощью следующего кода постройте график годовых доходов и расходов:

```
%matplotlib inline
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
sns.set()
```

```
points = customers.iloc[:, 3:5].values
```

```
x = points[:, 0]
```

```
y = points[:, 1]
```

```
plt.scatter(x, y, s=50, alpha=0.7)
```

```
plt.xlabel('Ежегодный доход, $')
```

```
plt.ylabel('Рейтинг расходов')
```

Из полученных результатов следует, что точки данных попадают примерно в пять кластеров (рис. 1.11).

⁴ Здесь и далее в этой главе названия столбцов в аналогичных таблицах такие: CustomerID — идентификатор; Gender — пол; Age — возраст; Annual Income — ежегодный доход; Spending Score — рейтинг расходов. — Прим. ред.

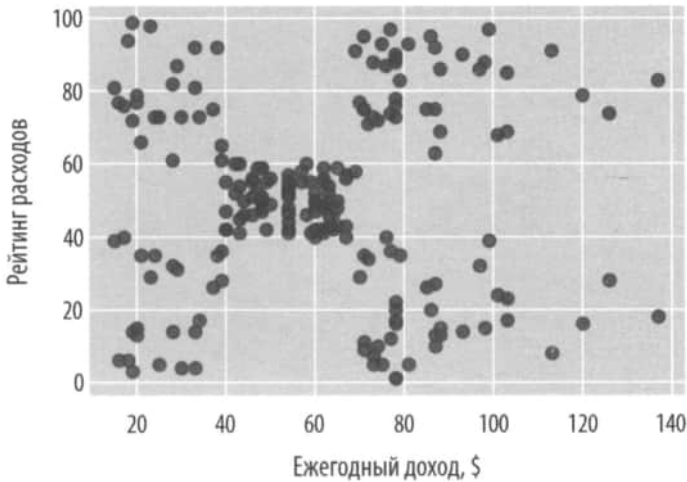


Рис. 1.11. График годовых доходов и расходов

Используйте следующий код для разбиения клиентов на пять кластеров и подсвечивания кластеров:

```
from sklearn.cluster import KMeans
```

```
kmeans = KMeans(n_clusters=5, random_state=0)
kmeans.fit(points)
predicted_cluster_indexes = kmeans.predict(points)
```

```
plt.scatter(x, y, c=predicted_cluster_indexes, s=50, alpha=0.7, cmap='viridis')
plt.xlabel('Ежегодный доход, $')
plt.ylabel('Рейтинг расходов')
```

```
centers = kmeans.cluster_centers_
plt.scatter(centers[:, 0], centers[:, 1], c='red', s=100)
```

Результат приведен на рис. 1.12.

Клиенты, с которыми ассоциированы точки в правом нижнем квадранте диаграммы, могут стать хорошей целью для проведения рекламной акции, направленной на увеличение их расходов. Почему? Потому что у них высокие доходы, но низкие показатели расходов. Используя следующий код, создайте копию фрейма данных и добавьте столбец Cluster (кластер), содержащий кластерные индексы:

```
df = customers.copy()
df['Cluster'] = kmeans.predict(points)
df.head()
```

Результат приведен на рис. 1.13.

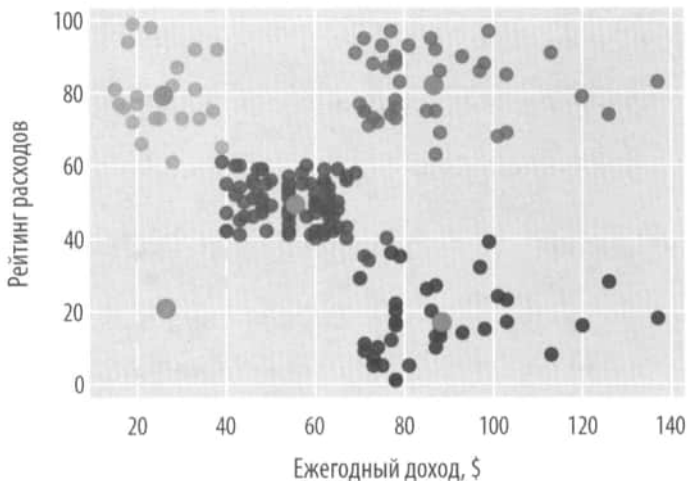


Рис. 1.12. График после разбиения клиентов на 5 кластеров и подсвечивания кластеров

	CustomerID	Gender	Age	Annual Income (k\$)	Spending Score (1-100)	Cluster
0	1	Male	19	15	39	4
1	2	Male	21	15	81	3
2	3	Female	20	16	6	4
3	4	Female	23	16	77	3
4	5	Female	31	17	40	4

Рис. 1.13. Второй результат в задаче сегментирования клиентов

Теперь с помощью следующего кода выведите идентификаторы клиентов с высокими доходами, но низкими показателями расходов:

```
import numpy as np
```

```
# Получение кластерного индекса для клиента с высоким уровнем дохода
```

```
# и низким уровнем расходов
```

```
cluster = kmeans.predict(np.array([[120, 20]]))[0]
```

```
# Фильтрация DataFrame для включения в него только клиентов,
```

```
# относящихся к данному кластеру
```

```
clustered_df = df[df['Cluster'] == cluster]
```

```
# Показать идентификаторы клиентов
```

```
clustered_df['CustomerID'].values
```

Полученные идентификаторы клиентов можно легко использовать для извлечения имен и адресов электронной почты из базы данных клиентов:

```
array([125, 129, 131, 135, 137, 139, 141, 145, 147, 149, 151, 153,
       155, 157, 159, 161, 163, 165, 167, 169, 171, 173, 175, 177,
       179, 181, 183, 185, 187, 189, 191, 193, 195, 197, 199],
      dtype=int64)
```

Ключевым моментом здесь является то, что применялась кластеризация для группировки клиентов по годовому доходу и рейтингу расходов. Если клиенты сгруппированы таким образом, то перечислить их в каждом кластере не составит труда.

Сегментирование клиентов по более чем двум измерениям

Предыдущий пример был прост, поскольку в нем использовались всего две переменные: годовой доход и рейтинг расходов. То же самое можно было сделать и без помощи машинного обучения. Но теперь давайте снова сегментируем клиентов, на этот раз используя всё, кроме идентификаторов клиентов. Начнем с замены строк *Male* (мужчина) и *Female* (женщина) в столбце *Gender* (пол) на 1 и 0 — процесс, называемый *кодированием меток*. Это необходимо, поскольку машинное обучение может работать только с числовыми данными:

```
from sklearn.preprocessing import LabelEncoder
```

```
df = customers.copy()
encoder = LabelEncoder()
df['Gender'] = encoder.fit_transform(df['Gender'])
df.head()
```

Теперь в столбце *Gender* содержатся 1 и 0 (рис. 1.14).

	CustomerID	Gender	Age	Annual Income (k\$)	Spending Score (1-100)
0	1	1	19	15	39
1	2	1	21	15	81
2	3	0	20	16	6
3	4	0	23	16	77
4	5	0	31	17	40

Рис. 1.14. Фрейм данных после кодирования меток

Извлеките столбцы *Gender*, *Age*, *Annual Income* и *Spending Score*. Затем с помощью метода локтя определите оптимальное количество кластеров на основе этих признаков:

```
points = df.iloc[:, 1:5].values
inertias = []
```

```
for i in range(1, 10):
    kmeans = KMeans(n_clusters=i, random_state=0)
    kmeans.fit(points)
    inertias.append(kmeans.inertia_)
```

```
plt.plot(range(1, 10), inertias)
plt.xlabel('Количество кластеров')
plt.ylabel('Итерация')
```

На этот раз локоть менее отчетлив, но 5 видится вполне приемлемым значением (рис. 1.15).

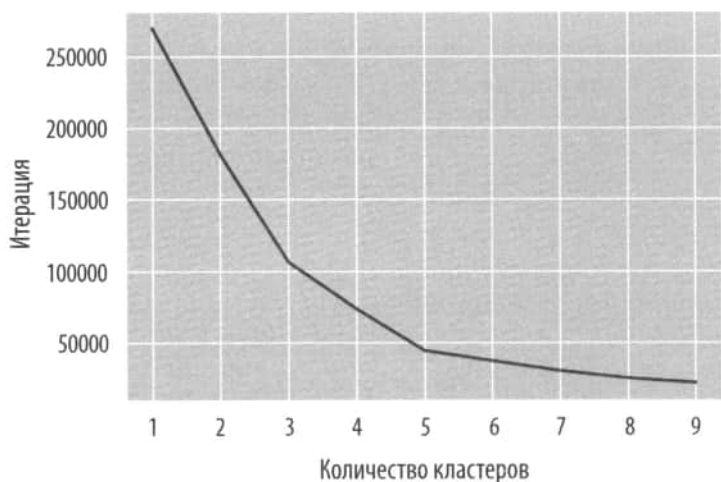


Рис. 1.15. График для определения оптимального количества кластеров

Разделите клиентов на пять кластеров и добавьте столбец с именем `Cluster`, содержащий индекс кластера (0–4), к которому был отнесен клиент:

```
kmeans = KMeans(n_clusters=5, random_state=0)
kmeans.fit(points)
```

```
df['Cluster'] = kmeans.predict(points)
df.head()
```

Результат приведен на рис. 1.16.

У вас есть значение кластера для каждого клиента, но что это означает? Вы не можете отразить пол, возраст, годовой доход и рейтинг расходов на двумерной диаграмме так же, как вы строили годовой доход и рейтинг расходов в предыдущем примере. Но можно получить среднее значение этих величин для каждого кластера по центроидам кластеров.

	CustomerID	Gender	Age	Annual Income (k\$)	Spending Score (1-100)	Cluster
0	1	1	19	15	39	0
1	2	1	21	15	81	4
2	3	0	20	16	6	0
3	4	0	23	16	77	4
4	5	0	31	17	40	0

Рис. 1.16. Фрейм данных после распределения клиентов по пяти кластерам

Создайте новый `DataFrame` со столбцами для среднего возраста, среднего дохода и т. д., а затем отобразите результаты в таблице:

```
results = pd.DataFrame(columns = ['Cluster', 'Average Age',
                                  'Average Income',
                                  'Average Spending Index',
                                  'Number of Females',
                                  'Number of Males'])

for i, center in enumerate(kmeans.cluster_centers_):
    age = center[1]    # Средний возраст для текущего кластера
    income = center[2] # Средний доход для текущего кластера
    spend = center[3]  # Средний рейтинг расходов для текущего кластера

    gdf = df[df['Cluster'] == i]
    females = gdf[gdf['Gender'] == 0].shape[0]
    males = gdf[gdf['Gender'] == 1].shape[0]

    results.loc[i] = ([i, age, income, spend, females, males])

results.head()
```

Результат приведен на рис. 1.17.

Исходя из этого, если бы вы собирались ориентироваться на покупателей с высоким доходом, но низким рейтингом расходов, какую группу покупателей (какой кластер) вы бы выбрали для проведения рекламной акции? Будет ли иметь значение, на кого ориентироваться — на мужчин или женщин? А если вы хотите создать программу лояльности, поощряющую клиентов с высоким рейтингом расходов, но при этом отдать предпочтение более молодым клиентам, которые могут быть лояльными клиентами в течение длительного времени? На какой кластер вы тогда будете ориентироваться?

Среди наиболее интересных моментов, выявленных в результате кластеризации, можно отметить такой: больше всех тратят молодые люди (средний возраст кото-

рых составляет 25,5 лет) со скромными доходами. Среди этих покупателей больше женщин, чем мужчин. Всё это полезная информация, которую необходимо иметь, если вы развиваете компанию и хотите лучше понять демографическую ситуацию.

	Cluster	Average Age	Average Income	Average Spending Index	Number of Females	Number of Males
0	0.0	45.217391	26.304348	20.913043	14.0	9.0
1	1.0	32.692308	86.538462	82.128205	21.0	18.0
2	2.0	43.088608	55.291139	49.569620	46.0	33.0
3	3.0	40.666667	87.750000	17.583333	17.0	19.0
4	4.0	25.521739	26.304348	78.565217	14.0	9.0

Рис. 1.17. Фрейм данных для средних значений⁵



Метод k средних может быть наиболее распространенным алгоритмом кластеризации, но не единственным. Среди других алгоритмов можно назвать *агломеративную кластеризацию* (<https://oreil.ly/zQpxZ>), при которой точки данных группируются иерархически, и *DBSCAN* (<https://oreil.ly/TanDh>) — пространственную кластеризацию для приложений с шумом на основе плотности. DBSCAN не требует предварительного задания количества кластеров. Эта кластеризация также может определять точки, которые попадают за пределы выделенных ею кластеров, что полезно для обнаружения выбросов — аномальных точек данных, которые не вписываются в остальные. Scikit-Learn предоставляет реализацию обоих алгоритмов в классах *AgglomerativeClustering* (<https://oreil.ly/7CztS>) и *DBSCAN* (<https://oreil.ly/D13gs>).

Используют ли реальные компании кластеризацию для извлечения информации из данных о клиентах? Конечно, используют. Во время учебы в аспирантуре мой сын, ныне аналитик данных в компании Delta Air Lines, проходил практику в компании по продаже товаров для домашних животных. С помощью кластеризации k средних он определил, что главной причиной того, что заявки, поступающие через веб-сайт компании, не конвертировались в продажи, был промежуток времени между моментом поступления заявки и первым контактом отдела продаж с клиентом. В результате его работодатель ввел дополнительную автоматизацию в рабочий процесс продаж для обеспечения быстрого реагирования на заявки. Это и есть неконтролируемое обучение в действии. И это прекрасный пример того, как компания использует машинное обучение для улучшения своих бизнес-процессов.

⁵ Названия столбцов: Cluster — кластер; Average Age — средний возраст; Average Income — средний доход; Average Spending Index — средний рейтинг расходов; Number of Females — количество женщин; Number of Males — количество мужчин. — Прим. ред.

Обучение с учителем

Обучение без учителя является важной ветвью машинного обучения, но когда большинство людей слышат термин "машинное обучение", они подразумевают контролируемое обучение, или *обучение с учителем* (supervised learning). Напомним, что модели контролируемого обучения делают прогнозы. Например, они предсказывают, является ли операция по кредитной карте мошеннической или придет ли рейс вовремя. Они также обучаются на данных с метками.

Модели контролируемого обучения бывают двух видов: *регрессионные* и *классификационные*. Цель регрессионной модели — предсказать числовой результат, например цену, по которой будет продан дом, или возраст человека на фотографии. Классификационные модели, напротив, предсказывают класс или категорию из конечного набора классов, определенных в обучающих данных. В качестве примера можно привести вопрос о том, является ли операция по кредитной карте законной или мошеннической, а также о том, какое число представляет собой написанная от руки цифра. Первая модель является *бинарной классификационной моделью*, поскольку существует только два возможных исхода: транзакция является законной или нет. Вторая модель — пример *многоклассовой (мультиклассовой) классификации*. Поскольку в западной арабской системе счисления имеется десять цифр (0–9), существует десять возможных классов, которые может представлять написанная от руки цифра.

На рис. 1.18 показаны два типа моделей контролируемого обучения. В модели слева задача состоит в том, чтобы ввести x и предсказать, каким будет y . Справа — ввести x и y и предсказать, какому классу соответствует точка: треугольнику или эллипсу. В обоих случаях целью применения машинного обучения к задаче является построение модели для прогнозирования. Вместо того чтобы строить эту модель самостоятельно, вы обучаете ML-модель с помощью помеченных данных и позволяете ей разработать математическую модель за вас.



Рис. 1.18. Регрессия в сравнении с классификацией

Для этих наборов данных можно легко построить математические модели, не прибегая к машинному обучению. Для регрессионной модели можно провести линию

через точки данных и использовать уравнение этой линии для прогнозирования значения y по значению x (рис. 1.19). Для классификационной модели можно провести линию, которая четко отделяет треугольники от эллипсов, что занимающиеся изучением данных ученые называют *классификационной границей*, и предсказать, к какому классу относится новая точка, определив, находится ли она выше или ниже этой линии. Точка, находящаяся выше линии, будет треугольником, а точка, находящаяся ниже линии, будет классифицироваться как эллипс.



Рис. 1.19. Линия регрессии и граница линейного разделения

В реальном мире наборы данных редко бывают настолько упорядоченными. Обычно они больше похожи на те, что приведены на рис. 1.20, где нет ни одной линии, которую можно было бы провести, чтобы соотнести значения x и y слева или четко разделить классы справа. Поэтому задача состоит в том, чтобы построить наилучшую модель. Это означает выбор алгоритма обучения, который дает наиболее точную модель.



Рис. 1.20. Наборы данных реального мира

Существует множество алгоритмов контролируемого обучения. Они имеют такие названия, как линейная регрессия, случайный лес, градиентный бустинг (gradient-boosting machines, GBM) и метод опорных векторов (support vector machines, SVM). Многие из них, но не все, могут использоваться для регрессии и классификации. Даже опытные специалисты по обработке данных часто экспериментируют, чтобы

определить, какой алгоритм обучения дает наиболее точную модель. Эти и другие алгоритмы обучения будут рассмотрены в последующих главах.

Метод k ближайших соседей

Одним из самых простых алгоритмов контролируемого обучения является *алгоритм k ближайших соседей* (k -nearest neighbors, <https://oreil.ly/dPhKi>). Суть его заключается в том, что при заданном наборе точек данных можно предсказать метку для новой точки, изучив ближайшие к ней точки. Для простой задачи регрессии, в которой каждая точка данных характеризуется координатами x и y , это означает, что при заданном x можно предсказать y , найдя n точек, ближайших по x , и усреднив их y . Для задачи классификации необходимо найти n точек, ближайших к точке, класс которой требуется предсказать, и выбрать класс с наибольшим числом встречаемости. Если $n = 5$ и пять ближайших соседей включают три треугольника и два эллипса, то решением будет треугольник, как показано на рис. 1.21.



Рис. 1.21. Классификация с помощью k ближайших соседей

Приведем пример с регрессией. Предположим, что у вас есть 20 точек данных, описывающих годовой заработок программистов в зависимости от стажа работы. На рис. 1.22 по оси x отмечены годы опыта (стаж работы), а по оси y — годовой доход (зарплата). Ваша цель — предсказать, сколько должен зарабатывать человек с 10-летним опытом работы. В данном примере $x = 10$, и вы хотите предсказать, каким должен быть доход y .

Применение метода k ближайших соседей при $n = 10$ позволяет определить точки, выделенные оранжевым цветом на рис. 1.23, как ближайших соседей — те 10 точек, чьи координаты x наиболее близки к $x = 10$. Среднее значение координат y этих точек равно 94 838. Таким образом, метод k ближайших соседей с $n = 10$ предсказывает, что программист с 10-летним опытом работы будет зарабатывать 94 838 долларов, что обозначено красной точкой.

Значение n , используемое в случае k ближайших соседей, часто влияет на результат. На рис. 1.24 показано то же решение при $n = 5$. На этот раз ответ несколько отличается, поскольку среднее значение y для пяти ближайших соседей составляет 98 713.

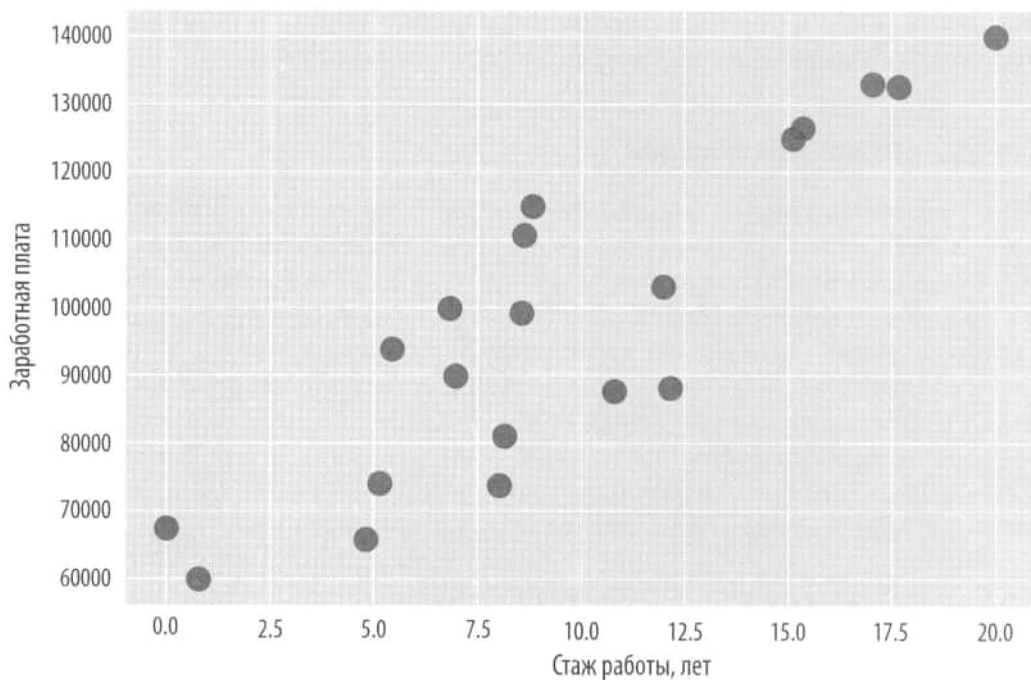


Рис. 1.22. Зарботная плата программистов в долларах в зависимости от стажа работы

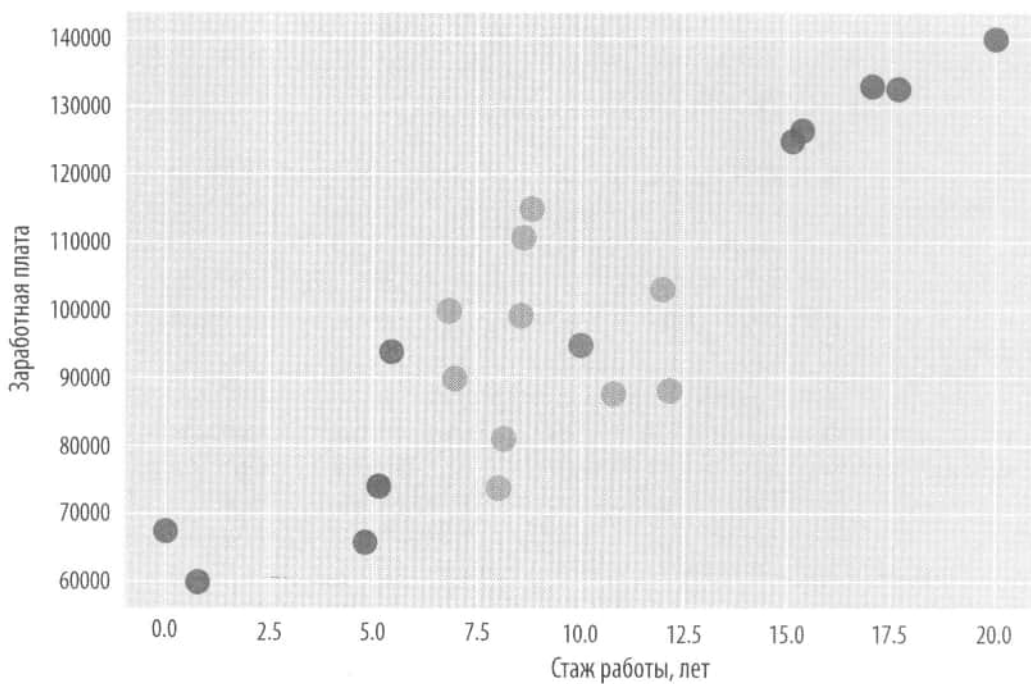


Рис. 1.23. Регрессия с k ближайшими соседями и $n = 10$

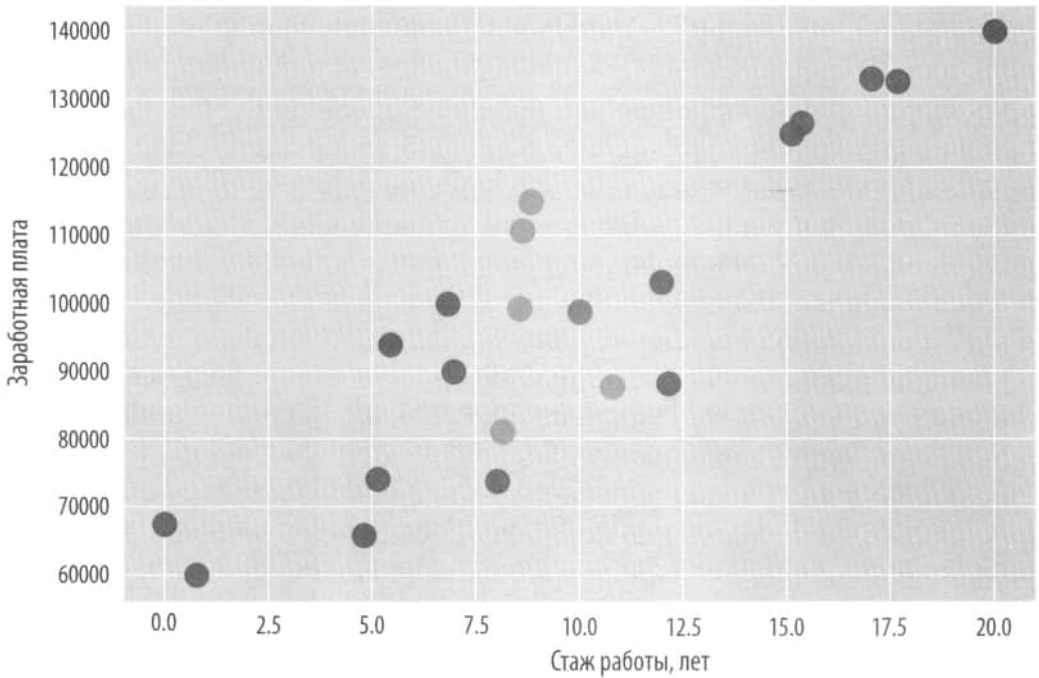


Рис. 1.24. Регрессия с k ближайшими соседями и $n = 5$

В реальной жизни все несколько сложнее, потому что, хотя набор данных содержит только один столбец меток, в нем, скорее всего, есть несколько столбцов признаков — не только x , но и x_1, x_2, x_3 и т. д. Вычислить расстояния в n -мерном пространстве достаточно просто, но существует несколько способов измерения расстояний для определения ближайших соседей точки, включая евклидово расстояние (<https://oreil.ly/36K7A>), манхэттенское расстояние (<https://oreil.ly/36K7A>) и расстояние Минковского (<https://oreil.ly/36K7A>). Можно даже использовать весовые коэффициенты, чтобы близкие точки вносили больший вклад в результат, чем далекие. Кроме того, можно не искать n ближайших соседей, а выбрать всех соседей в заданном радиусе — этот метод известен под названием "*соседи по радиусу*". Тем не менее принцип остается неизменным независимо от количества измерений в наборе данных, метода определения расстояния, выбора n ближайших соседей или всех соседей в пределах заданного радиуса: найдите точки данных, которые похожи на целевую точку, и используйте их для регрессии или классификации целевой точки.

Использование k ближайших соседей для классификации цветов

В состав Scikit-Learn входят классы `KNeighborsRegressor` (<https://oreil.ly/feXD0>) и `KNeighborsClassifier` (<https://oreil.ly/8o7Uv>), помогающие обучать регрессионные и классификационные модели с использованием алгоритма обучения k ближайших соседей. Также в него входят классы `RadiusNeighborsRegressor` (<https://oreil.ly/FP3wH>) и

RadiusNeighborsClassifier (<https://oreil.ly/g1TfE>), которые принимают радиус, а не количество соседей. Рассмотрим пример, в котором **KNeighborsClassifier** используется для классификации цветов на основе известного набора данных Iris (<https://oreil.ly/lpXFR>). Этот набор данных включает 150 образцов, каждый из которых представляет один из трех видов ирисов. Каждая строка содержит четыре параметра: длина чашелистика, ширина чашелистика, длина лепестка и ширина лепестка, — все в сантиметрах, а также метку: 0 — ирис *setosa*, 1 — *versicolor* и 2 — *virginica*. На рис. 1.25 показан пример каждого вида и проиллюстрирована разница между лепестками и чашелистиками.

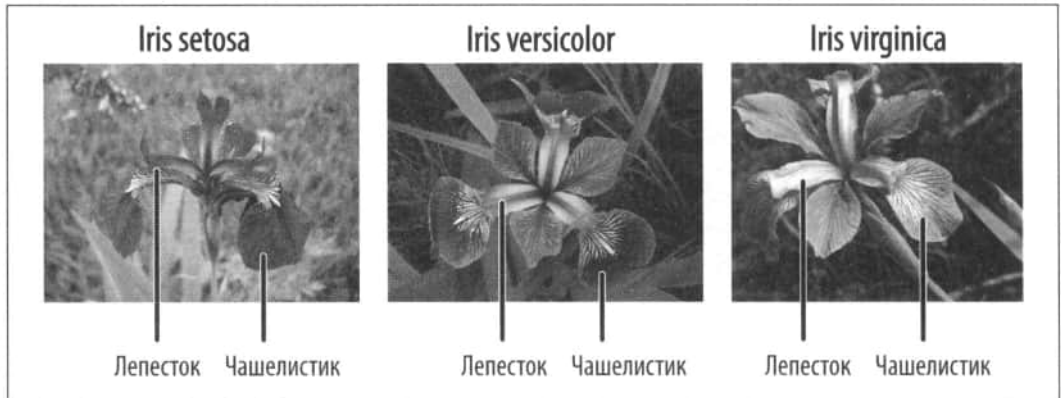


Рис. 1.25. Набор данных по ирисам (фотография в центре: "Blue Flag Flower Close-Up [Iris Versicolor]", автор — Danielle Langlois, лицензия — CC BY-SA 2.5, <https://creativecommons.org/licenses/by-sa/2.5/deed.en>; фотография справа: "Image of Iris Virginica Shrevei BLUE FLAG", автор — Frank Mayfield, лицензия — CC BY-SA 2.0, <https://creativecommons.org/licenses/by-sa/2.0/deed.en>)

Для подготовки модели машинного обучения, позволяющей различать виды ирисов по размерам чашелистиков и лепестков, запустите в блокноте Jupyter следующий код, загружающий набор данных, добавляющий столбец с названием класса и показывающий первые пять строк:

```
import pandas as pd
from sklearn.datasets import load_iris

iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['class'] = iris.target
df['class name'] = iris.target_names[iris['target']]
df.head()
```

Набор данных Iris является одним из нескольких примеров наборов данных (<https://oreil.ly/BMQYL>), входящих в состав Scikit. Поэтому его можно загрузить, вызвав функцию Scikit `load_iris` (<https://oreil.ly/NmpjR>), а не читать из внешнего файла. Результат работы этого кода приведен на рис. 1.26.

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	class	class name
0	5.1	3.5	1.4	0.2	0	setosa
1	4.9	3.0	1.4	0.2	0	setosa
2	4.7	3.2	1.3	0.2	0	setosa
3	4.6	3.1	1.5	0.2	0	setosa
4	5.0	3.6	1.4	0.2	0	setosa

Рис. 1.26. Фрейм данных с первыми пятью строками набора данных Iris⁶

Прежде чем строить модель машинного обучения на основе данных, необходимо разделить набор данных на два набора: обучающий и тестовый. Это очень важно, поскольку, не протестировав модель на данных, с которыми она еще не сталкивалась, т. е. на данных, которым она не обучалась, вы не будете иметь представления о том, насколько точно она может делать прогнозы.

К счастью, функция `train_test_split` (<https://oreil.ly/6TuKC>) в Scikit позволяет легко разделить набор данных с помощью заданного вами дробного разбиения. С помощью следующих операторов можно выполнить разбиение 80/20, при котором 80% строк отводится для обучения, а 20% — для тестирования:

```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.2, random_state=0)
```

Теперь `x_train` и `y_train` содержат 120 строк случайно выбранных измерений и меток, а `x_test` и `y_test` — оставшиеся 30. Хотя разбиение 80/20 является обычным для подобных небольших наборов данных, не существует правила, предписывающего разбиение 80/20. Чем больше данных вы используете для обучения, тем точнее модель. (Это не является абсолютной истиной, но, вообще говоря, всегда нужно иметь как можно больше обучающих данных.) Чем больше данных вы используете для тестирования, тем больше уверенности в точности модели. Для небольшого набора данных разумно начать с соотношения 80/20.

Следующий шаг — обучение ML-модели. Благодаря Scikit для этого требуется всего несколько строк кода:

```
from sklearn.neighbors import KNeighborsClassifier

model = KNeighborsClassifier()
model.fit(x_train, y_train)
```

В Scikit модель машинного обучения создается путем инстанцирования класса, содержащего выбранный алгоритм обучения, в данном случае `KNeighborsClassifier`.

⁶ Названия столбцов: `sepal length` — длина чашелистика; `sepal width` — ширина чашелистика; `petal length` — длина лепестка; `petal width` — ширина лепестка; `class` — класс; `class name` — имя класса. — Прим. ред.

Затем вызывается функция `fit` для обучения модели путем подгонки ее к обучающим данным. При наличии всего 120 строк обучающих данных обучение происходит очень быстро.

Последний шаг — использование 30 строк тестовых данных, выделенных из исходного набора данных, для измерения точности модели. В Scikit это осуществляется путем вызова метода `score` модели:

```
model.score(x_test, y_test)
```

В данном примере `score` возвращает значение 0,966667, что означает точность 97%, после того, как модель предсказала признаки в `x_test` и сравнила предсказанные метки с реальными метками в `y_test`.

Конечно, вся цель обучения прогнозирующей модели заключается в том, чтобы делать с ее помощью предсказания. В Scikit предсказание осуществляется путем вызова метода `predict` модели. Используйте следующую команду для получения предсказания класса (0 для *setosa*, 1 для *versicolor* и 2 для *virginica*), если необходимо определить вид ириса, длина чашелистика которого составляет 5,6 см, ширина чашелистика — 4,4 см, длина лепестка — 1,2 см, а ширина лепестка — 0,4 см:

```
model.predict([[5.6, 4.4, 1.2, 0.4]])
```

Метод `predict` может выполнять несколько предсказаний за один вызов. Именно поэтому ему передается список списков, а не просто список. Метод возвращает список, длина которого равна количеству переданных списков. Поскольку для `predict` был передан только один список, возвращаемое значение — это список с единственным значением. В данном примере предсказанный класс равен 0, т. е. модель предсказала, что ирис, длина чашелистика которого составляет 5,6 см, ширина чашелистика — 4,4 см, длина лепестка — 1,2 см, а ширина лепестка — 0,4 см, скорее всего, является ирисом *Setosa*.

При создании `KNeighborsClassifier` без указания количества соседей по умолчанию используется значение 5. Вы можете указать количество соседей следующим образом:

```
model = KNeighborsClassifier(n_neighbors=10)
```

Попробуйте выполнить настройку (обучение) и оценить модель еще раз, используя `n_neighbors=10`. Меняется ли оценка модели? По-прежнему ли модель предсказывает класс 0? Не бойтесь экспериментировать с другими значениями `n_neighbors`, чтобы почувствовать их влияние на результат.

Внутренние компоненты `KNeighborsClassifier`

Алгоритм *k* ближайших соседей иногда называют "ленивым" алгоритмом обучения, поскольку большая часть работы выполняется при вызове `predict`, а не при вызове `fit`. Фактически, при обучении технически не нужно делать ничего, кроме создания копии обучающих данных для вызова `predict`. Что же происходит внутри метода `fit` в `KNeighborsClassifier`?

В большинстве случаев `fit` строит в памяти двоичное дерево, которое ускоряет предсказание, избавляя от необходимости выполнять поиск соседних образцов методом перебора. Если выясняется, что двоичное дерево не помогает, `KNeighborsClassifier` прибегает к перебору при прогнозировании. Обычно это происходит в тех случаях, когда обучающие данные разрежены, т. е. содержат в основном нули с небольшим количеством ненулевых значений.

Одна из замечательных особенностей Scikit-Learn заключается в том, что у нее открытый исходный код. Если вы хотите узнать больше о том, как работает тот или иной класс или метод, можете обратиться к исходному коду на GitHub. На GitHub (<https://oreil.ly/mC4Rt>) можно найти исходный код `KNeighborsClassifier` и `RadiusNeighborsClassifier`.

Используемый здесь процесс — загрузка данных, разбиение данных, создание классификатора или регрессора, вызов `fit` для настройки на обучающие данные, вызов `score` для оценки точности модели на тестовых данных и, наконец, вызов `predict` для построения прогнозов — является тем процессом, который вы будете использовать в Scikit снова и снова. В реальном мире данные часто требуют очистки перед использованием для обучения и тестирования. Например, необходимо удалить строки с пропущенными значениями или провести дедупликацию данных, чтобы убрать лишние строки. В дальнейшем вы увидите множество примеров подобного рода, но в рассмотренных случаях данные были полными и хорошо структурированными, поэтому не требовали дополнительной подготовки.

Резюме

Машинное обучение предлагает инженерам и разработчикам программного обеспечения альтернативный подход к решению задач. Вместо того чтобы использовать традиционные компьютерные алгоритмы для преобразования входных данных в выходные, машинное обучение опирается на алгоритмы построения математических моделей на основе обучающих данных. Затем эти модели используются для преобразования будущих входных данных в выходные.

Большинство моделей машинного обучения относятся к одной из двух категорий. Модели обучения без учителя широко используются для анализа наборов данных путем выявления сходств и различий. Для них не требуются помеченные данные. Модели контролируемого обучения обучаются на основе помеченных данных, чтобы делать прогнозы, например предсказать, является ли операция по кредитной карте легитимной. Контролируемое обучение может использоваться для решения задач регрессии или классификации. Регрессионные модели предсказывают числовые результаты, а классификационные модели предсказывают классы (категории).

Кластеризация k средних — популярный алгоритм обучения без учителя, а k ближайших соседей — простой, но эффективный алгоритм обучения с учителем.

Многие, но не все алгоритмы контролируемого обучения могут быть использованы для регрессии и для классификации. Например, класс `KNeighborsRegressor` в Scikit-Learn применяет алгоритм k ближайших соседей для решения задач регрессии, а `KNeighborsClassifier` — для решения задач классификации.

Преподаватели часто используют алгоритм k ближайших соседей для знакомства с контролируемым обучением, поскольку он прост для понимания и достаточно хорошо работает в различных проблемных областях. После изучения k ближайших соседей следующим шагом на пути к освоению машинного обучения станет знакомство с другими алгоритмами обучения с учителем. Этому посвящена *глава 2*, в которой представлены несколько популярных алгоритмов обучения в контексте регрессионного моделирования.

Регрессионные модели

Из главы 1 вы узнали, что модели обучения с учителем (или контролируемого обучения) бывают двух видов: регрессионные и классификационные. Вы также узнали, что регрессионные модели предсказывают числовые результаты, например цену, за которую будет продан дом, или количество посетителей на сайте. Регрессионное моделирование — важный и порой недооцененный аспект машинного обучения. Розничные сети используют его для прогнозирования спроса (<https://oreil.ly/pqs2a>). Банки применяют его для проверки кредитных заявок, учитывая такие переменные, как кредитные баллы, соотношение долга и дохода, соотношение стоимости кредита и займа. Страховые компании используют его для установления страховых взносов. В любом случае, когда требуется получить числовые прогнозы, регрессионное моделирование — подходящий инструмент.

При построении регрессионной модели первым и наиболее важным решением является выбор алгоритма обучения. В главе 1 была представлена простая трехклассовая классификационная модель, использующая алгоритм k ближайших соседей для определения вида ириса по размерам чашелистиков и лепестков цветка. K ближайших соседей можно использовать и для регрессии, но это один из многих алгоритмов, которые можно выбрать для построения численных прогнозов. Другие алгоритмы обучения часто дают более точные модели.

В этой главе представлены распространенные алгоритмы регрессии, многие из которых могут использоваться и для классификации, и показан процесс построения регрессионной модели, предсказывающей стоимость проезда в такси на основе данных, опубликованных Комиссией по такси и лимузинам города Нью-Йорка (New York City Taxi and Limousine Commission). Также описываются различные способы оценки точности регрессионной модели и вводится важный метод измерения точности, называемый перекрестной проверкой.

Линейная регрессия

Линейная регрессия, наряду с k ближайшими соседями, — пожалуй, самый простой алгоритм обучения. Он лучше всего работает с данными, которые являются относительно линейными, т. е. с точками данных, расположенными примерно на одной прямой.

Обращаясь к школьным урокам математики, можно вспомнить, что уравнение прямой в двух измерениях имеет вид:

$$y = mx + b,$$

где m — наклон прямой, а b — место пересечения прямой с осью y . Набор данных "доход в зависимости от стажа", представленный на рис. 1.22, хорошо поддается линейной регрессии. На рис. 2.1 показана линия регрессии, соответствующая точкам данных. Прогнозирование дохода программиста с 10-летним опытом работы сводится к нахождению точки на линии, где $x = 10$. Уравнение прямой таково: $y = 3984x + 60\,040$. Подставив 10 в это уравнение для x , получим прогнозируемый доход 99 880 долларов.

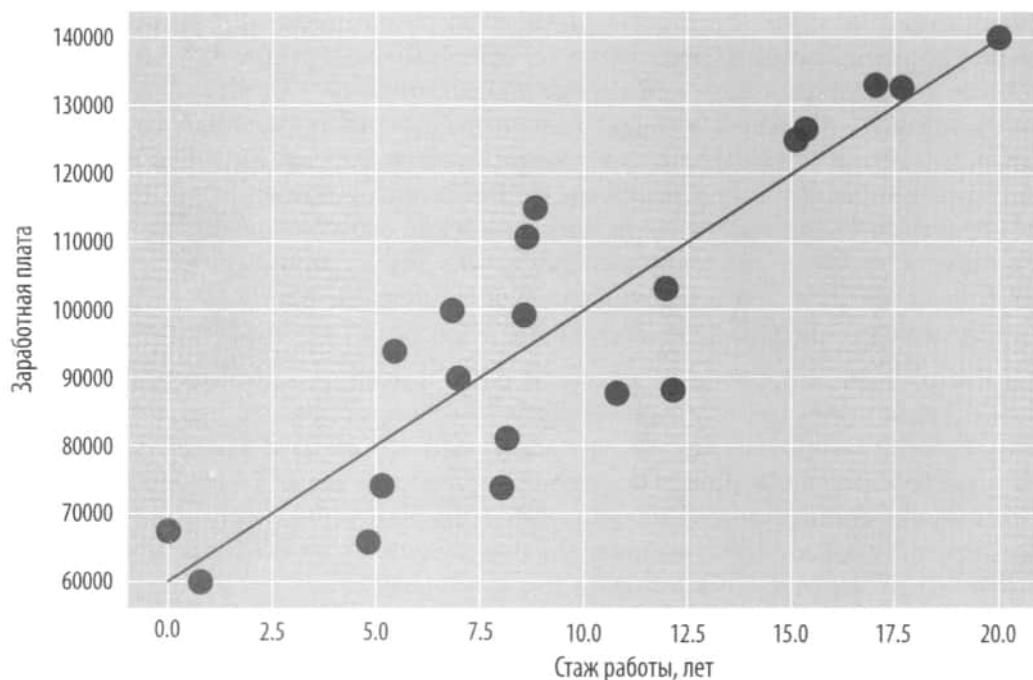


Рис. 2.1. Линейная регрессия

Цель обучения модели линейной регрессии — найти значения m и b , которые дают наиболее точные прогнозы. Для этого обычно используется итерационный процесс, который начинается с предполагаемых значений m и b и повторяется до тех пор, пока не будут найдены подходящие значения.

Наиболее распространенной методикой подгонки линии к набору точек является *регрессия по методу наименьших квадратов* (<https://oreil.ly/n5fx9>), или сокращенно МНК. Для этого необходимо возвести в квадрат расстояние по оси y между каждой точкой и линией регрессии, просуммировать квадраты и разделить на количество точек для вычисления *среднеквадратической ошибки* (mean squared error, MSE). (Возведение каждого расстояния в квадрат предотвращает компенсацию от-

рицательных расстояний положительными.) Затем следует произвести корректировку m и b для уменьшения среднеквадратической ошибки в следующий раз, и так повторять до тех пор, пока MSE не станет достаточно малой. Я не буду вдаваться в подробности того, как алгоритм определяет, в каком направлении корректировать m и b (это несложно, но включает в себя немного вычислений — в частности, использование *частных производных* (<https://oreil.ly/dnOhg>) функции среднеквадратической ошибки для определения того, следует ли увеличивать или уменьшать m и b на очередной итерации), но МНК часто может подогнать линию к набору точек за десяток итераций или менее.

В Scikit-Learn имеется ряд классов, помогающих строить модели линейной регрессии, в том числе класс `LinearRegression` (<https://oreil.ly/e8r8p>), реализующий МНК, и класс `PolynomialFeatures` (<https://oreil.ly/s3rJN>), который подгоняет к обучающим данным не прямую, а полиномиальную кривую. Обучение модели линейной регрессии может быть простым:

```
model = LinearRegression()
model.fit(x, y)
```

В Scikit есть и другие классы линейной регрессии с такими названиями, как `Ridge` (<https://oreil.ly/7I3ON>) и `Lasso` (<https://oreil.ly/Fnj2v>). Один из сценариев, в котором они могут быть полезны, — это ситуация, когда обучающие данные содержат выбросы. Вспомним из *главы 1*, что выбросы — это точки данных, которые не совпадают с остальными. Выбросы могут исказить модель или сделать ее менее точной. `Ridge` и `Lasso` добавляют *регуляризацию* (<https://oreil.ly/x4Dt2>), которая смягчает влияние выбросов, уменьшая их влияние на результат по мере корректировки коэффициентов в процессе обучения. Альтернативным подходом к решению проблемы выбросов является их полное удаление, что и будет сделано в примере с оплатой проезда на такси в конце этой главы.



Регрессия `Lasso` имеет и второе преимущество. Если обучающие данные страдают от *мультиколлинеарности* (<https://oreil.ly/qNDA0>) — состояния, при котором две или более входных переменных линейно коррелируют между собой, так что одна из них может быть предсказана на основе другой с достаточной степенью точности, то регрессия `Lasso` эффективно игнорирует избыточные данные.

Классическим примером мультиколлинеарности является ситуация, когда в наборе данных один столбец содержит информацию о количестве комнат в доме, а другой — о его площади. Большее количество комнат обычно означает большую площадь, поэтому эти две переменные в определенной степени коррелируют.

Линейная регрессия не ограничивается двумя измерениями (значениями x и y), она работает с любым числом измерений. Линейная регрессия с одной независимой переменной (x) называется *простой линейной регрессией*, а линейная регрессия с двумя и более независимыми переменными — например, x_1 , x_2 , x_3 и т. д. — называется *множественной линейной регрессией*. Если набор данных двумерный, то дос-

таточно просто построить график, чтобы определить его форму. Можно построить и трехмерные данные, но построение наборов данных с четырьмя или пятью измерениями является более сложной задачей, а наборы данных с сотнями или тысячами измерений невозможно визуализировать.

Как определить, поддается ли многомерный набор данных линейной регрессии? Один из способов — сократить n размерностей до двух или трех с помощью таких методов, как *анализ главных компонент* (principal component analysis, PCA) (<https://oreil.ly/jzc37>) и *стохастическое вложение соседей с t -распределением* (t -distributed stochastic neighbor embedding, t -SNE) (<https://oreil.ly/6mbe6>), чтобы можно было построить их график. Эти методы рассматриваются в *главе 6*, и оба позволяют уменьшить размерность набора данных без потери информации. Например, при использовании PCA нередко удается уменьшить число измерений на 90%, сохранив при этом 90% информации исходного набора данных. Это может показаться волшебством, но это не так. Это математика.

Если число измерений относительно невелико, то более простой техникой визуализации многомерных наборов данных являются *парные диаграммы* (<https://oreil.ly/kHiBd>), которые отображают пары измерений на обычных двумерных графиках. На рис. 2.2 показана парная диаграмма зависимости длины чашелистика (sepal length) от длины лепестка (petal length), ширины чашелистика (sepal width) от ширины лепестка (petal width) и других пар параметров (например, класса (class)) для набора данных Iris, представленного в *главе 1*.

Функция `pairplot` (<https://oreil.ly/2T9OS>) пакета Seaborn позволяет легко строить парные графики. График на рис. 2.2 был построен с помощью одной строки кода:

```
sns.pairplot(df)
```

Парная диаграмма не только помогает визуализировать взаимосвязи в наборе данных, но в конкретном примере гистограмма в правом нижнем углу показывает, что набор данных также сбалансирован. В нем имеется равное количество образцов всех трех классов, а по причинам, о которых вы узнаете в *главе 3*, всегда лучше обучать классификационные модели на сбалансированных наборах данных.

Линейная регрессия — *параметрический* алгоритм обучения, т. е. его цель — изучить набор данных и найти оптимальные значения параметров в уравнении, например m и b . К ближайших соседей — напротив, *непараметрический* алгоритм обучения, поскольку он не подгоняет данные под уравнение. Почему важно, является ли алгоритм обучения параметрическим или непараметрическим? Потому что наборы данных, используемые для обучения параметрических моделей, часто необходимо нормализовать. В самом простом случае *нормализация данных* означает, что все значения во всех столбцах имеют одинаковые диапазоны. О нормализации я расскажу в *главе 5*, а пока поймите, что обучение параметрических моделей с ненормализованными данными — например, с набором данных, содержащим в одном столбце значения от 0 до 1, а в другом — от 0 до 1 000 000, — может сделать эти модели менее точными или вообще не позволить им сходиться к решению. Это особенно характерно для машин опорных векторов и нейронных сетей, но применимо и к другим параметрическим моделям. Даже модели k ближайших соседей

лучше всего работают с нормализованными данными, поскольку, хотя алгоритм обучения и не является параметрическим, внутри него используются вычисления на основе расстояний.

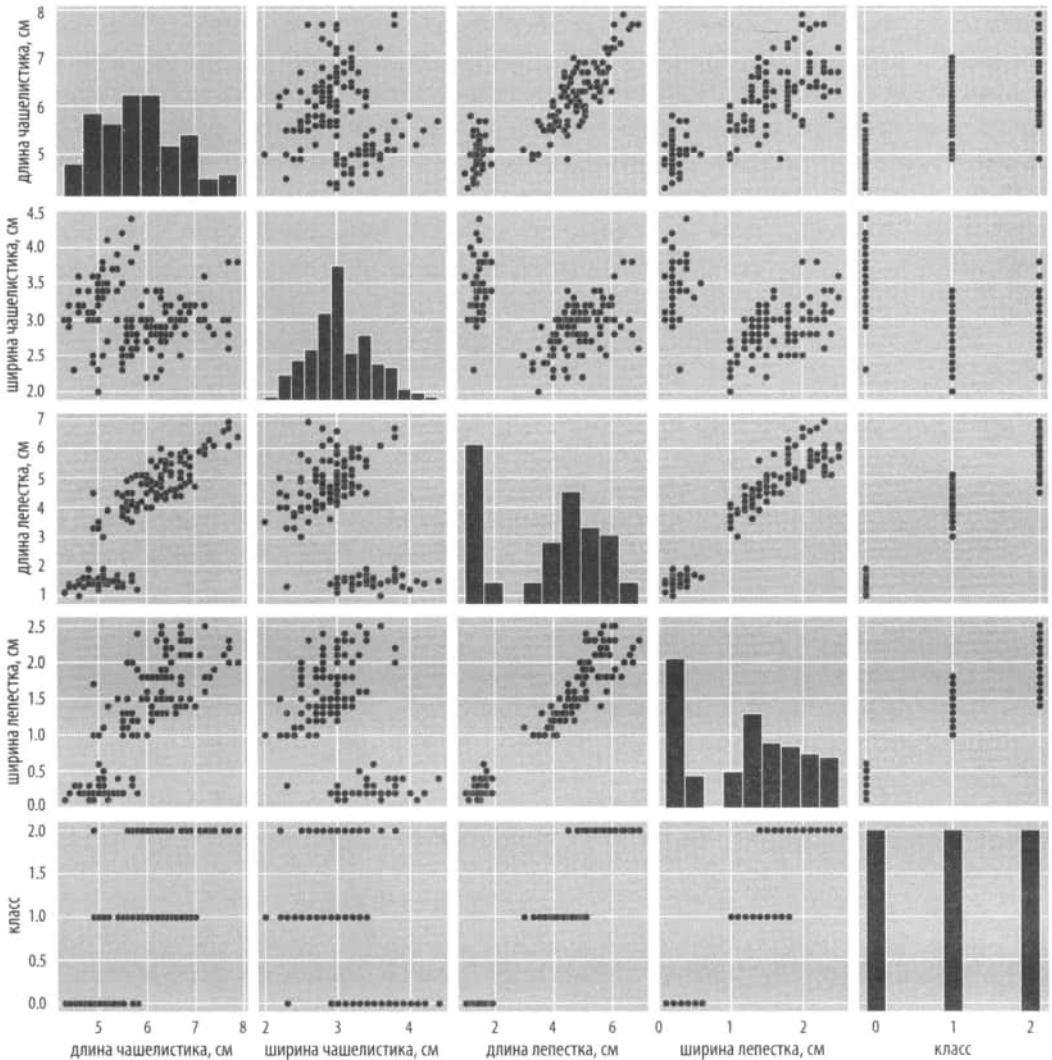


Рис. 2.2. Парная диаграмма, выявляющая взаимосвязи между парами переменных

Деревья решений

Даже если вы никогда не изучали информатику, вы наверняка знаете, что такое двоичное (бинарное) дерево (<https://oreil.ly/pE6E1>). В машинном обучении *дерево решений* — это древовидная структура, которая предсказывает результат, отвечая

на ряд вопросов. Большинство деревьев решений являются бинарными, в этом случае вопросы требуют простых ответов "да" или "нет".

На рис. 2.3 показано дерево решений, построенное программой Scikit на основе набора данных "доход в зависимости от опыта", представленного в *главе 1*. Дерево является простым, поскольку набор данных содержит только один столбец признаков (годы опыта), и я ограничил глубину дерева до 3, однако данная методика применима к деревьям неограниченного размера и сложности. В нашем примере для прогнозирования зарплаты программиста с 10-летним опытом работы требуется принять всего три решения "да/нет", как показано красными стрелками. Ответ составляет около 100 тыс. долларов, что довольно близко к тому, что предсказали алгоритм k ближайших соседей и линейная регрессия при применении к тому же набору данных.

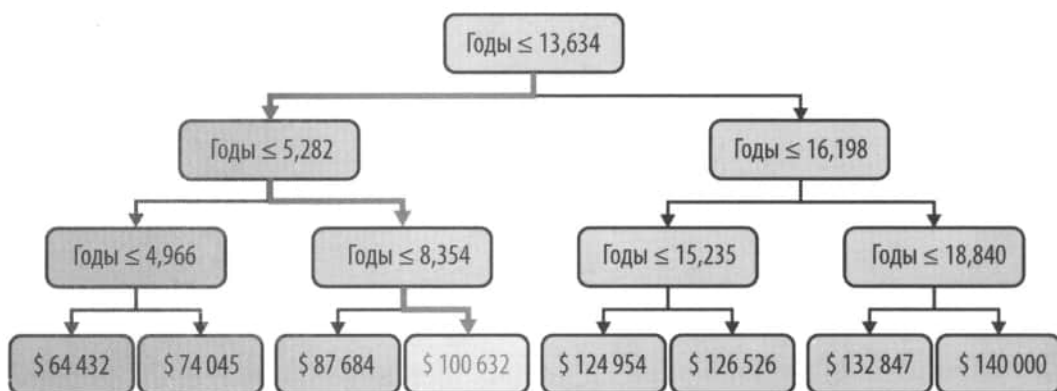


Рис. 2.3. Дерево решений

Деревья решений могут использоваться для регрессии и классификации. Для регрессора листовые узлы (узлы, у которых нет дочерних) представляют значения регрессии. Для классификатора они представляют классы. Выходные данные регрессора дерева решений не являются непрерывными. На выходе всегда будет одно из значений, присвоенных узлу листа, а количество узлов листа является конечным. Выходные данные линейной регрессионной модели, напротив, непрерывны. Она может принимать любое значение вдоль линии, подходящей к обучающим данным. В предыдущем примере вы получите один и тот же ответ, если попросите дерево предсказать зарплату для человека с 10-летним опытом работы и для человека с 13-летним опытом работы. Однако если увеличить количество лет опыта до 14, то прогнозируемая зарплата подскочит до 125 тыс. долларов (рис. 2.4). Если позволить дереву расти глубже, то ответы станут более точными. Однако слишком глубокий рост может привести к большим проблемам, о которых мы еще поговорим.

После обучения модели дерева решений, т. е. после построения дерева, прогнозы делаются быстро. Но по каким критериям принимать решения в каждом узле? Например, почему количество лет, представленное корневым узлом на рис. 2.3, равно 13,634? Почему не 10,000, не 8,742 или какое-то другое значение? Кроме того, если

в наборе данных имеется несколько столбцов признаков, как решить, на каком столбце делать разбиение в каждом узле принятия решений?

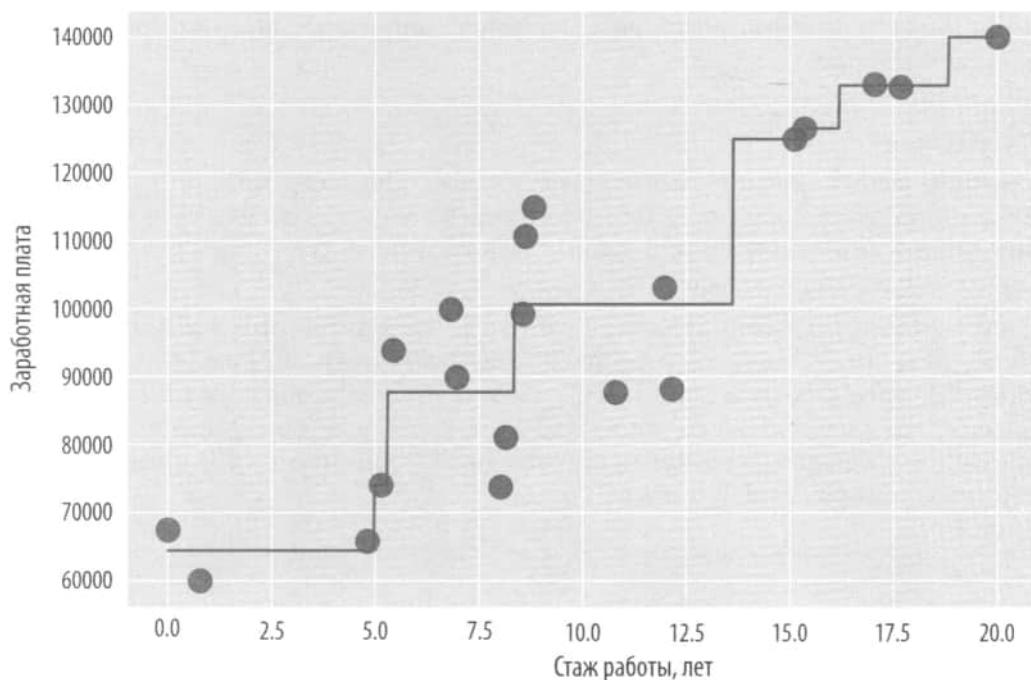


Рис. 2.4. Математическая модель, созданная на основе дерева решений

Деревья решений строятся путем рекурсивного разбиения обучающих данных. Основными решениями, которые принимает алгоритм разбиения при добавлении узла в дерево, являются: 1) какой столбец будет разделен этим узлом, и 2) на каком значении основывается разбиение. На каждой итерации целью является выбор столбца и значения разбиения, которые в наибольшей степени уменьшают "примесь" ("загрязненность") оставшихся данных для задач классификации или дисперсию оставшихся данных для задач регрессии. Общепринятой мерой "примеси" для классификаторов является *коэффициент Джини* (Gini) (<https://oreil.ly/bCuJx>), приблизительно определяющий процент выборок, которые неправильно классифицируются при использовании того или иного значения разбиения. Для регрессоров обычно применяется сумма квадратов ошибок или абсолютная ошибка, где "ошибка" — это разница между разделенным значением и значениями по обе стороны от разделения. Процесс построения дерева начинается с корневого узла и рекурсивно опускается вниз до тех пор, пока дерево не будет полностью очищено от листьев или внешние ограничения (например, ограничение на максимальную глубину) не помешают дальнейшему росту.

Класс `DecisionTreeRegressor` (<https://oreil.ly/jLyVQ>) и класс `DecisionTreeClassifier` (<https://oreil.ly/Ylrq4>) в Scikit позволяют легко строить деревья решений. Каждый из них реализует известный алгоритм CART для построения бинарных деревьев и

позволяет выбрать один из нескольких критериев для измерения примеси или дисперсии. Каждый из них также поддерживает такие параметры, как `max_depth`, `min_samples_split` и `min_samples_leaf`, которые позволяют ограничить рост дерева решений. Если принять значения по умолчанию, построить дерево решений можно следующим образом:

```
model = DecisionTreeRegressor()
model.fit(x, y)
```

Деревья решений являются непараметрическими. Обучение модели дерева решений заключается в построении бинарного дерева, а не в подгонке уравнения к набору данных. Это означает, что данные, используемые для построения дерева решений, не нужно нормализовать.

У деревьев решений есть большое преимущество: они работают с нелинейными данными так же хорошо, как и с линейными. Фактически, им практически безразлично, как сформированы данные. Но есть и обратная сторона. Она весьма значительна, и это одна из причин, по которой автономные деревья решений редко используются в машинном обучении. Эта причина — чрезмерное подстраивание (переобучение) (<https://oreil.ly/ycffk>).

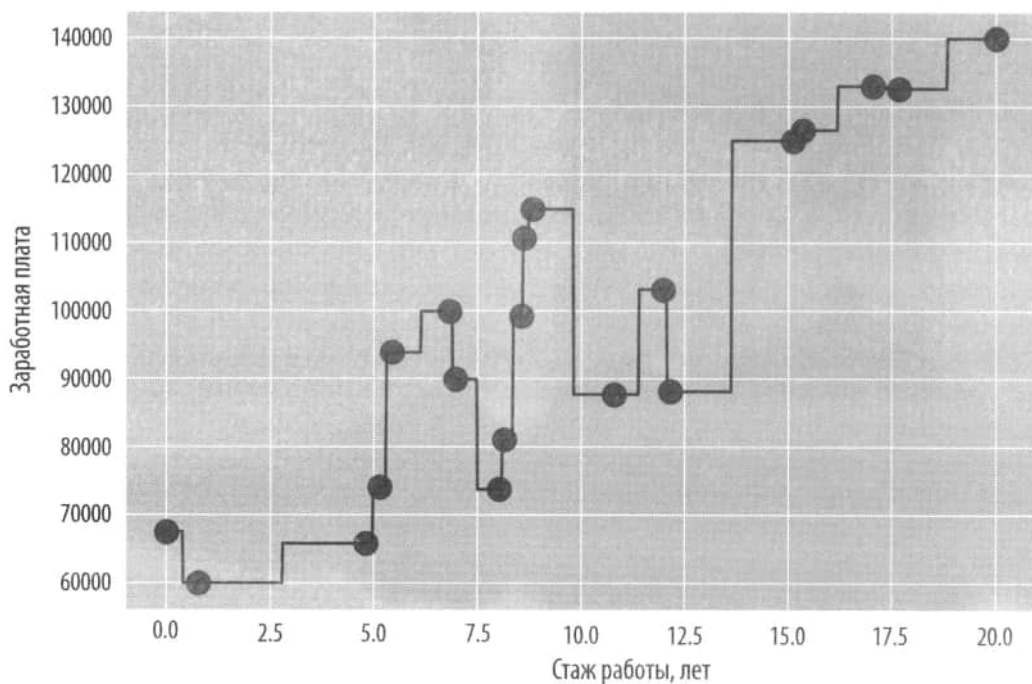


Рис. 2.5. Дерево принятия решений, чрезмерно подогнанное к обучающим данным

Деревья решений очень склонны к переобучению. Если позволить дереву решений достаточно разрастись, оно может, по сути, запомнить обучающие данные. Оно может казаться точным, но если его слишком плотно подогнать к обучающим дан-

ным, оно может плохо их обобщать. Это означает, что дерево будет не столь точным, когда ему потребуется сделать прогноз на основе данных, с которыми оно ранее не сталкивалось. На рис. 2.5 показано дерево решений, построенное на наборе данных "доход в зависимости от опыта" без ограничений на глубину. Ломаная траектория, по которой проходит красная линия через все точки, является явным признаком чрезмерного переобучения. Избыточное переобучение — это бич специалистов по исследованию данных. Хуже неточной модели может быть только та, которая кажется точной, но на самом деле таковой не является.

Одним из способов предотвращения избыточного переобучения при использовании деревьев решений является ограничение их роста, чтобы они *не могли* запоминать обучающие данные. Другой способ — использование групп деревьев решений, называемых случайными лесами.

Случайные леса

Как показано на рис. 2.6, *случайный лес* представляет собой набор деревьев решений (часто сотни), каждое из которых обучено по-разному на одних и тех же данных. Как правило, каждое дерево обучается на случайно выбранных строках набора данных, а ветвление происходит по столбцам, которые выбираются случайным образом при каждом разбиении. Модель не может слишком сильно привязываться к обучающим данным, поскольку каждое дерево обучается на своем подмножестве данных. Деревья строятся независимо друг от друга, и когда модель делает предсказание, она пропускает входные данные через все деревья решений и усредняет результат. Поскольку деревья строятся независимо друг от друга, обучение может быть распараллелено на поддерживающем это аппаратном обеспечении.

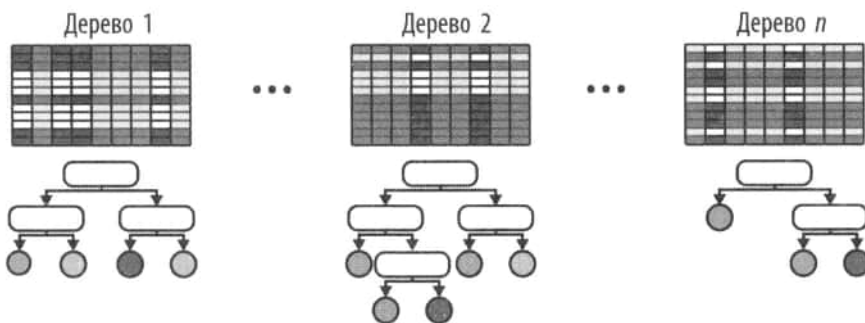


Рис. 2.6. Случайные леса

Это простая концепция, которая хорошо работает на практике. Случайные леса могут использоваться как для регрессии, так и для классификации, и Scikit предоставляет для этого такие классы, как `RandomForestRegressor` (<https://oreil.ly/QwaN1>) и `RandomForestClassifier` (<https://oreil.ly/xIXEp>). Они имеют ряд настраиваемых параметров, включая `n_estimators`, задающий количество деревьев в случайном лесу (по умолчанию равен 100); `max_depth`, ограничивающий глубину каждого дерева;

`max_samples`, задающий долю строк в обучающих данных, используемых для построения отдельных деревьев. На рис. 2.7 показано, как `RandomForestRegressor` подходит к набору данных "доход в зависимости от опыта" при `max_depth=3` и `max_samples=0,5`, т. е. ни одно дерево не видит более 50% строк в наборе данных.

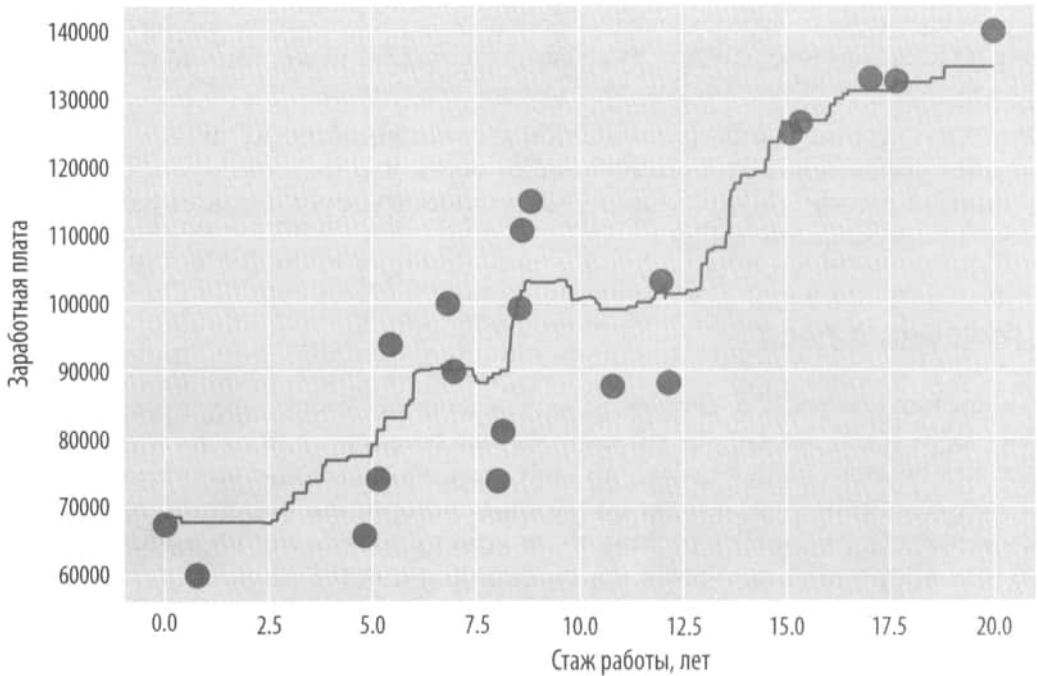


Рис. 2.7. Математическая модель, созданная на основе случайного леса

Поскольку деревья решений являются непараметрическими, случайные леса также непараметрические. И хотя на рис. 2.7 показано, как случайный лес работает с линейным набором данных, случайные леса прекрасно справляются с моделированием и нелинейных наборов данных.

Градиентно-бустинговые машины

Случайные леса служат подтверждением предположения о том, что можно взять множество слабых обучающихся моделей, которые сами по себе не являются сильными предсказателями, и объединить их для создания точных моделей. Ни одно отдельное дерево в случайном лесу не может предсказать результат с большой точностью. Но если собрать все деревья вместе и усреднить результаты, то они часто превосходят другие модели. Специалисты по исследованию данных называют это ансамблевым моделированием, или *ансамблевым обучением* (<https://oreil.ly/V9fqqa>).

Другим способом использования ансамблевого моделирования является *градиентный бустинг* (<https://oreil.ly/dzOiH>). Модели, использующие его, называются

градиентно-бустинговыми машинами (gradient-boosting machines, GBM). Большинство GBM используют деревья решений и иногда называются *деревьями решений с градиентным бустингом* (gradient-boosted decision trees, GBDT). Как и случайные леса, GBDT состоят из наборов деревьев решений. Однако вместо того, чтобы строить независимые деревья решений на основе случайных подмножеств данных, GBDT создают зависимые деревья решений одно за другим, обучая каждое из них на основе результатов предыдущего. Первое дерево решений моделирует набор данных. Второе дерево решений моделирует ошибку в выводе первого, третье — ошибку в выводе второго и т. д. Для того чтобы сделать прогноз, GBDT пропускает входные данные через каждое дерево решений и суммирует все выходы, получая результат. С каждым добавлением результат становится чуть более точным, что и послужило основанием для термина "*аддитивное моделирование*" (<https://oreil.ly/vyJyq>). Это похоже на то, как если вы вели бы мяч для гольфа по площадке и наносили бы удары всё короче и короче, пока, наконец, не достигли бы лунки.

Каждое дерево решений в модели GBDT является слабо обучаемым. Фактически, в GBDT обычно используются *гни деревья решений*, которые представляют собой деревья решений глубиной 1 (корневой узел и два дочерних узла), как показано на рис. 2.8.

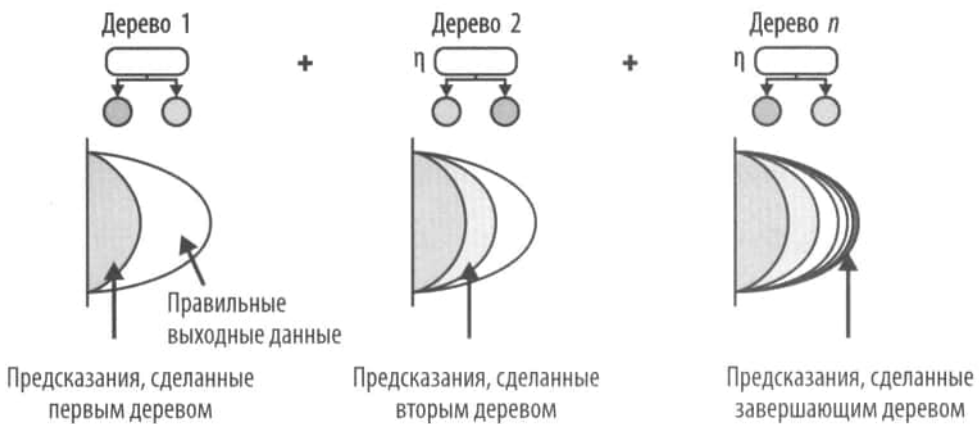


Рис. 2.8. Градиентно-бустинговые машины

В процессе обучения для создания базовой линии прогнозирования вначале берется среднее значение всех целевых значений в обучающих данных. Затем из целевых значений вычитается среднее значение для получения нового набора целевых значений или остатков для первого дерева прогнозирования. После обучения первого дерева входные данные прогоняются через него для получения набора прогнозов. Затем эти прогнозы добавляются к предыдущему набору прогнозов, генерируется новый набор остатков путем вычитания суммы из исходных (фактических) целевых значений, и обучается второе дерево для прогнозирования этих остатков. Повторяя этот процесс для n деревьев, где n обычно равно 100 и более, можно получить ансамблевую модель. Для того чтобы можно было убедиться в том, что каждое дерево решений является слабо обучаемым, в моделях GBDT выход каждого дерева

решений умножается на *коэффициент обучения* (коэффициент скорости обучения) с целью уменьшить их влияние на результат. Коэффициент обучения обычно представляет собой небольшое число, например 0,1, и является параметром, который можно задать при использовании классов, реализующих GBM.

Scikit включает классы `GradientBoostingRegressor` (<https://oreil.ly/ZhSop>) и `GradientBoostingClassifier` (<https://oreil.ly/9RIW5>), которые помогают строить GBDT. Но если вы действительно хотите понять, как работают GBDT, можете построить их самостоятельно с помощью класса `DecisionTreeRegressor` в Scikit. Код в примере 2.1 реализует GBDT со 100 пнями дерева решений и предсказывает годовой доход программиста с 10-летним стажем.

Пример 2.1. Реализация дерева решений с градиентным бустингом

```
learning_rate = 0.1 # Коэффициент обучения
n_trees = 100      # Количество деревьев решений
trees = []         # Деревья, составляющие модель

# Вычислите среднее значение всех целевых значений
y_pred = np.array([y.mean()] * len(y))
baseline = y_pred

# Создайте n деревьев (n_trees) и обучите каждое из них,
# используя ошибку в выводе предыдущего дерева
for i in range(n_trees):
    error = y - y_pred
    tree = DecisionTreeRegressor(max_depth=1, random_state=0)
    tree.fit(x, error)
    predictions = tree.predict(x)
    y_pred = y_pred + (learning_rate * predictions)
    trees.append(tree)

# Предскажите y для x=10
y_pred = np.array([baseline[0]] * len(x))

for tree in trees:
    y_pred = y_pred + (learning_rate * tree.predict([[10.0]]))

y_pred[0]
```

На диаграмме слева на рис. 2.9 показан результат работы одного дерева решений, примененного к набору данных "доход в зависимости от опыта". Эта модель является настолько слабо обучаемой, что может предсказать только два различных уровня дохода. На диаграмме справа показан результат работы модели из приме-

ра 2.1. Аддитивный эффект слабо обучающихся моделей дает сильную обучающуюся модель, которая предсказывает, что программист с 10-летним опытом работы должен зарабатывать 99 082 доллара в год, что согласуется с прогнозами, сделанными другими моделями.

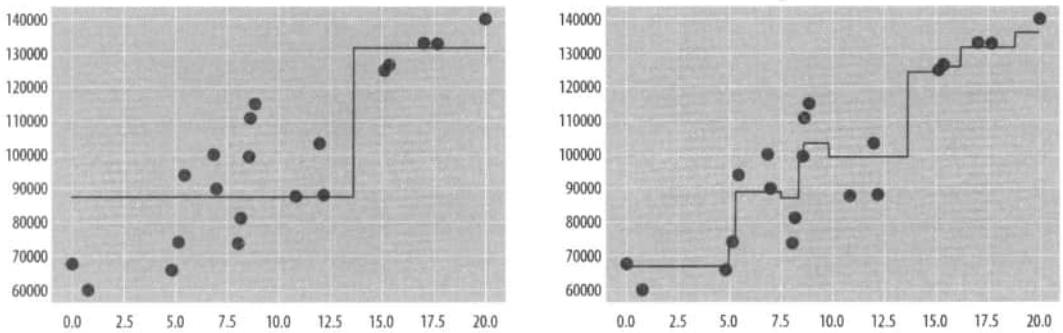


Рис. 2.9. Одиночное дерево решений в сравнении с деревьями решений с градиентным усилением

GBDT могут использоваться для регрессии и классификации и являются непараметрическими. Помимо нейронных сетей и машин опорных векторов, GBDT часто оказываются наиболее способными к моделированию сложных наборов данных.

В отличие от линейных регрессионных моделей и случайных лесов, GBDT подвержены чрезмерному обучению. Одним из способов борьбы с чрезмерным обучением в случае применения `GradientBoostingRegressor` и `GradientBoostingClassifier` является использование параметра `subsample`, не позволяющего отдельным деревьям просматривать весь набор данных аналогично тому, как это делает `max_samples` для случайных лесов. Другой способ — использовать параметр `learning_rate` для снижения скорости обучения, по умолчанию равный 0,1.

Метод опорных векторов

Я оставляю полное рассмотрение метода опорных векторов (SVM) для главы 5, но наряду с GBM он находится на острие статистического машинного обучения. Этот метод часто может создавать модели для сильно нелинейных наборов данных, которые не под силу другим алгоритмам обучения. Он настолько важен, что заслуживает отдельного рассмотрения от всех остальных алгоритмов. Для моделирования эффекта добавления размерности к данным в них используется математический аппарат, называемый *ядерным трюком* (kernel trick). Идея заключается в том, что данные, которые не поддаются разделению в m измерениях, могут быть разделены в n измерениях. Приведем небольшой пример.

Классы в двумерном наборе данных слева на рис. 2.10 невозможно разделить линейно. Но если добавить третье измерение, чтобы точки ближе к центру имели большие значения z , а точки дальше от центра — меньшие значения z , как показано

на рис. 2.10 справа, то можно провести плоскость между красными и фиолетовыми точками и добиться 100% разделения классов. Именно по такому принципу работает метод SVM. Он сложен с математической точки зрения, если использовать его для работы с произвольными наборами данных, но это чрезвычайно мощная техника, которая значительно упрощена в Scikit.

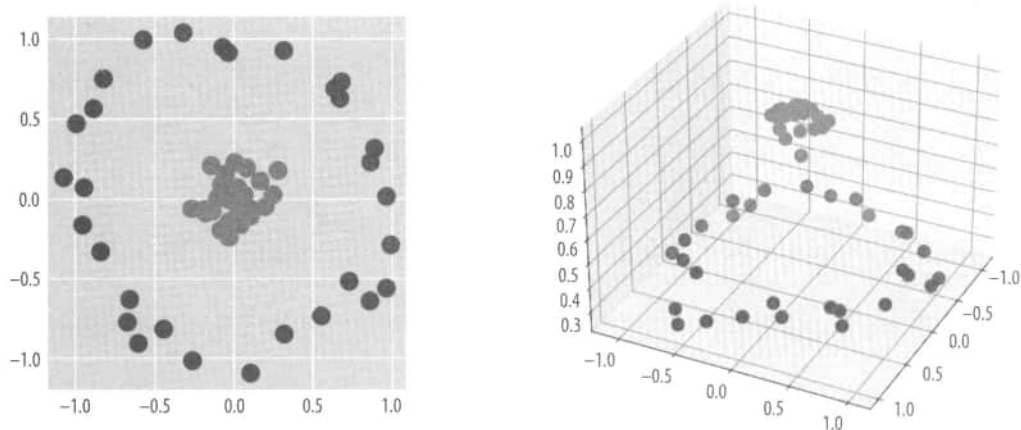


Рис. 2.10. Метод опорных векторов

SVM в основном используется для классификации, но его можно применять и для регрессии. В Scikit присутствуют классы для решения обеих задач, а именно SVC (<https://oreil.ly/MWsSX>) для решения задач классификации и SVR (<https://oreil.ly/YPvZa>) для решения задач регрессии. Об этих классах вы узнаете в *главе 5*. А пока на ближайшем мероприятии, посвященном машинному обучению, упомяните термин "метод опорных векторов" (или "машина опорных векторов"), и вы сразу же станете центром всеобщего внимания.

Меры точности для регрессионных моделей

Как вы узнали из *главы 1*, для обучения модели необходим один набор данных, а для ее тестирования — другой, и вы можете оценить точность модели, передав тестовые данные в метод оценки модели `score`. Тестирование позволяет определить, насколько точно модель делает прогнозы. Очень важно тестировать модель на наборе данных, отличном от того, на котором она обучалась, поскольку модель, скорее всего, достаточно хорошо усвоит обучающие данные, но это не означает, что она будет хорошо обобщать их, т. е. делать точные прогнозы. А если вы не протестируете модель, то не узнаете, насколько она точна.

Инженеры часто используют функцию `train_test_split` (<https://oreil.ly/AkvO3>) в Scikit для разделения набора данных на обучающий и тестовый. Однако при таком разбиении небольшого набора данных не всегда можно доверять оценке модели, возвращаемой методом `score`. Да и что вообще означает эта оценка? Ответ на этот

вопрос различен для регрессионных и классификационных моделей, и для тех и других существует множество способов оценки. Давайте разберемся, что означает число, которое выдает `score` для регрессионной модели, и что можно сделать, чтобы больше доверять этому числу.

Для того чтобы продемонстрировать, почему нужно скептически относиться к значению, возвращаемому `score` при работе с небольшими наборами данных, попробуем провести простой эксперимент. С помощью следующего кода загрузите в Scikit набор данных по жилью в Калифорнии (California housing dataset, <https://oreil.ly/mt7ch>), перемешайте строки и извлеките первые 1000 строк:

```
from sklearn.utils import shuffle
from sklearn.datasets import fetch_california_housing
```

```
df = fetch_california_housing(as_frame=True).frame
df = shuffle(df, random_state=0)
df = df.head(1000)
df.head()
```

Набор данных содержит столбцы с такими именами, как `MedInc` (медианный доход) и `MedHouseVal` (медианная стоимость жилья), как показано на рис. 2.11. Подробности сейчас не важны. Важно то, что вы собираетесь построить модель, которая будет использовать значения всех остальных столбцов для прогнозирования значений `MedHouseVal`:

	MedInc	HouseAge	AveRooms	AveBedrms	Population	AveOccup	Latitude	Longitude	MedHouseVal
14740	4.1518	22.0	5.663073	1.075472	1551.0	4.180593	32.58	-117.05	1.369
10101	5.7796	32.0	6.107226	0.927739	1296.0	3.020979	33.92	-117.97	2.413
20566	4.3487	29.0	5.930712	1.026217	1554.0	2.910112	38.65	-121.84	2.007
2670	2.4511	37.0	4.992958	1.316901	390.0	2.746479	33.20	-115.60	0.725
15709	5.0049	25.0	4.319261	1.039578	649.0	1.712401	37.79	-122.43	4.600

Рис. 2.11. Фрагмент набора данных California housing dataset¹

Используйте следующий код для разделения данных в соотношении 80/20, обучения линейной регрессионной модели на 80% данных для прогнозирования цены дома и оценки модели с 20% данных, выделенных для тестирования:

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split

x = df.drop(['MedHouseVal'], axis=1)
y = df['MedHouseVal']
```

¹ Названия столбцов: `MedInc` — медианный доход; `HouseAge` — возраст дома; `AveRooms` — среднее количество комнат; `AveBedrms` — среднее количество спален; `Population` — население; `AveOccup` — среднее количество жильцов; `Latitude` — широта; `Longitude` — долгота; `MedHouseVal` — медианная стоимость жилья. — *Прим. ред.*


```
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2,
                                                    random_state=0)
```

```
model = LinearRegression()
model.fit(x_train, y_train)
model.score(x_test, y_test)
```

В данном примере результат `score` равен 0,5863 и, по всей видимости, свидетельствует о том, что точность модели, использующей для прогнозирования признаки тестовых данных, составляет около 59%. Пока все хорошо.

Теперь изменим значение `random_state`, передаваемое в `train_test_split`, с 0 на 1 и снова запустим код:

```
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2,
                                                    random_state=1)
```

```
model = LinearRegression()
model.fit(x_train, y_train)
model.score(x_test, y_test)
```

На этот раз результат равен 0,6255. Как же так? Точность предсказаний модели составляет 59% или 63%? Почему `score` возвращает два разных значения?

Когда `train_test_split` разбивает набор данных, он случайным образом выбирает строки для обучающего и тестового наборов данных. Параметр `random_state` задает значение генератора случайных чисел, используемого для формирования выборки. Задавая два разных значения, вы обучаете модель на двух разных наборах данных и тестируете ее также на двух разных наборах данных. Конечно, между ними есть пересечение. Но факт остается фактом: число, с которым вы запускаете (засеиваете) генератор случайных чисел `train_test_split`, влияет на результат. Чем меньше набор данных, тем сильнее это влияние.

Решением проблемы является *перекрестная валидация* (кросс-валидация). Для нее модель должна разделить набор данных на *части* (folds), как показано на рис. 2.12.



Рис. 2.12. Пятикратная кросс-валидация

Обычно делят на пять частей, но можно использовать любое количество. Затем модель обучается пять раз — по одному разу для каждой части, используя для обуче-

ния разные 80% набора данных, а для тестирования — разные 20%, и полученные результаты усредняются, чтобы получить кросс-валидированную оценку. Эта оценка более надежна, чем оценка, выданная методом `score`, поскольку она менее чувствительна к тому, как разбиты данные. Этот процесс известен как *k-кратная перекрестная валидация*.

Недостатком перекрестной валидации является то, что она занимает больше времени. При 5-кратной перекрестной валидации модель обучается пять раз. Положительным моментом является то, что перекрестная валидация обычно применяется только для небольших наборов данных, и если набор данных небольшой, то модель, скорее всего, обучается быстро.

Можно написать код для перекрестной валидации самостоятельно, но это не обязательно, поскольку Scikit выполнит эту работу за вас. Для проведения перекрестной валидации модели требуется всего одна строка кода:

```
cross_val_score(model, x, y, cv=5).mean()
```

Кросс-валидированная оценка в данном примере должна составить около 0,61, что находится между двумя значениями, полученными методом `score`. Это более верная оценка точности модели, чем любая из других оценок.

Обучать модель перед кросс-валидацией не нужно, поскольку `cross_val_score` обучает ее за вас. Однако `cross_val_score` обучает копию модели, а не саму модель, поэтому после использования кросс-валидации для оценки точности модели все равно необходимо вызвать метод `fit`, прежде чем делать предсказания:

```
model.fit(x, y)
```

Обратите внимание, что в `fit` передается весь набор данных, а не отдельное подмножество. Это еще одно преимущество кросс-валидации: вы обучаете свою модель на *всех* данных, что очень важно, если набор данных изначально небольшой. Больше нет жесткого требования оставлять часть данных для тестирования. Однако даже при кросс-валидации полезно оценивать модель на данных, предназначенных исключительно для тестирования, если такие данные имеются. Помните: *вы не узнаете, насколько точна модель, пока не поймете, как она реагирует на данные, на которых она не обучалась*.



Как правило, кросс-валидацию следует применять для небольших наборов данных, а для больших наборов использовать разделение обучения и тестирования. Чем больше набор данных, тем менее чувствителен он к тому, как разделены данные.

В связи с этим возникает вопрос: что именно представляет собой значение, возвращаемое при оценке или кросс-валидации модели? Для регрессионной модели это *коэффициент детерминации* (<https://oreil.ly/W1UHd>), также известный как *R*-квадрат или просто R^2 . Коэффициент детерминации обычно представляет собой значение от 0 до 1 ("обычно" потому, что при определенных обстоятельствах он может быть отрицательным), количественно определяющее дисперсию выходных данных, которая может быть объяснена переменными на входе. Проще говоря, показатель R^2 , равный 0,8, означает, что модель в среднем должна быть точна при-

мерно в 80% предсказаний или давать правильный ответ в пределах 20%. Чем выше показатель R^2 , тем точнее модель. Существуют и другие способы измерения точности регрессионной модели, включая среднеквадратическую ошибку (mean squared error, MSE) и среднюю абсолютную ошибку (mean absolute error, MAE). Эти показатели имеют смысл только в контексте диапазона выходных значений, в то время как R^2 дает одно простое число, не зависящее от диапазона. Более подробно о метриках регрессии и методах их получения можно прочитать в документации по Scikit (<https://oreil.ly/G3LUI>).

Значение, возвращаемое методом `score` модели, совершенно иное для классификационной модели. Различные способы количественной оценки точности классификационных моделей будут рассмотрены в *главе 3*.

Использование регрессии для прогнозирования тарифов на такси

Представьте себе, что вы работаете в службе такси, и одной из самых больших претензий ваших клиентов является то, что они не знают, сколько будет стоить поездка, пока она не закончится. Это связано с тем, что расстояние не единственная переменная, определяющая стоимость проезда. Вы решили исправить эту проблему и создать мобильное приложение, с помощью которого клиенты, усаживаясь в такси, смогут оценить стоимость проезда. Для создания интеллектуального приложения вы собираетесь задействовать огромные массивы данных о стоимости проезда, которые компания собирала в течение многих лет, и использовать их для построения модели машинного обучения.

Давайте обучим регрессионную модель для предсказания суммы оплаты проезда с учетом времени суток, дня недели, а также места посадки и высадки пассажиров. Начнем с загрузки CSV-файла с набором данных (<https://oreil.ly/qgx9X>) и его копирования в подкаталог `Data`, где размещены ваши блокноты Jupyter. Затем используйте следующий код для загрузки набора данных в блокнот. Набор данных содержит около 55 000 строк и является подмножеством гораздо большего набора данных, использовавшегося в конкурсе Kaggle по предсказанию стоимости проезда на такси в Нью-Йорке (<https://oreil.ly/Dy9Ye>). Прежде чем использовать эти данные, необходимо провести значительную подготовительную работу, что нередко встречается в машинном обучении. Зачастую сбор и подготовка данных занимают 90% рабочего времени и более:

```
import pandas as pd
```

```
df = pd.read_csv('Data/taxi-fares.csv', parse_dates=['pickup_datetime'])
df.head()
```

Обратите внимание на использование параметра `parse_dates` функции `read_csv` для разбора строк в столбце `pickup_datetime` в объекты Python `datetime` (<https://oreil.ly/msqMQ>). Результат работы этого кода представлен на рис. 2.13.

	key	fare_amount	pickup_datetime	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitude	passenger_count
0	2014-06-15 17:11:00.000000107	7.0	2014-06-15 17:11:00+00:00	-73.995420	40.759662	-73.987607	40.751247	1
1	2011-03-14 22:43:00.000000095	4.9	2011-03-14 22:43:00+00:00	-73.993552	40.731110	-73.998497	40.737200	5
2	2011-02-14 15:14:00.000000067	6.1	2011-02-14 15:14:00+00:00	-73.972380	40.749527	-73.990638	40.745328	1
3	2009-10-29 11:29:00.000000040	6.9	2009-10-29 11:29:00+00:00	-73.973703	40.763542	-73.984253	40.758603	5
4	2011-07-02 10:38:00.000000028	10.5	2011-07-02 10:38:00+00:00	-73.921262	40.743615	-73.967383	40.765162	1

Рис. 2.13. Результат работы кода предподготовки набора данных²

Каждая строка представляет собой поездку на такси и содержит такую информацию, как стоимость проезда, места посадки и высадки (выраженные в виде широты и долготы), количество пассажиров. Мы хотим предсказать именно стоимость проезда. С помощью следующего кода можно построить гистограмму, показывающую, сколько строк содержит количество пассажиров, равное 1, сколько — 2 и т. д.:

```
%matplotlib inline
import matplotlib.pyplot as plt
import seaborn as sns
sns.set()

sns.countplot(x=df['passenger_count'])
```

На рис. 2.14 приведен результат.

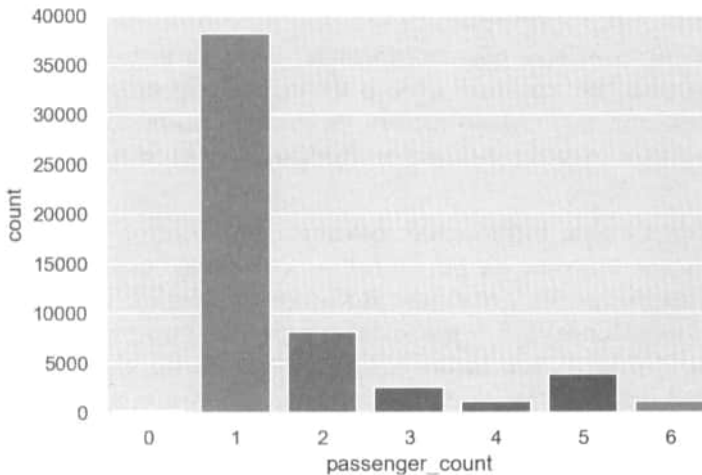


Рис. 2.14. Гистограмма встречаемости количества пассажиров

² Названия столбцов: key — ключ; fare_amount — стоимость проезда; pickup_datetime — время посадки; pickup_longitude — координаты посадки (широта); pickup_latitude — координаты посадки (долгота); dropoff_longitude — координаты высадки (широта); dropoff_latitude — координаты высадки (долгота); passenger_count — количество пассажиров. — Прим. ред.

Поскольку нас интересует прогнозирование стоимости проезда только для одного пассажира, с помощью следующего кода удалите все строки с несколькими пассажирами и удалите столбец `key` из набора данных, поскольку этот столбец не нужен — другими словами, он не является одним из признаков, по которым мы будем пытаться прогнозировать:

```
df = df[df['passenger_count'] == 1]
df = df.drop(['key', 'passenger_count'], axis=1)
df.head()
```

В результате в наборе данных остается 38 233 строки, в чем можно убедиться с помощью следующего оператора:

```
df.shape
```

Теперь с помощью метода `corr` библиотеки Pandas выясним, насколько сильно влияют входные переменные, такие как широта и долгота, на значения в столбце `fare_amount`:

```
corr_matrix = df.corr()
corr_matrix['fare_amount'].sort_values(ascending=False)
```

Результат выглядит следующим образом:

```
fare_amount      1.000000
dropoff_longitude 0.020438
pickup_longitude  0.015742
pickup_latitude  -0.015915
dropoff_latitude  -0.021711
Name: fare_amount, dtype: float64
```

Числа выглядят не слишком обнадеживающе. Широта и долгота мало связаны со стоимостью проезда, по крайней мере в их нынешнем виде. Однако интуитивно понятно, что они должны *сильно* влиять на стоимость, поскольку определяют начальный и конечный пункты, а более длительные поездки требуют более высоких тарифов.

Теперь начинается самое интересное: создание новых столбцов данных, которые оказывают большее влияние на результат, — столбцов, значения которых вычисляются из значений других столбцов. Добавим столбцы с указанием дня недели (0 — понедельник (Monday), 1 — воскресенье (Sunday) и т. д.), часа дня, когда пассажира забрали (0–23), и расстояния (по воздуху, а не по улице) в милях, которое потребовала поездка. При расчете расстояний в данном коде предполагается, что большинство поездок короткие и поэтому можно пренебречь кривизной Земли:

```
from math import sqrt
```

```
for i, row in df.iterrows():
    dt = row['pickup_datetime']
    df.at[i, 'day_of_week'] = dt.weekday()
    df.at[i, 'pickup_time'] = dt.hour
    x = (row['dropoff_longitude'] - row['pickup_longitude']) * 54.6
```

```
y = (row['dropoff_latitude'] - row['pickup_latitude']) * 69.0
distance = sqrt(x**2 + y**2)
df.at[i, 'distance'] = distance
```

```
df.head()
```

Теперь нет необходимости во всех столбцах, поэтому с помощью этих операторов можно удалить те, которые не будут использоваться:

```
df.drop(columns=['pickup_datetime', 'pickup_longitude', 'pickup_latitude',
                 'dropoff_longitude', 'dropoff_latitude'], inplace=True)
```

```
df.head()
```

Проверим корреляцию еще раз:

```
corr_matrix = df.corr()
corr_matrix['fare_amount'].sort_values(ascending=False)
```

По-прежнему нет сильной зависимости между расстоянием поездки и размером оплаты за проезд. Возможно, это объяснит причину:

```
df.describe()
```

На рис. 2.15 приведен результат.

	fare_amount	day_of_week	pickup_time	distance
count	38233.000000	38233.000000	38233.000000	38233.000000
mean	11.214115	2.951534	13.387989	12.018397
std	9.703149	1.932809	6.446519	217.357022
min	-22.100000	0.000000	0.000000	0.000000
25%	6.000000	1.000000	9.000000	0.762116
50%	8.500000	3.000000	14.000000	1.331326
75%	12.500000	5.000000	19.000000	2.402226
max	256.000000	6.000000	23.000000	4923.837280

Рис. 2.15. Измененный набор данных³

В наборе данных присутствуют выбросы, а выбросы часто искажают результаты моделей машинного обучения. Отфильтруйте данные, исключив отрицательные стоимости проезда и установив разумные ограничения на стоимость проезда и расстояние, а затем снова проведите корреляцию:

```
df = df[(df['distance'] > 1.0) & (df['distance'] < 10.0)]
df = df[(df['fare_amount'] > 0.0) & (df['fare_amount'] < 50.0)]
```

³ Названия столбцов: fare_amount — стоимость проезда; day_of_week — день недели; pickup_time — время посадки; distance — расстояние. — Прим. ред.

```
corr_matrix = df.corr()  
corr_matrix['fare_amount'].sort_values(ascending=False)
```

Еще раз, вот результат:

```
fare_amount    1.000000  
distance       0.851913  
day_of_week    -0.003570  
pickup_time    -0.023085  
Name: fare_amount, dtype: float64
```

Это уже лучше! Бóльшая часть (85%) дисперсии в стоимости проезда объясняется расстоянием. Корреляция между днем недели, временем суток и стоимостью проезда по-прежнему слабая, но это и неудивительно, поскольку именно расстояние поездки является основным фактором, определяющим стоимость проезда в такси. Оставим эти столбцы, поскольку вполне логично, что в час пик дорога из пункта А в пункт Б может занять больше времени, или что движение в 17:00 пятницы может отличаться от движения в 17:00 субботы.

Теперь пришло время обучить регрессионную модель. Давайте попробуем три различных алгоритма обучения, чтобы определить, какой из них дает наиболее точный результат, и использовать перекрестную валидацию для оценки точности. Начнем с линейной регрессионной модели:

```
from sklearn.linear_model import LinearRegression  
from sklearn.model_selection import cross_val_score
```

```
x = df.drop(['fare_amount'], axis=1)  
y = df['fare_amount']
```

```
model = LinearRegression()  
cross_val_score(model, x, y, cv=5).mean()
```

Попробуйте использовать RandomForestRegressor с тем же набором данных и сравните результаты. Напомним, что модели со случайным лесом обучают несколько деревьев решений на данных и усредняют результаты всех деревьев для получения прогноза:

```
from sklearn.ensemble import RandomForestRegressor
```

```
model = RandomForestRegressor(random_state=0)  
cross_val_score(model, x, y, cv=5).mean()
```

Наконец, воспользуемся GradientBoostingRegressor. Машины с градиентным бустингом работают на основе нескольких деревьев решений, каждое из которых обучено компенсировать ошибку в выходных данных предыдущего:

```
from sklearn.ensemble import GradientBoostingRegressor  
model = GradientBoostingRegressor(random_state=0)  
cross_val_score(model, x, y, cv=5).mean()
```

Предполагая, что `GradientBoostingRegressor` дал наибольший кросс-валидированный коэффициент детерминации, обучим его на всем наборе данных:

```
model.fit(x, y)
```

В завершение мы используем обученную модель для выполнения нескольких прогнозов. Во-первых, оцените, какой будет стоимость найма такси для поездки на расстояние 2 мили в пятницу в 17:00. Поскольку в метод `fit` вы передали фреймы данных `DataFrame` с именами столбцов, последние версии Scikit выдадут предупреждение, если вы передадите списки или массивы NumPy для прогнозирования. Поэтому вместо этого введите `DataFrame`:

```
model.predict(pd.DataFrame({ 'day_of_week': [4], 'pickup_time': [17],
                             'distance': [2.0] })))
```

Теперь спрогнозируем стоимость для поездки длиной 2 мили, совершенной в 17:00 через день (в субботу):

```
model.predict(pd.DataFrame({ 'day_of_week': [5], 'pickup_time': [17],
                             'distance': [2.0] })))
```

Предскажет ли модель более высокую или более низкую стоимость проезда для той же поездки в субботу днем? Имеют ли ответы смысл, если учесть, что данные получены от такси Нью-Йорка? Учитывайте, что в пятницу днем движение в часы пик, скорее всего, будет интенсивнее, чем в субботу.

Резюме

Регрессионные модели — это модели обучения с учителем, которые предсказывают числовые результаты, такие как стоимость поездки на такси. Вот основные алгоритмы обучения, используемые для регрессии.

Линейная регрессия.

Моделирует обучающие данные путем подгонки их под уравнение прямой.

Деревья решений.

Используют двоичные деревья для предсказания результата по ответам на ряд вопросов в формате "да" и "нет".

Случайные леса.

Используют несколько независимых деревьев решений для моделирования данных и устойчивы к избыточному обучению.

Градиентный бустинг.

Использует несколько зависимых деревьев решений, каждое из которых моделирует ошибку в выводе предыдущего.

Метод опорных векторов.

Использует совершенно иной подход к моделированию данных, добавляя размерность, исходя из предположения, что данные, которые не являются линейно разделимыми в исходном пространстве задач, могут быть линейно разделимы в более высокоразмерном пространстве.

Scikit предоставляет удобные реализации этих и других алгоритмов обучения в таких классах, как `LinearRegression`, `RandomForestRegressor` и `GradientBoostingRegressor`.

Общепринятой метрикой для количественной оценки точности регрессионных моделей является показатель R^2 , также известный как коэффициент детерминации. Обычно он имеет значение от 0 до 1, причем более высокие значения означают более высокую точность. Технически это мера дисперсии выходных данных, которая может быть объяснена значениями входных данных. Для небольших наборов данных перекрестная валидация дает больше уверенности в показателе R^2 , чем простое разделение данных для обучения и тестирования. Перекрестная валидация обучает модель k раз, каждый раз по-иному разбивая набор данных.

Реальные наборы данных, как правило, беспорядочны и часто требуют дополнительной подготовки для использования в машинном обучении. Как показал пример с такси, выбросы в обучающих данных могут повлиять на точность модели или вообще сделать ее бесполезной. Одним из решений является выявление выбросов и их удаление перед обучением модели. Другой способ — использовать алгоритм обучения, например гребневую (ridge) регрессию или регрессию Lasso, поддерживающий регуляризацию.

Регрессионные модели широко распространены в машинном обучении, но еще бóльшую популярность получили классификационные модели. Именно им и посвящена *глава 3*, в ней представлен один из ведущих алгоритмов обучения, называемый логистической регрессией, и использованы знания, которые вы получили из этой главы.

Классификационные модели

В модели машинного обучения, рассмотренной в предыдущей главе, использовались различные формы регрессии для прогнозирования стоимости проезда в такси в зависимости от расстояния поездки, дня недели и времени суток. Регрессионные модели предсказывают числовые результаты и широко используются в промышленности для прогнозирования продаж, цен, спроса и других показателей, определяющих бизнес-решения. Не менее важны и классификационные модели, которые предсказывают категориальные результаты, например, является ли операция по кредитной карте мошеннической или какую букву алфавита представляет тот или иной рукописный символ.

Большинство моделей классификации делятся на две категории: *бинарные (двоичные)*, в которых возможны только два исхода, и *мультиклассовые (многоклассовые)*, в которых возможны более двух исходов. В обоих случаях модель присваивает входным данным один класс, или метку класса. Реже встречаются *многометочные модели классификации*, которые могут классифицировать один входной сигнал как принадлежащий к нескольким классам, например предсказать, что документ является одновременно статьей по машинному обучению и статьей по геномике. Некоторые из них также могут предсказать, что входные данные не принадлежат ни к одному из возможных классов.

Многое из того, что вы знаете о регрессионных моделях, применимо и к классификационным моделям. Например, многие алгоритмы обучения, используемые в регрессионных моделях, одинаково хорошо работают и в классификационных моделях. Существенное различие между регрессией и классификацией заключается в способе измерения точности модели. Для классификационной модели не существует такого понятия, как показатель R^2 . Вместо него применяется множество показателей, таких как точность (precision), полнота (recall), специфичность (specificity), чувствительность (sensitivity), F1-мера (F1 score) и др. Одним из ключевых моментов в освоении классификационных моделей является умение работать с различными метриками точности и, что еще важнее, понимание того, какую из них (или какие из них) следует использовать в зависимости от целей применения модели.

Один из примеров мультиклассовой классификации — идентификацию ирисов — вы уже видели в *главе 1*. Настало время углубиться в классификаторы машинного обучения, начав с одного из самых проверенных алгоритмов, который работает *только* для классификационных моделей — логистической регрессии.

Логистическая регрессия

Для решения задач классификации существует множество алгоритмов обучения. В *главе 2* вы узнали, как деревья решений, случайные леса и градиентный бустинг (или градиентно-бустинговые машины, GBM) подбирают регрессионные модели к обучающим данным. Эти алгоритмы можно использовать и для классификации, а Scikit помогает в этом, предоставляя такие классы, как `DecisionTreeClassifier` (<https://oreil.ly/8N6MG>), `RandomForestClassifier` (<https://oreil.ly/3mNhv>) и `GradientBoostingClassifier` (<https://oreil.ly/ilXW4>). В *главе 1* вы использовали класс `KNeighborsClassifier` (<https://oreil.ly/zdDCH>) из Scikit для построения трехклассовой классификационной модели с k ближайшими соседями в качестве алгоритма обучения.

Это важные алгоритмы обучения, и они используются во многих современных моделях машинного обучения. Но одним из самых популярных алгоритмов классификации является *логистическая регрессия* (<https://oreil.ly/o5rBI>), которая анализирует распределение данных и подбирает под него уравнение, определяющее вероятность принадлежности данной выборки к каждому из двух возможных классов. Например, можно определить, что с вероятностью 10% значения в выборке относятся к классу 0, а с вероятностью 90% — к классу 1. В этом случае логистическая регрессия предскажет, что выборка соответствует классу 1. Несмотря на название, логистическая регрессия — это алгоритм *классификации*, а не алгоритм регрессии. Его цель не создание регрессионных моделей, а количественная оценка вероятностей для классификации входных выборок.

В качестве примера рассмотрим точки данных на рис. 3.1, которые принадлежат к двум классам: 0 (синий) и 1 (красный). Предположим, что ось x определяет количество часов, в течение которых человек готовился к экзамену, а ось y показывает, сдал он экзамен (1) или не сдал (0). Синие цвета попадают в диапазон от $x = 0$ до $x = 10$, а красные — в диапазон от $x = 5$ до $x = 15$. Вы не можете выбрать значение x , разделяющее классы, поскольку оба имеют значения между $x = 5$ и $x = 10$. (Попробуйте провести вертикальную линию, по одну сторону от которой будут лишь красные точки, а по другую — только синие.) Но можно построить кривую, которая, задавая x , показывает вероятность того, что точка с этим x принадлежит классу 1. С увеличением x растет и вероятность того, что точка относится к классу 1 ("зачет"), а не к классу 0 ("незачет"). Из кривой видно, что при $x = 2$ вероятность того, что точка соответствует классу 1, составляет менее 5%. Но если $x = 10$, то вероятность того, что она соответствует классу 1, составляет около 76%. Если бы вас попросили классифицировать эту точку как красную или синюю, вы бы пришли к выводу, что это красная точка, поскольку вероятность того, что она красная, гораздо выше, чем вероятность того, что она синяя.

Кривая на рис. 3.1 является *сигмоидной кривой*. Она представляет собой график функции, называемой *логистической функцией* (также известной как функция *logit*) (<https://oreil.ly/tZTVe>) и используемой в статистике уже несколько десятилетий.

Для логистической регрессии функция определяется следующим образом: x — входное значение, m и b — параметры, полученные в процессе обучения:

$$y = \frac{1}{1 + e^{-(mx+b)}}.$$

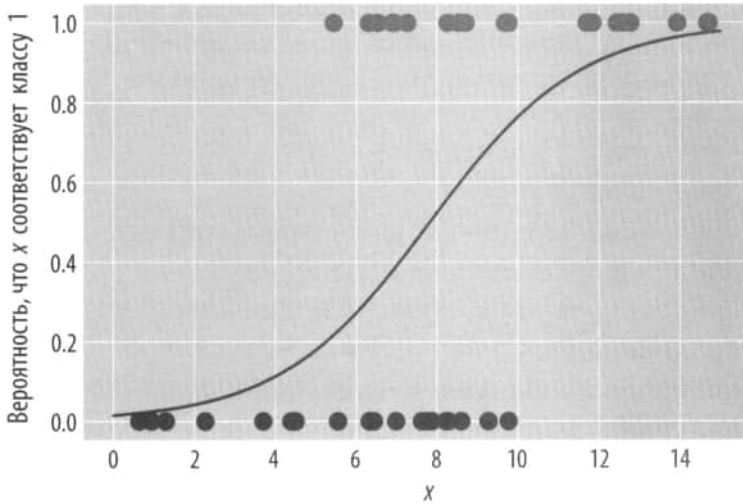


Рис. 3.1. Логистическая регрессия

Это уравнение показывает, почему логистическая *регрессия* называется именно так, несмотря на то, что это алгоритм классификации. Экспонента e соответствует уравнению линейной регрессии.

Алгоритм обучения логистической регрессии подгоняет логистическую функцию под распределение данных и использует полученные значения y в качестве вероятностей для классификации точек данных. Он работает с любым количеством признаков (не только x , но и x_1 , x_2 , x_3 и т. д.) и является параметрическим алгоритмом обучения, поскольку использует обучающие данные для поиска оптимальных значений m и b . Как он находит оптимальные значения — это деталь реализации, которую берут на себя такие библиотеки, как Scikit-Learn. По умолчанию Scikit использует алгоритм численной оптимизации, известный как *алгоритм ограниченной памяти Бройдена — Флетчера — Голдфарба — Шанно* (limited-memory Broyden — Fletcher — Goldfarb — Shanno, L-BFGS) (<https://oreil.ly/hhJw0>), но поддерживает и другие методы оптимизации. Это, кстати, одна из причин популярности Scikit в сообществе специалистов по машинному обучению. Нетрудно вычислить m и b по обучающим данным для модели линейной регрессии, но сложнее сделать это для модели логистической регрессии, не говоря уже о более сложных параметрических моделях, таких как метод опорных векторов (SVM).

Класс `LogisticRegression` (<https://oreil.ly/wpwGs>) в Scikit — это готовая логистическая регрессия.

С его помощью обучение модели логистической регрессии можно выполнить следующим образом:

```
model = LogisticRegression()
model.fit(x, y)
```

После обучения модели можно вызвать метод `predict`, чтобы предсказать, к какому классу относятся входные данные, или метод `predict_proba` с целью получения вычисленных вероятностей для каждого класса. Если для набора данных на рис. 3.1 используется модель `LogisticRegression`, то следующий оператор предскажет, соответствует $x = 10$ классу 0 или классу 1:

```
predicted_class = model.predict([[10.0]])[0]
print(predicted_class) # Дает на выходе 1
```

А эти операторы показывают вероятности, вычисленные для каждого класса:

```
predicted_probabilities = model.predict_proba([[10.0]])[0]
print(f'Class 0: {predicted_probabilities[0]}') # 0.23508543966167028
print(f'Class 1: {predicted_probabilities[1]}') # 0.7649145603383297
```

Scikit также включает класс `LogisticRegressionCV` (<https://oreil.ly/EPjhl>) для обучения моделей логистической регрессии со встроенной кросс-валидацией. (Если вам необходимо освежить эти вопросы в памяти, то перекрестная валидация (кросс-валидация) рассматривалась в главе 2.) За счет дополнительного времени обучения следующие операторы обучают модель логистической регрессии с использованием пяти частей:

```
model = LogisticRegressionCV(cv=5)
model.fit(x, y)
```

Логистическая регрессия технически является алгоритмом бинарной классификации, но ее можно использовать и для многоклассовой классификации. Подробнее об этом я расскажу в конце главы. Пока же считайте, что логистическая регрессия — это алгоритм машинного обучения, использующий хорошо известную логистическую функцию для количественной оценки вероятности того, что входные данные относятся к одному из двух классов, и вы будете иметь точное концептуальное представление о том, что такое логистическая регрессия.

Меры точности для классификационных моделей

Точность классификационной модели можно определить так же, как и для регрессионной модели: путем вызова метода `score` модели. Для классификатора `score` возвращает сумму истинноположительных и истинноотрицательных результатов, разделенную на общее количество образцов. Если тестовые данные включают 10 положительных (образцы класса 1) и 10 отрицательных (образцы класса 0) и модель правильно идентифицирует 8 положительных и 7 отрицательных образцов, то оценка будет равна $(8 + 7) / 20$, или 0,75. Это значение иногда называют *метрикой правильности модели* (accuracy score).

Существует множество других способов оценки классификационной модели, и выбор "правильного" способа часто зависит от того, как модель будет использоваться. Вместо того чтобы вычислять оценку точности, специалисты по изучению данных иногда измеряют *точность* (precision) и *полноту* (recall) (<https://oreil.ly/YJwCM>) классификационной модели.

Точность (precision).

Вычисляется путем деления числа истинноположительных результатов на сумму истинноположительных и ложноположительных результатов.

Полнота (recall).

Вычисляется путем деления числа истинноположительных результатов на сумму истинноположительных и ложноотрицательных результатов.

Таким образом, точность накладывает штраф на ложноположительные результаты (случаи, когда модель неверно предсказывает 1), а полнота штрафует ложноотрицательные результаты, снижая оценку, когда модель неверно предсказывает 0.

Рисунок 3.2 иллюстрирует разницу. Предположим, что вы обучили модель различать изображения белых медведей и моржей, а для ее тестирования предоставили три изображения белых медведей и три изображения моржей. Далее предположим, что модель правильно классифицирует два изображения белых медведей, но неправильно классифицирует два изображения моржей как изображения белых медведей, что отмечено красными рамками. В этом случае точность модели при идентификации белых медведей составляет 50%, поскольку только два из четырех изображений, классифицированных моделью как белые медведи, на самом деле являются белыми медведями. А вот полнота составляет 67%, поскольку модель правильно определила два из трех изображений белых медведей. Таковы вкратце точность и полнота. Первая определяет степень уверенности в точности положительного прогноза, а вторая — способность модели точно идентифицировать положительные образцы. Эти два показателя можно объединить в один, который называется *F1-мерой* (также известный как F score) (<https://oreil.ly/qGnR0>), используя простую формулу.



Рис. 3.2. Использование точности и полноты для измерения точности классификатора

Scikit предоставляет такие полезные функции, как `precision_score` (<https://oreil.ly/2x7yM>), `recall_score` (<https://oreil.ly/xp7tO>) и `f1_score` (<https://oreil.ly/fUu9A>), для получения метрик классификации. Что предпочесть — точность или полноту —

зависит от того, что выше: стоимость ложноположительных или стоимость ложноотрицательных результатов. Используйте точность, когда стоимость ложных срабатываний высока — например, когда "положительный" результат означает, что письмо является спамом. Вы предпочтете, чтобы спам-фильтр отправил несколько спамерских писем в папку **Входящие**, но не отправил нормальное (и потенциально важное) письмо в папку **Нежелательные**. В противоположность этому, если цена ложноотрицательных результатов высока, остановите свой выбор на полноте. Отличный пример — использование машинного обучения для обнаружения опухолей на рентгеновских и магнитно-резонансных снимках. Гораздо лучше ошибочно отправить пациента к врачу из-за ложноположительного результата, чем сказать ему, что опухоли нет, когда она действительно имеется.

Наблюдение за белыми медведями в дикой природе

Пример с белым медведем и моржом был взят из руководства, которое я написал для компании Microsoft и которое начиналось со следующего вступления:

"Вы являетесь руководителем группы ученых-климатологов, обеспокоенных сокращением популяции белых медведей в Арктике. Для решения этой проблемы ваша команда разместила сотни реагирующих на движение камер в стратегически важных локациях по всему региону. Вместо того чтобы вручную изучать каждую фотографию для определения, есть ли на ней белый медведь, вам необходимо разработать автоматизированную систему, которая будет обрабатывать данные с этих камер в режиме реального времени и выводить оповещение о том, что одной из камер зафиксирован белый медведь. Вам необходимо решение, использующее искусственный интеллект (ИИ), чтобы с высокой степенью точности определить, присутствует ли на фотографии белый медведь. И сделать это нужно быстро, поскольку изменение климата ждать не будет".

В этом руководстве используются несколько служб Azure для создания комплексного решения, а для визуализации применяется Power BI от Microsoft. Если вам интересно, можете ознакомиться с ним в Интернете (<https://oreil.ly/Q8Bwb>).

Метрика точности, точность, полнота и F1-мера применимы к моделям бинарной и многоклассовой классификации. Дополнительной метрикой, применимой только к бинарной классификации, является *кривая рабочей характеристики приемника* (receiver operating characteristic, ROC; <https://oreil.ly/BDvv9>), которая отображает соотношение истинноположительных результатов (true-positive rate, TPR) и ложноположительных результатов (false-positive rate, FPR) при различных пороговых значениях вероятности. Пример ROC-кривой показан на рис. 3.3. Ломаная линия, протянувшаяся из левого нижнего угла в правый верхний, указывает на то, что модель ошибается лишь в 50% случаев, что не лучше, чем угадывание для бинарного классификатора. Чем больше кривая загибается к левому верхнему углу, тем точнее модель. Специалисты по исследованию данных часто используют площадь под кривой (area under curve, AUC, или ROC AUC) в качестве общей меры точности.

Scikit предоставляет класс `RocCurveDisplay` (<https://oreil.ly/mJePr>) для построения ROC-кривых и функцию `roc_auc_score` (<https://oreil.ly/ryuxL>) для получения оценок ROC AUC. Баллы, возвращаемые этой функцией, представляют собой значения от 0,0 до 1,0. Чем выше балл, тем точнее модель.

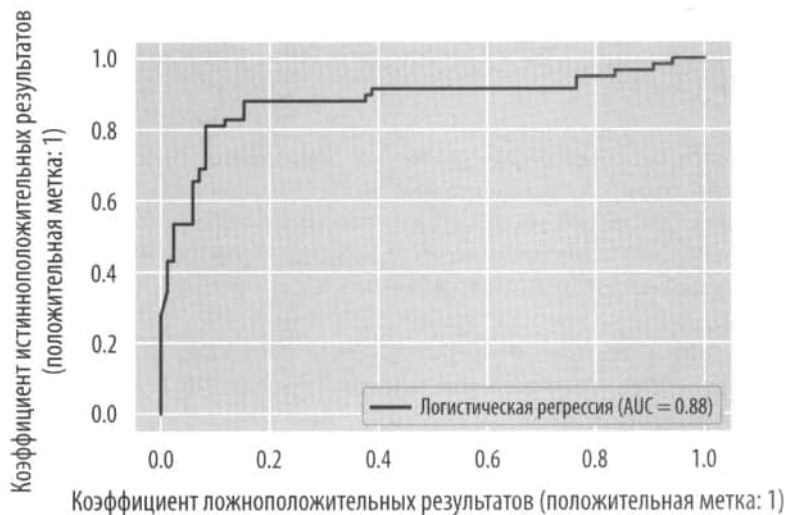


Рис. 3.3. Кривая рабочей характеристики приемника

Еще одним способом оценки точности классификационной модели является построение *матрицы ошибок* (confusion matrix; <https://oreil.ly/YxADc>), подобной приведенной на рис. 3.4. Она работает как для бинарной, так и для многоклассовой классификации и показывает для каждого класса, как работала модель во время тестирования.

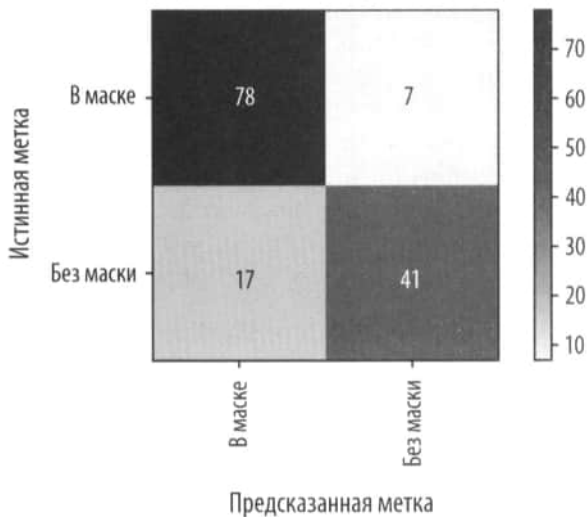


Рис. 3.4. Матрица ошибок

В данном примере модели было предложено отличить изображения, содержащие лица с масками, от изображений, содержащих лица без масок. Модель справилась с этой задачей 78 раз из 85, когда ей были представлены изображения людей в масках, и 41 раз из 58, когда ей были представлены изображения людей без масок. Scikit предлагает функцию `confusion_matrix` (<https://oreil.ly/U4JBb>) для вычисления матрицы ошибок и класс `ConfusionMatrixDisplay` (<https://oreil.ly/CLnP2>) с методами `from_estimator` (<https://oreil.ly/2XdMZ>) и `from_predictions` (<https://oreil.ly/6sXhx>) для построения матриц ошибок.



Бесчисленные примеры кода в Интернете используют функцию `plot_confusion_matrix` библиотеки Scikit для отображения матриц ошибок. Функция `ConfusionMatrixDisplay` появилась в Scikit 1.0 и является правильным способом генерации матриц ошибок. Функция `plot_confusion_matrix` будет удалена из библиотеки в версии 1.2.

Другие термины, которые можно встретить при обсуждении точности моделей классификации, включают *чувствительность* (sensitivity) и *специфичность* (specificity) (<https://oreil.ly/EhfoW>). Чувствительность идентична полноте, поэтому Scikit не предусматривает отдельного метода для ее расчета. Специфичность представляет собой полноту для отрицательного, а не положительного класса и рассчитывается путем деления числа истинноотрицательных результатов на сумму истинноотрицательных и ложноположительных результатов. В Scikit нет специальной функции для вычисления специфичности, но это можно сделать достаточно просто, вызвав `recall_score` с параметром `pos_label`, указывающим, что 0 (а не 1) является положительной меткой:

```
recall_score(y_test, y_predicted, pos_label=0)
```

Чувствительность и специфичность часто используются при тестировании лекарственных препаратов и обследовании на рак. Предположим, вы едете за границу, и перед возвращением домой вам необходим отрицательный тест на COVID. Если у вас нет COVID, какова вероятность того, что тест ошибочно покажет его наличие? (В последнее время я неоднократно задавал себе этот вопрос во время зарубежных поездок.) Ответ заключается в специфичности теста — насколько точно он определяет отрицательные образцы. Чувствительность, с другой стороны, показывает, насколько вероятно, что тест окажется верным, если сообщит, что у вас действительно есть COVID. Это тонкое различие, но очень важное, если вы обеспокоены тем, что ошибочный тест может помешать вам отправиться домой (специфичность) или если вы хотите быть уверены, что у вас нет COVID, прежде чем навестить пожилого родственника (чувствительность).

Категориальные данные

Машинное обучение находит закономерности в числах. Оно работает *только* с числами. Однако во многих наборах данных есть столбцы, содержащие строковые значения, такие как "мужчина" и "женщина" или "красный", "зеленый" и "синий". Специа-

листы по исследованию данных называют их *категориальными значениями*, а содержащие их столбцы — *категориальными столбцами*. Машинное обучение не может работать с категориальными значениями напрямую. Для того чтобы использовать их в модели, необходимо преобразовать их в числа.

Для преобразования категориальных значений в числовые существуют две популярные методики. Одна из них — кодирование меток, с которым вы кратко познакомились в примере кластеризации k средних в *главе 1*. При кодировании меток категориальные значения заменяются целыми числами. Если в столбце есть три уникальных значения, то при кодировании меток они заменяются на 0, 1 и 2. Для демонстрации выполните следующий код в блокноте Jupyter:

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

data = [[10, 'red'], [20, 'blue'], [12, 'red'], [16, 'green'], [22, 'blue']]
df = pd.DataFrame(data, columns=['Length', 'Color'])

encoder = LabelEncoder()
df['Color'] = encoder.fit_transform(df['Color'])
df.head()
```

Этот код создает `DataFrame`, содержащий числовой столбец `Length` (длина) и категориальный столбец `Color` (цвет), который содержит три различных категориальных значения. Как выглядит набор данных до начала работы с ним, показано на рис. 3.5.

А на рис. 3.6 продемонстрировано, как данные выглядят после того, как значения в столбце `Color` преобразованы (кодированы) в метки с помощью класса `LabelEncoder` от Scikit.

	Length	Color
0	10	red
1	20	blue
2	12	red
3	16	green
4	22	blue

Рис. 3.5. Набор данных до обработки

	Length	Color
0	10	2
1	20	0
2	12	2
3	16	1
4	22	0

Рис. 3.6. Набор данных после кодирования

Переработанный (кодированный) набор данных можно использовать для обучения ML-модели. Данные без преобразования — нет. Упорядоченный список классов, которые были переработаны, можно получить из атрибута `classes_` обработчика.

Другим, более распространенным способом кодирования категориальных значений в числовые является *горячее кодирование* (one-hot encoding), при котором в набор данных добавляется один столбец для каждого уникального значения в категориальном столбце, а преобразованные столбцы заполняются 1 и 0. Горячее кодирова-

ние можно выполнить с помощью класса `OneHotEncoder` (<https://oreil.ly/fkDgF>) из Scikit или путем вызова `get_dummies` из Pandas на `DataFrame`. Ниже показано, как последний способ используется для преобразования набора данных:

```
data = [[10, 'red'], [20, 'blue'], [12, 'red'], [16, 'green'], [22, 'blue']]
df = pd.DataFrame(data, columns=['Length', 'Color'])
```

```
df = pd.get_dummies(df, columns=['Color'])
df.head()
```

Результаты приведены на рис. 3.7.

	Length	Color_blue	Color_green	Color_red
0	10	0	0	1
1	20	1	0	0
2	12	0	0	1
3	16	0	1	0
4	22	1	0	0

Рис. 3.7. Фрейм данных после обработки `get_dummies`

Кодирование меток и горячее кодирование используются в задачах регрессии и классификации. Возникает очевидный вопрос: какое из них следует использовать? Как правило, специалисты по анализу данных отдают предпочтение горячему кодированию, а не кодированию меток. В первом случае каждому уникальному значению присваивается одинаковый вес, тогда как во втором подразумевается, что некоторые значения могут быть важнее других — например, "red" (2) важнее, чем "blue" (0). Кодирование меток, с другой стороны, более эффективно с точки зрения использования памяти. Количество столбцов до и после кодирования одинаково, в то время как при горячем кодировании добавляется по одному столбцу на каждое уникальное значение. Для очень больших наборов данных, содержащих тысячи уникальных значений в категориальном столбце, кодирование меток требует значительно меньше памяти.

Точность модели машинного обучения редко зависит от выбранного метода кодирования. Если вы сомневаетесь, то выбирайте горячее кодирование. Если же вы хотите быть уверенным, то можете закодировать данные обоими способами и сравнить результаты после обучения ML-модели.

Бинарная классификация

Бинарные классификаторы — это модели обучения с контролем, обучаемые на помеченных данных: нули для отрицательного класса и единицы для положительного. Прогнозы, которые они делают, тоже имеют значения 0 и 1. Они также предоставляют вероятность для каждого класса, которую можно учитывать в своих выво-

дах. Например, компания, выпускающая кредитные карты, может решить, что транзакция будет отклонена только в том случае, если модель с вероятностью не менее 99% предсказывает, что транзакция является мошеннической. В этом случае вероятность, которую вычислила модель, важнее, чем исходный прогноз, который она сделала.

Для того чтобы наглядно продемонстрировать все, что было рассказано о бинарной классификации, давайте с помощью Scikit построим несколько моделей: сначала простую, демонстрирующую основные принципы, а затем вторую, решающую реальную бизнес-задачу.

Классификация пассажиров, пливших на "Титанике"

Одним из наиболее известных открытых наборов данных в области машинного обучения является набор данных Titanic (<https://oreil.ly/JtxlV>), содержащий информацию о сотнях пассажиров, отправившихся в злополучное плавание на судне RMS Titanic, в том числе сведения о том, кто из них выжил, а кто — нет. Давайте с помощью логистической регрессии построим бинарную классификационную модель на основе этого набора данных и посмотрим, сможем ли мы предсказать вероятность того, что пассажир выживет, учитывая его пол, возраст и тарифный класс (ехал ли он первым, вторым или третьим классом).

Прежде всего, необходимо загрузить набор данных (<https://oreil.ly/vMnPI>) и скопировать его в подкаталог Data каталога, в котором размещены блокноты Jupyter. Затем выполните следующий код в блокноте для загрузки набора данных и ознакомления с его содержимым:

```
import pandas as pd
```

```
df = pd.read_csv('Data/titanic.csv')
df.head()
```

На рис. 3.8 приведен результат.

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked	
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cummings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

Рис. 3.8. Загруженный набор данных Titanic

Набор данных содержит 891 строку и 12 столбцов. Некоторые столбцы, такие как PassengerId и Name, не имеют значения для модели машинного обучения. Другие же очень важны. На них мы и сосредоточимся.

Survived (выживший).

Указывает, выжил ли пассажир во время плавания (1) или нет (0).

Pclass (класс).

Указывает, плыл ли пассажир первым классом (1), вторым классом (2) или третьим классом (3).

Sex (пол).

Указывает пол пассажира.

Age (возраст).

Указывает возраст пассажира.

Столбец Survived — это столбец-метка, который мы попытаемся предсказать. Остальные столбцы имеют значение, т. к. пассажиры первого класса имели больше шансов выжить после кораблекрушения (<https://oreil.ly/i0N6s>), поскольку их каюты находились ближе к верхней палубе корабля и ближе к спасательным шлюпкам. Кроме того, женщинам и детям чаще предоставляли места в спасательных шлюпках.

Теперь с помощью следующего оператора можно проверить, не пропущены ли в наборе данных какие-либо значения:

```
df.info()
```

Вот выходные данные:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  891 non-null   int64
1   Survived     891 non-null   int64
2   Pclass       891 non-null   int64
3   Name         891 non-null   object
4   Sex          891 non-null   object
5   Age          714 non-null   float64
6   SibSp        891 non-null   int64
7   Parch        891 non-null   int64
8   Ticket       891 non-null   object
9   Fare         891 non-null   float64
10  Cabin        204 non-null   object
11  Embarked     889 non-null   object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
```

В столбце Cabin (каюта) отсутствует множество значений, но это не важно, т. к. мы не будем использовать этот столбец. Мы *будем* использовать столбец Age (возраст), и в этом столбце также отсутствуют некоторые значения. Их можно было бы заменить средним значением всех остальных возрастов — такой подход специалиста по

обработке данных называют *вменением* недостающих значений, но мы воспользуемся более простым подходом — удалим строки с недостающими значениями. Используйте следующие операторы для удаления ненужных столбцов, удаления строк с пропущенными значениями и горячего кодирования значений в столбцах Sex и Pclass:

```
df = df[['Survived', 'Age', 'Sex', 'Pclass']]
df = pd.get_dummies(df, columns=['Sex', 'Pclass'])
df.dropna(inplace=True)
df.head()
```

На рис. 3.9 представлен полученный набор данных.

	Survived	Age	Sex_female	Sex_male	Pclass_1	Pclass_2	Pclass_3
0	0	22.0	0	1	0	0	1
1	1	38.0	1	0	1	0	0
2	1	26.0	1	0	0	0	1
3	1	35.0	1	0	1	0	0
4	0	35.0	0	1	0	0	1

Рис. 3.9. Набор данных после удаления строк с пропущенными значениями и горячего кодирования

Следующей задачей является разделение набора данных на обучающий и тестовый:

```
from sklearn.model_selection import train_test_split
```

```
x = df.drop('Survived', axis=1)
y = df['Survived']
```

```
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2,
                                                    stratify=y, random_state=0)
```

Обратите внимание на параметр `stratify=y`, переданный в `train_test_split`. Это важно, поскольку из 714 образцов, оставшихся после удаления строк с пропущенными значениями, 290 представляют выживших пассажиров и 424 — невыживших. Мы хотим, чтобы обучающая и тестовая выборки содержали одинаковые доли обоих классов, и `stratify=y` позволяет добиться этого. Без стратификации (формирования выборки) модель может показаться не такой точной, как она есть на самом деле.

Теперь создадим модель логистической регрессии, обучим ее на данных, выделенных для обучения, и оценим ее на тестовых данных:

```
from sklearn.linear_model import LogisticRegression
```

```
model = LogisticRegression(random_state=0)
model.fit(x_train, y_train)
model.score(x_test, y_test)
```

Повторно оцените модель с помощью кросс-валидации для большей уверенности в полученном результате. Помните, что это *метрика правильности* (accuracy score), вычисленная путем суммирования истинноположительных и истинноотрицательных результатов и деления на общее количество образцов:

```
from sklearn.model_selection import cross_val_score
```

```
cross_val_score(model, x, y, cv=5).mean()
```

Используйте следующий код для отображения матрицы ошибок, показывающей, как именно работала модель во время тестирования:

```
%matplotlib inline
```

```
from sklearn.metrics import ConfusionMatrixDisplay as cmd
```

```
cmd.from_estimator(model, x_test, y_test,
                  display_labels=['Perished', 'Survived'],
                  cmap='Blues', xticks_rotation='vertical')
```

Обратите внимание, что модель точнее предсказывает, что пассажиры не выживут, чем что выживут. Это объясняется тем, что использовавшийся для обучения модели набор данных содержал больше примеров погибших пассажиров, чем выживших. Всегда предпочтительнее обучать бинарную модель классификации на идеально сбалансированном наборе данных, содержащем равное количество положительных и отрицательных образцов, но вполне допустимо обучение на несбалансированном наборе данных, если при оценке точности модели этот дисбаланс учитывается.

Теперь с помощью функций Scikit `precision_score` и `recall_score` вычислите точность, полноту, чувствительность и специфичность модели:

```
from sklearn.metrics import precision_score, recall_score
```

```
y_pred = model.predict(x_test)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
sensitivity = recall
specificity = recall_score(y_test, y_pred, pos_label=0)
```

```
print(f'Precision: {precision}')      # точность
print(f'Recall: {recall}')           # полнота
print(f'Sensitivity: {sensitivity}')  # чувствительность
print(f'Specificity: {specificity}')  # специфичность
```

Согласуется ли высокое значение специфичности с тем, что модель лучше определяет пассажиров, которые не выживут, чем тех, которые выживут? Как бы вы объяснили относительно низкие показатели полноты и чувствительности?

Теперь давайте используем обученную модель для составления прогнозов. В первых, выясним, выживет ли 30-летняя женщина, путешествующая первым клас-

сом. Поскольку модель обучалась на основе содержащего имена столбцов фрейма данных, мы будем использовать эти же имена столбцов при формировании входных данных:

```
female = pd.DataFrame({ 'Age': [30], 'Sex_female': [1], 'Sex_male': [0],
                        'Pclass_1': [1], 'Pclass_2': [0], 'Pclass_3': [0] })
```

```
model.predict(female)[0]
```

Модель предсказывает, что женщина выживет, но каковы *шансы* того, что она выживет?

```
probability = model.predict_proba(female)[0][1]
print(f'Probability of survival: {probability:.1%}')
```

Вероятность того, что путешествующая первым классом 30-летняя женщина переживет крушение, составляет более 90%. А как насчет 60-летнего мужчины, путешествующего третьим классом?

```
male = pd.DataFrame({ 'Age': [60], 'Sex_female': [0], 'Sex_male': [1],
                      'Pclass_1': [0], 'Pclass_2': [0], 'Pclass_3': [1] })
```

```
probability = model.predict_proba(male)[0][1]
print(f'Probability of survival: {probability:.1%}')
```

Не бойтесь экспериментировать с другими исходными данными, чтобы увидеть, что показывает модель. Например, какова вероятность того, что путешествующий вторым классом 12-летний мальчик выживет при крушении "Титаника"?

Обнаружение мошенничества с кредитными картами

Одним из наиболее интересных направлений использования машинного обучения сегодня является выявление мошеннических финансовых операций. Компании, выпускающие кредитные карты, применяют ML для формирования решения о принятии или отклонении платежей. Хотя эти компании по понятным причинам не стремятся раскрывать подробности того, как они это делают и какие данные используются для обучения их моделей, по крайней мере один такой набор данных был опубликован для всеобщего пользования. Данные в нем были анонимизированы с помощью методики, называемой *анализом главных компонент* (principal component analysis, PCA), о котором я расскажу в главе 6.

Набор данных показан на рис. 3.10. Данные получены из реальных транзакций, совершенных европейскими держателями кредитных карт в сентябре 2013 года. Большинство столбцов имеют неинформативные названия, например V1 и V2, и содержат такие же непрозрачные значения. Три столбца — Time (время), Amount (сумма) и Class (класс) — имеют реальные названия и неизменные значения, показывающие время проведения операции, ее сумму, а также то, была ли операция законной (Class=0) или мошеннической (Class=1).

	Time	V1	V2	V3	...	V27	V28	Amount	Class
0	0.0	-1.359807	-0.072781	2.536347	...	0.133558	-0.021053	149.62	0
1	0.0	1.191857	0.266151	0.166480	...	-0.008983	0.014724	2.69	0
2	1.0	-1.358354	-1.340163	1.773209	...	-0.055353	-0.059752	378.66	0
3	1.0	-0.966272	-0.185226	1.792993	...	0.062723	0.061458	123.50	0
4	2.0	-1.158233	0.877737	1.548718	...	0.219422	0.215153	69.99	0
5	2.0	-0.425966	0.960523	1.141109	...	0.253844	0.081080	3.67	0
6	4.0	1.229658	0.141004	0.045371	...	0.034507	0.005168	4.99	0
7	7.0	-0.644269	1.417964	1.074380	...	-1.206921	-1.085339	40.80	0
8	7.0	-0.894286	0.286157	-0.113192	...	0.011747	0.142404	93.20	0
9	9.0	-0.338262	1.119593	1.044367	...	0.246219	0.083076	3.68	0

Рис. 3.10. Набор данных для обнаружения мошенничества

Каждая строка представляет собой одну транзакцию. Из 284 807 транзакций в наборе данных только 492 являются мошенническими. Этот набор данных очень несбалансирован, поэтому можно ожидать, что обученная на нем ML-модель будет гораздо лучше классифицировать легитимные транзакции, чем мошеннические. Это не обязательно проблема, поскольку занимающиеся кредитными картами компании скорее ошибутся в классификации мошеннических транзакций и пропустят 100 из них, чем ошибутся в классификации одной легитимной транзакции и разозлят клиента.

Начнем с загрузки ZIP-файла (<https://oreil.ly/EYbNK>), содержащего набор данных. Скопируйте файл `creditcard.csv` из ZIP-файла в подкаталог `Data` вашего блокнота, а затем выполните следующий код в блокноте Jupyter для загрузки набора данных и отображения первых нескольких строк:

```
import pandas as pd
```

```
df = pd.read_csv('Data/creditcard.csv')
df.head()
```

Выясните, сколько строк содержит набор данных и есть ли в этих строках пропущенные значения:

```
df.info()
```

Набор данных содержит 284 807 строк, и ни в одной из них нет пропущенных значений. Разделите данные на обучающие и тестовые и с помощью параметра `stratify` в `train_test_split` убедитесь, что соотношение легитимных и мошеннических транзакций в обучающем и тестовом наборах данных одинаково:

```
from sklearn.model_selection import train_test_split
```

```
x = df.drop(['Time', 'Class'], axis=1)
y = df['Class']
```

```
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2,
                                                    stratify=y, random_state=0)
```

Обучите модель логистической регрессии разделению классов:

```
from sklearn.linear_model import LogisticRegression
```

```
lr_model = LogisticRegression(random_state=0, max_iter=5000)
lr_model.fit(x_train, y_train)
```

Обратите внимание на параметр `max_iter=5000`, передаваемый функции `LogisticRegression`. Параметр `max_iter` задает максимальное количество итераций, допустимых для сходимости решения при настройке логистической функции на набор данных. По умолчанию это значение равно 100, что в данном примере недостаточно. Увеличение значения до 5000 дает внутреннему решателю необходимый запас для поиска результата.

Обычный показатель точности, рассчитываемый путем деления суммы истинноположительных и истинноотрицательных результатов на количество тестовых образцов, не очень полезен, поскольку набор данных крайне несбалансирован. Мошеннические транзакции составляют менее 0,2% всех образцов, а это означает, что модель может просто предположить, что каждая транзакция является законной, и сделать это правильно примерно в 99,8% случаев. Для визуализации работы модели в процессе тестирования можно использовать матрицу ошибок:

```
%matplotlib inline
```

```
from sklearn.metrics import ConfusionMatrixDisplay as cmd
```

```
labels = ['Легитимная', 'Мошенническая']
cmd.from_estimator(lr_model, x_test, y_test, display_labels=labels,
                  cmap='Blues', xticks_rotation='vertical')
```

Результат приведен на рис. 3.11.

Модель логистической регрессии правильно идентифицировала 56 853 транзакции как легитимные, а ошибочно классифицировала легитимные транзакции как мошеннические всего 11 раз. Мы хотим минимизировать последнее число, поскольку не желаем раздражать клиентов отклонением законных транзакций. Посмотрим, сможет ли классификатор со случайным лесом сделать это лучше:

```
from sklearn.ensemble import RandomForestClassifier
```

```
rf_model = RandomForestClassifier(random_state=0)
rf_model.fit(x_train, y_train)
```

```
cmd.from_estimator(rf_model, x_test, y_test, display_labels=labels,
                  cmap='Blues', xticks_rotation='vertical')
```

Результат приведен на рис. 3.12.



Рис. 3.11. Матрица ошибок для модели логистической регрессии

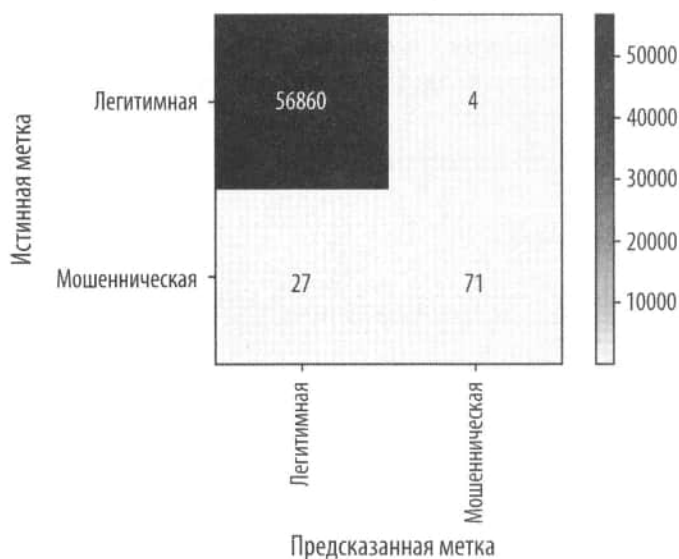


Рис. 3.12. Матрица ошибок классификатора со случайным лесом

Случайный лес принял за мошеннические всего 4 легитимные транзакции. Это улучшение по сравнению с логистической регрессией. Посмотрим, сможет ли классификатор с градиентным бустингом решить эту задачу еще лучше:

```
from sklearn.ensemble import GradientBoostingClassifier
```

```
gbm_model = GradientBoostingClassifier(random_state=0)
```

```
gbm_model.fit(x_train, y_train)
```

```
cmd.from_estimator(gbm_model, x_test, y_test, display_labels=labels,
                  cmap='Blues', xticks_rotation='vertical')
```

Результат приведен на рис. 3.13.



Рис. 3.13. Матрица ошибок классификатора с градиентным бустингом

Модель градиентного бустинга (GBM) ошибочно классифицировала больше легитимных транзакций, чем случайный лес, поэтому мы остановимся на случайном лесе. Из 56 864 легитимных транзакций случайный лес правильно классифицировал 56 860. Это означает, что легитимные транзакции классифицируются правильно более чем в 99,99% случаев. При этом модель уловила около 75% мошеннических транзакций.

Используйте следующий код, чтобы измерить точность, полноту, чувствительность и специфичность классификатора случайного леса:

```
from sklearn.metrics import precision_score, recall_score
```

```
y_pred = rf_model.predict(x_test)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
sensitivity = recall
specificity = recall_score(y_test, y_pred, pos_label=0)
```

```
print(f'Точность: {precision}')
print(f'Полнота: {recall}')
print(f'Чувствительность: {sensitivity}')
print(f'Специфичность: {specificity}')
```

Ниже приведен результат:

Точность: 0.9466666666666667

Полнота: 0.7244897959183674

Чувствительность: 0.7244897959183674

Специфичность: 0.9999296567248172

Учитывая стремление компаний — эмитентов кредитных карт сделать так, чтобы клиенты были довольны и тратили деньги, как вы думаете, какие из этих показателей их интересуют больше всего? Если вы ответили "специфичность", то вы ответили правильно. Специфичность — это показатель того, насколько надежен тест, не позволяющий ошибочно классифицировать отрицательный образец как положительный — в данном случае не позволяющий классифицировать легитимную транзакцию как мошенническую.

К сожалению, вы не можете делать прогнозы с помощью этой модели, поскольку не знаете значения чисел в столбцах V1–V28, и не можете сгенерировать значения столбцов из новой транзакции, поскольку не знаете примененного к исходному набору данных преобразования. Вы даже не знаете, как выглядит исходный набор данных. Скорее всего, каждая строка содержит информацию о владельце карты, например годовой доход, кредитный рейтинг, возраст, страну проживания, сумму, потраченную по карте за последний год, а также информацию о приобретенном товаре и месте его покупки. Поиск признаков в машинном обучении — выяснение того, какие данные имеют отношение к модели, которую вы пытаетесь построить, — не менее, если не более, сложная задача, чем подготовка данных и выбор алгоритма обучения.

В реальной жизни модели, которые используют компании — эмитенты кредитных карт для выявления мошенничества, более сложны, чем эта, и часто включают в себя несколько моделей, поскольку ни одна из них не является на 100% точной. Например, компания может построить три модели и позволить им голосовать за то, является ли та или иная транзакция легитимной. Как бы то ни было, вы доказали принцип, согласно которому при наличии необходимых характеристик можно построить классификационную модель, достаточно точно выявляющую мошенничество с кредитными картами. И вы на собственном опыте убедились, как легко Scikit позволяет экспериментировать с различными алгоритмами обучения, чтобы определить, какой из них дает наиболее полезную модель.

Мультиклассовая классификация

Настало время перейти к *мультиклассовой (многоклассовой) классификации*, в которой существует не два, а n возможных исходов. Отличным примером мультиклассовой классификации является оптическое распознавание символов: изучаем написанную от руки цифру и предсказываем, какой цифре от 0 до 9 она соответствует. Другой пример — фотография лица и идентификация человека на ней путем прогона ее через модель, обученную распознавать людей.

Практически все, что вы узнали о бинарной классификации, применимо и к мультиклассовой классификации. В Scikit любой классификатор, работающий с бинарной классификацией, работает и с моделями мультиклассовой классификации. Важность этого трудно переоценить. Некоторые алгоритмы обучения, например логистическая регрессия, работают только в сценариях бинарной классификации. Многие библиотеки машинного обучения заставляют писать явный код с целью применения логистической регрессии для мультиклассовой классификации или вообще использовать другую форму логистической регрессии. Scikit так не делает. Вместо этого библиотека гарантирует, что классификаторы типа `LogisticRegression` будут работать в любом сценарии, и чтобы добиться результата, при необходимости выполняет дополнительную работу за кулисами.

Для логистической регрессии Scikit использует одну из двух стратегий работы алгоритма в многоклассовых сценариях. (Вы можете указать используемую стратегию с помощью параметра `multi_class` класса `LogisticRegression` или принять значение по умолчанию `'auto'` и позволить Scikit выбирать самостоятельно.) Одна из них — *мультиномиальная логистическая регрессия*, которая заменяет логистическую функцию на функцию `softmax` (<https://oreil.ly/BXtMC>), дающую несколько вероятностей — по одной на класс. Другая модель — *one-vs-rest*, также известная как *one-vs-all* ("один против всех"), которая обучает n моделей бинарной классификации, где n — количество классов, которые может предсказать модель. Каждая из n моделей сопоставляет один класс со всеми остальными классами, и когда модель должна сделать прогноз, она прогоняет входные данные через все n моделей и использует результат той из них, которая дает наибольшую вероятность. Эта стратегия показана на рис. 3.14.



Рис. 3.14. Стратегии использования бинарных алгоритмов классификации для поддержки мультиклассовой классификации

Подход "один против всех" хорошо работает для логистической регрессии, но для некоторых алгоритмов бинарной классификации Scikit задействует подход "один против одного". Например, когда вы используете класс `SVC` (классификатор на основе машины опорных векторов, о котором вы узнаете в главе 5) для выполнения мультиклассовой классификации, Scikit строит одну модель для каждой пары клас-

сов. Если модель включает четыре возможных класса, то Scikit "под капотом" создает не менее семи моделей.

Для построения модели мультиклассовой классификации ничего этого знать не нужно. Но это объясняет, почему некоторые модели мультиклассовой классификации требуют больше памяти и обучаются медленнее, чем другие. Некоторые алгоритмы классификации, такие как случайные леса и градиентный бустинг, изначально поддерживают мультиклассовую классификацию. Для алгоритмов, которые этого не делают, Scikit выполнит работу за вас. Она восполняет этот пробел и делает это максимально прозрачно.

Повторим: все классификаторы Scikit способны выполнять бинарную классификацию и мультиклассовую классификацию. Это упрощает написанный код и позволяет сосредоточиться на построении и обучении моделей, а не на понимании глубокой механики алгоритмов.

Построение модели распознавания цифр

Хотите самостоятельно разобраться с мультиклассовой классификацией? Как насчет модели, которая изучает отсканированные рукописные цифры и предсказывает, каким цифрам от 0 до 9 они соответствуют? Почтовая служба США построила подобную модель много лет назад для распознавания рукописных почтовых индексов в рамках автоматизации сортировки почты. Мы будем использовать примерный набор данных, встроенный в Scikit: набор данных Optical Recognition of Handwritten Digits (<https://oreil.ly/i55Ob>) Калифорнийского университета в Ирвайне, содержащий почти 1800 рукописных цифр. Каждая цифра представлена массивом чисел 8×8 от 0 до 16, причем чем больше число, тем более темные пиксели оно означает. Мы будем использовать логистическую регрессию для построения прогнозов на основе этих данных. На рис. 3.15 показаны первые 50 цифр из набора данных.

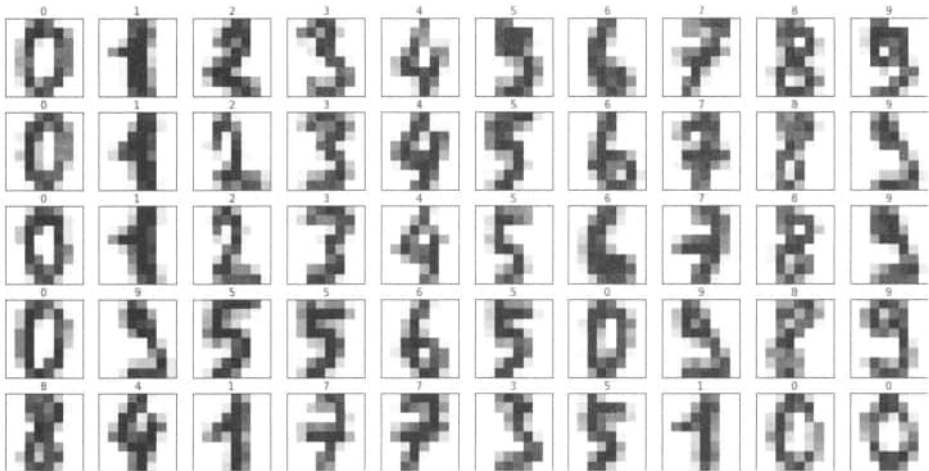


Рис. 3.15. Первые 50 цифр в наборе данных Optical Recognition of Handwritten Digits

Начните с создания блокнота Jupyter и выполнения следующих операторов в первой ячейке:

```
from sklearn import datasets
```

```
digits = datasets.load_digits()
print('digits.images: ' + str(digits.images.shape))
print('digits.target: ' + str(digits.target.shape))
```

Вот как выглядит первая цифра в числовом виде:

```
digits.images[0]
```

А вот как это выглядит на глаз:

```
%matplotlib inline
```

```
import matplotlib.pyplot as plt
```

```
plt.tick_params(axis='both', which='both', bottom=False, top=False,
                left=False, right=False, labelbottom=False, labelleft=False)
plt.imshow(digits.images[0], cmap=plt.cm.gray_r)
```

Очевидно, что это 0, но в этом можно убедиться по метке:

```
digits.target[0]
```

Постройте первые 50 изображений и покажите соответствующие метки:

```
fig, axes = plt.subplots(5, 10, figsize=(12, 7),
                        subplot_kw={'xticks': [], 'yticks': []})
```

```
for i, ax in enumerate(axes.flat):
    ax.imshow(digits.images[i], cmap=plt.cm.gray_r)
    ax.text(0.45, 1.05, str(digits.target[i]), transform=ax.transAxes)
```

Классификационные модели лучше всего работают со сбалансированными наборами данных. Используйте следующий код для построения графика распределения образцов (рис. 3.16).

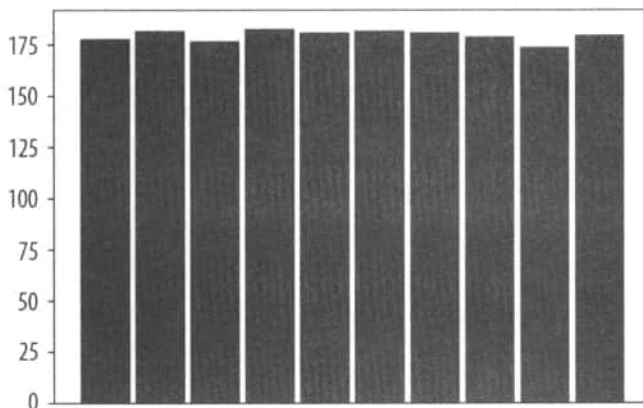


Рис. 3.16. Гистограмма распределения образцов

Набор данных практически идеально сбалансирован, поэтому давайте разделим его и обучим модель логистической регрессии:

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
```

```
x_train, x_test, y_train, y_test = train_test_split(
    digits.data, digits.target, test_size=0.2, random_state=0)
```

```
model = LogisticRegression(max_iter=5000)
model.fit(x_train, y_train)
```

Для количественной оценки точности модели используйте метод `score`:

```
model.score(x_test, y_test)
```

Используйте матрицу ошибок, чтобы увидеть, как модель работает на тестовом наборе данных:

```
%matplotlib inline
import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay as cmd
import seaborn as sns
sns.set()
```

```
fig, ax = plt.subplots(figsize=(8, 8))
ax.grid(False)
cmd.from_estimator(model, x_test, y_test, cmap='Blues', colorbar=False, ax=ax)
```

Полученный результат дает обнадеживающую картину: большие числа и темные цвета вдоль диагонали и маленькие числа и светлые цвета вне диагонали (рис. 3.17). В идеальной модели все нули были бы вне диагонали, но, конечно, идеальных моделей не существует.

Выберите одну из цифр из набора данных и отрисуйте ее, чтобы увидеть, как она выглядит:

```
sns.reset_orig() # Отмена sns.set()
plt.tick_params(axis='both', which='both', bottom=False, top=False,
                left=False, right=False, labelbottom=False, labelleft=False)
plt.imshow(digits.images[100], cmap=plt.cm.gray_r)
```

Передайте ее в модель и посмотрите, какую цифру предскажет модель:

```
model.predict([digits.data[100]])[0]
```

Какие вероятности предсказывает модель для каждой возможной цифры?

```
model.predict_proba([digits.data[100]])
```

Какова вероятность того, что цифра равна 4?

```
model.predict_proba([digits.data[100]])[0][4]
```

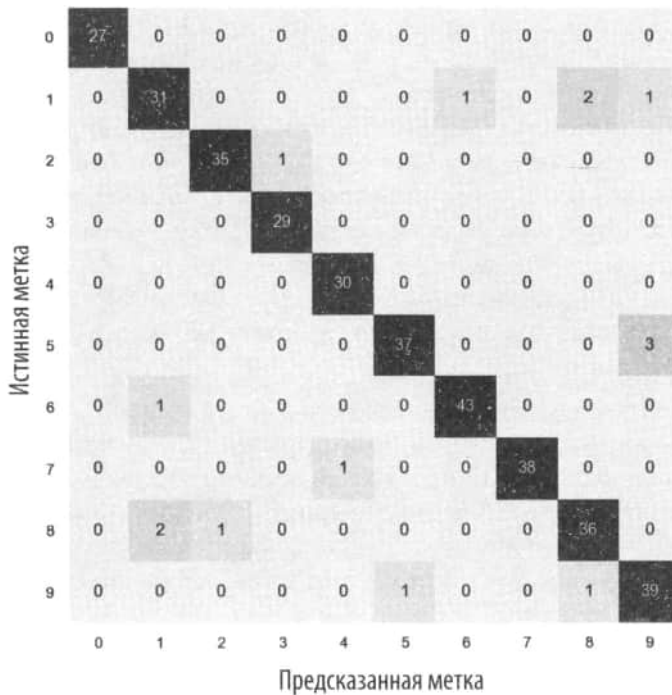


Рис. 3.17. Матрица ошибок для построенной модели распознавания цифр

При использовании бинарной классификации `predict_proba` возвращает две вероятности: одну для отрицательного класса и одну для положительного. При мультиклассовой классификации `predict_proba` возвращает вероятности для каждого возможного класса. Это позволяет оценить уверенность модели в прогнозе, возвращаемом функцией `predict`. Неудивительно, что `predict` возвращает класс, которому присвоена наибольшая вероятность.

Резюме

Классификационные модели широко используются в прикладных отраслях для предсказания категориальных результатов, таких как принятие или отклонение транзакции по кредитной карте. Модели бинарной классификации предсказывают один из двух исходов, в то время как модели мультиклассовой классификации предсказывают более двух исходов. Одним из наиболее распространенных алгоритмов обучения для классификационных моделей является логистическая регрессия, которая подстраивает уравнение под обучающие данные и использует его для прогнозирования исходов, вычисляя вероятность того, что каждый из них будет правильным, и выбирая исход с наибольшей вероятностью.

Существует множество способов оценки классификационных моделей. Выбор правильного метода зависит от того, как вы собираетесь использовать модель. Например, если стоимость ложных срабатываний высока, для оценки точности модели

следует использовать метрику точности (precision). Она рассчитывается путем деления числа истинноположительных результатов на сумму истинноположительных и ложноположительных результатов. С другой стороны, если стоимость ложноотрицательных результатов высока, вместо этого следует использовать полноту (recall). Она рассчитывается путем деления числа истинноположительных результатов на сумму истинноположительных и ложноотрицательных результатов. С метриками точности и полноты тесно связаны метрики чувствительности и специфичности. Чувствительность идентична полноте, в то время как специфичность — это полнота для отрицательного класса, а не для положительного. Матрицы ошибок позволяют наглядно представить, как модель работает на тестовых данных, не сводя точность к одному числу.

Некоторые алгоритмы обучения работают только с бинарной классификацией, но в Scikit добавлена программная магия, благодаря которой любой алгоритм обучения может быть использован для бинарной или мультиклассовой классификации. Это справедливо не для всех библиотек машинного обучения. Некоторые из них ограничивают применение алгоритмов обучения конкретными сценариями или требуют написания дополнительного кода для использования бинарной классификации в мультиклассовой модели.

Все модели, рассмотренные в этой главе, обучались на числовых данных, хотя некоторые наборы данных содержали категориальные значения — значения, являющиеся строками, а не числами, — которые приходилось преобразовывать в числа с помощью горячего кодирования. Вы можете задаться вопросом: как построить модель классификации, работающую исключительно с текстом, например модель, оценивающую отзывы о ресторане по настроению или классифицирующую электронные письма как спам или не-спам? Это вполне закономерный вопрос. И именно ему посвящена *глава 4*.

Классификация текста

Одним из наиболее распространенных применений бинарной классификации является анализ настроений, при котором образец текста, например отзыв о товаре, твит или комментарий, оставленный на сайте, оценивается по шкале от 0,0 до 1,0, где 0,0 — негативные настроения, а 1,0 — позитивные. Такой отзыв, как "отличный продукт по отличной цене", может получить 0,9 балла, а "продукт по завышенной цене, который почти не работает" — 0,1 балла. Балл — это вероятность того, что текст выражает положительные настроения. Модели анализа настроений сложно построить алгоритмически, но относительно легко — с помощью машинного обучения. Примеры использования анализа настроений в современном бизнесе приведены в статье Николаса Бианки "8 реальных примеров использования анализа настроений" (Bianchi Nicholas. 8 Sentiment Analysis Real-World Use Cases; <https://oreil.ly/nWq4a>).

Анализ настроения — это один из примеров задачи, в которой классифицируются текстовые, а не числовые данные. Поскольку машинное обучение работает с числами, перед обучением модели анализа настроений, модели идентифицирующей спам, или любой другой классифицирующей текст модели, необходимо преобразовать текст в числа. Распространенным подходом является построение таблицы частот слов, называемой *мешком слов* (bag of words). Scikit-Learn предоставляет классы, которые могут помочь в этом. В этой библиотеке также предусмотрена поддержка нормализации текста, чтобы, например, слова *awesome* и *Awesome* не считались двумя разными словами.

В начале этой главы описывается, как подготовить текст для использования в моделях классификации. После построения модели анализа настроений вы узнаете о другом популярном алгоритме обучения — Naive Bayes, который особенно хорошо работает с текстом, и воспользуетесь им для создания модели, позволяющей отличать обычные письма от спама. Наконец, вы узнаете о математической методике измерения сходства двух текстовых образцов и используете ее для создания приложения, которое рекомендует фильмы на основе других понравившихся вам фильмов.

Подготовка текста к классификации

Прежде чем обучать модель классификации текста, необходимо преобразовать текст в числа — этот процесс называется *векторизацией*. В *главе 1* была приведена

держит значения частоты терминов и обратной частоты документов (term frequency, inverse document frequency, TFIDF; <https://oreil.ly/NQrDJ>) от 0,0 до 1,0, отражающие относительную важность отдельных слов.

Все три класса способны преобразовывать текст в строчные буквы, удалять знаки препинания, удалять стоп-слова, разбивать предложения на отдельные слова и т. д. Они также поддерживают *n*-граммы, представляющие собой комбинации двух или более последовательных слов (число *n* задается пользователем), которые должны рассматриваться как одно слово. Идея заключается в том, что такие слова, как *credit* и *score*, могут быть более значимыми, если они встречаются в предложении рядом друг с другом, чем если они находятся далеко друг от друга. Без *n*-грамм относительная близость слов игнорируется. Недостатком использования *n*-грамм является увеличение объема памяти и времени обучения. Однако при разумном использовании они могут сделать модели классификации текста более точными.



Нейронные сети имеют другие, более мощные способы учета порядка слов, которые не требуют, чтобы связанные слова находились рядом друг с другом. Обычная модель машинного обучения не может связать слова "синий" и "небо" в предложении "Мне нравится синий, потому что это цвет неба", а нейронная сеть может. Более подробно я расскажу об этом в *главе 13*.

Вот пример, демонстрирующий работу CountVectorizer:

```
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer

lines = [
    'Four score and 7 years ago our fathers brought forth,',
    '... a new NATION, conceived in liberty $$$,',
    'and dedicated to the PrOpOsItIoN that all men are created equal',
    'One nation\'s freedom equals #freedom for another $nation!'
]

# Осуществляем векторизацию
vectorizer = CountVectorizer(stop_words='english')
word_matrix = vectorizer.fit_transform(lines)

# Показываем результирующую матрицу слов
feature_names = vectorizer.get_feature_names_out()
line_names = [f'Line {(i + 1):d}' for i, _ in enumerate(word_matrix)]

df = pd.DataFrame(data=word_matrix.toarray(), index=line_names, columns=feature_names)

df.head()
```

Выходные данные приведены на рис. 4.2.

	ago	brought	conceived	created	dedicated	equal	equals	fathers	forth	freedom	liberty	men	nation	new	proposition	score	years
Line 1	1	1	0	0	0	0	0	1	1	0	0	0	0	0	0	1	1
Line 2	0	0	1	0	0	0	0	0	0	0	1	0	1	1	0	0	0
Line 3	0	0	0	1	1	1	0	0	0	0	0	1	0	0	1	0	0
Line 4	0	0	0	0	0	0	1	0	0	2	0	0	2	0	0	0	0

Рис. 4.2. Словарь

В данном случае блок текста представляет собой четыре строки в списке языка Python. Класс `CountVectorizer` разбил строки на слова, удалил стоп-слова и символы, а все оставшиеся слова перевел в нижний регистр. Эти слова составляют столбцы набора данных, а числа в строках показывают, сколько раз данное слово встречается в каждой строке. Параметр `stop_words='english'` указывает `CountVectorizer` на удаление стоп-слов с помощью встроенного словаря, содержащего более 300 англоязычных стоп-слов. При желании можно предоставить собственный список стоп-слов в виде списка на языке Python. (Или можно оставить стоп-слова; часто это не имеет значения.) А если вы работаете с текстом, написанным на другом языке, списки мультязычных стоп-слов можно получить из других библиотек Python, таких как `Natural Language Toolkit` (NLTK, <https://oreil.ly/2WzKr>) и `Stop-words` (<https://oreil.ly/Z4mRJ>).

Из вывода видно, что слова `equal` и `equals` считаются отдельными словами, несмотря на то что они имеют схожее значение. При подготовке текста к машинному обучению специалисты по исследованию данных иногда идут еще дальше, выполняя *стемминг* (<https://oreil.ly/q5ZhR>) или *лемматизацию* (<https://oreil.ly/BbiUx>) слов. Если бы предыдущий текст был стемминговым, то все вхождения `equals` были бы преобразованы в `equal`. В Scikit отсутствует поддержка стемминга и лемматизации, но ее можно получить из других библиотек, например NLTK.

Класс `CountVectorizer` удаляет знаки препинания, но не удаляет цифры. Он пропустил 7 в строке 1, поскольку игнорирует одиночные символы. Но если заменить 7 на 777, то в словаре появится термин 777. Один из способов исправить это — определить функцию, удаляющую цифры, и передать ее в `CountVectorizer` через параметр `preprocessor`:

```
import re
```

```
def preprocess_text(text):
    return re.sub(r'\d+', '', text).lower()
```

```
vectorizer = CountVectorizer(stop_words='english', preprocessor=preprocess_text)
word_matrix = vectorizer.fit_transform(lines)
```

Обратите внимание на вызов `lower` для преобразования текста в нижний регистр. `CountVectorizer` не выполняет такое преобразование, если вы используете функцию предварительной обработки, поэтому указанная функция должна преобразовать данные самостоятельно. Однако она все равно удаляет знаки препинания.

Еще одним полезным параметром `CountVectorizer` является `min_df`, который позволяет игнорировать слова, встречающиеся меньше указанного количества раз. Это может быть целое число, задающее минимальное количество (например, игнорировать слова, встречающиеся в обучающем тексте менее пяти раз, `min_df=5`), или значение с плавающей точкой от 0,0 до 1,0, задающее минимальный процент образцов, в которых должно встречаться слово (например, игнорировать слова, встречающиеся менее чем в 10% образцов, `min_df=0,1`). Это позволяет отфильтровать слова, которые, вероятно, все равно не имеют смысла, а также сократить потребление памяти и время обучения за счет уменьшения размера словаря. `CountVectorizer` также поддерживает параметр `max_df` для исключения слов, которые встречаются слишком часто.

В предыдущих примерах использовался класс `CountVectorizer`, что, вероятно, заставило вас задуматься о том, когда (и почему) вместо него можно применить `HashingVectorizer` или `TfidfVectorizer`. Класс `HashingVectorizer` полезен при работе с большими наборами данных. Вместо того, чтобы хранить слова в памяти, он хеширует каждое слово и использует хеш в качестве индекса в массиве количества слов. Таким образом, он может делать больше, используя меньше памяти, и очень полезен для уменьшения размера векторизаторов при их сериализации с целью последующего восстановления — об этом я подробнее расскажу в *главе 7*. Недостатком `HashingVectorizer` является то, что он не разрешает работать в обратном направлении от векторизованного текста к исходному. А вот `CountVectorizer` позволяет, и для этого в нем предусмотрен метод `inverse_transform` (<https://oreil.ly/jlJxb>).

`TfidfVectorizer` часто используется для извлечения ключевых слов: документ или набор документов изучаются, и извлекаются ключевые слова, характеризующие их содержание. При этом словам присваиваются числовые веса, отражающие их важность, а для определения весов используются два фактора: частота появления слова в отдельных документах и частота его появления в общем наборе документов. Слова, которые чаще встречаются в отдельных документах, но в меньшем количестве документов, получают более высокие веса. Я не буду углубляться в эту тему, но если вам интересно узнать больше, то в репозитории GitHub этой книги есть блокнот (<https://oreil.ly/WNOiU>), в котором `TfidfVectorizer` используется для извлечения ключевых слов из текста *главы 1*.

Анализ настроений

Для обучения модели анализа настроений необходим набор данных с метками. В открытом доступе имеется несколько таких наборов данных (<https://oreil.ly/qYYGM>). Мы воспользуемся набором данных кинокритик IMDB (<https://oreil.ly/INB8o>), содержащем 25 000 образцов отрицательных и 25 000 образцов положительных рецензий, размещенных на сайте Internet Movie Database (<https://imdb.com/>). Каждый отзыв тщательно промаркирован: 0 — отрицательный отзыв или 1 — положительный отзыв. Для того чтобы продемонстрировать, как работает анализ настроений, построим модель бинарной классификации и обучим ее на этом наборе данных. В качестве алгоритма обучения будем использовать логистическую регрессию. Оценка анализа настроений, полученная с помощью этой модели, — это просто

вероятность того, что входные данные выражают положительные настроения, и ее легко получить, вызвав метод `predict_proba` из `LogisticRegression`.

Для начала загрузите набор данных (<https://oreil.ly/uex7A>) и скопируйте его в подкаталог `Data` каталога, в котором размещены ваши блокноты Jupyter. Затем выполните следующий код в блокноте, чтобы загрузить набор данных и показать первые пять строк:

```
import pandas as pd
```

```
df = pd.read_csv('Data/reviews.csv', encoding='ISO-8859-1')
df.head()
```

Атрибут `encoding` необходим, поскольку в CSV-файле используется кодировка символов ISO-8859-1, а не UTF-8. Выходные данные выглядят так, как представлено на рис. 4.3.

	Text	Sentiment
0	Once again Mr. Costner has dragged out a movie...	0
1	This is an example of why the majority of acti...	0
2	First of all I hate those moronic rappers, who...	0
3	Not even the Beatles could write songs everyon...	0
4	Brass pictures (movies is not a fitting word f...	0

Рис. 4.3. Выходные данные после предварительной обработки со столбцами `Text` (текст) и `Sentiment` (настроение)

Выясните, сколько строк содержит набор данных, и убедитесь, что в нем нет пропущенных значений:

```
df.info()
```

С помощью следующего оператора можно узнать, сколько существует экземпляров каждого класса (0 — отрицательный, 1 — положительный):

```
df.groupby('Sentiment').describe()
```

На рис. 4.4 приведен результат.

	Text			freq
	count	unique	top	
Sentiment				
0	25000	24697	Nickelodeon has gone down the toilet. They hav...	3
1	25000	24884	Loved today's show!!! It was a variety and not...	5

Рис. 4.4. Набор данных после подсчета частоты экземпляров каждого класса²

² Названия столбцов: `Sentiment` — настроение; `Text count` — количество текста; `unique` — уникальные; `top` — начало текста; `freq` — частота. — Прим. ред.

Существует четное количество положительных и отрицательных образцов, но в каждом случае количество уникальных образцов меньше, чем количество образцов для данного класса. Это означает, что в наборе данных есть дублирующиеся строки, и именно они могут исказить модель машинного обучения. Используйте следующие операторы для удаления дублирующихся строк и повторной проверки на сбалансированность:

```
df = df.drop_duplicates()
df.groupby('Sentiment').describe()
```

Теперь в строках нет дубликатов, а количество положительных и отрицательных образцов примерно одинаково.

Далее с помощью класса `CountVectorizer` подготовим и векторизуем текст в столбце `Text`. Установим `min_df` равным 20, чтобы игнорировать слова, которые редко встречаются в обучающем тексте. Это уменьшает вероятность возникновения ошибок нехватки памяти и, возможно, делает модель более точной. Также используйте параметр `ngram_range`, чтобы позволить векторизатору `CountVectorizer` включать пары слов, а также отдельные слова:

```
from sklearn.feature_extraction.text import CountVectorizer

vectorizer = CountVectorizer(ngram_range=(1, 2), stop_words='english', min_df=20)

x = vectorizer.fit_transform(df['Text'])
y = df['Sentiment']
```

Теперь разделите набор данных на обучающий и тестовый. Мы будем использовать разделение 50/50, поскольку всего имеется почти 50 000 образцов:

```
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.5,
                                                    random_state=0)
```

Следующий шаг — обучение классификатора. Мы воспользуемся классом `LogisticRegression` из Scikit, который использует логистическую регрессию для подгонки модели к данным:

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000, random_state=0)
model.fit(x_train, y_train)
```

Проверьте обученную модель на 50% набора отведенных для тестирования данных и отобразите результаты в виде матрицы ошибок:

```
%matplotlib inline

from sklearn.metrics import ConfusionMatrixDisplay as cmd

cmd.from_estimator(model, x_test, y_test,
                  display_labels=['Негативный', 'Позитивный'],
                  cmap='Blues', xticks_rotation='vertical')
```

Матрица ошибок показывает, что модель правильно идентифицировала 10 795 отрицательных отзывов, но при этом ошибочно классифицировала 1574 отзыва. Модель правильно определила 10 966 положительных отзывов и ошиблась 1456 раз (рис. 4.5).

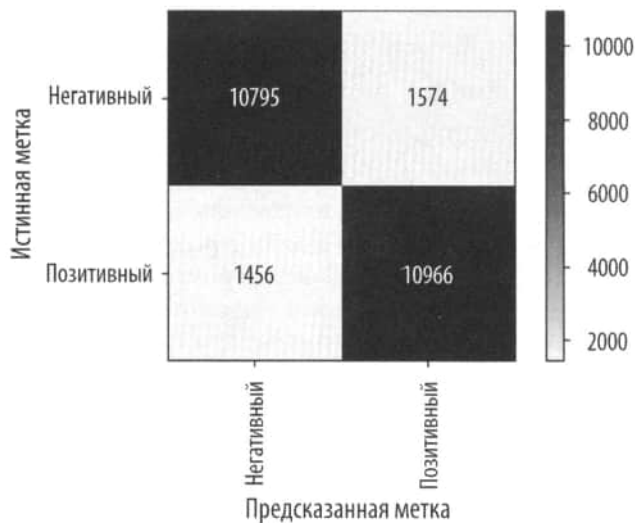


Рис. 4.5. Матрица ошибок построенной модели классификатора настроений

А теперь наступает самое интересное: анализ текста на предмет настроения. Используйте следующие операторы, чтобы получить оценку настроения для предложения "The long lines and poor customer service really turned me off" ("Длинные очереди и плохое клиентское обслуживание оттолкнули меня"):

```
text = 'The long lines and poor customer service really turned me off'
model.predict_proba(vectorizer.transform([text]))[0][1]
```

Вот результат:

```
0.09183447847778639
```

Теперь проделайте то же самое с фразой "The food was great and the service was excellent!" ("Еда была великолепной, а обслуживание — превосходным!"):

```
text = 'The food was great and the service was excellent!'
model.predict_proba(vectorizer.transform([text]))[0][1]
```

Если вы ожидали более высокой оценки, то не будете разочарованы:

```
0.8536277207125618
```

Не бойтесь тестировать свои предложения и проверять, согласны ли вы с оценками настроения, которые дает модель. Она не идеальна, но достаточно хороша, чтобы при прогоне через нее сотен отзывов или комментариев получить достоверное представление о настроениях, выраженных в тексте.



Иногда встроенный в `CountVectorizer` список стоп-слов снижает точность модели из-за его обширности. В качестве эксперимента удалите `stop_words='english'` из `CountVectorizer` и запустите код снова. Проверьте матрицу ошибок. Увеличилась или уменьшилась точность? Не стесняйтесь варьировать и другие параметры, такие как `min_df` и `ngram_range`. В реальном мире специалисты по исследованию данных часто пробуют множество различных комбинаций параметров, чтобы определить, какая из них дает наилучшие результаты.

Наивный Байес

Логистическая регрессия является основным алгоритмом для моделей классификации и часто очень эффективна при классификации текста. Однако в сценариях, связанных с классификацией текстов, специалисты по обработке данных часто обращаются к другому алгоритму обучения, называемому *"наивный Байес"* (Naive Bayes; <https://oreil.ly/MqGwH>). Это алгоритм классификации, основанный на теореме Байеса (<https://oreil.ly/dZxN3>), которая позволяет вычислять условные вероятности. Математически теорема Байеса выглядит следующим образом:

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}.$$

Она гласит, что вероятность того, что A истинно при условии, что B истинно, равна вероятности того, что B истинно при условии, что A истинно, умноженной на вероятность того, что A истинно, деленной на вероятность того, что B истинно. Это довольно сложная формулировка, которая, хотя и является точной, не объясняет, почему наивный Байес так полезен для классификации текста, и как его применить, например, к коллекции электронных писем, чтобы определить, какие из них являются спамом.

Начнем с простого примера. Предположим, что 10% всех полученных вами писем являются спамом. Это $P(A)$. Анализ показывает, что 5% полученных вами спамерских писем содержат слово *congratulations* (поздравления), но только 1% всех ваших писем содержат то же самое слово. Таким образом, $P(B|A)$ составляет 0,05, а $P(B)$ — 0,01. Вероятность того, что письмо является спамом, если оно содержит слово *congratulations*, равна $P(A|B)$, что составляет $(0,05 \cdot 0,10) / 0,01$, или 0,50.

Разумеется, спам-фильтр должен учитывать все слова в письме, а не только одно. Оказывается, если сделать несколько простых (наивных) предположений о том, что порядок слов в письме не имеет значения и каждое слово имеет одинаковый вес, то уравнение Байеса для классификатора спама можно записать таким образом:

$$P(S|\text{сообщение}) = P(S) \cdot P(\text{слово}_1 | S) \cdot P(\text{слово}_2 | S) \cdots P(\text{слово}_n | S).$$

Если говорить понятным языком, вероятность того, что сообщение является спамом, пропорциональна произведению:

- ◆ вероятности того, что любое сообщение в наборе данных является спамом, или $P(S)$;
- ◆ вероятности того, что каждое слово в сообщении встречается в спаме, или $P(\text{слово} | S)$.

$P(S)$ вычисляется достаточно просто: это доля сообщений в наборе данных, которые являются спамом. Если вы обучаете ML-модель на 1000 сообщений и 500 из них являются спамом, то $P(S) = 0,5$. Для заданного слова $P(\text{слово} | S)$ — это просто количество раз, которое это слово встречается в спамерских письмах, поделенное на количество слов во всех спамерских письмах. Вся проблема сводится к подсчету количества слов. Аналогичным образом можно вычислить вероятность того, что сообщение не является спамом, а затем использовать большую из двух вероятностей для прогноза.

Приведем пример с четырьмя образцами писем (табл. 4.1).

Таблица 4.1. Образцы писем

Текст	Спам
Raise your credit score in minutes (Повысьте свой кредитный рейтинг за несколько минут)	1
Here are the minutes from yesterday's meeting (Вот протокол вчерашней встречи)	0
Meeting tomorrow to review yesterday's scores (Завтра состоится встреча, на которой будут рассмотрены вчерашние показатели)	0
Score tomorrow's meds at yesterday's prices (Оцените завтрашние лекарства по вчерашним ценам)	1

Если удалить стоп-слова, преобразовать символы в строчные и расставить слова так, чтобы *tomorrow's* стало *tomorrow*, то получится результат, представленный в табл. 4.2.

Таблица 4.2. Обработанные сообщения

Текст	Спам
raise credit score minute	1
minute yesterday meeting	0
meeting tomorrow review yesterday score	0
score tomorrow med yesterday price	1

Поскольку два из четырех сообщений являются спамом, а два — нет, вероятность того, что любое сообщение является спамом ($P(S)$), составляет 0,5. То же самое относится и к вероятности того, что ни одно сообщение не является спамом ($P(N) = 0,5$). Кроме того, спам содержит девять уникальных слов, а не-спам — всего восемь.

Следующим шагом будет построение таблицы частот слов. В качестве примера возьмем слово *yesterday*. Оно встречается один раз в сообщении, помеченном как спам, поэтому $P(\text{yesterday} | S)$ составляет 1/9, или 0,111. В неспамерских сообщениях оно встречается дважды, поэтому $P(\text{yesterday} | N)$ составляет 2/8, или 0,250 (табл. 4.3).

Таблица 4.3. Таблица частоты слов

Слово	$P(\text{слово} S)$	$P(\text{слово} N)$
raise	1/9 = 0,111	0/8 = 0,000
credit	1/9 = 0,111	0/8 = 0,000
score	2/9 = 0,222	1/8 = 0,125
minute	1/9 = 0,111	1/8 = 0,125
yesterday	1/9 = 0,111	2/8 = 0,250
meeting	0/9 = 0,000	2/8 = 0,250
tomorrow	1/9 = 0,111	1/8 = 0,125
review	0/9 = 0,000	1/8 = 0,125
med	1/9 = 0,111	0/8 = 0,000
price	1/9 = 0,111	0/8 = 0,000

Это работает до определенного момента, но нули в таблице являются проблемой. Допустим, вы хотите определить, является ли "Scores must be reviewed by tomorrow" ("Оценка должна быть рассмотрена до завтра") спамом. Если убрать стоп-слова, то получится "score review tomorrow". Вероятность того, что сообщение является спамом, можно вычислить следующим образом:

$$P(S | \text{score review tomorrow}) = P(S) \cdot P(\text{score} | S) \cdot P(\text{review} | S) \cdot P(\text{tomorrow} | S);$$

$$P(S | \text{score review tomorrow}) = 0,5 \cdot 0,222 \cdot 0,0 \cdot 0,111 = 0;$$

$$P(S | \text{score review tomorrow}) = 0.$$

Результат равен 0, поскольку *review* не появляется в спамерском сообщении, а 0, умноженный на что-либо, дает 0. Алгоритм просто не может присвоить вероятность спама сообщению "Scores must be reviewed by tomorrow".

Распространенным способом решения этой проблемы является применение *сглаживания Лапласа* (<https://oreil.ly/iRt2y>), также известного как аддитивное сглаживание. Обычно для этого к каждому числителю добавляется 1, а к знаменателю — количество уникальных слов в наборе данных (в данном случае 10). Теперь $P(\text{review} | S)$ оценивается как $(0 + 1) / (9 + 10)$, что равно 0,053. Немного, но это лучше, чем ни-

чего (в буквальном смысле). В табл. 4.4 еще раз приведены частоты слов, на этот раз пересмотренные с помощью сглаживания Лапласа.

Таблица 4.4. Таблица частоты слов, пересмотренной с помощью сглаживания Лапласа

Слово	$P(\text{слово} S)$	$P(\text{слово} N)$
raise	$(1 + 1) / (9 + 10) = 0,105$	$(0 + 1) / (8 + 10) = 0,056$
credit	$(1 + 1) / (9 + 10) = 0,105$	$(0 + 1) / (8 + 10) = 0,056$
score	$(2 + 1) / (9 + 10) = 0,158$	$(1 + 1) / (8 + 10) = 0,111$
minute	$(1 + 1) / (9 + 10) = 0,105$	$(1 + 1) / (8 + 10) = 0,111$
yesterday	$(1 + 1) / (9 + 10) = 0,105$	$(2 + 1) / (8 + 10) = 0,167$
meeting	$(0 + 1) / (9 + 10) = 0,053$	$(2 + 1) / (8 + 10) = 0,167$
tomorrow	$(1 + 1) / (9 + 10) = 0,105$	$(1 + 1) / (8 + 10) = 0,111$
review	$(0 + 1) / (9 + 10) = 0,053$	$(1 + 1) / (8 + 10) = 0,111$
med	$(1 + 1) / (9 + 10) = 0,105$	$(0 + 1) / (8 + 10) = 0,056$
price	$(1 + 1) / (9 + 10) = 0,105$	$(0 + 1) / (8 + 10) = 0,056$

Теперь вы можете определить, является ли фраза "Scores must be reviewed by tomorrow" спамом, выполнив два простых расчета:

$$P(S | \text{score review tomorrow}) = 0,5 \cdot 0,158 \cdot 0,053 \cdot 0,105 = 0,000440;$$

$$P(N | \text{score review tomorrow}) = 0,5 \cdot 0,111 \cdot 0,111 \cdot 0,111 = 0,000684.$$

По этому показателю "Scores must be reviewed by tomorrow", скорее всего, не является спамом. Вероятность относительная, но можно нормализовать ее и сделать вывод, что вероятность того, что сообщение является спамом, составляет около 40%, а вероятность того, что оно не является таковым, — 60%, исходя из того, на каких письмах обучалась модель.

К счастью, эти вычисления не обязательно выполнять вручную. Scikit-Learn предоставляет несколько вспомогательных классов, в том числе класс `MultinomialNB` (<https://oreil.ly/twFtY>), который отлично работает с таблицами количества слов, полученными с помощью `CountVectorizer`.

Фильтрация спама

Современные спам-фильтры не случайно обладают высокой эффективностью в распознавании спама. Практически все они основаны на машинном обучении. Подобные модели трудно реализовать алгоритмически, поскольку алгоритм, использующий такие ключевые слова, как *credit* и *score*, для определения того, является ли письмо спамом, легко обмануть. Машинное обучение, напротив, рассматривает совокупность писем и использует полученные знания для классификации следую-

щего письма. Точность таких моделей часто превышает 99%. При этом со временем они становятся все более интеллектуальными, поскольку их тренируют на все большем количестве писем.

В одном из предыдущих примеров использовалась логистическая регрессия для предсказания того, выражает ли текст положительные или отрицательные настроения. Вероятность того, что текст выражает положительные настроения, использовалась в качестве оценки настроений, и вы видели, что такие выражения, как "Длинные очереди и плохое клиентское обслуживание оттолкнули меня", оцениваются в баллах, близких к 0,0, а такие выражения, как "Еда была отличной, а обслуживание — превосходным", — в баллах, близких к 1,0. Теперь построим бинарную модель классификации, которая классифицирует письма как спам или не-спам, и с помощью метода "наивный Байес" подгоним модель к обучающим данным.

В открытом доступе имеется несколько наборов данных для классификации спама. Каждый из них содержит коллекцию электронных писем, образцы которых помечены 1 — спам и 0 — не-спам. Мы будем использовать относительно небольшой набор данных, содержащий 1000 образцов. Для начала загрузите набор данных (<https://oreil.ly/hljvo>) и скопируйте его в подкаталог Data вашего блокнота. Затем загрузите данные и выведите на экран первые пять строк:

```
import pandas as pd
df = pd.read_csv('Data/ham-spam.csv')
df.head()
```

Теперь проверьте, нет ли в наборе данных дублирующихся строк:

```
df.groupby('IsSpam').describe()
```

Набор данных содержит одну дублирующуюся строку. Удалим ее и проверим набор на сбалансированность:

```
df = df.drop_duplicates()
df.groupby('IsSpam').describe()
```

Теперь набор данных содержит 499 образцов, не являющихся спамом, и 500 образцов, являющихся таковым. Следующим шагом будет использование `CountVectorizer` для векторизации писем. В очередной раз мы разрешаем `CountVectorizer` учитывать как пары слов, так и отдельные слова, и удалять стоп-слова, используя встроены в Scikit словарь английских стоп-слов:

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
vectorizer = CountVectorizer(ngram_range=(1, 2), stop_words='english')
x = vectorizer.fit_transform(df['Text'])
y = df['IsSpam']
```

Разбейте набор данных таким образом, чтобы 80% можно было использовать для обучения, а 20% — для тестирования:

```
from sklearn.model_selection import train_test_split
```

[illegible]

Следующий шаг — обучение классификатора наивного Байеса с использованием класса `MultinomialNB` (<https://oreil.ly/0CtOh>) из Scikit:

```
from sklearn.naive_bayes import MultinomialNB
```

```
model = MultinomialNB()
model.fit(x_train, y_train)
```

Проверьте модель, обученную на 20%, отведенных для тестирования набора данных, с помощью матрицы ошибок:

```
%matplotlib inline
```

```
from sklearn.metrics import ConfusionMatrixDisplay as cmd
```

```
cmd.from_estimator(model, x_test, y_test,
                  display_labels=['Не спам', 'Спам'],
                  cmap='Blues', xticks_rotation='vertical')
```

Модель правильно определила 101 из 102 нормальных писем как не-спам и 95 из 98 спамерских писем как спам (рис. 4.6).



Рис. 4.6. Матрица ошибок для классификатора спамерских писем на основе наивного Байеса

Оцените в баллах приблизительную точность модели:

```
model.score(x_test, y_test)
```

Теперь для визуализации ROC-кривой используйте класс `RocCurveDisplay` из Scikit:

```
from sklearn.metrics import RocCurveDisplay as rcd
```

```
import seaborn as sns
```

```
sns.set()
```

```
rcd.from_estimator(model, x_test, y_test)
```

Полученные результаты обнадеживают. После обучения всего на 999 образцах площадь под ROC-кривой (AUC) показывает, что модель более чем на 99,9% точно классифицирует письма как спам или не-спам (рис. 4.7).

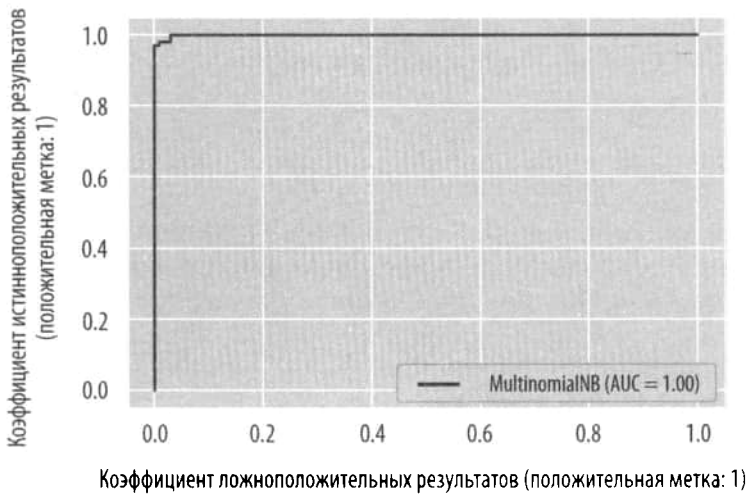


Рис. 4.7. ROC-кривая для построенной модели классификатора спамерских писем

Посмотрим, как модель классифицирует несколько писем, с которыми она еще не сталкивалась, начиная с того, которое не является спамом. Метод `predict` модели предсказывает класс 0 для не-спам и 1 для спама:

```
msg = 'Can you attend a code review on Tuesday to make sure the logic is solid?'
# Можете ли вы принять участие в обзоре кода во вторник, чтобы убедиться
# в надежности логики?
```

```
input = vectorizer.transform([msg])
model.predict(input)[0]
```

Модель говорит, что это письмо не является спамом, но какова вероятность того, что это не спам? Это можно узнать из функции `predict_proba`, которая возвращает массив, содержащий два значения: вероятность того, что предсказанный класс равен 0, и вероятность того, что предсказанный класс равен 1, в указанном порядке:

```
model.predict_proba(input)[0][0]
```

Модель, похоже, уверена, что это обычное письмо:

```
0.9999497111473539
```

Теперь протестируем модель на спам-сообщении:

```
msg = 'Why pay more for expensive meds when you can order them online and save $$$?'
# Зачем платить больше за дорогие лекарства, если можно заказать их онлайн
# и сэкономить $$$?
```

```
input = vectorizer.transform([msg])
model.predict(input)[0]
```

Какова вероятность того, что сообщение не является спамом?

```
model.predict_proba(input)[0][0]
```

Ответ:

```
0.00021423891260677753
```

Какова вероятность того, что сообщение *является* спамом?

```
model.predict_proba(input)[0][1]
```

Ответ:

```
0.9997857610873945
```

Обратите внимание, что методы `predict` и `predict_proba` принимают список входных данных. Исходя из этого, можно ли классифицировать целую партию писем одним вызовом любого из методов? Как бы вы получили результаты для каждого письма?

Рекомендательные системы

Еще одним направлением машинного обучения, хорошо показавшим себя в последние годы, являются *рекомендательные системы* — системы, рекомендуемые покупателям товары или услуги. По имеющимся данным, система рекомендаций компании Amazon обеспечивает 35% ее продаж (<https://oreil.ly/ue81Q>). Хорошая новость заключается в том, что для того, чтобы создать рекомендательную систему, не обязательно быть Amazon, равно как и не обязательно обладать ресурсами Amazon для ее разработки. Построить такую систему относительно просто, если освоить несколько основных принципов.

Рекомендательные системы бывают разных видов. *Системы, основанные на популярности*, предлагают покупателям варианты, основанные на том, какие товары и услуги популярны в данный момент, например "Бестселлеры этой недели". *Коллаборативные системы* дают рекомендации на основе того, что выбрали другие, например "Люди, купившие эту книгу, также купили эти книги". Ни одна из этих систем не требует машинного обучения.

Контентные системы, напротив, в значительной степени выигрывают от машинного обучения. Примером системы, основанной на контенте, является система, которая говорит: "Если вы купили эту книгу, то вам могут понравиться и эти книги". Такие системы требуют средств для количественной оценки сходства между элементами. Если вы прониклись к фильму "Крепкий орешек", то вам может понравиться или не понравиться фильм "Монти Пайтон и Священный Грааль". Если вы фанат "Истории игрушек", то, скорее всего, вам понравятся и "Приключения Флика". Но как определить это алгоритмически?

Для контентных рекомендательных систем необходимы два компонента: способ векторизации атрибутов, характеризующих услугу или товар, и способ вычисления сходства между полученными векторами. Первое несложно. `CountVectorizer` преобразует текст в таблицы количества слов. Достаточно найти способ измерения сход-

ства между строками количества слов, и можно строить рекомендательную систему. Одним из самых простых и эффективных способов является метод *косинусного сходства*.

Косинусное сходство

Косинусное сходство (<https://oreil.ly/948eP>) — это математический способ вычисления сходства между парами векторов (или рядов чисел, рассматриваемых как векторы). Основная идея состоит в том, чтобы взять каждое значение в выборке, например количество слов в строке векторизованного текста, и использовать их в качестве координат конечной точки вектора, причем другая конечная точка должна находиться в начале системы координат. Прodelайте это для двух выборок, а затем вычислите косинус между векторами в m -мерном пространстве, где m — количество значений в каждой выборке. Поскольку косинус 0 равен 1, два одинаковых вектора имеют сходство, равное 1. Чем более несхожи векторы, тем ближе косинус к 0.

В качестве иллюстрации приведем пример в двумерном пространстве. Предположим, что у вас есть три строки, содержащие по два значения:

1	2
2	3
3	1

Требуется определить, на что больше похож ряд 2 — на ряд 1 или на ряд 3? Это трудно сказать, просто взглянув на числа, а в реальной жизни таких чисел бывает гораздо больше. Если просто сложить числа в каждом ряду и сравнить суммы, то можно сделать вывод, что ряд 2 более похож на ряд 3. Но что если рассматривать каждый ряд как вектор, как показано на рис. 4.8?

◆ Ряд 1: $(0, 0) \rightarrow (1, 2)$.

◆ Ряд 2: $(0, 0) \rightarrow (2, 3)$.

◆ Ряд 3: $(0, 0) \rightarrow (3, 1)$.

Теперь можно представить каждый ряд в виде вектора, вычислить косинусы углов, образованных прямыми 1 и 2, 2 и 3, и определить, что ряд 2 больше похож на ряд 1, чем на ряд 3. Это и есть косинусное сходство в общих чертах.

Косинусное сходство не ограничивается двумя измерениями, оно работает и в многомерном пространстве. Для вычисления косинусного сходства независимо от числа измерений Scikit предлагает функцию `cosine_similarity` (<https://oreil.ly/vc1Uv>). Следующий код вычисляет косинусное сходство трех образцов из предыдущего примера:

```
data = [[1, 2], [2, 3], [3, 1]]
cosine_similarity(data)
```

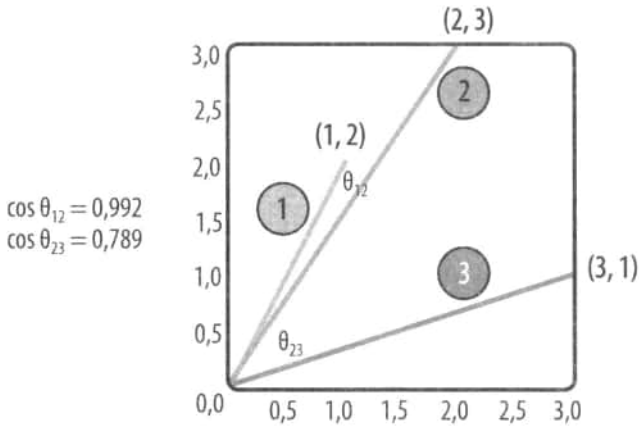


Рис. 4.8. Косинусное сходство

Возвращаемое значение — *матрица подобия* (similarity matrix), содержащая косинусы каждой пары векторов. Ширина и высота матрицы равны количеству образцов:

```
array([[1.          , 0.99227788, 0.70710678],
       [0.99227788, 1.          , 0.78935222],
       [0.70710678, 0.78935222, 1.          ]])
```

Отсюда видно, что сходство строк 1 и 2 составляет 0,992, а сходство строк 2 и 3 — 0,789. Другими словами, ряд 2 больше похож на ряд 1, чем на ряд 3. Сходство между рядами 2 и 3 (0,789) также больше, чем между рядами 1 и 3 (0,707).

Построение системы рекомендаций фильмов

Давайте применим косинусное сходство для построения системы рекомендаций фильмов на основе контента. Начнем с загрузки набора данных (<https://oreil.ly/ydE3v>), который является одним из нескольких наборов данных о фильмах с сайта **Kaggle.com**. Этот набор содержит информацию о примерно 4800 фильмах, включая название, бюджет, жанры, ключевые слова, актерский состав и т. д. Поместите CSV-файл в подкаталог Data вашего Jupyter-блокнота. Затем загрузите набор данных и просмотрите его содержимое:

```
import pandas as pd
```

```
df = pd.read_csv('Data/movies.csv')
df.head()
```

Набор данных содержит 24 столбца, лишь некоторые из них необходимы для описания фильма. Используйте следующие операторы для извлечения таких ключевых столбцов, как title (название) и genres (жанры), и заполнения отсутствующих значений пустыми строками:

```
df = df[['title', 'genres', 'keywords', 'cast', 'director']]
df = df.fillna('') # Заполнение отсутствующих значений пустыми строками
df.head()
```

Далее добавим столбец с именем `features`, который объединяет все слова в других столбцах:

```
df['features'] = df['title'] + ' ' + df['genres'] + ' ' + \
    df['keywords'] + ' ' + df['cast'] + ' ' + \
    df['director']
```

Используйте `CountVectorizer` для векторизации текста в столбце признаков:

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
vectorizer = CountVectorizer(stop_words='english', min_df=20)
word_matrix = vectorizer.fit_transform(df['features'])
word_matrix.shape
```

Таблица количества слов содержит 4803 строки — по одной для каждого фильма — и 918 столбцов. Следующей задачей является вычисление косинусного сходства для каждой пары строк:

```
from sklearn.metrics.pairwise import cosine_similarity
```

```
sim = cosine_similarity(word_matrix)
```

В конечном счете цель этой системы — задать название фильма и определить *n* фильмов, наиболее похожих на указанный фильм. Для этого определим функцию `get_recommendations`, которая принимает название фильма, содержащий информацию обо всех фильмах фрейм данных `DataFrame`, матрицу подобия и количество возвращаемых названий фильмов:

```
def get_recommendations(title, df, sim, count=10):
    # Получение индекса строки указанного заголовка в DataFrame
    index = df.index[df['title'].str.lower() == title.lower()]

    # Возвращение пустого списка, если для указанного названия
    # не существует записи
    if (len(index) == 0):
        return []

    # Получение соответствующей строки в матрице подобия
    similarities = list(enumerate(sim[index[0]]))

    # Сортировка оценок сходства в этой строке в порядке убывания
    recommendations = sorted(similarities, key=lambda x: x[1], reverse=True)

    # Получить n рекомендаций, игнорируя первую запись в списке,
    # поскольку она соответствует самому названию (и, следовательно,
    # имеет сходство 1.0)
    top_recs = recommendations[1:count + 1]
```

```
# Формирование списка названий из индексов в top_recs
titles = []

for i in range(len(top_recs)):
    title = df.iloc[top_recs[i][0]]['title']
    titles.append(title)

return titles
```

Эта функция сортирует косинусные сходства в порядке убывания, чтобы определить заданное количество фильмов count, наиболее похожих на тот, который был определен параметром title. Затем она возвращает названия этих фильмов.

Теперь используйте `get_recommendations` для поиска в базе данных похожих фильмов. Сначала запросите 10 фильмов, наиболее похожих на триллер "Skyfall" ("007: Координаты „Скайфолл“") о Джеймсе Бонде:

```
get_recommendations('Skyfall', df, sim)
```

Вот результат:

```
['Spectre',
 'Quantum of Solace',
 'Johnny English Reborn',
 'Clash of the Titans',
 'Die Another Day',
 'Diamonds Are Forever',
 'Wrath of the Titans',
 'I Spy',
 'Sanctum',
 'Blackthorn']
```

Вызовите `get_recommendations` еще раз, чтобы получить список фильмов, похожих на "Mulan" ("Мулан"):

```
get_recommendations('Mulan', df, sim)
```

Не поленитесь протестировать и другие фильмы. Обратите внимание, что можно вводить лишь те названия фильмов, которые есть в наборе данных. Для вывода полного списка фильмов используйте следующие операторы:

```
pd.set_option('display.max_rows', None)
print(df['title'])
```

Думаю, вы согласитесь с тем, что система достаточно качественно подбирает похожие фильмы. Неплохо для 20 строк кода!

Резюме

Классифицирующие текст модели машинного обучения широко распространены и находят применение как в прикладных отраслях, так и в повседневной жизни. На-

пример, какой здравомыслящий человек не мечтает о волшебной палочке, которая избавит его от спама?

Используемый для обучения модели классификации текст должен быть подготовлен и векторизован до начала обучения. Подготовка включает в себя перевод символов в нижний регистр и удаление знаков препинания, а также удаление стоп-слов, чисел, стемминг или лемматизацию. После подготовки текст векторизуется путем преобразования его в таблицу частот слов. Класс `CountVectorizer` из Scikit позволяет быстро выполнить процесс векторизации, а также справиться с некоторыми функциями подготовки.

Для классификации текста после его преобразования в числовую форму можно использовать логистическую регрессию и другие популярные алгоритмы классификации. Однако для задач классификации текста алгоритм обучения "наивный Байес" часто превосходит другие алгоритмы. Делая несколько "наивных" предположений, например о том, что порядок появления слов в образце текста не имеет значения, "наивный Байес" сводится к процессу подсчета слов. Класс `MultinomialNB` в Scikit предоставляет удобную реализацию метода "наивного Байеса".

Косинусное сходство — это математический способ вычисления сходства между двумя рядами чисел. Одно из применений этого метода — создание систем, рекомендуемых продукты или услуги на основе других приобретенных клиентом продуктов или услуг. Таблицы частот слов, созданные на основе текстовых описаний с помощью `CountVectorizer`, могут быть использованы в сочетании с косинусным сходством для создания интеллектуальных рекомендательных систем, призванных повысить прибыль компании.

Не бойтесь использовать примеры этой главы в качестве отправной точки для собственных экспериментов. Например, посмотрите, можно ли изменить параметры, передаваемые в `CountVectorizer` в каком-либо из примеров, и увеличить точность получаемой модели. Специалисты по исследованию данных называют поиск оптимальной комбинации параметров *настройкой гиперпараметров*, и об этом мы поговорим в следующей главе.

Метод опорных векторов

Метод опорных векторов (или *машины опорных векторов* — support vector machines, SVM) находится на передовом рубеже машинного обучения. Чаще всего он используется для решения задач классификации, но может применяться и для регрессии. Благодаря уникальному способу подгонки математических моделей к данным, SVM часто удается найти разделение между классами, которое не удастся отыскать другим моделям. Технически метод выполняет только бинарную классификацию, но Scikit-Learn позволяет ему выполнять и мультиклассовую классификацию, используя приемы, рассмотренные в *главе 3*.

Scikit-Learn упрощает построение SVM с помощью таких классов, как SVC (support vector classifier; <https://oreil.ly/IAiWs>) для классификационных моделей и SVR (support vector regressor; <https://oreil.ly/f8B8K>) для регрессионных моделей. Вы можете использовать эти классы, не понимая, как работает SVM, но вы получите больше пользы, если *разберетесь* в его работе. Также важно знать, как настраивать SVM для отдельных наборов данных и как подготавливать данные перед обучением модели. В конце этой главы мы построим SVM-модель, которая будет выполнять распознавание лиц. Но сначала давайте заглянем за кулисы и узнаем, почему SVM часто становится лучшим механизмом для моделирования реальных наборов данных.

Принцип работы метода опорных векторов

Во-первых, почему метод называется "машины опорных векторов"? Задача SVM-классификатора такая же, как и любого другого классификатора: найти границу принятия решения, которая четко разделяет классы. Для этого метод SVM находит линию в двумерном пространстве, плоскость в трехмерном пространстве или гиперплоскость в многомерном пространстве, которые позволяют с наибольшей уверенностью разделять различные классы. В примере на рис. 5.1 для разделения двух классов можно провести бесконечное число линий, но наилучшей будет та, которая дает наибольшую ширину границы (та, что показана справа). Ширина границы — это расстояние между ближайшими к границе точками в каждом классе вдоль линии, перпендикулярной границе. Эти точки называются *опорными векторами* и обведены красным цветом.

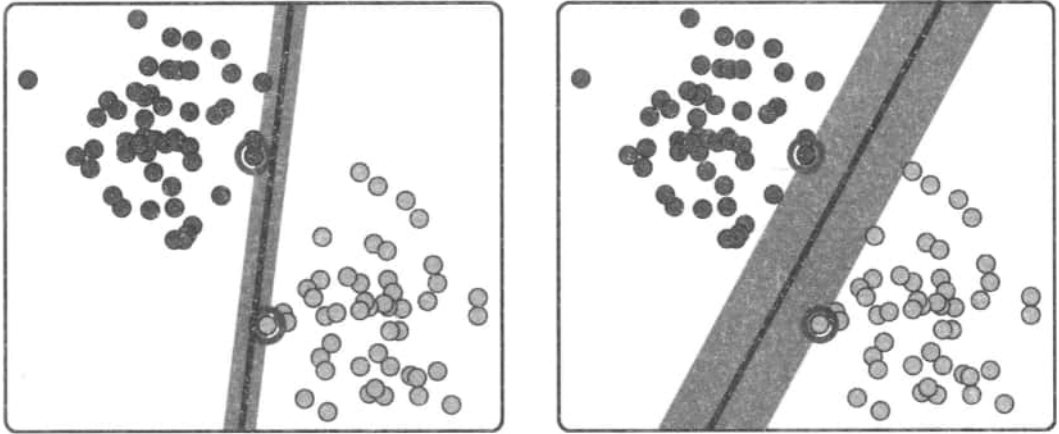


Рис. 5.1. Классификация по максимальной границе

Конечно, реальные данные редко поддаются такому чистому разделению. Перекрытие между классами неизбежно препятствует получению идеальной подгонки. Для решения этой проблемы SVM поддерживает параметр регуляризации, обычно называемый C , который может быть изменен для ослабления или усиления соответствия. Более низкие значения C дают более широкое поле с большим количеством ошибок по обе стороны от границы принятия решения, как показано на рис. 5.2. Более высокие значения обеспечивают более плотное прилегание к обучающим данным с соответственно меньшей шириной поля и меньшим количеством ошибок. Если значение C слишком велико, модель может плохо обобщаться. Оптимальное значение зависит от набора данных. Специалисты по исследованию данных обычно пробуют различные значения C , чтобы определить, какое из них лучше всего работает на тестовых данных.

Все вышесказанное верно, но ничего из этого не объясняет, почему метод опорных векторов так хорош в своей работе. SVM не единственная модель, которая математически ищет границы, разделяющие классы. Особенностью SVM являются ядра (kernels), некоторые из них добавляют размерность к данным, чтобы найти границы, не существующие в более низких измерениях. Рассмотрим рис. 5.3. Невозможно провести линию, полностью отделяющую красные точки от фиолетовых. Но если добавить третье измерение (как показано справа) — измерение z , значение которого зависит от расстояния точки от центра, — то можно провести плоскость между фиолетовыми и красными точками и добиться 100-процентного разделения. В этом примере данные, не поддающиеся линейному разделению в двух измерениях, линейно разделяются в трех измерениях. Принцип работы заключается в *теореме Ковера* (<https://oreil.ly/BASz2>), которая утверждает, что данные, не поддающиеся линейному разделению, могут быть линейно разделены, если спроецировать их в пространство более высокой размерности с помощью нелинейного преобразования.

Используемое в этом примере преобразование ядра, которое проецирует двумерные данные на трехмерные путем добавления z к каждому x и y , хорошо работает с этим

конкретным набором данных. Но для того чтобы метод SVM был полезным в широком смысле, необходимо ядро, не привязанное к форме конкретного набора данных.

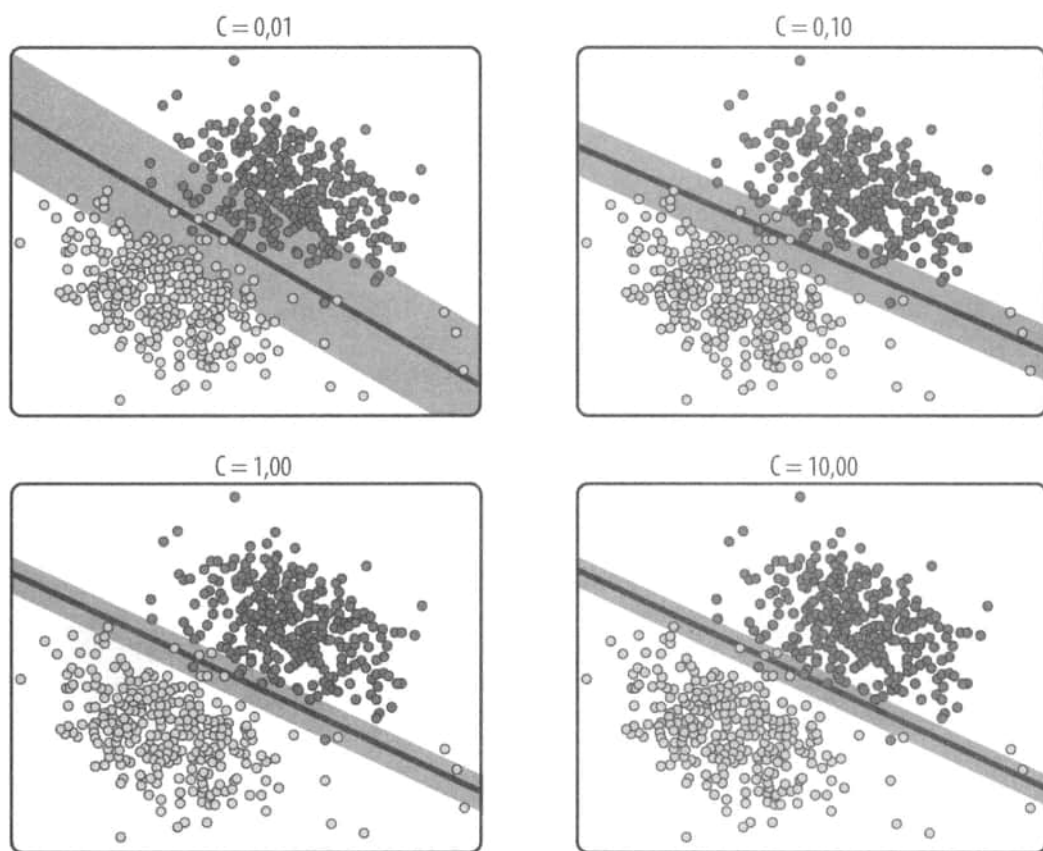


Рис. 5.2. Влияние C на ширину границы

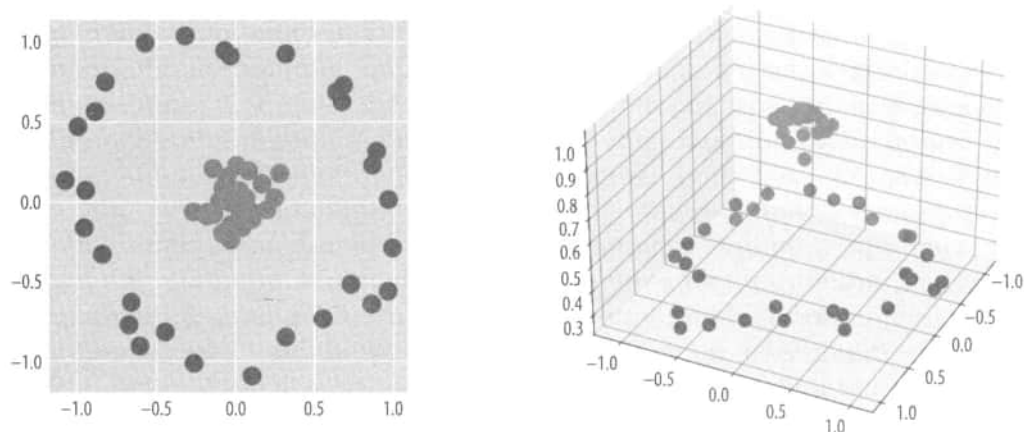


Рис. 5.3. Добавление измерений для достижения линейной разделяемости

Ядра

В Scikit-Learn встроено несколько ядер общего назначения, включая линейное ядро, RBF-ядро (radial basis function — радиальная базисная функция), полиномиальное ядро и сигмоидное ядро. Линейное ядро не добавляет размерности. Оно хорошо работает с данными, которые линейно разделяются, но не очень хорошо с данными, которые не разделяются. Его применение к задаче на рис. 5.3 дает границу принятия решения слева на рис. 5.4. Применение RBF-ядра к тем же данным дает границу принятия решения справа. Ядро RBF проецирует значения x и y в более высокоразмерное пространство и находит гиперплоскость, которая четко отделяет фиолетовые точки от красных. При обратном проецировании в двумерное пространство граница принятия решения имеет форму круга. Аналогичные результаты можно получить на этом наборе данных при использовании правильно настроенного полиномиального ядра, но в целом ядро RBF может находить границы принятия решений в нелинейных данных, чего не способно сделать полиномиальное ядро. Именно поэтому RBF является типом ядра по умолчанию в Scikit, если не указано иное.

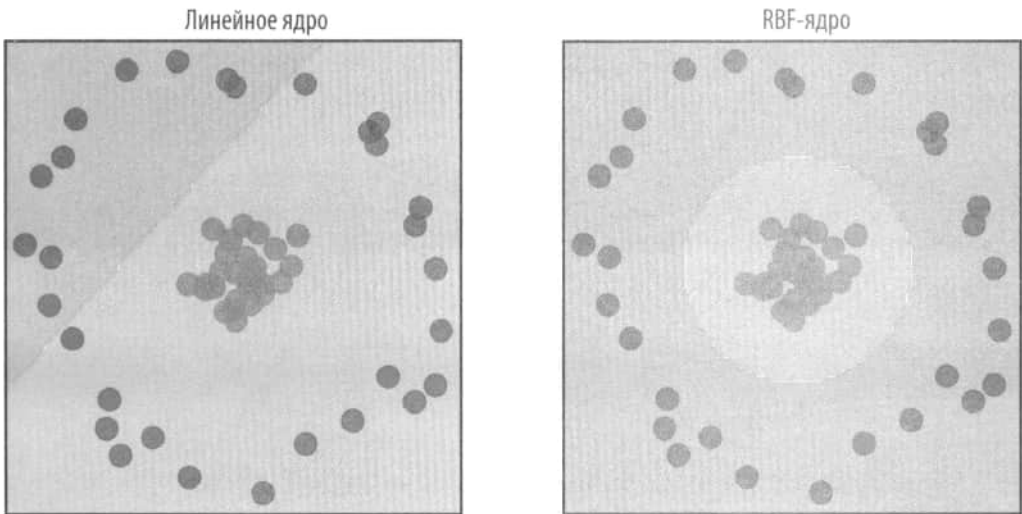


Рис. 5.4. Линейное ядро в сравнении с RBF-ядром

Логично задать вопрос: добавляет ли ядро RBF z к каждому x и y ? Короткий ответ — нет. Оно эффективно проецирует точки данных в пространство с бесконечным числом измерений. Ключевое слово — *эффективно*. Ядра используют математические приемы, называемые *ядерными трюками*, для определения эффекта от добавления новых измерений без фактического вычисления их значений. Именно здесь математика SVM становится сложной. Ядра тщательно разработаны для вычисления *скалярного произведения* (<https://oreil.ly/yGUYS>) между двумя n -мерными векторами в m -мерном пространстве (где m больше n и даже может быть бесконечным) без генерации всех этих новых измерений, и в конечном итоге скалярные произведения — это все, что нужно SVM для вычисления границы принятия реше-

ния. Это математический эквивалент сидения на двух стульях, и это та секретная приправа, которая делает SVM потрясающим методом. Обучение SVM-моделей на больших наборах данных может занимать много времени, но одним из преимуществ метода опорных векторов является то, что он лучше работает на небольших наборах данных с меньшим количеством строк или образцов, чем другие алгоритмы обучения.

Ядерные трюки

Хотите увидеть пример использования ядерных трюков для вычисления точечных произведений в высокоразмерных пространствах без вычисления значений для новых размерностей? Приведенное ниже объяснение совершенно необязательно. Но если вы, как и я, лучше учитесь на конкретных примерах, то этот раздел может оказаться для вас полезным.

Начнем с двумерного кругового набора данных, представленного ранее, но на этот раз спроецируем его в трехмерное пространство с помощью следующих уравнений:

$$\begin{aligned}x' &= x^2; \\y' &= y^2; \\z &= xy\sqrt{2}.\end{aligned}$$

Другими словами, мы вычислим x и y в трехмерном пространстве (x' и y'), возведя в квадрат x и y в двумерном пространстве, и добавим z , равное произведению исходных x и y и квадратного корня из 2. Проецирование данных таким образом позволяет получить четкое разделение между фиолетовыми и красными цветами, как показано на рис. 5.5.

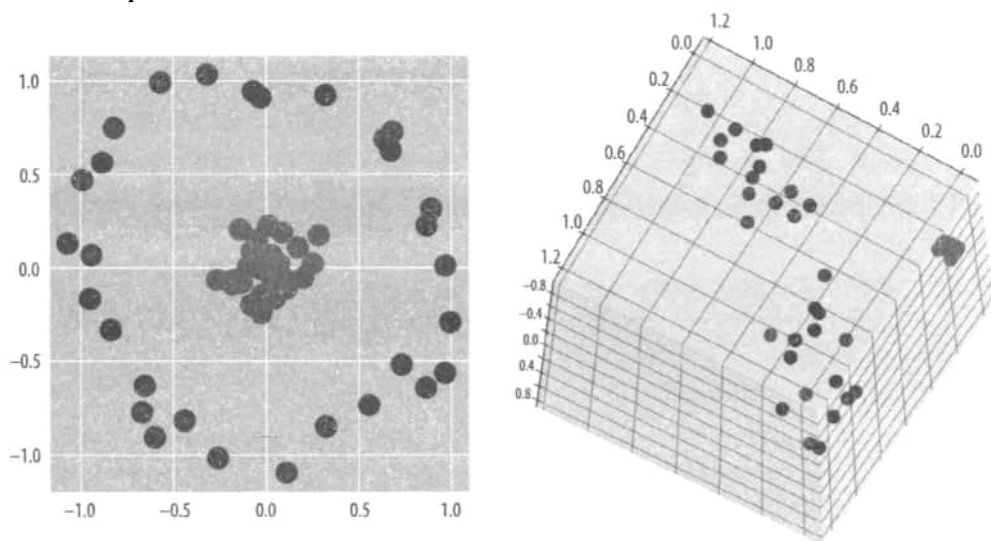


Рис. 5.5. Проецирование двумерных точек в трехмерное пространство для разделения двух классов

Эффективность SVM зависит от ее способности вычислять точечное произведение двух векторов (или точек, которые можно рассматривать как векторы) в пространстве более высокой размерности, не проецируя их в это пространство, т. е. используя только значения в исходном пространстве. Возьмем для работы несколько точек:

$$a = (3, 7);$$

$$b = (2, 4).$$

Точечное произведение этих двух точек можно вычислить следующим образом:

$$(3 \cdot 2) + (7 \cdot 4) = 34.$$

Конечно, точечное произведение в двух измерениях не очень полезно. Для SVM необходимо точечное произведение этих точек в трехмерном пространстве. Используя предыдущие уравнения, спроецируем точки a и b в трехмерное пространство, а затем вычислим точечное произведение полученных результатов:

$$a = (3^2; 7^2; 3 \cdot 7 \cdot \sqrt{2}) = (9; 49; 29,6984);$$

$$b = (2^2; 4^2; 2 \cdot 4 \cdot \sqrt{2}) = (4; 16; 11,3137);$$

$$(9 \cdot 4) + (49 \cdot 16) + (29,6984 \cdot 11,3137) = 1156.$$

Теперь мы имеем точечное произведение пары двумерных точек в трехмерном пространстве, но для его получения необходимо было сгенерировать координаты в трехмерном пространстве. Вот тут-то и начинается самое интересное. Следующая функция, или ядерный трюк, дает тот же результат, используя только значения в исходном двумерном пространстве:

$$K(a, b) = \langle a, b \rangle^2.$$

$\langle a, b \rangle$ — это просто точечное произведение a и b , поэтому $\langle a, b \rangle^2$ — это квадрат точечного произведения a и b . Мы уже знаем, как вычислить точечное произведение a и b . Следовательно,

$$K((3, 7), (2, 4)) = ((3 \cdot 2) + (7 \cdot 4))^2 = 34^2 = 1156.$$

Это совпадает с результатом, вычисленным при явном проецировании точек, но при этом проецирование не требуется. Таков вкратце ядерный трюк. Он позволяет экономить время и память при переходе от двух измерений к трем. Представьте себе экономию при проецировании на бесконечное число измерений, а это, как вы помните, как раз то, что делает ядро RBF.

Представленный здесь ядерный трюк был создан не на пустом месте. Он используется в полиномиальном ядре степени 2. В Scikit классификатор SVM с полиномиальным ядром степени 2 можно подогнать к набору данных следующим образом:

```
model = SVC(kernel='poly', degree=2)
model.fit(x, y)
```

Если применить эту модель к предыдущему круговому набору данных и построить границу принятия решения (рис. 5.6, справа), то результат будет практически иден-

тичен результату, полученному с помощью RBF-ядра. Интересно, что полиномиальное ядро степени 1 (рис. 5.6, слева) дает ту же границу принятия решения, что и линейное ядро, поскольку линия — это просто полином первой степени.

Ядерные трюки необычны. Каждый из них предназначен для моделирования определенной проекции в более высокие измерения. Scikit предоставляет несколько ядер для работы, но есть и другие, которые в Scikit не встроены. Вы можете расширить Scikit собственными ядрами, но для подавляющего большинства случаев достаточно тех, которые библиотека предоставляет.

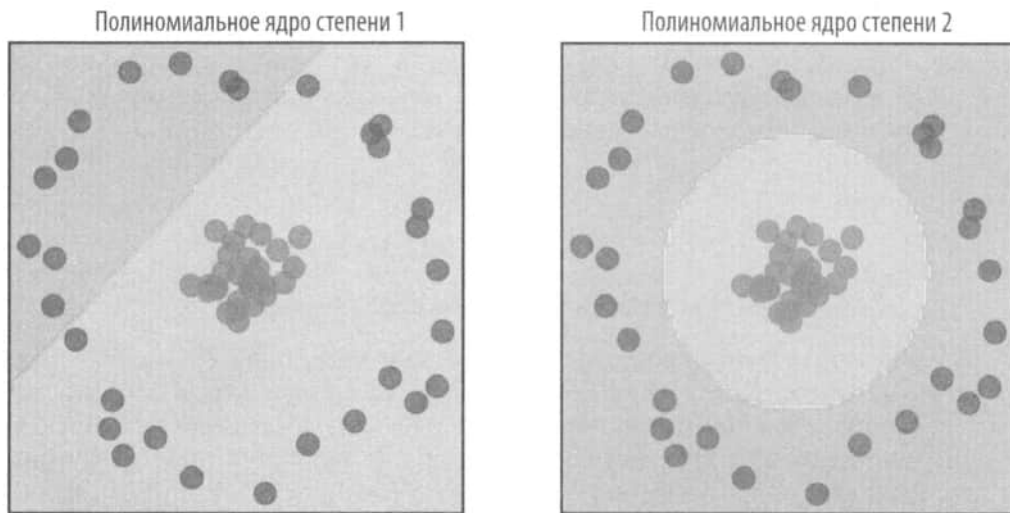


Рис. 5.6. Сравнение полиномиальных ядер степени 1 и степени 2

Настройка гиперпараметров

С самого начала трудно определить, какое из встроенных ядер даст наиболее точную модель. Также трудно определить правильное значение C , т. е. то, которое обеспечивает оптимальный баланс между недооценкой и переоценкой обучающих данных и дает наилучшие результаты при работе модели с тестовыми данными. Для RBF- и полиномиальных ядер существует третье значение, называемое *gamma*, которое влияет на точность. А для полиномиальных ядер параметр *degree* влияет на способность модели обучаться на обучающих данных.

Параметр C определяет, насколько плотно модель прилегает к обучающим данным. Чем выше значение параметра, тем плотнее подгонка и тем выше риск переоценки. На рис. 5.7 показано, как RBF-ядро подгоняет модель к набору обучающих данных, содержащему три класса, при различных значениях параметра C . По умолчанию в Scikit используется $C=1$, но можно задать другое значение для настройки подгонки. На правом нижнем рисунке видна опасность чрезмерной подгонки. Точка, распо-

ложенная крайне справа, будет классифицирована как синяя, хотя, скорее всего, она относится к классу желтых или коричневых. Недооценка также является проблемой. В левом верхнем примере практически любая точка данных, не являющаяся коричневой, будет классифицирована как синяя.

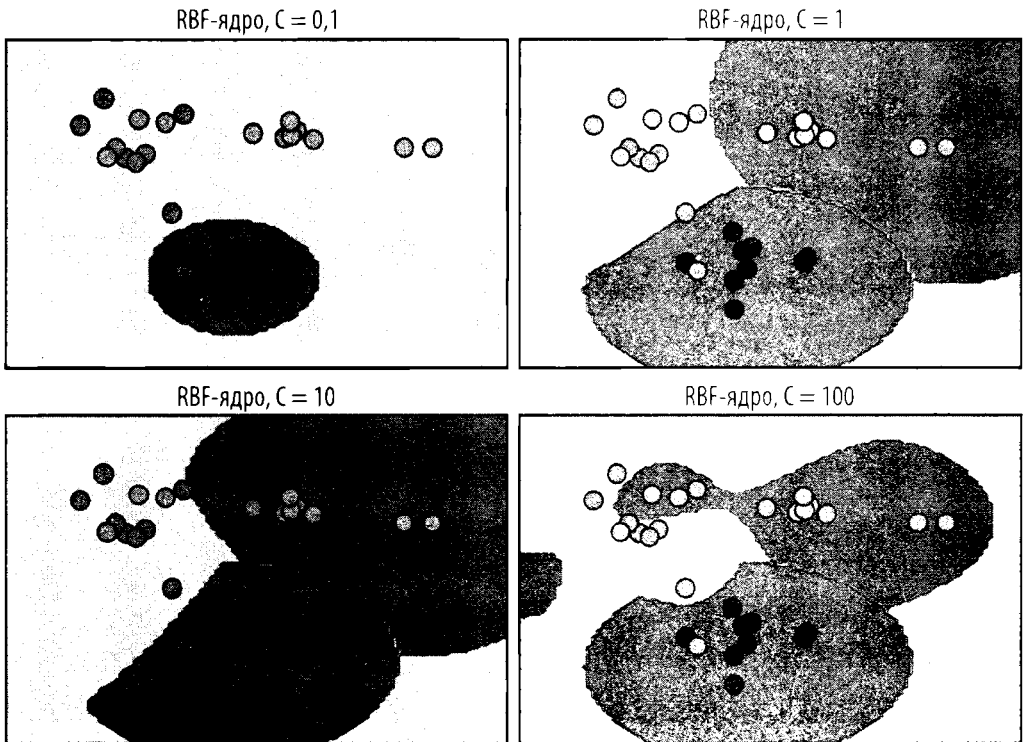


Рис. 5.7. Влияние C на RBF-ядро

SVM-модель, использующая ядро RBF, не будет правильно настроена до тех пор, пока вы не подберете верное значение γ . Этот параметр определяет, насколько далеко распространяется влияние одной точки данных при вычислении границ принятия решения. При меньших значениях используется больше точек и получаются более гладкие границы принятия решений, при больших значениях — меньше точек и более жесткое соответствие обучающим данным. Это показано на рис. 5.8, где увеличение значения γ при неизменном значении C более плотно обводит границы принятия решений вокруг кластеров классов. γ может быть любым ненулевым положительным значением, но наиболее распространены значения от 0 до 1. Вместо того чтобы жестко задавать значение γ по умолчанию, Scikit выбирает его по умолчанию алгоритмически, если оно не указано.

На практике специалисты по исследованию данных экспериментируют с различными ядрами и разными значениями параметров, чтобы найти комбинацию, дающую наиболее точную модель, этот процесс называется *настройкой гиперпараметров*. Ее полезность характерна не только для SVM, но практически всегда мож-

но сделать SVM-модель более точной, найдя оптимальную комбинацию типа ядра, C и γ (а для полиномиальных ядер — и degree).

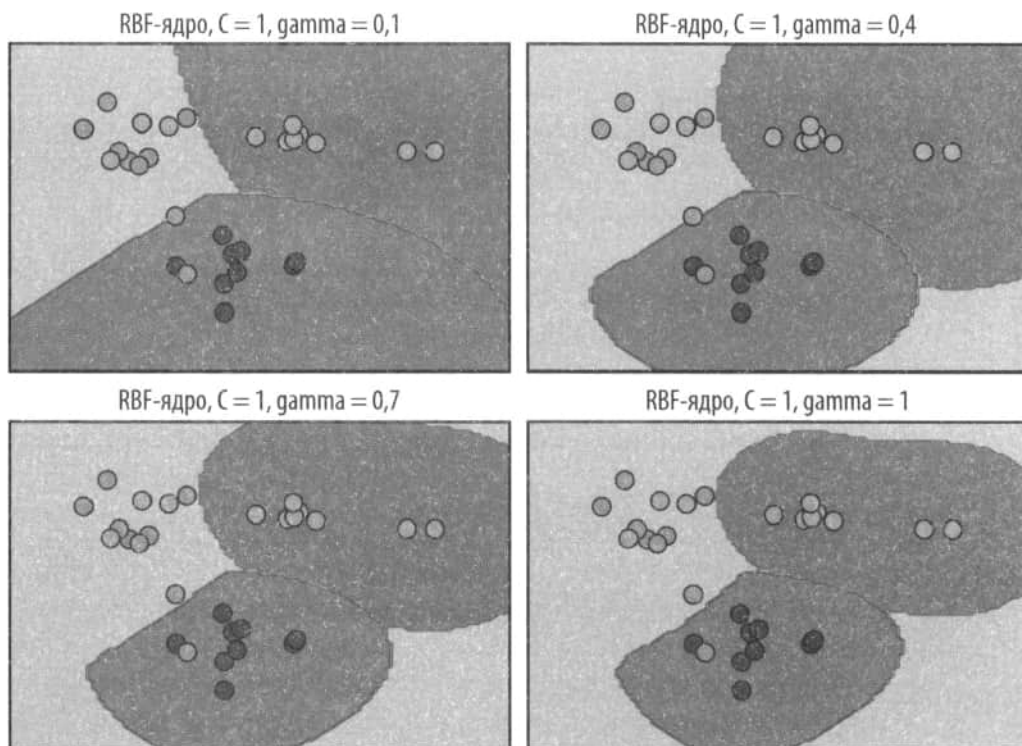


Рис. 5.8. Влияние γ на RBF-ядро

Для облегчения процесса настройки гиперпараметров Scikit предоставляет семейство оптимизаторов (<https://oreil.ly/IFpOA>), в которое входит класс `GridSearchCV` (<https://oreil.ly/n32OC>), перебирающий все комбинации заданного набора значений параметров со встроенной кросс-валидацией для определения наиболее точной модели. Эти оптимизаторы избавляют вас от необходимости писать код для перебора всех уникальных комбинаций значений параметров. Для ясности, они сами выполняют перебор, обучая модель в несколько заходов, каждый раз с различными комбинациями значений. В итоге из атрибута `best_estimator_` можно получить наиболее точную модель, из атрибута `best_params_` — значения параметров, давшие наиболее точную модель, а из атрибута `best_score_` — наилучшую оценку.

Приведем пример, в котором класс `SVC` из Scikit используется для реализации SVM-классификатора. Для начала можно создать SVM-классификатор, использующий стандартные значения параметров, и подогнать его к набору данных с помощью двух строк кода:

```
model = SVC()
model.fit(x, y)
```

При этом используется RBF-ядро с $C=1$. Тип ядра и значения C и γ можно задать таким образом:

```
model = SVC(kernel='poly', C=10, gamma=0.1)
model.fit(x, y)
```

Предположим, что необходимо попробовать два разных ядра и по пять значений C и γ , чтобы понять, какая комбинация дает наилучшие результаты. Вместо того чтобы писать вложенный цикл `for`, можно поступить следующим образом:

```
model = SVC()

grid = {
    'C': [0.01, 0.1, 1, 10, 100],
    'gamma': [0.01, 0.25, 0.5, 0.75, 1.0],
    'kernel': ['rbf', 'poly']
}
```

```
grid_search = GridSearchCV(estimator=model, param_grid=grid, cv=5, verbose=2)
grid_search.fit(x, y)  # Обучение модели с различными комбинациями параметров
```

Вызов `fit` некоторое время не осуществляется. Модель обучается 250 раз, т. к. существует 50 различных комбинаций ядра, C и γ , а $cv=5$ означает, что для оценки результатов необходимо использовать пятикратную кросс-валидацию. После завершения обучения наилучшая модель определяется следующим образом:

```
best_model = grid_search.best_estimator_
```

Нередко поиск, подобный этому, приходится проводить несколько раз — первый раз с заданными значениями параметров, а затем каждый раз с более узкими диапазонами значений, ориентируясь на значения, полученные из `best_params_`. Больше время обучения — это цена, которую приходится платить за точную модель. Повторимся, что практически всегда можно сделать SVM более точной, найдя оптимальную комбинацию параметров. И, как бы то ни было, грубая сила (*brute force*) является наиболее эффективным способом определения оптимальной комбинации.



Один нюанс, о котором следует знать в связи с использованием класса `SVC`, заключается в том, что по умолчанию он не вычисляет вероятности. Если вы хотите вызвать `predict_proba` для экземпляра `SVC`, то при создании экземпляра необходимо установить значение `probability` равным `True`:

```
model = SVC(probability=True)
```

Модель будет обучаться медленнее, но вы сможете получать вероятности, а также предсказания. Кроме того, документация Scikit предупреждает, что "`predict_proba` может быть несовместима с `predict`". Более подробная информация приведена в разделе 1.4.1.2 документации (<https://oreil.ly/Jg8X0>).

Нормализация данных

В главе 2 я отметил, что некоторые алгоритмы обучения лучше работают с нормализованными данными. Ненормализованные данные содержат столбцы чисел с сильно различающимися диапазонами, например значения от 0 до 1 в одном столбце и от 0 до 1 000 000 в другом. SVM — это параметрический обучающийся алгоритм. Обучение на нормализованных данных важно, поскольку SVM использует расстояния для расчета полей. Если одно измерение охватывает гораздо большие расстояния, чем другое, то внутренний алгоритм, используемый для поиска максимальных значений полей, может с трудом находить решение.

Важность обучения ML-моделей на нормализованных данных не ограничивается методом опорных векторов. Деревья решений и такие алгоритмы обучения, как случайные леса и деревья решений с градиентным бустингом, основанные на деревьях решений, являются непараметрическими, поэтому они одинаково хорошо работают как с нормализованными, так и с ненормализованными данными. Однако они являются исключением. Большинство других алгоритмов обучения в той или иной степени выигрывают от использования нормализованных данных. К ним относится и метод k ближайших соседей, который, хотя и является непараметрическим, использует для различения классов вычисления на основе расстояний.

Scikit предлагает несколько классов для нормализации данных. Наиболее часто используемыми являются `MinMaxScaler` (<https://oreil.ly/nz2wR>) и `StandardScaler` (<https://oreil.ly/OTTm>). Первый нормализует данные путем пропорционального уменьшения значений в каждом столбце до значений от 0,0 до 1,0. Математически это выглядит просто. Для каждого столбца набора данных `MinMaxScaler` вычитает из всех значений столбца его минимальное значение, затем делит каждое значение на разницу между минимальным и максимальным значениями. В результирующем столбце минимальное значение равно 0,0, а максимальное — 1,0.

Для демонстрации я извлек подмножества двух столбцов с сильно различающимися диапазонами из набора данных по раку молочной железы (<https://oreil.ly/IQAC9>), встроенного в Scikit. Каждый столбец содержит 100 значений. Вот первые 10 строк:

```
[[1.001e+03 3.001e-01]
 [1.326e+03 8.690e-02]
 [1.203e+03 1.974e-01]
 [3.861e+02 2.414e-01]
 [1.297e+03 1.980e-01]
 [4.771e+02 1.578e-01]
 [1.040e+03 1.127e-01]
 [5.779e+02 9.366e-02]
 [5.198e+02 1.859e-01]
 [4.759e+02 2.273e-01]]
```

Значения в первом столбце находятся в диапазоне от 201,9 до 1878,0; значения во втором столбце — от 0,000692 до 0,3754. На рис. 5.9 показано, как выглядят эти данные, если построить график с одинаковым масштабом по осям x и y . Поскольку

значения в первом столбце значительно больше значений во втором, точки данных выглядят как линия. Если изменить масштаб осей в соответствии с диапазонами значений в каждом столбце, то получится совершенно другая картина (рис. 5.10).

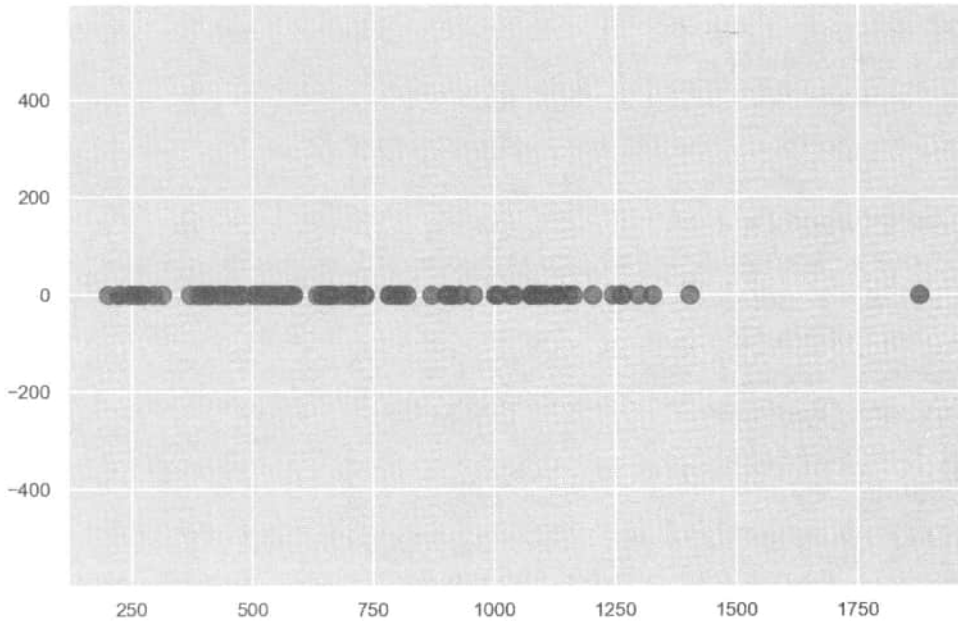


Рис. 5.9. Ненормализованные данные, построенные с одинаковым масштабом по осям

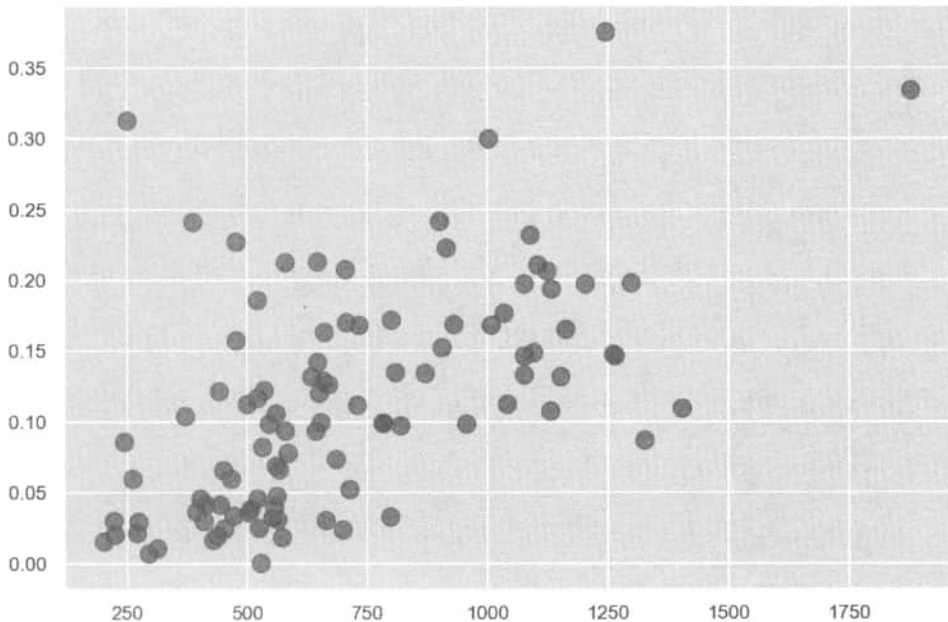


Рис. 5.10. Ненормализованные данные, построенные по пропорционально масштабированным осям

Такие ненормализованные данные могут представлять проблему для алгоритмов параметрического обучения. Один из способов решения этой проблемы — применить `MinMaxScaler` для данных:

```
from sklearn.preprocessing import MinMaxScaler
```

```
scaler = MinMaxScaler()  
normalized_data = scaler.fit_transform(data)
```

Вот первые 10 строк после min-max-нормализации:

```
[[0.47676153 0.79904352]  
 [0.67066404 0.23006715]  
 [0.5972794 0.52496344]  
 [0.10989798 0.64238821]  
 [0.65336197 0.52656469]  
 [0.16419068 0.41928115]  
 [0.50002983 0.29892076]  
 [0.22433029 0.24810786]  
 [0.18966649 0.49427287]  
 [0.16347473 0.60475891]]
```

На рис. 5.11 показан график нормализованных данных с равными осями. Форма данных не изменилась. Изменения коснулись содержимого обоих столбцов: теперь в них значения от 0,0 до 1,0.

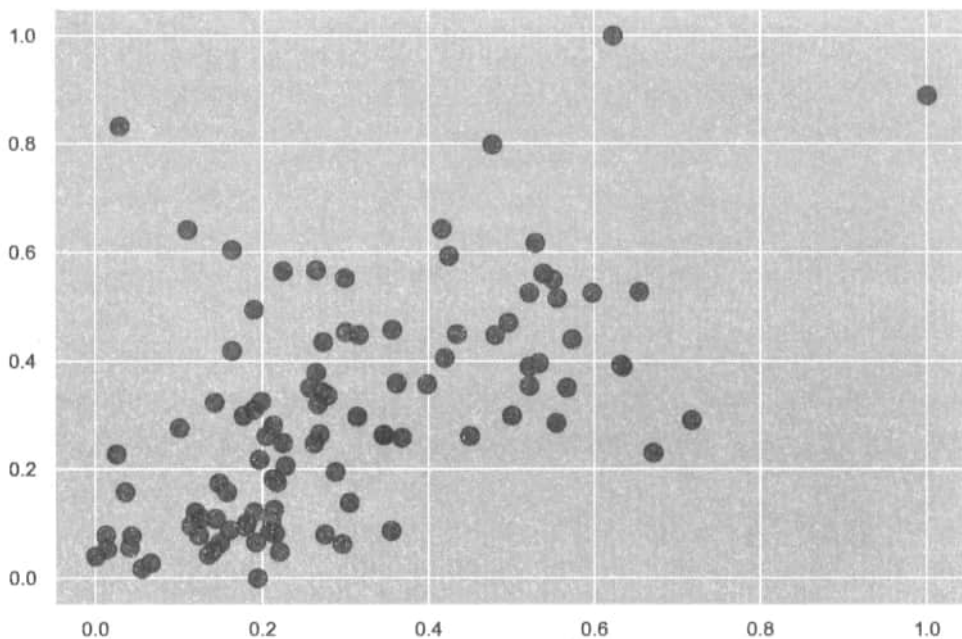


Рис. 5.11. Данные, нормализованные с помощью `MinMaxScaler`

SVM почти всегда лучше обучаются на нормализованных данных, но простой нормализации, выполняемой `MinMaxScaler`, иногда бывает недостаточно. Метод SVM, как правило, лучше реагирует на данные, нормализованные до *единичной дисперсии* с помощью техники, называемой *стандартизацией* (<https://oreil.ly/UrsYP>) или *нормализацией Z-score* (Z-оценки). Единичная дисперсия достигается путем выполнения следующих действий с каждым столбцом набора данных:

- ♦ вычислить среднее значение и стандартное отклонение всех значений в столбце;
- ♦ вычесть среднее значение из каждого значения в столбце;
- ♦ разделить каждое значение в столбце на стандартное отклонение.

Именно такое преобразование выполняет класс `StandardScaler` в Scikit для набора данных. Применить единичную дисперсию к набору данных очень просто:

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
normalized_data = scaler.fit_transform(data)
```

Значения в исходном наборе данных могут сильно различаться от столбца к столбцу, но преобразованный набор данных будет содержать столбцы чисел, привязанных к 0, с диапазонами, пропорциональными стандартному отклонению каждого столбца. Применение `StandardScaler` к набору данных приводит к следующим значениям в первых 10 строках:

```
[[ 0.93457642  2.36212718]
 [ 1.95483237 -0.35495682]
 [ 1.56870474  1.05328794]
 [-0.99574783  1.61403698]
 [ 1.86379415  1.06093451]
 [-0.71007617  0.5486138 ]
 [ 1.05700714 -0.02615397]
 [-0.39363986 -0.26880538]
 [-0.57603023  0.90672853]
 [-0.71384326  1.4343424 ]]
```

В результате получается распределение, показанное на рис. 5.12. И снова форма данных не изменилась, но существенно изменились значения, *определяющие* эту форму.

Как правило, метод SVM лучше всего работает при обучении на стандартизированных данных, даже если все столбцы имеют одинаковые диапазоны. (То же самое, кстати, справедливо и для нейронных сетей.) Классическим случаем, когда столбцы имеют схожие диапазоны, но нормализация все равно полезна, являются данные изображений, где каждый столбец содержит значения пикселей от 0 до 255. Есть и исключения, но обычно ошибочно бросать в SVM кучу данных, не разобравшись в их распределении, в частности, в том, имеет ли оно единичную дисперсию.

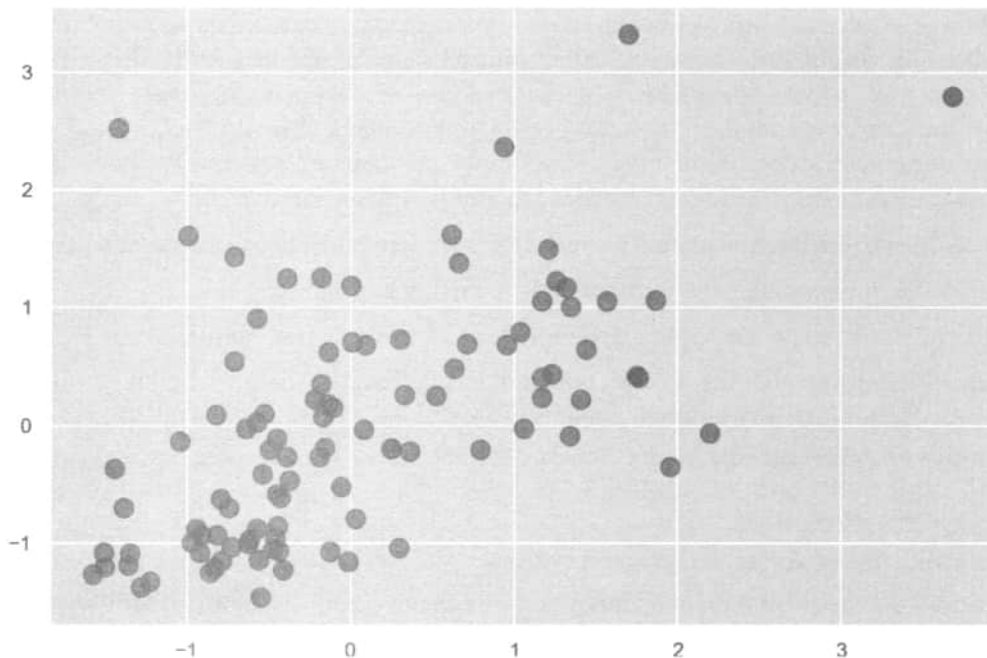


Рис. 5.12. Данные, нормализованные с помощью `StandardScaler`

Конвейеризация

Если вы нормализуете или стандартизируете значения, используемые для обучения ML-модели, то такое же преобразование необходимо применить и к значениям, вводимым в метод предсказания модели. Другими словами, модель обучается следующим образом:

```
model = SVC()
scaler = StandardScaler()
x = scaler.fit_transform(x)
model.fit(x, y)
```

и с ее помощью прогнозирование выполняется так:

```
input = [0, 1, 2, 3, 4]
model.predict([scaler.transform([input])])
```

В противном случае будут получены нелепые предсказания.

Для того чтобы упростить код и не забыть одинаково преобразовать данные обучения и прогнозирования, Scikit предлагает функцию `make_pipeline` (<https://oreil.ly/HNN2p>). Она позволяет объединить модели прогнозирования, которые Scikit называет *эстиматорами* (оценщиками), или экземпляры классов, таких как `SVC`, с преобразованиями, применяемыми к вводимым в эти модели данным.

Вот как можно использовать `make_pipeline`, чтобы гарантировать, что все вводимые в модель данные будут преобразованы с помощью `StandardScaler`:

```
# Обучение модели
```

```
pipe = make_pipeline(StandardScaler(), SVC())
```

```
pipe.fit(x, y)
```

```
# Выполнение предсказания с помощью модели
```

```
input = [0, 1, 2, 3, 4]
```

```
pipe.predict([input])
```

Теперь к используемым для обучения модели данным применяется `StandardScaler`, и таким же образом преобразуются данные, вводимые для прогнозирования.

А что если вы хотите использовать `GridSearchCV` для поиска оптимального набора параметров для конвейера, сочетающего в себе преобразование данных и эстиматор? Это несложно, но есть одна хитрость, о которой необходимо знать. Она заключается в использовании имен классов с двойными подчеркиваниями в словаре `param_grid`, передаваемом в `GridSearchCV`. Вот пример:

```
pipe = make_pipeline(StandardScaler(), SVC())
```

```
grid = {
    'svc__C': [0.01, 0.1, 1, 10, 100],
    'svc__gamma': [0.01, 0.25, 0.5, 0.75, 1.0],
    'svc__kernel': ['rbf', 'poly']
}
```

```
grid_search = GridSearchCV(estimator=pipe, param_grid=grid, cv=5, verbose=2)
```

```
grid_search.fit(x, y) # Обучение модели с различными комбинациями параметров
```

В этом примере модель обучается 250 раз, чтобы найти наилучшую комбинацию ядра, `C` и `gamma` для экземпляра `SVC` в конвейере. Обратите внимание на обозначение `"svc__"`, которое соответствует экземпляру `SVC`, переданному в функцию `make_pipeline`.

Использование SVM для распознавания лиц

Современное распознавание лиц часто осуществляется с помощью нейронных сетей, однако метод опорных векторов также может успешно выполнять данную работу. Продемонстрируем это на примере построения модели, распознающей лица. Мы будем использовать набор данных `Labeled Faces in the Wild (LFW)`, <https://oreil.ly/xG3LG>, который содержит более 13 000 изображений лиц известных людей, собранных из Интернета, и встроен в `Scikit` в качестве примера набора данных. У 1680 из более чем 5000 представленных в наборе человек есть два изображения лица и более, и только у пяти — 100 и более. Зададим минимальное количество лиц на одного человека равным 100, и это означает, что будут импортиро-

ваны пять наборов лиц, соответствующих пяти известным людям. Каждое изображение лица помечено именем человека, которому оно принадлежит.

Начнем с создания нового блокнота Jupyter и использования следующих операторов для загрузки набора данных и обрезки изображений лиц:

```
import numpy as np
import pandas as pd
from sklearn.datasets import fetch_lfw_people

faces = fetch_lfw_people(min_faces_per_person=100, slice_=None)
faces.images = faces.images[:, 35:97, 39:86]
faces.data = faces.images.reshape(faces.images.shape[0],
                                  faces.images.shape[1] * faces.images.shape[2])

print(faces.target_names)
print(faces.images.shape)
```

Всего было загружено 1140 изображений лиц. Размер каждого изображения составляет 47×62 пиксела, т. е. всего 2914 пикселей на изображение. Это означает, что набор данных содержит 2914 признаков. Используйте следующий код, чтобы показать первые 24 изображения в наборе данных и людей, которым принадлежат эти лица:

```
%matplotlib inline
import matplotlib.pyplot as plt

fig, ax = plt.subplots(3, 8, figsize=(18, 10))
for i, axi in enumerate(ax.flat):
    axi.imshow(faces.images[i], cmap='gist_gray')
    axi.set(xticks=[], yticks=[],
            xlabel=faces.target_names[faces.target[i]])
```

Результат представлен на рис. 5.13.

Для проверки баланса в наборе данных постройте гистограмму, демонстрирующую, сколько изображений лиц было импортировано для каждого человека:

```
import seaborn as sns
sns.set()

from collections import Counter
counts = Counter(faces.target)
names = {}

for key in counts.keys():
    names[faces.target_names[key]] = counts[key]

df = pd.DataFrame.from_dict(names, orient='index')
df.plot(kind='bar')
```



Рис. 5.13. Набор изображений лиц известных людей

Полученный результат показывает, что в наборе данных гораздо больше изображений Джорджа Буша-младшего, чем кого-либо другого (рис. 5.14).

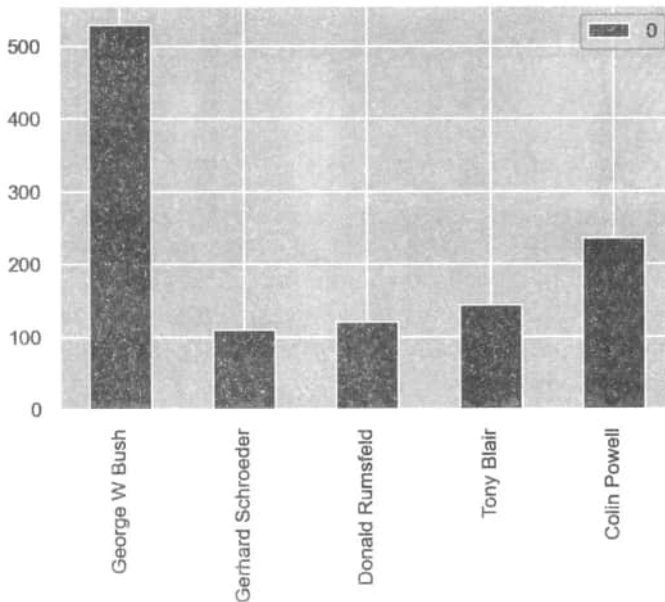


Рис. 5.14. Гистограмма частоты встречаемости изображений

Классификационные модели лучше всего обучаются на сбалансированных наборах данных.

Используйте следующий код для уменьшения набора данных до 100 изображений для каждого человека:

```
mask = np.zeros(faces.target.shape, dtype=bool)

for target in np.unique(faces.target):
    mask[np.where(faces.target == target)[0][:100]] = 1

x = faces.data[mask]
y = faces.target[mask]
x.shape
```

Обратим внимание на то, что *x* содержит 500 изображений лиц, а *y* — метки, которые к ним прилагаются: 0 для Колина Пауэлла, 1 для Дональда Рамсфельда и т. д. Теперь посмотрим, сможет ли SVM разобраться с этими данными. Мы обучим три различные модели: одну с линейным ядром, другую с полиномиальным ядром и третью с RBF-ядром. В каждом случае мы будем использовать GridSearchCV для оптимизации гиперпараметров. Начнем с линейной модели и четырех различных значений *C*:

```
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV

svc = SVC(kernel='linear')

grid = {
    'C': [0.1, 1, 10, 100]
}

grid_search = GridSearchCV(estimator=svc, param_grid=grid, cv=5, verbose=2)
grid_search.fit(x, y) # Обучение модели с различными параметрами
grid_search.best_score_
```

Точность этой модели при кросс-валидации составляет 84,2%. Вероятно, точность можно повысить за счет стандартизации данных изображений. Повторите `grid_search`, но на этот раз используйте `StandardScaler` для применения единичной дисперсии ко всем значениям пикселей:

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
svc = SVC(kernel='linear')
pipe = make_pipeline(scaler, svc)

grid = {
    'svc__C': [0.1, 1, 10, 100]
}
```

```
grid_search = GridSearchCV(estimator=pipe, param_grid=grid, cv=5, verbose=2)
grid_search.fit(x, y)
grid_search.best_score_
```

Стандартизация данных дала прирост точности. Какое значение C обеспечило такую точность?

```
grid_search.best_params_
```

Возможно ли, чтобы полиномиальное ядро превосходило линейное? Есть простой способ это выяснить. Обратите внимание на введение в набор параметров γ и degree . Эти параметры, наряду с C , могут существенно повлиять на способность полиномиального ядра подходить к обучающим данным:

```
scaler = StandardScaler()
svc = SVC(kernel='poly')
pipe = make_pipeline(scaler, svc)

grid = {
    'svc__C': [0.1, 1, 10, 100],
    'svc__gamma': [0.01, 0.25, 0.5, 0.75, 1],
    'svc__degree': [1, 2, 3, 4, 5]
}
```

```
grid_search = GridSearchCV(estimator=pipe, param_grid=grid, cv=5, verbose=2)
```

```
grid_search.fit(x, y) # Обучение модели с различными комбинациями параметров
grid_search.best_score_
```

Полиномиальное ядро достигло той же точности, что и линейное. Какие значения параметров привели к такому результату?

```
grid_search.best_params_
```

Атрибут `best_params_` показывает, что оптимальное значение `degree` равно 1, т. е. полиномиальное ядро работает как линейное. Поэтому неудивительно, что оно достигло такой же точности. Может ли RBF-ядро работать лучше?

```
scaler = StandardScaler()
svc = SVC(kernel='rbf')
pipe = make_pipeline(scaler, svc)

grid = {
    'svc__C': [0.1, 1, 10, 100],
    'svc__gamma': [0.01, 0.25, 0.5, 0.75, 1.0]
}
```

```
grid_search = GridSearchCV(estimator=pipe, param_grid=grid, cv=5, verbose=2)
grid_search.fit(x, y)
grid_search.best_score_
```

Ядро RBF оказалось не таким эффективным, как линейное и полиномиальное ядра. В этом кроется свой урок. Ядро RBF часто подходит к нелинейным данным лучше, чем другие ядра, но не всегда. Поэтому наилучшая стратегия работы с SVM — это пробовать разные ядра с различными значениями параметров. Наилучшая комбинация будет отличаться от набора данных к набору данных. Для набора данных LFW, по-видимому, лучше всего подходит линейное ядро. Это удобно, поскольку линейное ядро является самым быстрым из всех ядер, предоставляемых Scikit.



В дополнение к классу SVC в Scikit-Learn входят SVM-классификаторы LinearSVC и NuSVC. Последний поддерживает тот же набор ядер, что и класс SVC, но заменяет C на параметр регуляризации nu, который по-другому управляет плотностью подгонки. NuSVC не так хорошо масштабируется, как SVC, на большие массивы данных и, по моему опыту, используется редко. LinearSVC реализует только линейное ядро, но использует другой алгоритм оптимизации, который обучает быстрее. Если обучение с помощью SVC идет медленно, а вы определили, что линейное ядро дает наилучшую модель, подумайте о замене SVC на LinearSVC. Более быстрое время обучения имеет значение даже для наборов данных скромного размера, особенно если вы используете GridSearchCV для обучения модели сотни раз. Отличный обзор функциональных различий между двумя классами можно найти в статье "SVM в Scikit-Learn: Что вы должны знать", написанной Анжелой Ши (Shi Angela. SVM with Scikit-Learn: What You Should Know; <https://oreil.ly/OQ11k>).

Матрицы ошибок (confusion matrix) — отличный способ визуализации точности модели. Давайте разделим набор данных, обучим оптимизированную линейную модель на 80% изображений, протестируем ее на оставшихся 20% и представим результаты в виде матрицы ошибок.

Первым шагом является разбиение набора данных. Обратите внимание на параметр `stratify=y`, который гарантирует, что обучающий и тестовый наборы данных будут иметь ту же долю образцов каждого класса, что и исходный набор данных. В данном примере обучающий набор данных будет содержать по 20 образцов каждого из пяти людей:

```
from sklearn.model_selection import train_test_split
```

```
x_train, x_test, y_train, y_test = train_test_split(x, y, train_size=0.8,
                                                    stratify=y, random_state=0)
```

Теперь обучим линейную SVM с оптимальным значением C, выявленным в результате поиска по сетке (grid search):

```
scaler = StandardScaler()
svc = SVC(kernel='linear', C=0.1)
pipe = make_pipeline(scaler, svc)
pipe.fit(x_train, y_train)
```

Проведем кросс-валидацию модели для подтверждения ее точности:

```
from sklearn.model_selection import cross_val_score
```

```
cross_val_score(pipe, x, y, cv=5).mean()
```

Используйте матрицу ошибок, чтобы увидеть, как модель работает с тестовыми данными:

```
from sklearn.metrics import ConfusionMatrixDisplay as cmd
```

```
fig, ax = plt.subplots(figsize=(6, 6))
```

```
ax.grid(False)
```

```
cmd.from_estimator(pipe, x_test, y_test,  
                  display_labels=faces.target_names,  
                  cmap='Blues', xticks_rotation='vertical', ax=ax)
```

На рис. 5.15 представлен результат.

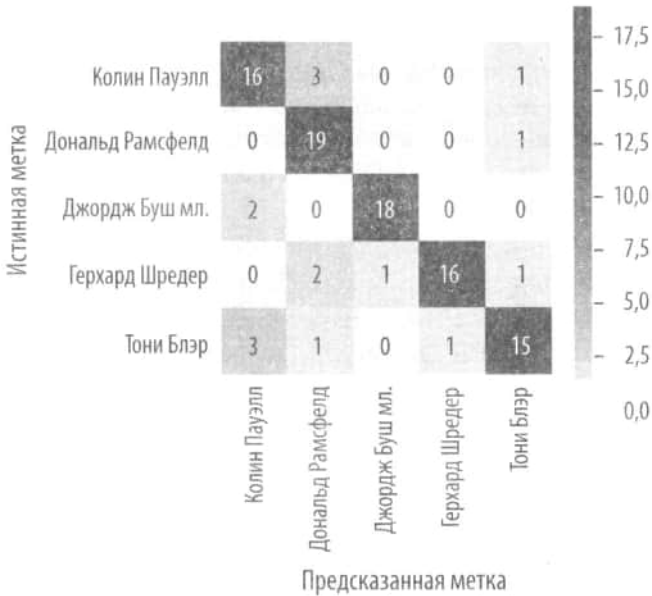


Рис. 5.15. Матрица ошибок для построенной модели

Модель правильно определила Колина Пауэлла 16 раз из 20, Дональда Рамсфелда 19 раз из 20 и т. д. Неплохо. И это отличный пример работы метода опорных векторов.

Резюме

Метод опорных векторов, или SVM, часто подходит к наборам данных лучше, чем другие алгоритмы обучения. SVM — это классификаторы с максимальной границей, использующие хитрости при работе с ядром для имитации добавления размер-

ности к данным. Теоретически данные, не поддающиеся линейному разделению в m измерениях, могут быть разделены в n измерениях, если $n > m$. Метод SVM чаще всего используется для классификации, но он может выполнять и регрессию. В качестве эксперимента попробуйте заменить `GradientBoostingRegressor` в примере с такси в главе 2 на `SVR` и использовать `GridSearchCV` для оптимизации гиперпараметров модели. Какая модель дает наибольший коэффициент детерминации при кросс-валидации?

SVM-модели обычно лучше обучаются на нормализованных к единичной дисперсии данных. Это верно, даже если значения во всех столбцах имеют схожие диапазоны, но особенно верно, если они не имеют схожих диапазонов. Класс `StandardScaler` в Scikit применяет к данным единичную дисперсию. Она достигается путем деления значений в столбце на среднее значение всех значений в столбце и деления на стандартное отклонение. Функция `make_pipeline` в Scikit позволяет объединить такие преобразователи, как `StandardScaler`, и классификаторы, такие как `SVC`, в одну логическую единицу, чтобы данные, передаваемые в `fit`, `predict` и `predict_proba`, подвергались одинаковым преобразованиям.

Для достижения оптимальной точности SVM-модели требуют настройки. Под настройкой понимается поиск правильных значений таких параметров, как `C`, `gamma` и ядро, а также перебор различных комбинаций параметров и оценка результатов. Scikit предоставляет в помощь такие классы, как `GridSearchCV`, но они увеличивают время обучения за счет того, что модель обучается один раз для каждой уникальной комбинации значений параметров.

Метод SVM может показаться волшебным в своей способности подгонять математические модели под сложные наборы данных. Но, на мой взгляд, эта магия отходит на второй план по сравнению с численной гимнастикой, которую выполняет анализ главных компонент (`principal component analysis`, PCA), позволяющий решить целый ряд проблем, регулярно возникающих в машинном обучении. Я часто презентую PCA слушателям, говоря, что это самый сокровенный секрет машинного обучения. После главы 6 этот секрет перестанет быть секретом.

Анализ главных компонент

Анализ главных компонент (principal component analysis, PCA) — одно из маленьких чудес машинного обучения. Это метод уменьшения размерности, позволяющий сократить число измерений в наборе данных без потери соразмерного количества информации. Хотя на первый взгляд это может показаться невероятным, метод очень важен для инженеров и разработчиков программного обеспечения, стремящихся построить прогностические модели на основе своих данных.

Допустим, я скажу, что можно взять набор данных с 1000 столбцами, с помощью PCA сократить его до 100 столбцов и сохранить 90% или даже больше информации из исходного набора данных. Это относительно распространенная ситуация, хотите верить, хотите нет. И такой подход можно использовать в различных практических целях, в том числе:

- ◆ для сокращения высокоразмерных данных до двух-трех измерений, чтобы их можно было отображать и исследовать;
- ◆ для уменьшения числа измерений в наборе данных с последующим восстановлением исходного числа измерений, что находит применение при обнаружении аномалий и фильтрации шумов;
- ◆ с целью анонимизации наборов данных, позволяющей предоставлять их другим лицам без раскрытия природы и смысла данных.

И это еще не все. Побочным эффектом применения PCA к набору данных является удаление менее важных признаков (столбцов данных, которые имеют меньшее значение для результатов прогностической модели), а также устранение зависимостей между столбцами. В наборах данных с низким соотношением выборок (строк) и признаков (столбцов) PCA можно применять для увеличения этого соотношения. Как правило, набор данных, используемый для машинного обучения, должен содержать в пять раз больше строк, чем столбцов. Если нет возможности добавить строки, альтернативой может стать PCA для сокращения количества столбцов.

Узнав о PCA, вы будете удивляться, как жили без него. Давайте вкратце разберемся, что это такое и как это работает. Затем рассмотрим несколько примеров, демонстрирующих, почему он является таким незаменимым инструментом.

Что такое анализ главных компонент

Один из способов понять суть PCA — посмотреть, как он сводит двумерный набор данных к одному измерению. На рис. 6.1 показан двумерный набор данных, представляющий собой случайное множество значений x и y . Если свести этот набор данных к одному измерению, просто отбросив столбец x или столбец y , то получится горизонтальная или вертикальная линия, мало похожая на исходный набор данных.

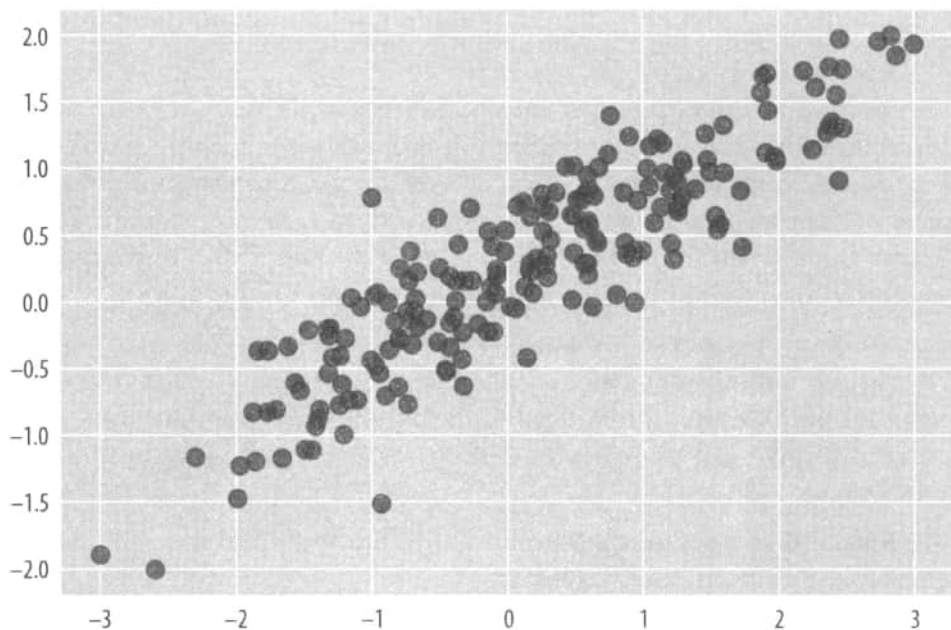


Рис. 6.1. Двумерный набор данных

На рис. 6.2 добавлены стрелки, обозначающие две главные компоненты набора данных. По сути, система координат преобразована таким образом, что одна ось (более длинная из двух стрелок) охватывает большую часть дисперсии набора данных. Это и есть *главная компонента набора данных*. Другая ось содержит более узкий диапазон значений и представляет собой вторичную главную компоненту. Количество главных компонент равно количеству измерений в наборе данных, поэтому в данном примере имеется две главные компоненты.

Для того чтобы свести двумерный набор данных к одному измерению, PCA находит две главные компоненты и удаляет ту из них, которая имеет меньшую дисперсию. Это позволяет спроецировать точки данных на ось главных компонент, как показано на рис. 6.3. Красные точки данных сохраняют не всю информацию исходного набора данных, но большую ее часть. В данном примере набор данных, обработанный методом PCA, сохранил более 95% содержавшейся в оригинале информации. PCA сократил число измерений на 50%, но при этом было потеряно менее 5% значимой информации в наборе данных. В этом и заключается суть PCA: уменьшение числа измерений без потери информации.

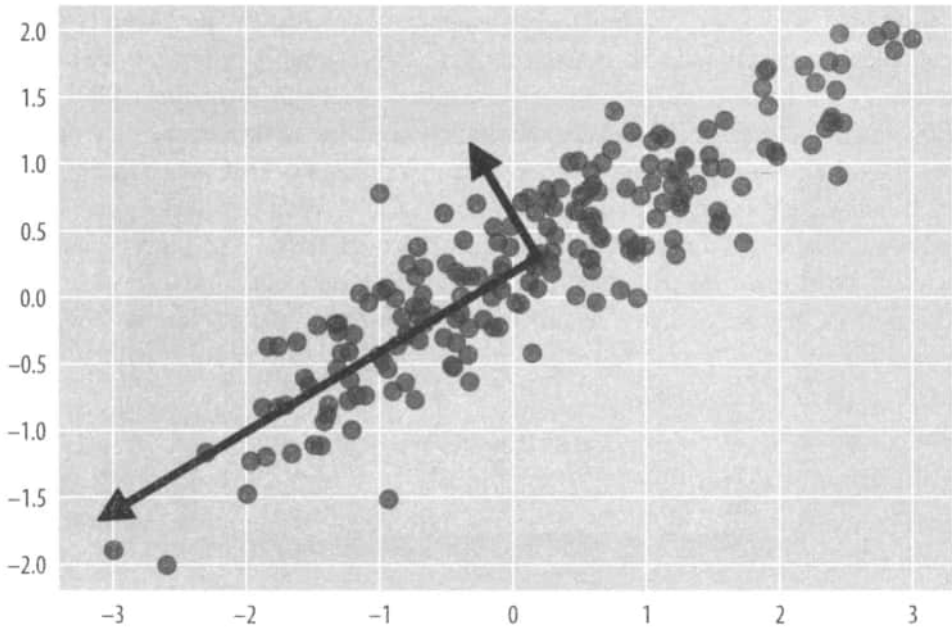


Рис. 6.2. Стрелки, изображающие главные компоненты двумерного набора данных

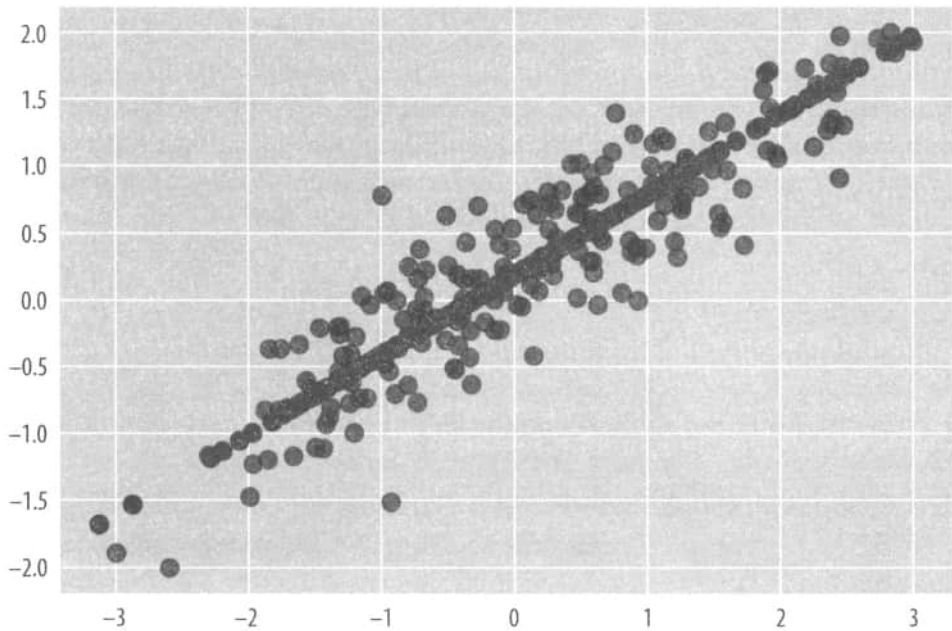


Рис. 6.3. Двумерный набор данных (синий),
сведенный к одному измерению (красный)¹ с помощью PCA

¹ Цветные иллюстрации к книге размещены на странице книги издательства по адресу bhv.ru. —
Прим. ред.

В основе PCA лежит построение *ковариационной матрицы* (<https://oreil.ly/Rz9oI>), определяющей дисперсию каждого измерения по отношению к другим, и вычисление на ее основе собственных векторов и собственных значений (<https://oreil.ly/LGSv1>), определяющих главные компоненты набора данных. Если вы хотите углубиться в эту тему, рекомендую прочитать статью Закарии Джаади "Пошаговое объяснение анализа главных компонент (PCA)" (Jaadi Zakaria. A Step-by-Step Explanation of Principal Component Analysis (PCA); <https://oreil.ly/zLREI>). Хорошая новость заключается в том, что для работы PCA не нужно разбираться в математике, поскольку класс PCA в Scikit-Learn выполняет расчеты за вас. Следующие операторы сокращают набор данных `x` до пяти измерений, независимо от того, сколько измерений он содержит изначально:

```
pca = PCA(n_components=5)
x = pca.fit_transform(x)
```

Для восстановления исходного числа измерений можно также инвертировать преобразование PCA:

```
x = pca.inverse_transform(x)
```

Метод `inverse_transform` восстанавливает исходную размерность набора данных, но не восстанавливает исходный набор данных. Информация, которая была отброшена при применении преобразования PCA, будет отсутствовать в восстановленном наборе данных.

Потерю информации при применении преобразования PCA и последующей инверсии можно наглядно продемонстрировать на примере набора данных Labeled Faces in the Wild (LFW, <https://oreil.ly/USXRc>), представленного в *главе 5*. Для демонстрации запустите блокнот Jupyter и выполните следующий код:

```
%matplotlib inline
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_lfw_people

faces = fetch_lfw_people(min_faces_per_person=100, slice_=None)
faces.images = faces.images[:, 35:97, 39:86]
faces.data = faces.images.reshape(faces.images.shape[0],
                                  faces.images.shape[1] * faces.images.shape[2])
fig, ax = plt.subplots(3, 8, figsize=(18, 10))

for i, axi in enumerate(ax.flat):
    axi.imshow(faces.images[i], cmap='gist_gray')
    axi.set(xticks=[], yticks=[], xlabel=faces.target_names[faces.target[i]])
```

На выходе получаем первые 24 изображения из набора данных (рис. 6.4).



Рис. 6.4. Первые 24 изображения из набора данных LFW

Размер каждого изображения составляет 478×62 пикселей, т. е. всего 2914 пикселей на одно изображение. Таким образом, набор данных имеет 2914 измерений. Теперь с помощью следующего кода уменьшим число измерений до 150 (оставив примерно 5% исходного числа), восстановим исходные 2914 измерений и построим график восстановленных изображений:

```
from sklearn.decomposition import PCA
```

```
pca = PCA(n_components=150, random_state=0)
```

```
pca_faces = pca.fit_transform(faces.data)
```

```
unpca_faces = pca.inverse_transform(pca_faces).reshape(1140, 62, 47)
```

```
fig, ax = plt.subplots(3, 8, figsize=(18, 10))
```

```
for i, axi in enumerate(ax.flat):
```

```
    axi.imshow(unpca_faces[i], cmap='gist_gray')
```

```
    axi.set(xticks=[], yticks=[], xlabel=faces.target_names[faces.target[i]])
```

Несмотря на то что в наборе данных было удалено почти 95% измерений, значимой информации было потеряно мало. Восстановленные изображения несколько более размыты, чем оригиналы, но лица все равно узнаваемы (рис. 6.5).

Повторим, что количество измерений уменьшилось с 2914 до 150, но поскольку PCA обнаружил 2914 главных компонент и удалил наименее важные (с наименьшей дисперсией), то основная часть информации из исходного набора данных сохранилась. В связи с этим возникает вопрос: насколько точно была сохранена исходная информация?

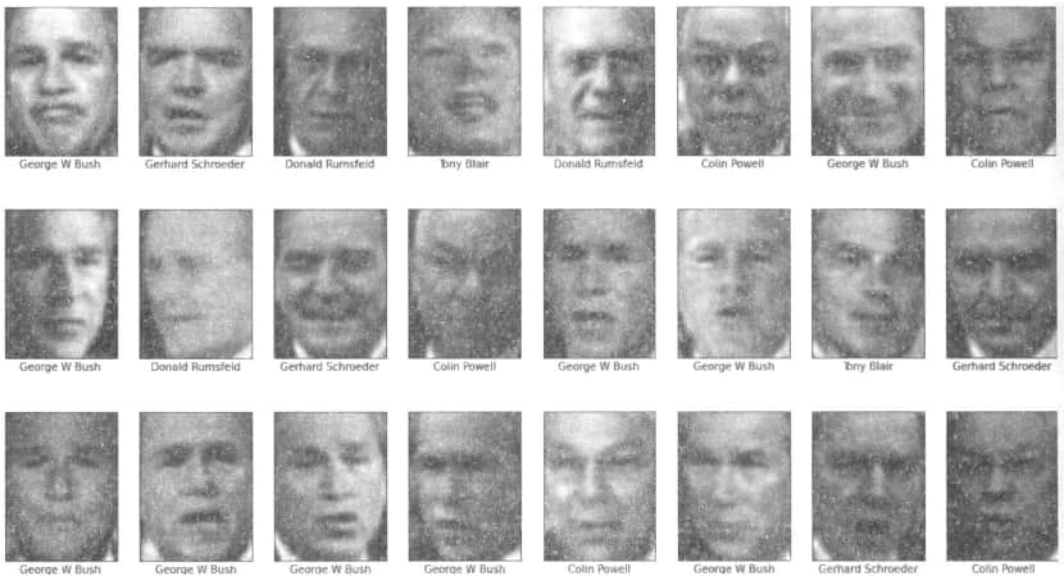


Рис. 6.5. Первые 24 изображения из набора данных LFW после уменьшения и затем восстановления числа измерений

После подгонки объекта PCA к набору данных по атрибуту `explained_variance_ratio_` можно узнать, сколько дисперсии закодировано в каждой главной компоненте. Это массив с одним элементом для каждой главной компоненты в преобразованном наборе данных. Вот как это выглядит после сокращения набора данных LFW до 150 измерений:

```
array([0.20098166, 0.1436709 , 0.0694095 , 0.0554688 , 0.04888214,
       0.02838693, 0.02344352, 0.02056908, 0.01904505, 0.01790946,
       0.01446775, 0.0141357 , 0.01173403, 0.01033751, 0.00927581,
       0.00900304, 0.00895557, 0.00830898, 0.00770731, 0.00712525,
       0.0064077 , 0.00619189, 0.00582111, 0.00557892, 0.00535471,
       0.00494034, 0.00482188, 0.00446195, 0.00443723, 0.00404091,
       0.00382839, 0.00370596, 0.00363874, 0.00352478, 0.00335927,
       0.00328615, 0.00315452, 0.00309412, 0.00290268, 0.00284517,
       0.00278296, 0.00267253, 0.00258336, 0.00249151, 0.00243766,
       0.00239778, 0.00237984, 0.00231506, 0.00223432, 0.00220306,
       0.00208555, 0.00207567, 0.00204288, 0.00196099, 0.00192303,
       0.00189352, 0.0018381 , 0.00180081, 0.00178862, 0.00174389,
       0.00168321, 0.00165759, 0.00162565, 0.00159976, 0.00153559,
       0.00152782, 0.00150262, 0.0014841 , 0.00147757, 0.00144323,
       0.00140246, 0.00138122, 0.00136053, 0.00132581, 0.00130121,
       0.00128062, 0.00126851, 0.00123904, 0.00123427, 0.00120644,
       0.00118998, 0.00117278, 0.00116551, 0.00115161, 0.00111428,
```

```
0.00108951, 0.00107443, 0.00105793, 0.00104903, 0.00104119,
0.00099986, 0.00098006, 0.00097077, 0.00095622, 0.00093874,
0.00092516, 0.00091716, 0.00091061, 0.00090051, 0.00087887,
0.00086778, 0.0008543 , 0.00084502, 0.00082587, 0.00081203,
0.00080346, 0.00079375, 0.00077893, 0.00077295, 0.00077045,
0.00075456, 0.00073704, 0.00073038, 0.00072013, 0.0007093 ,
0.00070115, 0.00069389, 0.00067964, 0.00067382, 0.00065503,
0.0006506 , 0.00063969, 0.00063328, 0.00062684, 0.00062352,
0.0006103 , 0.00060463, 0.00059769, 0.00058182, 0.00057901,
0.00056648, 0.00056551, 0.00054979, 0.00054543, 0.00053753,
0.0005361 , 0.00053067, 0.00051841, 0.00051382, 0.00050711,
0.00049933, 0.0004919 , 0.00048888, 0.00047992, 0.00047919,
0.00046916, 0.00046408, 0.00046142, 0.00045397, 0.0004432 ],
dtype=float32)
```

Таким образом, выясняется, что 20% дисперсии набора данных объясняется первичной главной компонентой, 14% — вторичной и т. д. Обратите внимание, что с увеличением индекса значения уменьшаются. По определению каждая главная компонента в наборе данных, подвергнутом PCA, содержит больше информации, чем следующая за ней главная компонента. В нашем примере отброшенные 2764 главные компоненты содержали настолько мало информации, что их потеря была едва заметной при инвертировании преобразования. Действительно, сумма 150 чисел в предыдущем примере равна 0,938. Это означает, что при сокращении набора данных с 2914 измерений до 150 сохранилось 93,8% информации, содержащейся в исходном наборе данных. Другими словами, вы сократили число измерений почти на 95% и при этом сохранили почти 94% информации в наборе данных. Если это не потрясает, то я не знаю, что это такое.

Логично задать вопрос: какое количество компонентов является "правильным"? Другими словами, какое количество компонентов обеспечивает оптимальный баланс между сокращением числа измерений в наборе данных и сохранением большей части информации? Одним из способов поиска такого числа является построение *графика осыпи* (или *осыпной диаграммы* — *scree plot*), который показывает долю объясненной дисперсии для каждого измерения. Приведенный ниже код выдает осыпную диаграмму для PCA-преобразования, использованного для изображений лиц:

```
import seaborn as sns
sns.set()

plt.plot(pca.explained_variance_ratio_)
plt.xlabel('Главная компонента')
plt.ylabel('Объясненная дисперсия')
```

На рис. 6.6 приведен результат.

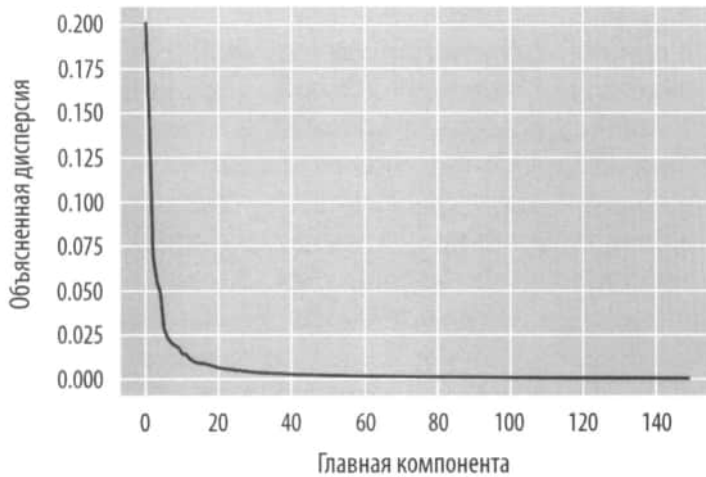


Рис. 6.6. График осыпи

Другой способ рассмотреть этот вопрос — построить график кумулятивной суммы дисперсий в зависимости от количества компонентов:

```
import numpy as np
plt.plot(np.cumsum(pca.explained_variance_ratio_))
plt.xlabel('Число компонентов')
plt.ylabel('Кумулятивная объясненная дисперсия')
```

На рис. 6.7 приведен результат.

Какой бы способ вы ни использовали, основная часть информации содержится в первых 50–100 измерениях. Исходя из этих графиков, если уменьшить число измерений до 50, а не до 150, можно ли ожидать, что восстановленные изображения лиц будут выглядеть существенно иначе? Если вы не уверены, попробуйте и посмотрите.

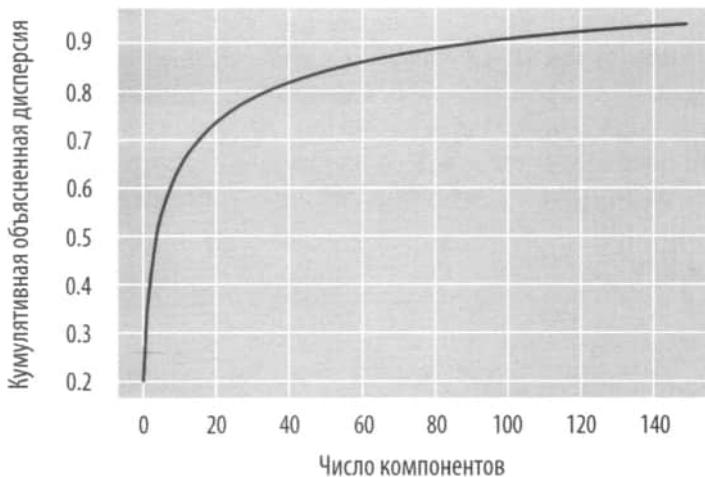


Рис. 6.7. График кумулятивной суммы дисперсий

Фильтрация шумов

Одним из очень практичных применений PCA является фильтрация шумов в данных. Шум (<https://oreil.ly/yjLYD>) — это случайные, искаженные или иным образом бессмысленные данные, которые особенно часто возникают при использовании физических устройств, таких как датчики давления или акселерометры. Основной подход к использованию PCA для уменьшения шума заключается в том, что PCA преобразует данные, а затем инвертирует преобразование, уменьшая набор данных с m размерностей до n , а затем восстанавливая его до m . Поскольку при уменьшении размерностей PCA отбрасывает наименее важную информацию, а шум, как правило, не имеет или имеет незначительную информационную ценность, в идеале это позволяет устранить большую часть шума, сохранив при этом большую часть значимых данных.

Проверить это предположение можно на наборе данных LFW. Посредством следующих операторов добавьте шум к изображениям лиц с помощью генератора случайных чисел и постройте первые 24 изображения:

```
%matplotlib inline
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_lfw_people
import numpy as np

faces = fetch_lfw_people(min_faces_per_person=100, slice_=None)
faces.images = faces.images[:, 35:97, 39:86]
faces.data = faces.images.reshape(faces.images.shape[0],
                                  faces.images.shape[1] * faces.images.shape[2])

np.random.seed(0)
noisy_faces = np.random.normal(faces.data, 0.0765)

fig, ax = plt.subplots(3, 8, figsize=(18, 10))

for i, axi in enumerate(ax.flat):
    axi.imshow(noisy_faces[i].reshape(62, 47), cmap='gist_gray')
    axi.set(xticks=[], yticks=[], xlabel=faces.target_names[faces.target[i]])
```

Полученные изображения лиц напоминают неподвижный экран телевизора шестидесятых годов прошлого века (рис. 6.8).

Теперь используем PCA для уменьшения числа размерностей. Вместо того чтобы устанавливать количество измерений (компонентов), мы укажем, что хотим сократить количество информации в наборе данных до 80%.

Мы позволим Scikit решить, сколько размерностей останется, а затем покажем результат:

```
from sklearn.decomposition import PCA

pca = PCA(0.8, random_state=0)
pca_faces = pca.fit_transform(noisy_faces)
pca.n_components_
```



Рис. 6.8. Первые 24 изображения после добавления шума

PCA сократил число измерений с 2914 до 179, но оставшиеся измерения содержат 80% информации, той, что была в исходных 2914. Теперь восстановим изображения лиц из изображений, полученных с помощью PCA, и приведем результаты:

```
unpca_faces = pca.inverse_transform(pca_faces)

fig, ax = plt.subplots(3, 8, figsize=(18, 10))

for i, axi in enumerate(ax.flat):
    axi.imshow(unpca_faces[i].reshape(62, 47), cmap='gist_gray')
    axi.set(xticks=[], yticks=[], xlabel=faces.target_names[faces.target[i]])
```

На рис. 6.9 приведен результат.

Восстановленный набор данных не так безупречен, как оригинал, но он достаточно хорош, чтобы можно было различить лица на фотографиях.



Рис. 6.9. Изображения после применения PCA

Анонимизация данных

В главе 3 было показано, как с помощью различных алгоритмов обучения построить модель бинарной классификации, позволяющую обнаружить мошенничество с кредитными картами. Набор данных, использованный в примере, содержал реальные данные кредитных карт, которые были анонимизированы для защиты владельцев карт (и интеллектуальной собственности кредитной компании). Первые 10 строк этого набора данных показаны на рис. 6.10.

	Time	V1	V2	V3	...	V27	V28	Amount	Class
0	0.0	-1.359807	-0.072781	2.536347	...	0.133558	-0.021053	149.62	0
1	0.0	1.191857	0.266151	0.166480	...	-0.008983	0.014724	2.69	0
2	1.0	-1.358354	-1.340163	1.773209	...	-0.055353	-0.059752	378.66	0
3	1.0	-0.966272	-0.185226	1.792993	...	0.062723	0.061458	123.50	0
4	2.0	-1.158233	0.877737	1.548718	...	0.219422	0.215153	69.99	0
5	2.0	-0.425966	0.960523	1.141109	...	0.253844	0.081080	3.67	0
6	4.0	1.229658	0.141004	0.045371	...	0.034507	0.005168	4.99	0
7	7.0	-0.644269	1.417964	1.074380	...	-1.206921	-1.085339	40.80	0
8	7.0	-0.894286	0.286157	-0.113192	...	0.011747	0.142404	93.20	0
9	9.0	-0.338262	1.119593	1.044367	...	0.246219	0.083076	3.68	0

Рис. 6.10. Анонимизированный набор данных для обнаружения мошенничества²

² Названия столбцов: Time — время; Amount — объем; Class — класс. — Прим. ред.

Другим практическим применением PCA является анонимизация данных. Обычно этот процесс состоит из двух этапов:

1. "Сокращение" набора данных с помощью PCA с m размерностей до n , где n — исходное число размерностей (а также число размерностей после "сокращения").
2. Нормализация данных таким образом, чтобы дисперсия была равна единице.

Второй шаг не является обязательным, но он позволяет сделать диапазоны значений более однородными. Обезличенные таким образом данные все равно можно использовать для обучения ML-модели, но их первоначальное значение невозможно определить.

Попробуйте выполнить эту процедуру самостоятельно. Во-первых, используйте следующий код для загрузки набора данных Scikit по раку молочной железы (<https://oreil.ly/siksH>) и отображения первых пяти строк:

```
import pandas as pd
from sklearn.datasets import load_breast_cancer
```

```
data = load_breast_cancer()
df = pd.DataFrame(data=data.data, columns=data.feature_names)
pd.set_option('display.max_columns', 6)
df.head()
```

На рис. 6.11 приведен результат.

	mean radius	mean texture	mean perimeter	...	worst concave points	worst symmetry	worst fractal dimension
0	17.99	10.38	122.80	...	0.2654	0.4601	0.11890
1	20.57	17.77	132.90	...	0.1860	0.2750	0.08902
2	19.69	21.25	130.00	...	0.2430	0.3613	0.08758
3	11.42	20.38	77.58	...	0.2575	0.6638	0.17300
4	20.29	14.34	135.10	...	0.1625	0.2364	0.07678

5 rows × 30 columns

Рис. 6.11. Первые пять строк загруженного набора данных³

Набор данных содержит 30 столбцов, не считая столбца с метками. Теперь с помощью следующих операторов найдите 30 главных компонент и примените `StandardScaler` к преобразованным данным:

```
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
```

³ Названия столбцов: mean radius — средний радиус; mean texture — средняя текстура; mean perimeter — средний периметр; worst concave points — наихудшие вогнутые точки; worst symmetry — наихудшая симметрия; worst fractal dimension — наихудшая фрактальная размерность. — *Прим. ред.*

```
pca = PCA(n_components=30, random_state=0)
pca_data = pca.fit_transform(df)

scaler = StandardScaler()
anon_df = pd.DataFrame(scaler.fit_transform(pca_data))
pd.set_option('display.max_columns', 8)
anon_df.head()
```

На рис. 6.12 приведен результат.

	0	1	2	3	...	26	27	28	29
0	1.743043	-3.440692	1.832695	-1.179529	...	-1.033900	0.767070	1.406020	0.841434
1	1.906779	0.182972	-1.335313	2.418269	...	-0.043492	-0.798802	0.484854	-1.267746
2	1.496120	0.458381	-0.064503	0.568556	...	0.092680	0.010964	-0.547972	0.484234
3	-0.611764	-0.788775	0.327197	-1.592188	...	0.008095	0.811865	-1.511794	-1.978890
4	1.397781	2.216483	0.051866	1.150718	...	1.716566	0.161769	1.260500	0.390467

5 rows × 30 columns

Рис. 6.12. Фрагмент набора данных после поиска 30 главных компонент и применения единичной дисперсии

После преобразования PCA набор данных становится неузнаваемым. Без преобразования невозможно работать в обратном направлении и восстановить исходные данные. Тем не менее сумма значений объясненной дисперсии `explained_variance_ratio_` равна 1,0, что означает отсутствие потери информации. Это можно доказать следующим образом:

```
import numpy as np
np.sum(pca.explained_variance_ratio_)
```

Набор данных, обработанный с помощью PCA, так же полезен для машинного обучения, как и исходный. Более того, если вы захотите поделиться этим набором данных с другими людьми, чтобы они могли обучать собственные модели, то не будет риска разглашения конфиденциальной или служебной информации.

Визуализация высокоразмерных данных

Еще одно применение PCA — это приведение набора данных к двум или трем измерениям, чтобы его можно было отобразить на графике с помощью таких библиотек, как Matplotlib (<https://oreil.ly/p7u82>). Вы не можете построить график для набора данных, содержащего 1000 столбцов. Можно построить набор данных, состоящий из двух или трех столбцов. Тот факт, что PCA позволяет свести высокоразмерные данные к двум или трем измерениям, сохранив при этом большую часть исходной

информации, делает его отличным инструментом для исследования данных и визуализации связей между классами.

Предположим, вы строите модель классификации и хотите заранее оценить, достаточно ли разделения между классами для поддержки такой модели. Возьмем, к примеру, набор данных Optical Recognition of Handwritten Digits (<https://oreil.ly/RuXLJ>), встроенный в Scikit. Каждая цифра в этом наборе представлена массивом пиксельных значений 8×8 , т. е. набор имеет 64 измерения. Если бы существовала возможность построить 64-мерную диаграмму, то можно было бы проанализировать набор данных и найти разделение между классами. Но 64 измерения — это слишком много для большинства людей.

В следующем коде загружается набор данных, с помощью PCA он сводится к двум измерениям, а затем и строится график, на котором различными цветами обозначены разные классы (цифры):

```
from sklearn.decomposition import PCA
from sklearn.datasets import load_digits
import matplotlib.pyplot as plt
%matplotlib inline

digits = load_digits()
pca = PCA(n_components=2, random_state=0)
pca_digits = pca.fit_transform(digits.data)

plt.figure(figsize=(12, 8))
plt.scatter(pca_digits[:, 0], pca_digits[:, 1], c=digits.target,
            cmap=plt.cm.get_cmap('Paired', 10))

plt.colorbar(ticks=range(10))
plt.clim(-0.5, 9.5)
```

Полученная диаграмма внушает оптимизм и свидетельствует о том, что на этих данных можно обучить классификатор. Несмотря на то что между классами наблюдается некоторое перекрытие, различные классы образуют довольно четкие кластеры. Значительное перекрытие наблюдается между красным (цифра 4) и светло-фиолетовым (цифра 6) цветами, что указывает на то, что модель может испытывать определенные трудности с различением 4 и 6. Однако цифры 0 и 1 расположены сверху и снизу, а цифры 3 и 4 — слева и справа. Предполагается, что модель будет хорошо различать эти цифры (рис. 6.13).

Взаимосвязи между классами можно лучше представить с помощью трехмерной диаграммы. Следующий код использует PCA для сокращения набора данных до трех измерений и Mplot3D (<https://oreil.ly/sNdI9>) для создания интерактивного графика. Обратите внимание, что если вы запускаете этот код в Jupyter Lab, то вам, вероятно, придется изменить первую строку на `%matplotlib widget`:

```
%matplotlib notebook
from mpl_toolkits.mplot3d import Axes3D
```

```
digits = load_digits()
pca = PCA(n_components=3, random_state=0)
pca_digits = pca.fit_transform(digits.data)

ax = plt.figure(figsize=(12, 8)).add_subplot(111, projection='3d')
ax.scatter(xs = pca_digits[:, 0], ys = pca_digits[:, 1], zs = pca_digits[:, 2],
          c=digits.target, cmap=plt.cm.get_cmap('Paired', 10))
```

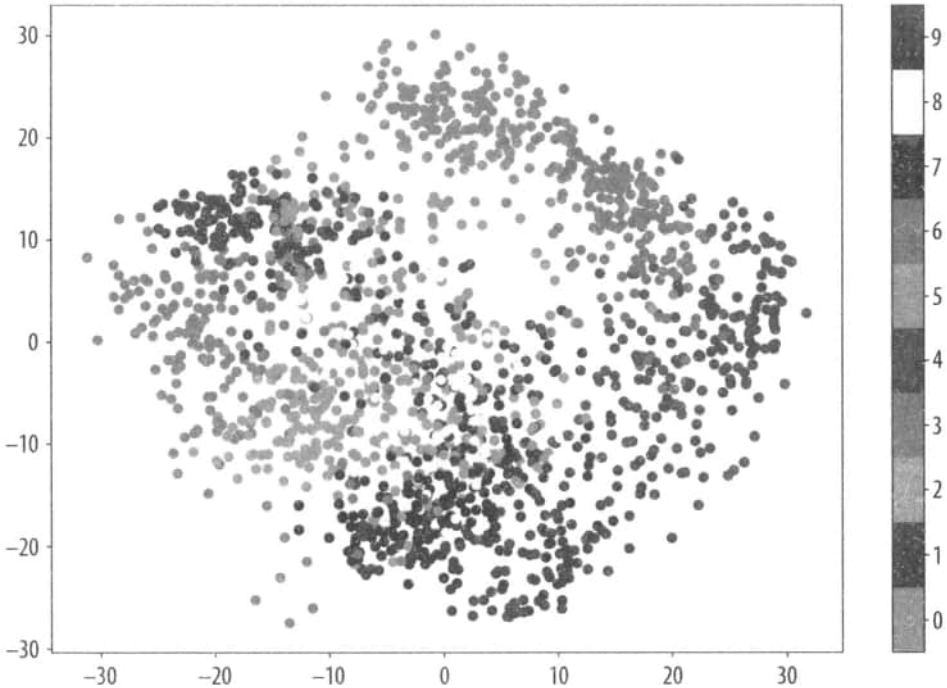


Рис. 6.13. Диаграмма классификации цифр

Полученный график можно повернуть в 3D и посмотреть на него под разными углами. На рис. 6.14 мы видим, что разделение между 4 и 6 более заметно, чем в двух измерениях.

PCA не единственный способ уменьшить набор данных до двух или трех измерений для построения графика. Можно также использовать классы *Isomap* (<https://oreil.ly/f9TL6>) или *tSNE* (<https://oreil.ly/vt1cf>) из *Scikit*. *tSNE* реализует *стохастическое вложение соседей с t-распределением*, или сокращенно *t-SNE* (<https://oreil.ly/ncz7O>). *t-SNE* — это алгоритм уменьшения размерности, который используется почти исключительно для визуализации высокоразмерных данных. В то время как PCA использует линейную функцию для преобразования данных, *t-SNE* применяет нелинейное преобразование, которое стремится усилить разделение между классами, удерживая похожие точки данных близко друг к другу в низкоразмерном пространстве. (PCA, напротив, фокусируется на том, чтобы разнести разнородные точки подальше друг от друга.)

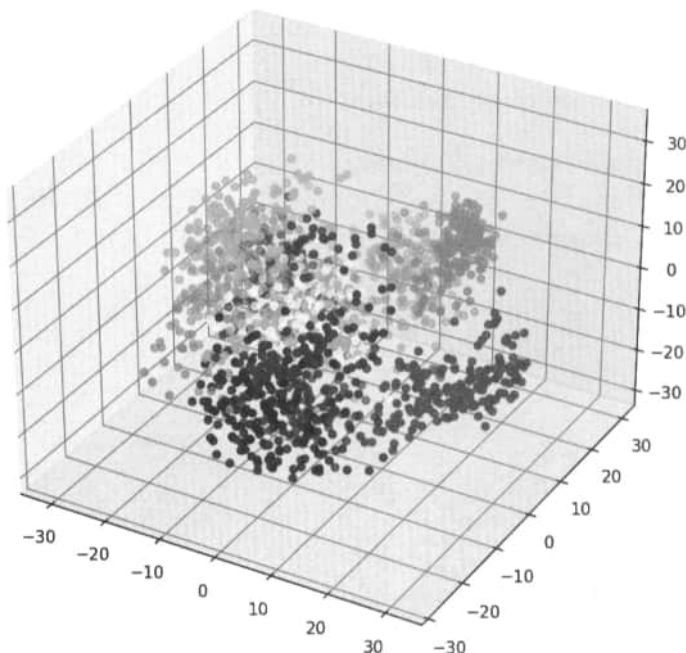


Рис. 6.14. Трехмерная диаграмма классификации цифр

Приведем пример двумерного графика набора данных Digits после его преобразования с помощью *t*-SNE:

```
%matplotlib inline
from sklearn.manifold import TSNE

digits = load_digits()
tsne = TSNE(n_components=2, init='pca', learning_rate='auto',
            random_state=0)
tsne_digits = tsne.fit_transform(digits.data)

plt.figure(figsize=(12, 8))
plt.scatter(tsne_digits[:, 0], tsne_digits[:, 1], c=digits.target,
            cmap=plt.cm.get_cmap('Paired', 10))
plt.colorbar(ticks=range(10))
plt.clim(-0.5, 9.5)
```

На рис. 6.15 приведен результат.

t-SNE лучше справляется с задачей разделения групп цифр на кластеры, что свидетельствует о наличии в данных закономерностей, которые может использовать машинное обучение. Основным недостатком *t*-SNE является то, что метод требует больших вычислительных затрат, а значит, его выполнение на больших массивах

данных может занять непомерно много времени. Одним из способов снижения этого недостатка является выполнение t -SNE не на всем наборе данных, а на подмножестве строк. Другая стратегия заключается в использовании PCA для уменьшения числа размерностей, а затем в обработке полученного набора данных методом t -SNE.

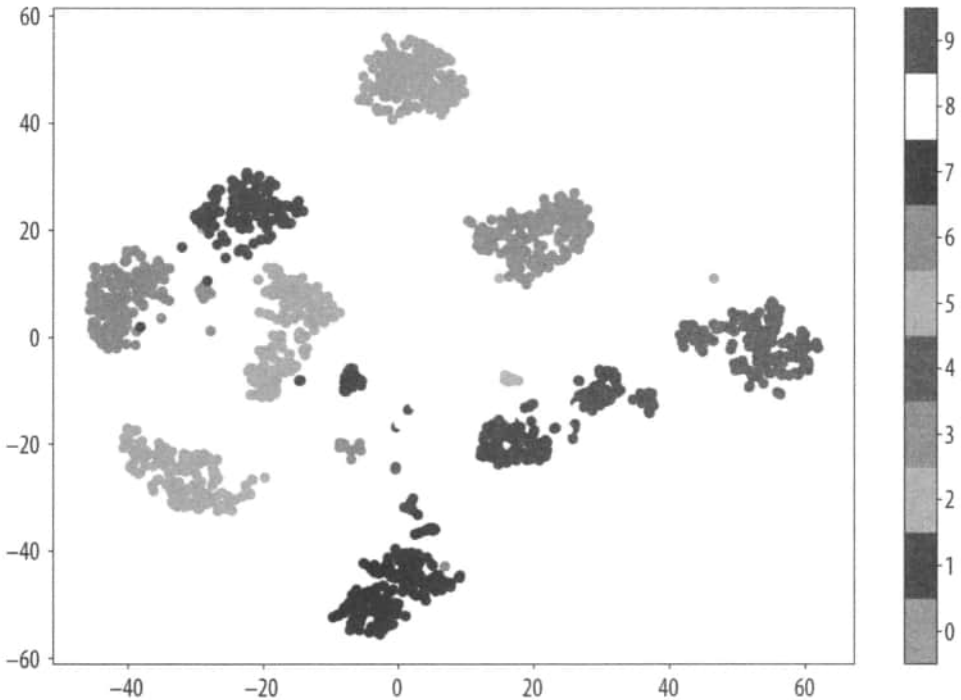


Рис. 6.15. Двумерный график набора данных Digits после его преобразования с помощью t -SNE

Обнаружение аномалий

Обнаружение аномалий (anomaly detection) — это направление машинного обучения, целью которого является выявление аномалий в наборах или потоках данных. Компания Airbus использует его для прогнозирования отказов реактивных двигателей (<https://oreil.ly/Aafuz>) и обнаружения аномалий в передаваемых с Международной космической станции телеметрических данных (<https://oreil.ly/OiEKG>). Финансовые компании применяют его для выявления мошенничества с кредитными картами (<https://oreil.ly/tjAtA>). Цель обнаружения аномалий — выявление выбросов в выборках данных, которые не являются "нормальными" по сравнению с другими. В случае мошенничества с кредитными картами предполагается, что если транзакции подвергнуть алгоритму обнаружения аномалий, то мошеннические транзакции будут показаны как аномальные, а нормальные — нет.

Существует множество алгоритмов обнаружения аномалий. Они имеют такие названия, как изолированные леса (<https://oreil.ly/PtdRg>), одноклассовые SVM (<https://oreil.ly/KFoT4>) и локальный фактор выброса (local outlier factor, LOF) (<https://oreil.ly/LumWj>). Большинство из них основаны на методах обучения без контроля и поэтому не требуют использования помеченных данных. Они просто просматривают набор образцов и определяют, какие из них являются аномальными. Обнаружение аномалий без контроля особенно интересно тем, что оно не требует априорных знаний о том, что такое аномалия, и не обязывает нас тщательно маркировать немаркированный набор данных.

Одна из наиболее популярных форм обнаружения аномалий основана на анализе главных компонент. Вы уже знаете, что с помощью PCA можно уменьшить размерность данных с m до n , а преобразование PCA можно инвертировать, чтобы восстановить исходную размерность m . Также известно, что инвертирование преобразования не восстанавливает данные, которые были потеряны при его применении. Суть обнаружения аномалий на основе PCA заключается в том, что аномальная выборка должна иметь большую потерю или *ошибку восстановления* (разницу между исходными данными и теми же данными после применения и инвертирования PCA-преобразования), чем нормальная. Другими словами, потери при PCA-преобразовании аномальной выборки должны быть выше, чем потери при применении той же операции к нормальной выборке. Посмотрим, насколько это предположение соответствует реальным условиям.

Использование PCA для обнаружения мошенничества с кредитными картами

Обучение с учителем (или контролируемое обучение — supervised learning) не единственный вариант обнаружения мошенничества с кредитными картами. Существует альтернативный подход, в котором для выявления мошеннических транзакций используется метод обнаружения аномалий на основе PCA. Начнем с загрузки набора данных, разделяя образцы по классам в один набор данных, представляющий нормальные транзакции, и в другой, представляющий мошеннические транзакции, и отбрасывая столбцы `Time` и `Class`. Если вы не загрузили набор данных в главе 3, можете извлечь его сейчас из ZIP-файла (<https://oreil.ly/RTNYn>).

```
import pandas as pd
```

```
df = pd.read_csv('Data/creditcard.csv')
df.head()
```

```
# Разделение образцов по классам
```

```
legit = df[df['Class'] == 0]
```

```
fraud = df[df['Class'] == 1]
```

```
# Удаление столбцов Time и Class
```

```
legit = legit.drop(['Time', 'Class'], axis=1)
```

```
fraud = fraud.drop(['Time', 'Class'], axis=1)
```

Используйте PCA для уменьшения двух наборов данных с 29 до 26 измерений, а затем инвертируйте это преобразование для восстановления каждого набора данных до 29 измерений. Преобразование применяется только для нормальных транзакций, поскольку нам необходимо базовое значение ошибки восстановления, позволяющее отличить нормальные транзакции от мошеннических. Однако оно применяется к обоим наборам данных:

```
from sklearn.decomposition import PCA
```

```
pca = PCA(n_components=26, random_state=0)
```

```
legit_pca = pd.DataFrame(pca.fit_transform(legit), index=legit.index)
```

```
fraud_pca = pd.DataFrame(pca.transform(fraud), index=fraud.index)
```

```
legit_restored = pd.DataFrame(pca.inverse_transform(legit_pca), index=legit_pca.index)
```

```
fraud_restored = pd.DataFrame(pca.inverse_transform(fraud_pca), index=fraud_pca.index)
```

При этом была потеряна часть информации. Остается надеяться, что мошеннические транзакции понесли бóльшие потери, чем нормальные, и мы можем использовать это для их различения. Следующий шаг — вычисление потерь для каждой строки в двух наборах данных путем суммирования квадратов разностей между значениями в исходной и восстановленной строках:

```
import numpy as np
```

```
def get_anomaly_scores(df_original, df_restored):
```

```
    loss = np.sum((np.array(df_original) - np.array(df_restored)) ** 2, axis=1)
```

```
    loss = pd.Series(data=loss, index=df_original.index)
```

```
return loss
```

```
legit_scores = get_anomaly_scores(legit, legit_restored)
```

```
fraud_scores = get_anomaly_scores(fraud, fraud_restored)
```

Теперь построим график потерь, понесенных при преобразовании и восстановлении законных сделок:

```
%matplotlib inline
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
sns.set()
```

```
legit_scores.plot(figsize = (12, 6))
```

На рис. 6.16 приведен результат.

Далее построим график потерь для мошеннических операций:

```
fraud_scores.plot(figsize = (12, 6))
```

На рис. 6.17 приведен результат.

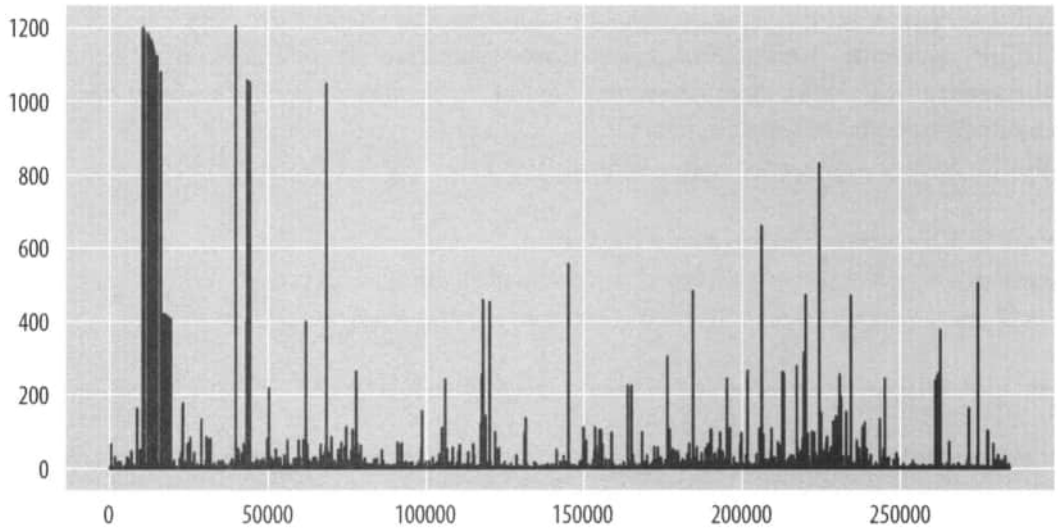


Рис. 6.16. График потерь, понесенных при преобразовании и восстановлении законных сделок

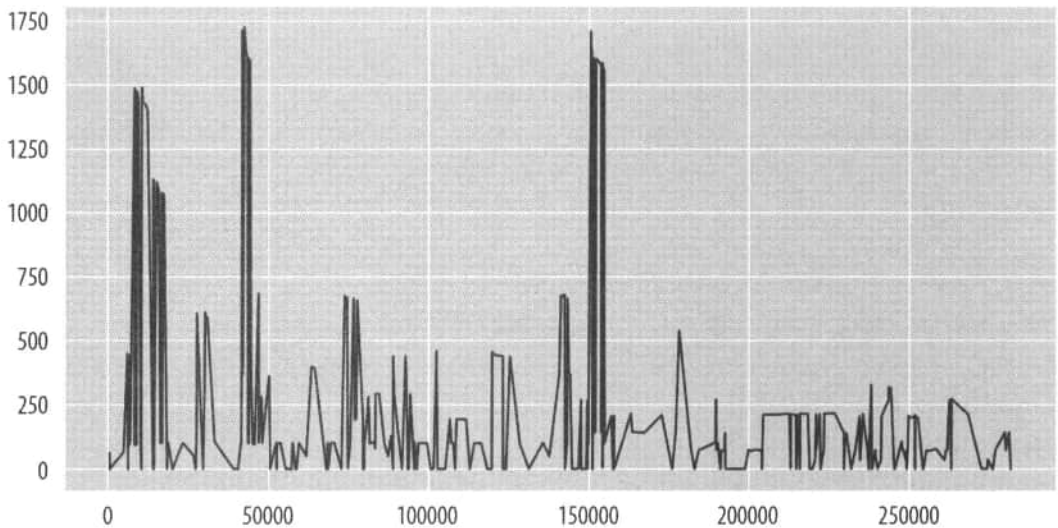


Рис. 6.17. График потерь, понесенных при преобразовании и восстановлении мошеннических сделок

Графики показывают, что большинство строк набора данных, представляющих нормальные (законные) транзакции, имеют потери менее 200, в то время как многие строки набора данных мошеннических транзакций имеют потери более 200. Разделите строки по этому признаку, определив транзакции с потерями менее 200

как нормальные, а с большими потерями — как мошеннические, и используйте матрицу ошибок для визуализации результатов:

```
threshold = 200
```

```
true_neg = legit_scores[legit_scores < threshold].count()
false_pos = legit_scores[legit_scores >= threshold].count()
true_pos = fraud_scores[fraud_scores >= threshold].count()
false_neg = fraud_scores[fraud_scores < threshold].count()
```

```
labels = ['Легитимная', 'Мошенническая']
```

```
mat = [[true_neg, false_pos], [false_neg, true_pos]]
```

```
sns.heatmap(mat, square=True, annot=True, fmt='d', cbar=False,
             cmap='Blues', xticklabels=labels, yticklabels=labels)
```

```
plt.xlabel('Предсказанная метка')
```

```
plt.ylabel('Истинная метка')
```

На рис. 6.18 приведен результат.



Рис. 6.18. Матрица ошибок построенной модели

Результаты не столь хороши, как в случае со случайным лесом, но модель все же уловила около 50% мошеннических транзакций, ошибившись при этом всего в 76 из 284 315 законных транзакций. Таким образом, коэффициент ошибок для нормальных транзакций составляет менее 0,03% по сравнению с 0,007% для модели с обучением с учителем.

Два параметра в этой модели определяют уровень ошибок: количество размерностей, к которым были приведены наборы данных (26), и порог, выбранный для раз-

личения нормальных и мошеннических транзакций (200). Точность можно регулировать, экспериментируя с различными значениями. Я провел неформальное тестирование и пришел к выводу, что это вполне приемлемое сочетание. Выбор более низкого порога повышает способность модели выявлять мошеннические транзакции, но за счет ошибочной классификации большего числа нормальных транзакций. В конце концов, вы должны решить, с каким коэффициентом ошибок вы готовы смириться, помня о том, что отказ в допустимой покупке по карте, скорее всего, разозлит клиента.

Использование PCA для прогнозирования выхода из строя подшипников

Одним из классических способов обнаружения аномалий является прогнозирование отказов вращающихся механизмов. Применим обнаружение аномалий на основе PCA к подмножеству опубликованного NASA набора данных для прогнозирования отказов подшипников (<https://oreil.ly/B9jOD>). Набор содержит данные о вибрации четырех подшипников, поддерживающих вращающийся вал с приложенной к нему радиальной нагрузкой 6000 фунтов. Подшипники были обкатаны до отказа, а данные о вибрации снимались высокочувствительными кварцевыми акселерометрами через равные промежутки времени до момента отказа.

Сначала загрузите CSV-файл (<https://oreil.ly/vOUCc>) с подмножеством, которое я выбрал из большого набора данных NASA. Затем создайте блокнот Jupyter и загрузите данные:

```
import pandas as pd
df = pd.read_csv('Data/bearings.csv', index_col=0, parse_dates=[0])
df.head()
```

На рис. 6.19 приведены первые пять строк набора данных.

	Bearing 1	Bearing 2	Bearing 3	Bearing 4
2004-02-12 10:32:39	0.058333	0.071832	0.083242	0.043067
2004-02-12 10:42:39	0.058995	0.074006	0.084435	0.044541
2004-02-12 10:52:39	0.060236	0.074227	0.083926	0.044443
2004-02-12 11:02:39	0.061455	0.073844	0.084457	0.045081
2004-02-12 11:12:39	0.061361	0.075609	0.082837	0.045118

Рис. 6.19. Фрагмент набора данных для прогнозирования отказов подшипников (Bearing 1, ..., Bearing 4)

Набор данных хранит сведения о 984 образцах. Каждая выборка содержит данные о вибрации четырех подшипников, взятые с интервалом 10 минут.

Построим временной ряд данных вибрации для всех четырех подшипников:

```
%matplotlib inline
import matplotlib.pyplot as plt
import seaborn as sns
sns.set()
```

```
df.plot(figsize = (12, 6))
```

На рис. 6.20 приведен результат.

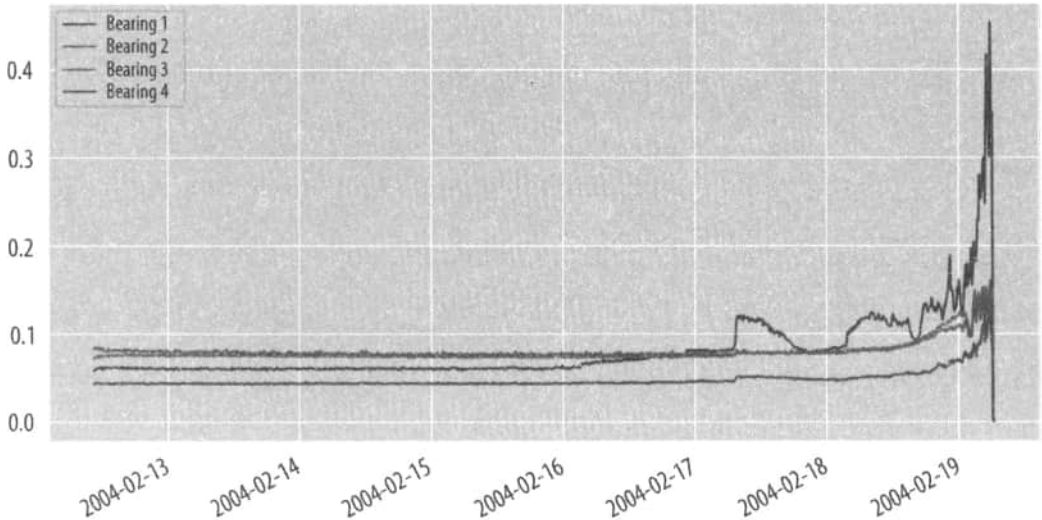


Рис. 6.20. Графики временных рядов данных вибрации для всех подшипников

Примерно через четыре дня испытаний вибрации в подшипнике 1 стали возрастать. Через день они резко возросли, а примерно через два дня после этого произошел катастрофический отказ подшипника 1. Наша цель — построить модель, распознающую повышенную вибрацию в любом подшипнике как признак приближающегося отказа, причем сделать это *без* набора помеченных данных.

Следующим шагом будет извлечение из набора данных (`x_train` в коде ниже) образцов, представляющих "нормальную" работу, и сведение четырех измерений к одному с помощью PCA — по сути, объединение данных по всем четырем подшипникам. Затем применим то же преобразование PCA к оставшейся части набора данных (`x_test`), объединим два частичных набора данных и построим график:

```
from sklearn.decomposition import PCA
```

```
x_train = df['2004-02-12 10:32:39':'2004-02-13 23:42:39']
```

```
x_test = df['2004-02-13 23:52:39':]
```

```
pca = PCA(n_components=1, random_state=0)
x_train_pca = pd.DataFrame(pca.fit_transform(x_train))
x_train_pca.index = x_train.index

x_test_pca = pd.DataFrame(pca.transform(x_test))
x_test_pca.index = x_test.index
df_pca = pd.concat([x_train_pca, x_test_pca])
df_pca.plot(figsize = (12, 6))
plt.legend().remove()
```

На рис. 6.21 приведен результат.

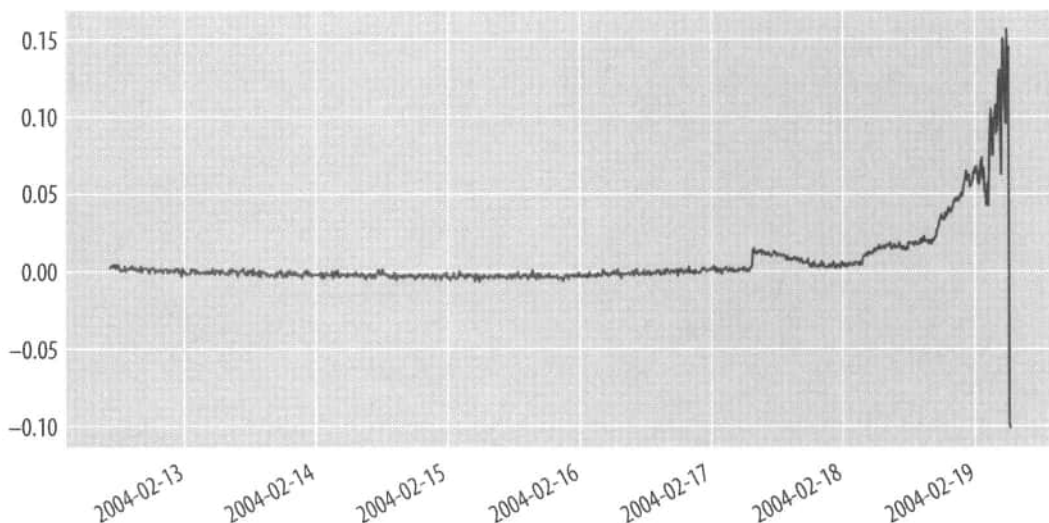


Рис. 6.21. График после PCA-преобразования

Теперь инвертируем преобразование PCA и построим график "восстановленного" набора данных:

```
df_restored = pd.DataFrame(pca.inverse_transform(df_pca), index=df_pca.index)
df_restored.plot(figsize = (12, 6))
```

Результаты выглядят так, как показано на рис. 6.22.

Очевидно, что при применении преобразования и его инвертировании были потеряны данные. Определим функцию, которая вычисляет потери в диапазоне выборок, затем применим эту функцию ко всем выборкам исходного и восстановленного наборов данных и построим график различий во времени:

```
import numpy as np
```

```
def get_anomaly_scores(df_original, df_restored):
    loss = np.sum((np.array(df_original) - np.array(df_restored)) ** 2, axis=1)
```

```
loss = pd.Series(data=loss, index=df_original.index)
return loss
```

```
scores = get_anomaly_scores(df, df_restored)
scores.plot(figsize = (12, 6))
```

На рис. 6.23 приведены результаты.

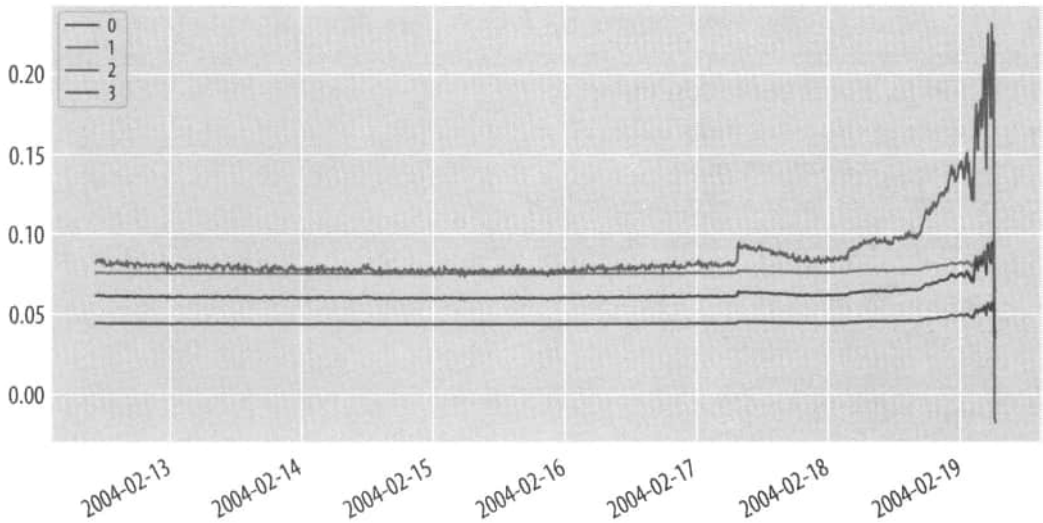


Рис. 6.22. Графики "восстановленного" набора данных после инвертирования PCA-преобразования

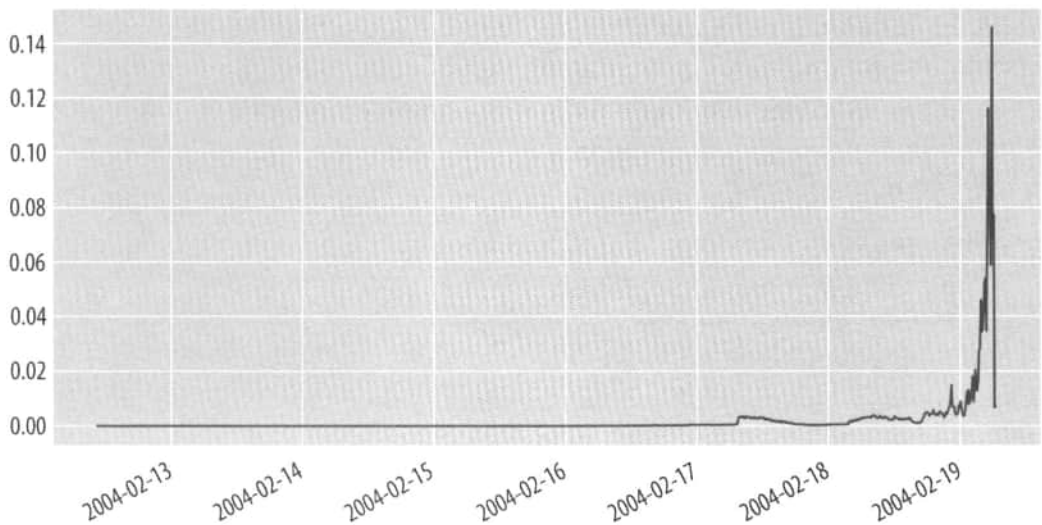


Рис. 6.23. График потерь данных

Потери очень малы, когда все четыре подшипника работают нормально, но они начинают возрастать, когда один или несколько подшипников испытывают превышающую норму вибрацию. Из графика видно, что когда потери превышают пороговое значение, равное примерно 0,002, это свидетельствует о возможном выходе подшипника из строя.

Теперь после выбора предварительного порогового значения потерь можно использовать его для обнаружения аномального поведения подшипников. Начните с определения функции, которая берет образец и возвращает True или False (что указывает на то, является ли образец отклоняющимся от нормы) путем применения и инвертирования преобразования PCA, измерения потерь для каждого подшипника и сравнения их с заданным порогом потерь:

```
def is_anomaly(row, pca, threshold):
    pca_row = pca.transform(row)
    restored_row = pca.inverse_transform(pca_row)
    losses = np.sum((row - restored_row) ** 2)

    for loss in losses:
        if loss > threshold:
            return True;
    return False
```

Примените функцию к началу временного ряда, которое представляет собой нормальное поведение, и убедитесь, что она возвращает значение False:

```
x = df.loc[['2004-02-16 22:52:39']]
is_anomaly(x, pca, 0.002)
```

Примените функцию к более позднему временному периоду, который представляет собой аномальное поведение, и убедитесь, что она возвращает значение True:

```
x = df.loc[['2004-02-18 22:52:39']]
is_anomaly(x, pca, 0.002)
```

Теперь примените функцию ко всем данным и закрасьте аномальные данные красным цветом, чтобы наглядно показать, когда обнаружено аномальное поведение:

```
df.plot(figsize = (12, 6))
```

```
for index, row in df.iterrows():
    if is_anomaly(pd.DataFrame([row]), pca, 0.002):
        plt.axvline(row.name, color='r', alpha=0.2)
```

На рис. 6.24 приведены результаты.

Повторите эту процедуру, но на этот раз используйте порог потерь не 0,002, а 0,0002:

```
df.plot(figsize = (12, 6))
for index, row in df.iterrows():
    if is_anomaly(pd.DataFrame([row]), pca, 0.0002):
        plt.axvline(row.name, color='r', alpha=0.2)
```

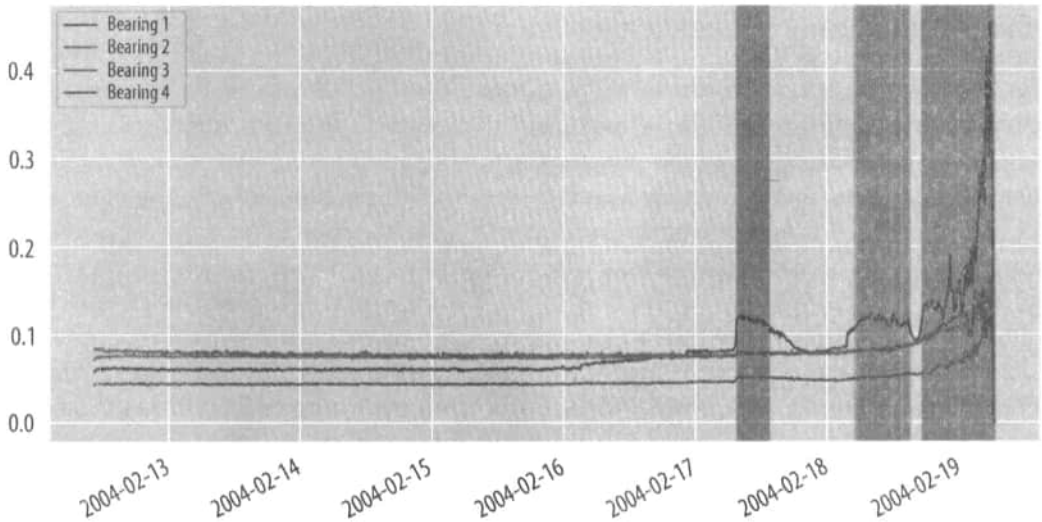


Рис. 6.24. Графики после применения функции обнаружения отклонений с порогом 0,002

На рис. 6.25 приведены результаты.

Чувствительность модели можно регулировать путем изменения порогового значения, используемого для обнаружения аномалий. Использование порога потерь 0,002 позволяет предсказать отказ подшипника примерно за два дня до его возникновения, а порог потерь 0,0002 — примерно за три дня. Как правило, необходимо выбрать такой порог потерь, который позволяет предсказать отказ как можно раньше, не вызывая ложных срабатываний.

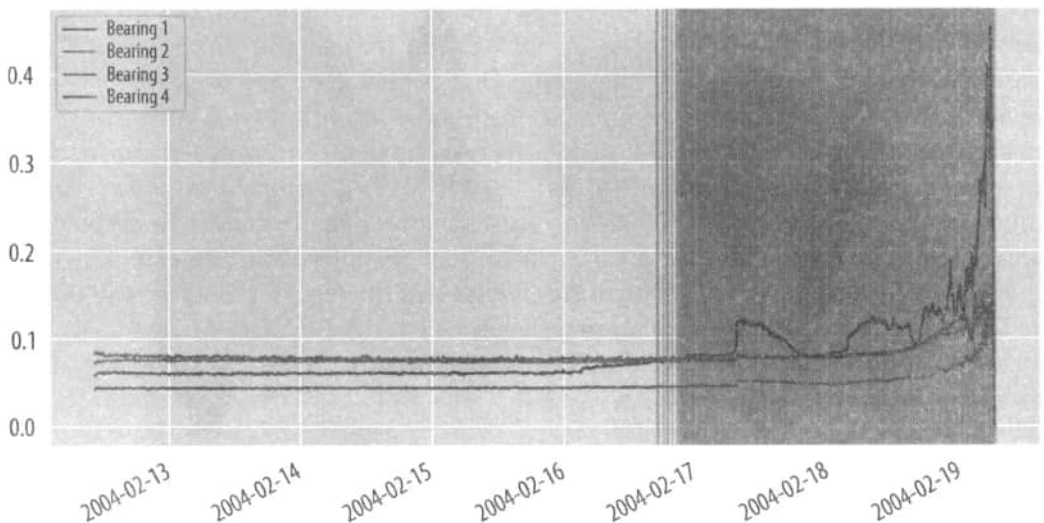


Рис. 6.25. Графики после применения функции обнаружения отклонений с порогом 0,0002

Обнаружение многомерных аномалий

Могли ли мы предсказать отказ в предыдущем примере, просто наблюдая за отдельными подшипниками? Возможно. Но что делать, если на приближающийся отказ указывают незначительно повышенные вибрации не одного, а двух подшипников? Инженеры часто сталкиваются с тем, что о приближающейся неисправности сигнализируют не отдельные датчики, а комбинация показаний нескольких датчиков. Эти показания могут поступать от датчиков разных типов: например, от датчиков температуры и манометров в автомобильной и аэрокосмической промышленности (<https://oreil.ly/18v1V>) или от кардиомониторов и датчиков артериального давления в здравоохранении (<https://oreil.ly/UoeIc>). Сокращение числа измерений до одного с помощью PCA — это попытка уловить взаимосвязь между данными, поступающими от отдельных датчиков, и рассматривать показания системно, что известно как *обнаружение многомерных аномалий*.

Одно из ограничений использования PCA для обнаружения аномалий в многомерных системах состоит в том, что, поскольку в нем используются линейные преобразования, PCA лучше моделирует линейные связи между переменными, чем нелинейные. Нейронные сети, напротив, прекрасно справляются с моделированием нелинейных данных. Это основная причина, по которой современные методы обнаружения многомерных аномалий обычно опираются на глубокое обучение.

С увеличением числа переменных возрастает и сложность моделирования взаимозависимостей между ними. Нередки случаи, когда общее состояние системы определяется десятками независимых друг от друга переменных. В сентябре 2020 года группа исследователей из Microsoft и Пекинского университета опубликовала работу "Обнаружение многомерных аномалий временных рядов с помощью графовой сети внимания" ("Multivariate Time-series Anomaly Detection via Graph Attention Network", <https://oreil.ly/T5qMi>), в которой была предложена новая архитектура для обнаружения многомерных аномалий. Она объединяет две модели глубокого обучения: одну, основанную на ошибке предсказания, и другую, основанную на ошибке восстановления. Microsoft использует эту архитектуру в своем сервисе Azure Multivariate Anomaly Detector (<https://oreil.ly/5ixTv>), который позволяет моделировать зависимости между 300 независимыми источниками данных и применяется такими компаниями, как Airbus и Siemens, для обнаружения нарушений в телеметрии космических станций (<https://oreil.ly/xO1nI>) и тестирования медицинских приборов перед их выпуском на рынок. Сервис Azure Multivariate Anomaly Detector является частью Azure Cognitive Services (<https://oreil.ly/rwRSv>), о котором мы поговорим в *главе 14*.

Резюме

Анализ главных компонент — это метод сокращения числа измерений в наборе данных без соразмерной потери информации. Он находит широкое применение в

машинном обучении, включая визуализацию высокоразмерных данных, анонимизацию данных, уменьшение шума и увеличение отношения строк к столбцам за счет уменьшения числа измерений. Он также может использоваться для обнаружения аномалий путем измерения потерь, возникающих при применении преобразования PCA и последующей инверсии. Аномальные выборки, как правило, несут большие потери.

Когда я провожу занятия, то часто называю PCA "самым сокровенным секретом машинного обучения". Он не должен оставаться секретом, поскольку это незаменимый инструмент в руках инженеров машинного обучения. Теперь, когда вы знаете о нем, я могу почти наверняка гарантировать, что вы найдете способы применить его на практике.

Операционализация моделей машинного обучения

Все модели машинного обучения, представленные до сих пор в этой книге, написаны на языке Python. Модели не обязательно *должны* быть разработаны на основе Python, но многие из них используют именно его, в том числе благодаря многочисленным библиотекам Python высочайшего уровня, включая Pandas (<https://oreil.ly/ISq91>) и Scikit-Learn (<https://oreil.ly/rg9GI>). ML-модели, написанные на Python, легко используются в приложениях на Python. Вызов их из других языков, таких как C++, Java и C#, требует немного больше усилий. Вы не можете просто вызвать функцию Python из C++, как если бы это была функция C++. Так как же вызывать модели, написанные на Python, из приложений, разработанных на других языках? Другими словами, как *операционализировать* (внедрить) модели Python, чтобы они могли использоваться в любом приложении на любой платформе, написанном на любом языке программирования?

Схема слева на рис. 7.1 показывает одну из стратегий: обернуть модель в веб-сервис и раскрыть ее функциональность через REST API (<https://oreil.ly/CEvxK>). Тогда любой клиент, способный сформировать HTTP(S)-запрос, сможет вызвать модель. Обернуть модель в веб-сервис довольно просто с помощью таких фреймворков Python, как Flask (<https://oreil.ly/x5vWh>). Веб-сервис можно разместить локально или в облаке, а для удобства развертывания его можно контейнеризировать с помощью таких инструментов, как Docker (<https://oreil.ly/wVDsr>).



Рис. 7.1. Архитектуры использования моделей Python в других языках

На рис. 7.1 справа показана еще одна стратегия — относительно новая, но быстро набирающая популярность. Она заключается в экспорте модели Python в формат

ONNX (Open Neural Network Exchange, <https://oreil.ly/qdwhs>), не зависящий от платформы, и последующем использовании среды выполнения ONNX для загрузки модели в Java, C++, C# и другие языки программирования. После загрузки модель можно вызвать через среду выполнения ONNX.

Конечно, если клиентское приложение и модель написаны на одном языке, то ни веб-сервис, ни ONNX не нужны. В этой главе я рассмотрю несколько сценариев:

- ◆ как сохранить обученную Python-модель и вызвать ее из Python-клиента;
- ◆ как вызвать Python-модель от не-Python-клиента с помощью веб-сервиса;
- ◆ как контейнеризировать Python-модель (и веб-сервис) для упрощения развертывания;
- ◆ как использовать ONNX для вызова Python-модели из других языков программирования;
- ◆ как писать модели машинного обучения на C#, а не на Python.

В завершение я продемонстрирую новый способ операционализации модели машинного обучения путем раскрытия ее функциональности через Microsoft Excel. Многое еще предстоит рассмотреть, так что давайте приступим.

Обращение к Python-модели из Python-клиента

Казалось бы, вызвать Python-модель из Python-клиента очень просто: достаточно вызвать команду `predict` (или, для классификатора, `predict_proba`) на модели. Конечно, не нужно заново обучать модель каждый раз при ее использовании. Можно обучить ее однократно, а затем дать возможность клиентскому приложению пересоздать модель в обученном состоянии. Для этого программисты на языке Python используют Python-модуль `pickle` (<https://oreil.ly/DgFfH>).

В качестве демонстрации ниже приведен код создания и обучения модели Titanic, рассмотренной в *главе 3*. Однако вместо того чтобы использовать модель для прогнозирования, он сохраняет ее в файле `.pkl` ("пиклинг"-модель) с вызовом `pickle.dump` в последней строке:

```
import pickle
import pandas as pd
from sklearn.linear_model import LogisticRegression

df = pd.read_csv('Data/titanic.csv')
df = df[['Survived', 'Age', 'Sex', 'Pclass']] # выживший, пол, возраст, класс каюты
df = pd.get_dummies(df, columns=['Sex', 'Pclass'])
df.dropna(inplace=True)

x = df.drop('Survived', axis=1)
y = df['Survived']
```

```
model = LogisticRegression(random_state=0)
model.fit(x, y)
```

```
pickle.dump(model, open('titanic.pkl', 'wb'))
```

Для вызова модели Python-клиент использует `pickle.load` для десериализации модели из файла `.pkl`, *пересоздает модель в обученном состоянии* и вызывает `predict_proba` для вычисления шансов на выживание пассажира:

```
import pickle
import pandas as pd

model = pickle.load(open('titanic.pkl', 'rb'))

female = pd.DataFrame({ 'Age': [30],                                # возраст
                        'Sex_female': [1], 'Sex_male': [0],        # пол: женский, мужской
                        'Pclass_1': [1], 'Pclass_2': [0], 'Pclass_3': [0] }) # класс
```

```
probability = model.predict_proba(female)[0][1]
print(f'Probability of survival: {probability:.1%}') # Вероятность выживания
```

Теперь клиент может использовать модель для прогнозирования без ее переобучения. А после загрузки модели она может сохраняться в течение всего времени существования клиента и вызываться для прогнозирования в любое время.

В *главе 5* была представлена функция `Scikit make_pipeline` (<https://oreil.ly/wiBqe>), которая позволяет объединять в общий блок, или конвейер, эстиматоры-оценщики (объекты, делающие прогнозы) и трансформеры (объекты, преобразующие данные, поступающие в модель). Модуль `pickle` также может использоваться для сериализации и десериализации конвейеров. В примере 7.1 рассмотренная в *главе 4* модель анализа настроений с помощью `make_pipeline` объединяет класс `CountVectorizer` (<https://oreil.ly/zZcAM>) для векторизации текста с объектом `LogisticRegression` (<https://oreil.ly/bqimV>) для классификации текста. Вызов `pickle.dump` сохраняет модель, `CountVectorizer` и все остальное.

Пример 7.1. Обучение и сохранение конвейера анализа настроений

```
import pickle
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline

df = pd.read_csv('Data/reviews.csv', encoding="ISO-8859-1")
df = df.drop_duplicates()

x = df['Text']          # текст
y = df['Sentiment']     # настроение
```

```
vectorizer = CountVectorizer(ngram_range=(1, 2), stop_words='english', min_df=20)
model = LogisticRegression(max_iter=1000, random_state=0)
pipe = make_pipeline(vectorizer, model)
pipe.fit(x, y)
```

```
pickle.dump(pipe, open('sentiment.pkl', 'wb'))
```

Python-клиент может десериализовать конвейер и вызвать `predict_proba` для оценки строки текста на предмет настроения с помощью нескольких простых строк кода:

```
import pickle
pipe = pickle.load(open('sentiment.pkl', 'rb'))
score = pipe.predict_proba(['Great food and excellent service!'])[0][1]
# Еда была отличной, а обслуживание - превосходным!
print(score)
```

Такой пиклинг работает не только с `CountVectorizer`, но и с другими преобразователями, например `StandardScaler`.



Пиклинг конвейера, содержащего `CountVectorizer`, может привести к созданию большого файла `.pkl`. В данном примере `sentiment.pkl` имеет размер 50 Мбайт, поскольку содержит весь словарь, построенный `CountVectorizer` на основе обучающего текста. Если убрать параметр `min_df=20`, то файл увеличится почти до 90 Мбайт.

Решением является замена `CountVectorizer` на `HashingVectorizer`, который не создает словарь, а использует хеши слов для индексации таблицы частот слов. Это позволяет уменьшить размер файла `sentiment.pkl` до 8 Мбайт без параметра `min_df`, который `HashingVectorizer` все равно не поддерживает.

Если вы хотите написать автономный Python-клиент, выполняющий анализ настроений, запустите код из примера 7.1 в блокноте Jupyter, чтобы сгенерировать файл `sentiment.pkl`. По желанию можете заменить `CountVectorizer` на `HashingVectorizer` и удалить параметр `min_df`, чтобы уменьшить размер файла `.pkl`. Затем создайте Python-скрипт с именем `sentiment.py`, содержащий следующий код:

```
import pickle, sys

# Получение текста для анализа
if len(sys.argv) > 1:
    text = sys.argv[1]
else:
    text = input('Text to analyze: ') # Текст для анализа

# Загрузка конвейера, содержащего модель и векторизатор
pipe = pickle.load(open('sentiment.pkl', 'rb'))
```


Передача входного текста в конвейер и вывод результата

```
score = pipe.predict_proba([text])[0][1]
print(score)
```

Скопируйте `sentiment.pkl` в тот же каталог, что и `sentiment.py`, а затем перейдите в командную строку и запустите сценарий:

```
python sentiment.py "Great food and excellent service!"
```

Результат должен выглядеть примерно так, как показано на рис. 7.2, и это доказательство того, что вам удалось воссоздать модель в обученном состоянии и вызвать ее для анализа входного текста на предмет анализа настроения.

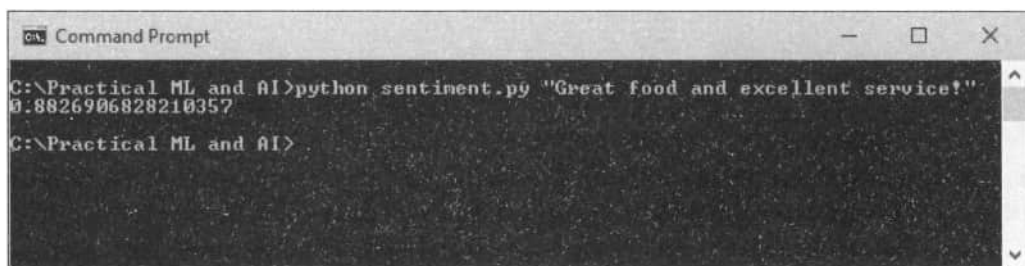


Рис. 7.2. Результат работы сценария

Обратите внимание, что оценка настроения будет несколько отличаться, если заменить `CountVectorizer` на `HashingVectorizer`, отчасти из-за отсутствия параметра `min_df`.

Версионирование файлов Pickle

Вообще говоря, модель, сохраненная в одной версии Scikit, не может быть распакована в другой версии. Время от времени это приводит к появлению предупреждающих сообщений, а иногда вообще возникает проблема с работой. Обязательно сохраняйте модели в той же версии Scikit, которая была использована для их создания. Это требует определенного планирования с инженерной точки зрения, поскольку если вы храните сериализованные модели в централизованном репозитории и обновляете версию Scikit, используемую в ваших приложениях, то вам придется обновить и сохраненные модели.

В связи с этим возникает несколько интересных вопросов. Как организовать репозиторий для моделей машинного обучения? Как развернуть модели из репозитория на устройствах, где они хранятся? И, наконец, как *версионировать* эти модели, а также наборы данных, на которых они обучаются?

Ответы на эти вопросы дает зарождающаяся область, известная как MLOps, что является сокращением от ML operations (ML-операции). Я не рассматриваю MLOps здесь, поскольку это обширная тема, о которой можно написать отдельную книгу. Если вы хотите узнать больше, рекомендую прочитать книгу "Практический MLOps: операционализация моделей машинного обучения" Ноя Гифта и Альфредо

Дезы (Gift Noah, Deza Alfredo. Practical MLOps: Operationalizing Machine Learning Models; <https://oreil.ly/bT3FE>).

Обращение к Python-модели из C#-клиента

Предположим, что вы хотите вызвать описанную в предыдущем разделе модель анализа настроений из приложения, созданного на другом языке, например на C#. Не получится напрямую вызвать функцию Python из C#, но вы можете обернуть модель Python в веб-сервис и открыть ее метод `predict` (или `predict_proba`) с помощью REST API. Одним из способов создания веб-сервиса является использование Flask — популярного фреймворка для создания сайтов и веб-сервисов на Python.

Попробуем проверить это, но сперва убедитесь, что Flask установлен на вашем компьютере. Затем создайте файл с именем `app.py` и поместите в него следующий код. Этот код использует Flask для реализации написанного на Python веб-сервиса, который прослушивает порт 5000:

```
import pickle
from flask import Flask, request

app = Flask(__name__)
pipe = pickle.load(open('sentiment.pkl', 'rb'))

@app.route('/analyze', methods=['GET'])
def analyze():
    if 'text' in request.args:
        text = request.args.get('text')
    else:
        return 'No string to analyze'

    score = pipe.predict_proba([text])[0][1]
    return str(score)

if __name__ == '__main__':
    app.run(debug=True, port=5000, host='0.0.0.0')
```

При запуске сервис десериализует конвейер, содержащий модель анализа настроений в файле `sentiment.pkl`. Оператор `@app.route`, сопровождающий функцию `analyze`, указывает Flask на вызов этой функции при вызове метода `analyze` сервиса. Если сервис размещен локально, то следующий запрос вызывает функцию `analyze` и возвращает строку, содержащую оценку настроения для текста в строке запроса:

`http://localhost:5000/analyze?text=Great food and excellent service!`

Для наглядности перейдите в каталог, где расположен файл `app.py` (убедитесь, что `sentiment.pkl` находится там же), и запустите Flask, набрав:

```
flask run
```

Затем перейдите в командную строку и с помощью команды `curl` выполните запрос к URL-адресу:

```
curl -G -w "%n" http://localhost:5000/analyze --data-urlencode "text=Great food and excellent service!"
```

На рис. 7.3 приведен результат.

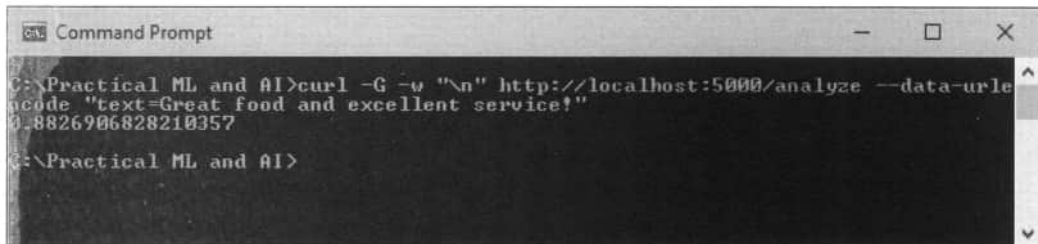


Рис. 7.3. Результат запроса к URL-адресу

Если на вашем компьютере установлена среда разработки Visual Studio или Visual Studio Code и настроены компиляция и запуск приложений на языке C#, можно использовать следующий код консольного приложения на C# для вызова веб-сервиса и оценки текстовой строки. Разумеется, нет ограничений на вызов веб-сервиса (и, соответственно, модели) только из C#. Для этого подойдет любой язык, поскольку практически все современные языки программирования предоставляют средства для отправки HTTP-запросов.

```
using System;
```

```
using System.Net.Http;
```

```
using System.Threading.Tasks;
```

```
class Program
```

```
{
    static async Task Main(string[] args)
    {
        string text;

        // Получение текста для анализа
        if (args.Length > 0)
        {
            text = args[0];
        }
        else
        {
            Console.Write("Text to analyze: "); // Анализируемый текст
            text = Console.ReadLine();
        }
    }
}
```

```

// Передача текста веб-сервису
var client = new HttpClient();
var url = $"http://localhost:5000/analyze?text={text}";
var response = await client.GetAsync(url);
var score = await response.Content.ReadAsStringAsync();

// Отображение оценки настроения
Console.WriteLine(score);
}
}

```

Этот веб-сервис представляет собой простую службу, которая считывает входные данные из строки запроса и возвращает строку. Для более сложного ввода и вывода можно сериализовать входные данные в JSON и передать их в теле запроса HTTP POST, а в ответе вернуть полезную информацию в формате JSON. Руководство, демонстрирующее, как это сделать на языке Python, приведено в статье "Python Post JSON Using Requests Library" (<https://oreil.ly/rOJcF>).

Сохранение модели машинного обучения в контейнере

Недостатком обертыивания модели машинного обучения в веб-сервис и ее локального запуска является то, что на компьютере клиента должен быть установлен Python, а также все пакеты, необходимые для работы модели и веб-сервиса. Альтернативным вариантом является размещение веб-сервиса в облаке, где с ним можно работать через Интернет. Нет ничего сложного в том, чтобы зайти в Azure или AWS, создать виртуальную машину (virtual machine, VM) и установить туда программное обеспечение. Но есть и более эффективный способ — контейнеры.

Контейнеры (<https://oreil.ly/7AKWf>) произвели революцию в способах создания и развертывания программного обеспечения. Контейнер включает в себя приложение и все необходимое для его работы, в том числе среду выполнения (например, Python), пакеты, на которые опирается приложение, и даже виртуальную файловую систему. Если вы не знакомы с контейнерами, то считайте, что это легкие виртуальные машины, которые быстро запускаются и потребляют гораздо меньше памяти. Самой популярной контейнерной платформой в мире является Docker (<https://oreil.ly/w2mU6>), хотя ее стремительно теснит Kubernetes (<https://oreil.ly/IRpur>).

Контейнеры создаются из *образов контейнеров*, которые служат "чертежами" для контейнеров подобно тому, как классы в объектно-ориентированных языках программирования являются "чертежами" для объектов. Первым шагом в создании образа контейнера Docker, содержащего модель анализа настроений и веб-сервис,

является создание файла с именем Dockerfile (без расширения имени файла) в том же каталоге, что и app.py и sentiment.pkl, а затем размещение в нем следующих операторов:

```
FROM python:3.8
RUN pip install flask numpy scipy scikit-learn && \
    mkdir /app
COPY app.py /app
COPY sentiment.pkl /app
WORKDIR /app
EXPOSE 5000
ENTRYPOINT ["python"]
CMD ["app.py"]
```

Dockerfile содержит инструкции по созданию образа контейнера. В данном случае создается образ контейнера, включающий среду выполнения Python, несколько пакетов Python, таких как Flask и Scikit-Learn, а также файлы app.py и sentiment.pkl. Он также указывает среде выполнения Docker, в которой размещается контейнер, открыть порт 5000 для HTTP-запросов и запускать app.py при запуске контейнера.

Существует несколько способов создания образа контейнера из Dockerfile. Если на компьютере установлена платформа Docker, то можно воспользоваться командой `docker build`:

```
docker build -t sentiment-server .
```

В качестве альтернативы можно загрузить Dockerfile в облачную платформу, например Microsoft Azure, и собрать его там. Это избавит вас от необходимости устанавливать Docker на локальной машине и позволит легко хранить полученный образ контейнера в облаке. Образы контейнеров хранятся в *реестрах контейнеров*, а современные облачные платформы содержат как реестры контейнеров, так и сами контейнеры. Если запустить экземпляр контейнера в Azure, то веб-сервис в контейнере можно вызвать с помощью URL-адреса, подобного этому:

```
http://wintellect.northcentralus.azurecontainer.io:5000/analyze?text=Great food and excellent service!
```

Одним из преимуществ размещения экземпляра контейнера в облаке является то, что к нему можно обратиться из любого клиентского приложения, работающего на любой машине и под управлением любой операционной системой, а на компьютере, где размещено приложение, не нужно устанавливать ничего специального. Контейнеры могут быть полезны даже в том случае, если веб-сервис размещается не в облаке, а локально. Если на локальной машине развернут контейнерный стек, например среда выполнения Docker, то не нужно устанавливать Python и все необходимые веб-сервису пакеты. Достаточно запустить экземпляр контейнера и направить к нему HTTP-запросы через localhost.

Использование ONNX для преодоления языковой лакуны

Можно ли преодолеть разрыв между клиентом на C# и ML-моделью на Python без использования веб-сервиса в качестве посредника? Да! Решение кроется в аббревиатуре из четырех букв: *ONNX*. Как уже упоминалось, *ONNX* расшифровывается как *Open Neural Network Exchange*, и изначально он был разработан для того, чтобы модели глубокого обучения, написанные на одном фреймворке, например *TensorFlow*, можно было использовать с другими фреймворками, такими как *PyTorch*. Но сегодня его можно применять и для моделей *Scikit*. Подробнее об *ONNX* я расскажу в *главе 12*, а пока давайте воспользуемся им для вызова Python-модели непосредственно из приложения, написанного на C#, — и веб-сервис не потребуется.

Первым шагом в использовании *ONNX* для преодоления языкового разрыва является установка пакета *Sklearn2onnx* (<https://oreil.ly/LIOiY>) в среде Python. Затем используйте метод *convert_sklearn* этого пакета для сохранения обученной модели *Scikit* в файл *.onnx*. Приведем небольшой фрагмент кода, который сохраняет модель анализа настроений из примера 7.1 в файл с именем *sentiment.onnx*. Параметр *initial_types* указывает, что модель ожидает одно входное значение: строку с текстом, который нужно оценить на предмет смысловой нагрузки:

```
from sklearn2onnx import convert_sklearn
from sklearn2onnx.common.data_types import StringTensorType

initial_type = [('string_input', StringTensorType([None, 1]))]
onnx = convert_sklearn(pipe, initial_types=initial_type)
```

```
with open('sentiment.onnx', 'wb') as f:
    f.write(onnx.SerializeToString())
```

Если вы хотите использовать модель из Python, то сначала установите пакет Python под названием *Onnxruntime* (<https://oreil.ly/aEax9>), содержащий среду выполнения *ONNX* (также известную как *ORT*). Она обеспечивает поддержку загрузки и запуска моделей *ONNX*. Затем вызовите метод *InferenceSession* среды выполнения с указанием пути к файлу *.onnx* для десериализации модели, а далее осуществите вызов *run* на возвращенном объекте сессии для вызова метода *predict* или *predict_proba* модели. Вот как это выглядит в коде:

```
import numpy as np
import onnxruntime as rt

session = rt.InferenceSession('sentiment.onnx')
input_name = session.get_inputs()[0].name
label_name = session.get_outputs()[1].name # 0 = predict, 1 = predict_proba
input = np.array('Great food and excellent service!').reshape(1, -1)
```

Еда была отличной, а обслуживание – превосходным!

```
score = session.run([label_name], { input_name: input })[0][0][1]
print(score)
```

Обратите внимание, что входящая в модель строка передается в виде массива NumPy. Возвращаемое методом `run` значение показывает оценку настройки, возвращаемую `predict_proba`. Если вы предпочитаете вызывать `predict` для прогнозирования класса, измените `session.get_outputs()[1].name` на `session.get_outputs()[0].name`.



В повседневной жизни вы не будете загружать модель каждый раз при ее вызове. Вы загружаете ее один раз, позволяете ей сохраняться в течение всего времени жизни клиента и вызываете ее всякий раз, когда хотите выполнить прогноз.

Разумеется, весь смысл этого разбора заключается в том, чтобы вызывать модель из C#, а не из Python. Это тоже несложно, благодаря пакету NuGet от Microsoft под названием `Microsoft.ML.OnnxRuntime` (<https://oreil.ly/UYHqr>). Его можно установить из командной строки или с помощью интегрированной среды разработки, например Visual Studio. Затем достаточно просто написать консольное приложение на C#, которое воссоздает обученную модель анализа настроений из файла `.onnx` и вызывает ее для оценки текстовой строки:

```
using System;
using Microsoft.ML.OnnxRuntime;
using Microsoft.ML.OnnxRuntime.Tensors;
using System.Collections.Generic;
using System.Linq;
class Program
{
    static void Main(string[] args)
    {
        string text;

        // Получение текста для анализа
        if (args.Length > 0)
        {
            text = args[0];
        }
        else
        {
            Console.WriteLine("Text to analyze: "); // Текст для анализа
            text = Console.ReadLine();
        }
    }
}
```

```

// Создание модели и передача в нее текста
var tensor = new DenseTensor<string>(new string[]
    { text }, new int[] { 1, 1 });

var input = new List<NamedOnnxValue>
{
    NamedOnnxValue.CreateFromTensor<string>("string_input", tensor)
};

var session = new InferenceSession("sentiment.onnx");
var output = session.Run(input)
    .ToList().Last().AsEnumerable<NamedOnnxValue>();
var score = output.First().AsDictionary<Int64, float>()[1];

// Отображение оценки настроения
Console.WriteLine(score);
}
}

```

В данном коде предполагается, что файл `sentiment.onnx` находится в текущем каталоге. Инстанцирование объекта `InferenceSession` создает модель, а выполнение команды `Run` на этом объекте вызывает модель. "Под капотом" `Run` вызывает методы модели `predict` и `predict_proba` и делает доступными результаты обоих методов. Приходится немного потрудиться, чтобы преобразовать типы C# в типы ONNX и наоборот, но если разобраться, то можно вызвать модель машинного обучения, написанную на Python, прямо из C#.

Приведем еще один пример. Предположим, вы хотите использовать на языке C# регрессионную модель для расчета стоимости проезда на такси, описанную в главе 2. Напомним, что модель принимает на вход три значения с плавающей точкой — день недели (0–6), час суток (0–23) и расстояние в милях — и выдает прогнозируемую стоимость проезда на такси. Следующий код на языке Python сохраняет обученную модель в файле ONNX:

```

from skl2onnx import convert_sklearn
from skl2onnx.common.data_types import FloatTensorType

initial_type = [('float_input', FloatTensorType([None, 3]))]
onnx = convert_sklearn(model, initial_types=initial_type)

with open('taxi.onnx', 'wb') as f:
    f.write(onnx.SerializeToString())

```


Приведенный ниже код на языке C# загружает модель и делает прогноз. Существенное отличие этого примера от предыдущего заключается в том, что модели передается не строка, а массив из трех значений с плавающей точкой:

```
// Упаковка входных данных
var input = new float[] {
    4.0f,    // День недели
    17.0f,   // Время встречи пассажира (час суток)
    2.0f     // Расстояние до места
};

var tensor = new DenseTensor<float>(input, new int[] { 1, 3 });

// Создание модели и передача ей входных данных
var session = new InferenceSession("taxi.onnx");

var output = session.Run(new List<NamedOnnxValue>
{
    NamedOnnxValue.CreateFromTensor("float_input", tensor)
});

var score = output.First().AsTensor<float>().First();

// Демонстрация прогнозируемого тарифа
Console.WriteLine($"{score:#.00}");
```

Так что же быстрее: вызов обернутой в веб-сервис модели машинного обучения или вызов той же модели с помощью ONNX? Для ответа на этот вопрос я написал простой тестовый набор. На моем компьютере вызов модели анализа настроений в локально запущенном веб-сервисе Flask потребовал чуть более 2 секунд на один цикл. Вызов той же модели через среду выполнения Python ONNX занял в среднем 0,001 секунды. Это разница более чем на три порядка. А если веб-сервис размещается на удаленном сервере, то возникает дополнительная задержка.

Важно отметить, что ONNX не ограничивается только C# и Python. В настоящее время доступны среды исполнения ONNX для Python, C, C++, C#, Java, JavaScript и Objective-C, которые работают под управлением Windows, Linux, macOS, Android и iOS. Когда речь заходит о проецировании моделей машинного обучения, написанных на Python, на другие платформы и языки, ONNX становится лучшим решением. Более подробную информацию можно найти на сайте среды исполнения ONNX (<https://oreil.ly/mnSO5>).

Построение моделей машинного обучения на C# с помощью ML.NET

Scikit-Learn — это, пожалуй, самый популярный в мире фреймворк машинного обучения. Эффективность этой библиотеки, сопровождающая ее документация, а также тот факт, что она окружена большим количеством людей, являются основными причинами того, что на Python написано больше ML-моделей, чем на любом другом языке. Однако Scikit не единственный фреймворк машинного обучения. Существуют и другие фреймворки для других языков, и если вы в состоянии написать модель машинного обучения на том же языке программирования, что и использующий ее клиент, то сможете избежать лишних хлопот по ее операционализации.

ML.NET (<https://oreil.ly/YhliZ>) — это бесплатная кроссплатформенная библиотека машинного обучения с открытым исходным кодом от Microsoft для разработчиков .NET. Она делает почти все то же, что и Scikit, и кое-что из того, что Scikit не делает. И когда речь заходит о написании ML/AI-решений на C#, лучшего инструмента для этой работы не найти.

ML.NET основана на внутренней библиотеке, разработанной Microsoft и используемой в продуктах Microsoft уже более десяти лет. Реализуемые в ней алгоритмы машинного обучения были опробованы в реальных условиях и настроены на оптимальные производительность и точность. Поскольку ML.NET создана на основе C#, вы получаете все преимущества компилируемого языка программирования, включая безопасность типов и быстрое выполнение.

В документе, опубликованном командой ML.NET в Microsoft в 2019 году (<https://oreil.ly/ixrrY>), рассматриваются мотивы и цели разработки ML.NET. В нем также проводится сравнение точности и производительности ML.NET с Scikit-Learn и другим фреймворком машинного обучения — H2O (<https://oreil.ly/vDbtD>). На базе 9 Гбайт набора данных отзывов Amazon ML.NET обучила модель анализа настроений с точностью до 95%. Ни Scikit, ни H2O не смогли обработать этот набор данных из-за его размера. Когда все три фреймворка были обучены на 10% набора данных, ML.NET достигла наивысшей точности, обучаясь в 6 раз быстрее, чем Scikit, и почти в 10 раз быстрее, чем H2O.

ML.NET совместима с Windows, Linux и macOS. Благодаря инновационной разработке под названием *IDataView* она может оперировать наборами данных практически неограниченного размера. Хотя с ее помощью нельзя создавать нейронные сети с нуля, в ней присутствует возможность загрузки существующих нейронных сетей и использования техники, называемой *трансферным обучением* (transfer learning), для перепрофилирования этих сетей с целью решения специфических задач в конкретной области. (Трансферное обучение будет рассмотрено в *главе 10*.) В ней также реализована поддержка ONNX, позволяющая загружать сложные модели, в том числе модели глубокого обучения, сохраненные в формате ONNX. Наконец, она может быть использована в Python и даже совмещена со Scikit-Learn с помощью связующего модуля для Python под названием NimbusML (<https://oreil.ly/CsG6N>).

Если вы являетесь разработчиком .NET и интересуетесь машинным обучением, то сейчас как никогда подходящее время для знакомства с ML.NET. Этот раздел не претендует на исчерпывающее описание ML.NET, а призван познакомить вас с ней, показать основы построения с ее помощью моделей машинного обучения и, надеюсь, разжечь ваш интерес настолько, чтобы побудить вас к дальнейшему изучению. В Интернете можно найти множество замечательных ресурсов, включая официальную документацию по ML.NET (<https://oreil.ly/ay04E>), репозиторий GitHub с примерами ML.NET (<https://oreil.ly/IjAw7>) и книгу рецептов по ML.NET (<https://oreil.ly/06YMb>).

Анализ настроек с помощью ML.NET

Следующий код на языке C# использует ML.NET для построения и обучения модели анализа настроек. Он эквивалентен реализации на языке Python в примере 7.1:

```
var context = new MLContext(seed: 0);

// Загрузка данных
var data = context.Data.LoadFromTextFile<Input>("reviews.csv",
    hasHeader: true, separatorChar: ',', allowQuoting: true);

// Разделение данных на обучающий и тестовый наборы
var trainTestData = context.Data.TrainTestSplit(data,
    testFraction: 0.2, seed: 0);
var trainData = trainTestData.TrainSet;
var testData = trainTestData.TestSet;

// Построение и обучение модели
var pipeline = context.Transforms.Text.FeaturizeText
    (outputColumnName: "Features", inputColumnName: "Text")
    .Append(context.BinaryClassification.Trainers
        .SdcaLogisticRegression());

var model = pipeline.Fit(trainData);

// Оценка модели
var predictions = model.Transform(testData);
var metrics = context.BinaryClassification.Evaluate(predictions);
Console.WriteLine($"AUC: {metrics.AreaUnderPrecisionRecallCurve:P2}");

// Оценка настройки строки текста
var predictor = context.Model.CreatePredictionEngine<Input, Output>(model);
var input = new Input { Text = "Among the best movies I have ever seen";
```

```
// Текст - "Один из лучших просмотренных фильмов"
```

```
var prediction = predictor.Predict(input);
```

```
// Отображение оценки
```

```
Console.WriteLine($"Sentiment score: {prediction.Probability}");
```

Каждое приложение ML.NET начинается с создания экземпляра класса `MLContext` (<https://oreil.ly/1qkYh>). Необязательный параметр `seed` (<https://oreil.ly/CsKBJ>) инициализирует используемый ML.NET генератор случайных чисел, что позволяет получать повторяющиеся результаты от одного запуска к другому. `MLContext` раскрывает ряд свойств, через которые осуществляется доступ к основным частям API ML.NET. Одним из примеров является вызов `LoadFromTextFile` (<https://oreil.ly/zzzIB>), представляющий собой метод класса `DataOperationsCatalog` (<https://oreil.ly/EkbKU>), доступ к которому осуществляется через свойство `Data` (<https://oreil.ly/rsAOo>) класса `MLContext`.

`LoadFromTextFile` — это один из нескольких методов ML.NET, предназначенных для загрузки данных из текстовых файлов, баз и других источников данных. Он возвращает *представление данных* (data view), являющееся реализующим интерфейс `IDataView` (<https://oreil.ly/Q0k2S>) объектом. Представления данных в ML.NET похожи на фреймы данных в Pandas с одним важным отличием: если фреймы данных должны помещаться в памяти, то представления данных — нет. Внутри представления данных для доступа к данным используется SQL-подобный курсор. Это означает, что теоретически может быть обернут неограниченный объем данных. Именно поэтому ML.NET смогла обработать весь набор данных Amazon объемом 9 Гбайт, а Scikit и H2O — нет.

После загрузки данных и разделения их на обучающие и тестовые в предыдущем коде создается конвейер, содержащий объект `TextFeaturizingEstimator` (<https://oreil.ly/D2EqI>), созданный с помощью метода `FeaturizeText` (<https://oreil.ly/pnKic>), и объект `SdcaLogisticRegressionBinaryTrainer` (<https://oreil.ly/6yMxg>), созданный с помощью метода `SdcaLogisticRegression` (<https://oreil.ly/4qn9u>). В Scikit это аналогично созданию конвейера, содержащего объект `CountVectorizer` для векторизации входного текста и объект `LogisticRegression` для подгонки модели к данным. Вызов `Fit` на конвейере обучает модель, как и вызов `fit` в Scikit. Не случайно в ML.NET используются те же шаблоны, что и в Scikit. Это было сделано специально, чтобы создать ощущение узнаваемости для программистов, использующих Scikit.

После оценки точности модели путем вычисления площади под кривой "точность — полнота" вызов `ModelOperationsCatalog.CreatePredictionEngine` (<https://oreil.ly/nFQoT>) создает механизм прогноза, чей метод `Predict` (<https://oreil.ly/HOGQp>) выполняет прогнозирование. В отличие от Scikit, в которой метод `predict` вызывается на самом эстиматоре (оценщике), в ML.NET возможность прогнозирования заключена в отдельном объекте, отчасти для того, чтобы можно было создать несколько механизмов прогнозирования для достижения масштабируемости в сценариях с высокой интенсивностью использования.

В данном примере `Predict` принимает на вход объект `Input` и возвращает объект `Output`. Одним из преимуществ построения моделей с помощью `ML.NET` является сильная типизация. `LoadFromTextFile` — это общий метод, принимающий в качестве параметра типа имя класса — в данном случае `Input`. Аналогично, `CreatePredictionEngine` использует параметры типа для задания схем ввода и вывода. Классы `Input` и `Output` зависят от конкретного приложения и в данном случае определяются следующим образом:

```
public class Input
{
    [LoadColumn(0)]
    public string Text;

    [LoadColumn(1), ColumnName("Label")]
    public bool Sentiment;
}

public class Output
{
    [ColumnName("PredictedLabel")]
    public bool Prediction { get; set; }

    public float Probability { get; set; }
}
```

Атрибуты `LoadColumn` (<https://oreil.ly/d32Bc>) связывают столбцы файла данных со свойствами класса `Input`. Они указывают `ML.NET`, что значения для поля `Text` берутся из столбца 0 входного файла, а значения для поля настроения `Sentiment` (1 и 0, указывающие на то, является ли настроение, выраженное в тексте, положительным или отрицательным) — из столбца 1. Атрибут `ColumnName("Label")` определяет второй столбец как столбец метки — столбец со значениями, которые будет пытаться предсказать модель.

Класс `Output` определяет схему вывода. В данном примере он содержит свойства `Prediction` и `Probability`, которые после прогнозирования содержат метку прогноза (0 или 1) и вероятность того, что образец принадлежит к положительному классу, что удваивает оценку настроения. Атрибут `ColumnName("PredictedLabel")` связывает возвращаемое `Predict` значение со свойством `Prediction` объекта `Output`.



В названиях классов `Input` и `Output` нет ничего магического. Можно назвать их `SentimentData` и `SentimentPrediction`, и код будет работать точно так же.

Сохранение и загрузка моделей ML.NET

Ранее мы познакомились с тем, как использовать модуль `pickle` в Python для сохранения и загрузки обученных моделей. То же самое можно сделать и в ML.NET, вызвав `ModelOperationsCatalog.Save` (<https://oreil.ly/dIGXl>) и `ModelOperationsCatalog.Load` (<https://oreil.ly/PKo3g>) через свойство `Model` (<https://oreil.ly/yp9JL>) объекта `MLContext`:

```
// Сохранение обученной модели в ZIP-файл
context.Model.Save(model, data.Schema, "model.zip");
// Загрузка обученной модели из ZIP-файла
var model = context.Model.Load("model.zip", out DataViewSchema schema);
```

Это позволяет клиентам пересоздавать модель в обученном состоянии и использовать ее для прогнозирования без необходимости повторного обучения модели.

Добавление возможностей машинного обучения в Excel

Хотите узнать еще один способ практической реализации модели машинного обучения? Представьте, что вы — разработчик программного обеспечения в занимающейся арендой жилья интернет-компании. Отдел по связям с общественностью компании попросил вас создать электронную таблицу, позволяющую анализировать текст на предмет настроений. Идея заключается в том, что если в социальных сетях возникнет негативное отношение к компании, то отдел по связям с общественностью сможет вовремя среагировать.

Вы уже знаете, как построить, обучить и сохранить модель анализа настроений в Python с помощью Scikit. Excel поддерживает функции, определяемые пользователем (user-defined functions, UDF; <https://oreil.ly/8Pv4R>), которые позволяют пользователям создавать собственные функции, вызываемые так же, как `SUM()`, `AVG()` и другие встроенные в Excel функции. Но UDF написаны на языке Visual Basic for Applications (VBA, <https://oreil.ly/cJt6d>), а не на Python. Для того чтобы объединить Scikit с Excel, необходимо программировать UDF на Python.

К счастью, существуют библиотеки, позволяющие это сделать. Одна из них — `Xlwings` (<https://oreil.ly/uSmSL>), библиотека с открытым исходным кодом, сочетающая в себе мощь Excel и универсальность Python. С ее помощью можно писать Python-код, загружающий или создающий таблицы Excel и манипулирующий их содержимым, создавать Python-макросы, запускаемые нажатием кнопок в Excel, получать доступ к таблицам Excel из блокнотов Jupyter и многое другое. Кроме того, с помощью `Xlwings` можно разрабатывать UDF-файлы на языке Python для Excel под Windows.

Первым шагом в создании электронной таблицы, необходимой команде по связям с общественностью, является изменение конфигурации Excel для обеспечения доверия к надстройкам VBA. Запустите Microsoft Excel и с помощью команды **File** → **Options** (Файл → Параметры) откройте диалоговое окно **Excel Options** (Параметры

Excel). В списке разделов слева выберите пункт **Trust Center** (Центр управления безопасностью), а затем нажмите кнопку **Trust Center Settings** (Параметры центра управления безопасностью). Слева выберите раздел **Macro Settings** (Параметры макросов) и установите флажок **Trust access to the VBA project object model** (Доверять доступ к объектной модели проекта VBA), как показано на рис. 7.4. Нажмите кнопку **OK**, чтобы закрыть диалог **Trust Center** (Центр управления безопасностью), а затем **OK**, чтобы закрыть диалог **Excel Options** (Параметры Excel).

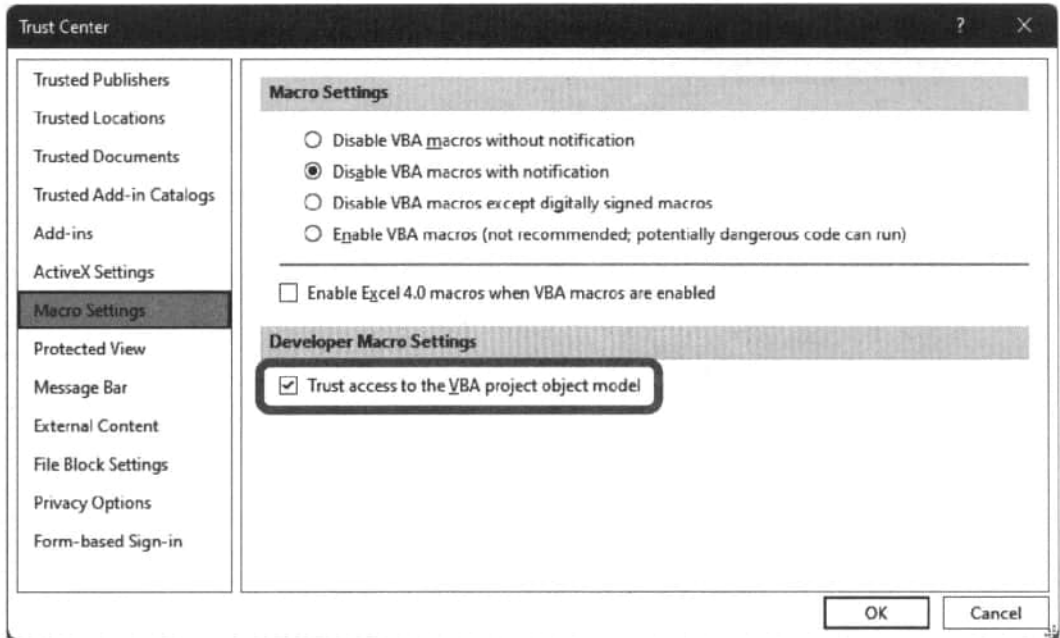


Рис. 7.4. Настройка Excel на доверие к надстройкам VBA

Следующим шагом будет установка Xlwings на компьютер с помощью команды `pip install xlwings` или эквивалентной команды для вашей среды Python. После этого перейдите в командную строку и выполните следующую команду для установки надстройки Xlwings в Excel:

```
xlwings addin install
```

Теперь создайте на компьютере каталог проекта и перейдите в него по команде `cd`. Затем выполните следующую команду:

```
xlwings quickstart sentiment
```

Эта команда создает в текущем каталоге подкаталог `sentiment` и помещает в него пару файлов: электронную таблицу `sentiment.xlsm` и файл на языке Python `sentiment.py`. Именно в последнем из них будет написана UDF, анализирующая текст на предмет смысловой нагрузки.

Далее скопируйте файл `sentiment.pkl` в каталог `sentiment`. (Если этот файл не был создан ранее, запустите код из примера 7.1 в блокноте Jupyter, чтобы сгенерировать

его сейчас.) Затем откройте файл `sentiment.py` в используемом текстовом редакторе и замените его содержимое кодом, приведенным далее. Этот код загружает модель анализа настроений из файла `.pkl` и сохраняет ссылку на нее в переменной с именем `model`. Затем определяется UDF с именем `analyze_text`, которая может быть вызвана из Excel. `analyze_text` векторизует переданный ей текст и отправляет его в метод `predict_proba` модели. Этот метод возвращает число от 0,0 до 1,0, причем 0,0 означает негативное настроение, а 1,0 — позитивное. После завершения работы сохраните изменения в файле `sentiment.py`. UDF написана. Теперь пришло время вызвать ее из Excel.

```
import pickle, os
import xlwings as xw

# Загрузка модели и словаря и создание векторизатора CountVectorizer
model_path = os.path.abspath(os.path.join(os.path.dirname(__file__), 'sentiment.pkl'))

model = pickle.load(open(model_path, 'rb'))

@xw.func
def analyze_text(text):
    score = model.predict_proba([text])[0][1]
    return score
```

Элегантность Xlwings заключается в том, что, написав UDF, например `analyze_text`, вы можете вызывать ее так же, как и функции, встроенные в Excel. Но сначала необходимо импортировать UDF. Для этого откройте файл `sentiment.xlsm` в Excel. Перейдите на вкладку **xlwings** и нажмите кнопку **Import Functions**, чтобы импортировать функцию `analyze_text` (рис. 7.5). Excel не уведомляет об успешном импорте, но сообщает о неудачном импорте — например, если вы забыли скопировать файл `sentiment.pkl` в каталог, где хранится файл `sentiment.py`.

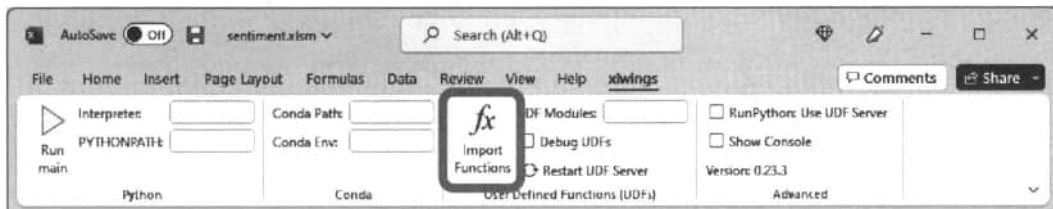


Рис. 7.5. Импорт функции `analyze_text`

Введите в ячейку A1 (рис. 7.6) предложение "Great food and excellent service" ("Еда была отличной, а обслуживание - превосходным!").

Введите в ячейку B1 следующее выражение, чтобы передать текст из ячейки A1 в импортированную из `sentiment.py` функцию `analyze_text`:

```
=analyze_text(A1)
```

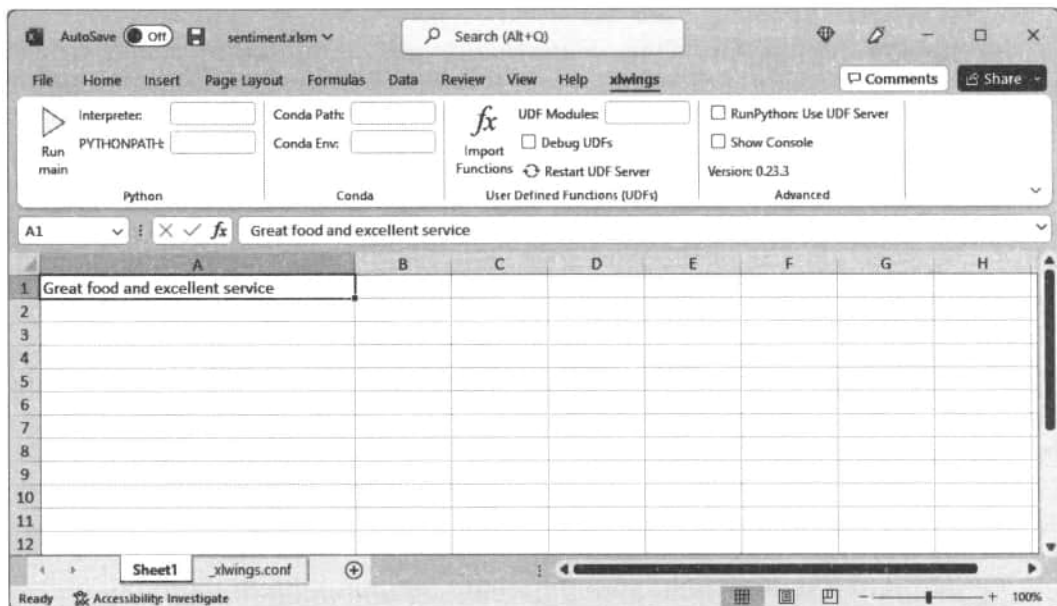



Рис. 7.6. Ввод строки текста для анализа

Убедитесь, что в ячейке B1 появилось число от 0,0 до 1,0 (рис. 7.7). Это оценка, которую модель машинного обучения присвоила тексту "Great food and excellent service!" ("Еда была отличной, а обслуживание - превосходным!"). Согласны ли вы с такой оценкой? В завершение введите несколько собственных текстовых строк и посмотрите, как будут оценены они.

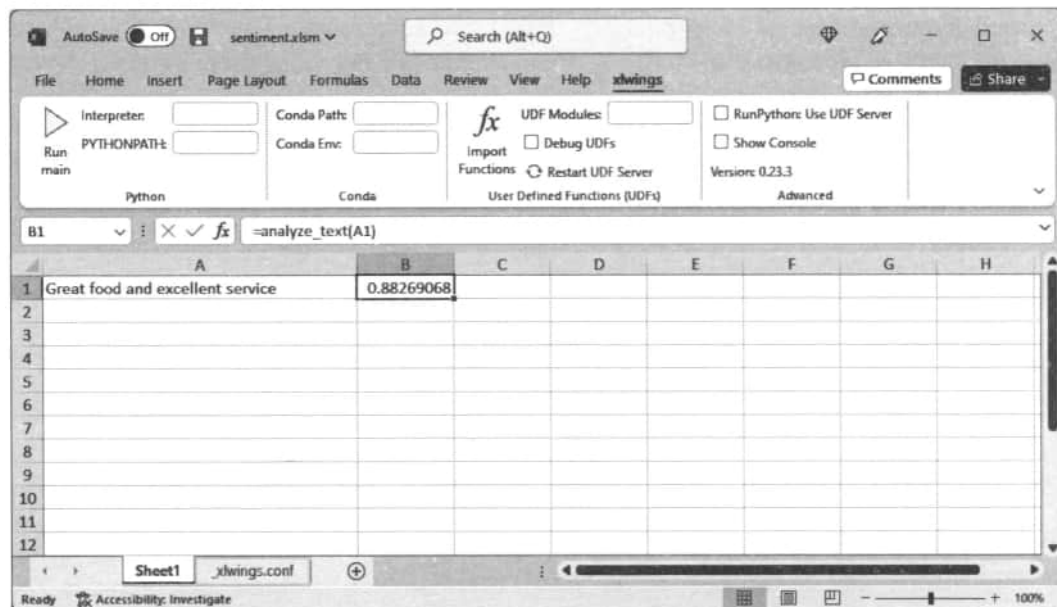


Рис. 7.7. Анализ настроений в Excel

UDF, написанные на языке Python, открывают перед пользователями Excel целый мир новых возможностей благодаря богатой экосистеме библиотек Python, предназначенных для машинного обучения, статистического анализа и других задач. Они предоставляют пользователям Excel прекрасную возможность операционализировать модели машинного обучения, написанные на Python.

Резюме

Для написания Python-клиента, вызывающего модель машинного обучения на Python, требуется не более одной строки кода для десериализации модели из файла .pkl. Один из способов вызова Python-модели для не-Python-клиентов — обернуть модель в написанный на Python веб-сервис и вызывать ее с помощью REST API. Веб-сервис может быть размещен локально или в облаке, а контейнеризация веб-сервиса (и модели) упрощает развертывание и делает программное обеспечение более переносимым.

Альтернативным подходом является использование ONNX для преодоления языковой лакуны. С помощью ONNX можно сохранять модель Scikit в файл .onnx и загружать ее из различных языков программирования, включая C, C++, C#, Java и JavaScript. После загрузки модель можно вызвать так же, как если бы она была вызвана из Python.

Другим вариантом вызова моделей машинного обучения из клиентов, не использующих Python, является написание модели на том же языке, что и клиент. ML.NET от Microsoft, которая является бесплатной, кроссплатформенной и с открытым исходным кодом, — отличный вариант для разработчиков на C#. Другие библиотеки включают Java-ML (<https://oreil.ly/7NDrq>) для разработчиков Java и Caret (<https://oreil.ly/SB6Hn>) и Tidymodels (<https://oreil.ly/KcMVW>) для разработчиков R. Поддерживаемые этими библиотеками API отличаются от API в Scikit, но заложенные в них принципы одинаковы.

- ЧАСТЬ II -

Глубокое обучение с помощью Keras и TensorFlow

Глубокое обучение

В каждой модели, представленной в *части 1* книги, использовались классические алгоритмы машинного обучения, составляющие основу этих моделей: логистическая регрессия, случайный лес и т. д. Такие модели часто называют *традиционными* моделями машинного обучения, чтобы отличить их от моделей глубокого обучения. Вспомним из *главы 1*, что глубокое обучение (deep learning) — это подмножество машинного обучения, которое опирается в основном на нейронные сети, и что большая часть того, что сегодня считается искусственным интеллектом, реализована с помощью глубокого обучения. От распознавания объектов на фотографиях до перевода речи в реальном времени и использования компьютеров для создания произведений искусства, музыки, поэзии и фотореалистичных портретов (<https://oreil.ly/2cDLY>)¹ — глубокое обучение позволяет компьютерам совершать подвиги, которые не под силу традиционному машинному обучению.

Я часто знакоблю разработчиков программного обеспечения с глубоким обучением, предлагая им придумать алгоритм, позволяющий определить, есть ли на фотографии собака. Если они предложат решение, я представлю в качестве контраргумента фотографию собаки, которая опровергнет алгоритм. Традиционные модели машинного обучения могут частично решить эту проблему, но когда дело доходит до распознавания объектов на изображениях, глубокое обучение оказывается на высоте. Обучить нейронную сеть распознавать изображения собак не так уж сложно, иногда это получается даже точнее, чем у человека. Как только вы научитесь это делать, то совершите небольшой шаг вперед к распознаванию бракованных деталей, сходящих с конвейера, или велосипедов, проезжающих перед самоуправляемым автомобилем.

Нейронные сети существуют уже несколько десятилетий, но только в последние 10 лет или около того появились вычислительные мощности, достаточные для обучения сложных сетей. Современные нейронные сети обучаются на графических процессорах (graphics processing units, GPU; <https://oreil.ly/25jZk>) и тензорных процессорах (tensor processing units, TPU; <https://oreil.ly/t9pV6>), которые часто подключаются к высокопроизводительным вычислительным кластерам. Графические процессоры отлично подходят для игр, поскольку обеспечивают высокопроиз-

¹ Вводя в адресной строке своего браузера эту ссылку, вы всякий раз будете получать новое фото. — Прим. ред.

водительную графику. Кроме того, они являются эффективными машинами параллельной обработки данных, позволяющими ученым обучать нейронные сети в разы быстрее, чем это требуется на обычных процессорах. Сегодня любой обладающий финансами исследователь может приобрести графический процессор NVIDIA либо запустить GPU в Azure или AWS и получить доступ к вычислительным мощностям, о которых 20 лет назад можно было только мечтать. Это, как ничто другое, стало причиной возрождения искусственного интеллекта и ускорило постоянный прогресс в его развитии.

Эта глава — первая из нескольких, посвященных глубокому обучению. В ней вы узнаете:

- ◆ что такое нейронная сеть и откуда взялось слово "глубокое" в глубоком обучении;
- ◆ как нейронная сеть преобразует входные данные в выходные с помощью простых математических операций;
- ◆ что происходит при обучении нейронной сети, а также какие проблемы возникают при обучении.

К созданию и обучению нейронных сетей мы пока не будем приступать — об этом речь пойдет в *главе 9*. Прежде чем строить дом, необходимо заложить фундамент. Работа с фундаментом начинается прямо сейчас.

Понимание нейронных сетей

Нейронные сети бывают разных типов. Например, *сверточные (конволюционные) нейронные сети* (convolutional neural network, CNN; <https://oreil.ly/Kdts5>) отлично справляются с задачами компьютерного зрения, такими как классификация изображений. *Рекуррентные нейронные сети* (recurrent neural network, RNN; <https://oreil.ly/0nUec>) находят применение в распознавании рукописного текста и обработке естественного языка (natural language processing, NLP), а *генеративные состязательные сети* (generative adversarial network, GAN; <https://oreil.ly/LL9Wt>) позволяют компьютерам создавать художественные, музыкальные и другие произведения. Но для начала необходимо понять, что такое нейронная сеть и как она работает.

Простейшим типом нейронной сети является *многослойный перцептрон* (multilayer perceptron; <https://oreil.ly/KqFX1>). Он состоит из узлов, или *нейронов*, расположенных слоями. Глубина сети — это количество слоев, а ширина — количество нейронов в каждом слое, которое может быть разным для каждого слоя. Современные нейронные сети иногда содержат 100 и более слоев и тысячи нейронов в отдельных слоях. *Глубокая нейронная сеть* (deep neural network) — это сеть, содержащая много слоев, и именно от нее происходит термин "*глубокое обучение*".

Многослойный перцептрон на рис. 8.1 содержит три слоя: входной слой с двумя нейронами, средний слой (также известный как *скрытый слой*) с тремя нейронами и выходной слой с одним нейроном. Поскольку при подсчете слоев входной слой

часто игнорируется, некоторые могут утверждать, что в этой сети *два* слоя, а не три. Независимо от этого, задача сети состоит в том, чтобы принять на вход два значения с плавающей точкой и выдать на выходе одно число с плавающей точкой. Нейронные сети работают с числами с плавающей точкой. Они работают *только* с числами с плавающей точкой. Как и в случае традиционных моделей машинного обучения, нейронная сеть может обрабатывать нечисловые данные, например текстовые строки, только в том случае, если эти данные предварительно преобразованы в числа.

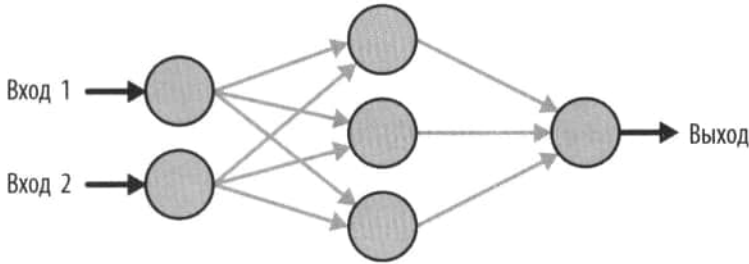


Рис. 8.1. Многослойный перцептрон

Оранжевые стрелки на рис. 8.1 обозначают связи между нейронами. Каждый нейрон в каждом слое связан с каждым нейроном в следующем слое, что послужило основанием для термина "*полносвязные слои*". Каждому соединению присваивается *вес*, который обычно представляет собой небольшое число с плавающей точкой. Кроме того, каждому нейрону вне входного слоя присваивается *смещение*, которое также является небольшим числом с плавающей точкой. На рис. 8.2 показан набор весов и смещений, позволяющий сети суммировать два входных сигнала (например, сложить 2 и 2).

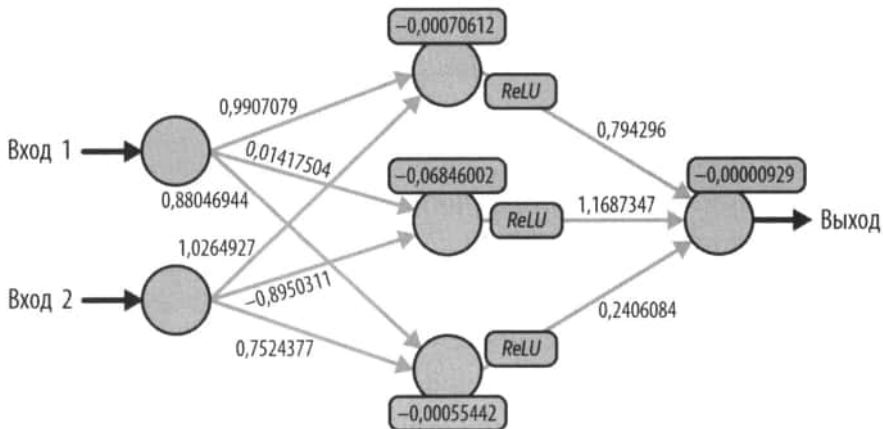


Рис. 8.2. Веса и смещения

Блоки с надписью "ReLU" представляют собой *функции активации* (activation function; <https://oreil.ly/Q4wqT>), которые применяют простые нелинейные преобразо-

вания к передаваемым через сеть значениям. Наиболее часто используемой функцией активации является функция *выпрямленных линейных единиц* (rectified linear unit, ReLU; <https://oreil.ly/KdPE0>), которая пропускает через себя положительные числа без изменений, а отрицательные преобразует в 0. Без функций активации нейронным сетям было бы трудно моделировать нелинейные данные. А ведь известно, что реальные данные, как правило, нелинейны.

Нейроны выполняют простые линейные преобразования данных, поступающих на их вход. Для нейрона с одним входом x значение y вычисляется путем умножения x на присвоенный входу вес m и добавления b — смещения нейрона (рис. 8.3).

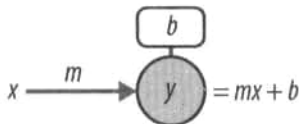


Рис. 8.3. Линейное преобразование данных нейроном

Знакомо? Это уравнение линейной регрессии. В Scikit-Learn есть класс `Perceptron` (<https://oreil.ly/HLCb9>), который моделирует подобное поведение и может быть использован для построения нейронных моделей линейной регрессии. В нем даже есть классы `MLPRegressor` (<https://oreil.ly/vJUCI>) и `MLPClassifier` (<https://oreil.ly/XY5GE>) для построения простых многослойных перцептронов. Однако Scikit не является библиотекой глубокого обучения. Настоящие библиотеки глубокого обучения в большей степени поддерживают продвинутые нейронные сети.



Сочетание выполняющих линейные преобразования нейронов и применяющих нелинейные преобразования активационных функций является воплощением универсальной теоремы аппроксимации (<https://oreil.ly/xfDMb>), которая гласит, что любую функцию f можно аппроксимировать путем суммирования выходов линейных функций и преобразования их с помощью нелинейной функции. В учебниках часто пишут, что активационные функции "добавляют нелинейность" в нейронные сети. Теперь вы знаете, почему.

Для того чтобы превратить входные данные в выходные, нейронная сеть присваивает входные значения нейронам входного слоя. Затем она умножает значения входных нейронов на веса, соединяющие их с нейронами следующего слоя, суммирует входы для каждого нейрона и добавляет смещения. Этот процесс повторяется для распространения значений слева направо до выходного слоя. На рис. 8.4 показано, что происходит в первых двух слоях, когда сеть на рис. 8.2 складывает 2 и 2.

Значения передаются от скрытого слоя к выходному аналогичным образом, за одним исключением: перед умножением на веса они преобразуются с помощью активационной функции. Помните, что функция активации ReLU превращает отрицательные числа в 0. На рис. 8.5 значение $-1,83$, рассчитанное для среднего нейрона скрытого слоя, при передаче на выходной слой преобразуется в 0, что фактически исключает вклад этого нейрона в выходной сигнал.

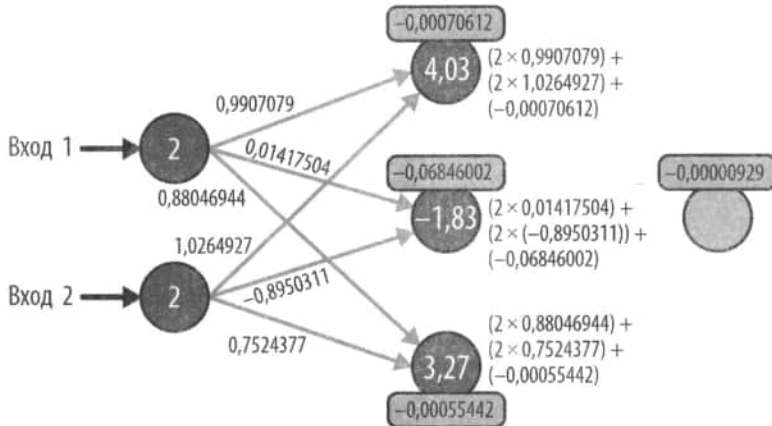


Рис. 8.4. Поток данных от входного слоя к скрытому при сложении 2 и 2

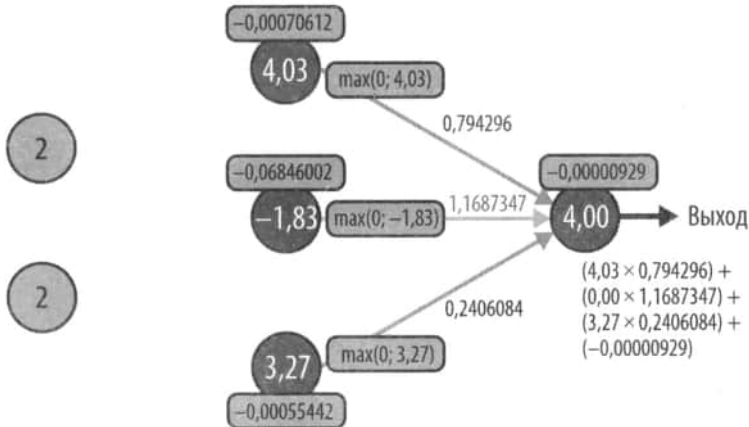


Рис. 8.5. Поток данных от скрытого слоя к выходному слою при сложении 2 и 2

Имея набор весов и смещений, несложно написать нейронную сеть вручную. Следующий код на языке Python моделирует сеть, показанную на рис. 8.2:

Веса

`w0 = 0.9907079`

`w1 = 1.0264927`

`w2 = 0.01417504`

`w3 = -0.8950311`

`w4 = 0.88046944`

`w5 = 0.7524377`

`w6 = 0.794296`

`w7 = 1.1687347`

`w8 = 0.2406084`

Смещения

b0 = -0.00070612

b1 = -0.06846002

b2 = -0.00055442

b3 = -0.00000929

```
def relu(x):
```

```
    return max(0, x)
```

```
def predict(x1, x2):
```

```
    h1 = (x1 * w0) + (x2 * w1) + b0
```

```
    h2 = (x1 * w2) + (x2 * w3) + b1
```

```
    h3 = (x1 * w4) + (x2 * w5) + b2
```

```
    y = (relu(h1) * w6) + (relu(h2) * w7) + (relu(h3) * w8) + b3
```

```
    return y
```

Если хотите убедиться в этом, поместите код в блокнот Jupyter и вызовите функцию `predict` с входными данными 2 и 2. Ответ должен быть очень близок к реальной сумме 2 и 2.

Для данной задачи существует бесконечное множество комбинаций весов и смещений, которые приводят к желаемому результату. На рис. 8.6 показана та же сеть с совершенно другим набором весов и смещений. Однако если подставить эти значения в предыдущий код (или прогнать значения через сеть вручную), то можно обнаружить, что сеть в равной степени способна складывать 2 и 2, а также другие малые значения.

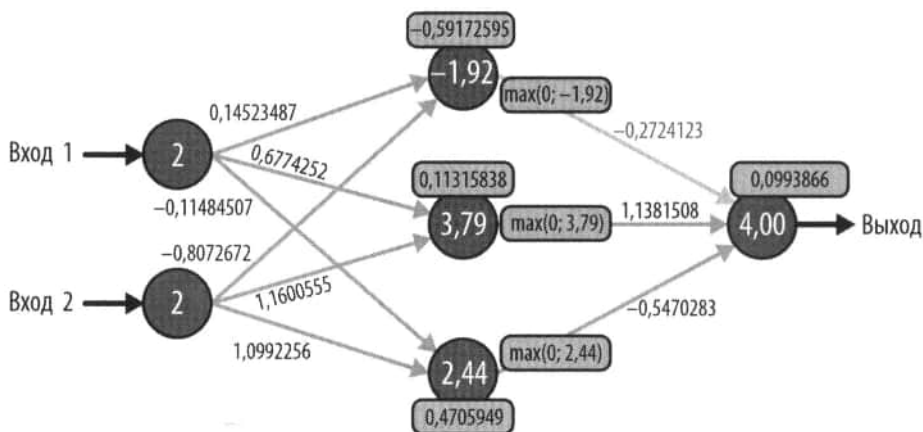


Рис. 8.6. Сложение 2 и 2
с другим набором весов и смещений

При заданных весах и смещениях использование нейронной сети для прогнозирования оказывается очень простым. Это не более чем умножение и сложение. Но

подобрать набор весов и смещений для начала — сложная задача. Поэтому нейронные сети необходимо обучать.

Обучение нейронных сетей

Обучение традиционной ML-модели позволяет приспособить ее к набору данных. Нейронные сети тоже требуют обучения, и именно в процессе обучения рассчитываются веса и смещения. Веса обычно инициализируются небольшими случайными числами. Смещения — как правило, нулями. В необученном состоянии нейронная сеть способна лишь генерировать случайные выходные сигналы. После завершения обучения весовые коэффициенты и смещения позволяют сети отличать собак от кошек, переводить книжную рецензию на другой язык или делать всё то, для чего она была создана.

Что происходит при обучении нейронной сети? На высоком уровне обучающие выборки подаются через сеть, ошибка (разница между вычисленным и правильным выходом) вычисляется с помощью функции потерь, и *алгоритм обратного распространения ошибки* (<https://oreil.ly/ScACh>) проходит через сеть назад, регулируя веса и смещения (рис. 8.7). Эта процедура повторяется до тех пор, пока ошибка не станет достаточно малой. С каждой итерацией веса и смещения становятся все более точными, а ошибка, соответственно, меньше.

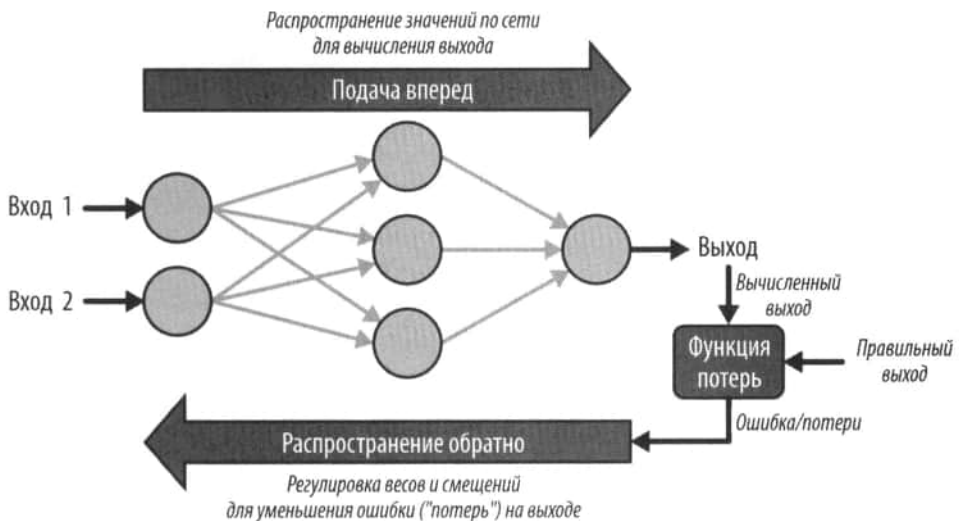


Рис. 8.7. Настройка весов и смещений в процессе обучения

Наиболее важным компонентом режима обратного распространения является *оптимизатор*, который на каждом проходе назад решает, насколько и в каком направлении, положительном или отрицательном, корректировать веса и смещения. Исследователи данных постоянно работают над поиском лучших и более эффективных оптимизаторов, позволяющих обучать сети более точно и за меньшее время.

Наберите в поисковике "нейронные сети", и вы обнаружите множество статей с большим количеством сложной математики. Большая часть математики связана с оптимизацией. Оптимизатор не может просто угадать, как настроить веса и смещения, в силу их огромного количества. Нейронная сеть, содержащая два скрытых слоя по 1000 нейронов в каждом, имеет 1 000 000 связей между слоями, а значит, и 1 000 000 весов, которые необходимо настроить. Обучение заняло бы вечность, если бы стратегия оптимизации сводилась к случайному угадыванию. Оптимизатор должен быть достаточно интеллектуальным, чтобы на каждой последующей итерации вносить коррективы, уменьшающие ошибку.

Для визуализации работы оптимизаторов специалисты по исследованию данных используют графики, подобные приведенному на рис. 8.8. Этот график называется *ландшафтом потерь* (loss landscape). В целях визуализации он уменьшен до трех измерений, но в действительности он содержит множество измерений, иногда миллионы. Разноцветный контур отражает ошибку для различных комбинаций весов и смещений. Задача оптимизатора — перемещаться по контуру и находить комбинацию, дающую наименьшую ошибку, что соответствует нижней точке, или глобальному минимуму, на ландшафте потерь.

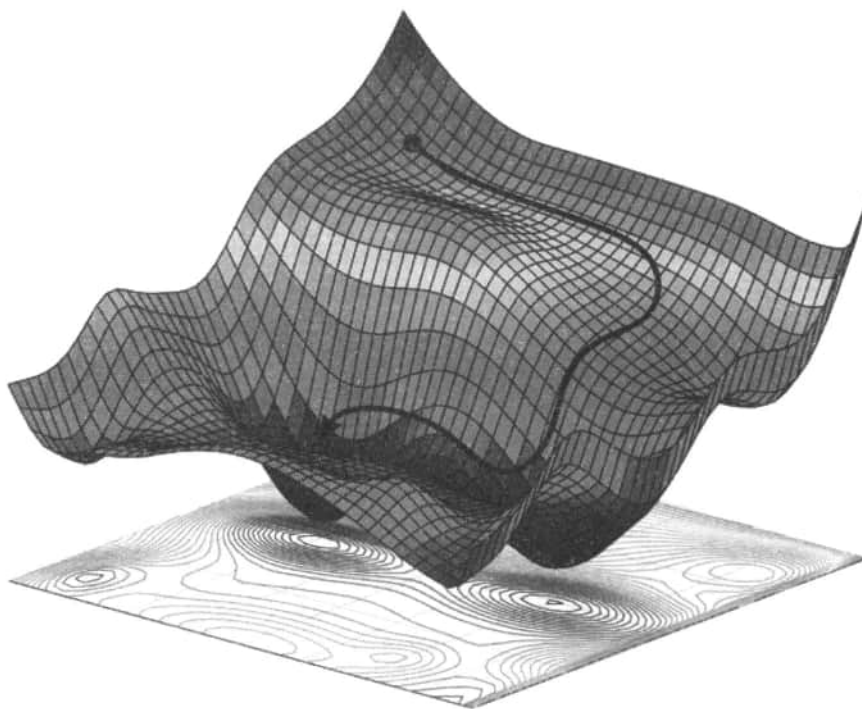


Рис. 8.8. Ландшафт потерь

(источник: Amini A., Soleimany A., Karaman S., Rus D. Spatial Uncertainty Sampling for End-to-End Control. — NeurIPS Bayesian Deep Learning, 2017; <https://arxiv.org/pdf/1805.04829>)

Работа оптимизатора не является простой. Она включает в себя вычисление частных производных (расчет наклона контура относительно каждого веса и смеще-

ния), градиентный спуск (корректировка весов и смещений, чтобы они двигались *вниз* по склону, а не вверх или в сторону) и *скорость обучения*, которая определяет дробную корректировку весов и смещений на каждом проходе обратного распространения. Если скорость обучения слишком велика, то оптимизатор может пропустить глобальный минимум. Если же она слишком мала, то обучение сети займет много времени. Современные оптимизаторы используют *адаптивные скорости обучения*, которые делают меньшие шаги по мере приближения к минимуму, где наклон контура равен нулю. Еще более усложняет ситуацию то, что оптимизатор не должен застревать в локальных минимумах, чтобы он мог продолжать движение по контуру к глобальному минимуму, где ошибка наименьшая. Также необходимо опасаться седловых точек, когда наклон увеличивается в одном направлении, но падает в перпендикулярном.



При изучении градиентного спуска можно встретить такие термины, как стохастический градиентный спуск (stochastic gradient descent, SGD; <https://oreil.ly/Cawlk>) и мини-пакетный градиентный спуск (mini-batch gradient descent, MBGD; <https://oreil.ly/TLiCP>). Оптимизация с помощью градиентного спуска — это итерационный процесс, в ходе которого через сеть подаются образцы, вычисляются градиенты, а затем градиенты объединяются со скоростью обучения для обновления весов и смещений. Обновление весов и смещений после каждого поступающего через сеть образца требует больших вычислительных затрат, поэтому при обучении обычно прогоняются через сеть партии из 30–40 образцов, усредняется ошибка, а затем выполняется обратное распространение. Это и есть MBGD. Он ускоряет обучение и помогает оптимизатору обходить локальные минимумы. Более подробную информацию, а также очень доступное изложение связанных с обучением нейронных сетей проблем можно найти в статье "How Neural Networks Are Trained" ("Как обучаются нейронные сети", <https://oreil.ly/HXX57>).

Нейронные сети принципиально просты. Сложно их математическое обучение. К счастью, для их построения не обязательно понимать все, что происходит в процессе обучения. Библиотеки глубокого обучения, такие как Keras (<https://oreil.ly/7K0Ko>) и TensorFlow (<https://oreil.ly/dWamB>), ограждают вас от математики и предоставляют современные оптимизаторы для выполнения тяжелой работы. Но теперь, когда при использовании одной из этих библиотек вас попросят выбрать функцию потерь и оптимизатор, вы будете понимать, что это и зачем нужно.

Резюме

Глубокое обучение — это подмножество машинного обучения, которое опирается на глубокие нейронные сети, и оно является основой современного искусственного интеллекта. Именно с его помощью компьютеры распознают объекты на изображениях, переводят текст и речь на другие языки, создают произведения искусства и

музыку, а также выполняют другие задачи, которые еще несколько лет назад были практически невозможны.

Многослойный перцептрон — это простая нейронная сеть, состоящая из слоев нейронов. Каждый нейрон превращает входные данные в выходные с помощью простой математической формулы. Функции активации дополнительно преобразуют данные при прохождении между слоями, вводя нелинейность, что позволяет нейронным сетям подстраиваться под самые разные наборы данных. Скрытые слои между входным и выходным слоями выполняют основную вычислительную работу, а многослойный перцептрон с большим количеством скрытых слоев называется глубокой нейронной сетью.

Обучение нейронной сети позволяет приспособить ее к набору данных путем итеративного изменения весов и смещений — весов, соединяющих нейроны в соседних слоях, и смещений, назначаемых самим нейронам, — для получения желаемого результата. Сердцем процесса обучения являются проходы обратного распространения, которые регулируют веса и смещения. Компонент, отвечающий за корректировку, — оптимизатор, конечной целью которого является нахождение оптимальной комбинации весов и смещений при минимальном количестве проходов обратного распространения.

Теперь, когда вы поняли, как работают нейронные сети, следующий шаг — научиться их строить и обучать. Для этого специалисты по обработке данных используют такие библиотеки, как Keras и TensorFlow. В *главе 9* начинается глубокое погружение в оба этих фреймворка.

Нейронные сети

Машинное обучение не представляет собой ничего сложного, если у вас есть правильно составленный набор данных для работы. *Причина*, по которой это несложно, заключается в таких библиотеках, как Scikit-Learn и ML.NET, сводящих сложные алгоритмы обучения к нескольким строкам кода. Глубокое обучение также не представляет проблемы благодаря таким библиотекам, как Microsoft Cognitive Toolkit (CNTK, <https://oreil.ly/3jPyE>), Theano (<https://oreil.ly/l4V0X>) и PyTorch (<https://oreil.ly/grwxl>). Но наиболее распространенной библиотекой для построения нейронных сетей является TensorFlow (<https://oreil.ly/xuxYG>) — фреймворк с открытым исходным кодом, созданный компанией Google и выпущенный в 2015 году под лицензией Apache License 2.0.

TensorFlow не ограничивается построением нейронных сетей. Это фреймворк для выполнения быстрых математических операций в масштабе с использованием *тензоров*, которые представляют собой обобщенные массивы. Тензоры могут представлять скалярные значения (0-мерные тензоры), векторы (одномерные тензоры), матрицы (двумерные тензоры) и т. д. Нейронная сеть, по сути, является рабочим процессом для преобразования тензоров. Трехслойный перцептрон, рассмотренный в *главе 8*, принимает на вход одномерный тензор, содержащий два значения, преобразует его в одномерный тензор, содержащий три значения, и выдает на выходе 0-мерный тензор. TensorFlow позволяет задавать направленные графы, которые, в свою очередь, определяют способ вычисления тензоров. В отличие от Scikit, библиотека поддерживает графические процессоры.

Кривая обучения для TensorFlow довольно крутая. Другая библиотека, названная Keras (<https://oreil.ly/goUK8>), предоставляет упрощенный Python-интерфейс для TensorFlow и стала своего рода Scikit для глубокого обучения. Keras — это всё для нейронных сетей. Она начала свою жизнь как отдельный проект в 2015 году, но в 2019 году была интегрирована в TensorFlow. Любой код, написанный с использованием встроенного в TensorFlow модуля Keras, в конечном итоге выполняется в TensorFlow (и оптимизируется под него). Даже Google рекомендует использовать API Keras.

Keras предлагает два API для построения нейронных сетей: последовательный API (<https://oreil.ly/6C1Mp>) и функциональный API (<https://oreil.ly/2vOt2>). Первый является более простым и достаточным для большинства нейронных сетей. Второй полезен в более сложных сценариях, таких как сети с несколькими входами или выходами — например, выход классификации и выход регрессии, что характерно

для нейронных сетей, выполняющих обнаружение объектов, — или с общими слоями. В большинстве примеров, приведенных в этой книге, используется последовательный API. Если любопытство заставит вас узнать больше о функциональном API, обратитесь к статье Джейсона Браунли "Как использовать функциональный API Keras для глубокого обучения" (Brownlee Jason. How to Use the Keras Functional API for Deep Learning; <https://oreil.ly/QQJ5M>), содержащей очень понятное вступление.

Построение нейронных сетей с помощью Keras и TensorFlow

Создание нейронной сети с помощью последовательного API Keras очень простое. Сначала создается экземпляр класса `Sequential` (<https://oreil.ly/23D4a>). Затем вызывается функция `add` (<https://oreil.ly/Zx0me>) на объекте `Sequential` для добавления слоев. Слои являются экземплярами таких классов, как `Dense` (<https://oreil.ly/ITzcp>), который представляет собой полностью связанный слой с заданным количеством нейронов. Следующие операторы создают трехслойную сеть, описанную в *главе 8*:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential()
model.add(Dense(3, activation='relu', input_dim=2))
model.add(Dense(1))
```

Эта сеть содержит входной слой с двумя нейронами, скрытый слой с тремя нейронами и выходной слой с одним нейроном. Значения, передаваемые из скрытого слоя в выходной, преобразуются с помощью функции активации ReLU, которая, как вы помните, превращает отрицательные числа в 0 и помогает модели подстраиваться под нелинейные наборы данных. Обратите внимание, что входной слой не обязательно добавлять явно. Параметр `input_dim=2` в первом скрытом слое неявно создает входной слой с двумя нейронами.



`relu` (<https://oreil.ly/dQUt0>) — одна из нескольких активационных функций (<https://oreil.ly/uAIZt>), входящих в состав Keras. К ним относятся `tanh` (<https://oreil.ly/IGwZp>), `sigmoid` (<https://oreil.ly/WXtNw>) и `softmax` (<https://oreil.ly/QlGs9>). В скрытых слоях редко, если вообще когда-либо, будут использоваться какие-либо другие функции, кроме `relu`. Позже в этой главе будет показано, почему такие функции, как `sigmoid` и `softmax`, полезны в выходных слоях сетей, выполняющих классификацию, а не регрессию.

Когда все слои добавлены, следующим шагом будет вызов команды `compile` (<https://oreil.ly/YZvZy>) и указание важных атрибутов, таких как оптимизатор и функция потерь, которые будут использоваться при обучении. Приведем пример:

```
model.compile(optimizer='adam', loss='mae', metrics=['mae'])
```

Давайте последовательно пройдемся по параметрам.

```
optimizer='adam'
```

Указывает Keras использовать оптимизатор Adam (<https://oreil.ly/ebDXm>) для настройки весов и смещений в каждом проходе обратного распространения в процессе обучения. Adam — один из восьми встроенных в Keras оптимизаторов (<https://oreil.ly/XpozA>), и он является одним из самых совершенных. Он использует адаптивную скорость обучения, и я всегда начинаю с него, если нет веских причин поступать иначе.

```
loss='mae'
```

Указывает Keras использовать среднюю абсолютную ошибку (mean absolute error, MAE) для измерения потерь. Это характерно для нейронных сетей, предназначенных для решения задач регрессии. Другой часто используемой опцией для регрессионных моделей является `loss='mse'` для среднеквадратической ошибки (mean squared error, MSE).

```
metrics=['mae']
```

Указывает Keras фиксировать значения MAE в процессе обучения сети. Эта информация используется после завершения обучения для оценки его эффективности.

Строковые значения, такие как `'adam'` и `'mae'`, являются сокращениями для встроенных в Keras функций. Например, `optimizer='adam'` эквивалентно `optimizer=Adam()`. Полная форма удобна для вызова функции со значениями параметров не по умолчанию, например `optimizer=Adam(learning_rate=2e-5)` для создания оптимизатора Adam с заданной скоростью обучения. Пример этого мы увидим в *главе 13*, когда будем проводить тонкую настройку модели, обучая ее с низкой скоростью обучения.

Внутри метода `compile` Keras создает объектный граф TensorFlow для ускорения выполнения. После того как сеть скомпилирована, ее можно обучить путем вызова функции `fit`:

```
hist = model.fit(x, y, epochs=100, batch_size=100, validation_split=0.2)
```

Метод `fit` принимает множество параметров. Ниже приведены те, которые используются в данном примере.

```
x
```

Столбцы признаков набора данных.

```
y
```

Столбец метки набора данных — столбец, содержащий значения, которые будет пытаться предсказать сеть.

```
epochs=100
```

Указывает Keras обучать сеть в течение 100 итераций, или *эпох*. В каждой эпохе все обучающие данные проходят через сеть один раз.

```
batch_size=100
```

Указывает Keras пропустить через сеть 100 обучающих образцов, прежде чем выполнять обратное распространение для настройки весов и смещений. При

большом размере набора обучение занимает меньше времени, но при этом может пострадать точность. Обычно для нахождения оптимального баланса между временем обучения и точностью можно поэкспериментировать с различными размерами наборов. *Не следует полагать*, что уменьшение размера набора приведет к повышению точности. Это происходит часто, но иногда — нет.

`validation_split=0.2`

Указывает Keras, что в каждой эпохе нужно проводить обучение на 80% строк набора данных и проверять точность сети на оставшихся 20%. При желании можно самостоятельно разделить набор данных и использовать параметр `validation_data` для передачи данных для проверки. В Keras нет явной функции для разбиения набора данных, но для этого можно использовать функцию `train_test_split` (<https://oreil.ly/ObHAM>) из Scikit. Разница между `train_test_split` и `validation_split` заключается в том, что первая разбивает данные случайным образом и включает возможность стратифицированного разбиения. `validation_split`, напротив, просто делит набор данных на две части и не пытается перемешать или стратифицировать. Не используйте `validation_split` для упорядоченных данных, не перетасовав их предварительно.

Возможно, вас удивит тот факт, что если обучать одну и ту же сеть на одном и том же наборе данных несколько раз, то результаты будут каждый раз разными. По умолчанию веса инициализируются случайными значениями, и разные начальные точки приводят к разным результатам. Заложенная в процесс обучения некоторая случайность означает, что сеть будет обучаться по-разному, даже если она инициализирована одними и теми же случайными весами. Вместо того чтобы бороться с этим явлением, специалисты по исследованию данных учатся "принимать случайность". Если вы будете работать по приведенному в следующем разделе учебнику, ваши результаты будут отличаться от моих. Они не должны отличаться намного, но все-таки будут разниться.



Случайные веса, присваиваемые связям между нейронами, не являются абсолютно случайными. В состав Keras входит около десятка инициализаторов (<https://oreil.ly/SPGKn>), каждый из которых инициализирует параметры по-своему. По умолчанию в плотных слоях для инициализации смещений используется инициализатор Zeroes (<https://oreil.ly/a4mXG>), а для инициализации весов — инициализатор GlorotUniform (<https://oreil.ly/FZTzQ>). Последний генерирует случайные числа, попадающие в равномерное распределение, границы которого вычисляются на основе топологии сети.

Об эффективности обучения можно судить по информации, возвращаемой методом `fit`. Он возвращает объект истории, содержащий метрики обучения и проверки, указанные в переданном в метод `compile` параметре `metrics`. Например, `metrics=['mae']` фиксирует MAE в конце каждой эпохи. Построение графиков этих показателей позволяет определить, правильное ли количество эпох длилось обучение. Кроме того, графики дают возможность выяснить, недостаточно или чрезмерно обучена сеть. На рис. 9.1 показан график MAE в течение 30 эпох обучения.

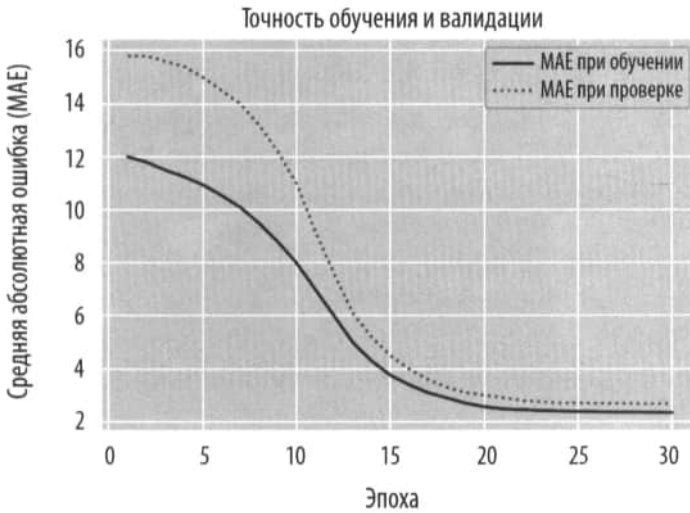


Рис. 9.1. Точность обучения и валидации во время обучения

Синяя кривая на рис. 9.1 показывает, как сеть подходила к обучающим данным. Оранжевая кривая демонстрирует, как она тестировалась на проверочных данных. Большая часть обучения пришлось на первые 20 эпох, но по мере обучения MAE продолжала снижаться. В конце обучения MAE при проверке почти сравнялась с MAE при обучении, что свидетельствует о том, что сеть не переобучена. Как правило, вам не важно, насколько хорошо сеть соответствует обучающим данным. Существенно, насколько хорошо сеть соответствует данным проверки, поскольку это показывает, как сеть работает с данными, с которыми она раньше не сталкивалась. Чем больше разрыв между точностью обучения и точностью проверки, тем выше вероятность того, что сеть переобучена.

После того как нейронная сеть обучена, можно вызвать ее метод `predict` (<https://oreil.ly/UO1uW>), чтобы выполнить прогноз:

```
prediction = model.predict(np.array([[2, 2]]))
```

В данном примере сеть принимает на вход два значения с плавающей точкой, а на выходе выдает одно значение с плавающей точкой. Возвращаемое функцией `predict` значение и есть этот выход.

Определение размеров нейронной сети

Нейронная сеть характеризуется количеством слоев (*глубиной* сети), количеством нейронов в каждом слое (*шириной* слоев), типом слоев (в данном примере — плотный (Dense) слой полностью связанных нейронов) и используемыми функциями активации. Существуют и другие типы слоев (<https://oreil.ly/7xehq>), многие из которых будут представлены в последующих главах. Например, слои Dropout (<https://oreil.ly/HsN2s>) позволяют повысить способность сети к обобщению за счет случайного сброса связей между слоями при обновлении весов, а слои Conv2D (<https://oreil.ly/>

avOeo) позволяют создавать сверточные нейронные сети (convolutional neural network, CNN), которые отлично справляются с обработкой изображений.

Как при проектировании сети выбрать правильное количество слоев и количество нейронов для каждого слоя? Краткий ответ заключается в том, что "правильная" ширина и глубина зависят от проблемы, которую необходимо решить, от набора данных, на котором проводится обучение, и от желаемой точности. Как правило, для достижения точности *требуется минимальная ширина и глубина слоя*, а достигается это с помощью интуиции и экспериментов. При этом следует помнить о нескольких рекомендациях.

- ◆ Большая ширина и глубина дают сети больше возможностей для "обучения" за счет более плотного прилегания к обучающим данным. Они также увеличивают вероятность чрезмерного обучения. Важны результаты проверки, и иногда ослабление подгонки к обучающим данным позволяет сети лучше обобщать данные. Самый простой способ ослабить подгонку — уменьшить количество нейронов.
- ◆ Как правило, лучше предпочесть большую ширину, чем большую глубину, чтобы избежать проблемы исчезающего градиента (vanishing gradient, <https://oreil.ly/9q1xK>), которая снижает эффективность дополнительных слоев. Функция активации ReLU обеспечивает определенную защиту от исчезающих градиентов, которая, правда, не является абсолютной. Объяснение этому см. в статье "Как решить проблему исчезающих градиентов с помощью ReLU" ("Brownlee Jason. How to Fix the Vanishing Gradients Problem Using the ReLU", <https://oreil.ly/yHIqy>). Кроме того, сеть, содержащая, скажем, 100 нейронов в одном слое, обучается быстрее, чем сеть с пятью слоями по 20 нейронов в каждом, поскольку в первом случае весов меньше. Вдумайтесь: между нейронами в одном слое нет связей, а между пятью слоями, содержащими по 20 нейронов, — 1600 связей ($20^2 \times 4$).
- ◆ Меньшее количество нейронов означает меньшее время обучения. Современные нейронные сети, обученные на больших массивах данных, иногда требуют нескольких дней или недель для обучения на высокопроизводительных графических процессорах, поэтому время обучения имеет большое значение.

В повседневной жизни специалисты по обработке данных экспериментируют с различной шириной и глубиной, чтобы найти оптимальный баланс между временем обучения, точностью и способностью сети к обобщению. Для многослойного перцептрона редко требуется более двух скрытых слоев, часто достаточно одного. Сеть с одним или двумя скрытыми слоями способна решать даже сложные нелинейные задачи. Например, два слоя по 128 нейронов в каждом дают 16 384 (128^2) настраиваемых весов плюс 256 смещений. Это очень большая подгоночная мощность. Я часто начинаю с одного или, в крайнем случае, двух слоев по 512 нейронов в каждом и уменьшаю ширину или глубину в два раза до тех пор, пока точность проверки не упадет ниже приемлемого порога.

Использование нейронной сети для прогнозирования стоимости проезда в такси

Применим полученные знания для построения и обучения нейронной сети. Мы будем решать ту же задачу, что и в *главе 2*: использовать данные Комиссии по такси и лимузинам города Нью-Йорка для прогнозирования стоимости проезда в такси. С этой целью будем использовать нейронную сеть в качестве регрессионной модели.

Загрузите CSV-файл с набором данных (<https://oreil.ly/LZ4OO>), если вы еще не сделали это в *главе 2*, и скопируйте его в каталог Data, где размещены ваши блокноты Jupyter. Затем с помощью следующего кода загрузите набор данных и покажите первые пять строк. Этот набор содержит около 55 000 строк и является подмножеством гораздо большего набора данных, который недавно использовался в конкурсе Kaggle по предсказанию стоимости проезда на такси в Нью-Йорке (<https://oreil.ly/1vIVL>):

```
import pandas as pd
```

```
df = pd.read_csv('Data/taxi-fares.csv', parse_dates=['pickup_datetime'])
df.head()
```

Для того чтобы данные стали полезными, необходимо провести значительную подготовительную работу, что довольно часто встречается в машинном обучении, а также в глубоком обучении. Используйте следующие операторы для преобразования исходного набора данных в пригодный для обучения, а для пошагового объяснения применяемых преобразований обратитесь к примеру с оплатой проезда на такси в *главе 2*:

```
from math import sqrt
```

```
df = df[df['passenger_count'] == 1]
df = df.drop(['key', 'passenger_count'], axis=1)
```

```
for i, row in df.iterrows():
    dt = row['pickup_datetime']
    df.at[i, 'day_of_week'] = dt.weekday()
    df.at[i, 'pickup_time'] = dt.hour
    x = (row['dropoff_longitude'] - row['pickup_longitude']) * 54.6
    y = (row['dropoff_latitude'] - row['pickup_latitude']) * 69.0
    distance = sqrt(x**2 + y**2)
    df.at[i, 'distance'] = distance
```

```
df.drop(['pickup_datetime', 'pickup_longitude', 'pickup_latitude',
        'dropoff_longitude', 'dropoff_latitude'], axis=1, inplace=True)
```

```
df = df[(df['distance'] > 1.0) & (df['distance'] < 10.0)]
df = df[(df['fare_amount'] > 0.0) & (df['fare_amount'] < 50.0)]
df.head()
```

Полученный набор данных содержит столбцы дня недели (0–6, где 0 соответствует понедельнику), часа суток (0–23) и пройденного расстояния в милях, из которых были удалены выбросы (рис. 9.2).

	fare_amount	day_of_week	pickup_time	distance
2	6.1	0.0	15.0	1.038136
4	10.5	5.0	10.0	2.924341
5	15.3	4.0	20.0	4.862893
8	7.7	5.0	1.0	2.603493
9	8.9	3.0	16.0	1.365739

Рис. 9.2. Полученный набор данных после предобработки¹

Очередным шагом является создание нейронной сети. Используйте следующие операторы для создания сети с входным слоем, принимающим три значения (день, время и расстояние), двумя скрытыми слоями с 512 нейронами в каждом и выходным слоем с одним нейроном (прогнозируемая сумма тарифа):

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential()
model.add(Dense(512, activation='relu', input_dim=3))
model.add(Dense(512, activation='relu'))
model.add(Dense(1))
model.compile(optimizer='adam', loss='mae', metrics=['mae'])
model.summary()
```

Вызов метода `summary` (<https://oreil.ly/ZAQMT>) в последней строке позволяет получить краткий итог топологии сети, включая количество *обучаемых параметров* — весов и смещений, которые могут быть настроены для подгонки сети к набору данных (рис. 9.3). Для данного слоя количество параметров равно произведению числа нейронов в этом слое и предыдущем (число весов, соединяющих нейроны в двух слоях) плюс число нейронов в слое (смещения, связанные с этими нейронами). Эта сеть относительно проста, но в ней более четверти миллиона настроек, которые можно регулировать, подстраивая ее под набор данных.

Теперь отделим столбцы признаков от столбцов меток и используем их для обучения сети. Установите значение `validation_split` равным 0,2, чтобы проверить сеть на 20% обучающих данных. Проведите обучение в течение 100 эпох и используйте размер набора, равный 100. Поскольку набор данных содержит более 38 000 образ-

¹ Названия столбцов: `fare_amount` — стоимость проезда; `day_of_week` — день недели; `pickup_time` — время посадки; `distance` — расстояние. — Прим. ред.

цов, то в каждой эпохе будет выполнено около 380 проходов обратного распространения:

```
x = df.drop('fare_amount', axis=1)
y = df['fare_amount']
```

```
hist = model.fit(x, y, validation_split=0.2, epochs=100, batch_size=100)
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 512)	2048
dense_1 (Dense)	(None, 512)	262656
dense_2 (Dense)	(None, 1)	513
=====		
Total params: 265,217	(512 × 1 вес) + 1 смещение	
Trainable params: 265,217	(512 × 512 весов) + 512 смещений	
Non-trainable params: 0	(3 × 512 весов) + 512 смещений	
2048 + 262 656 + 513		

Рис. 9.3. Обучаемые параметры в простой нейронной сети

Используйте возвращаемый функцией `fit` объект `history` для построения графика точности обучения и проверки для каждой эпохи:

```
%matplotlib inline
import matplotlib.pyplot as plt
import seaborn as sns
sns.set()

err = hist.history['mae']
val_err = hist.history['val_mae']
epochs = range(1, len(err) + 1)

plt.plot(epochs, err, '-', label='MAE при обучении')
plt.plot(epochs, val_err, ':', label='MAE при проверке')
plt.title('Точность обучения и валидации')
plt.xlabel('Эпоха')
plt.ylabel('Средняя абсолютная ошибка (MAE)')
plt.legend(loc='upper right')
plt.plot()
```

Ваши результаты будут несколько отличаться от моих, но они должны выглядеть примерно так, как показано на рис. 9.4.

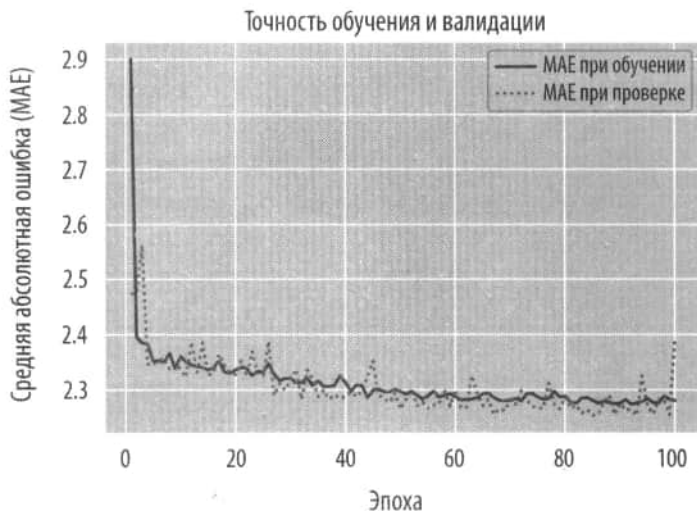


Рис. 9.4. График средней абсолютной ошибки после создания нейронной сети

Окончательная валидация MAE составила около 2,25, что означает, что в среднем предсказанная этой сетью стоимость проезда в такси должна приближаться к истинной стоимости с точностью около 2,25 доллара.

Вспомним из главы 2, что общей мерой точности для регрессионных моделей является коэффициент детерминации, или R^2 . В Keras нет функции для вычисления коэффициента R^2 , но в Scikit она есть. В связи с этим для вычисления R^2 для сети используйте следующие операторы:

```
from sklearn.metrics import r2_score
```

```
r2_score(y, model.predict(x))
```

Опять же, ваши результаты будут отличаться от моих, но, скорее всего, они окажутся в районе 0,75.

В завершение с использованием модели спрогнозируйте, сколько будет стоить нанять такси для поездки на расстояние 2 мили в 17:00 в пятницу:

```
import numpy as np
```

```
model.predict(np.array([[4, 17, 2.0]]))
```

Теперь спрогнозируйте стоимость проезда по маршруту длиной 2 мили, совершенного в 17:00 через день (в субботу):

```
model.predict(np.array([[5, 17, 2.0]]))
```

Предсказывает ли модель более высокую или более низкую сумму тарифа для той же поездки в субботу днем? Имеют ли результаты смысл, если учесть, что данные получены от такси Нью-Йорка?

Прежде чем закрыть этот блокнот, используйте его как основу для дальнейших экспериментов. Вот несколько шагов, которые можно пройти, чтобы получить дополнительные сведения о нейронных сетях.

- ◆ Несколько раз запустите блокнот от начала до конца и обратите внимание на разницу в показателях R^2 , а также на кривые MAE. Помните, что нейронные сети инициализируются со случайными весами при каждом создании, а дополнительная случайность в процессе обучения еще больше гарантирует, что результаты будут отличаться от прогона к прогону.
- ◆ Варьируйте ширину скрытых слоев. Я использовал 512 нейронов в каждом слое и пришел к выводу, что это дает приемлемые результаты. Смогут ли 128, 256 или 1024 нейрона на слой повысить точность? Попробуйте и узнайте. Поскольку результаты будут немного отличаться от одного прогона к другому, возможно, будет полезно обучить сеть несколько раз в каждой конфигурации и усреднить результаты.
- ◆ Варьируйте размер набора. Как это влияет на время обучения и почему? Как это влияет на точность?

Наконец, попробуйте сократить сеть до одного скрытого слоя, содержащего всего 16 нейронов. Снова проведите обучение и проверьте R^2 . Удивил ли вас результат? Сколько обучаемых параметров содержит эта сеть?

Бинарная классификация с помощью нейронных сетей

Одним из распространенных видов машинного обучения является бинарная классификация, когда на основе входных данных делается прогноз о том, к какому из двух возможных классов они относятся. Практическое применение — анализ настроений, фильтрация спама и выявление мошенничества. Такие модели обучаются на наборах данных, помеченных 1 и 0 (представляющими два класса), используют популярные алгоритмы обучения, такие как логистическая регрессия (<https://oreil.ly/EGn82>) и наивный Байес (<https://oreil.ly/1pyWP>), и часто строятся с помощью таких библиотек, как Scikit-Learn.

Глубокое обучение можно использовать и для бинарной классификации. По сути, построение нейронной сети, исполняющей роль бинарного классификатора, мало чем отличается от построения сети, играющей роль регрессора. В предыдущем разделе вы построили нейронную сеть, которая решала задачу регрессии. Эта сеть имела входной слой, который принимал три значения: расстояние до места поездки, час дня и день недели — и выдавал на выходе прогнозируемую стоимость проезда на такси. Построение нейронной сети, выполняющей бинарную классификацию, предполагает два простых изменения:

- ◆ Добавить в выходной слой функцию активации, а именно, функцию активации `sigmoid`. Она выдает значение от 0,0 до 1,0, представляющее собой вероятность

того, что входные данные принадлежат положительному классу. Для того чтобы вспомнить, что делает сигмоидная функция, обратитесь к обсуждению логистической регрессии в *главе 3*.

- ♦ Изменить функцию потерь на `binary_crossentropy` (<https://oreil.ly/Kid2S>), которая предназначена для бинарных классификаторов. Соответственно, измените метрику на `'accuracy'`, чтобы вычисленная по функции потерь точность фиксировалась в возвращаемом функцией `fit` объекте истории.

Вот сеть, предназначенная для выполнения бинарной классификации, а не регрессии:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```
model = Sequential()
model.add(Dense(512, activation='relu', input_dim=3))
model.add(Dense(512, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

Вот и всё. Всё, что требуется для создания исполняющей роль бинарного классификатора нейронной сети. Вы по-прежнему вызываете `fit` для обучения сети и используете возвращаемый объект истории для построения графика точности обучения и проверки, чтобы определить, в течение достаточного ли количества эпох вы обучили сеть и насколько хорошо она соответствует данным.



Что такое *бинарная кросс-энтропия* (*перекрестная энтропия*) и как она помогает бинарному классификатору прийти к решению? В процессе обучения функция потерь кросс-энтропии экспоненциально увеличивает штраф за неправильные выходы, чтобы более агрессивно направить веса и смещения в нужную сторону.

Допустим, образец принадлежит к положительному классу (его метка равна 1), и сеть предсказывает, что вероятность того, что это 1, равна 0,9. *Кросс-энтропийная потеря*, известная также как логарифмическая потеря (*log-потеря*, *log loss*), равна $-\log(0.9)$, что составляет 0,04. Если же сеть выдает вероятность 0,1 для той же выборки, то ошибка составляет $-\log(0.1)$, что равно 1. Важно, что если предсказанная вероятность *действительно* неверна, то штраф будет гораздо выше. Если образец равен 1, а сеть говорит, что вероятность того, что он равен 1, составляет всего 0,0001, то потеря кросс-энтропии составит $-\log(0.0001)$, или 4. Кросс-энтропийные потери, по сути, похлопывают оптимизатора по спине, когда он близок к правильному ответу, и бьют его по рукам, когда это не так. Чем хуже предсказание, тем сильнее удар.

Итак, построим нейронную сеть для бинарной классификации, включив в выходной слой один нейрон с активацией `sigmoid` и задав в качестве функции потерь `binary_crossentropy`. Выходным сигналом сети является вероятность от 0,0 до 1,0 того, что входной сигнал относится к положительному классу. Проще не бывает!

Выполнение прогнозирования

Одно из преимуществ нейронной сети заключается в том, что она может легко подстраиваться под нелинейные наборы данных. При этом не нужно пробовать различные алгоритмы обучения, как это приходится делать с обычными моделями машинного обучения; сеть сама является алгоритмом обучения. В качестве примера рассмотрим набор данных на рис. 9.5, в котором каждая точка данных состоит из пары координат $(x; y)$ и принадлежит к одному из двух классов.

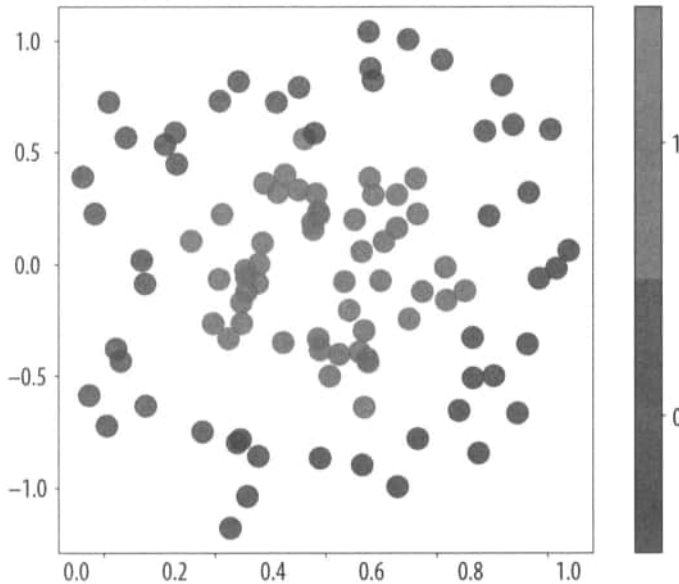


Рис. 9.5. Содержащий два класса нелинейный набор данных

Следующий код обучает нейронную сеть предсказывать класс по координатам x и y точки:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential()
model.add(Dense(128, activation='relu', input_dim=2))
model.add(Dense(1, activation='sigmoid'))
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
hist = model.fit(x, y, epochs=40, batch_size=10, validation_split=0.2)
```

Эта сеть содержит всего один скрытый слой со 128 нейронами, но график точности обучения и валидации показывает, что она очень успешно разделяет классы (рис. 9.6).

После обучения бинарного классификатора можно делать предсказания путем вызова его метода `predict`. Благодаря функции активации `sigmoid` метод `predict` возвращает число от 0,0 до 1,0, представляющее собой вероятность принадлежности

входных данных к положительному классу. В данном примере фиолетовые точки на рис. 9.5 данных представляют отрицательный класс (0), а красные — положительный (1). Здесь сеть должна предсказать вероятность того, что точка данных в точке $(-0,5; 0,0)$ относится к классу красных:

```
model.predict(np.array([[-0.5, 0.0]]))
```

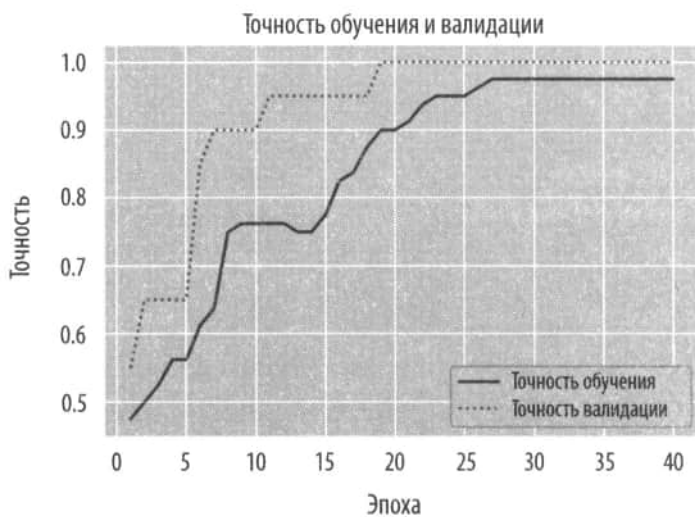


Рис. 9.6. Графики точности обучения и валидации нейронной сети, предсказывающей класс точки

Ответ 0,57, что говорит о том, что $(-0,5; 0,0)$ с большей вероятностью будет красной, чем фиолетовой. Если нужно просто узнать, к какому классу относится точка, можно поступить следующим образом:

```
(model.predict(np.array([[-0.5, 0.0]])) > 0.5).astype('int32')
```

Ответ — 1, что соответствует красному цвету. В старых версиях Keras присутствовал метод `predict_classes`, который делал то же самое без приведения `astype`, но недавно этот метод стал устаревшим и был удален.

Обучение нейронной сети для обнаружения мошенничества с кредитными картами

Давайте обучим нейронную сеть обнаруживать мошенничество с кредитными картами. Для начала скачайте ZIP-файл с набором данных (<https://oreil.ly/uMFFJ>), если вы еще не сделали этого, и извлеките файл `creditcard.csv` из ZIP-архива в подкаталог `Data` вашего блокнота. Это тот же самый файл, который использовался в главах 3 и 6. В нем содержится информация о 284 808 транзакциях по кредитным картам, включая сумму каждой транзакции и метку: 0 — нормальные (законные) транзакции, 1 — мошеннические. Кроме того, он содержит 28 столбцов с именами `V1–V28`, смысл которых был скрыт с помощью анализа главных компонент. Набор

данных сильно несбалансирован и содержит менее 500 примеров мошеннических транзакций.

Теперь загрузим набор данных:

```
import pandas as pd
```

```
df = pd.read_csv('Data/creditcard.csv')
df.head(10)
```

С помощью следующих операторов можно убрать столбец `Time` (время), разделить набор данных на признаки `x` и метки `y`, а также разделить набор данных на два набора: один — для обучения, другой — для тестирования. Вместо того чтобы позволить Keras выполнить разделение за нас, мы сделаем это сами, чтобы впоследствии прогнать тестовые данные через сеть и использовать матрицу ошибок для анализа результатов:

```
from sklearn.model_selection import train_test_split
```

```
x = df.drop(['Time', 'Class'], axis=1)
y = df['Class']
```

```
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, stratify=y, random_state=0)
```

Создадим нейронную сеть для бинарной классификации:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```
model = Sequential()
model.add(Dense(128, activation='relu', input_dim=29))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
model.summary()
```

Следующим шагом является обучение модели. Обратите внимание на передаваемый в `fit` параметр `validation_data`, который использует выделенные из большого набора данных тестовые данные для оценки точности модели в процессе обучения:

```
hist = model.fit(x_train, y_train, validation_data=(x_test, y_test),
                epochs=10, batch_size=100)
```

Теперь построим графики точности обучения и валидации, используя значения для каждой эпохи в объекте истории:

```
%matplotlib inline
import matplotlib.pyplot as plt
import seaborn as sns
sns.set()
```

```

acc = hist.history['accuracy']
val = hist.history['val_accuracy']
epochs = range(1, len(acc) + 1)
plt.plot(epochs, acc, '-', label='Точность обучения')
plt.plot(epochs, val, ':', label='Точность валидации')
plt.title('Точность обучения и валидации')
plt.xlabel('Эпоха')
plt.ylabel('Точность')
plt.legend(loc='lower right')
plt.plot()

```

Для меня результат выглядел так, как показано на рис. 9.7. Помните, что ваши результаты будут отличаться благодаря присущей обучению нейронных сетей случайности.

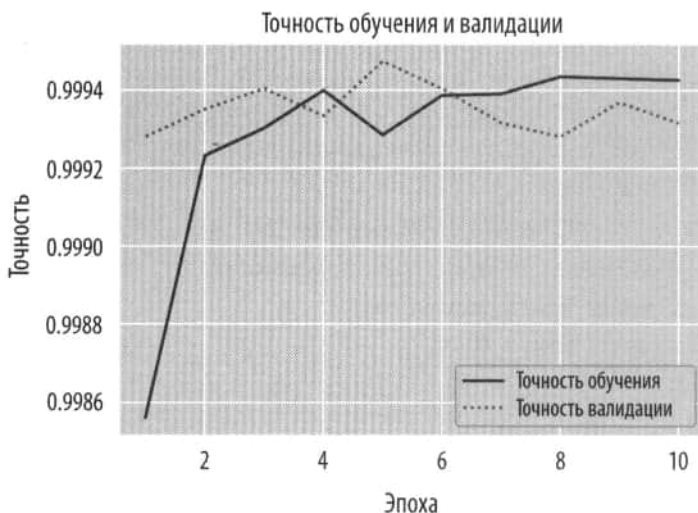


Рис. 9.7. Графики точности обучения и валидации нейронной сети, обнаруживающей мошенничество с кредитными картами

На первый взгляд, точность валидации (около 0,9994) очень высока. Однако следует помнить, что набор данных несбалансирован. Мошеннические транзакции составляют менее 0,2% всех образцов, а это значит, что модель может просто предположить, что каждая транзакция является легитимной, и сделать это правильно примерно в 99,8% случаев. Используйте матрицу ошибок для визуализации работы модели во время тестирования на данных, которым она не обучалась:

```
from sklearn.metrics import ConfusionMatrixDisplay as cmd
```

```

sns.reset_orig()
y_predicted = model.predict(x_test) > 0.5
labels = ['Легитимная', 'Мошенническая']

```

```
cmd.from_predictions(y_test, y_predicted, display_labels=labels,
                     cmap='Blues', xticks_rotation='vertical')
```

Вот что получилось в моем случае (рис. 9.8).



Рис. 9.8. Матрица ошибок нейронной сети, обнаруживающей мошенничество с кредитными картами

Вероятно, ваши результаты будут отличаться. Обучите модель несколько раз, и каждый раз результаты будут другими. В данном случае модель правильно определила 56 859 транзакций как легитимные, а ошибочно классифицировала легитимные транзакции всего 5 раз. Это означает, что легитимные транзакции классифицируются правильно примерно в 99,99% случаев. В то же время более 73% мошеннических транзакций были распознаны с помощью модели. Это вполне приемлемо, поскольку обслуживающие кредитные карты компании скорее допустят 100 мошеннических операций, чем отклонят одну легитимную транзакцию.



Для обучения и оценки точности нейронной сети специалисты по исследованию данных часто используют не два, а три набора данных: обучающий набор данных для обучения, валидационный набор данных для проверки сети (и оценки ее прогресса) в процессе обучения и тестовый набор данных для оценки точности сети по завершении обучения.

В предыдущем примере для проверки и тестирования использовался один и тот же набор данных — 20% отделенных от исходного набора данных с помощью команды `train_test_split`. На первый взгляд, это нормально, поскольку валидационные данные не используются для корректировки весов и смещений сети в процессе обучения. Однако если вы действительно хотите быть уверенными в точности сети, никогда не помешает протестировать ее на третьем наборе данных, не использо-

вавшемся для обучения или валидации. В реальном мире окончательной проверкой точности модели глубокого обучения является ее работа с данными, которые она никогда раньше не видела.

Заключительное замечание по рассмотренному примеру связано с дополнительным параметром, который можно передать методу `fit` и который особенно полезен при работе с несбалансированными наборами данных. В качестве эксперимента попробуйте заменить вызов метода `fit` следующим образом:

```
hist = model.fit(x_train, y_train, validation_data=(x_test, y_test),
                epochs=10, batch_size=100,
                class_weight={ 0: 1.0, 1: 0.01 })
```

Затем снова прогоните блокнот от начала до конца. По всей вероятности, результирующая матрица ошибок покажет ноль (или, в крайнем случае, одну-две) неправильно классифицированных легитимных транзакций, но и процент правильно идентифицированных мошеннических транзакций уменьшится. Параметр `class_weight` в данном примере говорит модели, что для вас гораздо важнее правильно классифицировать легитимные транзакции, чем правильно идентифицировать мошеннические. Вы можете поэкспериментировать с различными весами для двух классов и найти баланс, который наилучшим образом соответствует бизнес-требованиям, побудившим вас к созданию модели.

Мультиклассовая классификация с помощью нейронных сетей

Перед нами простой бинарный классификатор, который принимает пару входных данных, имеет скрытый слой из 128 нейронов и выдает значение от 0,0 до 1,0, представляющее собой вероятность принадлежности входа к положительному классу:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```
model = Sequential()
model.add(Dense(128, activation='relu', input_dim=2))
model.add(Dense(1, activation='sigmoid'))
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

Ключевыми элементами являются выходной слой с одним нейроном, которому назначена функция активации `sigmoid`, и функция потерь `binary_crossentropy`. Три простые модификации позволяют перепрофилировать эту сеть для мультиклассовой классификации:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```

model = Sequential()
model.add(Dense(128, activation='relu', input_dim=2))
model.add(Dense(4, activation='softmax'))
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

```

Изменения заключаются в следующем.

- ◆ Выходной слой содержит *по одному нейрону на класс*, а не только один нейрон. Если набор данных содержит четыре класса, то выходной слой содержит четыре нейрона. Если набор данных содержит 10 классов, то выходной слой содержит 10 нейронов. Каждый нейрон соответствует одному классу.
- ◆ В выходном слое используется функция активации **softmax** (<https://oreil.ly/sdtB9>), а не функция активации **sigmoid**. Каждый нейрон выходного слоя дает вероятность для соответствующего класса, и благодаря функции **softmax** сумма всех вероятностей равна 1,0.
- ◆ Функция потерь — **sparse_categorical_crossentropy** (<https://oreil.ly/WpSp5>). В процессе обучения эта функция потерь экспоненциально штрафует за ошибку в вероятностях, предсказанных многоклассовым классификатором, подобно тому, как это делает **binary_crossentropy** для бинарных классификаторов.

Определив сеть, вы вызываете метод `fit` для ее обучения и метод `predict` для выполнения прогнозов. Поскольку пример состоит из тысячи слов, давайте применим нейронную сеть к двумерному набору данных, состоящему из четырех классов (рис. 9.9).

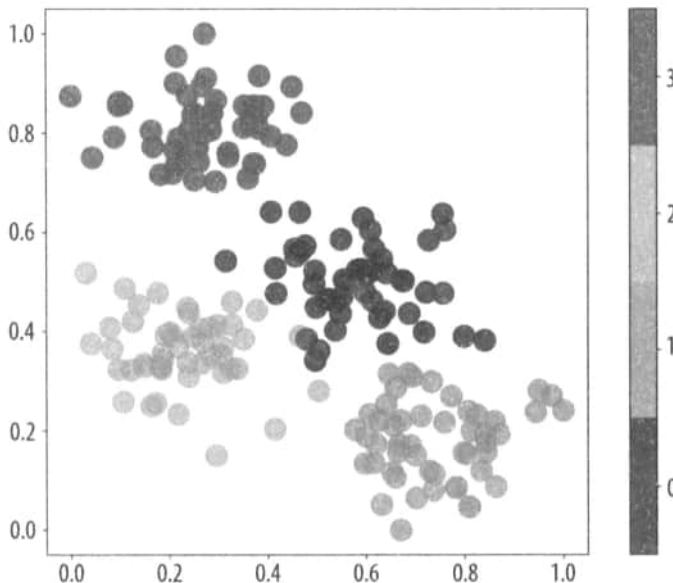


Рис. 9.9. Нелинейный набор данных, содержащий четыре класса

Следующий код обучает нейронную сеть предсказывать класс по координатам x и y точки:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

model = Sequential()
model.add(Dense(128, activation='relu', input_dim=2))
model.add(Dense(4, activation='softmax'))
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

```
hist = model.fit(x, y, epochs=40, batch_size=10, validation_split=0.2)
```

Графики точности обучения и валидации показывают, что сеть без особых проблем разделяет классы (рис. 9.10).

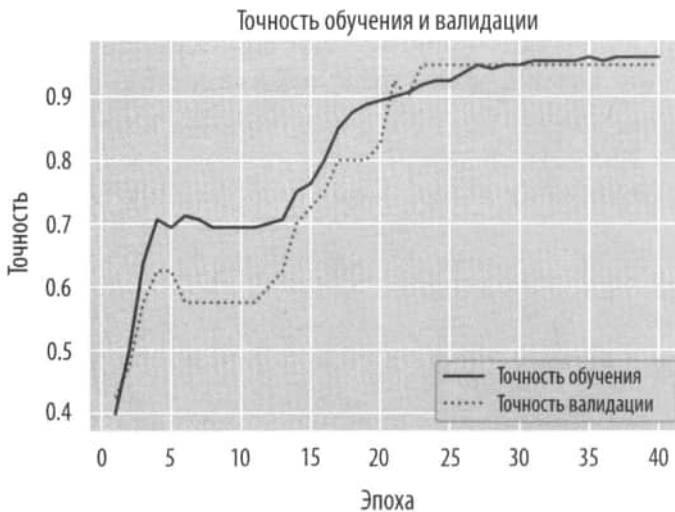


Рис. 9.10. Графики точности обучения и валидации нейронной сети для двумерного набора данных, состоящего из четырех классов

Прогнозирование осуществляется путем вызова метода `predict` классификатора. Для каждого входных данных метод `predict` возвращает массив вероятностей — по одной на класс. Предсказанным классом считается тот, которому присвоена наибольшая вероятность. В рассматриваемом примере фиолетовые точки данных (см. рис. 9.9) представляют класс 0, светло-голубые — класс 1, салатовые — класс 2, а красные — класс 3. Предложим сети классифицировать точку, расположенную в координатах (0,2; 0,8):

```
model.predict(np.array([[0.2, 0.8]]))
```

Результатом будет массив из четырех вероятностей, соответствующих классам 0, 1, 2 и 3, в таком порядке:

```
[2.1877741e-02, 5.3804164e-05, 5.0240371e-02, 9.2782807e-01]
```

Сеть предсказала, что вероятность того, что точка с координатами (0,2; 0,8) соответствует классу 0 составляет 2%, классу 1 — 0%, классу 2 — 5%, классу 3 — 93%. Если посмотреть на график, то это кажется разумным ответом.

Если необходимо просто узнать, к какому классу относится точка, то это можно сделать следующим образом:

```
np.argmax(model.predict(np.array([[0.2, 0.8]]))), axis=1)
```

Ответом будет 3, что соответствует красному цвету. В старых версиях Keras присутствовал метод `predict_classes`, который делал то же самое без вызова `argmax`, но в настоящее время этот метод признан устаревшим и удален.



В состав Keras также входит функция потерь `categorical_crossentropy` (<https://oreil.ly/Ue2bC>), часто используемая для мультиклассовой классификации. Она работает аналогично `sparse_categorical_crossentropy`, но требует, чтобы метки были однократно закодированы. Например, вместо того чтобы передавать метку в столбце, содержащем значения от 0 до 3, нужно передать ей четыре столбца, содержащие нули и единицы. Для выполнения такого кодирования Keras предоставляет служебную функцию `to_categorical` (<https://oreil.ly/yt6hu>). Однако если вы используете `sparse_categorical_crossentropy`, можете использовать столбец меток как есть.

Обучение нейронной сети для распознавания лиц

В главе 5 были описаны шаги по обучению машины опорных векторов для распознавания лиц. Давайте обучим нейронную сеть тому же самому. Мы будем использовать тот же набор данных, что и раньше: набор `Labeled Faces in the Wild` (LFW, <https://oreil.ly/vkEpV>), содержащий более 13 000 изображений лиц известных людей и встроенный в Scikit в качестве примера. Напомним, что из более чем 5000 человек, представленных в наборе, 1680 имеют два и более изображений лиц, и только пятеро — 100 и более. Мы установим минимальное количество лиц на одного человека равным 100, что означает, что будут импортированы пять наборов лиц, соответствующих пяти известным людям.

Начнем с создания нового блокнота Jupyter и загрузки набора данных с помощью следующих операторов:

```
import pandas as pd
from sklearn.datasets import fetch_lfw_people
```

```
faces = fetch_lfw_people(min_faces_per_person=100, slice_=None)
faces.images = faces.images[:, 35:97, 39:86]
```

```

faces.data = faces.images.reshape(faces.images.shape[0],
                                   faces.images.shape[1] * faces.images.shape[2])
image_count = faces.images.shape[0]
image_height = faces.images.shape[1]
image_width = faces.images.shape[2]
class_count = len(faces.target_names)

```

Всего было загружено 1140 изображений лиц. После обрезки каждое из них имеет размер 47×62 пиксела. Используйте следующий код для отображения первых 24 изображений из набора данных и людей, которым принадлежат эти лица:

```

%matplotlib inline
import matplotlib.pyplot as plt

fig, ax = plt.subplots(3, 8, figsize=(18, 10))
for i, axi in enumerate(ax.flat):
    axi.imshow(faces.images[i], cmap='gist_gray')
    axi.set(xticks=[], yticks=[], xlabel=faces.target_names[faces.target[i]])

```

Проверьте баланс в наборе данных путем построения гистограммы, показывающей, сколько изображений лиц было импортировано для каждого человека:

```

from collections import Counter
import seaborn as sns
sns.set()

counts = Counter(faces.target)
names = {}

for key in counts.keys():
    names[faces.target_names[key]] = counts[key]

df = pd.DataFrame.from_dict(names, orient='index')
df.plot(kind='bar')

```

В наборе данных гораздо больше изображений Джорджа Буша-младшего, чем кого-либо другого. Классификационные модели лучше всего обучаются на сбалансированных наборах данных. Используйте следующий код для уменьшения набора данных до 100 изображений каждого человека:

```

import numpy as np

mask = np.zeros(faces.target.shape, dtype=bool)

for target in np.unique(faces.target):
    mask[np.where(faces.target == target)[0][:100]] = 1

```

```
x_faces = faces.data[mask]
y_faces = faces.target[mask]
x_faces.shape
```

`x_faces` содержит 500 изображений лиц, а `y_faces` — соответствующие им метки: 0 для Колина Пауэлла, 1 для Дональда Рамсфелда и т. д.

Следующий шаг — разделение данных на обучающие и валидационные. Мы выделим 20% данных для тестирования, позволим Keras использовать их для проверки модели в процессе обучения, а затем будем применять их для оценки результатов с помощью матрицы ошибок:

```
from sklearn.model_selection import train_test_split
```

```
x_train, x_test, y_train, y_test = train_test_split(
    face_images, y_faces, train_size=0.8, stratify=y_faces, random_state=0)
```



Обычно все значения пикселей делят на 255, поскольку нейронные сети часто лучше обучаются на нормализованных данных, а деление на 255 — это простой способ нормализации значений пикселей. Нередко вместо этого используется класс `StandardScaler` из Scikit для применения единичной дисперсии. В данном примере деление на 255 не требуется, поскольку пиксели в наборе данных LFW уже были нормализованы таким образом.

Создайте нейронную сеть, содержащую один скрытый слой с 512 нейронами. В качестве функции потерь используйте `sparse_categorical_crossentropy`, а в качестве функции активации на выходном слое — `softmax`, поскольку это задача мультиклассовой классификации:

```
from tensorflow.keras.layers import Dense
from tensorflow.keras.models import Sequential
```

```
model = Sequential()
model.add(Dense(512, activation='relu', input_shape=(image_width * image_height,)))
model.add(Dense(class_count, activation='softmax'))
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
model.summary()
```

Затем обучите сеть:

```
hist = model.fit(x_train, y_train, validation_data=(x_test, y_test),
                epochs=100, batch_size=20)
```

Постройте графики точности обучения и валидации:

```
val_acc = hist.history['val_accuracy']
epochs = range(1, len(acc) + 1)

plt.plot(epochs, acc, '-', label='Точность обучения')
plt.plot(epochs, val_acc, ':', label='Точность валидации')
```

```
plt.title('Точность обучения и валидации')
plt.xlabel('Эпоха')
plt.ylabel('Точность')
plt.legend(loc='lower right')
plt.plot()
```

В завершение используйте матрицу ошибок для визуализации результатов работы сети на тестовых данных:

```
from sklearn.metrics import ConfusionMatrixDisplay as cmd
```

```
sns.reset_orig()
y_pred = model.predict(x_test)
fig, ax = plt.subplots(figsize=(5, 5))
ax.grid(False)
cmd.from_predictions(y_test, y_pred.argmax(axis=1),
                    display_labels=faces.target_names, colorbar=False,
                    cmap='Blues', xticks_rotation='vertical', ax=ax)
```

Сколько раз модель правильно идентифицировала Джорджа Буша-младшего? Сколько раз она идентифицировала его как другого человека? Будет ли сеть столь же точной при 128 нейронах в скрытом слое, как и при 512?

Отсев

Цель любой модели машинного обучения — делать точные предсказания. В идеальном мире на поздних этапах обучения нейронной сети разрыв между точностью обучения и точностью проверки был бы близок к 0. В реальном мире такое случается редко. Точность обучения рассмотренной в предыдущем разделе модели приближается к 100%, но точность валидации, скорее всего, находится в пределах 80–85%. Это означает, что модель обобщает не так хорошо, как хотелось бы. Она очень хорошо усвоила обучающие данные, но когда ей были представлены данные, с которыми она раньше не сталкивалась (данные для проверки), она показала низкую эффективность. Это может быть признаком того, что модель слишком переобучена. В конечном итоге важна не точность обучения, а то, насколько точно модель обрабатывает новые данные.

Одним из способов борьбы с избыточным обучением является уменьшение глубины сети, ширины отдельных слоев или того и другого. Меньшее количество нейронов означает меньшее количество обучаемых параметров, а меньшее количество параметров не позволяет сети слишком плотно приспосабливаться к обучающим данным.

Другим способом защиты от перебора является введение в сеть функции *отсева* (dropout). Отсев случайным образом разрывает связи между слоями во время обучения, чтобы сеть не смогла слишком хорошо усвоить обучающие данные. Это по-

хоже на чтение книги с пропуском каждой второй страницы в надежде, что вы усвоите высокоуровневые концепции и не запутаетесь в деталях. Метод Dropout был представлен в 2014 году в статье "Dropout: простой способ предотвратить переобучение нейронных сетей" ("Dropout: A Simple Way to Prevent Neural Networks from Overfitting", <https://oreil.ly/yqLRk>).

Класс Dropout (<https://oreil.ly/O2crM>) в Keras реализует добавление отсева в сеть. Для демонстрации вернемся к примеру из предыдущего раздела и переопределим сеть следующим образом:

```
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.models import Sequential
```

```
model = Sequential()
model.add(Dense(512, activation='relu', input_shape=(image_width * image_height,)))
model.add(Dropout(0.2))
model.add(Dense(class_count, activation='softmax'))
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

Теперь снова обучите сеть и постройте график точности обучения и валидации. Вот как это получилось у меня (рис. 9.11).



Рис. 9.11. Графики точности обучения и валидации нейронной сети для распознавания лиц с применением отсева

Разрыв не сильно сократился (если вообще сократился), но иногда добавление отсева таким образом *позволяет* повысить точность валидации. Ключевой оператор в этом примере — `model.add(Dropout(0.2))`, который добавляет слой Dropout, случайным образом отсеивающий (игнорирующий) 20% связей между нейронами скрытого слоя и нейронами выходного слоя в каждом проходе обратного распространения.

Можно быть более агрессивным и отбрасывать большее количество связей (например, увеличить 0,2 до 0,4), но при этом достичь хорошего результата. Обратите внимание, что если использовать слишком агрессивный коэффициент отсева (например, 0,5), то, скорее всего, придется обучать модель большее количество эпох. Если усложнить процесс обучения, то и времени на него уйдет больше.

На практике единственный способ узнать, улучшит ли отсев способность модели к обобщению, — это экспериментировать. Кроме использования различных процентов отсева, можно попробовать ввести отсев между двумя или более слоями в надежде найти подходящую комбинацию.

Сохранение и загрузка моделей

В главе 7 вы узнали, как сериализовать (сохранить) обученную модель Scikit и загрузить ее в клиентское приложение. Это же требование применимо и к нейронным сетям: необходимо иметь возможность сохранить обученную сеть и загрузить ее позже, чтобы использовать в работе.

Весы и смещения модели можно получить с помощью метода `get_weights` (<https://oreil.ly/7twR>) в Keras, а восстановить их — посредством `set_weights` (<https://oreil.ly/JyNu9>). Однако сохранение обученной модели для последующего ее воссоздания требует дополнительных действий. В частности, необходимо сохранить архитектуру сети: количество и типы слоев, количество нейронов в каждом слое, функции активации, используемые в каждом слое, и т. д. К счастью, все это требует всего одной строки кода. Эта строка отличается в зависимости от того, в каком из двух форматов вы хотите сохранить модель:

```
model.save('my_model.h5')           # Сохранить модель в формате H5 от Keras
model.save('my_model')               # Сохранить модель в формате TensorFlow
```

Загрузка сохраненной модели также проста:

```
from tensorflow.keras.models import load_model
```

```
model = load_model('my_model.h5')   # Загрузка модели,
                                     # сохраненной в формате H5
model = load_model('my_model')       # Загрузка модели,
                                     # сохраненной в формате TensorFlow
```

При сохранении модели в формате H5 создается один файл .h5, содержащий всю модель. При сохранении в собственном формате TensorFlow, также известном как формат SavedModel (<https://oreil.ly/hI3Rt>), создается серия файлов и подкаталогов, содержащих сериализованную модель. Google рекомендует использовать последний формат, хотя все еще часто встречается формат H5 от Keras. Функциональной разницы между ними нет, но файлы .h5 могут быть прочитаны только написанными на Python приложениями Keras, в то время как сохраненные в формате SavedModel модели могут быть загружены другими фреймворками. Например, приложения, написанные на C# с помощью ML.NET от Microsoft, могут загружать модели, со-

храненные в формате `SavedModel`, независимо от языка программирования, на котором была создана модель.



Формат H5 изначально был разработан для того, чтобы модели Keras можно было сохранять независимо от используемого в качестве бэкенда фреймворка глубокого обучения. Keras по-прежнему доступен в самостоятельной версии, поддерживающей бэкенды, отличные от TensorFlow (в частности, CNTK и Theano), но эти фреймворки уже устарели — они больше не разрабатываются — и сегодня редко используются, кроме как в старых моделях. Версия Keras, встроенная в TensorFlow, поддерживает *только* бэкенды TensorFlow.

После загрузки сохраненной модели она действует один в один с оригиналом. Например, выполняемые ею прогнозы идентичны сделанным исходной моделью. Можно даже продолжить обучение модели и пропустить через нее дополнительные обучающие выборки. Это подчеркивает одно из основных отличий нейронных сетей от традиционных моделей машинного обучения, построенных с помощью Scikit. Поскольку состояние сети определяется ее весами и смещениями, а загрузка модели восстанавливает веса и смещения, нейронные сети по своей сути поддерживают инкрементное обучение (известное также как *непрерывное обучение*), что позволяет им становиться более интеллектуальными с течением времени. Большинство моделей Scikit так не могут, поскольку сериализация моделей не сохраняет накопленное в процессе обучения внутреннее состояние.

Повторим: вы можете прогнать через нейронную сеть миллион обучающих образцов, сохранить ее, загрузить, прогнать через нее еще миллион обучающих образцов, и сеть продолжит работу с того же места, на котором остановилась. Результаты будут идентичны тому, как если бы вы изначально прогнали через сеть 2 млн обучающих образцов, за исключением небольших различий, обусловленных всегда присутствующей обучению случайностью.

Обратные вызовы Keras

Когда вы обучаете нейронную сеть и она достигает пика точности проверки, этот пик трудно зафиксировать. Вместо того чтобы постепенно выравниваться в последующие эпохи, точность проверки может снижаться или колебаться между пиками и долинами. Учитывая стохастическую (случайную) природу нейронных сетей, если отметить эпоху, в которой была достигнута максимальная точность проверки, и провести повторное обучение ровно столько эпох, то во второй раз вы не получите тех же результатов. Как же обучить *именно то* количество эпох, которое необходимо для получения наилучшей (наиболее точной) сети?

Эlegantным решением является API обратных вызовов (callbacks) Keras (<https://oreil.ly/RP1Ht>), позволяющий создавать функции обратного вызова, выполняемые в различные моменты обучения, например в конце каждой эпохи, и способные изменять и даже останавливать процесс обучения. Приведем пример созда-

ния дочернего класса `StopCallback`, который наследуется от класса `Callback` (<https://oreil.ly/CULyB>) в Keras. Дочерний класс реализует функцию `on_epoch_end`, вызываемую в конце каждой эпохи обучения и останавливающую обучение, если точность валидации достигает 95%:

```
from tensorflow.keras.callbacks import Callback
```

```
class StopCallback(Callback):
```

```
    accuracy_threshold = None
```

```
    def __init__(self, threshold):
```

```
        self.accuracy_threshold = threshold
```

```
    def on_epoch_end(self, epoch, logs=None):
```

```
        if (logs.get('val_accuracy') >= self.accuracy_threshold):
```

```
            self.model.stop_training = True
```

```
callback = StopCallback(0.95)
```

```
model.fit(x, y, validation_split=0.2, epochs=100, batch_size=20, callbacks=[callback])
```

```
model.save('best_model.h5')
```

Обратите внимание на порог точности валидации (0,95), переданный в конструктор `StopCallback`. Вызов функции `fit` предполагает обучение сети в течение 100 эпох, но если точность валидации достигает 0,95 до этого времени, обучение останавливается. Заключительный оператор сохраняет достигшую этой точности модель.

`on_epoch_end` — это одна из нескольких функций, которые можно реализовать в классах-наследниках `Callback` для получения обратного вызова (callback) при достижении заданной контрольной точки в процессе обучения. Другие функции включают в себя `on_epoch_begin`, `on_train_begin`, `on_train_end`, `on_train_batch_begin` и `on_train_batch_end`. Полный список с примерами приведен в разделе "Witching Your Own Callbacks" ("Написание собственных обратных вызовов", <https://oreil.ly/X7lf4>) документации Keras.

Помимо базового класса `Callback`, на основе которого можно создавать свои классы обратных вызовов, Keras предоставляет несколько собственных классов обратных вызовов. Один из них — класс `EarlyStopping`, позволяющий без написания большого количества кода прекратить обучение по заданному критерию, например по снижению точности проверки или увеличению потерь при обучении. В следующем примере обучение прекращается досрочно, если точность проверки не улучшается в течение пяти эпох подряд (`patience=5`). При остановке обучения веса и смещения сети автоматически восстанавливаются до тех значений, которые были при достижении пика точности валидации в последние пять эпох (`restore_best_weights=True`):

```
from tensorflow.keras.callbacks import EarlyStopping
```

```
callback = EarlyStopping(monitor='val_accuracy', patience=5,
```

```
                        restore_best_weights=True)
```

```
model.fit(x, y, validation_split=0.2, epochs=100, batch_size=20, callbacks=[callback])
```

Остановка процесса обучения на основе роста потерь при обучении, а не снижения точности валидации в конце каждой эпохи требует незначительного изменения кода:

```
from tensorflow.keras.callbacks import EarlyStopping

callback = EarlyStopping(monitor='loss', patience=5, restore_best_weights=True)
model.fit(x, y, validation_split=0.2, epochs=100, batch_size=20,
          callbacks=[callback])
```

Обратный вызов, который используется, пожалуй, чаще других, — это `ModelCheckpoint` (<https://oreil.ly/cLMwN>), сохраняющий модель через заданные интервалы времени в процессе обучения или, если задать для параметра `save_best_only` значение `True`, сохраняющий наиболее точную модель. В следующем примере модель обучается в течение 100 эпох и сохраняет в файле `best_model.h5` модель, показавшую наибольшую точность валидации:

```
from tensorflow.keras.callbacks import ModelCheckpoint

callback = ModelCheckpoint(filepath='best_model.h5',
                           monitor='val_accuracy', save_best_only=True)
model.fit(x, y, validation_split=0.2, epochs=100, batch_size=20, callbacks=[callback])
```

Другим часто используемым обратным вызовом является `TensorBoard` (<https://oreil.ly/h3sHp>), который в процессе обучения модели записывает определенную информацию в указанное место в файловой системе. В следующем примере запись ведется в подкаталог `logs` текущего каталога:

```
from tensorflow.keras.callbacks import TensorBoard

callback = TensorBoard(log_dir='logs', histogram_freq=1)
model.fit(x_train, y_train, validation_split=0.2, epochs=100,
          batch_size=20, callbacks=[callback])
```

Для отслеживания точности и потерь, изменений весов и смещений модели, а также других параметров в процессе обучения или после его завершения можно использовать инструмент `TensorBoard`. `TensorBoard` можно запустить из блокнота `Jupyter` и указать ему на подкаталог `logs` с помощью следующей команды:

```
%tensorboard --logdir logs
```

Или можно запустить его из командной строки, выполнив ту же команду без знака процента. Затем перейдите в браузере по адресу `http://localhost:6006`, чтобы открыть консоль `TensorBoard` (рис. 9.12). Раздел "Get Started with TensorBoard" ("Начало работы с TensorBoard"; <https://oreil.ly/1Q0G4>) в документации по TensorFlow содержит полезный учебник по основам работы с `TensorBoard`. Это незаменимый инструмент в руках профессионалов, особенно при обучении сложных моделей, для полного обучения которых требуются часы, дни и даже недели.

Другие классы обратных вызовов Keras включают `LearningRateScheduler` (<https://oreil.ly/OMEMh>) для регулировки скорости обучения в начале каждой эпохи и `CSVLogger` (<https://oreil.ly/BjUny>) для записи результатов каждой эпохи обуче-

ния в CSV-файл. Полный список обратных вызовов приведен в документации API (<https://oreil.ly/Mm3fY>). Кроме того, обратите внимание, что параметр `callbacks` метода `fit` представляет собой список Python, и это означает, что при обучении модели можно указать несколько обратных вызовов. Например, можно использовать один обратный вызов для остановки обучения при выполнении определенных условий, а другой — для регистрации показателей обучения в CSV-файле.

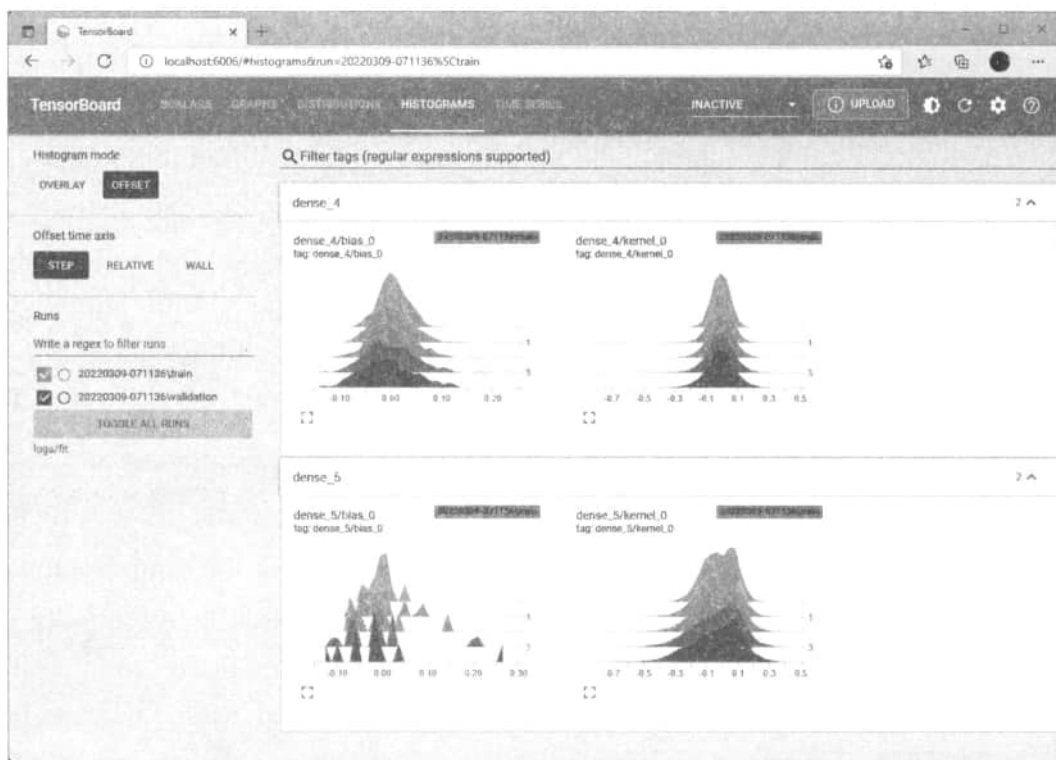


Рис. 9.12. TensorBoard, отображающий результаты обучения нейронной сети

Резюме

Keras и TensorFlow — это широко применяемые фреймворки с открытым исходным кодом, позволяющие строить, обучать, сохранять, загружать и использовать (делать прогнозы) нейронные сети. Нейронные сети можно строить на различных языках программирования, используя собственные API TensorFlow, но Keras абстрагирует эти API и делает глубокое обучение гораздо более доступным. Приложения Keras написаны на языке Python.

Нейронные сети, как и традиционные модели машинного обучения, могут использоваться для решения задач регрессии и классификации. Выполняющая регрессию сеть имеет один нейрон в выходном слое без функции активации; на выходе сеть

представляет число с плавающей точкой. Сеть, выполняющая бинарную классификацию, также имеет один нейрон в выходном слое, но функция активации `sigmoid` обеспечивает получение на выходе значения от 0,0 до 1,0, представляющего собой *вероятность* того, что входные данные относятся к положительному классу. Для мультиклассовой классификации количество нейронов в выходном слое равно количеству классов, которые может предсказать сеть. Функция активации `softmax` преобразует исходные присвоенные выходным нейронам значения в массив вероятностей для каждого класса.

Если вы обнаружили, что нейронная сеть слишком плотно подстраивается под обучающие данные, бороться с переобучением и повысить способность сети к обобщению поможет снижение сложности сети: уменьшение числа слоев, числа нейронов в отдельных слоях или того и другого. Другой подход заключается в добавлении в сеть отсева. Он целенаправленно снижает способность сети к обучению на обучающих данных путем случайного игнорирования подмножества связей между слоями при обновлении весов и смещений.

API обратных вызовов Keras позволяет настраивать процесс обучения. Например, обрабатывая обратные вызовы, возникающие в конце каждой эпохи обучения, можно проверить точность модели и прекратить обучение, если она достигла приемлемого уровня. Keras также включает простой и удобный API для сохранения и загрузки обученных моделей. Это необходимо для практической реализации обучаемых моделей — их внедрения в производство и использования полученных в процессе обучения прогностических возможностей.

Описанная в этой главе модель распознавания лиц превысила 80% в точности валидации, но современные модели глубокого обучения часто достигают 99% точности на том же наборе данных. Не стоит удивляться, если вы узнаете, что глубокое обучение — это не только многослойные перцептроны. В *главе 11* мы подробно рассмотрим распознавание лиц, но сначала изучим другой тип нейронных сетей, особенно хорошо подходящих для решения задач компьютерного зрения. Такая нейронная сеть называется сверточной (конволюционной), и именно ей посвящена *глава 10*.

Классификация изображений с помощью сверточных нейронных сетей

Компьютерное зрение (<https://oreil.ly/SKGgL>) — это направление глубокого обучения, в котором компьютеры распознают информацию на изображениях. В реальном мире это позволяет идентифицировать объекты на фотографиях, удалять неприемлемые изображения с сайтов социальных сетей, подсчитывать количество автомобилей в очереди на пункте оплаты и распознавать лица на фотографиях. Модели компьютерного зрения можно даже объединить с моделями обработки естественного языка (natural language processing, NLP) для создания подписей к фотографиям. Во время отпуска я сделал снимок и попросил службу компьютерного зрения Azure (Computer Vision service, <https://oreil.ly/IRCW5>) сделать к нему подпись. Результат показан на рис. 10.1. Это довольно удивительно, учитывая, что вмешательство человека не потребовалось.



Рис. 10.1. "Водоем с причалом и зданием на заднем плане" — Azure AI

В последние годы область компьютерного зрения развивается быстрыми темпами, в основном благодаря *сверточным нейронным сетям*, известным также как CNN (convolutional neural networks) или ConvNets. В 2012 году восьмислойная CNN под названием AlexNet (<https://oreil.ly/JCNBM>) превзошла традиционные модели машинного обучения, участвовавшие в ежегодном конкурсе ImageNet Large Scale Visual Recognition Challenge (ILSVRC, <https://oreil.ly/8OK4o>), достигнув ошибки в 15,3% при идентификации объектов на фотографиях. В 2015 году состоящая из 152 слоев модель ResNet-152 (<https://oreil.ly/4rBgb>) победила в этом соревновании с ошибкой всего в 3,5%, что превосходит способность человека классифицировать изображения, представленные на конкурсе.

Магия CNN заключается в том, что они рассматривают изображения как *изображения*, а не просто массивы пиксельных значений. Для извлечения "признаков" из изображений они используют десятилетиями разрабатывавшуюся технологию сверточных ядер (convolution kernels, <https://oreil.ly/upd6y>), что позволяет им распознавать форму головы кошки или контур хвоста собаки. Кроме того, они легко создаются с помощью Keras и TensorFlow.

Современные CNN, такие как ResNet-152, обучаются с большими затратами на миллионах изображений на графических процессорах, но многое можно сделать и на обычном центральном процессоре (central processor unit, CPU). В этой главе вы узнаете, что такое CNN и как они работают, а также постройте и обучите несколько собственных CNN. Кроме того, вы узнаете, как использовать передовые CNN, открытые для общего пользования такими компаниями, как Google и Microsoft, и как с помощью техники, называемой *трансферным обучением*, перепрофилировать эти CNN для решения специфических задач в конкретной области.

Понимание работы сверточных нейронных сетей

На рис. 10.2 показана топология базовой CNN. Она начинается с одного или нескольких наборов *сверточных слоев* и *пулинговых слоев* (pooling layers). *Сверточные слои* извлекают признаки из изображений, создавая преобразованные изображения, называемые картами признаков, поскольку они выделяют отличительные признаки, такие как формы и контуры. *Слои пулинга* (син.: слои субдискретизации, объединяющие слои) уменьшают размер карты признаков в два раза, что позволяет извлекать признаки при различных разрешениях и делает их менее чувствительными к небольшим изменениям положения. Выходные данные, полученные на последнем слое объединения, сглаживаются до одного измерения и поступают на один или несколько плотных слоев для классификации. Сверточные и слои объединения называют "бутылочными" (от "бутылочного горлышка", bottleneck), поскольку они уменьшают размерность поступающих на их вход изображений. Кроме того, на них приходится большая часть вычислительного времени в процессе обучения.

Сверточные слои (<https://oreil.ly/dc5b7>) извлекают признаки из изображений, пропуская по ним сверточные ядра (или ядра свертки) — та же технология, которая используется в инструментах редактирования изображений для их размытия, по-

вышения резкости и рельефности. Ядро — это просто матрица значений. Обычно она имеет размер 3×3 , но может быть и больше. Для обработки изображения необходимо поместить ядро в левый верхний угол изображения, умножить значения ядра на значения пикселей под ним и вычислить новое значение для центрального пиксела путем суммирования произведений (рис. 10.3). Затем ядро перемещается на один пиксел вправо, и процесс повторяется строка за строкой и столбец за столбцом до тех пор, пока не будет обработано все изображение.

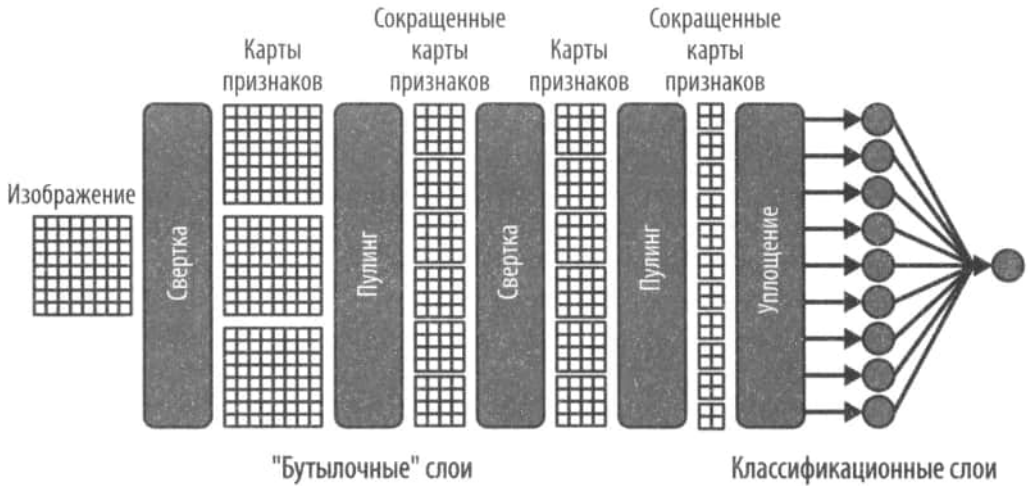


Рис. 10.2. Сверточная нейронная сеть

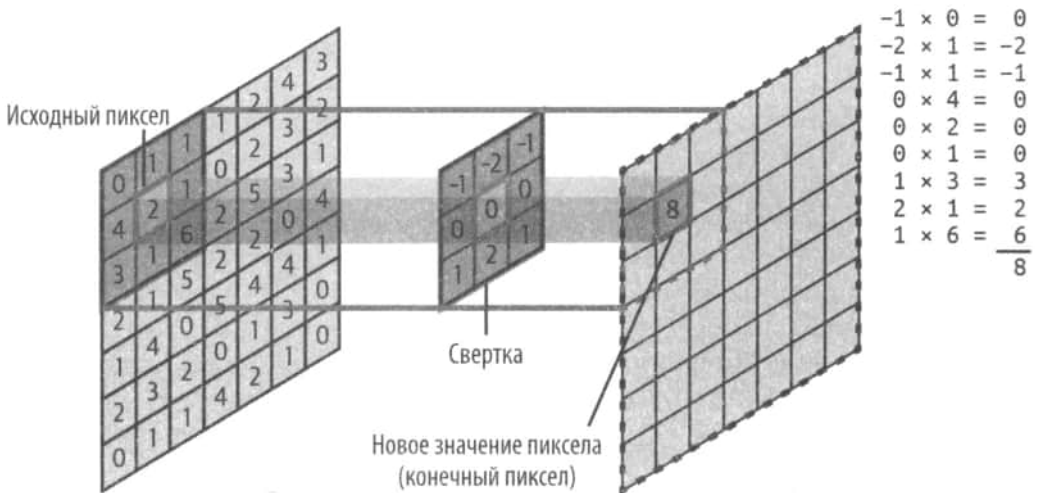


Рис. 10.3. Обработка пикселей изображения с помощью сверточного ядра 3×3

На рис. 10.4 показано, что происходит при применении ядра 3×3 к изображению хот-дога. Это ядро называется *нижним ядром Собеля* и предназначено для обнару-

жения краев путем выделения их таким образом, как будто свет падает снизу. Сверточные слои CNN используют подобные ядра для извлечения признаков, помогающих отличить один класс от другого.

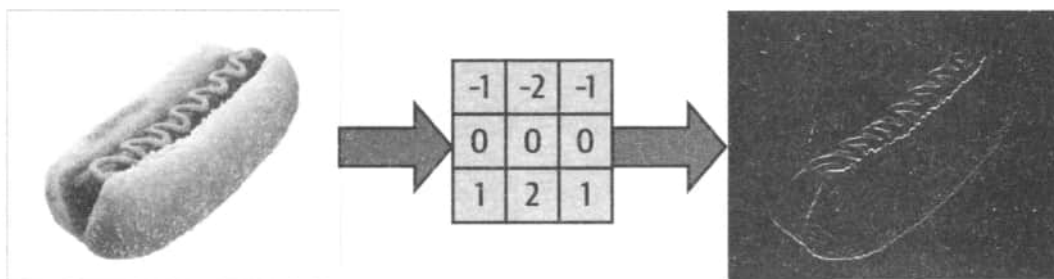


Рис. 10.4. Обработка изображения с помощью нижнего ядра Собеля

Сверточный слой использует не одно ядро для обработки изображений, а множество — иногда 100 и более. Значения ядер не определяются заранее. Они инициализируются случайными значениями, а затем обучаются (корректируются) в процессе обучения CNN, подобно тому как определяются веса, соединяющие нейроны в плотных слоях. Каждое ядро также имеет связанное с ним смещение, как и нейрон в плотном слое. Изображения на рис. 10.5 были сгенерированы первым сверточным слоем обученной CNN. Видно, как различные сверточные ядра позволяют сети по-разному воспринимать одно и то же изображение хот-дога и выделять некоторые особенности, например форму булочки и ленточку горчицы сверху.

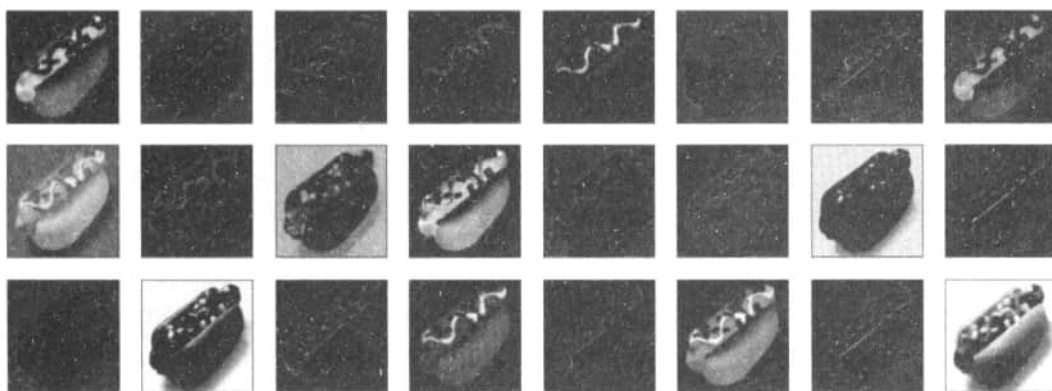


Рис. 10.5. Изображения, формируемые сверточными ядрами в CNN

Объединяющие слои (пулинга, <https://oreil.ly/KW5rW>) уменьшают размер изображений. Наиболее распространенной методикой уменьшения размера является *максимальный пулинг* (max pooling), при котором изображение делится на блоки пикселей размером 2×2 , и в каждом блоке выбирается наибольшее из четырех значений. Альтернативой является *средний пулинг* (average pooling), который усредняет значения в каждом блоке.

На рис. 10.6 показано, как "сжимается" изображение при прохождении через последовательные объединяющие слои. Первая строка получена на первом объединяющем слое, вторая строка — на втором объединяющем слое и т. д.

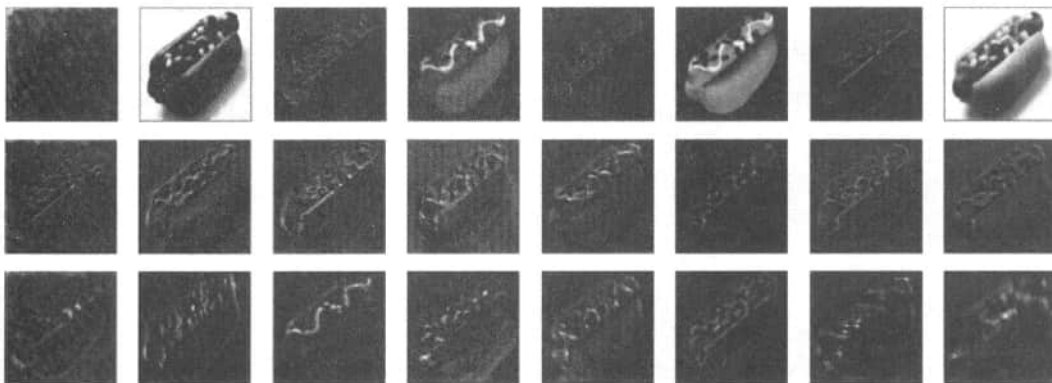


Рис. 10.6. Изображения, сгенерированные объединяющими слоями в CNN

Пулинг не единственный способ уменьшения размера изображения. Хотя подобное встречается реже, уменьшение можно выполнить и без объединяющих слоев, установив для сверточного слоя значение `stride` (шага) равным 2. `Stride` — это количество пикселей, на которое перемещается сверточное ядро при прохождении по изображению. По умолчанию оно равно 1, но при установке значения 2 размер изображения уменьшается вдвое за счет игнорирования каждой второй строки и каждого второго столбца пикселей.

Плотные слои в конце сети классифицируют признаки, извлеченные из "бутылочных" слоев, и называются *классификационными слоями CNN*. Они ничем не отличаются от рассмотренных в *главе 9* многослойных перцептронов. Для бинарной классификации выходной слой содержит один нейрон и использует функцию активации `sigmoid`. Для мультиклассовой классификации выходной слой содержит по одному нейрону на класс и использует функцию активации `softmax`.



Не существует закона, по которому "бутылочные" слои должны объединяться с классификационными слоями. Можно взять полученные на выходе "бутылочных" слоев карты признаков и классифицировать их с помощью метода опорных векторов, а не многослойным перцептроном. Это не так уж и надуманно, как может показаться. В *главе 12* я представлю одну известную модель, которая именно так и работает.

Использование Keras и TensorFlow для построения CNN

Для того чтобы упростить построение классифицирующих изображений CNN, Keras предлагает класс `Conv2D` (<https://oreil.ly/zXy3K>), моделирующий сверточные слои, и класс `MaxPooling2D` (<https://oreil.ly/FreyC>), реализующий слой с максималь-

ным пулингом. Следующие операторы создают CNN с двумя парами сверточных и объединяющих слоев, уплощающим слоем для преобразования выходных данных в одномерный массив для ввода в плотный слой, плотным слоем для классификации извлеченных из "бутылочных" слоев признаков, и выходным слоем softmax для классификации:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense

model = Sequential()
model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))
model.add(MaxPooling2D(2, 2))
model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(MaxPooling2D(2, 2))
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dense(10, activation='softmax'))
```

Первым передаваемым функции Conv2D-параметром является количество включаемых в слой сверточных ядер. Большее количество ядер означает большую подготовленную мощность, аналогично количеству нейронов в плотном слое. Второй параметр — размерность каждого ядра. Иногда можно добиться большей точности при использовании ядер 5×5 , однако ядро такого размера увеличивает время обучения, поскольку требует 25 операций умножения для каждого пиксела по сравнению с девятью для ядра 3×3 . Параметр `input_shape` в первом слое задает размер изображений, подаваемых на вход CNN: в данном случае это одноканальные (градации серого) изображения 28×28 . Все используемые для обучения CNN изображения должны быть одинакового размера.



Conv2D обрабатывает изображения, являющиеся двумерными. В Keras также существует класс Conv1D (<https://oreil.ly/TgzPk>) для обработки одномерных данных и Conv3D (<https://oreil.ly/WshPD>) для трехмерных данных. Первый класс находит применение при обработке текстовых данных и временных рядов (<https://oreil.ly/6gw8k>). Канонические примеры использования Conv3D включают анализ видео и трехмерных медицинских изображений.

Если имеется набор изображений с относительно высокой степенью разделения классов, то вполне реально обучить CNN классифицировать эти изображения на обычном ноутбуке или ПК. В качестве примера можно привести набор данных MNIST (<https://oreil.ly/WAePv>), содержащий 60 000 обучающих изображений с отсканированными рукописными цифрами размером 28×28 пикселей каждое, а также 10 000 тестовых изображений. На рис. 10.7 показаны первые 50 сканов из обучающего набора.

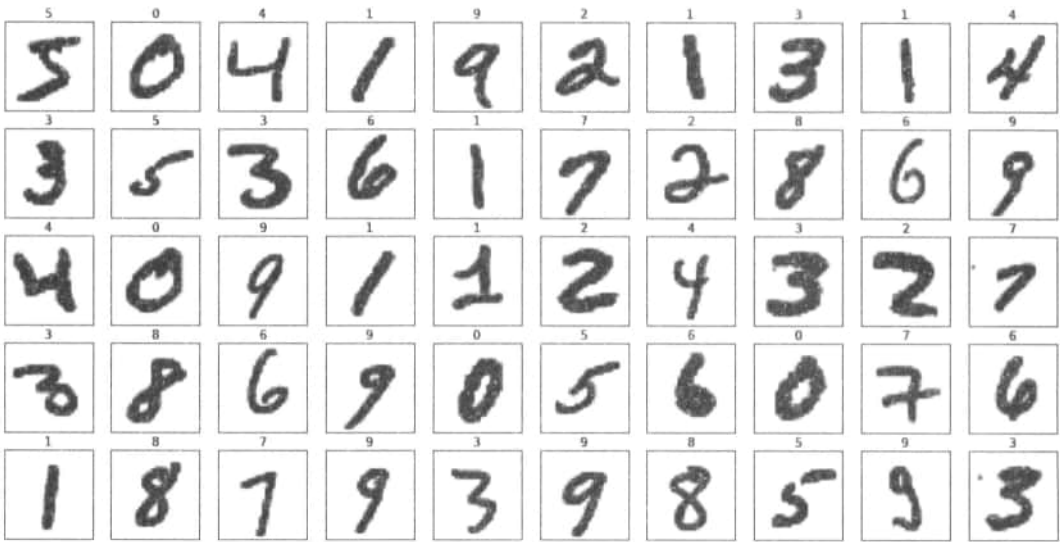


Рис. 10.7. Набор данных MNIST

Давайте обучим CNN распознаванию цифр в наборе данных MNIST, который, что очень удобно, является одним из нескольких встроенных в Keras образцов наборов данных. Начнем с создания нового блокнота Jupyter и использования следующих операторов для загрузки набора данных, преобразования изображений 28×28 в массивы $28 \times 28 \times 1$ (28×28 изображений, содержащих один цветовой канал) и деления значений пикселей на 255 в качестве простой нормализации:

```
from tensorflow.keras.datasets import mnist
```

```
(train_images, y_train), (test_images, y_test) = mnist.load_data()
```

```
x_train = train_images.reshape(60000, 28, 28, 1) / 255
```

```
x_test = test_images.reshape(10000, 28, 28, 1) / 255
```

Далее определим CNN, которая принимает на вход массивы значений пикселей размером $28 \times 28 \times 1$, содержит две пары сверточных и объединяющих слоев и имеет выходной слой softmax с 10 нейронами, поскольку набор данных содержит сканы 10 различных цифр:

```
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Dense, Flatten
```

```
from tensorflow.keras.models import Sequential
```

```
model = Sequential()
```

```
model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))
```

```
model.add(MaxPooling2D(2, 2))
```

```
model.add(Conv2D(64, (3, 3), activation='relu'))
```

```
model.add(MaxPooling2D(2, 2))
```

```
model.add(Flatten())
```

```
model.add(Dense(128, activation='relu'))
```

```
model.add(Dense(10, activation='softmax'))
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
model.summary(line_length=80)
```

На рис. 10.8 показан результат вызова `summary` в последней строке. Результат говорит многое о том, как эта CNN обрабатывает изображения. Каждый объединяющий слой уменьшает размер изображения в два раза, а каждый сверточный слой уменьшает высоту и ширину изображения на два пиксела. Почему так происходит? По умолчанию сверточное ядро не начинает свою работу с центральной ячейки над пикселем в левом верхнем углу изображения, а выравнивает свой левый верхний угол с левым верхним углом изображения. Для ядра 3×3 по краям имеется граница шириной 1 пиксел, которая не выдерживает свертки (для ядра 5×5 ширина не выдерживающей свертки границы составляет 2 пиксела). Для обозначения этой границы используется термин *padding* (отступ), и при желании можно переопределить поведение по умолчанию, чтобы центральная ячейка ядра вплотную прилегалась к краям изображения. В Keras это достигается путем включения параметра `padding='same'` в вызов `Conv2D`.

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204928
dense_1 (Dense)	(None, 10)	1290
Total params: 225,034		
Trainable params: 225,034		
Non-trainable params: 0		

Рис. 10.8. Выходные данные метода `summary`

Еще один вывод: каждое изображение 28×28 выходит из первого сверточного слоя в виде трехмерного массива или тензора размером $26 \times 26 \times 32$: по одной карте признаков 26×26 для каждого из 32 ядер. После максимального объединения тензор уменьшается до $13 \times 13 \times 32$ и поступает на второй сверточный слой, где еще 64 ядра отфильтровывают признаки из 32 карт признаков размером 13×13 и объединяют их для получения 64 новых карт признаков (тензор размером $11 \times 11 \times 64$). Послед-

ний объединяющий слой уменьшает это число до $5 \times 5 \times 64$. Эти значения уплощаются в одномерный тензор, содержащий 1600 значений, и поступают в плотный слой для классификации.



Общая картина заключается в том, что CNN преобразует каждое изображение размером 28×28 , состоящее из 784 значений пикселей, в массив из 1600 чисел с плавающей запятой, которые (надеемся) выделяют содержимое изображения более четко, чем обычные значения пикселей. Именно этим и занимаются "бутылочные" слои: они преобразуют *матрицы целочисленных значений пикселей* в *тензоры чисел с плавающей точкой*, которые лучше характеризуют поступающие на вход изображения. Как будет показано в *главе 13*, сети NLP используют *вкрапления слов* для создания плотных векторных представлений слов в документе. *Плотное векторное представление* (dense vector representation) — это термин, часто встречающийся в глубоком обучении. Это не что иное, как массивы чисел с плавающей точкой, характеризующие входные данные в большей степени, чем сами входные данные.

Выходные данные в итоге будут выглядеть точно так же, как если бы на вход сети подавались трехканальные цветные изображения, а не одноканальные полутоновые. Применение сверточного слоя с n ядрами к изображению дает n карт признаков независимо от глубины изображения, так же как применение сверточного слоя с n ядрами к полученным предыдущими слоями картам признаков дает n новых карт признаков независимо от глубины входного сигнала. Внутри CNN используются *тензорные точечные (скалярные) произведения* для получения 2D-карт признаков из 3D-карт признаков. Библиотека NumPy (<https://oreil.ly/Hek8J>) в Python включает функцию `tensor_dot` (<https://oreil.ly/6NyCk>) для быстрого вычисления тензорных точечных произведений.

Теперь обучим сеть и построим графики точности обучения и валидации:

```
%matplotlib inline
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
sns.set()
```

```
hist = model.fit(x_train, y_train, validation_data=(x_test, y_test),
                 epochs=10, batch_size=50)
```

```
acc = hist.history['accuracy']
```

```
val_acc = hist.history['val_accuracy']
```

```
epochs = range(1, len(acc) + 1)
```

```
plt.plot(epochs, acc, '-', label='Точность обучения')
```

```
plt.plot(epochs, val_acc, ':', label='Точность валидации')
```

```
plt.title('Точность обучения и валидации')
```

```
plt.xlabel('Эпоха')
```

```
plt.ylabel('Точность')  
plt.legend(loc='lower right')
```

После обучения эта простая CNN может с точностью 99% классифицировать рукописные цифры (рис. 10.9).

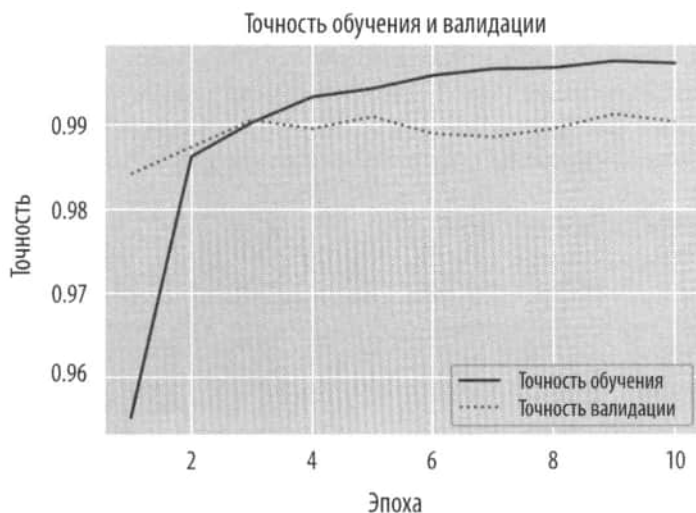


Рис. 10.9. Графики точности обучения и валидации для CNN, распознающей рукописные цифры

Одной из причин такой точности является количество обучающих образцов — около 6000 на каждый класс (в качестве эксперимента я обучил сеть всего на 100 образцах каждого класса и получил точность 92%). Другим фактором является то, что цифра 2 внешне сильно отличается, скажем, от 8. Если человек может достаточно легко различить эти две цифры, то и CNN сможет это сделать.

Обучение CNN распознаванию арктических диких животных

Базовая CNN может легко достичь 99% точности на наборе данных MNIST. Однако все оказывается не так просто, когда задача становится более сложной, например, если необходимо определить, кто изображен на фотографии: собака или кошка. Одна из причин заключается в том, что большинство восьмерок выглядят очень похоже, в то время как собаки и кошки бывают разных видов. Другой фактор заключается в том, что каждая цифра в наборе данных MNIST тщательно обрезается, чтобы точно заполнить кадр, в то время как собаки и кошки могут появляться в любом месте кадра и могут быть сфотографированы в различных позах и с бесконечного числа ракурсов.

Для наглядности давайте обучим CNN различать песцов, белых медведей и моржей. Представьте, что перед вами поставлена задача создать систему, которая с помощью искусственного интеллекта будет изучать снимки, сделанные активирова-

мыми движением камерами, установленными в Арктике для регистрации активности белых медведей.

Начните с загрузки ZIP-файла (<https://oreil.ly/p7Y0k>), содержащего изображения для обучения и тестирования CNN. Распакуйте ZIP-файл и поместите его содержимое в подкаталог Wildlife, где размещены ваши Jupyter-блокноты. ZIP-файл содержит папки train, test и samples. Каждая папка имеет вложенные папки arctic_fox, polar_bear и walrus. Обучающие папки содержат по 100 изображений, а папки для тестирования — по 40 изображений. На рис. 10.10 показаны некоторые из обучающих изображений белого медведя. Это общедоступные изображения, которые были загружены из Интернета, обрезаны и приведены к размеру 224×224 пикселей.



Рис. 10.10. Изображения белого медведя

Теперь создайте блокнот Jupyter и с помощью следующего кода определите пару вспомогательных функций: одна загружает пакет изображений из указанного места в файловой системе и присваивает им метки, а другая показывает первые восемь изображений из пакета изображений:

```
import os
from tensorflow.keras.preprocessing import image
import matplotlib.pyplot as plt
%matplotlib inline

def load_images_from_path(path, label):
    images, labels = [], []

    for file in os.listdir(path):
        img = image.load_img(os.path.join(path, file), target_size=(224, 224, 3))
        images.append(image.img_to_array(img))
        labels.append((label))

    return images, labels
```

```
def show_images(images):
    fig, axes = plt.subplots(1, 8, figsize=(20, 20),
                             subplot_kw={'xticks': [], 'yticks': []})

    for i, ax in enumerate(axes.flat):
        ax.imshow(images[i] / 255)
```

```
x_train, y_train, x_test, y_test = [], [], [], []
```

С помощью следующих операторов загрузите 100 обучающих изображений песка и отобразите их подмножество:

```
images, labels = load_images_from_path('Wildlife/train/arctic_fox', 0)
show_images(images)
```

```
x_train += images
y_train += labels
```

Аналогичным образом загрузите и промаркируйте обучающие изображения белых медведей:

```
images, labels = load_images_from_path('Wildlife/train/polar_bear', 1)
show_images(images)
```

```
x_train += images
y_train += labels
```

А затем — обучающие изображения моржей:

```
images, labels = load_images_from_path('Wildlife/train/walrus', 2)
show_images(images)
```

```
x_train += images
y_train += labels
```

Также необходимо загрузить изображения, используемые для валидации CNN. Начните с 40 тестовых изображений песка:

```
images, labels = load_images_from_path('Wildlife/test/arctic_fox', 0)
show_images(images)
```

```
x_test += images
y_test += labels
```

Теперь тестовые изображения белого медведя:

```
images, labels = load_images_from_path('Wildlife/test/polar_bear', 1)
show_images(images)
```

```
x_test += images
y_test += labels
```


И, наконец, тестовые изображения моржей:

```
images, labels = load_images_from_path('Wildlife/test/walrus', 2)
show_images(images)
```

```
x_test += images
y_test += labels
```

Следующим шагом является нормализация обучающих и тестовых изображений путем деления их пиксельных значений на 255:

```
import numpy as np
```

```
x_train = np.array(x_train) / 255
x_test = np.array(x_test) / 255
```

```
y_train = np.array(y_train)
y_test = np.array(y_test)
```

Теперь пришло время построить CNN. Поскольку размеры изображений 224×224 и мы хотим, чтобы итоговые карты признаков сжали как можно больше информации в небольшое пространство, мы будем использовать пять пар сверточных и объединяющих слоев для извлечения признаков из обучающих изображений с пятью разрешениями: 224×224 , 111×111 , 54×54 , 26×26 и 12×12 . За ними следует плотный слой и выходной содержащий три нейрона слой softmax — по одному для каждого из трех классов:

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
```

```
model = Sequential()
model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(224, 224, 3)))
model.add(MaxPooling2D(2, 2))
model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(MaxPooling2D(2, 2))
model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(MaxPooling2D(2, 2))
model.add(Conv2D(128, (3, 3), activation='relu'))
model.add(MaxPooling2D(2, 2))
model.add(Conv2D(128, (3, 3), activation='relu'))
model.add(MaxPooling2D(2, 2))
model.add(Flatten())
model.add(Dense(1024, activation='relu'))
model.add(Dense(3, activation='softmax'))
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
model.summary(line_length=80)
```

Вызовите `fit` для обучения модели:

```
hist = model.fit(x_train, y_train, validation_data=(x_test, y_test),
                 batch_size=10, epochs=20)
```

Если обучать модель на центральном процессоре, то на это, вероятно, потребуется от 10 до 20 секунд на эпоху (вспомните все эти вычисления пикселей на всех этих изображениях с использованием всех этих сверточных ядер). По окончании обучения постройте график точности обучения и валидации с помощью следующих утверждений:

```
import seaborn as sns
sns.set()
```

```
acc = hist.history['accuracy']
val_acc = hist.history['val_accuracy']
epochs = range(1, len(acc) + 1)
```

```
plt.plot(epochs, acc, '-', label='Точность обучения')
plt.plot(epochs, val_acc, ':', label='Точность валидации')
plt.title('Точность обучения и валидации')
plt.xlabel('Эпоха')
plt.ylabel('Точность')
plt.legend(loc='lower right')
plt.plot()
```

На рис. 10.11 приведен результат.



Рис. 10.11. Графики точности обучения и валидации CNN, распознающей белых медведей

Такой результат вы ожидали? Точность валидации приличная, но не лучшая. Вероятно, она находится в пределах 60–70%. Современные CNN часто классифицируют подобные изображения с точностью 95% и выше. Возможно, из этой модели удастся извлечь больше пользы, если нарастить количество сверточных слоев или увеличить число ядер, а также добиться более высокой обобщающей способности, добавив слой отсева. Но 95% с этой сетью и этим набором данных вы не достигнете.

Одна из причин, по которой современные CNN могут так точно классифицировать изображения, заключается в том, что они обучаются на миллионах изображений. Вам не нужны миллионы образцов каждого класса, но вам, вероятно, потребуется *как минимум* на порядок больше — если не на два порядка больше, чем те 300, на которых проводилось обучение здесь. Можно порыться в Интернете в поисках большего количества изображений, но большее количество изображений означает большее время обучения. Если целью является достижение точности 95% и более, то вы быстро придете к тому, что обучение CNN будет занимать слишком много времени, или к необходимости покупки графического процессора NVIDIA (<https://oreil.ly/fkWUQ>).

Это не означает, что CNN непригодны для решения бизнес-задач. Это просто означает, что есть чему поучиться. Следующий раздел — это первый шаг к пониманию того, как достичь высокой точности без обучения CNN с нуля.

Предварительно обученные CNN

Microsoft, Google и другие технологические компании используют содержащее более 1 млн изображений подмножество набора данных ImageNet (<https://oreil.ly/UZuxt>) для обучения современных CNN распознаванию сотен объектов, включая песцов и белых медведей. Затем они размещаются в открытом доступе. Такие CNN называются *предварительно обученными* и являются более сложными, чем те, которые вы могли бы обучить самостоятельно. И если это недостаточно удивительно, то Keras сводит процесс загрузки предварительно обученной CNN к одной строке кода.

Keras предоставляет классы, в которые встроены более двух десятков популярных предварительно обученных CNN (полный список задокументирован на сайте Keras (<https://oreil.ly/Jpl0Q>)). Большинство из этих CNN описаны в таких научных работах, как "Глубокое остаточное обучение для распознавания изображений" ("Deep Residual Learning for Image Recognition", <https://oreil.ly/7np6D>) и "EfficientNet: переосмысление масштабирования модели для сверточных нейронных сетей" ("Rethinking Model Scaling for Convolutional Neural Networks", <https://oreil.ly/GCdfo>). Некоторые из них стали победителями престижных конкурсов, таких как ImageNet Large Scale Visual Recognition Challenge и COCO Detection Challenge (<https://oreil.ly/RWOSu>). Среди наиболее известных — семейство сетей ResNet от Microsoft и сети Inception от Google. Заслуживает внимания и сеть MobileNet (<https://oreil.ly/m1UFj>), которая благодаря малому объему памяти выигрывает в точности и идеально подходит для мобильных устройств. Более подробно о ней можно узнать в блоге Google AI (<https://oreil.ly/eoofX>).

Архитектуры предварительно обученных CNN

Предварительно обученные CNN достигают впечатляющих уровней точности не только потому, что они обучены на миллионах изображений, но и потому, что они глубже и архитектурно более совершенны, чем представленные до сих пор CNN. Например, они используют последовательные сверточные слои для дальнейшего уточнения извлекаемых из каждого изображения характеристик, а многие из них применяют *пакетную нормализацию* (batch normalization, <https://oreil.ly/jHAKA>) для нормализации (приведения к единичной дисперсии) значений, передаваемых между слоями в процессе обучения.

Однако предварительно обученные CNN более сложны и по другим причинам. Например, в сетях ResNet была впервые применена концепция *остаточных слоев* (residual layers), которые добавляют свои входные данные к выходным. Это простое нововведение помогло устранить проблему исчезающего градиента, из-за которой обновления становятся все меньше по мере того, как оптимизатор идет назад по сети, обновляя веса и смещения, и сделало возможным создание гораздо более глубоких сетей.

Кроме того, существует семейство CNN Inception, главным новшеством которого стало использование ядер свертки 1×1 для минимизации вычислительных затрат больших сверточных ядер за счет уменьшения глубины входных карт признаков. В последних версиях Inception также были использованы возможности ResNets, а производная архитектура, известная под названием Xception, усовершенствовала Inception путем введения в нее разделяемых по глубине свертываний (<https://oreil.ly/8wfRD>). Если вы хотите узнать больше, прочитайте статьи "Простое руководство по версиям сети Inception" ("A Simple Guide to the Versions of the Inception Network", <https://oreil.ly/iwv4L>) и "Xception: глубокое обучение с разделяемыми по глубине свертками" ("Xception: Deep Learning with Depthwise Separable Convolutions", <https://oreil.ly/U2OCm>).

Следующий оператор инстанцирует класс MobileNetV2 (<https://oreil.ly/Oq0yb>) в Keras и инициализирует его весами, смещениями и значениями ядра, полученными при обучении сети на наборе данных ImageNet:

```
from tensorflow.keras.applications import MobileNetV2
```

```
model = MobileNetV2(weights='imagenet')
```

Параметр `weights='imagenet'` указывает Keras, какие параметры необходимо загрузить для пересоздания сети в обученном состоянии. Можно также передать путь к файлу с пользовательскими весами, но `imagenet` — это единственный набор предопределенных весов, который поддерживается в настоящее время.

Прежде чем изображение будет передано в предварительно обученную CNN для классификации, оно должно быть приведено к ожидаемым CNN размерам (обычно 224×224) и предварительно обработано. Различные CNN по-разному требуют предварительной обработки изображений, поэтому Keras предоставляет функцию

`preprocess_input` для каждой предварительно обученной CNN. Кроме того, в состав Keras входят утилиты для загрузки и изменения размеров изображений. Следующие операторы загружают изображение из файловой системы и выполняют его предварительную обработку для отправки в сеть MobileNetV2:

```
import numpy as np
from tensorflow.keras.applications.mobilenet import preprocess_input
from tensorflow.keras.preprocessing import image
```

```
x = image.load_img('arctic_fox.jpg', target_size=(224, 224))
x = image.img_to_array(x)
x = np.expand_dims(x, axis=0)
x = preprocess_input(x)
```

В большинстве случаев `preprocess_input` выполняет всю необходимую работу, часто заключающуюся в применении единичной дисперсии к значениям пикселей и преобразовании RGB-изображений в формат BGR. Однако в некоторых случаях все же требуется разделить значения пикселей на 255. В качестве примера можно привести ResNet50V2 (<https://oreil.ly/IanRd>):

```
import numpy as np
from tensorflow.keras.applications.resnet50 import preprocess_input
from tensorflow.keras.preprocessing import image
```

```
x = image.load_img('arctic_fox.jpg', target_size=(224, 224))
x = image.img_to_array(x)
x = np.expand_dims(x, axis=0)
x = preprocess_input(x) / 255
```

После предварительной обработки изображения сделать прогноз достаточно просто путем вызова метода `predict` сети:

```
y = model.predict(x)
```



```
[['n02120079', 'Arctic_fox', 0.8888298),
 ('n02114548', 'white_wolf', 0.10683839),
 ('n02111889', 'Samoyed', 0.0024216734),
 ('n02119789', 'kit_fox', 0.0004960838),
 ('n02114367', 'timber_wolf', 0.00025730868)]]
```

Рис. 10.12. Результат работы `decode_predictions`

Для того чтобы помочь вам интерпретировать полученные результаты, Keras также предоставляет метод `decode_predictions`, предназначенный для конкретной сети. На

рис. 10.12 показан результат, полученный этим методом для переданной в ResNet50V2 фотографии.

Функция ResNet50V2 на 89% уверена, что на фотографии изображен песок, что, собственно, и есть. MobileNetV2 с 92% уверенностью предсказала, что на фотографии изображен песок. Обе сети были обучены на одном и том же наборе данных, но разные предварительно обученные CNN классифицируют изображения по-разному.

Использование ResNet50V2 для классификации изображений

Давайте используем Keras для загрузки предварительно обученной CNN и классификации пары изображений. Откройте ноутбук и с помощью следующих операторов загрузите ResNet50V2:

```
from tensorflow.keras.applications import ResNet50V2
```

```
model = ResNet50V2(weights='imagenet')
```

```
model.summary()
```

Далее загрузите изображение песка и укажите это в блокноте:

```
%matplotlib inline
```

```
import matplotlib.pyplot as plt
```

```
from tensorflow.keras.preprocessing import image
```

```
x = image.load_img('wildlife/samples/arctic_fox/arctic_fox_140.jpeg',
                    target_size=(224, 224))
```

```
plt.xticks([])
```

```
plt.yticks([])
```

```
plt.imshow(x)
```

Теперь выполните предварительную обработку изображения (помните, что для ResNet50V2 необходимо также разделить все значения пикселей на 255 после вызова метода `preprocess_input`, предоставляемого Keras) и передайте его в CNN для классификации:

```
import numpy as np
```

```
from tensorflow.keras.applications.resnet50 import preprocess_input
```

```
from tensorflow.keras.applications.resnet50 import decode_predictions
```

```
x = image.img_to_array(x)
```

```
x = np.expand_dims(x, axis=0)
```

```
x = preprocess_input(x) / 255
```

```
y = model.predict(x)
```

```
decode_predictions(y)
```

Результат должен выглядеть примерно так:

```
[[('n02120079', 'Arctic_fox', 0.9999944),
 ('n02114548', 'white_wolf', 4.760021e-06),
 ('n02119789', 'kit_fox', 2.3306782e-07),
 ('n02442845', 'mink', 1.2460312e-07),
 ('n02111889', 'Samoyed', 1.1914468e-07)]]
```

ResNet50V2 практически уверена, что на изображении находится песец. А теперь загрузите изображение моржа:

```
x = image.load_img('wildlife/samples/walrus/walrus_143.png', target_size=(224, 224))
plt.xticks([])
plt.yticks([])
plt.imshow(x)
```

Попросите ResNet50V2 классифицировать его:

```
x = image.img_to_array(x)
x = np.expand_dims(x, axis=0)
x = preprocess_input(x) / 255
```

```
y = model.predict(x)
decode_predictions(y)
```

Ниже приведен результат:

```
[[('n02454379', 'armadillo', 0.63758147),
 ('n01704323', 'triceratops', 0.16057032),
 ('n02113978', 'Mexican_hairless', 0.07795086),
 ('n02398521', 'hippopotamus', 0.022284042),
 ('n01817953', 'African_grey', 0.016944142)]]
```

ResNet50V2 считает, что на картинке, скорее всего, изображен броненосец, но она не очень уверена в этом. Можете ли предположить, почему?

Для обучения ResNet50V2 было использовано почти 1,3 млн изображений. Однако ни одно из них не содержало моржа. Список ImageNet 1000 Class List (<https://oreil.ly/mj9Zs>) показывает полный список классов, использовавшихся в обучении сети. Предварительно обученная CNN отлично справляется с классификацией изображений по классам, на которые была обучена, но она бессильна решать специфические задачи, для которых ее не обучали.

Но не все потеряно. Метод, называемый *трансферным обучением*, позволяет пере-профилировать предварительно обученные CNN для решения специфических задач. При этом перепрофилирование может осуществляться на обычном CPU без использования GPU. Иногда точность трансферного обучения достигает 95% при использовании всего нескольких сотен обучающих изображений. Зная это, вы сможете совершенно по-другому взглянуть на эффективность применения CNN для решения бизнес-задач.

Трансферное обучение

Ранее мы использовали набор данных с фотографиями песцов, белых медведей и моржей для обучения CNN распознаванию дикой природы Арктики. Обученная на 300 изображениях — по 100 для каждого из трех классов — CNN достигла точности около 60%. Этого недостаточно для большинства целей.

Одним из решений является обучение CNN на десятках тысяч фотографий. Более эффективным решением, способным обеспечить точность высочайшего уровня на имеющихся у вас 300 фотографиях и не требующим дорогостоящего оборудования, является трансферное обучение (transfer learning). В руках разработчиков программного обеспечения и инженеров трансферное обучение делает CNN практичным решением для множества задач компьютерного зрения. При этом оно требует на порядки меньше времени и вычислительных мощностей, чем CNN, обученные с нуля. Давайте разберемся, что такое трансферное обучение и как оно работает, а затем применим его для идентификации арктических диких животных.

Предварительно обученные CNN на базе данных ImageNet способны идентифицировать песцов и белых медведей, но не могут идентифицировать моржей, поскольку изображения моржей не использовались для обучения. Трансферное обучение позволяет перепрофилировать предварительно обученные CNN для идентификации объектов, которые изначально не использовались в обучении. При этом применяются заложенные в предварительно обученные CNN интеллектуальные возможности, но они переопределяются для решения новых задач.

Напомним, что CNN состоит из двух групп слоев: "бутылочных" слоев, содержащих сверточные слои и объединяющие слои, извлекающие признаки из изображений с различным разрешением, и классификационных слоев, которые классифицируют признаки, полученные от "бутылочных" слоев, как принадлежащие песцу, белому медведю или чему-то другому. В сверточных слоях для извлечения признаков используются сверточные ядра, значения которых определяются в процессе обучения. На это обучение приходится большая часть времени. Когда сложные CNN обучаются на миллионах изображений, сверточные ядра становятся очень эффективными при извлечении признаков. Однако за эту эффективность приходится платить.

Принцип трансферного обучения показан на рис. 10.13. Вы загружаете бутылочные слои предварительно обученной CNN, но не загружаете классификационные слои. Вместо этого вы предоставляете свои собственные, которые обучаются на порядки быстрее, чем вся CNN. Затем обучающие изображения проходят через бутылочные слои для выделения признаков и на полученных признаках обучаются классификационные слои. Предварительно обученная CNN, возможно, была обучена извлекать признаки из изображений яблок и апельсинов, но эти же слои, вероятно, неплохо справятся с извлечением признаков и из фотографий собак и кошек. Используя предварительно обученные бутылочные слои для извлечения признаков, а затем используя эти признаки для обучения собственных классификационных слоев, можно научить модель тому, что определенный извлеченный из изображения признак может быть признаком собаки, а не яблока.

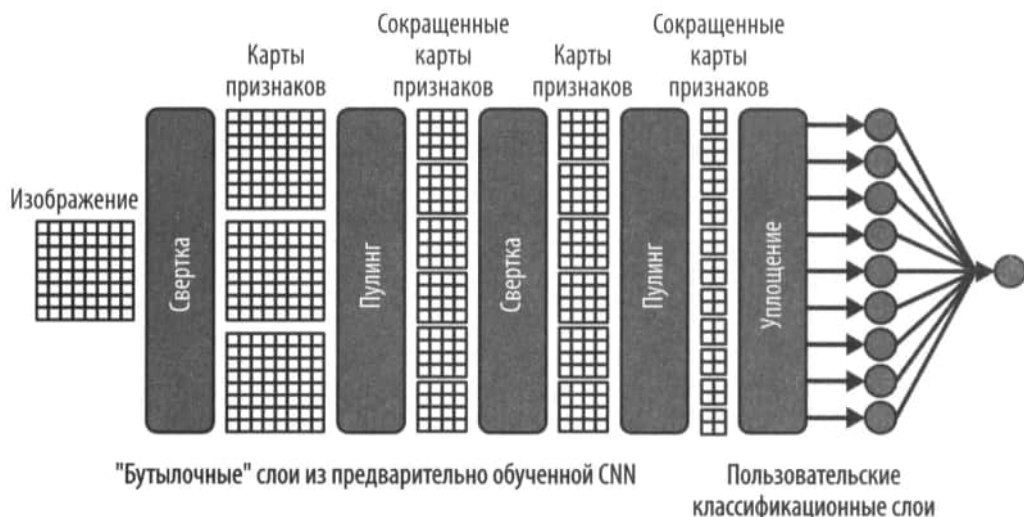


Рис. 10.13. Архитектура нейронной сети для трансферного обучения

Трансферное обучение относительно просто реализуется в Keras и TensorFlow. Напомним, что следующий оператор загружает ResNet50V2 и инициализирует ее весами (включая значения ядра) и смещениями, полученными при обучении сети на подмножестве набора данных ImageNet:

```
base_model = ResNet50V2(weights='imagenet')
```

Для того чтобы загрузить ResNet50V2 (или любую другую поддерживаемую Keras предварительно обученную CNN) без слоев классификации, достаточно добавить атрибут `include_top=False`:

```
base_model = ResNet50V2(weights='imagenet', include_top=False)
```

С этого момента можно пойти двумя путями трансферного обучения. Первый заключается в добавлении классификационных слоев к "бутылочным" слоям базовой модели и установке атрибута `trainable` каждого базового слоя в значение `False`, чтобы веса, смещения и ядра сверточных слоев не обновлялись при обучении сети:

```
for layer in base_model.layers:
```

```
    layer.trainable = False
```

```
model = Sequential()
```

```
model.add(base_model)
```

```
model.add(Flatten())
```

```
model.add(Dense(1024, activation='relu'))
```

```
model.add(Dense(3, activation='softmax'))
```

```
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

```
model.fit(x, y, validation_split=0.2, epochs=10, batch_size=10)
```

Второй путь заключается в том, чтобы прогнать все обучающие изображения через базовую модель для извлечения признаков, а затем прогнать эти признаки через отдельную сеть, содержащую классификационные слои:

```
features = base_model.predict(x)
```

```
model = Sequential()
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dense(3, activation='softmax'))
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

```
model.fit(features, y, validation_split=0.2, epochs=10, batch_size=10)
```

Какой путь лучше? Второй путь быстрее, поскольку обучающие изображения проходят через "бутылочные" слои для выделения признаков не один раз за эпоху, а всего один раз. Именно этот вариант следует использовать при отсутствии веских причин для иного. Первый путь немного медленнее, но он допускает тонкую настройку (<https://oreil.ly/pIQrN>), при которой после завершения обучения размораживается один или несколько бутылочных слоев, и обучение продолжается еще несколько эпох с очень низкой скоростью обучения. Кроме того, этот метод позволяет осуществлять аугментацию ("раздувание") данных, о чем я расскажу в следующем разделе.



Тонкая настройка часто применяется к моделям трансферного обучения после завершения обучения с целью получения дополнительных процента-двух точности. В *главе 13* мы будем использовать тонкую настройку для повышения точности модели NLP, использующей предварительно обученную нейронную сеть.

В случае выбора первой техники для реализации трансферного обучения прогнозирование осуществляется путем предварительной обработки изображений и передачи их в метод `predict` модели. При использовании второй техники прогнозирование выполняется в два этапа. После предварительной обработки изображений они передаются в метод `predict` базовой модели, а затем выходные данные этого метода передаются в метод `predict` вашей модели:

```
x = image.img_to_array(x)
x = np.expand_dims(x, axis=0)
x = preprocess_input(x) / 255
```

```
features = base_model.predict(x)
predictions = model.predict(features)
```

На этом трансферное обучение завершено. Осталось только применить его на практике.

Использование трансферного обучения для идентификации диких животных Арктики

Применим трансферное обучение для решения той же задачи, которую мы пытались решить ранее с помощью собранной на скорую руку CNN: построим модель, определяющую, кто изображен на фотографии — песец, белый медведь или морж.

Создайте блокнот Jupyter и используйте тот же код, что и ранее, для загрузки обучающих и валидационных изображений и присвоения им меток: 0 — для песцов, 1 — для белых медведей и 2 — для моржей. После этого необходимо выполнить предварительную обработку изображений. В качестве предварительно обученной CNN мы будем использовать ResNet50V2, поэтому для первичной обработки пикселей используйте версию ResNet метода `preprocess_input`. Затем разделите значения пикселей на 255:

```
import numpy as np
from tensorflow.keras.applications.resnet50 import preprocess_input
```

```
x_train = preprocess_input(np.array(x_train)) / 255
x_test = preprocess_input(np.array(x_test)) / 255
```

```
y_train = np.array(y_train)
y_test = np.array(y_test)
```

На следующем этапе загрузите ResNet50V2, стараясь загружать "бутылочные" слои, но *не* классификационные слои, и используйте ее для извлечения признаков из обучающих и проверочных изображений:

```
from tensorflow.keras.applications import ResNet50V2

base_model = ResNet50V2(weights='imagenet', include_top=False)
```

```
x_train = base_model.predict(x_train)
x_test = base_model.predict(x_test)
```

Теперь обучите нейронную сеть классифицировать признаки, извлеченные из обучающих изображений:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Flatten, Dense

model = Sequential()
model.add(Flatten())
model.add(Dense(1024, activation='relu'))
model.add(Dense(3, activation='softmax'))
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

hist = model.fit(x_train, y_train, validation_data=(x_test, y_test),
                batch_size=10, epochs=10)
```

Насколько хорошо обучилась сеть? Постройте графики точности обучения и валидации для каждой эпохи:

```
import seaborn as sns
sns.set()

acc = hist.history['accuracy']
val_acc = hist.history['val_accuracy']
epochs = range(1, len(acc) + 1)

plt.plot(epochs, acc, '-', label='Точность обучения')
plt.plot(epochs, val_acc, ':', label='Точность валидации')
plt.title('Точность обучения и валидации')
plt.xlabel('Эпоха')
plt.ylabel('Точность')
plt.legend(loc='lower right')
plt.plot()
```

Ваши результаты будут отличаться от моих, но я получил точность около 97% (рис. 10.14). Если у вас не получилось, попробуйте обучить сеть еще раз.

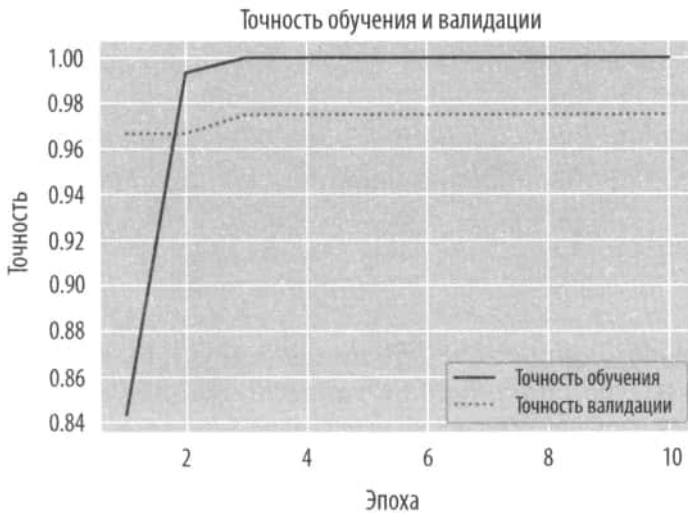


Рис. 10.14. Графики точности обучения и валидации CNN, идентифицирующей диких животных Арктики с применением трансферного обучения

Наконец, используйте матрицу ошибок для визуализации того, насколько хорошо сеть проводит различия между классами:

```
from sklearn.metrics import ConfusionMatrixDisplay as cmd

sns.reset_orig()
fig, ax = plt.subplots(figsize=(4, 4))
ax.grid(False)
```

```

y_pred = model.predict(x_test)
class_labels = ['песец', 'белый медведь', 'морж']

cmd.from_predictions(y_test, y_pred.argmax(axis=1),
                    display_labels=class_labels, colorbar=False,
                    cmap='Blues', xticks_rotation='vertical', ax=ax)

```

Вот как это получилось у меня (рис. 10.15).

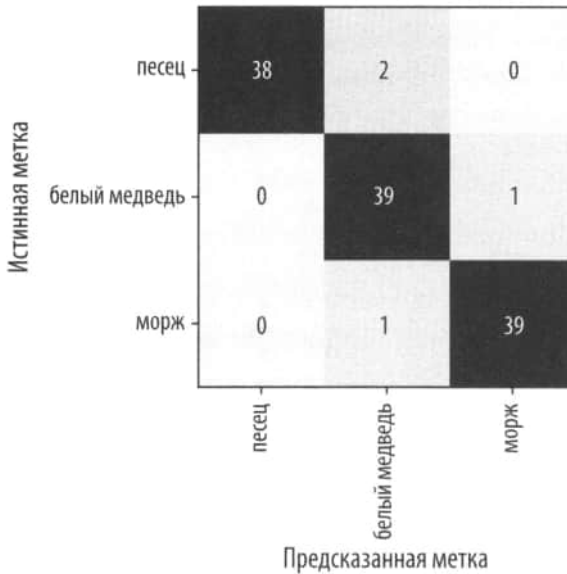


Рис. 10.15. Матрица ошибок построенной CNN, идентифицирующей диких животных Арктики с применением трансферного обучения

Для того чтобы увидеть, как работает трансферное обучение, загрузите одно из изображений песка из папки `samples`. В ней находятся изображения диких животных, на которых модель не обучалась и не проверялась:

```

x = image.load_img('wildlife/samples/arctic_fox/arctic_fox_140.jpeg',
                  target_size=(224, 224))

```

```

plt.xticks([])
plt.yticks([])
plt.imshow(x)

```

Теперь выполните предварительную обработку изображения, пропустите его через слои извлечения признаков ResNet50V2 и пропустите выходные данные через вновь обученные классификационные слои:

```

x = image.img_to_array(x)
x = np.expand_dims(x, axis=0)
x = preprocess_input(x) / 255

```

```
y = base_model.predict(x)
predictions = model.predict(y)
```

```
for i, label in enumerate(class_labels):
    print(f'{label}: {predictions[0][i]}')
```

В моем случае сеть почти со 100% уверенностью предсказала, что на изображении находится песец:

```
arctic fox: 1.0
polar bear: 0.0
walrus: 0.0
```

Возможно, это покажется закономерным, ведь ResNet50V2 обучался на изображениях песцов. Но теперь загрузим изображение моржа, которое, как вы помните, ResNet50V2 не смог классифицировать:

```
x = image.load_img('wildlife/samples/walrus/walrus_143.png',
                    target_size=(224, 224))
```

```
plt.xticks([])
plt.yticks([])
plt.imshow(x)
```

Предварительно обработайте изображение и сделайте прогноз:

```
x = image.img_to_array(x)
x = np.expand_dims(x, axis=0)
x = preprocess_input(x) / 255
```

```
y = base_model.predict(x)
predictions = model.predict(y)
```

```
for i, label in enumerate(class_labels):
    print(f'{label}: {predictions[0][i]}')
```

Вот как получилось на этот раз:

```
arctic fox: 0.0
polar bear: 0.0
walrus: 1.0
```

ResNet50V2 не была обучена распознавать моржей, но ваша сеть прошла это обучение. Это и есть трансферное обучение в двух словах. Это эквивалент глубокого обучения, когда можно усидеть сразу на двух стульях. И это тот самый секретный ингредиент, который делает CNN жизнеспособным инструментом для любого, у кого есть ноутбук и несколько сотен обучающих изображений.

Это не значит, что трансферное обучение *всегда* будет давать 97% точности при 100 изображениях на класс. Не обольщайтесь. Если в наборе данных не хватает информации для достижения такого уровня разделения, то ни построенные с нуля CNN, ни трансферное обучение не смогут сделать это по щелчку пальцев. Это все-

гда так в машинном обучении и искусственном интеллекте. Нельзя получить воду из камня. И нельзя построить точную модель на основе не поддерживающих ее данных.

Аугментация данных

В предыдущем примере было показано, как с помощью трансферного обучения построить модель, которая на основе всего 300 обучающих изображений может классифицировать фотографии трех различных видов арктической дикой природы с точностью 97%. Одно из преимуществ трансферного обучения заключается в том, что оно позволяет сделать больше с меньшим количеством изображений. Однако эта особенность является и недостатком. При наличии всего 100 или около того образцов каждого класса разнообразие изображений невелико. Модель может распознать белого медведя, если его голова идеально расположена в центре фотографии. Но если в число обучающих изображений не входят фото, на которых голова медведя расположена по-другому или наклонена под разными углами, то модель может столкнуться с трудностями при классификации фотографии.

Одним из решений является *аугментация (расширение, раздутие) данных*. Вместо того чтобы набирать дополнительные обучающие изображения, можно поворачивать, изменять и масштабировать имеющиеся изображения. Это *не всегда* повышает точность CNN, но часто помогает, особенно при работе с небольшими наборами данных. Keras позволяет легко произвольно трансформировать предоставляемые сети обучающие изображения. В каждой эпохе изображения преобразуются по-разному, поэтому при обучении в течение 10 эпох сеть видит 10 различных вариаций каждого обучающего изображения. Это может повысить способность модели к обобщению при незначительном влиянии на время обучения. На рис. 10.16 показан эффект применения случайных преобразований к изображению хот-дога. Видно, что при предъявлении модели одного и того же изображения разными способами модель может лучше распознавать хот-доги независимо от того, как они оформлены.



Рис. 10.16. Изображения хот-дога с применением случайных преобразований

В Keras имеется встроенная поддержка аугментации данных изображений. Рассмотрим несколько способов аугментации изображений, а затем применим их к модели дикой природы Арктики.

Аугментация изображений с помощью *ImageDataGenerator*

Одним из способов применения аугментации изображений при обучении модели является использование класса *ImageDataGenerator* (<https://oreil.ly/C78UD>) в Keras.

`ImageDataGenerator` генерирует пакеты обучающих изображений "на лету" из загруженных изображений (например, с помощью функции `Keras load_img`) либо из указанного места в файловой системе. Последний способ особенно полезен при обучении CNN на миллионах изображений, поскольку в этом случае изображения загружаются в память не все сразу, а пачками. Однако независимо от того, откуда поступают изображения, `ImageDataGenerator` с удовольствием применяет преобразования по мере их получения.

Приведем простой пример, который можно опробовать самостоятельно. Используйте следующий код для загрузки изображения из файловой системы, оберните вокруг него `ImageDataGenerator` и сгенерируйте 24 версии изображения:

```
import numpy as np
from tensorflow.keras.preprocessing import image
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import matplotlib.pyplot as plt
%matplotlib inline

# Загрузка изображения
x = image.load_img('Wildlife/train/polar_bear/polar_bear_010.jpeg')
x = image.img_to_array(x)
x = np.expand_dims(x, axis=0)

# Оборачивание ImageDataGenerator вокруг изображения
idg = ImageDataGenerator(rescale=1./255, horizontal_flip=True, rotation_range=30,
                        width_shift_range=0.2, height_shift_range=0.2,
                        zoom_range=0.2)

idg.fit(x)

# Генерация 24 версий изображения
generator = idg.flow(x, [0], batch_size=1, seed=0)
fig, axes = plt.subplots(3, 8, figsize=(16, 6),
                        subplot_kw={'xticks': [], 'yticks': []})

for i, ax in enumerate(axes.flat):
    img, label = generator.next()
    ax.imshow(img[0])
```

На рис. 10.17 приведен результат.

Параметры, передаваемые в `ImageDataGenerator`, указывают ему, как преобразовать каждое передаваемое изображение.

`rescale=1./255`

Делит значение каждого пиксела на 255.

`horizontal_flip=True`

Случайным образом переворачивает изображение по горизонтали (вокруг вертикальной оси).

`rotation_range=30`

Случайным образом поворачивает изображение на угол от -30 до 30 градусов.

`width_shift_range=0.2` и `height_shift_range=0.2`

Случайным образом сдвигает изображение на величину от -20 до 20% .

`zoom_range=0.2`

Случайным образом масштабирует изображение на величину от -20 до 20% .

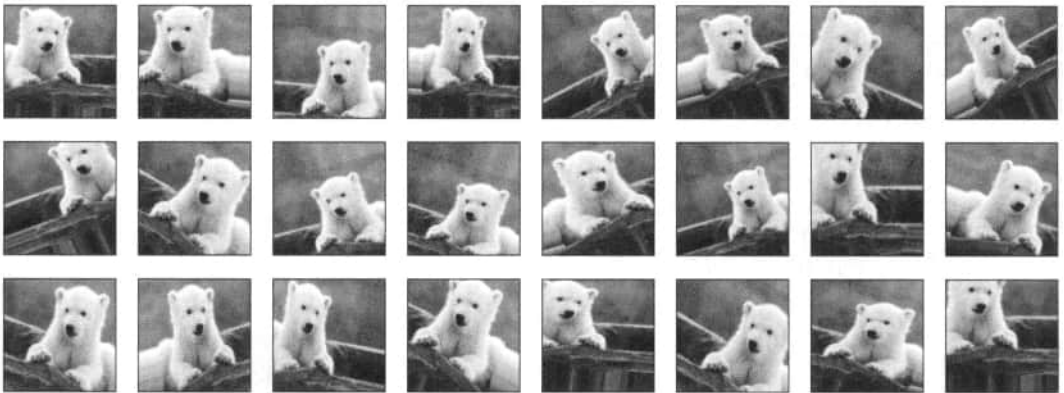


Рис. 10.17. Двадцать четыре версии изображения белого медведя

Существуют и другие параметры, такие как `vertical_flip`, `shear_range`, `brightness_range`, но в целом картина понятна. Используемый в данном примере метод `flow` (<https://oreil.ly/djx5s>) генерирует изображения из переданных в `fit` изображений. Связанный с ним метод `flow_from_directory` (<https://oreil.ly/RBleY>) загружает изображения из файловой системы и опционально маркирует их в зависимости от подкаталогов, в которых они находятся.

Возвращаемый методом `flow` генератор может быть передан непосредственно в метод `fit` модели, чтобы по мере обучения предоставлять модели случайным образом преобразованные изображения. Предположим, что `x_train` и `y_train` содержат коллекцию обучающих изображений и меток. Следующий код оборачивает вокруг них `ImageDataGenerator` и использует их для обучения модели:

```
idg = ImageDataGenerator(rescale=1./255, horizontal_flip=True, rotation_range=30,
                        width_shift_range=0.2, height_shift_range=0.2,
                        zoom_range=0.2)
```

```
idg.fit(x_train)
```

```
image_batch_size = 10
```

```
generator = idg.flow(x_train, y_train, batch_size=image_batch_size, seed=0)
```

```
model.fit(generator,  
            steps_per_epoch=len(x_train) // image_batch_size,  
            validation_data=(x_test, y_test),  
            batch_size=20, epochs=10)
```

Параметр `steps_per_epoch` является ключевым, поскольку `ImageDataGenerator` может создавать бесконечное число версий каждого изображения. В данном примере передаваемый в `flow` параметр `batch_size` указывает генератору на создание 10 изображений в каждой партии. Деление количества изображений на размер партии изображений для расчета `steps_per_epoch` гарантирует, что в каждую эпоху обучения модель получает по одной преобразованной версии каждого изображения из набора данных.



Версии Keras, предшествующие 2.1, не позволяли передавать генератор в метод `fit`. Вместо этого предоставлялся отдельный метод `fit_generator`. Этот метод устарел и будет удален в будущих версиях.

Обратите внимание, что вызов метода `fit` включает параметр `validation_data`, определяющий отдельный набор изображений и меток для проверки сети в процессе обучения. Как правило, *не стоит* увеличивать количество изображений для проверки, поэтому при передаче генератора в `fit` следует избегать использования `validation_split`.

Аугментация изображений с помощью аугментационных слоев

Для создания преобразованных изображений в модели можно использовать `ImageDataGenerator`, однако последние версии Keras предоставляют альтернативу в виде слоев предварительной обработки изображений (<https://oreil.ly/scoED>) и аугментационных слоев изображений (<https://oreil.ly/ikQMF>). Вместо того чтобы преобразовывать обучающие изображения отдельно, можно интегрировать преобразования непосредственно в модель. Приведем пример:

```
from tensorflow.keras.models import Sequential  
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense  
from tensorflow.keras.layers import Rescaling, RandomFlip, RandomRotation  
from tensorflow.keras.layers import RandomTranslation, RandomZoom
```

```
model = Sequential()  
model.add(Rescaling(1./255))  
model.add(RandomFlip(mode='horizontal'))  
model.add(RandomTranslation(0.2, 0.2))  
model.add(RandomRotation(0.2))
```

```

model.add(RandomZoom(0.2))
model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(224, 224, 3)))
model.add(MaxPooling2D(2, 2))
model.add(Conv2D(128, (3, 3), activation='relu'))
model.add(MaxPooling2D(2, 2))
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dense(3, activation='softmax'))

```

Каждое используемое для обучения CNN изображение имеет деленные на 255 значения пикселей, а затем оно случайным образом переворачивается, сдвигается, поворачивается и масштабируется. Важно отметить, что слои `RandomFlip`, `RandomTranslation`, `RandomRotation` и `RandomZoom` работают только с обучающими изображениями. Они неактивны, когда сеть проверяется или ей требуется выполнить прогнозирование. Следовательно, при обучении модели, содержащей аугментационные слои изображений, можно использовать `validation_split`. Слой `Rescaling` активен всегда, поэтому больше не нужно помнить о необходимости делить значения пикселей на 255 перед обучением модели или отправкой изображения на классификацию.

Применение аугментации для изображений дикой природы Арктики

Может ли аугментация изображений сделать трансферное обучение еще лучше? Есть один способ узнать это.

Создайте блокнот Jupyter и скопируйте код, который загружает обучающее и валидационное изображения из примера трансферного обучения. Затем используйте следующие операторы для подготовки данных. Обратите внимание, что в этот раз нет необходимости делить на 255, поскольку об этом позаботится слой `Rescaling`:

```

import numpy as np
from tensorflow.keras.applications.resnet50 import preprocess_input

```

```

x_train = preprocess_input(np.array(x_train))
x_test = preprocess_input(np.array(x_test))

```

```

y_train = np.array(y_train)
y_test = np.array(y_test)

```

Теперь загрузите `ResNet50V2` без слоев классификации и инициализируйте ее весами ImageNet. Ключевым элементом здесь является предотвращение обучения булылочных слоев в процессе обучения сети путем установки их атрибута обучаемости в значение `False`, что фактически замораживает эти слои. Вместо того чтобы устанавливать значение `False` для атрибута `trainable` каждого отдельного слоя, мы

установим `False` у `trainable` для самой модели и позволим этой настройке "наследоваться" отдельными слоями:

```
from tensorflow.keras.applications import ResNet50V2
```

```
base_model = ResNet50V2(weights='imagenet', include_top=False)
```

```
base_model.trainable = False
```

Определите сеть, включающую в себя слои масштабирования и аугментации, "бутылочные" слои `ResNet50V2` и плотные слои для классификации. Затем обучите сеть:

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Flatten, Dense, Rescaling, RandomFlip
```

```
from tensorflow.keras.layers import RandomRotation, RandomTranslation, RandomZoom
```

```
model = Sequential()
```

```
model.add(Rescaling(1./255))
```

```
model.add(RandomFlip(mode='horizontal'))
```

```
model.add(RandomTranslation(0.2, 0.2))
```

```
model.add(RandomRotation(0.2))
```

```
model.add(RandomZoom(0.2))
```

```
model.add(base_model)
```

```
model.add(Flatten())
```

```
model.add(Dense(1024, activation='relu'))
```

```
model.add(Dense(3, activation='softmax'))
```

```
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',  
              metrics=['accuracy'])
```

```
hist = model.fit(x_train, y_train,
```

```
validation_data=(x_test, y_test),
```

```
batch_size=10, epochs=10)
```

Насколько хорошо обучилась сеть? Постройте графики точности обучения и валидации для каждой эпохи:

```
import seaborn as sns
```

```
sns.set()
```

```
acc = hist.history['accuracy']
```

```
val_acc = hist.history['val_accuracy']
```

```
epochs = range(1, len(acc) + 1)
```

```
plt.plot(epochs, acc, '-', label='Точность обучения')
```

```
plt.plot(epochs, val_acc, ':', label='Точность валидации')
```

```
plt.title('Точность обучения и валидации')
```

```
plt.xlabel('Эпоха')
```

```
plt.ylabel('Точность')
plt.legend(loc='lower right')
plt.plot()
```

Если повезло, то точность немного превысила точность модели, обученной без аугментации данных (рис. 10.18).

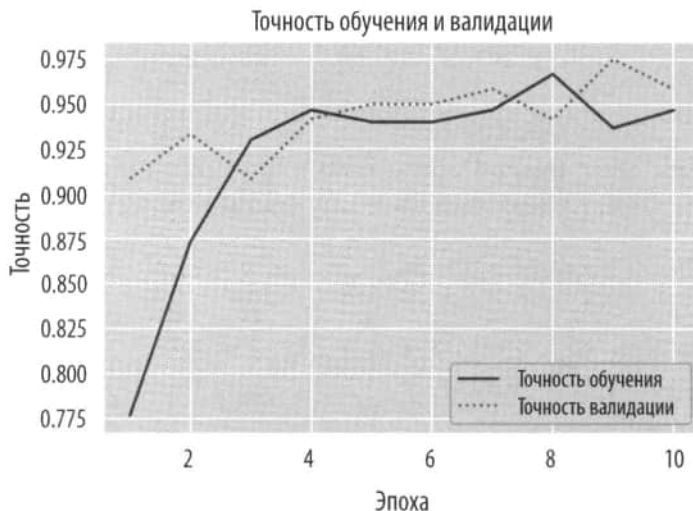


Рис. 10.18. Графики точности обучения и валидации CNN, идентифицирующей диких животных Арктики с применением аугментации данных

Вы можете обнаружить, что версия модели, использующая аугментацию данных, менее точна, чем та, которая ее не применяет. Для того чтобы проверить это, я обучил каждую версию десять раз и усреднил результаты. Оказалось, что версия с аугментацией данных дает в среднем на 0,5% больше точности, чем версия без аугментации. Это не так уж много, но специалисты по исследованию данных часто идут на многое, чтобы повысить точность всего на доли процента.

Используйте матрицу ошибок для визуализации того, насколько хорошо работала сеть во время тестирования:

```
from sklearn.metrics import ConfusionMatrixDisplay as cmd

sns.reset_orig()
fig, ax = plt.subplots(figsize=(4, 4))
ax.grid(False)

y_pred = model.predict(x_test)
class_labels = ['невец', 'белый медведь', 'морж']
cmd.from_predictions(y_test, y_pred.argmax(axis=1),
                    display_labels=class_labels, colorbar=False,
                    cmap='Blues', xticks_rotation='vertical', ax=ax)
```

Вот что получилось у меня (рис. 10.19).

Истинная метка	песец	39	1	0
	белый медведь	1	38	1
	морж	1	1	38
		песец	белый медведь	морж
		Предсказанная метка		

Рис. 10.19. Матрица ошибок построенной CNN, идентифицирующей диких животных Арктики с применением аугментации данных

Специалисты по исследованию данных иногда прибегают к аугментации данных даже при обучении CNN с нуля, а не в случае трансферного обучения, особенно если набор данных относительно невелик. Это полезный инструмент, о котором следует знать, и который может иметь значение, когда вы пытаетесь выжать из модели глубокого обучения все до последней капли точности.

Глобальный пулинг

Цель включения слоя Flatten в CNN — преобразовать трехмерные содержащие итоговые карты признаков тензоры в одномерные тензоры, пригодные для ввода в слой Dense. Однако Flatten не единственный способ сделать это. Иногда Flatten приводит к переобучению (overfitting), предоставляя слоям классификации слишком много информации.

Одним из способов борьбы с избыточным обучением является введение слоя Dropout. Другая стратегия заключается в уменьшении ширины слоя Dense. Третий вариант — замена слоя Flatten слоем GlobalMaxPooling2D (<https://oreil.ly/hjPM3>) или слоем GlobalAveragePooling2D (<https://oreil.ly/8hOPF>). Они тоже выводят одномерные тензоры, но генерируют их другим способом. И этот способ менее подвержен переобучению.

Для того чтобы продемонстрировать это, изменим пример с набором данных MNIST, приведенный ранее в этой главе так, чтобы использовать слой GlobalMaxPooling2D, а не слой Flatten:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D,
    GlobalMaxPooling2D, Dense

model = Sequential()
model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)))
model.add(MaxPooling2D(2, 2))
model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(MaxPooling2D(2, 2))
model.add(GlobalMaxPooling2D()) # In lieu of Flatten()
model.add(Dense(128, activation='relu'))
model.add(Dense(10, activation='softmax'))
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
    metrics=['accuracy'])
model.summary(line_length=120)
```

В итоге (рис. 10.20) видно, что на выходе слоя GlobalMaxPooling2D получается тензор, содержащий не $5 \times 5 \times 64$, или 1600, значений, как для слоя Flatten, а 64 значения — по одному на каждую карту признаков, созданную завершающим слоем MaxPooling2D. Каждое значение является максимальным из 25 значений в каждой карте признаков 5×5 . Если бы вместо этого использовался слой GlobalAveragePooling2D, то каждое значение было бы средним из 25 значений в каждой карте признаков.

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
global_max_pooling2d (GlobalMaxPooling2D)	(None, 64)	0
dense (Dense)	(None, 128)	8320
dense_1 (Dense)	(None, 10)	1290
Total params: 28,426		
Trainable params: 28,426		
Non-trainable params: 0		

Рис. 10.20. Выходные данные метода summary

Глобальный пулинг иногда повышает способность CNN к обобщению, а иногда — нет. Для набора данных MNIST он несколько снижает точность. Как это часто бывает в машинном обучении, единственный способ узнать — это попробовать. А из-

за случайности, присущей обучению нейронных сетей, всегда рекомендуется обучать сеть несколько раз в каждой конфигурации и усреднять результаты, прежде чем делать выводы.

Классификация аудио с помощью CNN

Представьте себе, что вы возглавляете группу ученых-климатологов, обеспокоенных сокращением тропических лесов на планете. Ежегодно мир теряет из-за незаконной вырубki до 10 млн акров¹ старых тропических лесов. Ваша команда планирует превратить тысячи выброшенных смартфонов в подслушивающие устройства на солнечных батареях и разместить их по всей Амазонке, чтобы передавать сигналы тревоги в случае появления звуков бензопил и двигателей грузовиков. Вам необходимо использующее искусственный интеллект программное обеспечение для идентификации таких звуков в режиме реального времени. И сделать это нужно быстро, поскольку изменение климата ждать не будет.

Эффективным способом классификации звуков является преобразование аудиопотоков в изображения спектрограмм (<https://oreil.ly/gpCij>), являющихся визуальным представлением спектров частот в зависимости от их изменения во времени, и использование CNN для классификации спектрограмм. Спектрограммы на рис. 10.21 были получены из WAV-файлов, содержащих звуки бензопилы. Используя трансферное обучение, создадим модель, способную идентифицировать характерные звуки лесозаготовительных работ и отличать их от окружающих звуков, таких как шум дикой природы и грозы.

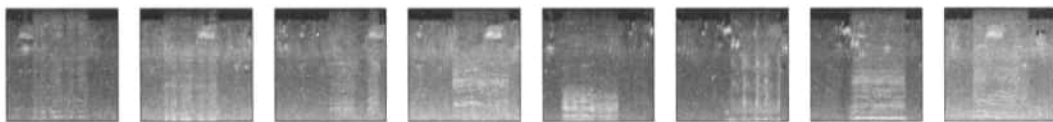


Рис. 10.21. Спектрограммы, полученные из аудиофайлов, содержащих звуки бензопилы



Руководство в этом разделе было создано под влиянием проекта Rainforest Connection (<https://oreil.ly/SkoZO>), в котором используются переработанные телефоны на базе Android для мониторинга тропических лесов на предмет обнаружения звуков незаконной деятельности. Анализом звука с телефонов занимается CNN, размещенная в облаке TensorFlow. Предполагается, что в будущем на самих телефонах (<https://oreil.ly/cN3Cv>) сможет работать TensorFlow Lite (<https://oreil.ly/2q9qJ>) — сокращенная версия TensorFlow, предназначенная для мобильных, встраиваемых и прочих устройств. Более подробную информацию можно найти в блоге Google AI "Борьба с незаконной вырубкой лесов с помощью TensorFlow" ("The Fight Against Illegal Deforestation with TensorFlow", <https://oreil.ly/aL4Ga>). Это лишь один из примеров того, как искусственный интеллект делает мир лучше.

¹ Более 40 тыс. км². — Прим. ред.

Для начала загрузите ZIP-файл (<https://oreil.ly/dkSeJ>), содержащий набор данных со звуками тропических лесов. (Внимание: размер файла составляет 666 Мбайт.) Создайте подкаталог Sounds в каталоге, где размещены ваши блокноты, и скопируйте содержимое ZIP-файла в этот подкаталог. Теперь Sounds содержит подкаталоги с названиями background, chainsaw, engine и storm. Каждый подкаталог хранит 100 WAV-файлов. WAV-файлы в подкаталоге background содержат только фоновые шумы тропического леса, а файлы в других подкаталогах — звуки бензопил (chainsaw), двигателей (engine) и грозы (thunderstorm), наложенные на фоновые шумы. Я создал эти файлы с помощью пакета синтеза звуковых ландшафтов Scarper (<https://oreil.ly/fs6os>) для комбинирования звуков из общедоступного набора данных UrbanSound8K (<https://oreil.ly/mNQit2>) со звуками тропического леса. Проиграйте несколько WAV-файлов на своем компьютере, чтобы получить представление о содержащихся в них звуках.

Теперь создайте блокнот Jupyter и вставьте в первую ячейку следующий код:

```
import numpy as np
import librosa.display, os
import matplotlib.pyplot as plt
%matplotlib inline

def create_spectrogram(audio_file, image_file):
    fig = plt.figure()
    ax = fig.add_subplot(1, 1, 1)
    fig.subplots_adjust(left=0, right=1, bottom=0, top=1)

    y, sr = librosa.load(audio_file)
    ms = librosa.feature.melspectrogram(y=y, sr=sr)
    log_ms = librosa.power_to_db(ms, ref=np.max)
    librosa.display.specshow(log_ms, sr=sr)

    fig.savefig(image_file)
    plt.close(fig)

def create_pngs_from_wavs(input_path, output_path):
    if not os.path.exists(output_path):
        os.makedirs(output_path)

    dir = os.listdir(input_path)

    for i, file in enumerate(dir):
        input_file = os.path.join(input_path, file)
        output_file = os.path.join(output_path,
            file.replace('.wav', '.png'))
        create_spectrogram(input_file, output_file)
```

В данном коде определена пара функций для преобразования WAV-файлов в изображения спектрограмм. `create_spectrogram` использует пакет Python Librosa (<https://oreil.ly/vbAc8>) для создания изображения спектрограммы из WAV-файла. `create_pngs_from_wavs` преобразует все WAV-файлы в указанном каталоге в изображения спектрограмм. Вам потребуется установить Librosa, если он еще не установлен.

С помощью следующих операторов можно создать PNG-файлы со спектрограммами из всех WAV-файлов, находящихся в подкаталогах каталога Sounds:

```
create_pngs_from_wavs('Sounds/background', 'Spectrograms/background')
create_pngs_from_wavs('Sounds/chainsaw', 'Spectrograms/chainsaw')
create_pngs_from_wavs('Sounds/engine', 'Spectrograms/engine')
create_pngs_from_wavs('Sounds/storm', 'Spectrograms/storm')
```

Проверьте каталог Spectrograms на наличие содержащих спектрограммы подкаталогов, и убедитесь, что каждый подкаталог имеет 100 PNG-файлов. Затем с помощью следующего кода определите две новые вспомогательные функции для загрузки и отображения спектрограмм и объявите два списка Python — один для хранения изображений спектрограмм, другой для хранения меток классов:

```
from tensorflow.keras.preprocessing import image

def load_images_from_path(path, label):
    images, labels = [], []

    for file in os.listdir(path):
        images.append(image.img_to_array(image.load_img(os.path.join(path, file),
                                                                target_size=(224, 224, 3))))
        labels.append((label))

    return images, labels

def show_images(images):
    fig, axes = plt.subplots(1, 8, figsize=(20, 20),
                             subplot_kw={'xticks': [], 'yticks': []})

    for i, ax in enumerate(axes.flat):
        ax.imshow(images[i] / 255)

x, y = [], []
```

С помощью следующих операторов можно загрузить изображения фоновых спектрограмм, добавить их в список с именем `x` и пометить нулями:

```
images, labels = load_images_from_path('Spectrograms/background', 0)
show_images(images)
```

```
x += images
y += labels
```

Повторите этот процесс для загрузки спектрограмм бензопилы из каталога Spectrograms\chainsaw, спектрограмм двигателя из каталога Spectrograms\engine и спектрограмм грозы из каталога Spectrograms\storm. Пометьте спектрограммы бензопилы единицами, спектрограммы двигателя — двойками, а спектрограммы грозы — тройками. В табл.10.1 перечислены метки для четырех классов изображений.

Таблица 10.1. Метки для четырех классов изображений

Тип спектрограммы	Метка
Фон (Background)	0
Бензопила (Chainsaw)	1
Двигатель (Engine)	2
Гроза (Storm)	3

Поскольку данная модель в перспективе может работать на мобильных телефонах, в качестве базовой сети мы будем использовать MobileNetV2. Примените следующий код для предварительной обработки пикселей и разделения изображений и меток на два набора данных — один для обучения, другой для тестирования:

```
from sklearn.model_selection import train_test_split
from tensorflow.keras.applications.mobilenet import preprocess_input
```

```
x = preprocess_input(np.array(x))
y = np.array(y)
```

```
x_train, x_test, y_train, y_test = train_test_split(x, y, stratify=y,
                                                    test_size=0.3, random_state=0)
```

Вызовите Keras-функцию MobileNetV2 (<https://oreil.ly/UyRAC>) для инстанцирования MobileNetV2 без классификационных слоев. Затем прогоните обучающие и тестовые данные через MobileNetV2 для извлечения признаков из изображений спектрограмм:

```
from tensorflow.keras.applications import MobileNetV2
```

```
base_model = MobileNetV2(weights='imagenet', include_top=False,
                          input_shape=(224, 224, 3))
```

```
train_features = base_model.predict(x_train)
test_features = base_model.predict(x_test)
```

Определите нейронную сеть для классификации извлеченных MobileNetV2 признаков:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
model = Sequential()
model.add(Flatten())
model.add(Dense(512, activation='relu'))
model.add(Dense(4, activation='softmax'))
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```



В качестве эксперимента я заменил слой Flatten слоем GlobalAveragePooling2D. Точность валидации несколько повысилась, но при тестировании модели на аудиозаписях, извлеченных из документального видео, обобщение оказалось не таким хорошим. Это подтверждает важный тезис из главы 9: полностью доверять модели можно только тогда, когда она тестируется на данных, с которыми ранее не сталкивалась, причем желательно на полученных из другого источника данных.

Обучите сеть с помощью функций:

```
hist = model.fit(train_features, y_train,
                 validation_data=(test_features, y_test),
                 batch_size=10, epochs=10)
```

Постройте графики точности обучения и валидации:

```
import seaborn as sns
sns.set()

acc = hist.history['accuracy']
val_acc = hist.history['val_accuracy']
epochs = range(1, len(acc) + 1)
plt.plot(epochs, acc, '-', label='Точность обучения')
plt.plot(epochs, val_acc, ':', label='Точность валидации')
plt.title('Точность обучения и валидации')
plt.xlabel('Эпоха')
plt.ylabel('Точность')
plt.legend(loc='lower right')
plt.plot()
```

Точность валидации должна достигать 95% и выше (рис. 10.22).

Прогоните тестовые изображения через сеть и используйте матрицу ошибок для оценки результатов:

```
from sklearn.metrics import ConfusionMatrixDisplay as cmd

sns.reset_orig()
fig, ax = plt.subplots(figsize=(4, 4))
ax.grid(False)
y_pred = model.predict(test_features)
class_labels = ['фон', 'бензопила', 'двигатель', 'гроза']

cmd.from_predictions(y_test, y_pred.argmax(axis=1),
                    display_labels=class_labels, colorbar=False,
                    cmap='Blues', xticks_rotation='vertical', ax=ax)
```

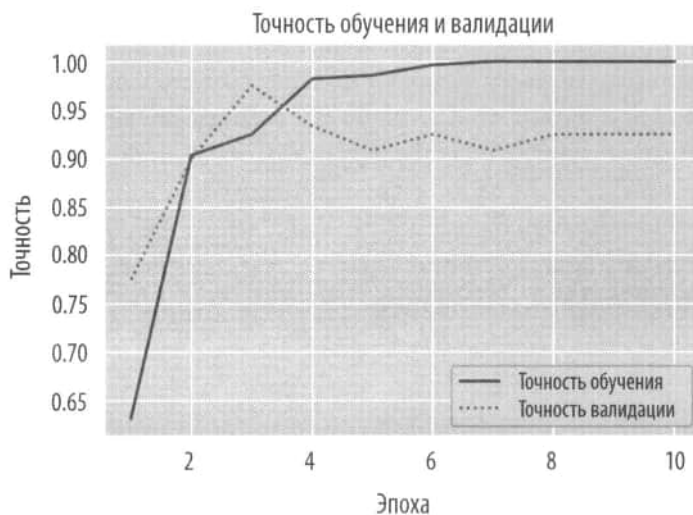


Рис. 10.22. Графики точности обучения и валидации CNN, распознающей звуки вырубki леса

Сеть достаточно хорошо умеет определять ролики, не содержащие звуков бензопил или двигателей (рис. 10.23). Иногда она путает звуки бензопилы и двигателя. Это нормально, поскольку наличие любого из них может свидетельствовать о незаконной деятельности в тропическом лесу.

В каталоге Sounds имеется подкаталог samples, содержащий WAV-файлы, на которых CNN не обучалась и не проверялась. Эти WAV-файлы не имеют ничего общего с образцами, использовавшимися для обучения и тестирования: они взяты из видеоролика YouTube, документирующего усилия Бразилии по борьбе с незаконной вырубкой леса. Давайте используем только что обученную модель для анализа этих файлов на предмет наличия в них звуков лесозаготовительной деятельности.

Начнем с создания спектрограммы первого образца WAV-файла, содержащего звук лесорубов, вырубаящих деревья в Амазонии:

```
create_spectrogram('Sounds/samples/sample1.wav',
                   'Spectrograms/sample1.png')
```

```
x = image.load_img('Spectrograms/sample1.png', target_size=(224, 224))
plt.xticks([])
plt.yticks([])
plt.imshow(x)
```

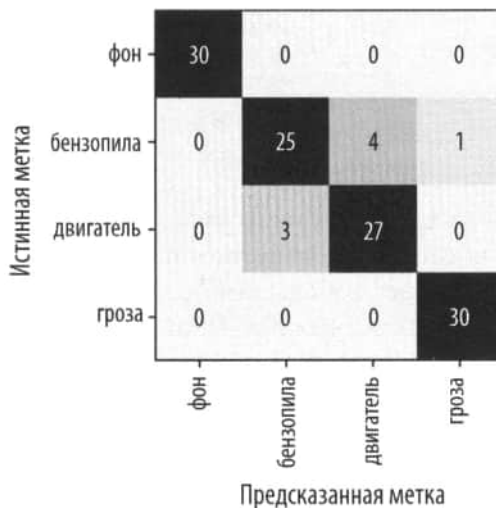


Рис. 10.23. Матрица ошибок CNN, распознающей звуки вырубki леса

Предварительно обработайте изображение спектрограммы, передайте его в MobileNetV2 для извлечения признаков и классифицируйте их:

```
x = image.img_to_array(x)
x = np.expand_dims(x, axis=0)
x = preprocess_input(x)
```

```
y = base_model.predict(x)
predictions = model.predict(y)
```

```
for i, label in enumerate(class_labels):
    print(f'{label}: {predictions[0][i]}')
```

Теперь создайте спектрограмму из WAV-файла, содержащего звук проносящегося сквозь тропический лес лесовоза:

```
create_spectrogram('Sounds/samples/sample2.wav',
                   'Spectrograms/sample2.png')
```

```
x = image.load_img('Spectrograms/sample2.png', target_size=(224, 224))
plt.xticks([])
plt.yticks([])
plt.imshow(x)
```

Предварительно обработайте изображение, передайте его в MobileNetV2 для извлечения признаков и классифицируйте их:

```
x = image.img_to_array(x)
x = np.expand_dims(x, axis=0)
x = preprocess_input(x)
y = base_model.predict(x)
predictions = model.predict(y)
```

```
for i, label in enumerate(class_labels):
    print(f'{label}: {predictions[0][i]}')
```

Если сеть ошиблась в одном из примеров, попробуйте обучить ее еще раз. Помните, что каждый раз нейронная сеть будет обучаться по-разному, в том числе и потому, что Keras инициализирует веса небольшими случайными значениями. В реальном мире специалисты по обработке данных часто обучают нейронную сеть несколько раз и усредняют результаты, чтобы оценить ее точность.

Резюме

Сверточные нейронные сети отлично справляются с классификацией изображений, поскольку они используют сверточные ядра для извлечения признаков из изображений с разными разрешениями — признаков, призванных подчеркнуть различия между классами. Сверточные слои используют сверточные ядра для извлечения признаков, а объединяющие слои уменьшают размер карт признаков, получаемых на выходе сверточных слоев. Выходные данные этих слоев поступают на вход полносвязных слоев для классификации. Keras предоставляет реализацию сверточных и объединяющих слоев в таких классах, как Conv2D и MaxPooling2D.

Обучение CNN с нуля при относительно высокой степени разделения между классами, например на наборе данных MNIST, вполне возможно на обычном ноутбуке или ПК. Обучение CNN для решения более сложных задач требует большего количества обучающих изображений и, соответственно, большей вычислительной мощности. Трансферное обучение является практической альтернативой обучению CNN с нуля. Оно использует знание, уже присутствующее в "бутылочных" слоях предварительно обученных CNN, для извлечения признаков из изображений, а затем применяет собственные классификационные слои для интерпретации результатов.

Аугментация данных может повысить точность CNN, обученной на относительно небольшом количестве изображений, и особенно полезна при трансферном обуче-

нии. Аугментация данных включает в себя применение случайных преобразований, таких как смещение и поворот, к обучающим изображениям. Вы можете преобразовывать изображения перед подачей их в сеть с помощью класса `ImageDataGenerator` в Keras либо встроить преобразования в сеть с помощью таких слоев, как `RandomRotation` и `RandomTranslation`. Слои, преобразующие изображения, активны во время обучения, но неактивны, когда сеть делает прогнозы.

CNN применимы к широкому кругу задач компьютерного зрения и практически единолично ответственны за быстрый прогресс, достигнутый в этой области за последнее десятилетие. Они играют важную роль и в современных системах распознавания лиц. Хотите узнать больше? Обнаружение и идентификация лиц на фотографиях — тема следующей главы.

Обнаружение и распознавание лиц

Не так давно я летел в Европу и был удивлен тем, что мне не пришлось предъявлять паспорт. Я прошел перед камерой и благополучно поднялся на борт самолета. Это было частью пилотного проекта компании Delta Air Lines, направленного на внедрение системы распознавания лиц (<https://oreil.ly/tRoSq>) и обеспечение бесконтактного перемещения от "двери до двери".

Распознавание лиц встречается повсеместно. Это одно из наиболее распространенных, а иногда и противоречивых применений искусственного интеллекта. Компания Meta¹, ранее известная как Facebook, использует его для маркировки друзей на фотографиях (по крайней мере, так было до тех пор, пока она не отказалась от этой функции из-за проблем с конфиденциальностью, <https://oreil.ly/ffn8n>). Apple использует его для разблокировки iPhone (<https://oreil.ly/X9j5J>), а Microsoft — для разблокировки компьютеров Windows. Компания Uber применяет эту функцию для подтверждения личности своих водителей (<https://oreil.ly/ZkRAw>). При правильном использовании система распознавания лиц обладает огромным потенциалом для того, чтобы сделать мир лучше, безопаснее и надежнее.

Предположим, вы хотите создать систему, идентифицирующую людей на фотографиях или видеокадрах. Возможно, это часть системы безопасности, разрешающей доступ в общежития колледжа только имеющим на это право студентам и сотрудникам (<https://oreil.ly/NXt8q>). А может быть, вы пишете приложение, которое ищет на жестком диске фотографии знакомых вам людей ("Покажи мне все фотографии меня и моей дочери"). Для создания подобных систем требуются алгоритмы или модели, способные:

- ♦ находить лица на фотографиях или в видеокадрах — процесс, известный как *обнаружение лиц*;
- ♦ идентифицировать обнаруженные лица — процесс, известный как *распознавание* или *идентификация лиц*.

Существует множество известных алгоритмов для поиска и идентификации лиц на фотографиях. Некоторые из них основаны на глубоком обучении, в частности, на сверточных нейронных сетях, а некоторые — нет. В конце концов, распознавание лиц появилось на несколько десятилетий раньше, чем глубокое обучение. Однако

¹ В России компания признана экстремистской организацией. — Прим. ред.

глубокое обучение сделало технологию распознавания лиц более совершенной и практичной, чем она была прежде.

В начале этой главы мы познакомимся с двумя популярными методами обнаружения лиц. Затем перейдем к распознаванию лиц и представим трансферное обучение как средство этой технологии. В конце главы приводится руководство, с помощью которого читатель сможет собрать всё вместе и создать собственную систему идентификации лиц. Звучит заманчиво? Тогда давайте приступим.

Обнаружение лиц

В последующих разделах представлены два широко используемых алгоритма обнаружения лиц — один, основанный на машинном обучении, и другой, использующий глубокое обучение, — а также библиотеки, реализующие их. Задача состоит в том, чтобы уметь находить все лица на фото- или видеокadre (рис. 11.1). Далее я представлю простую в использовании функцию, которую можно вызвать для извлечения всех изображений лиц из фотографии и сохранения их на диске или передачи в модель распознавания лиц.



Рис. 11.1. Обнаружение лиц

Обнаружение лиц с помощью алгоритма Виолы — Джонса

Один из самых быстрых и популярных алгоритмов обнаружения лиц на фотографиях основан на опубликованной в 2001 году работе "Быстрое обнаружение объектов с помощью усиленного каскада простых признаков" ("Rapid Object Detection

Using a Boosted Cascade of Simple Features"; <https://oreil.ly/uLz4E>). Алгоритм, известный под названием *"алгоритм Виолы — Джонса"* (Пол Виола и Майкл Джонс — авторы статьи), основан на использовании относительных интенсивностей соседних блоков пикселей. Например, средняя интенсивность пикселей в прямоугольнике вокруг глаз обычно темнее, чем средняя интенсивность пикселей в прямоугольнике непосредственно под этой областью. Аналогично, переносица обычно светлее, чем область вокруг глаз, поэтому два темных прямоугольника со светлым прямоугольником посередине могут представлять два глаза и нос. Наличие в кадре множества таких Хаар-подобных признаков (Haar-like feature, <https://oreil.ly/zrFxR>) в нужных местах является индикатором того, что кадр содержит лицо (рис. 11.2).



Рис. 11.2. Обнаружение лиц с использованием Хаар-подобных признаков

Алгоритм Виолы — Джонса работает по принципу скольжения окон различных размеров по изображению в поисках рамок с Хаар-подобными признаками в подходящих местах. При каждой остановке пиксели в окне масштабируются до заданного размера (обычно 24×24), признаки извлекаются и поступают в бинарный классификатор, который выдает положительный результат, указывающий на наличие в кадре лица, или отрицательный, указывающий на его отсутствие. Затем окно перемещается в следующую позицию, и процесс обнаружения начинается сначала.

Ключевым моментом в работе алгоритма Виолы — Джонса является бинарный классификатор. Кадр шириной и высотой по 24 пиксела содержит более 160 000 комбинаций прямоугольников, представляющих собой потенциальные Хаар-подобные признаки. Вместо того чтобы вычислять значения для каждой комбинации, алгоритм Виолы — Джонса вычисляет лишь те, которые требуются классификатору. Более того, количество требуемых классификатором признаков зависит от содержимого кадра. Фактически классификатор представляет собой несколько бинарных классификаторов, расположенных поэтапно. На первом этапе может потребоваться всего один признак. На втором этапе — 10 признаков, на третьем — 20 и т. д. Признаки извлекаются и передаются на этап n только в том случае, если на этапе $n - 1$ получен положительный результат, что и дало основание использовать термин *"каскадный классификатор"*.

На рис. 11.3 показан трехступенчатый каскадный классификатор. Каждый этап тщательно настраивается, чтобы достичь 100%-ного уровня обнаружения при ис-

пользовании ограниченного числа признаков даже при высоком уровне ложных срабатываний. На первом этапе один из признаков определяет, содержит ли кадр лицо. Положительный ответ означает, что кадр *может* содержать лицо, отрицательный — что, скорее всего, нет, и в этом случае дальнейшие проверки не проводятся. Однако если на первом этапе получен положительный ответ, то извлекаются еще 10 признаков, которые передаются на второй этап. Кадр считается содержащим лицо только в том случае, если *все* этапы дали положительный ответ, что дает суммарный коэффициент ложных срабатываний, близкий к нулю. В машинном обучении такая схема известна как "*высокая полнота, затем точность*", поскольку, хотя отдельные этапы настраиваются на высокую полноту, суммарный эффект дает высокую точность.



Рис. 11.3. Обнаружение лиц с помощью каскадного классификатора²

Преимуществом такой архитектуры является то, что не содержащие лиц рамки быстро отбрасываются, поскольку вызывают негативную реакцию на ранних этапах каскада. Так как большинство рамок *не* содержат лиц, алгоритм работает очень быстро, пока не встретит рамку с лицом. При тестировании 38-ступенчатого классификатора, обученного на 6061 признаке из 4916 изображений лиц, алгоритм Виолы — Джонса обнаружил, что в среднем из каждой рамки извлекается всего 10 признаков.

Эффективность алгоритма Виолы — Джонса зависит от каскадного классификатора, который, по сути, является моделью машинного обучения, обученной на изображениях лиц и изображениях без лиц. Обучение происходит медленно, но прогнозы выполняются быстро. В каком-то смысле алгоритм действует как CNN, созданная для извлечения минимального количества признаков, необходимых для определения того, содержит ли рамка лицо. Для ускорения извлечения признаков алгоритм Виолы — Джонса использует хитрый математический прием под названием "*интегральные изображения*" (<https://oreil.ly/7Z79V>), позволяющий быстро вычислить разницу в интенсивности между двумя блоками пикселей. В результате система может с достаточно высокой точностью определять граничные рамки, окружающие лица на изображении, и делать это достаточно быстро для обнаружения лиц в кадрах транслируемого в реальном времени видео.

² FP-срабатывание — ложноположительное срабатывание (от англ. *false-positive*). — Прим. ред.

Использование реализации алгоритма Виолы — Джонса в OpenCV

OpenCV (<https://oreil.ly/IWvGS>) — это популярная библиотека компьютерного зрения с открытым исходным кодом, бесплатная для коммерческого использования. Она предоставляет реализацию алгоритма Виолы — Джонса в классе `CascadeClassifier` (<https://oreil.ly/NJA1z>), а также XML-файл, содержащий обученный распознавать лица каскадный классификатор. В следующих операторах `CascadeClassifier` используется в блокноте Jupyter для обнаружения лиц на изображении и отрисовки прямоугольников вокруг них. Вы можете взять собственное изображение или загрузить с GitHub использованное в моем примере изображение (<https://oreil.ly/PM24X>):

```
import cv2
from cv2 import CascadeClassifier
from matplotlib.patches import Rectangle
import matplotlib.pyplot as plt
%matplotlib inline

image = plt.imread('Data/Amsterdam.jpg')
fig, ax = plt.subplots(figsize=(12, 8), subplot_kw={'xticks': [], 'yticks': []})
ax.imshow(image)

model = CascadeClassifier(cv2.data.harcascades +
                          'haarcascade_frontalface_default.xml')
faces = model.detectMultiScale(image)

for face in faces:
    x, y, w, h = face
    rect = Rectangle((x, y), w, h, color='red', fill=False, lw=2)
    ax.add_patch(rect)
```

На рис. 11.4 представлены выходные данные с фотографией матери и ее дочери (фото сделано в Амстердаме несколько лет назад).

`CascadeClassifier` обнаружил два лица на фотографии, но при этом произошло несколько ложных срабатываний. Одним из способов борьбы с ними является использование параметра `minNeighbors`. По умолчанию он равен 3, но более высокие значения делают `CascadeClassifier` избирательнее. При `minNeighbors=20` функция `detectMultiScale` находит только лица двух людей:

```
faces = model.detectMultiScale(image, minNeighbors=20)
```

На рис. 11.5 приведен результат.



Рис. 11.4. Результат обнаружения лиц на фотографии с применением класса CascadeClassifier (с ложными срабатываниями)



Рис. 11.5. Результат обнаружения лиц на фотографии (без ложных срабатываний)

При анализе изображения метод `detectMultiScale`, как правило, обнаруживает лицо несколько раз, и в каждом случае оно определяется выравненной по-разному ограничительной рамкой. Параметр `minNeighbors` задает минимальное количество раз обнаружения лица, чтобы оно было признано таковым. Более высокие значения

обеспечивают более высокую точность (меньшее количество ложных срабатываний), но при этом снижается полнота, т. е. некоторые лица могут быть не обнаружены.

CascadeClassifier часто приходится настраивать таким образом, чтобы отыскать оптимальный баланс между нахождением слишком большого количества лиц и слишком малого их числа. При этом он является одним из самых быстрых среди существующих алгоритмов обнаружения лиц. Его можно использовать и для обнаружения иных объектов, кроме лиц, загружая XML-файлы, содержащие другие предварительно обученные классификаторы. В репозитории OpenCV на GitHub (<https://oreil.ly/Aj0At>) можно найти XML-файлы для обнаружения столовых приборов и других объектов с помощью Хаар-подобных признаков, а также XML-файлы для обнаружения объектов с помощью другого типа дискриминатора, называемого локальным бинарным шаблоном (<https://oreil.ly/a9p27>).

Обнаружение лиц с помощью сверточных нейронных сетей

Несмотря на более высокие вычислительные затраты, методы глубокого обучения часто справляются с обнаружением лиц на изображениях лучше, чем алгоритм Виолы — Джонса. В частности, *многозадачные каскадные сверточные нейронные сети* (multitask cascaded convolutional neural networks, MTCNN) доказали свою эффективность в обнаружении лиц в различных тестах. Они также определяют такие ориентиры на лице, как глаза, нос и рот.

Рисунок 11.6 воспроизводит схему из статьи 2016 года "Совместное обнаружение и выравнивание лиц с помощью многозадачных каскадных сверточных сетей" ("Joint Face Detection and Alignment Using Multitask Cascaded Convolutional Networks", <https://oreil.ly/QxjCe>), в которой предлагается схема работы MTCNN. В MTCNN для обнаружения лиц используются три последовательно расположенные CNN. Первая из них, называемая *сетью предложений* (proposal network, P-Net), представляет собой неглубокую CNN, просматривающую изображение различного разрешения в поисках указывающих на лица признаков. Выявленные P-Net прямоугольники объединяются в прямоугольники-кандидаты на лица и поступают на вход *сети усовершенствований* (refine network, R-Net) — более глубокой CNN, которая более тщательно изучает каждый прямоугольник и отбрасывает те, в которых отсутствуют лица. Наконец, выходные данные из R-Net поступают в *выходную сеть* (output network, O-Net), осуществляющую дальнейшую фильтрацию прямоугольников-кандидатов и выявление лицевых ориентиров. MTCNN — это многозадачные CNN, поскольку они производят не один, а три выхода — классификационный (определяющий степень уверенности в том, что прямоугольник содержит лицо) и два регрессионных, определяющих местоположение лица и лицевых ориентиров. И они каскадируются, как классификаторы Виолы — Джонса, чтобы быстро исключить рамки, не содержащие лица.

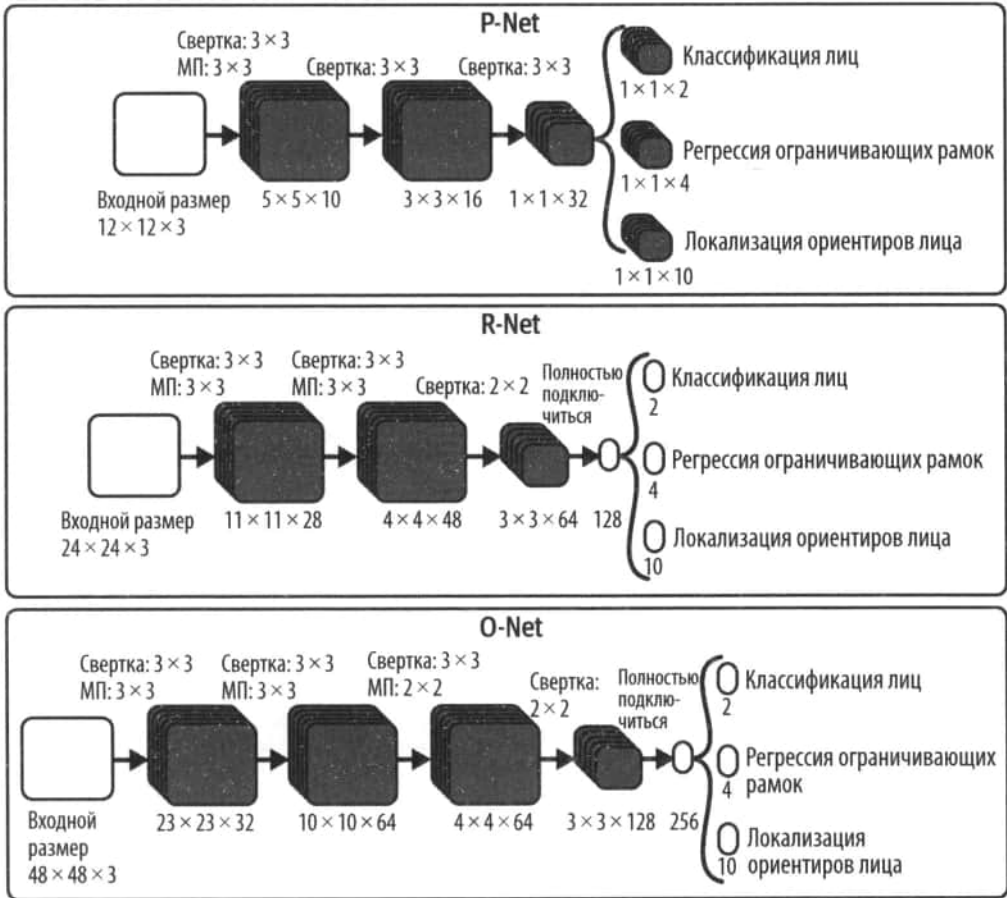


Рис. 11.6. Многозадачная каскадная сверточная нейронная сеть³

Практичная реализация MTCNN доступна в пакете Python, носящем название MTCNN. В следующем коде она используется для обнаружения лиц на той же фотографии, что и в предыдущем примере:

```
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle
from mtcnn.mtcnn import MTCNN
%matplotlib inline

image = plt.imread('Data/Amsterdam.jpg')
fig, ax = plt.subplots(figsize=(12, 8), subplot_kw={'xticks': [], 'yticks': []})
ax.imshow(image)
detector = MTCNN()
faces = detector.detect_faces(image)
```

³ МП — максимальный пулинг. — Прим. ред.

for face in faces:

```
x, y, w, h = face['box']
```

```
rect = Rectangle((x, y), w, h, color='red', fill=False, lw=2)
```

```
ax.add_patch(rect)
```

На рис. 11.7 приведен результат.



Рис. 11.7. Результат обнаружения лиц с помощью MTCNN (с ложными срабатываниями)

MTCNN обнаружила не только лица двух людей, но и лицо статуи, отражающееся в двери позади них. Вот что на самом деле возвращает `detect_faces` — список, содержащий три словаря, каждый из которых соответствует одному из лиц на фотографии:

```
[
    {
        'box': [723, 248, 204, 258],
        'confidence': 0.9997798800468445,
        'keypoints': {
            'left_eye': (765, 341),
            'right_eye': (858, 343),
            'nose': (800, 408),
            'mouth_left': (770, 432),
            'mouth_right': (864, 433)
        }
    },
    {
        'box': [723, 248, 204, 258],
        'confidence': 0.9997798800468445,
        'keypoints': {
            'left_eye': (765, 341),
            'right_eye': (858, 343),
            'nose': (800, 408),
            'mouth_left': (770, 432),
            'mouth_right': (864, 433)
        }
    },
    {
        'box': [723, 248, 204, 258],
        'confidence': 0.9997798800468445,
        'keypoints': {
            'left_eye': (765, 341),
            'right_eye': (858, 343),
            'nose': (800, 408),
            'mouth_left': (770, 432),
            'mouth_right': (864, 433)
        }
    }
]
```

```
{
  'box': [538, 258, 183, 232],
  'confidence': 0.9997591376304626,
  'keypoints': {
    'left_eye': (601, 353),
    'right_eye': (685, 344),
    'nose': (662, 394),
    'mouth_left': (614, 433),
    'mouth_right': (689, 424)
  }
},
{
  'box': [1099, 84, 40, 41],
  'confidence': 0.8863282203674316,
  'keypoints': {
    'left_eye': (1108, 101),
    'right_eye': (1123, 96),
    'nose': (1116, 102),
    'mouth_left': (1114, 115),
    'mouth_right': (1127, 111)
  }
}
]
```

Устранить лицо в отражении можно одним из двух способов: игнорируя лица с уровнем уверенности (*confidence*) ниже определенного порога или передавая параметр *min_face_size* в функцию *MTCNN*, чтобы *detect_faces* игнорировала лица меньше указанного размера. Вот модифицированный цикл *for*, реализующий первый вариант:

```
for face in faces:
    if face['confidence'] > 0.9:
        x, y, w, h = face['box']
        rect = Rectangle((x, y), w, h, color='red', fill=False, lw=2)
        ax.add_patch(rect)
```

На рис. 11.8 приведен результат.

Прямоугольники вокруг лиц на рис. 11.1 были сгенерированы с использованием настроек *MTCNN* по умолчанию, т. е. без какой-либо фильтрации на основе уровней уверенности или размеров лица. В целом *MTCNN* справляется с задачей обнаружения лиц лучше, чем *CascadeClassifier*.



Рис. 11.8. Результат обнаружения лиц с помощью MTCNN (без ложных срабатываний)

Извлечение лиц из фотографий

Если вы знаете, как находить лица на фотографиях, то извлечение изображений лиц для обучения модели или передачи их обученной модели для идентификации не составит труда. В примере 11.1 представлена функция Python, которая принимает путь к файлу изображения и возвращает список изображений лиц. По умолчанию она обрезает изображения лиц так, чтобы они были квадратными (идеально подходит для передачи в CNN), но вы можете отключить обрезку, указав в функции параметр `crop=False`. Вы также можете задать минимальный уровень уверенности с помощью параметра `min_confidence`, который по умолчанию равен 0,9.

Пример 11.1. Функция для извлечения изображений лиц из фотографии

```
import numpy as np
from PIL import Image, ImageOps
from mtcnn.mtcnn import MTCNN

def extract_faces(input_file, min_confidence=0.9, crop=True):
    # Загрузка изображения и его правильная ориентация
    pil_image = Image.open(input_file)
    exif = pil_image.getexif()

    for k in exif.keys():
        if k != 0x0112:
```

```
exif[k] = None
del exif[k]
```

```
pil_image.info["exif"] = exif.tobytes()
pil_image = ImageOps.exif_transpose(pil_image)
image = np.array(pil_image)
```

```
# Определение лиц на изображении
```

```
detector = MTCNN()
faces = detector.detect_faces(image)
faces = [face for face in faces if face['confidence'] >= min_confidence]
results = []
```

```
for face in faces:
```

```
    x1, y1, w, h = face['box']
```

```
    if (сгор):
```

```
        # Вычисление координат обрезки
```

```
        if w > h:
```

```
            x1 = x1 + ((w - h) // 2)
```

```
            w = h
```

```
        elif h > w:
```

```
            y1 = y1 + ((h - w) // 2)
```

```
            h = w
```

```
# Извлечение изображения лица и добавление его в список
```

```
x2 = x1 + w
```

```
y2 = y1 + h
```

```
results.append(Image.fromarray(image[y1:y2, x1:x2]))
```

```
# Возврат всех изображений лиц
```

```
return results
```

Я передал в функцию фотографию (<https://oreil.ly/gI4r2>) с рис. 11.9, и она вернула все лица. Возвращаемые функцией `extract_faces` элементы являются изображениями Python Imaging Library (PIL, <https://oreil.ly/VsHPz>), поэтому вы можете изменить их размер или сохранить на диск с помощью одной строки кода. Вот фрагмент кода, извлекающий все лица из фотографии, изменяющий их размер до 224×224 пикселей и сохраняющий измененные изображения:

```
faces = extract_faces('PATH_TO_IMAGE_FILE')
```

```
for i, face in enumerate(faces):
```

```
    face.resize((224, 224)).save(f'face{i}.jpg')
```



Рис. 11.9. Изображения лиц, извлеченные из фотографии

С помощью `extract_faces` можно сравнительно просто сгенерировать набор изображений лиц для обучения CNN из набора фотографий с жесткого диска или извлечь лица из фотографии и передать их CNN для идентификации.

Распознавание лиц

Теперь, когда вы знаете, как обнаруживать лица на фотографиях, следующий шаг — научиться их идентифицировать. За прошедшие годы было разработано несколько алгоритмов распознавания лиц на фотографиях (<https://oreil.ly/E7D32>). Некоторые из них опираются на биометрические показатели, такие как расстояние между глазами или текстура кожи, в то время как другие используют более целостный подход, рассматривая идентификацию лиц как проблему распознавания образов. Современные модели сегодня часто опираются на глубокие сверточные нейронные сети. Одним из основных эталонов для моделей распознавания лиц является набор данных Labeled Faces in the Wild (LFW, <https://oreil.ly/0hylZ>), часть которого представлена на рис. 11.10 – содержащий более 13 000 собранных из Ин-

тернета изображений лиц более 5000 человек. Модели глубокого обучения, такие как MobiFace (<https://oreil.ly/9TwRP>) и FaceNet (<https://oreil.ly/cXIF6>), регулярно достигают более 99% точности при работе с этим набором данных. Это равняется способности человека идентифицировать лица на фотографиях LFW или превосходит ее (<https://oreil.ly/WEMnj>).



Рис. 11.10. Фрагмент набора данных Labeled Faces in the Wild (цветные фото)

В главе 5 была представлен метод опорных векторов (SVM), достигший 85% точности при использовании подмножества из 500 изображений (по 100 изображений пяти известных людей из этого набора данных). В главе 9 та же проблема была решена с помощью нейронной сети, и результаты были аналогичными. Эти модели лишь поверхностно затрагивают возможности современного распознавания лиц. Давайте применим CNN и трансферное обучение к тому же поднабору LFW и посмотрим, смогут ли они лучше распознавать лица на фотографиях. Попутно вы получите ценный урок о предварительно обученных CNN и специфике весов, генерируемых при обучении этих CNN.

Применение трансферного обучения для распознавания лиц

Первым шагом в изучении распознавания лиц на основе CNN является загрузка набора данных LFW. На этот раз мы загрузим полноразмерные цветные изображения и обрежем их до 128×128 пикселей. Вот код:

```
import pandas as pd
from sklearn.datasets import fetch_lfw_people
```

```

faces = fetch_lfw_people(min_faces_per_person=100, slice_=None,
                        resize=1.0, color=True)
faces.images = faces.images[:, 60:188, 60:188]
faces.data = faces.images.reshape(faces.images.shape[0],
                                faces.images.shape[1] * faces.images.shape[2],
                                faces.images.shape[3])
class_count = len(faces.target_names)

print(faces.target_names)
print(faces.images.shape)

```

Поскольку мы установили `min_faces_per_person` равным 100, всего было загружено 1140 изображений лиц, соответствующих пяти людям. Используйте следующие операторы для вывода на экран первых нескольких изображений и сопровождающих их меток:

```

import matplotlib.pyplot as plt
%matplotlib inline

fig, ax = plt.subplots(3, 6, figsize=(18, 10))

for i, axi in enumerate(ax.flat):
    axi.imshow(faces.images[i])
    axi.set(xticks=[], yticks=[], xlabel=faces.target_names[faces.target[i]])

```

Набор данных является несбалансированным и содержит почти столько же фотографий Джорджа Буша-младшего, сколько и всех остальных вместе взятых. Используйте следующий код, чтобы уменьшить набор данных до 100 изображений на каждого человека для всего набора из 500 изображений лиц:

```

import numpy as np

mask = np.zeros(faces.target.shape, dtype=bool)

for target in np.unique(faces.target):
    mask[np.where(faces.target == target)[0][:100]] = 1

x_faces = faces.data[mask]
y_faces = faces.target[mask]
x_faces = np.reshape(x_faces, (x_faces.shape[0], faces.images.shape[1],
                              faces.images.shape[2], faces.images.shape[3]))
x_faces.shape

```

Теперь обработайте значения пикселей для ввода в предварительно обученную CNN ResNet50 и используйте функцию `train_test_split` из Scikit-Learn для разделения набора данных и получения 400 обучающих и 100 тестовых образцов.

```

from sklearn.model_selection import train_test_split
from tensorflow.keras.applications.resnet50 import preprocess_input

```

```
face_images = preprocess_input(np.array(x_faces * 255))
```

```
x_train, x_test, y_train, y_test = train_test_split(
    face_images, y_faces, train_size=0.8, stratify=y_faces, random_state=0)
```

При желании вы можете разделить значения прошедших предварительную обработку пикселей на 255 и обучить CNN с нуля прямо сейчас на этих данных. Вот как это можно сделать (на случай, если захотите попробовать):

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Conv2D, MaxPooling2D

model = Sequential()
model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(x_train.shape[1:])))
model.add(MaxPooling2D(2, 2))
model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(MaxPooling2D(2, 2))
model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(MaxPooling2D(2, 2))
model.add(Flatten())
model.add(Dense(1024, activation='relu'))
model.add(Dense(class_count, activation='softmax'))
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.fit(x_train / 255, y_train, validation_data=(x_test / 255, y_test),
          epochs=20, batch_size=10)
```

Я проделал это, а затем построил графики точности обучения и валидации (рис. 11.11).

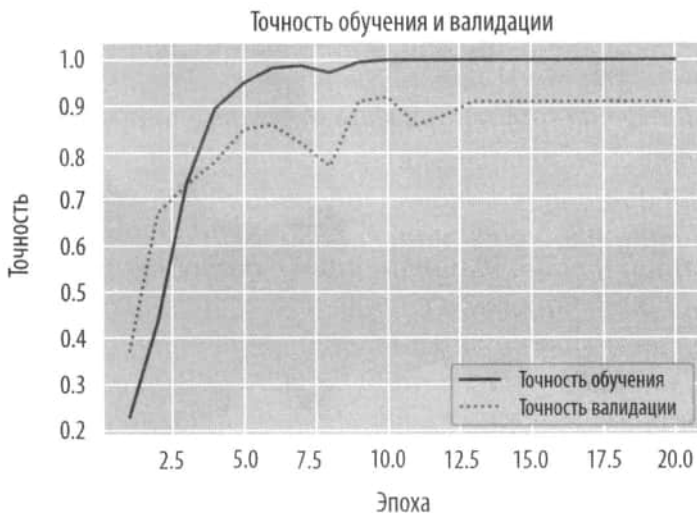


Рис. 11.11. Графики точности обучения и валидации модели CNN, распознающей лица по цветным фото

Точность проверки выше, чем у метода опорных векторов или обычной нейронной сети, но она и близко не стоит с тем, чего достигают современные CNN на наборе данных LFW. Поэтому очевидно, что есть лучший способ.

И им, конечно же, является трансферное обучение, о котором мы рассказывали в *главе 10*. ResNet50 была обучена на более чем 1 млн изображений из набора данных ImageNet, поэтому она должна быть искусна в извлечении признаков из фотографий — в большей степени, чем самодельная CNN, обученная на 400 изображениях. Давайте проверим, так ли это. С помощью следующих операторов загрузите слои извлечения признаков ResNet50, инициализируйте их весами ImageNet и заморозьте их, чтобы веса не изменялись во время обучения:

```
from tensorflow.keras.applications import ResNet50
```

```
base_model = ResNet50(weights='imagenet', include_top=False)
```

```
base_model.trainable = False
```

Теперь добавьте классификационные слои к базовой модели и включите слой Resizing для изменения размера поступающих в сеть изображений до размера, ожидаемого ResNet50:

```
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import Flatten, Dense, Resizing
```

```
model = Sequential()
```

```
model.add(Resizing(224, 224))
```

```
model.add(base_model)
```

```
model.add(Flatten())
```

```
model.add(Dense(1024, activation='relu'))
```

```
model.add(Dense(class_count, activation='softmax'))
```

```
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

Обучите модель и постройте графики точности обучения и валидации:

```
import seaborn as sns
```

```
sns.set()
```

```
hist = model.fit(x_train, y_train, validation_data=(x_test, y_test),
                batch_size=10, epochs=10)
```

```
acc = hist.history['accuracy']
```

```
val_acc = hist.history['val_accuracy']
```

```
epochs = range(1, len(acc) + 1)
```

```
plt.plot(epochs, acc, '-', label='Точность обучения')
```

```
plt.plot(epochs, val_acc, ':', label='Точность валидации')
```

```
plt.title('Точность обучения и валидации')
plt.xlabel('Эпоха')
plt.ylabel('Точность')
plt.legend(loc='lower right')
plt.plot()
```

Результаты могут быть разными, но в моем случае точность валидации составила около 94% (рис. 11.12).

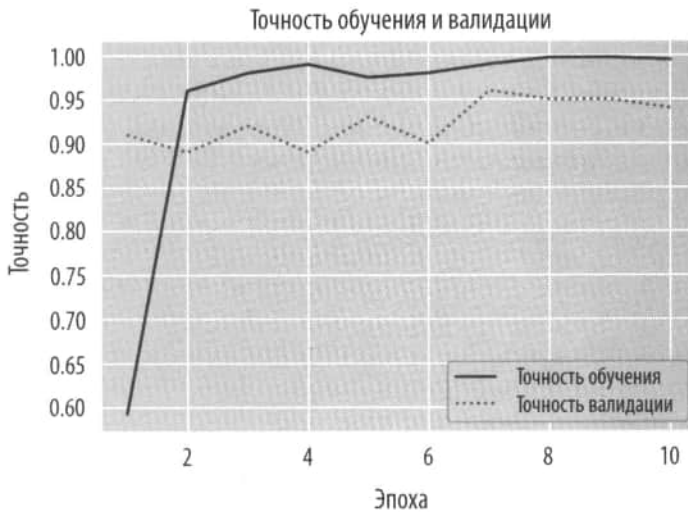


Рис. 11.12. Графики точности обучения и валидации модели, распознающей лица, с применением предобученной CNN

Это лучше, чем при использовании обученной с нуля CNN, и свидетельствует о том, что ResNet50 лучше справляется с извлечением признаков из изображений лиц. Но это все еще не лучший результат. Можно ли еще приблизиться к идеалу?

Совершенствование трансферного обучения с помощью весов для конкретной задачи

Инициализированная с весами ImageNet, ResNet50 выполняет надежную работу по извлечению признаков. Эти веса были получены в результате обучения ResNet50 на более чем 1 млн фотографий объектов — от баскетбольных мячей до бабочек. Однако она не обучалась на изображениях лиц. Будет ли она лучше извлекать признаки из изображений лиц, если ее обучить на них?

В 2017 году исследователи из группы визуальной геометрии Оксфордского университета опубликовали работу под названием "VGGFace2: набор данных для распознавания лиц разных положений и возрастов" ("VGGFace2: A Dataset for Recognising Faces Across Pose and Age", <https://oreil.ly/ssMNY>). Собрав набор данных, состоящий из нескольких миллионов изображений лиц, они обучили на нем две вариации

ResNet50 и опубликовали результаты. Исследователи также опубликовали весовые коэффициенты, упакованные в удобную библиотеку Python под названием Keras-vggface (<https://oreil.ly/46cYu>). Эта библиотека включает класс VGGFace, инкапсулирующий ResNet50 с весами, совместимыми с TensorFlow. Буквально сразу модель VGGFace способна распознавать лица тысяч знаменитостей — от Бри Ларсон до Дженнифер Энистон. Но ее настоящая ценность заключается в использовании трансферного обучения, позволяющего перенастроить ее на распознавание лиц, которым она не была обучена ранее.

Для того чтобы упростить задачу, я установил Keras-vggface, создал экземпляр VGGFace без классификационных слоев, инициализировал веса и сохранил модель в H5-файл с именем vggface.h5. Загрузите этот файл (<https://oreil.ly/DV3Wc>) и поместите его в подкаталог Data вашего блокнота. Затем используйте следующий код для создания экземпляра VGGFace, построенного поверх ResNet50, и добавьте пользовательские классификационные слои:

```
from tensorflow.keras.models import load_model

base_model = load_model('Data/vggface.h5')
base_model.trainable = False

model = Sequential()
model.add(Resizing(224, 224))
model.add(base_model)
model.add(Flatten())
model.add(Dense(1024, activation='relu'))
model.add(Dense(class_count, activation='softmax'))
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

Затем обучите модель и постройте графики точности обучения и валидации:

```
hist = model.fit(x_train, y_train, validation_data=(x_test, y_test),
                batch_size=10, epochs=10)

acc = hist.history['accuracy']
val_acc = hist.history['val_accuracy']
epochs = range(1, len(acc) + 1)

plt.plot(epochs, acc, '-', label='Точность обучения')
plt.plot(epochs, val_acc, ':', label='Точность валидации')
plt.title('Точность обучения и валидации')
plt.xlabel('Эпоха')
plt.ylabel('Точность')
plt.legend(loc='lower right')
plt.plot()
```

Результаты потрясающие (рис. 11.13).

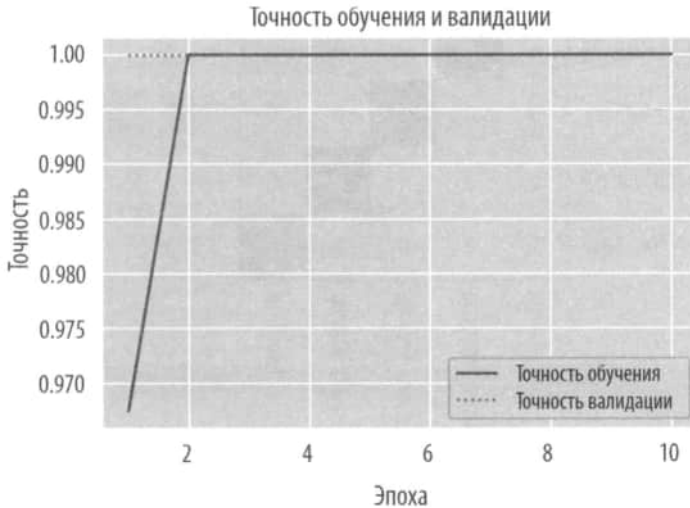


Рис. 11.13. Графики точности обучения и валидации модели, распознающей лица, с использованием библиотеки Keras-vggface

Для полной уверенности пропустите тестовые данные через сеть и оцените результаты с помощью матрицы ошибок:

```
from sklearn.metrics import ConfusionMatrixDisplay as cmd
```

```
sns.reset_orig()
y_pred = model.predict(x_test)
fig, ax = plt.subplots(figsize=(5, 5))
ax.grid(False)

cmd.from_predictions(y_test, y_pred.argmax(axis=1),
                    display_labels=faces.target_names, colorbar=False,
                    cmap='Blues', xticks_rotation='vertical', ax=ax)
```

Поскольку VGGFace была настроена на извлечение признаков из изображений лиц, она получила отличный результат на 100 тестовых изображениях. Это не значит, что она никогда не сможет ошибиться в распознавании лица. Но это говорит о том, что на том наборе данных, на котором вы ее обучали, она прекрасно справляется с извлечением признаков из изображений лиц (рис. 11.14).

И в этом кроется важный урок. CNN, обученные для конкретной задачи, часто обеспечивают лучшую базу для трансферного обучения, чем CNN, обученные более общим способом. Если задача состоит в распознавании лиц, то CNN, обученная на изображениях лиц, почти всегда будет работать лучше, чем CNN, обученная на фотографиях тысяч разнородных объектов. Для нейронной сети веса оказываются важнее всего.

Истинная метка	Колин Пауэлл	20	0	0	0	0
	Дональд Рамсфелд	0	20	0	0	0
	Джордж Буш мл.	0	0	20	0	0
	Герхард Шредер	0	0	0	20	0
	Тони Блэр	0	0	0	0	20
	Предсказанная метка	Колин Пауэлл	Дональд Рамсфелд	Джордж Буш мл.	Герхард Шредер	Тони Блэр

Рис. 11.14. Матрица ошибок модели, распознающей лица, с использованием библиотеки Keras-vggface

ArcFace

VGGFace не единственная предварительно обученная CNN, которая отлично справляется с извлечением признаков из изображений лиц. Еще одна технология — ArcFace, которая была представлена в 2019 году в работе под названием "ArcFace: аддитивная потеря разницы распознавания в глубоком распознавании лиц" ("ArcFace: Additive Angular Margin Loss for Deep Face Recognition", <https://oreil.ly/fWe7Y>). Удобная реализация доступна в пакете Python под названием Arcface (<https://oreil.ly/KySCA>).

Каждое изображение лица, переданное в ArcFace, преобразуется в плотный вектор из 512 значений, известный как *эмбединг* (*встраивание*) лица (face embedding). Код для создания эмбединга прост:

```
from arcface import ArcFace
```

```
af = ArcFace.ArcFace()
embedding = af.calc_emb(image)
```

Эмбединги можно использовать для обучения моделей машинного обучения, а также для прогнозирования с помощью этих моделей. Благодаря функции потерь, упомянутой в заголовке статьи, созданные ArcFace эмбединги часто лучше передают уникальность лица, чем эмбединги, созданные обычными CNN.

Еще одно применение эмбедингов, созданных ArcFace, — *верификация лица* (face verification), при которой сравниваются два изображения лица и вычисляется вероятность того, что на них изображен один и тот же человек. Следующие операторы генерируют эмбединги для двух изображений лиц и используют представленную

в главе 4 функцию `cosine_similarity` (<https://oreil.ly/iGDKx>) для количественной оценки сходства между ними:

```
af = ArcFace.ArcFace()
face_emb1 = af.calc_emb(image1)
face_emb2 = af.calc_emb(image2)
sim = cosine_similarity([face_emb1, face_emb2])[0][1]
```

Результатом является значение от 0,0 до 1,0, при этом более высокие значения отражают большее сходство между лицами.

Собираем все вместе: обнаружение и распознавание лиц на фотографиях

Если в ходе тестирования модель получает идеальные результаты, следует относиться к этому скептически. Ни одна модель не совершенна, и даже если она достигла 100-процентной точности на тестовом наборе данных, она не сможет повторить это в реальных условиях. Учитывая, что VGGFace обучалась на изображениях тех же знаменитых людей, что и в наборе данных LFW, возможно ли, что она предвзята по отношению к этим людям? А трансферное обучение с помощью VGGFace будет ли таким успешным, если его обучить на изображениях обычных людей? И как оно будет работать при наличии всего нескольких обучающих изображений?

Для того чтобы ответить на эти вопросы, давайте создадим блокнот, обучающий модель распознавания лиц на основе VGGFace и использующий MTCNN для обнаружения лиц на фотографиях, а также применяющий эту модель для идентификации обнаруженных лиц. Используемый набор данных содержит восемь фотографий, на каждой из которых изображены три обычных человека в возрасте до 20 лет в очках и без них в немного разных позах (рис. 11.15). Эти изображения были извлечены из фотографий с помощью функции `extract_faces` из примера 11.1 и приведены к размеру 224×224 .



Рис. 11.15. Фотографии для обучения модели распознавания лиц

Начните с загрузки ZIP-файла (<https://oreil.ly/AA6Gd>) с изображениями лиц и скопируйте его содержимое в подкаталог Faces, где размещены ваши блокноты. ZIP-файл содержит четыре папки: одну с названием Jeff, одну с названием Lori, одну с названием Abby и одну с названием Samples, содержащую необрезанные фотографии для тестирования.

Теперь создайте новый блокнот и выполните следующий код в первой ячейке, чтобы определить вспомогательные функции для загрузки и отображения изображений лиц из подкаталогов, в которые вы их скопировали, и объявить несколько списков Python для хранения изображений и меток:

```
import os
from tensorflow.keras.preprocessing import image
import matplotlib.pyplot as plt
%matplotlib inline

def load_images_from_path(path, label):
    images, labels = [], []

    for file in os.listdir(path):
        images.append(image.img_to_array(image.load_img(os.path.join(path, file),
                                                                    target_size=(224, 224, 3))))
        labels.append((label))

    return images, labels

def show_images(images):
    fig, axes = plt.subplots(1, 8, figsize=(20, 20),
                             subplot_kw={'xticks': [], 'yticks': []})

    for i, ax in enumerate(axes.flat):
        ax.imshow(images[i] / 255)

x, y = [], []
Затем загрузите изображения Джефа и пометьте их нулями:
images, labels = load_images_from_path('Faces/Jeff', 0)
show_images(images)

x += images
y += labels
Загрузите изображения Лори и обозначьте их единицами:
images, labels = load_images_from_path('Faces/Lori', 1)
show_images(images)

x += images
y += labels
```

Загрузите изображения Эбби и обозначьте их двойками:

```
images, labels = load_images_from_path('Faces/Abby', 2)
show_images(images)
```

```
x += images
y += labels
```

Наконец, предварительно обработайте пиксели для ResNet50-версии VGGFace и разбейте данные пополам, чтобы сеть обучалась на четырех случайно выбранных изображениях каждого человека и проверялась на таком же количестве изображений:

```
import numpy as np
from sklearn.model_selection import train_test_split
from tensorflow.keras.applications.resnet50 import preprocess_input
```

```
faces = preprocess_input(np.array(x))
labels = np.array(y)
```

```
x_train, x_test, y_train, y_test = train_test_split(
    faces, labels, train_size=0.5, stratify=labels, random_state=0)
```

Следующим шагом будет загрузка сохраненной модели VGGFace и заморозка "бутылочных" слоев. Если вы не загрузили файл vggface.h5 ранее, скачайте его сейчас (<https://oreil.ly/noZoo>) и поместите в подкаталог Data вашего блокнота. Затем выполните следующий код:

```
from tensorflow.keras.models import load_model
```

```
base_model = load_model('Data/vggface.h5')
base_model.trainable = False
```

Теперь определите сеть, использующую трансферное обучение с VGGFace для идентификации лиц. Слой изменения размера Resizing гарантирует, что каждое изображение имеет размер 224×224 пиксела. Слой Dense содержит всего восемь нейронов, поскольку набор обучающих данных невелик и мы не хотим, чтобы сеть слишком плотно подстраивалась под него:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Flatten, Dense, Resizing
```

```
model = Sequential()
model.add(Resizing(224, 224))
model.add(base_model)
model.add(Flatten())
model.add(Dense(8, activation='relu'))
model.add(Dense(3, activation='softmax'))
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```


Обучите модель:

```
hist = model.fit(x_train, y_train, validation_data=(x_test, y_test),
                batch_size=2, epochs=10)
```

Постройте графики точности обучения и валидации:

```
import seaborn as sns
sns.set()
```

```
acc = hist.history['accuracy']
val_acc = hist.history['val_accuracy']
epochs = range(1, len(acc) + 1)
```

```
plt.plot(epochs, acc, '-', label='Точность обучения')
plt.plot(epochs, val_acc, ':', label='Точность валидации')
plt.title('Точность обучения и валидации')
plt.xlabel('Эпоха')
plt.ylabel('Точность')
plt.legend(loc='lower right')
plt.plot()
```

Надеюсь, вы получили что-то подобное рис. 11.16.

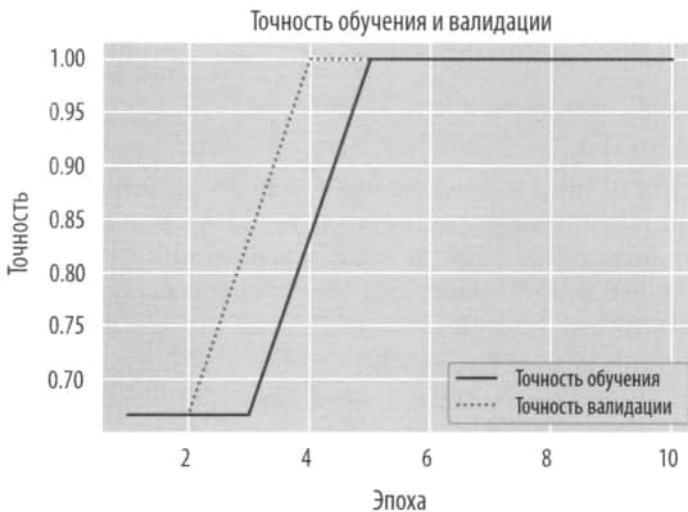


Рис. 11.16. Графики точности обучения и валидации модели, использующей трансферное обучение с VGGFace

А теперь начинается самое интересное: использование MTCNN для обнаружения лиц на фотографии и обученной модели для идентификации этих лиц. Сначала убедитесь, что пакет MTCNN установлен в вашей среде. Затем определите пару вспомогательных функций — одну, извлекающую лицо из указанного места на изобра-

жении (`get_face`), и другую, загружающую фотографию и аннотирующую лица на ней именами и уровнями уверенности (`label_faces`):

```
from mtcnn.mtcnn import MTCNN
from PIL import Image, ImageOps
from tensorflow.keras.preprocessing import image
from matplotlib.patches import Rectangle

def get_face(image, face):
    x1, y1, w, h = face['box']
    if w > h:
        x1 = x1 + ((w - h) // 2)
        w = h
    elif h > w:
        y1 = y1 + ((h - w) // 2)
        h = w

    x2 = x1 + h
    y2 = y1 + w

    return image[y1:y2, x1:x2]

def label_faces(path, model, names, face_threshold=0.9,
                prediction_threshold=0.9,
                show_outline=True, size=(12, 8)):
    # Загрузка изображения и его правильная ориентация
    pil_image = Image.open(path)
    exif = pil_image.getexif()

    for k in exif.keys():
        if k != 0x0112:
            exif[k] = None
            del exif[k]

    pil_image.info["exif"] = exif.tobytes()
    pil_image = ImageOps.exif_transpose(pil_image)
    np_image = np.array(pil_image)

    fig, ax = plt.subplots(figsize=size, subplot_kw={'xticks': [], 'yticks': []})
    ax.imshow(np_image)

    detector = MTCNN()
    faces = detector.detect_faces(np_image)
    faces = [face for face in faces if face['confidence'] > face_threshold]
```

```
for face in faces:
```

```
    x, y, w, h = face['box']
```

```
    # Использование модели для идентификации лица
```

```
    face_image = get_face(np_image, face)
```

```
    face_image = image.array_to_img(face_image)
```

```
    face_image = preprocess_input(np.array(face_image))
```

```
    predictions = model.predict(np.expand_dims(face_image, axis=0))
```

```
    confidence = np.max(predictions)
```

```
if (confidence > prediction_threshold):
```

```
    # Опциональное отображение рамки вокруг лица
```

```
    if show_outline:
```

```
        rect = Rectangle((x, y), w, h, color='red', fill=False, lw=2)
```

```
        ax.add_patch(rect)
```

```
    # Пометка лица
```

```
    index = int(np.argmax(predictions))
```

```
    text = f'{names[index]} ({confidence:.1%})'
```

```
    ax.text(x + (w / 2), y, text, color='white',
```

```
           backgroundcolor='red', ha='center', va='bottom',
```

```
           fontweight='bold', bbox=dict(color='red'))
```

Теперь передайте в `label_faces` первый образец изображения из папки `Samples`:

```
labels = ['Jeff', 'Lori', 'Abby']
```

```
label_faces('Faces/Samples/Sample-1.jpg', model, labels)
```

Результат должен выглядеть примерно так, как на рис. 11.17, хотя процентное соотношение может быть и другим.



Рис. 11.17. Распознавание лиц Лори и Джефа

Попробуйте еще раз, но на этот раз с другой фотографией:

```
label_faces('Faces/Samples/Sample-2.jpg', model, labels)
```

На рис. 11.18 приведен результат.

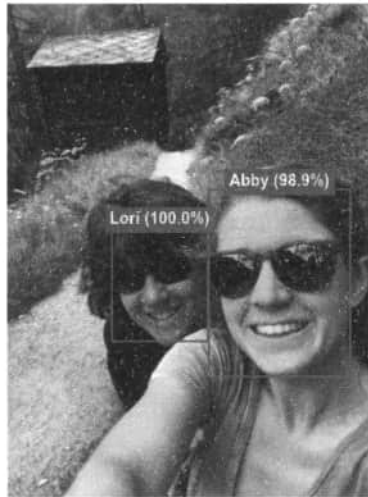


Рис. 11.18. Распознавание лиц Лори и Эбби

И наконец, отправьте фотографию, на которой изображены все три человека, по которым обучалась модель:

```
label_faces('Faces/Samples/Sample-3.jpg', model, labels)
```

Обученная всего на 12 изображениях лиц — по четыре на человека — модель отлично справляется с идентификацией лиц на фотографиях. Конечно, вы можете создать собственный набор данных, пропустив фотографии друзей и членов семьи через функцию в примере 11.1 и обучив модель на полученных изображениях.

Работа с неизвестными лицами: классификация по закрытым и открытым наборам

Теперь плохие новости. Модель распознавания лиц VGGFace отлично справляется с идентификацией лиц, на которые она была обучена, но она не знает, что делать, когда встречает лицо, на которое она *не была* обучена. Попробуйте: передайте свою фотографию. Модель, вероятно, опознает вас как Джефа, Лори или Эбби, причем сделает это с большой долей уверенности. Она буквально не знает того, чего не знает. Это особенно верно, когда набор данных мал, и сеть может переобучаться.

Причина этого не имеет ничего общего с VGGFace. Она связана с тем, что нейронная сеть с выходным слоем softmax — это *классификатор с закрытым набором*, т. е. она классифицирует любой образец, представленный ей для прогнозирования, как один из классов, на которые она была обучена. (Помните: softmax гарантирует,

что сумма вероятностей для всех классов равна 1,0.) Альтернативой является *классификатор с открытым набором* (рис. 11.19), который может сказать: "Этот образец не принадлежит ни к одному из классов, которым я был обучен".

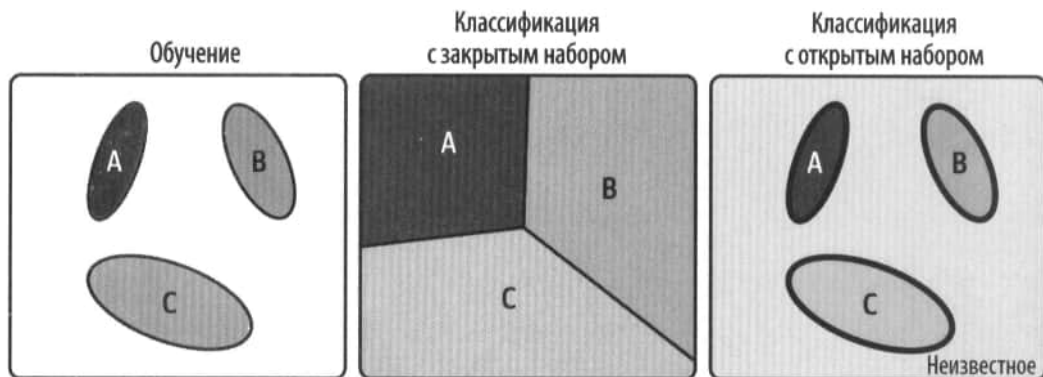


Рис. 11.19. Классификация с закрытым и открытым набором

На сегодняшний день в сообществе глубокого обучения нет универсального решения для построения классификаторов с открытым набором. В 2016 году в статье "В направлении глубоких сетей с открытым набором" ("Towards Open Set Deep Networks", <https://oreil.ly/IFmIF>) было предложено одно из решений в виде выходных слоев *openmax*, которые заменяют выходные слои softmax и "оценивают вероятность того, что входные данные принадлежат неизвестному классу". По сути, если сеть обучена на 10 классах, выходной слой *openmax* добавляет 11-й выход, представляющий неизвестный класс. Он работает, беря активации из последнего классификационного слоя и корректируя их с помощью распределения Вейбулла (<https://oreil.ly/plWN2>), а не нормализуя вероятности, как это делает softmax.

Другое потенциальное решение было предложено в статье 2018 года под названием "Уменьшение сетевой агностофобии" ("Reducing Network Agnostophobia", <https://oreil.ly/hL9jk>) исследователей из Университета Колорадо. В ней предлагается заменить кросс-энтропийные потери новой функцией потерь под названием "*энтропийные потери открытых наборов*" (entropic open-set loss), приводящей оценки softmax для неизвестных классов к равномерному распределению вероятности. Используя эту технику, можно более надежно определять принадлежность образцов к неизвестному классу с помощью пороговых значений вероятности в связке с обычными выходными слоями softmax. Отличный итог проблем классификации по открытым наборам в глубоком обучении и обзор *openmax* и энтропийных потерь открытых наборов можно найти в статье "Знает ли нейронная сеть, когда не знает?" Тивадара Данки (Danka Tivadar. Does a Neural Network Know When It Doesn't Know?; <https://oreil.ly/dHb7m>).

Еще одно решение — использовать ArcFace для проверки каждого идентифицированного моделью лица путем сравнения эмбединга, созданного на основе этого лица, с эталонным эмбедингом для того же лица. Вы можете отклонить вывод модели, если косинусное сходство окажется ниже заранее определенного порога.

Более наивный подход заключается в том, чтобы не дать сети слишком хорошо изучить обучающие данные в надежде, что неизвестные классы дадут более низкие вероятности softmax. Именно поэтому в предыдущем примере я включил в классификационный слой всего восемь нейронов. (Вы можете пойти еще дальше, добавив слой отсева.) Это работает до определенного момента, но не идеально. Функция `label_faces` имеет порог прогнозирования по умолчанию 0,9, т. е. она помечает лицо только в том случае, если модель классифицирует его с уверенностью не менее 90%. Вы можете установить `prediction_threshold` на 0,99, чтобы исключить больше неизвестных лиц, но ценой отказа от идентификации большего количества известных лиц. Такая настройка, позволяющая найти правильный баланс между распознаванием известных лиц и игнорированием неизвестных, является неизбежной частью подготовки модели распознавания лиц.

Резюме

Создание комплексной системы распознавания лиц требует наличия средств для обнаружения лиц на фотографиях, а также средств для классификации (идентификации) этих лиц. Одним из способов обнаружения лиц является алгоритм Виолы — Джонса, удобная реализация которого предоставляется библиотекой OpenCV. Основанной на глубоком обучении альтернативой является многозадачная каскадная сверточная нейронная сеть, или MTCNN. Пакет Python под названием MTCNN содержит готовую к использованию реализацию MTCNN. Алгоритм Виолы — Джонса быстрее и лучше подходит для приложений реального времени (например, для идентификации лиц в прямой трансляции с веб-камеры), но MTCNN в целом более точны и имеют меньше ложных срабатываний.

Глубокое обучение может быть применено к задаче распознавания лиц с помощью сверточных нейронных сетей. Трансферное обучение посредством предварительно обученной CNN, такой как ResNet50, может идентифицировать лица с относительно высокой степенью точности, но трансферное обучение с помощью CNN, обученной на миллионах изображений лиц, обеспечивает непревзойденную точность. Основная причина заключается в том, что "бутылочные" слои CNN оптимизированы для извлечения признаков из изображений лиц.

С высоко оптимизированным набором весов нейронная сеть может делать практически все. Если нет ничего лучшего, достаточно общих весов, но если для начала подобрать набор весов для конкретной задачи, распознавание лиц с помощью трансферного обучения может достичь точности, подобной человеческой.

Обнаружение объектов

В предыдущей главе были представлены два популярных алгоритма обнаружения лиц на фотографиях: основанный на машинном обучении алгоритм Виолы — Джонса и основанный на глубоком обучении MTCNN. Обнаружение лиц — это частный случай обнаружения объектов (<https://oreil.ly/4H8yw>), когда компьютеры обнаруживают и идентифицируют объекты на изображениях. Идентификация объекта — это проблема классификации изображений, в которой CNN преуспели. Но сам поиск объектов для идентификации представляет собой другую задачу.

Обнаружение объектов является сложной задачей, поскольку не предполагается, что объекты идеально обрезаны и выровнены, как это происходит в задачах классификации изображений. Также на изображении может быть не один объект. На рис. 12.1 показано, что может увидеть автономно движущийся автомобиль при сканировании видеокадров с направленной вперед камеры. CNN, обученная классификации обычных изображений с помощью тщательно подготовленных обучающих изображений, бессильна помочь. Она может классифицировать изображение как городскую улицу, но не сможет определить, что на нем есть машины, люди и светофоры, и уж тем более не сможет точно определить их местоположение.

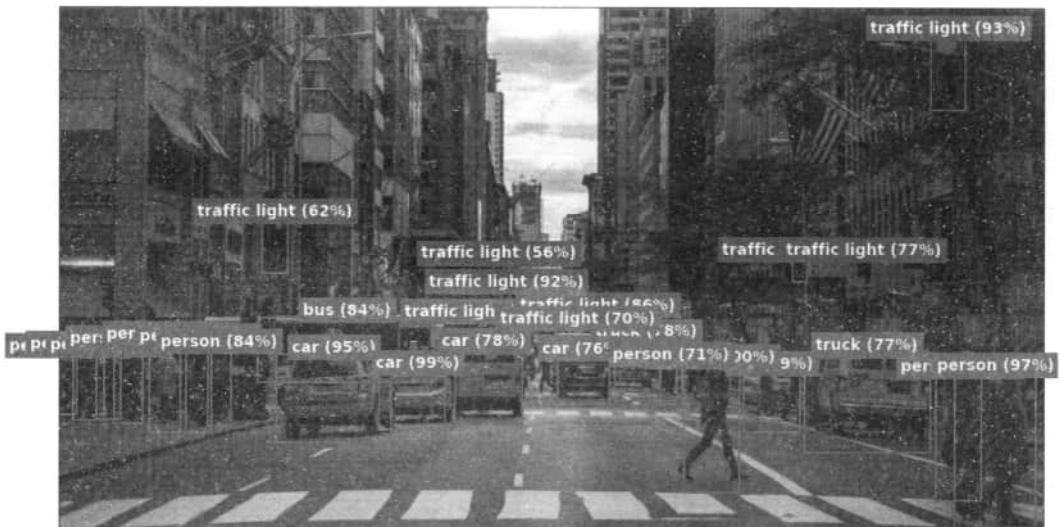


Рис. 12.1. Обнаружение и идентификация объектов в кадре

В последние годы скорость и точность обнаружения объектов выросли, и современные методы основаны на глубоком обучении. В частности, они используют CNN, которые были представлены в *главе 10*. Давайте обсудим, как CNN выполняют обнаружение и идентификацию объектов, и попробуем свои силы, используя передовые модели обнаружения объектов.

R-CNN

Одним из способов применения глубокого обучения для задачи обнаружения объектов является использование *региональных CNN*, также известных как *R-CNN* (region-based/region CNN, <https://oreil.ly/ozIoy>). Первая R-CNN была представлена в 2014 году в работе под названием "Богатые иерархии признаков для точного обнаружения объектов и семантической сегментации" ("Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation"). Описанная в статье модель состоит из трех этапов. На первом этапе сканируется изображение и определяется до 2000 ограничительных рамок, представляющих области интереса — регионы, в которых могут находиться объекты. Второй этап — работа глубокой CNN, извлекающей признаки из областей интереса. Третий — применение метода опорных векторов, классифицирующего признаки. На выходе получается набор ограничительных рамок с метками классов и оценками доверия. Алгоритм под названием "*подавление немаксимумов*" (non-maximum suppression, NMS; <https://oreil.ly/ku8SU>) фильтрует выходные данные и выбирает наилучшее ограничительное поле для каждого объекта.

NMS является важнейшим элементом практически всех современных систем обнаружения объектов. Детектор неизменно выдает несколько ограничительных рамок для каждого объекта. Если фотография содержит один экземпляр заданного класса, например одну зебру, NMS выбирает ограничительную рамку с наивысшей оценкой достоверности. Если фотография содержит две зебры (рис. 12.2), NMS делит ограничительные рамки на две группы и выбирает в каждой группе рамку с наивысшей оценкой достоверности. Алгоритм группирует рамки в зависимости от степени перекрытия между ними. Перекрытие вычисляется путем деления площади пересечения двух рамок на площадь, образованную объединением рамок. Если результирующий *показатель пересечения и объединения* (intersection-over-union, IoU) превышает заранее установленный порог (обычно 0,5), NMS относит рамки к одной группе. В противном случае она относит их к разным группам.

Когда два объекта одного класса практически не пересекаются, NMS легко разделяет их. Если два экземпляра одного класса значительно перекрываются (например, одна зебра стоит позади другой), для разделения может потребоваться корректировка порога IoU. Порог IoU — это гиперпараметр, который можно настроить для нахождения правильного баланса между слишком агрессивным разделением перекрывающихся объектов и недостаточно агрессивным.

На первом этапе большинства реализаций R-CNN используется алгоритм *селективного поиска* (selective search, <https://oreil.ly/3S1K0>) для определения областей инте-

реса путем ориентировки на сходство цвета, текстуры, формы и размера. На рис. 12.3 показаны первые 500 ограничительных рамок, созданных при изучении изображения с помощью реализации селективного поиска в OpenCV (<https://oreil.ly/7XDD8>). Передача целевого списка регионов в CNN происходит быстрее, чем грубый подход со скользящим окном, при котором содержимое окна вводится в CNN при каждой остановке.

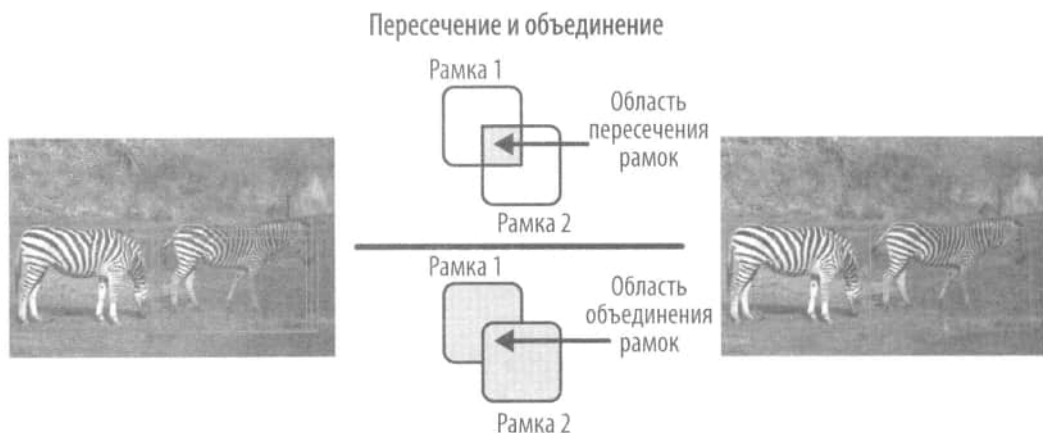


Рис. 12.2. Подавление немаксимумов

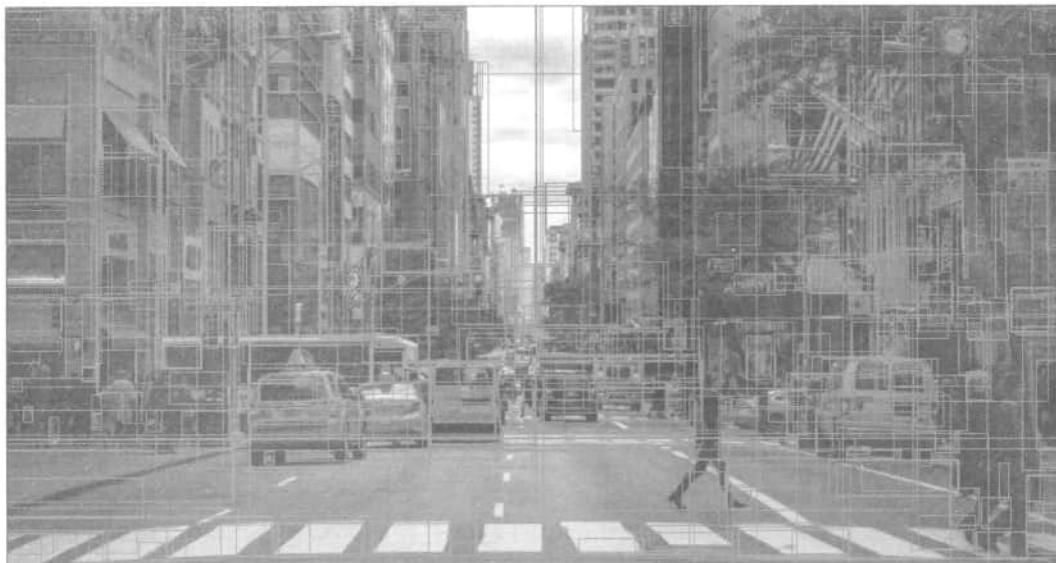


Рис. 12.3. Ограничительные рамки, созданные с помощью селективного поиска

Даже при селективном поиске, сужающем список регионов-кандидатов на этапе 2, R-CNN не может выполнять обнаружение объектов в реальном времени. Почему? Потому что CNN по отдельности обрабатывает 2000 или около того областей инте-

реса, определенных на этапе 1. Эти регионы неизменно перекрываются, поэтому CNN обрабатывает одни и те же пиксели несколько раз.

В работе 2015 года под названием "Fast R-CNN" (<https://oreil.ly/ih0x2>) эта проблема была решена путем предложения использовать модифицированную архитектуру, в которой всё изображение проходит через CNN один раз (рис. 12.4). Селективный поиск или аналогичный алгоритм определяет области интереса на изображении, и эти области проецируются на созданную CNN карту признаков. Затем слой объединения областей (регионов) интереса (region of interest, ROI) использует форму максимального пулинга для сокращения признаков в каждой области интереса до вектора фиксированной длины, не зависящего от размера и формы области (в отличие от этого, R-CNN масштабирует каждый регион в изображение заранее определенного размера перед передачей его в CNN, что значительно дороже, чем пулинг ROI). Классификация векторов признаков выполняется полносвязными слоями, а не SVM, и на выходе получают классификатор softmax и регрессор граничных ячеек. NMS фильтрует ограничительные рамки до тех, которые имеют значение. В результате получается система, которая обучается на порядок быстрее, чем R-CNN, делает прогнозы на два порядка быстрее и немного точнее, чем R-CNN.

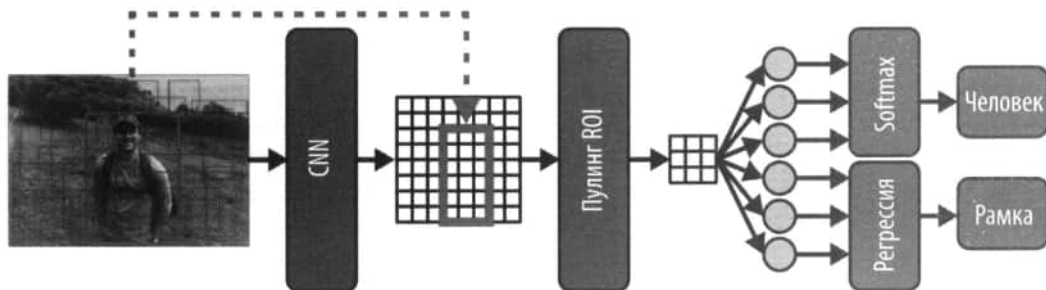


Рис. 12.4. Архитектура Fast R-CNN



Пулинг ROI сводит любую область интереса к вектору признаков определенного размера, независимо от высоты и ширины области. Представьте, что у вас есть область 8×16 на карте признаков, и вы хотите уменьшить ее до 4×4 . Вы можете разделить область 8×16 на сетку 4×4 , каждая ячейка которой имеет размер 2×4 . Затем можете взять максимальное из восьми значений в каждой ячейке и вставить их в сетку 4×4 . Это и есть пулинг ROI. Просто, быстро и эффективно. Он работает с областями любого размера и соотношения сторон.

В 2016 году была опубликована статья под названием "Faster R-CNN: обнаружение объектов в реальном времени с помощью сетей предложений областей" ("Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks"; <https://oreil.ly/wWMOy>), позволившая еще больше увеличить производительность путем замены селективного поиска на *сеть предложений областей* (region proposal network, RPN). RPN — это неглубокая CNN, имеющая общие слои с основной CNN (рис. 12.5). Для того чтобы генерировать предложения областей, она скользит ок-

ном по карте признаков, созданной последним общим слоем. При каждой остановке окна RPN оценивает n регионов-кандидатов, называемых *якорями* (anchors) или *якорными рамками* (anchor boxes), и вычисляет *оценку объектности* (objectness score) для каждого из них на основе оценок IoU с рамками-источниками. Оценки объектности и якорные рамки поступают на вход полносвязных слоев для классификации (содержит ли якорь объект, и если да, то какого типа?) и регрессии. Выходные данные этих слоев в конечном итоге определяют области интереса, проецируемые на созданную основной CNN карту признаков, и передаются на слой пулинга ROI.

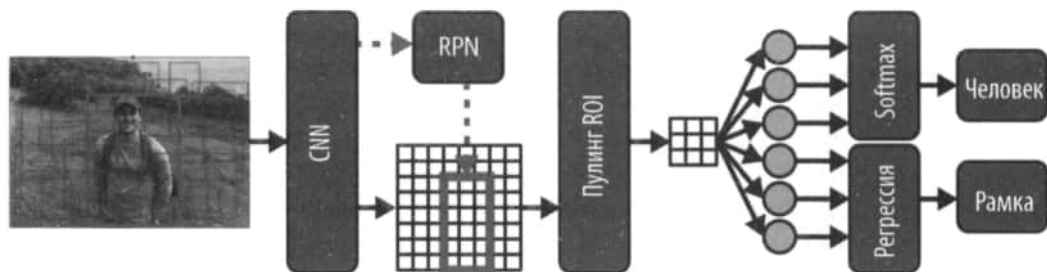


Рис. 12.5. Архитектура Faster R-CNN

Faster R-CNN может работать в 10 раз быстрее, чем Fast R-CNN, и выполнять обнаружение объектов практически в реальном времени. Как правило, они также более точны, чем Fast R-CNN, благодаря превосходной способности RPN определять регионы-кандидаты. Селективный поиск статичен, но RPN становится умнее по мере обучения сети. RPN также выполняется параллельно с основной CNN, а это означает, что предложения областей мало влияют на производительность.

Mask R-CNN

Следующая глава в истории R-CNN была написана в 2017 году в работе под названием "Mask R-CNN" (<https://oreil.ly/33ktB>). Mask R-CNN расширяют Faster R-CNN, добавляя *сегментацию экземпляров* (instance segmentation), идентифицирующую обнаруженные на изображении формы объектов с помощью *масок сегментации* (segmentation masks), подобных тем, что показаны на рис. 12.6. Влияние на производительность минимально, поскольку сегментация экземпляров выполняется параллельно с оценкой областей. Преимущество Mask R-CNN заключается в том, что они предоставляют более подробную информацию об объектах, которые они обнаруживают. Например, можно определить, вытянуты ли руки человека, чего нельзя сделать с помощью простой ограничительной рамки. Они также немного более точны, чем Faster R-CNN, поскольку заменяют пулинг ROI *выравниванием ROI* (ROI alignment), что позволяет отбрасывать меньше информации при генерации векторов признаков, границы которых не полностью совпадают с границами областей, которые они представляют.



Рис. 12.6. Ограничительные рамки и маски сегментации, созданные Mask R-CNN

Сервис Zoom использует сегментацию экземпляров для отображения разнообразных фонов позади вас во время видеотрансляций. Сегментация экземпляров также используется в графических редакторах на основе искусственного интеллекта, которые вырезают фигуры с переднего плана одних изображений и помещают их на другие. Давайте продемонстрируем это на примере предварительно обученной сети Mask R-CNN.

Начните с загрузки ZIP-файла (<https://oreil.ly/NKGlg>), содержащего необходимые для этого упражнения файлы.

adam.jpg

Фотография молодого человека с рюкзаком.

maui.jpg

Фотография, сделанная у бассейна на Гавайях.

mask.py

Содержит вспомогательные функции для предварительной обработки изображений перед их вводом в Mask R-CNN и функции постобработки для интерпретации результатов.

MaskRCNN-12-int8.onnx

Содержит предварительно обученную Mask R-CNN, сохраненную в формате ONNX.

Поместите *mask.py* в каталог, где находятся ваши блокноты Jupyter, а остальные файлы — в подкаталог Data ваших блокнотов.

Важнейшим файлом в этом ансамбле является *MaskRCNN-12-int8.onnx*. В *главе 7* был представлен Open Neural Network Exchange (ONNX, <https://oreil.ly/UwqD3>) как

средство для загрузки написанных на Python моделей Scikit-Learn и их использования с другими языками. Но ONNX не просто устраняет языковой разрыв с Scikit. Это нейтральный формат, позволяющий экспортировать нейронные сети, написанные с помощью одного фреймворка глубокого обучения, в другие. Например, можно сохранить модель PyTorch в файл .onnx, а затем преобразовать файл .onnx в модель TensorFlow и использовать ее так, как будто она изначально была написана в TensorFlow. Или можно установить среду выполнения ONNX, загрузить и вызвать модель без преобразования ее в TensorFlow. В *главе 7* показывалось, как сохранить модель Scikit в формате ONNX и использовать среду выполнения Python ONNX (<https://oreil.ly/U79N2>) для загрузки модели и вызова ее из Python. В ней также применялась среда исполнения .NET ONNX (<https://oreil.ly/u5yYL>) для загрузки модели и ее вызова из C#. Как отмечалось в *главе 7*, среды выполнения ONNX доступны для различных платформ и языков программирования.

MaskRCNN-12-int8.onnx содержит сложную реализацию Mask R-CNN (<https://oreil.ly/bz31O>) от Facebook¹ Research. Она использует основу ResNet50 для извлечения признаков из поступающих на вход изображений и содержит ветвь, вычисляющую маски сегментации экземпляров параллельно с вычислением ограничительных рамок. Она была обучена на более чем 200 000 изображений из набора данных COCO (<https://oreil.ly/C22xv>). Эти изображения содержат более 1,5 млн объектов, относящихся к 80 категориям (<https://oreil.ly/D7Bzd>) — от кошек и собак до людей, велосипедов и автомобилей. А поскольку обученная модель была опубликована в "зоопарке" моделей ONNX Model Zoo на GitHub (<https://oreil.ly/ytRcn>), вы можете загрузить ее и передать ей изображения с помощью всего нескольких строк кода.

Первый шаг к использованию модели в Python — убедиться, что среда выполнения Python ONNX установлена в вашей системе. Для этого упражнения также убедитесь, что библиотека OpenCV установлена. Затем создайте блокнот Jupyter и запустите следующий код в первой ячейке, чтобы загрузить и отобразить adam.jpg:

```
import matplotlib.pyplot as plt
from PIL import Image
%matplotlib inline

image = Image.open('Data/adam.jpg')
fig, ax = plt.subplots(figsize=(12, 8), subplot_kw={'xticks': [], 'yticks': []})
ax.imshow(image)
```

На рис. 12.7 приведен результат.

Задача состоит в том, чтобы пропустить это изображение через Mask R-CNN, обнаружить на нем объекты и соответствующим образом аннотировать изображение. Используйте следующий код для предварительной обработки пикселей изображе-

¹ Facebook запрещена на территории России. — Прим. ред.

ния в соответствии с ожиданиями модели, загрузите Mask R-CNN из файла ONNX и передайте ей предварительно обработанное изображение:

```
from mask import *
import onnxruntime as rt

image_data = preprocess(image)
session = rt.InferenceSession('Data/MaskRCNN-12-int8.onnx')
input_name = session.get_inputs()[0].name
result = session.run(None, { input_name: image_data })
```



Рис. 12.7. Исходное изображение

Возвращаемое методом `gui` значение представляет собой массив из четырех массивов. Первый массив содержит ограничительные рамки для обнаруженных на изображении объектов. В данном примере было обнаружено всего 13 объектов:

```
array([[ 377.50537, 174.36874, 801.177 , 787.7125 ],
       [ 428.15994, 344.70105, 699.99097, 599.8499 ],
       [ 757.1859 , 529.4938 , 814.2776 , 658.8041 ],
       [ 432.57123, 351.229 , 672.2242 , 484.63702],
       [ 435.53653, 357.34235, 494.73557, 516.4939 ],
       [ 672.56714, 516.2991 , 822.82153, 663.68726],
       [ 608.1904 , 361.0429 , 686.1207 , 552.70105],
       [ 629.84894, 355.3375 , 799.4963 , 718.9458 ],
       [ 437.99078, 331.19318, 709.4969 , 748.2785 ],
       [ 438.44028, 348.14404, 579.8056 , 546.0588 ],
       [1152.0518 , 203.89978, 1163.7583 , 223.2485 ],
       [1151.8087 , 132.45256, 1164.0013 , 150.6876 ],
```

```
[1151.8087 , 404.0875 , 1164.0013 , 423.058 ],
[ 416.03802, 396.8123 , 694.8975 , 511.93896],
[ 683.35236, 765.0078 , 701.3403 , 784.47546]], dtype=float32)
```

Второй массив — это массив целочисленных идентификаторов классов, соответствующих классам в наборе данных СОСО: 1 — человек, 2 — велосипед и т. д. Они идентифицируют объекты на изображении:

```
array([ 1, 25, 27, 25, 25, 27, 25, 27, 27, 25, 1, 1, 1, 25, 75], dtype=int64)
```

Третий массив содержит оценки достоверности для 13 объектов. Они варьируются от высокого уровня в 99,9% для человека на изображении до низкого в 5,5% для логотипа на его кепке, который, по мнению модели, может быть часами. Обратите внимание, что только два объекта получили уровень уверенности 70% или выше:

```
array([0.99911565, 0.8009716 , 0.4624603 , 0.19400069, 0.17042953,
       0.11972303, 0.11626374, 0.10352837, 0.08243025, 0.08089489,
       0.07266886, 0.07266886, 0.07266886, 0.06278796, 0.0549276 ],
      dtype=float32)
```

Наконец, четвертый массив содержит маски сегментации для каждого объекта. Каждая маска представляет собой массив 28×28 значений с плавающей точкой, иногда называемый *мягкой маской* из-за мягких краев. На рис. 12.8 показана маска для человека, обнаруженного на изображении. При обратном сопоставлении с исходным размером и соотношением сторон изображения она позволяет определить, какие пикселы соответствуют этому человеку.

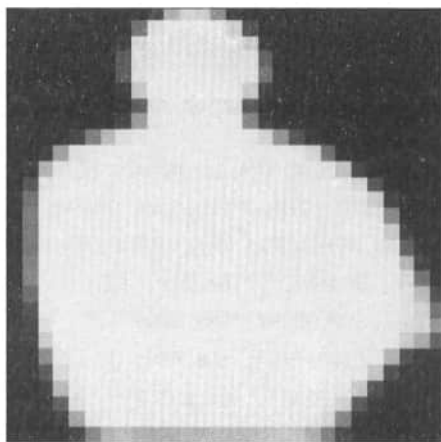


Рис. 12.8. Мягкая маска размером 28×28 , соответствующая человеку на предыдущей фотографии

Следующим шагом будет сбор ограничительных рамок, прогнозируемых меток классов, доверительных оценок и масок сегментации и передача их в функцию `annotate_image` для визуализации результатов:

```
boxes = result[0]   # Ограничительные рамки
labels = result[1]  # Метки классов
```

```
scores = result[2] # Доверительные оценки
masks = result[3] # Маски сегментации

annotate_image(image, boxes, labels, scores, masks)
```

Результат представлен на рис. 12.9.



Рис. 12.9. Визуализация ограничительных рамок, прогнозируемых меток классов, доверительных оценок и масок сегментации

По умолчанию `annotate_image` игнорирует объекты, идентифицированные с уверенностью менее 70%. Это можно отменить, включив в вызов параметр `min_confidence`. Модель обнаружила на изображении два объекта с уверенностью, равной или превышающей 70%: человека (`person`) и рюкзак (`backpack`). `annotate_image` рисует ограничительные рамки и аннотирует их именами классов и уровнями уверенности. Она также затеняет объекты, накладывая на них частично прозрачные пиксели. Это возвращаемые программой `gui` маски сегментации.

`annotate_image` — одна из нескольких вспомогательных функций, содержащихся в `mask.py`. Другая функция — `preprocess`, которая подготавливает каждое изображение так, как ожидает Mask R-CNN: изменяет размер изображения, проверяет, чтобы ширина и высота были кратны 32, конвертирует изображение в формат BGR и нормализует значения пикселей. Я перенес обе эти функции с GitHub-страницы модели (<https://oreil.ly/06Nmo>) с небольшими изменениями. Поскольку Mask R-CNN часто используются для обрезки объектов с изображений, я также добавил функцию `change_background`, которая извлекает все объекты, обнаруженные на изображении с заданным уровнем уверенности (по умолчанию он равен 0,7), и компонует их на новом фоне. Для того чтобы убедиться в этом, выполните приведенные ниже операторы в следующей ячейке блокнота:

```
fg_image = Image.open('Data/adam.jpg')
bg_image = Image.open('Data/maui.jpg')

change_background(session, fg_image, bg_image)
```


Результат должен выглядеть так, как показано на рис. 12.10.



Рис. 12.10. Компоновка изображения человека и нового фона

`change_background` отправляет изображение переднего плана (переданное во втором параметре функции) в модель и копирует все находящиеся в масках сегментации пиксели в фоновое изображение. Лучше всего эта функция работает, когда изображения переднего и заднего планов имеют одинаковый размер, но при необходимости она изменит размер фонового изображения, чтобы оно соответствовало изображению переднего плана. Фоновое изображение будет искажено, если его соотношение сторон отличается от соотношения сторон изображения переднего плана.

То, что размер создаваемых Mask R-CNN масок сегментации составляет всего 28×28 , объясняет "разрывы" вокруг пикселей, скопированных с изображения переднего плана: разрешение маски меньше, чем у человека на фотографии. Можно модифицировать Mask R-CNN для использования масок с более высоким разрешением, но для этого придется заново обучать модель. Это займет много времени для такой сложной модели, как эта, и с таким большим набором данных, как COCO, даже если вы будете проводить обучение на GPU.

YOLO

Хотя семейство систем обнаружения объектов R-CNN обеспечивает непревзойденную точность, оно оставляет желать лучшего, когда речь идет об обнаружении объектов в реальном времени, как это требуется, скажем, для автономных автомобилей. В 2015 году была опубликована работа "Ты смотришь только один раз: унифицированное обнаружение объектов в реальном времени" ("You Only Look Once: Unified, Real-Time Object Detection"; <https://oreil.ly/qtiJj>). В ней была предложена

альтернатива R-CNN, известная как *YOLO* (You Only Look Once), которая произвела революцию в представлениях инженеров об обнаружении объектов. Из введения к статье:

"Человек смотрит на изображение и мгновенно понимает, какие объекты на нем изображены, где они находятся и как взаимодействуют. Зрительная система человека работает быстро и точно, что позволяет нам выполнять такие сложные задачи, как вождение автомобиля, практически не задумываясь об этом. Быстрые и точные алгоритмы обнаружения объектов позволили бы компьютерам управлять автомобилями без специализированных датчиков, вспомогательным устройствам — передавать пользователям информацию о происходящем в реальном времени, а также раскрыли бы потенциал для создания реагирующих на изменения роботизированных систем общего назначения.

Существующие системы обнаружения перепрофилируют классификаторы для выполнения обнаружения. Для того чтобы обнаружить объект, эти системы берут классификатор для этого объекта и оценивают его в различных локациях и масштабах на тестовом изображении. Такие системы, как модели деформируемых частей (*deformable parts models, DPM*), используют подход скользящего окна, при котором классификатор запускается в равномерно распределенных местах по всему изображению.

Более современные подходы, такие как R-CNN, используют методы предложения областей, чтобы сначала сгенерировать потенциальные ограничительные рамки на изображении, а затем запустить классификатор на этих предложенных рамках. После классификации используется постобработка для уточнения границ, устранения дубликатов и повторной оценки границ на основе других объектов в сцене. Эти сложные конвейеры работают медленно и трудно поддаются оптимизации, поскольку каждый компонент должен обучаться отдельно.

Мы же рассматриваем обнаружение объектов как единую задачу регрессии — от пикселей изображения к координатам ограничительной рамки и вероятностям классов. Используя нашу систему, вы смотрите на изображение только один раз (*YOLO*), чтобы предсказать, какие объекты присутствуют и где они находятся".

Системы *YOLO* отличаются своей производительностью:

"Наша базовая сеть работает со скоростью 45 кадров в секунду без пакетной обработки на GPU Titan X, а быстрая версия — со скоростью более 150 кадров в секунду. Это означает, что можно обрабатывать потоковое видео в режиме реального времени с задержкой менее 25 миллисекунд. Более того, *YOLO* достигает более чем в два раза большей средней точности, чем другие системы реального времени".

На высоком уровне *YOLO* работает, разделяя карты признаков на сетки ячеек и оценивая каждую ячейку на наличие объекта, как показано на рис. 12.11. Используя заимствованную из *Faster R-CNN* концепцию якорных рамок, *YOLO* анализирует ограничительные рамки различных форм и размеров вокруг каждой ячейки и присваивает рамкам классы и вероятности. Одна CNN обрабатывает все, включая извлечение признаков, классификацию и регрессию, определяя размеры и располо-

жение ограничительных рамок, и изображение проходит через CNN всего один раз. В конце NMS сокращает количество ограничительных рамок до одной на объект, и каждой ограничительной рамке присваивается класс, а также уровень уверенности — вероятность того, что в рамке действительно находится объект указанного класса.

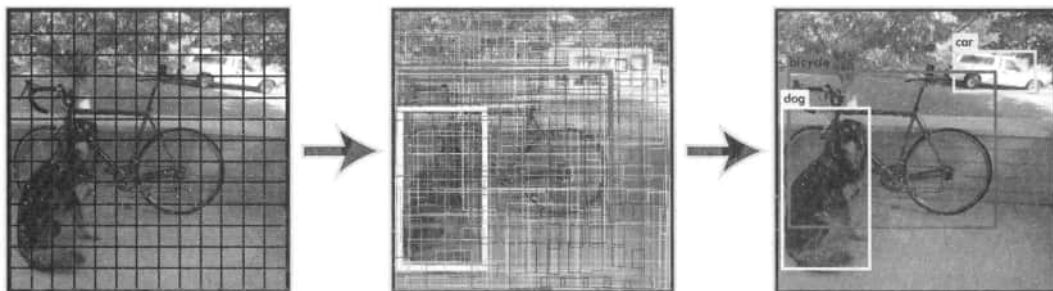


Рис. 12.11. Подход YOLO к анализу изображения (<https://oreil.ly/4dfXD>)

В настоящее время существует семь версий YOLO, именуемых с YOLOv1 по YOLOv7. Каждая новая версия улучшает предыдущую с точки зрения точности и производительности. Существуют также такие разновидности, как PP-YOLO (<https://oreil.ly/AMxGm>) и YOLO9000 (<https://oreil.ly/GOBMc>). YOLOv3 (<https://oreil.ly/jBHYy>) была последней версией, в разработке которой принимал участие создатель YOLO Джозеф Редмон (<https://oreil.ly/MnoZw>), и считается своего рода эталонной реализацией. Извлекая карты признаков из определенных слоев CNN, YOLOv3 анализирует изображение с помощью сеток 13×13 , 26×26 и 52×52 , пытаясь обнаружить объекты различных размеров. Она использует якоря для прогнозирования девяти ограничительных рамок на ячейку. Основным недостатком YOLO — трудности с обнаружением очень маленьких объектов, расположенных близко друг к другу, хотя YOLOv3 в этом отношении стала лучше, чем YOLOv1 и YOLOv2. Более подробную информацию о YOLO можно найти на сайте (<https://oreil.ly/x3EzO>) ее создателя. Отдельная статья под названием "Копаясь в глубинах YOLO V3" ("Digging Deep into YOLO V3"; <https://oreil.ly/eRCUE>) предлагает глубокое погружение в архитектуру YOLOv3.

YOLOv3 и Keras

Изначально YOLO была написана с использованием фреймворка глубокого обучения под названием Darknet (<https://oreil.ly/YfeXf>), но ее можно реализовать и с помощью других фреймворков. Проект `keras-yolo3` (<https://oreil.ly/yZZMt>) на GitHub содержит реализацию YOLOv3 в Keras, которую можно обучить с нуля или инициализировать с заданными весами и использовать для вывода, т. е. для прогнозирования. Эта версия принимает изображения 416×416 . Для того чтобы упростить использование, я создал файл `yolov3.py`, содержащий вспомогательные классы и вспомогательные функции. Это модифицированная версия файла, доступного в

проекте keras-yolo3. Я удалил ненужные для прогнозирования элементы, переписал часть кода для повышения практичности и производительности, а также добавил немного собственного кода, но основная заслуга принадлежит Huynh Ngoc Anh, чей GitHub ID — [experiencor](https://oreil.ly/UxTFz) (<https://oreil.ly/UxTFz>). С помощью yolov3.py и подготовленного набора весов вы можете загрузить YOLOv3 и делать прогнозы с помощью всего нескольких строк кода.

Для того чтобы увидеть YOLOv3 в действии, начните со скачивания ZIP-файла (<https://oreil.ly/mQPZS>), содержащего нужные вам файлы. Откройте ZIP-файл и поместите файл yolov3.py в каталог с вашими блокнотами. Остальные файлы поместите в подкаталог Data ваших блокнотов. Затем загрузите файл yolov3.weights (<https://oreil.ly/ukHHF>), содержащий веса, полученные при обучении YOLOv3 на наборе данных COCO, и также поместите его в подкаталог Data.

Теперь создайте новый блокнот Jupyter и вставьте следующий код в первую ячейку:

```
from yolov3 import *
```

```
model = make_yolov3_model()
weight_reader = WeightReader('Data/yolov3.weights')
weight_reader.load_weights(model)
model.summary()
```

Запустите код для импорта вспомогательных классов и функций в yolov3.py, создайте модель и инициализируйте ее весами COCO. Затем с помощью следующих операторов загрузите фотографию пары, катающейся на велосипеде по городской стене вокруг китайского города Сиань:

```
import matplotlib.pyplot as plt
%matplotlib inline
```

```
image = plt.imread('Data/xian.jpg')
width, height = image.shape[1], image.shape[0]
fig, ax = plt.subplots(figsize=(12, 8), subplot_kw={'xticks': [], 'yticks': []})
ax.imshow(image)
```

Теперь давайте посмотрим, сможет ли YOLOv3 обнаружить объекты на изображении. Используйте следующий код, чтобы снова загрузить изображение, изменить его размер до 416×416 , предварительно обработать пиксели и отправить полученное изображение в модель для прогнозирования:

```
import numpy as np
from tensorflow.keras.preprocessing.image import load_img, img_to_array

x = load_img('Data/xian.jpg', target_size=(YOLO3.width, YOLO3.height))
x = img_to_array(x) / 255
x = np.expand_dims(x, axis=0)
y = model.predict(x)
```

Функция `predict` возвращает массивы с информацией об объектах, обнаруженных на изображении в трех разрешениях (т. е. с ячейками сетки размером 8, 16 и 32 квадратных пиксела), но эти массивы необходимо декодировать в ограничительные рамки и отфильтрованные с помощью NMS рамки. Для этого в `yolov3.py` есть функция `decode_predictions` (вдохновленная функцией `decode_predictions` из Keras, <https://oreil.ly/hQwXP>), выполняющая постобработку. Для `decode_predictions` требуются ширина и высота исходного изображения в качестве входных данных, чтобы она могла масштабировать ограничительные рамки в соответствии с размерами исходного изображения. Возвращаемое значение — список объектов `BoundingBox`, каждый из которых содержит пиксельные координаты рамки, окружающей обнаруженный в сцене объект, а также идентифицирующую объект метку, и значение уверенности от 0,0 до 1,0.

Следующим шагом будет передача полученных от `predict` прогнозов в `decode_predictions` и составление списка ограничительных рамок:

```
boxes = decode_predictions(y, width, height)

for box in boxes:
    print(f'({box.xmin}, {box.ymin}), ({box.xmax}, {box.ymax}), ' +
          f'{box.label}, {box.score}')
```

Ниже приведен результат:

```
(692, 232), (1303, 1490), person, 0.9970048069953918
(1314, 327), (1920, 1496), person, 0.9957388639450073
(716, 786), (1277, 1634), bicycle, 0.9924144744873047
(1210, 845), (2397, 1600), bicycle, 0.9957170486450195
```

Модель обнаружила на изображении четыре объекта: двоих людей и два велосипеда. Метки взяты из набора данных COCO. Всего их 80, и они встроены в класс `YOLO3` в файле `yolov3.py`. Используйте следующую команду, чтобы отобразить список и увидеть все различные типы объектов, которые может обнаружить модель:

```
YOLO3.labels
```

`yolov3.py` также содержит вспомогательную функцию `annotate_image`, загружающую изображение из файловой системы и рисующую возвращаемые `decode_predictions` ограничительные рамки, а также метки и значения уверенности. Используйте следующий оператор для визуализации того, что модель нашла в изображении:

```
annotate_image('Data/xian.jpg', boxes)
```

Результат должен выглядеть примерно так, как показано на рис. 12.12.

Теперь попробуем проделать это с другим изображением — на этот раз с фотографией девушки и домашней собаки. Сначала выведите изображение и сохраните его ширину и высоту:

```
image = plt.imread('Data/abby-lady.jpg')
width, height = image.shape[1], image.shape[0]
fig, ax = plt.subplots(figsize=(12, 8), subplot_kw={'xticks': [], 'yticks': []})
ax.imshow(image)
```



Рис. 12.12. Результат распознавания объектов на фото людей на велосипедах

Предварительно обработайте изображение и передайте его в метод `predict` модели:

```
x = load_img('Data/abby-lady.jpg', target_size=(YOLO3.width, YOLO3.height))
x = img_to_array(x) / 255
x = np.expand_dims(x, axis=0)
y = model.predict(x)
```

Снова выведите изображение, на этот раз с аннотациями обнаруженных на нем объектов:

```
boxes = decode_predictions(y, width, height)
annotate_image('Data/abby-lady.jpg', boxes)
```

На рис. 12.13 приведен результат.

Модель обнаружила девушку и ее ноутбук, но не обнаружила собаку, несмотря на то, что обучающие изображения COCO включали собак. На самом деле модель *обнаружила* собаку, но с уверенностью менее 90%. Метод `predict` модели обычно возвращает информацию о тысячах ограничительных рамок, большинство из которых можно игнорировать из-за низкого уровня уверенности. По умолчанию `decode_predictions` игнорирует ограничительные рамки с уровнями уверенности менее 0,9, но это можно отменить, включив в вызов параметр `min_score`. Используйте следующие операторы для декодирования прогнозов и их визуализации, на этот раз с минимальным уровнем уверенности 55%:

```
boxes = decode_predictions(y, width, height, min_score=0.55)
annotate_image('Data/abby-lady.jpg', boxes)
```

При снижении порога уверенности до 0,55 модель обнаружила не только собаку, но и диван (рис. 12.14).

Поскольку модель была обучена на наборе данных COCO, она может обнаруживать и множество других объектов, включая светофоры, знаки остановки, различные виды еды и животных, и даже бутылки и бокалы для вина. Попробуйте использо-

вать ее на своих изображениях, чтобы убедиться в том, что это самое современное средство обнаружения объектов.

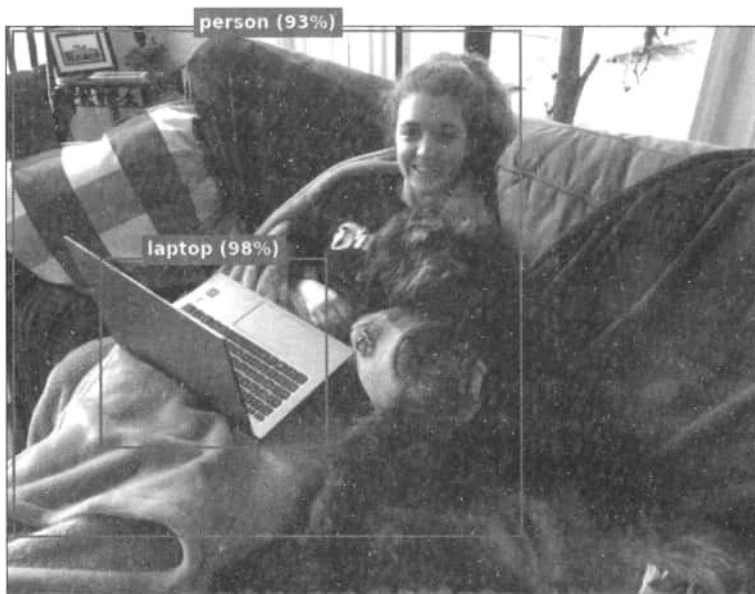


Рис. 12.13. Результат обнаружения человека и ноутбука на фотографии девушки и домашней собаки

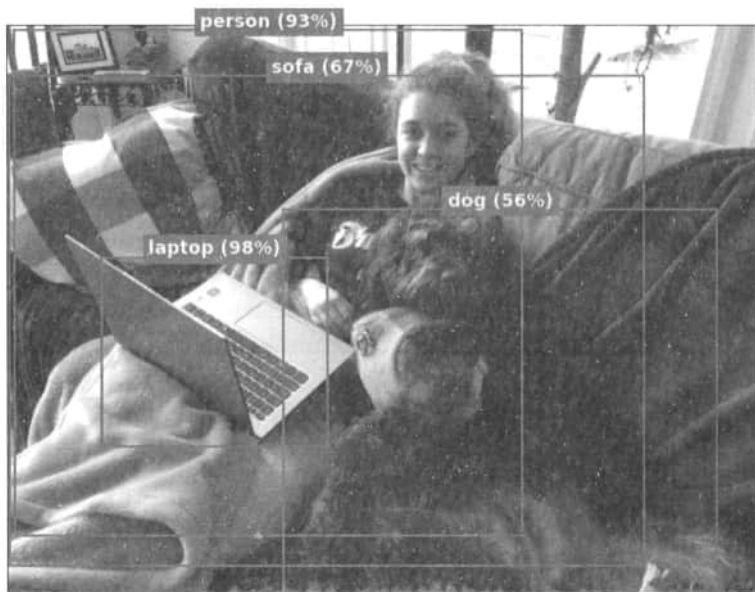


Рис. 12.14. Результат обнаружения человека, собаки, дивана и ноутбука на фотографии девушки и домашней собаки

Обнаружение нестандартных объектов

Модель обнаружения объектов, обученная на наборе данных COCO, может легко обнаружить и идентифицировать 80 различных типов объектов. Но что, если вам нужна модель обнаружения, которая должна идентифицировать другие объекты? Например, если требуется модель, обнаруживающая номерные знаки на автомобилях, пакеты, оставленные на крыльце, или раковые клетки в образцах тканей? Все эти способы обнаружения объектов используются сегодня в прикладных отраслях. И ни один из них не опирается на модели, обученные исключительно на изображениях COCO.

Обучение модели обнаружения объектов с нуля — тяжелый труд даже для исследователей из Microsoft, Facebook и Google. Можно немного сократить этот процесс, начав с весов COCO и постепенно обучая сеть новым классам, но даже это требует больших затрат времени и вычислений.

Есть более легкий путь. В основе всей этой книги лежит предпосылка, что вы разработчик программного обеспечения или инженер, а не изучающий данные доктор наук. Вы хотите использовать машинное обучение (или глубокое обучение) для повышения ценности вашего бизнеса. Вам нужны результаты, а не уравнения на доске. Графический процессор у вас может быть в наличии, а может нет, но у вас есть работа, которую необходимо выполнить, и в данном случае эта работа включает в себя пользовательское обнаружение объектов.

Более легкий путь — это Azure Cognitive Services (<https://oreil.ly/98V37>), а точнее, представитель семейства Cognitive Services под названием Custom Vision service (<https://oreil.ly/cMAHe>). Я представляю Azure Cognitive Services в *главе 14*, но если вашей целью является создание пользовательского детектора объектов, то сейчас самое время для первого знакомства с миром искусственного интеллекта как службы. На мой взгляд, ничто не делает создание пользовательских моделей обнаружения объектов проще, чем служба Custom Vision. Для этого не требуется особых знаний, а в итоге вы получаете обученную модель, которую можно легко использовать в Python, C# или других языках программирования, например Swift.

Служба Azure Custom Vision — один из самых секретных секретов в области искусственного интеллекта. Давайте используем его для обучения пользовательской модели обнаружения объектов, а затем представим себя морскими биологами и запустим модель в работу.

Обучение пользовательской модели обнаружения объектов с помощью службы Custom Vision

Служба Custom Vision — одна из более чем 20 служб и API, входящих в состав Azure Cognitive Services. Службы Cognitive Services позволяют создавать интеллектуальные приложения, не требуя глубоких знаний в области машинного обучения и искусственного интеллекта. Служба Custom Vision позволяет создавать модели

классификации изображений и обнаружения объектов и обучать их в облаке на графических процессорах. После завершения обучения вы можете развернуть модели в виде веб-сервисов и вызывать их с помощью REST API или загрузить их в различных форматах, включая TensorFlow и Core ML (<https://oreil.ly/uSkBE>), чтобы использовать локально. В загрузку включены примеры исходного кода, демонстрирующие, как использовать модель.

Для того чтобы использовать службу Custom Vision, необходимо иметь учетную запись Microsoft и подписку Azure. Если у вас нет учетной записи Microsoft, вы можете создать ее бесплатно (<https://oreil.ly/n5T1n>). Если у вас нет подписки Azure, вы можете создать бесплатную пробную подписку (<https://oreil.ly/izl8G>). Для этого необходимо предоставить кредитную или дебетовую карту, но вы получите 200 долларов в виде бесплатных кредитов на 30 дней и доступ ко многим уровням бесплатных услуг. Даже если вы оформили платную подписку или перевели бесплатную пробную подписку в платную, вы не будете платить за использование сервиса Custom Vision, особенно если вы экспортируете созданные модели.



Если вы предпочитаете не создавать подписку Azure, можете пропустить оставшуюся часть этого раздела и продолжить изучение со следующего раздела, где мы используем обученную модель для обнаружения объектов. Все файлы, необходимые для использования модели, находятся в папке главы 12 (<https://oreil.ly/IPwtZ>) в репозитории GitHub этой книги.

Готовы создать собственную модель обнаружения объектов? Как насчет модели, позволяющей обнаруживать морских черепах в дикой природе? Начните с загрузки набора данных (<https://oreil.ly/5L2g4>) с изображениями морских черепах. Я использовал поиск изображений Bing, чтобы найти эти фотографии, и ограничил поиск изображениями, которые можно свободно распространять и использовать в коммерческих целях. Скопируйте изображения из ZIP-файла в подходящее место на жестком диске.

Затем перейдите на портал Custom Vision (<https://oreil.ly/a9aMy>) в браузере и войдите в систему под своей учетной записью Microsoft. Нажмите + **NEW PROJECT** (Новый проект), а в диалоговом окне **Create new project** (Создать новый проект) щелкните по ссылке **create new** (создать новый), чтобы создать новый ресурс Azure для проекта (рис. 12.15).

Введите имя ресурса, например "turtle-detector", и выберите подписку Azure, как показано на рис. 12.16. Выберите существующую группу ресурсов Azure для нового ресурса или щелкните по ссылке **create new** и создайте новый ресурс. Убедитесь, что в качестве типа ресурса выбраны Cognitive Services, и либо примите расположение Azure по умолчанию, либо выберите наиболее близкое к вам. Наконец, выберите F0 в качестве ценового уровня, если он доступен, или S1 в противном случае. (F0 — это бесплатный уровень, обычно он ограничен одним ресурсом на подписку Azure.) Затем нажмите кнопку **Create resource**.

Create new project

Name*

Enter project name

Description

Enter project description

Resource*

create new

Manage Resource Permissions

Рис. 12.15. Создание нового ресурса Azure в проекте службы Custom Vision

Create New Resource

Name*

turtle-detector

Subscription*

Resource Group*

create new

custom-vision-rg

Kind

CognitiveServices

Location

South Central US

Pricing Tier

S0

Create resource

Рис. 12.16. Создание нового ресурса Cognitive Services



В Azure *группа ресурсов* — это набор связанных ресурсов, которые служат общей цели и имеют общее время жизни. При удалении группы ресурсов удаляются все ресурсы в этой группе. Кроме того, вы можете легко получить информацию о биллинге для группы в целом, а не суммировать затраты на отдельные ресурсы.

Create new project [X]

Name*

Object Detection (Sea Turtles)

Description

Enter project description

Resource* create new

turtle-detector [S0]

Manage Resource Permissions

Project Types ⓘ

☐ Classification

☒ Object Detection

Domains:

☐ General [A1]

☐ General

☐ Logo

☐ Products on Shelves

☐ General (compact) [S1]

☒ General (compact)

Pick the domain closest to your scenario. Compact domains are lightweight models that can be exported to iOS/Android and other platforms. [Learn More](#)

Export Capabilities: ⓘ

☒ Basic platforms (Tensorflow, CoreML, ONNX, ...)

☐ Vision AI Dev Kit

Cancel Create project

Рис. 12.17. Создание проекта, содержащего детектор объектов

В диалоговом окне **Create new project** введите название проекта и выберите ресурс, который вы только что создали (рис. 12.17). В качестве типа проекта устано-

вите переключатель **Object Detection** (Обнаружение объектов), а в качестве домена — **General (compact)** (Общий (компактный)). Выбор одного из "компактных" доменов необходим, если целью является обучение модели, которую можно экспортировать и использовать локально. Убедитесь, что в разделе **Export Capabilities** (Возможности экспорта) выбрана опция **Basic platforms** (Основные платформы), а затем нажмите кнопку **Create project** (Создать проект).

Теперь, когда проект Custom Vision создан, следующий шаг — загрузить изображения, используемые для обучения модели, и аннотировать их именами классов и ограничительными рамками. Для того чтобы начать, нажмите **Add Images** (Добавить изображения). Затем перейдите в папку, где хранятся изображения морских черепах, и загрузите все 50. После завершения загрузки нажмите на первое изображение, чтобы открыть его в редакторе изображений. С помощью мыши нарисуйте рамку вокруг морской черепахи (рис. 12.18).



Рис. 12.18. Идентификация морских черепах на обучающих изображениях

Затем введите **sea-turtle** в появившемся редакторе тегов и нажмите на знак плюса справа от него. Теперь нажмите стрелку  в правой части окна, чтобы перейти к следующему изображению, и повторите этот процесс для остальных обучающих изображений. Вам не нужно снова вводить название тега; вы просто выбираете его

из списка существующих тегов. Кроме того, обычно не нужно рисовать рамки вокруг морских черепах. Наведите курсор на морскую черепаху, и редактор с помощью искусственного интеллекта сам предложит создать ограничительную рамку.



Самая утомительная часть обучения пользовательской модели обнаружения объектов — определение объектов и ограничительных рамок на всех учебных изображениях. Служба Custom Vision помогает в этом, предоставляя интеллектуальный редактор, минимизирующий количество необходимых щелчков и перетаскиваний. Но представьте себе, как это проделать для сотен тысяч изображений!

В случаях, когда фотография содержит две морские черепахи или более, обязательно пометьте их всех, как показано на рис. 12.19. Ничего страшного, если ограничительные рамки накладываются друг на друга. Ведь в реальном мире объекты часто пересекаются. Специалисты по исследованию данных называют перекрывающиеся ограничительные рамки *окклюзиями*.

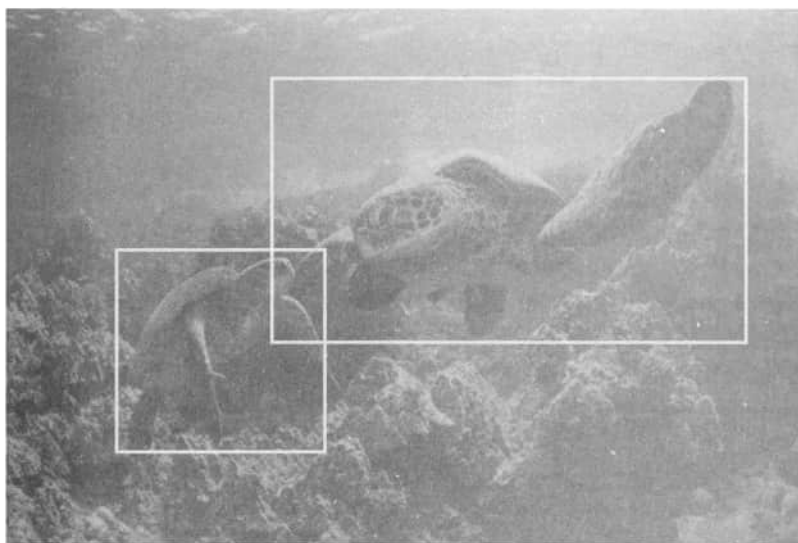


Рис. 12.19. Морские черепахи с перекрывающимися ограничительными рамками

После того как вы отметите все 50 учебных изображений, нажмите кнопку **Train** (Обучение) в правом верхнем углу страницы. Когда вам будет предложено выбрать тип обучения (рис. 12.20), выберите вариант **Quick Training** (Быстрое обучение), чтобы сэкономить время и деньги, а затем нажмите кнопку **Train** (Обучение) в правом нижнем углу диалогового окна, чтобы начать обучение. Вариант расширенного обучения обычно позволяет получить более точную модель, но занимает больше времени и оплачивается в пределах количества часов, заложенных в бюджет. Расширенное обучение может быть недоступно, если при создании модели вы выбрали бесплатный уровень F0. F0 предоставляет один час обучения в месяц бесплатно.

Choose Training Type

Training Types ⓘ

☒ Quick Training
 ☐ Advanced Training

In most cases, the more time you select the better the model will be. You're charged based on the compute time used to train your model, so choose your budget based on your need.

Training budget: 1 hour ⓘ

1 hour | | | 96 hours

☐ Send me an email notification after training completes

Email address

Train

Рис. 12.20. Выбор варианта обучения

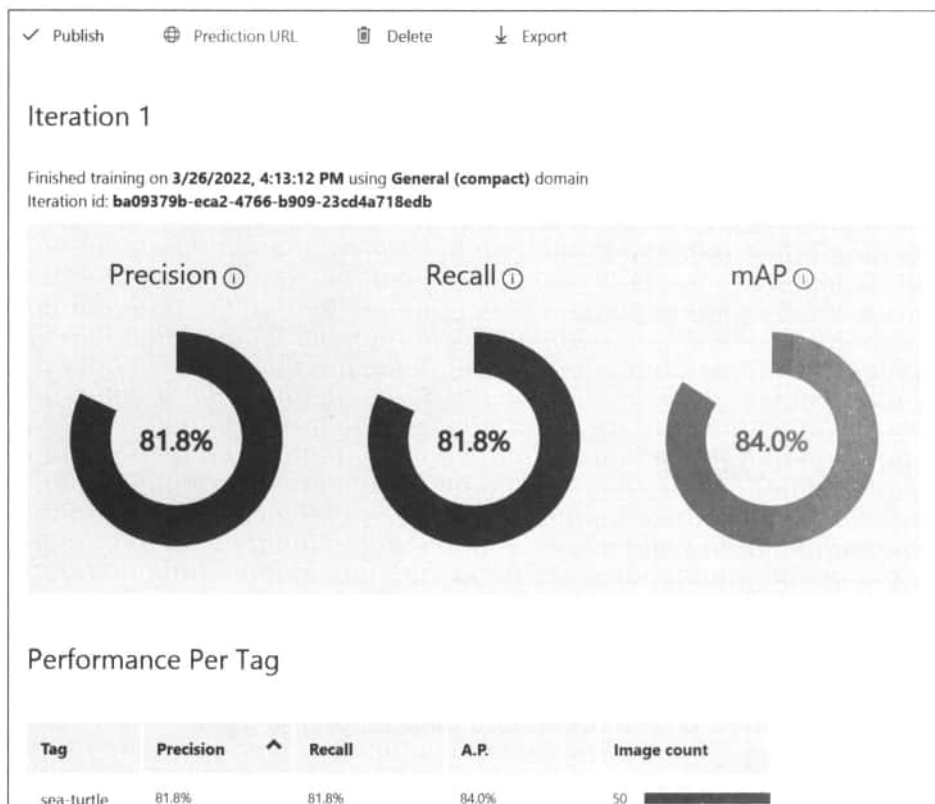


Рис. 12.21. Результаты обучения

Быстрое обучение обычно занимает от 5 до 10 минут. По окончании обучения вы увидите итог, подобный приведенному на рис. 12.21. Вы уже знаете, что такое точность и полнота. Третья метрика — *средняя точность* (mean average precision, mAP) — является общей метрикой для оценки точности моделей обнаружения объектов. Она отражает способность модели классифицировать объекты и определять их ограничительные рамки. Чем выше процент, тем лучше. Если этот показатель недостаточно высок, то у вас есть два варианта: тренироваться на большем количестве изображений или попробовать продвинутое обучение вместо быстрого (или и то и другое). Современные модели обнаружения объектов обычно обучаются на тысячах примеров каждого класса.



В качестве эксперимента я обучил ту же модель, используя опцию расширенного обучения и выделив на обучение один час. Обучение заняло менее 30 минут, а показатель mAP достиг 97,2%. На момент написания этой главы расширенное обучение стоит 10 долларов за час, поэтому стоимость моей подписки на Azure составила менее 5 долларов. Более подробную информацию о ценах можно найти на странице **Custom Vision** (<https://oreil.ly/DCAvD>).

Вы можете проверить точность модели, нажав кнопку **Quick Test** в верхней части страницы и выбрав пару фотографий, на которых модель не обучалась. Или можете нажать кнопку **Publish** в левом верхнем углу страницы и развернуть модель как веб-сервис. Но наша цель — скачать модель и запустить ее локально. Для этого нажмите **Export** в верхней части страницы. Выберите TensorFlow в качестве типа экспорта, а затем нажмите кнопку **Export**. Подождите, пока экспорт будет подготовлен, а затем нажмите кнопку **Download** (рис. 12.22).

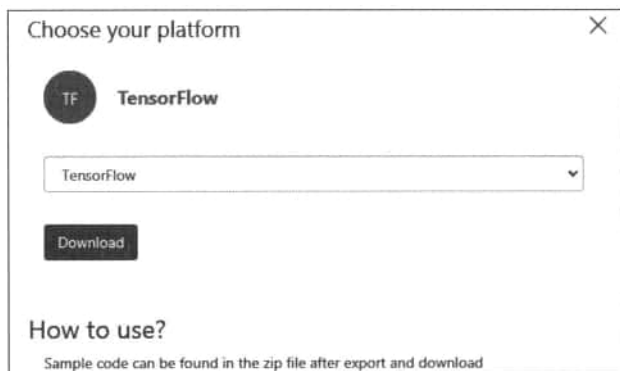


Рис. 12.22. Загрузка обученной модели TensorFlow

Когда загрузка завершится, откройте скачанный ZIP-файл. Скопируйте следующие файлы из архива в каталог, где размещены ваши блокноты Jupyter.

labels.txt

Содержит метку, созданную вами (sea-turtle, морская черепаха) при маркировке изображений.

model.pb

Содержит обученную и сериализованную модель TensorFlow.

object_detection.py

Находится в папке python ZIP-файла и содержит вспомогательные функции для использования модели в приложениях Python.

После загрузки этих файлов удалите проект на портале Custom Vision, если вы не хотите вернуться и доработать модель, обучив ее на большем количестве изображений (или предоставив ей больше времени на обучение). Для того чтобы быть уверенным в отсутствии дополнительных расходов на подписку Azure, удалите содержащую модель группу ресурсов Azure. Удалить группы ресурсов можно через портал Azure (<https://oreil.ly/wYcWH>).

Обратите внимание, что при подготовке модели с помощью службы Custom Vision вы не ограничены одним классом объектов. Вы можете загружать изображения с десятками различных классов и помечать их соответствующим образом в редакторе деталей изображения. В этом случае файл labels.txt будет содержать все метки, а не только одну.

Использование экспортированной модели

Теперь вы готовы использовать модель в работе. Начните с создания нового блокнота Jupyter и вставьте следующий код в первую ячейку. Эти операторы определяют класс `SeaTurtleDetector`, представляющий модель TensorFlow, и функцию `annotate_image`, аннотирующую обнаруженное моделью изображение. Класс `SeaTurtleDetector` является дочерним классом класса `ObjectDetection`, реализованного в файле `object_detection.py`. В нем имеется метод `predict`, который можно вызвать для обнаружения объектов на изображении:

```
import sys
import tensorflow as tf
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle
from object_detection import ObjectDetection
from PIL import Image
import numpy as np

class SeaTurtleDetector(ObjectDetection):
    def __init__(self, graph_def, labels):
        super(SeaTurtleDetector, self).__init__(labels)
        self.graph = tf.compat.v1.Graph()
        with self.graph.as_default():
            input_data = tf.compat.v1.placeholder(
                tf.float32, [1, None, None, 3], name='Placeholder')
```



```

    tf.import_graph_def(graph_def,
                        input_map={"Placeholder:0": input_data}, name="")
def predict(self, preprocessed_image):
    inputs = np.array(preprocessed_image, dtype=float)[: , : , (2, 1, 0)]

    with tf.compat.v1.Session(graph=self.graph) as sess:
        output_tensor = sess.graph.get_tensor_by_name('model_outputs:0')
        outputs = sess.run(output_tensor,
                            {'Placeholder:0': inputs[np.newaxis, ...]})

    return outputs[0]

def annotate_image(image, predictions, min_score=0.7, figsize=(12, 8)):
    fig, ax = plt.subplots(figsize=figsize, subplot_kw={'xticks': [], 'yticks': []})
    img_width, img_height = image.size
    ax.imshow(image)

    for p in predictions:
        score = p['probability']

        if score >= min_score:
            x1 = p['boundingBox']['left'] * img_width
            y1 = p['boundingBox']['top'] * img_height
            width = p['boundingBox']['width'] * img_width
            height = p['boundingBox']['height'] * img_height
            label = p['tagName']

            rect = Rectangle((x1, y1), width, height, fill=False, color='red', lw=2)
            ax.add_patch(rect)
            label = f'{label} ({score:.0%})'
            ax.text(x1 + (width / 2), y1, label, color='white',
                    backgroundcolor='red', ha='center', va='bottom',
                    fontweight='bold', bbox=dict(color='red'))

Далее с помощью следующих операторов загрузите сериализованную модель в
model.pb, а метки в labels.txt, и инициализируйте модель обнаружения объектов:
# Загрузка сериализованного графа модели
graph_def = tf.compat.v1.GraphDef()
with tf.io.gfile.GFile('model.pb', 'rb') as f:
    graph_def.ParseFromString(f.read())

# Загрузка меток
with open('labels.txt', 'r') as f:
    labels = [l.strip() for l in f.readlines()]

# Создание модели
model = SeaTurtleDetector(graph_def, labels)

```

Наконец, воспользуйтесь Интернетом и скачайте фотографию морской черепахи. Затем загрузите фотографию в свой блокнот и посмотрите, обнаружит ли модель морскую черепаху на ней:

```
%matplotlib inline
```

```
image = Image.open('PATH_TO_IMAGE_FILE')
predictions = model.predict_image(image)
annotate_image(image, predictions)
```

Справилась ли модель с задачей? В моем случае — да (рис. 12.23).



Рис. 12.23. Обнаруженная на фото черепаха

Помните, что высочайшая точность обнаружения объектов — это результат обучения на большом количестве помеченных изображений. 50 использованных для обучения изображений — это хорошее начало, но чтобы обучить модель с достаточной точностью для использования в реальном мире, вам, вероятно, потребуется как минимум в 10 раз больше. Хорошей новостью является то, что после подготовки изображений служба Custom Vision делает процесс маркировки изображений и обучения модели относительно простым.

Резюме

Обнаружение объектов представляет собой передовой рубеж в мире компьютерного зрения. Именно с его помощью автономные автомобили "видят" объекты перед собой, и именно эта технология лежит в основе множества реальных приложений. Современное обнаружение объектов достигается с помощью глубокого обучения с использованием специально разработанных CNN, которые выполняют нечто боль-

шее, чем простая классификация изображений. Вы можете использовать предварительно обученные модели, такие как YOLO и Mask R-CNN, для обнаружения объектов на изображениях и видеокдрах. С помощью Mask R-CNN можно даже генерировать маски сегментации, раскрывающие дополнительную информацию об этих объектах.

Обнаружение нестандартных объектов требует обучения подобных моделей на тщательно промаркированных изображениях с указанием объектов и ограничительных рамок. Служба Azure Custom Vision упрощает процесс обучения пользовательских моделей обнаружения объектов и позволяет размещать обучаемые модели в облаке и вызывать их с помощью REST API или скачивать и использовать локально. Это одна из нескольких служб, входящих в состав Azure Cognitive Services, и она является подходящим инструментом для работы в случае необходимости обнаружения объектов, на которых не были обучены предварительно подготовленные модели.

Обработка естественного языка

С помощью Scikit-Learn несложно построить модели машинного обучения, которые анализируют текст на предмет настроения, выявляют спам и классифицируют текст другими способами. Но сегодня современная классификация текстов чаще всего выполняется с помощью нейронных сетей. Вы уже знаете, как создавать нейронные сети, принимающие на вход числа и изображения. Давайте продолжим эту тему и научимся строить обрабатывающие текст модели глубокого обучения — сегмента глубокого обучения, известного как *обработка естественного языка* (natural language processing, NLP).

NLP включает в себя различные виды задач, в том числе классификацию текстов, распознавание именованных сущностей, извлечение ключевых слов, ответы на вопросы и перевод на другой язык. В последние годы точность моделей NLP повысилась по целому ряду причин, не последнюю роль среди которых играют новые и более совершенные способы преобразования слов и предложений во включающие смысл плотные векторные представления, а также относительно новая архитектура нейронных сетей под названием *трансформер* (transformer), позволяющая выделять наиболее значимые слова и даже различать разные значения одного и того же слова.

Одним из общих элементов практически всех обрабатывающих текст нейронных сетей является *эмбединговый слой*, использующий *эмбединги слов* для преобразования массивов или представляющих слова последовательностей скалярных значений в массивы чисел с плавающей точкой, называемых *векторами слов*. Эти векторы кодируют информацию о значениях слов и связях между ними. Выходные данные эмбедингового слоя могут быть поданы на классификационный слой или на другие типы слоев нейронной сети, чтобы извлечь из них больше смысла, прежде чем подвергать их дальнейшей обработке.

Трансформеры? Эмбединговые слои? Векторы слов? Здесь есть много чего интересного, но как только вы освоите несколько основных понятий, обрабатывающие язык нейронные сети станут настоящим волшебством. Давайте погрузимся в процесс. Для начала мы узнаем, как подготовить текст к обработке моделью глубокого обучения и как создавать эмбединги слов. Затем мы применим эти знания для построения нейронных сетей, которые классифицируют текст, переводят его с одного языка на другой и многое другое — все это классическое применение NLP.

Подготовка текста

В главе 4 был представлен класс `CountVectorizer` (<https://oreil.ly/EcATK>) от Scikit-Learn, преобразующий строки текста в строки количества слов, которые может использовать модель машинного обучения. `CountVectorizer` также преобразует символы в строчные, удаляет цифры и знаки препинания и, по желанию, удаляет стоп-слова — такие распространенные слова, как *and* и *the*, которые, скорее всего, не окажут большого влияния на результат.

Текст также должен быть очищен и векторизован перед использованием в обучении нейронной сети, но векторизация обычно выполняется по-другому. Вместо того чтобы создавать таблицу количества слов, вы создаете таблицу последовательностей, содержащих представляющие отдельные слова *токены*. Токены часто являются индексами *словаря*, или лексикона, созданного на основе совокупности слов в наборе данных. Для того чтобы помочь в этом, Keras предоставляет класс `Tokenizer` (<https://oreil.ly/nfVsh>), который можно рассматривать как эквивалент `CountVectorizer` в глубоком обучении. Вот пример, в котором `Tokenizer` используется для создания последовательностей из четырех строк текста:

```
from tensorflow.keras.preprocessing.text import Tokenizer
```

```
lines = [
    'The quick brown fox',
    'Jumps over $$$ the lazy brown dog',
    'Who jumps high into the blue sky after counting 123',
    'And quickly returns to earth'
]
```

```
tokenizer = Tokenizer()
tokenizer.fit_on_texts(lines)
sequences = tokenizer.texts_to_sequences(lines)
```

Метод `fit_on_texts` (<https://oreil.ly/Nbbba>) создает словарь, содержащий все слова входного текста. Метод `texts_to_sequences` (<https://oreil.ly/JijdJ>) возвращает список последовательностей, являющихся просто массивами индексов в словаре:

```
[[1, 4, 2, 5],
 [3, 6, 1, 7, 2, 8],
 [9, 3, 10, 11, 1, 12, 13, 14, 15, 16],
 [17, 18, 19, 20, 21]]
```

Слово *brown* встречается в строках 1 и 2 и представлено индексом 2. Следовательно, 2 встречается в обеих последовательностях. Аналогично, индекс 3, обозначающий слово *jumps*, встречается в последовательностях 2 и 3. Индекс 0 не используется для обозначения слов; он зарезервирован. Подробнее об этом чуть позже.

Вы можете использовать метод `sequences_to_texts` (<https://oreil.ly/cxSdq>) класса `Tokenizer` для обратного процесса и преобразования последовательностей снова в текст:

```
['the quick brown fox',
 'jumps over the lazy brown dog',
 'who jumps high into the blue sky after counting 123',
 'and quickly returns to earth']
```

Одним из открытий оказывается то, что `Tokenizer` преобразует текст в строчные буквы и удаляет символы, но не удаляет стоп-слова или числа. Если вам нужно удалить стоп-слова, можете использовать отдельную библиотеку, например `Natural Language Toolkit (NLTK)`, <https://oreil.ly/Ic2ON>). Также можно удалить слова, содержащие цифры:

```
from tensorflow.keras.preprocessing.text import Tokenizer
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
```

```
lines = [
    'The quick brown fox',
    'Jumps over $$$ the lazy brown dog',
    'Who jumps high into the blue sky after counting 123',
    'And quickly returns to earth'
]
```

```
def remove_stop_words(text):
    text = word_tokenize(text.lower())
    stop_words = set(stopwords.words('english'))
    text = [word for word in text if word.isalpha() and not word in stop_words]
    return ' '.join(text)
```

```
lines = list(map(remove_stop_words, lines))
```

```
tokenizer = Tokenizer()
tokenizer.fit_on_texts(lines)
tokenizer.texts_to_sequences(lines)
```

Полученные последовательности выглядят примерно следующим образом:

```
[[3, 1, 4],
 [2, 5, 1, 6],
 [2, 7, 8, 9, 10],
 [11, 12, 13]]
```

И они же, затем преобразованные в текст, выглядят так:

```
['quick brown fox',
 'jumps lazy brown dog',
```

```
'jumps high blue sky counting',
'quickly returns earth']
```

Длина последовательностей варьируется от трех до пяти значений, но нейронная сеть ожидает, что все последовательности будут одинаковой длины. Функция Keras `pad_sequences` (<https://oreil.ly/nQoQZ>) выполняет этот последний шаг, обрезая последовательности, длина которых больше указанной, и заполняя нулями последовательности, длина которых меньше указанной:

```
from tensorflow.keras.preprocessing.sequence import pad_sequences
```

```
padded_sequences = pad_sequences(sequences, maxlen=4)
```

Полученные в результате последовательности выглядят следующим образом:

```
array([[ 0,  3,  1,  4],
       [ 2,  5,  1,  6],
       [ 7,  8,  9, 10],
       [ 0, 11, 12, 13]])
```

Преобразование этих последовательностей обратно в текст дает следующее:

```
['quick brown fox',
'jumps lazy brown dog',
'high blue sky counting',
'quickly returns earth']
```

По умолчанию `pad_sequences` выполняет вставку и усечение слева, но вы можете включить параметр `padding='post'`, если хотите, чтобы вставка и усечение выполнялись справа. Вставка справа иногда важна при использовании нейронных сетей для перевода текста.



Удаление стоп-слов часто практически не влияет на задачи классификации текста. Если вам нужно просто удалить числа из поступающего в нейронную сеть текста и при этом не удалять стоп-слова, создайте `Tokenizer` следующим образом:

```
tokenizer = Tokenizer(
    filters='!"#$%&()*+,-./:;<=>?@[\\]^_`\' \
    \'{|}~\t\n0123456789')
```

Параметр `filters` указывает `Tokenizer`, какие символы удалять. По умолчанию он принимает значение `'!"#$%&()*+,-./:;<=>?@[\\]^_`\'{|}~\t\n'`. Код выше просто добавляет 0–9 в список. Обратите внимание, что по умолчанию `Tokenizer` не удаляет апострофы, но вы можете удалить их, добавив апостроф в список `filters`.

Важно, чтобы текст, поступающий в нейронную сеть для прогнозирования, был токенизирован и снабжен теми же символами, что и текст, поступающий в модель для обучения. Если вы думаете, что было бы неплохо не заниматься токенизацией и упорядочиванием вручную, есть способ обойти эту проблему. Вместо того чтобы писать много кода, можно включить в модель слой `TextVectorization`

(<https://oreil.ly/rydux>). Сейчас я покажу, как это сделать. Но сначала вам нужно узнать об эмбединговых слоях.

Эмбединги слов

После того как текст обработан и преобразован в последовательности, он готов для обучения нейронной сети. Но, вероятно, вы не сможете далеко продвинуться, используя необработанные последовательности.

Одним из важнейших элементов обрабатывающей текст нейронной сети является эмбединговый слой (<https://oreil.ly/vd83A>), задача которого — преобразовать заполненные последовательности словарных токенов в массивы векторов слов (<https://oreil.ly/bA39w>), представляющих каждое слово не одним целым числом, а массивом (вектором) чисел с плавающей точкой. Каждое слово во входном тексте представлено вектором в эмбединговом слое, и по мере обучения сети векторы, представляющие отдельные слова, корректируются, чтобы можно было отразить их связь друг с другом. Если вы создаете модель анализа настроений и такие слова, как *"превосходный"* и *"потрясающий"*, имеют схожий смысл, то векторы, представляющие эти слова в эмбединговом пространстве, должны быть расположены относительно близко друг к другу, чтобы фразы типа *"превосходное обслуживание"* и *"потрясающее обслуживание"* оценивались одинаково.

Реализация эмбедингового слоя вручную — сложная задача (особенно в плане обучения), поэтому Keras предлагает класс `Embedding` (<https://oreil.ly/OQMzf>). В Keras создание обучаемого эмбедингового слоя требует всего одной строки кода:

```
Embedding(input_dim=10000, output_dim=32, input_length=100)
```

В функцию `Embedding` передаются три параметра по следующему порядку:

- ◆ размер словаря, или количество слов в словаре, созданном `Tokenizer`;
- ◆ количество измерений m в эмбединговом пространстве;
- ◆ длина n каждой заполненной последовательности.

Вы выбираете число измерений m , и каждое слово кодируется в эмбединговом пространстве как m -мерный вектор. Большее число измерений обеспечивает большую мощность подгонки, но также увеличивает время обучения. На практике m обычно представляет собой число от 32 до 512.

Векторы, представляющие отдельные слова в эмбединговом слое, изучаются в процессе обучения, так же как изучаются веса, соединяющие нейроны в соседних плотных слоях. Если количество обучающих образцов достаточно велико, при обучении сети обычно создаются эффективные векторные представления всех слов. Однако если у вас всего несколько сотен обучающих образцов, эмбединговый слой может не обладать информацией в достаточном количестве для правильной векторизации текста.

В этом случае вы можете инициализировать эмбединговый слой предварительно обученными эмбедингами слов, а не полагаться на то, что он сам освоит эмбе-

динги слов. В открытом доступе существует несколько популярных предварительно обученных эмбеддингов слов, включая разработанные в Стэнфорде векторы слов GloVe (<https://oreil.ly/9xBhl>) и разработку Word2Vec (<https://oreil.ly/NzrSo>) от Google. Предварительно обученные эмбеддинги, как правило, моделируют семантические отношения между словами, определяя, например, что король и королева — родственные термины, а лестница и зебра — нет. Хотя это может быть полезно, обученная для классификации текста сеть обычно работает лучше, когда эмбеддинги слов получены из обучающих данных, поскольку такие эмбеддинги специфичны для конкретных задач. Пример применения предварительно обученных эмбеддингов приведен в статье "Использование предварительно обученных эмбеддингов слов" ("Using Pretrained Word Embeddings", <https://oreil.ly/0VC3Q>), написанной автором Keras.

Классификация текста

На рис. 13.1 показана базовая архитектура нейронной сети, выполняющей классификацию текста. Токенизированные текстовые последовательности поступают на вход эмбеддингового слоя. Выходом эмбеддингового слоя является двумерная матрица значений с плавающей точкой размером m на n , где m — количество измерений в эмбеддинговом пространстве, а n — длина последовательности. Слой Flatten (<https://oreil.ly/2fsvZ>), следующий за эмбеддинговым слоем, "сплюсчивает" двумерный выход в одномерный массив, пригодный для ввода в плотный слой, а плотный слой классифицирует значения, полученные из слоя Flatten. Вы можете экспериментировать с различными размерами эмбеддингового слоя и разной шириной плотного слоя для достижения максимальной точности. При необходимости можно добавить больше плотных слоев.

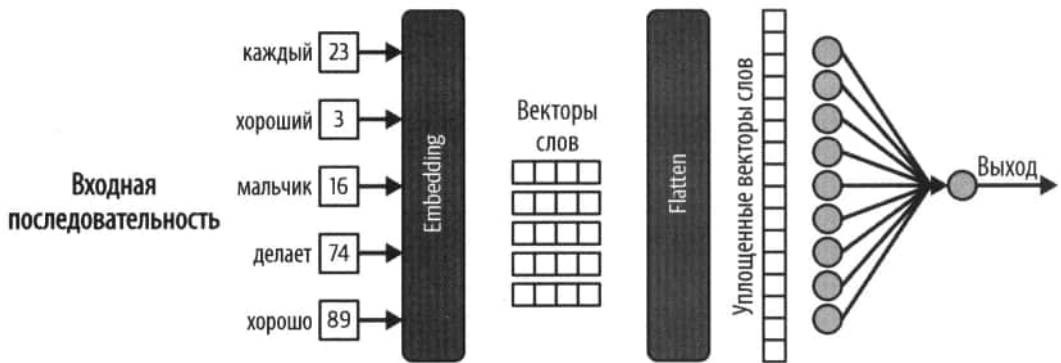


Рис. 13.1. Нейронная сеть для классификации текста

Представленная на рис. 13.1 нейронная сеть может быть реализована следующим образом:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Embedding
```

```

model = Sequential()
model.add(Embedding(10000, 32, input_length=100))
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

```

Одним из применений классифицирующей текст нейронной сети является фильтрация спама. В *главе 4* было показано, как с помощью Scikit построить модель машинного обучения, отделяющую спам от нормальных писем. Давайте построим эквивалентную модель глубокого обучения с помощью Keras и TensorFlow. Мы будем использовать тот же набор данных, что и раньше: набор, содержащий 1000 писем, половина из которых является спамом (обозначается единицами в столбце меток), а половина — нет (обозначается нулями в столбце меток).

Для начала загрузите набор данных (<https://oreil.ly/jWEyN>) и скопируйте его в подкаталог Data, где размещены ваши блокноты Jupyter. Затем с помощью следующих операторов загрузите набор данных и перетасуйте строки, чтобы распределить положительные и отрицательные образцы по всему набору. Перемешивание важно, потому что вместо того, чтобы использовать `train_test_split` для создания проверочного набора данных, мы будем использовать параметр `validation_split` из `fit`. Он не перемешивает данные, как это делает `train_test_split`:

```

import pandas as pd

df = pd.read_csv('Data/ham-spam.csv')
df = df.sample(frac=1, random_state=0)
df.head()

```

Используйте следующие операторы для удаления дубликатов строк из набора данных и проверки баланса:

```

df = df.drop_duplicates()
df.groupby('IsSpam').describe()

```

Далее извлеките электронные письма из столбца Text фрейма данных DataFrame и метки из столбца IsSpam. Затем используйте класс Tokenizer из Keras для токенизации текста и преобразования его в последовательности, а также `pad_sequences` для получения последовательностей одинаковой длины. Удалять стоп-слова не нужно, поскольку это не влияет на результат:

```

from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

```

```

x = df['Text']
y = df['IsSpam']

```

```

max_words = 10000 # Ограничение словарного запаса
                  # 10 000 наиболее употребительных слов
max_length = 500

```

```
tokenizer = Tokenizer(num_words=max_words)
tokenizer.fit_on_texts(x)
sequences = tokenizer.texts_to_sequences(x)
x = pad_sequences(sequences, maxlen=max_length)
```

Определите модель бинарной классификации, содержащую эмбединговый слой с 32 измерениями, упрощающий слой для сглаживания выходных данных эмбедингового слоя, плотный слой для классификации и выходной слой с одним нейроном и сигмоидной активацией:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Embedding
```

```
model = Sequential()
model.add(Embedding(max_words, 32, input_length=max_length))
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
model.summary()
```

Обучите сеть и позвольте Keras использовать 20% обучающих образцов для валидации:

```
hist = model.fit(x, y, validation_split=0.2, epochs=5, batch_size=20)
```

Используйте возвращаемый методом fit объект history, чтобы построить графики точности обучения и валидации в каждой эпохе:

```
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
sns.set()
```

```
acc = hist.history['accuracy']
val = hist.history['val_accuracy']
epochs = range(1, len(acc) + 1)
```

```
plt.plot(epochs, acc, '-', label='Точность обучения')
plt.plot(epochs, val, ':', label='Точность валидации')
plt.title('Точность обучения и валидации')
plt.xlabel('Эпоха')
plt.ylabel('Точность')
plt.legend(loc='lower right')
plt.plot()
```

Надеюсь, сеть достигла точности проверки, превышающей 95%. Если это не так, обучите ее снова. Вот как это получилось у меня (рис. 13.2).

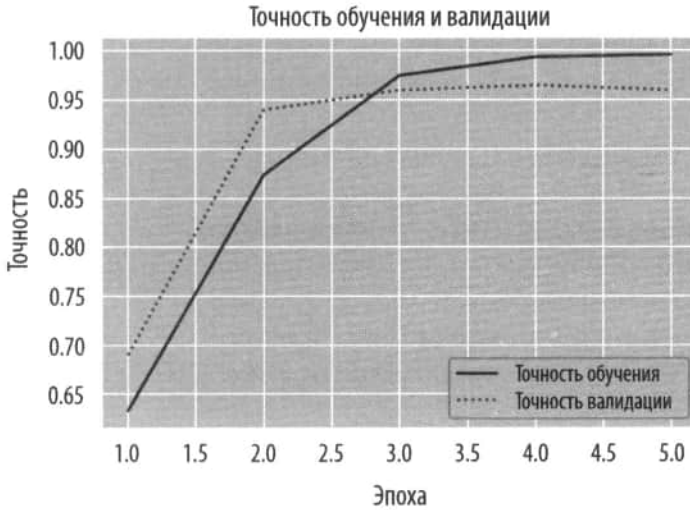


Рис. 13.2. Графики точности обучения и валидации модели нейронной сети для отделения спама от нормальных писем

Когда вы убедитесь в точности, используйте следующие операторы для вычисления вероятности того, что связанное с обзором кода письмо является спамом:

```
text = 'Can you attend a code review on Tuesday? ' \
      'Need to make sure the logic is rock solid.'
```

```
sequence = tokenizer.texts_to_sequences([text])
padded_sequence = pad_sequences(sequence, maxlen=max_length)
model.predict(padded_sequence)[0][0]
```

Затем проделайте то же самое с другим письмом:

```
text = 'Why pay more for expensive meds when ' \
      'you can order them online and save $$$?'
```

```
sequence = tokenizer.texts_to_sequences([text])
padded_sequence = pad_sequences(sequence, maxlen=max_length)
model.predict(padded_sequence)[0][0]
```

Что сеть предсказала для первого письма? А что насчет второго? Согласны ли вы с прогнозами? Помните, что число, близкое к 0,0, означает, что письмо не является спамом, а число, близкое к 1,0, — что является.

Автоматизация векторизации текста

Вместо того чтобы запускать `Tokenizer` и `pad_sequences` вручную, можно предварить эмбединговый слой слоем `TextVectorization` (<https://oreil.ly/1hDHa>). Вот пример:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten, Embedding
```

```

from tensorflow.keras.layers import TextVectorization, InputLayer
import tensorflow as tf

model = Sequential()
model.add(InputLayer(input_shape=(1,), dtype=tf.string))
model.add(TextVectorization(max_tokens=max_words, output_sequence_length=max_length))
model.add(Embedding(max_words, 32, input_length=max_length))
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
model.summary()

```

Обратите внимание, что входной слой (экземпляр `InputLayer`, <https://oreil.ly/uZ5ic>) теперь явно определен, и он настроен на прием строковых данных. Кроме того, перед обучением модели слой `TextVectorization` необходимо подогнать под входные данные путем вызова оператора `adapt` (<https://oreil.ly/BxRAa>):

```
model.layers[0].adapt(x)
```

Теперь больше не нужно предварительно обрабатывать обучающий текст и можно передавать необработанные текстовые строки в `predict`:

```

text = 'Why pay more for expensive meds when ' \
      'you can order them online and save $$$?'
model.predict([text])[0][0]

```

`TextVectorization` не удаляет стоп-слова, поэтому, если необходимо удалить их, можно проделать это отдельно или использовать параметр `standardize` слоя `TextVectorization` для определения функции обратного вызова, которая сделает это за вас.

Использование `TextVectorization` в модели анализа настроений

Для того чтобы продемонстрировать, как слои `TextVectorization` упрощают обработку текста, давайте используем ее для построения бинарного классификатора, выполняющего анализ настроений. Мы возьмем тот же набор данных, что и в главе 4: набор данных отзывов IMDB (<https://oreil.ly/tfjoP>), содержащий 25 000 положительных и 25 000 отрицательных отзывов.

Скачайте набор данных, если вы этого еще не сделали, и поместите его в подкаталог `Data` вашего блокнота Jupyter. Затем создайте новый блокнот и используйте следующие операторы для загрузки и перемешивания набора данных:

```
import pandas as pd
```

```

df = pd.read_csv('Data/reviews.csv', encoding="ISO-8859-1")
df = df.sample(frac=1, random_state=0)
df.head()

```

Удалите дублирующиеся строки и проверьте сбалансированность:

```
df = df.drop_duplicates()
df.groupby('Sentiment').describe()
```

Теперь создайте модель и включите в нее слой `TextVectorization` для предварительной обработки входного текста:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import TextVectorization, InputLayer
from tensorflow.keras.layers import Dense, Flatten, Embedding
import tensorflow as tf
```

```
max_words = 20000
max_length = 500
```

```
model = Sequential()
model.add(InputLayer(input_shape=(1,), dtype=tf.string))
model.add(TextVectorization(max_tokens=max_words,
                             output_sequence_length=max_length))
model.add(Embedding(max_words, 32, input_length=max_length))
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])
model.summary()
```

Извлеките отзывы из столбца `Text` фрейма данных `DataFrame` и метки (0 для негативного настроения, 1 — для положительного) из столбца `Sentiment` и используйте первые для подгонки слоя `TextVectorization` к тексту. Затем обучите модель:

```
x = df['Text']
y = df['Sentiment']
model.layers[0].adapt(x)
```

```
hist = model.fit(x, y, validation_split=0.5, epochs=5, batch_size=250)
```

После завершения обучения постройте графики точности обучения и валидации:

```
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
sns.set()
```

```
acc = hist.history['accuracy']
val = hist.history['val_accuracy']
epochs = range(1, len(acc) + 1)
```

```
plt.plot(epochs, acc, '-', label='Точность обучения')
plt.plot(epochs, val, ':', label='Точность валидации')
plt.title('Точность обучения и валидации')
plt.xlabel('Эпоха')
plt.ylabel('Точность')
plt.legend(loc='lower right')
plt.plot()
```

Модель очень хорошо подходит к обучающему тексту, но точность проверки обычно достигает 85–90% (рис. 13.3).

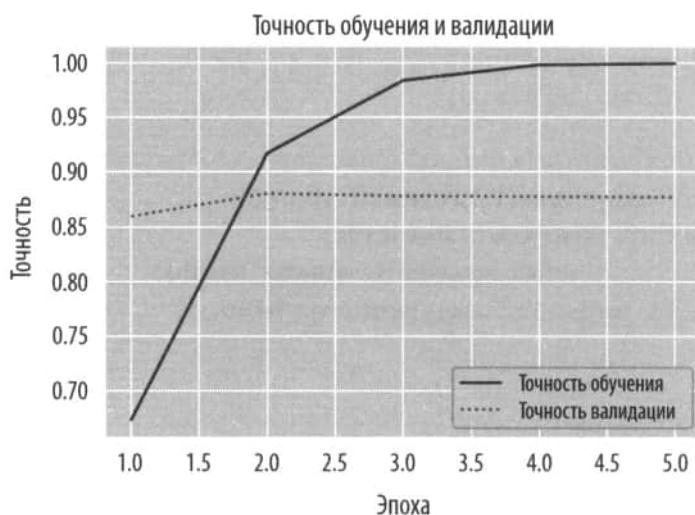


Рис. 13.3. Графики точности обучения и валидации бинарного классификатора, выполняющего анализ настроений

Используйте модель для оценки положительного комментария по настроению:

```
text = 'Excellent food and fantastic service!'
```

```
# Превосходная еда и фантастическое обслуживание!
```

```
model.predict([text])[0][0]
```

Теперь сделайте то же самое с негативным комментарием:

```
text = 'The long lines and poor customer service really turned me off'
```

```
# Мне не понравились длинные очереди и плохое обслуживание клиентов
```

```
model.predict([text])[0][0]
```

Обратите внимание, насколько упростился код. Эксплуатация модели также упрощается, поскольку вам больше не нужен `Tokenizer`, настраиваемый на обучающие данные для подготовки текста, передаваемого в модель для прогнозирования. Однако имейте в виду, что модель со слоем векторизации текста нельзя сохранить в используемом в Keras формате H5. Ее можно сохранить в формате `SavedModel` (TensorFlow).

Следующий оператор сохраняет модель в подкаталоге `saved_model` текущего каталога:

```
model.save('saved_model')
```

Как только модель будет перезагружена, ей можно передавать текст для составления прогнозов.

Учет порядка слов в прогнозах

Обе модели, которые вы только что построили, представляют собой модели *мешка слов*, игнорирующие порядок слов. Такие модели широко распространены и зачастую являются более точными, чем модели других типов. Но это не всегда так. Относительное расположение слов в предложении иногда имеет значение. Слова "*кредит*" и "*карта*", вероятно, должны влиять на классификатор спама одним образом, если они встречаются в предложении далеко друг от друга, и другим, если они встречаются вместе.

Один из способов улучшения — или, по крайней мере, *попытки* улучшения — простой модели мешка слов — это использование n -грамм, как описано в *главе 4*. N -грамма — это набор из n слов, идущих в последовательном порядке. Класс `TextVectorization` в Keras имеет параметр `ngrams`, который упрощает применение n -грамм. Следующий оператор создает слой `TextVectorization`, учитывающий как пары слов, так и отдельные слова:

```
model.add(TextVectorization(max_tokens=max_words,
                             output_sequence_length=max_length,
                             ngrams=2))
```

Одно из ограничений n -грамм заключается в том, что они учитывают только те слова, которые непосредственно примыкают друг к другу. Несколько более надежный способ учесть положение слов в задаче классификации — заменить плотные слои слоями Conv1D (<https://oreil.ly/4h0M8>) и MaxPooling1D (<https://oreil.ly/2KMId>), превратив сеть в сверточную нейронную сеть. CNN чаще всего используются для классификации изображений, но одномерные сверточные слои хорошо работают и с текстовыми последовательностями. Вот сеть, представленная в предыдущем примере, переделанная в CNN:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv1D, MaxPooling1D, GlobalMaxPooling1D
from tensorflow.keras.layers import TextVectorization, InputLayer
from tensorflow.keras.layers import Dense, Embedding
import tensorflow as tf
```

[illegible]


```

model.add(Embedding(max_words, 32, input_length=max_length))
model.add(Conv1D(32, 7, activation='relu'))
model.add(MaxPooling1D(5))
model.add(Conv1D(32, 7, activation='relu'))
model.add(GlobalMaxPooling1D())
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy', optimizer='adam',
              metrics=['accuracy'])

```

Вместо того чтобы обрабатывать отдельные слова, слои Conv1D в этом примере извлекают признаки из групп представляющих слова векторов (семь в первом слое и еще семь во втором), подобно тому как слои Conv2D извлекают признаки из блоков пикселей. Слой MaxPooling1D сжимает выходной сигнал первого слоя Conv1D, чтобы выявить более высокоуровневую структуру входных последовательностей, подобно тому, как уменьшение разрешения изображения подчеркивает макрохарактеристики, такие как форма тела человека, в то же время минимизируя или отфильтровывая совсем незначительные характеристики, такие как разрез его глаз. Простая CNN, подобная этой, иногда классифицирует текст точнее, чем модели мешка слов, а иногда — нет. Как это часто бывает в машинном обучении, единственный способ узнать — это попробовать.

Рекуррентные нейронные сети (RNN)

Еще один способ учесть позицию слова в классификаторе — включить в сеть рекуррентные слои. Они изначально были придуманы для обработки данных временных рядов, например для просмотра погодных данных за последние пять дней и предсказания того, какой будет максимальная температура завтра. Если бы вы просто взяли все погодные данные за эти пять дней и рассматривали каждый день независимо, очевидные в данных тенденции были бы потеряны. Однако рекуррентный слой может обнаружить эти тенденции и учесть их в своем выводе. Выводимая эмбединговым слоем последовательность векторов может считаться временным рядом, поскольку слова во фразе упорядочены последовательно, и слова, используемые в начале фразы, могут повлиять на интерпретацию слов, встречающихся позже.

На рис. 13.4 показано, как рекуррентный слой преобразует эмбединги слов в вектор, на который влияет порядок слов. В этом примере последовательности поступают на вход эмбедингового слоя, преобразующего каждое слово (токен) последовательности в вектор чисел с плавающей точкой. Эти эмбединги слов поступают в рекуррентный слой, в результате чего образуется другой выходной вектор. Для того чтобы вычислить этот вектор, ячейки рекуррентного слоя в цикле перебирают составляющие последовательность эмбединговые слова. На входе $n + 1$ итерации цикла находятся текущий эмбединговый вектор и выход итерации n — так называемое *скрытое состояние*. Выход рекуррентного слоя — это выход последней итерации цикла. Результат отличается от того, каким он был бы, если бы каж-

дый эмбединговый вектор обрабатывался независимо, потому что каждая итерация использует информацию из предыдущей итерации для вычисления выхода. Контекст из слова в начале последовательности может переноситься на слова, которые встречаются позже.

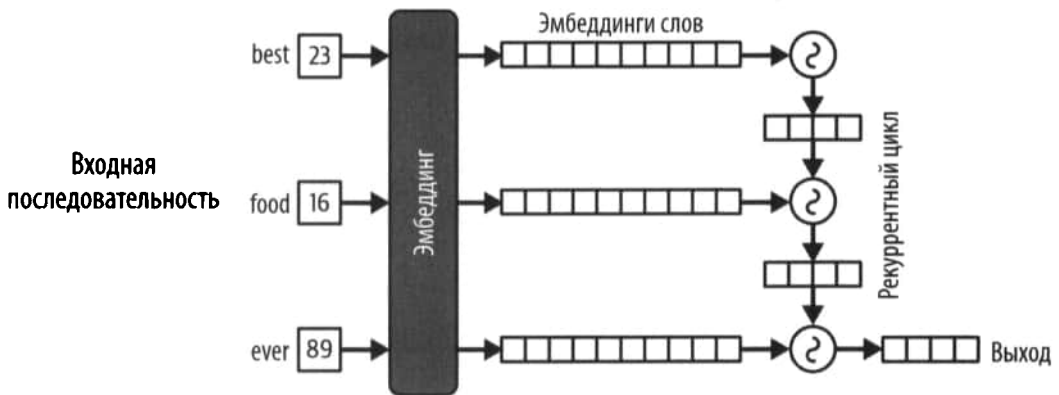


Рис. 13.4. Обработка эмбединговых слоев слов с помощью рекуррентного слоя

Нетрудно вручную построить очень простой рекуррентный слой, который будет достаточно хорошо работать с короткими последовательностями (скажем, из четырех или пяти слов), но более длинные последовательности будут страдать от эффекта исчезающего градиента, означающего, что расположенные далеко друг от друга слова будут оказывать незначительное влияние друг на друга. Одно из решений — рекуррентный слой, состоящий из ячеек Long Short-Term Memory (LSTM, долгая краткосрочная память), которые были представлены в 1997 году в статье, озаглавленной так же — "Долгая краткосрочная память" (<https://oreil.ly/W7vrb>).

Ячейки LSTM сами по себе являются миниатюрными нейронными сетями. По мере того как клетки перебирают слова в последовательности, они узнают (точно так же, как узнают веса, соединяющие нейроны в плотных слоях), какие слова важны, и отдают этой информации приоритет в последующих итерациях цикла. Плотный слой не распознает связь между синим и небом во фразе "Я люблю синий, потому что в ясный и солнечный день это цвет неба". Слой LSTM распознает и может учесть это в своем выводе. Однако его мощность уменьшается по мере удаления слов друг от друга.

Keras предоставляет удобную реализацию LSTM-слоев в своем классе LSTM (<https://oreil.ly/Q6zlb>). Библиотека также предоставляет класс GRU (<https://oreil.ly/LtHDE>), реализующий слои с управляемыми рекуррентными блоками (gated recurrent unit, GRU). GRU представляют собой упрощенные LSTM-слои, которые обучаются быстрее и часто дают результаты, равные или превосходящие результаты LSTM-слоев. Вот как выглядел бы наш классификатор спама, если бы он был модифицирован для использования LSTM:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import TextVectorization, InputLayer
```

```

from tensorflow.keras.layers import Dense, Embedding, LSTM
import tensorflow as tf

model = Sequential()
model.add(InputLayer(input_shape=(1,), dtype=tf.string))
model.add(TextVectorization(max_tokens=max_words,
                             output_sequence_length=max_length))
model.add(Embedding(max_words, 32, input_length=max_length))
model.add(LSTM(32))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy', optimizer='adam',
              metrics=['accuracy'])

```

LSTM требуют значительных вычислительных затрат, особенно при работе с длинными последовательностями. Если вы попробуете этот код, то обнаружите, что обучение модели занимает *гораздо* больше времени. Также вы увидите, что точность проверки улучшается незначительно, если вообще улучшается. Это связано с тем, что модели типа "мешок слов", несмотря на свою простоту, обычно эффективны при классификации текста.

Слои LSTM можно складывать, как и плотные слои. Хитрость заключается в том, чтобы включить атрибут `return_sequences=True` во все слои LSTM, кроме последнего, чтобы предыдущие слои возвращали все сгенерированные векторы (все скрытые состояния), а не только конечный вектор. Когда-то сервис Google Translate использовал два стека LSTM глубиной в восемь слоев для кодирования фраз на одном языке и декодирования их на другой.

Использование предварительно обученных моделей для классификации текста

Если обучение собственной модели анализа настроения не дает требуемой точности, вы всегда можете обратиться к предварительно обученной модели. Подобно тому, как существуют предварительно обученные на миллионах помеченных изображений модели компьютерного зрения, существуют предварительно обученные модели анализа настроения, которые были подготовлены на миллионах помеченных образцов текста. Keras не предоставляет удобную обертку для таких моделей, как для предварительно обученных CNN, но, тем не менее, их относительно легко использовать.

Много таких моделей можно найти у Hugging Face (<https://oreil.ly/ly7sQ>), компании, ориентированной на искусственный интеллект, целью которой является развитие и демократизация искусственного интеллекта. Изначально Hugging Face специализировалась на моделях NLP, но затем расширила свою библиотеку, включив в нее модели других типов, в том числе модели классификации изображений (<https://oreil.ly/2ZQFT>) и модели обнаружения объектов (<https://oreil.ly/A5i4m>). В

настоящее время Hugging Face содержит более 400 моделей анализа настроения (<https://oreil.ly/omrnl>), обученных на различных типах входных данных — от твитов до отзывов о товарах на разных языках. Есть даже модели, анализирующие текст на предмет радости, гнева, удивления и других эмоций.

Все эти модели бесплатны для использования. Хотите попробовать? Начните с установки пакета Transformers (<https://oreil.ly/NoiNp>) от Hugging Face в среду Python (например, `pip install transformers`). Затем запустите блокнот Jupyter и используйте следующий код для загрузки стандартной модели анализа настроения Hugging Face:

```
from transformers import pipeline
```

```
model = pipeline('sentiment-analysis')
```

При первой загрузке модели возникнет небольшая задержка. Как только завершите загрузку, оцените это предложение:

```
model('The long lines and poor customer service really turned me off')
```

Ниже приведен результат:

```
[{'label': 'NEGATIVE', 'score': 0.9995430707931519}]
```

Попробуйте использовать положительный комментарий:

```
model('Great food and excellent service!')
```

Ниже приведен результат:

```
[{'label': 'POSITIVE', 'score': 0.9998843669891357}]
```

В возвращаемом значении `label` указывает, является ли настроение позитивным или негативным, а `score` показывает уверенность модели в метке.

Проанализировать текстовую строку на предмет эмоций так же просто, загрузив другую предварительно обученную модель. Для того чтобы убедиться в этом, сделайте следующее:

```
model = pipeline('text-classification',
                  model='bhadresh-savani/distilbert-base-uncased-emotion',
                  return_all_scores=True)
```

```
model('The long lines and poor customer service really turned me off')
```

Вот какой результат я получил:

```
[{'label': 'sadness', 'score': 0.10837080329656601},      # печаль
 {'label': 'joy', 'score': 0.002373947761952877},       # радость
 {'label': 'love', 'score': 0.0006029471987858415},     # любовь
 {'label': 'anger', 'score': 0.8861245512962341},      # гнев
 {'label': 'fear', 'score': 0.0019340706057846546},    # страх
 {'label': 'surprise', 'score': 0.0005936296074651182}]] # удивление
```

Проще не бывает! И, как вы увидите через некоторое время, предварительно обученные модели Hugging Face могут использоваться не только для классификации текстов.

Нейронный машинный перевод

Во вселенной обработки естественного языка классификация текстов — относительно простая задача. На противоположном конце спектра находится *нейромашинный перевод* (neural machine translation, NMT), использующий глубокое обучение для перевода текста с одного языка на другой. NMT доказал свое превосходство над системами машинного перевода на основе правил (rules-based machine translation, RBMT; <https://oreil.ly/V6IIIG>) и статистического машинного перевода (statistical machine translation, SMT, <https://oreil.ly/vKhKW>), которые появились до бурного развития глубокого обучения, и сегодня является основой практически всех современных сервисов перевода текстов.

Суть классификации текстов заключается в том, что вы преобразуете входную последовательность в вектор, характеризующий эту последовательность, а затем передаете этот вектор в классификатор. Существует несколько способов создания этого вектора. Можно преобразовать двумерный вывод эмбедингового слоя в одномерный вектор, а можно направить этот вывод в рекуррентный слой или сверточный слой в надежде получить вектор, в большей степени учитывающий контекст. Какой бы путь вы ни выбрали, цель проста: преобразовать строку текста в массив чисел с плавающей точкой, которые уникально описывают ее, и использовать выходной слой на основе сигмоидной функции или слой softmax для классификации.

NMT — это, по сути, расширение классификации текста. Вы начинаете с преобразования текстовой последовательности в вектор. Но вместо того чтобы классифицировать вектор, вы используете его для создания новой последовательности. Один из способов сделать это — использовать кодировщик-декодировщик (кодер-декодер) LSTM.

Кодировщик-декодировщик LSTM

Еще несколько лет назад большинство моделей NMT, включая модель, лежащую в основе Google Translate, представляли собой модели "последовательность — последовательность" (sequence-to-sequence, <https://oreil.ly/5rvCF>) на основе LSTM, подобные той, что показана на рис. 13.5. В таких моделях один или несколько слоев LSTM кодируют токенизированную входную последовательность, представляющую фразу для перевода, в вектор. Второй набор рекуррентных слоев использует этот вектор в качестве входных данных и декодирует его в токенизированную фразу на другом языке. Модель принимает последовательности в качестве входных данных и возвращает последовательности в качестве выходных, отсюда и термин "модель „последовательность — последовательность“". Выходной слой softmax в конце выдает набор вероятностей для каждого токена в выходной последовательности. Если максимальная поддерживаемая длина выходной фразы составляет, например, 20 токенов, а словарный запас языка вывода содержит 20 000 слов, то на выходе получается 20 наборов (по одному на токен) из 20 000 вероятностей. Слово,

выбранное для каждого выходного токена, — это слово, которому присвоена наибольшая вероятность.

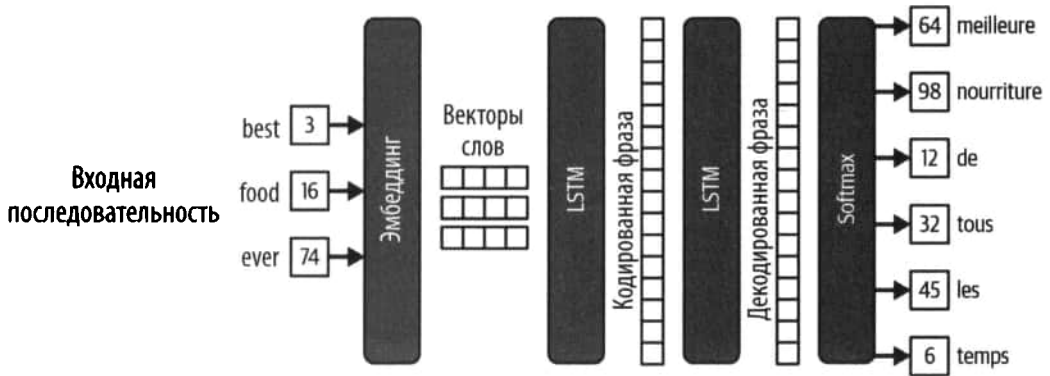


Рис. 13.5. Кодировщик-декодировщик на основе LSTM для нейронного машинного перевода

Основанные на технологии LSTM модели "последовательность — последовательность" относительно легко построить с помощью Keras и TensorFlow. В репозитории GitHub этой книги содержится блокнот (<https://oreil.ly/qieYB>), использующий 50 000 образцов для обучения LSTM-модели переводу с английского на французский. Модель определяется следующим образом:

```
model = Sequential()
model.add(Embedding(en_vocab_size, 256, input_length=en_max_len, mask_zero=True))
model.add(LSTM(256))
model.add(RepeatVector(fr_max_len))
model.add(LSTM(256, return_sequences=True))
model.add(Dropout(0.4))
model.add(TimeDistributed(Dense(fr_vocab_size, activation='softmax')))
model.compile(loss='sparse_categorical_crossentropy', optimizer='adam',
              metrics=['accuracy'])model.summary(line_length=100)
```

Первый слой — это эмбе́ддинговый слой, который преобразует словесные токены в векторы, содержащие 256 значений с плавающей точкой. Параметр `mask_zero=True` указывает, что нули во входных последовательностях обозначают вставку, так что следующий слой может их игнорировать (в итоге нет необходимости переводить эти токены). Далее следует слой LSTM, кодирующий поступающие в модель английские фразы. Второй слой LSTM декодирует фразы в плотные векторы, представляющие французские эквиваленты. Между ними находится слой `RepeatVector` (<https://oreil.ly/3szHO>), который перестраивает выходные данные первого LSTM-слоя для ввода во второй путем повторения выходных данных заданное количество раз. Последний слой — это слой классификации `softmax`, выводящий вероятности для каждого слова из французского словаря. Обертка `TimeDistributed` (<https://oreil.ly/Da2jW>) гарантирует, что модель выводит набор вероятностей для каждого токена в выходных данных, а не только один набор для всей последовательности.

После 34 эпох обучения модель перевела 10 тестовых фраз таким образом:

its fall now => cest maintenant maintenant
 im losing => je suis en train
 it was quite funny => cetait fut amusant amusant
 thats not unusual => ce nest pas inhabituel
 i think ill do that => je pense que je le
 tom looks different => tom a lair different
 its worth a try => ca vaut le coup
 fortune smiled on him => la la lui a souri
 lets hit the road => taillons la
 i love winning => jadore gagner

Модель не идеальна, отчасти из-за ограниченного размера обучающего набора. Настоящие модели NMT обучаются на сотнях миллионов или даже миллиардах фраз (<https://oreil.ly/bOZOK>). Но является ли эта модель действительно репрезентативной для моделей, используемых с целью современного перевода текста? Рисунок 13.6 адаптирован из изображения в документе 2016 года (<https://oreil.ly/gXD9A>), написанном инженерами Google и документирующем архитектуру Google Translate. Архитектура очень похожа на архитектуру только что представленной модели. Она более глубокая, с восемью слоями LSTM в кодировщике и восемью — в декодировщике. В ней также используются остаточные связи между слоями для поддержания большей глубины сети.

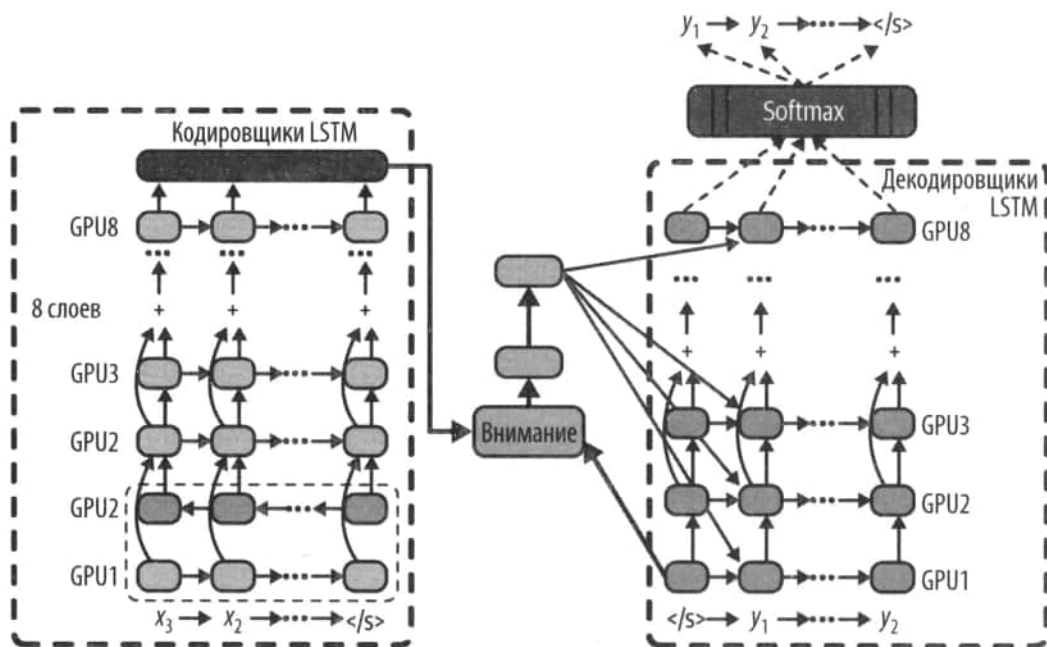


Рис. 13.6. Google Translate приблизительно в 2016 году

Одно из отличий нашей модели от модели, описанной в статье, — блок с меткой "Внимание" (Attention) между кодировщиком и декодировщиком. В глубоком обучении *внимание* — это механизм, позволяющий сфокусировать внимание модели на наиболее важных частях фразы, признавая, что одно слово может иметь разное значение в различных контекстах и значение слова иногда зависит от того, что находится вокруг него.

Внимание было представлено в глубоком обучении в основополагающей статье 2014 года под названием "Нейронный машинный перевод путем совместного обучения выравниванию и переводу" ("Neural Machine Translation by Jointly Learning to Align and Translate", <https://oreil.ly/moOX9>), но только несколько лет спустя внимание заняло ведущую позицию в качестве способа *заменить* слои LSTM, а не дополнить их. Появилась, возможно, самая значительная на сегодняшний день разработка в области NMT и NLP в целом: модель трансформера.

Трансформерные кодировщики-декодировщики

Знаковая статья 2017 года под названием "Внимание — это все, что вам нужно" ("Attention Is All You Need", <https://oreil.ly/OUfbX>) изменила подход специалистов по обработке данных к NMT и другим задачам нейронной обработки текста. В ней был предложен лучший способ обработки последовательности, основанный на моделях трансформеров, в которых отказываются от рекуррентных слоев и применяют механизмы внимания для моделирования используемого контекста слов. Сегодня модели трансформеров практически полностью вытеснили модели на основе LSTM.

Рисунок 13.7 адаптирован из изображения в вышеупомянутой статье. Слева — кодировщик, принимающий на вход текстовые последовательности и генерирующий плотные векторные представления этих последовательностей. Справа — декодировщик, преобразующий плотные векторные представления входных последовательностей в выходные последовательности. На высоком уровне модель трансформера использует ту же архитектуру кодировщика-декодировщика, что и модель на основе LSTM. Разница заключается в том, как она выполняет кодирование и декодирование.

Главным новшеством модели трансформера является использование слоев *многоголового внимания* (multi-head attention, MHA) вместо слоев LSTM. Слои MHA воплощают концепцию *самовнимания*, позволяющую модели анализировать входную последовательность и фокусироваться на наиболее важных словах, а также на контексте, в котором эти слова используются. Например, в предложении "We took a walk in the park" ("Мы прогулялись по парку") слово *park* (парк) имеет другое значение, чем в предложении "Where did you park the car?" ("Где вы припарковали машину?"). Эмбединговый слой хранит одно векторное представление для слова *park*, но в модели трансформера слой MHA изменяет выведенный эмбединговым слоем вектор, так что в двух предложениях слово *park* представлено двумя разными векторами. Неудивительно, что значения, используемые для придания эмбединговым векторам контекстной осведомленности, узнаются в процессе обучения.

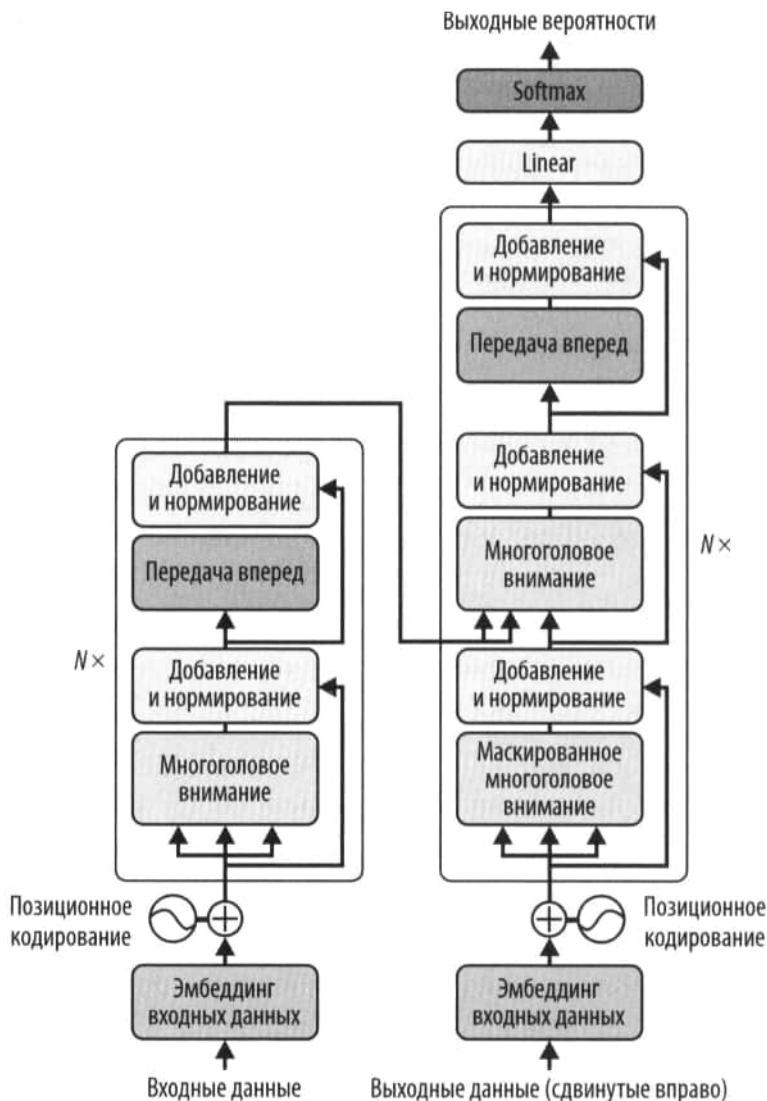


Рис. 13.7. Модель трансформера



Как работает самовнимание? Суть в том, что слой МНА использует точечные произведения для вычисления оценок сходства для каждой пары слов в последовательности. После нормализации оценок он применяет их для вычисления взвешенных версий каждого эмбединга слов в последовательности. Затем он снова изменяет их, используя веса, полученные в процессе обучения.

Первая часть *multi* в словосочетании *multi-head attention* означает, что слой МНА обучается нескольким наборам весов, а не одному, в отличие от сверточных слоев в CNN. Это дает МНА возможность различать контекст в длинных последовательностях, где слова могут иметь множество связей друг с другом.

МНА также предоставляет дополнительный контекст для слов, ссылающихся на другие слова. Эмбединговый слой представляет местоимение *it* одним вектором, а слой МНА помогает модели понять, что в предложении "I love my car because it is fast" ("Я люблю свою машину, потому что она быстрая") *it* относится к автомобилю. Он также добавляет вес слову *fast*, поскольку оно имеет решающее значение для смысла предложения. Без него непонятно, *почему* вы любите свою машину.

В отличие от слоев LSTM, способность слоев МНА моделировать отношения между словами во входной последовательности не зависит от длины последовательности. Слои МНА также поддерживают параллельную работу и поэтому быстрее обучаются на нескольких GPU. Однако они не кодируют информацию о расположении слов во фразе. Для того чтобы компенсировать это, трансформер использует простое векторное сложение для добавления информации о положении слова в последовательности к каждому вектору, выводимому из эмбедингового слоя. Это называется *позиционным кодированием* или *позиционным эмбедингом*. На рис. 13.7 это обозначено знаками "плюс", помеченными как "Позиционное кодирование".

Трансформеры не ограничиваются нейронным машинным переводом; сегодня они используются практически во всех аспектах NLP. Кодировочная половина трансформера выдает плотные векторные представления входных последовательностей. Классификацию текста можно выполнить с помощью трансформера, а не отдельного эмбедингового слоя для кодирования входных последовательностей. Построенные таким образом модели часто превосходят модели типа "мешок слов", особенно если отношение количества образцов к длине образца (количество обучающих образцов, деленное на среднюю длину каждого образца) превышает 1500. Эта так называемая золотая константа была обнаружена группой исследователей из Google и задокументирована в руководстве по классификации текстов (<https://oreil.ly/MSxf5>).

Построение NMT-модели на основе трансформера

Keras предоставляет некоторые, но не все строительные блоки, из которых состоит сквозной трансформер. Например, в классе `MultiHeadAttention` (<https://oreil.ly/OgTvn>) удобно реализованы эмбединговые слои, но не реализован позиционный эмбединг. Однако для этого существует отдельный пакет `KerasNLP` (<https://oreil.ly/Wx58Y>). Среди прочего, он включает следующие классы, представляющие слои в сети на основе трансформеров.

`TransformerEncoder` (<https://oreil.ly/CFsaO>).

Представляет кодировщик трансформера.

`TransformerDecoder` (<https://oreil.ly/vhL4m>).

Представляет декодировщик трансформера.

`TokenAndPositionEmbedding` (<https://oreil.ly/nxCRU>).

Реализует эмбединговый слой, поддерживающий позиционный эмбединг.

С помощью указанных классов модели NMT на основе трансформеров строить относительно просто. Можно продемонстрировать это на примере создания переводчика с английского на французский. Начните с установки KerasNLP, если он еще не установлен. Затем загрузите en-fr.txt (<https://oreil.ly/K4mTF>) — файл данных, содержащий 50 000 английских фраз и их французские эквиваленты, — и поместите его в подкаталог Data, где находятся ваши блокноты Jupyter. Файл en-fr.txt является подмножеством большего файла (<https://oreil.ly/WZDMQ>), содержащего более 190 000 фраз и соответствующих им переводов, составленного в рамках проекта Tatoeba (<https://oreil.ly/WHT6F>). Разделителем в файле является табуляция. Каждая строка содержит английскую фразу, эквивалентную французскую фразу, и атрибут, указывающий источник перевода. Нам не нужны атрибуты, поэтому загрузите набор данных в DataFrame, удалите столбец с атрибутами, перетасуйте и переиндексируйте строки:

```
import pandas as pd

df = pd.read_csv('Data/en-fr.txt', names=['en', 'fr', 'attr'],
                usecols=['en', 'fr'], sep='\t')
df = df.sample(frac=1, random_state=42)
df = df.reset_index(drop=True)
df.head()
```

На рис. 13.8 приведен результат.

	en	fr
0	You're very clever.	Vous êtes fort ingénieuse.
1	Are there kids?	Y a-t-il des enfants ?
2	Come in.	Entrez !
3	Where's Boston?	Où est Boston ?
4	You see what I mean?	Vous voyez ce que je veux dire ?

Рис. 13.8. Фрагмент файла en-fr.txt после предобработки

Перед тем как использовать набор данных для обучения модели, его необходимо очистить. С помощью следующих операторов удалите цифры и знаки препинания, преобразуйте слова с символами Юникода, например *où*, в их эквиваленты ASCII (*ou*), переведите символы в нижний регистр и вставьте токены [start] и [end] в начало и конец каждой французской фразы:

```
import re
from unicodedata import normalize

def clean_text(text):
    text = normalize('NFD', text.lower())
    text = re.sub('[^A-Za-z ]+', '', text)
    return text
```

```
def clean_and_prepare_text(text):
    text = '[start] ' + clean_text(text) + ' [end]'
    return text

df['en'] = df['en'].apply(lambda row: clean_text(row))
df['fr'] = df['fr'].apply(lambda row: clean_and_prepare_text(row))
df.head()
```

После этого результат выглядит немного чище (рис. 13.9).

	en	fr
0	youre very clever	[start] vous etes fort ingenieuse [end]
1	are there kids	[start] y atil des enfants [end]
2	come in	[start] entrez [end]
3	wheres boston	[start] ou est boston [end]
4	you see what i mean	[start] vous voyez ce que je veux dire [end]

Рис. 13.9. Фрагмент файла en-fr.txt после очистки

Следующий шаг — сканирование набора данных и определение максимальной длины английских и французских фраз. Эти длины будут определять длины последовательностей, подаваемых на вход и получаемых на выходе из модели:

```
en = df['en']
fr = df['fr']
en_max_len = max(len(line.split()) for line in en)
fr_max_len = max(len(line.split()) for line in fr)
sequence_len = max(en_max_len, fr_max_len)
print(f'Max phrase length (English): {en_max_len}')
print(f'Max phrase length (French): {fr_max_len}')
print(f'Sequence length: {sequence_len}')
```

В этом примере самая длинная английская фраза содержит семь слов, а самая длинная французская — 16 (включая токены [start] и [end]). Модель сможет переводить английские фразы длиной до семи слов во французские фразы длиной до четырнадцати слов.

Теперь настройте с `fit` один `Tokenizer` на английские фразы и другой `Tokenizer` на их французские эквиваленты и сгенерируйте последовательности со вставками из всех фраз. Обратите внимание на параметр `filters`, переданный французскому токенизатору. Он настраивает токенизатор на удаление всех знаков препинания, которые он обычно удаляет, *за исключением* квадратных скобок, используемых для разделения лексем [start] и [end]:

```
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
```

```

en_tokenizer = Tokenizer()
en_tokenizer.fit_on_texts(en)
en_sequences = en_tokenizer.texts_to_sequences(en)
en_x = pad_sequences(en_sequences, maxlen=sequence_len, padding='post')

fr_tokenizer = Tokenizer(filters='!"#$%&()*+,-./:;<=>?@\^_`{|}~\t\n')
fr_tokenizer.fit_on_texts(fr)
fr_sequences = fr_tokenizer.texts_to_sequences(fr)
fr_y = pad_sequences(fr_sequences, maxlen=sequence_len + 1, padding='post')

```

Затем рассчитайте размер словаря для каждого языка на основе экземпляров `Tokenizer`:

```

en_vocab_size = len(en_tokenizer.word_index) + 1
fr_vocab_size = len(fr_tokenizer.word_index) + 1

print(f'Vocabulary size (English): {en_vocab_size}')
print(f'Vocabulary size (French): {fr_vocab_size}')

```

В результате выяснилось, что словарный запас английского языка составляет 6033 слова, а французского — 12 197. Эти значения будут использованы для определения размеров двух эмбединговых слоев модели. Последние также будут использованы для размера выходного слоя.

Наконец, создайте признаки и метки, на которых будет обучаться модель. Признаки — это английские последовательности со вставками и французские последовательности со вставками за вычетом токенов [end]. Метки — это французские последовательности со вставками за вычетом токенов [start]. Упакуйте признаки в словарь, чтобы их можно было передать в модель, принимающую множество входных данных:

```

inputs = { 'encoder_input': en_x, 'decoder_input': fr_y[:, :-1] }
outputs = fr_y[:, 1:]

```

Теперь давайте определим модель. На этот раз мы воспользуемся функциональным API Keras, а не последовательным API. Это необходимо, потому что у данной модели два входа: один принимает токенизированную английскую фразу, а другой — токенизированную французскую фразу. Мы также запустим с определенным параметром используемые в Keras и TensorFlow генераторы случайных чисел, чтобы получить повторяющиеся результаты, по крайней мере на CPU. Это отклонение от всех других примеров в данной книге, но оно гарантирует, что при обучении модели вы получите те же результаты, что и я. Вот код:

```

import numpy as np
import tensorflow as tf
from tensorflow.keras import Model
from tensorflow.keras.layers import Input, Dense, Dropout
from keras_nlp.layers import TokenAndPositionEmbedding, TransformerEncoder
from keras_nlp.layers import TransformerDecoder

```

```

np.random.seed(42)
tf.random.set_seed(42)

num_heads = 8
embed_dim = 256

encoder_input = Input(shape=(None,), dtype='int64', name='encoder_input')
x = TokenAndPositionEmbedding(en_vocab_size, sequence_len, embed_dim)(encoder_input)
encoder_output = TransformerEncoder(embed_dim, num_heads)(x)
encoded_seq_input = Input(shape=(None, embed_dim))

decoder_input = Input(shape=(None,), dtype='int64', name='decoder_input')
x = TokenAndPositionEmbedding(fr_vocab_size, sequence_len, embed_dim,
                              mask_zero=True)(decoder_input)
x = TransformerDecoder(embed_dim, num_heads)(x, encoded_seq_input)
x = Dropout(0.4)(x)

decoder_output = Dense(fr_vocab_size, activation='softmax')(x)
decoder = Model([decoder_input, encoded_seq_input], decoder_output)
decoder_output = decoder([decoder_input, encoder_output])

model = Model([encoder_input, decoder_input], decoder_output)
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
model.summary(line_length=100)

```

Функциональный API Keras

Функциональный API — это более богатая версия последовательного API Keras. Помимо прочего, он позволяет создавать модели с несколькими входами или выходами, а также модели с общими слоями, как в сети предложения областей Faster R-CNN. Вот простой бинарный классификатор, созданный с помощью последовательного API:

```

model = Sequential()
model.add(Dense(128, activation='relu', input_dim=3))
model.add(Dense(1, activation='sigmoid'))

```

Вот та же сеть, созданная с помощью функционального API:

```

input = Input(shape=(3,))
hidden = Dense(128, activation='relu')(input)
output = Dense(1, activation='sigmoid')(hidden)
model = Model(inputs=input, outputs=output)

```

При создании модели вы указываете входы и выходы, а поскольку параметры `input` и `output` принимают списки Python, создать модель с несколькими входами и выходами не составит труда. Краткое введение в функциональный API и примеры, демонстрирующие расширенные возможности его использования, включая множественные входы и выходы и общие слои, можно найти в статье Джейсона Браунли "Как использовать функциональный API Keras для глубокого обучения" (Brownlee Jason. How to Use the Keras Functional API for Deep Learning; <https://oreil.ly/fpbgm>).

Модель разработана таким образом, чтобы она могла функционировать итеративно. Для перевода текста сначала передается английская фраза на английский вход и слово "[start]" на французский вход. Затем добавляется следующее предсказанное моделью французское слово к предыдущему французскому входу и снова вызывается модель, и этот процесс повторяется до тех пор, пока вся фраза не будет переведена, т. е. пока следующим словом, спрогнозированным моделью, не окажется "[end]".

На рис. 13.10 показана архитектура модели. Модель включает два эмбединговых слоя: один для английских и один для французских последовательностей. Оба преобразуют словарные токены в векторы из 256 значений с плавающей точкой каждый, и оба являются экземплярами класса `TokenAndPositionEmbedding` от `KerasNLP`, добавляющего позиционную информацию к эмбедингам слов. Выходные данные английского эмбедингового слоя проходят через кодировщик (экземпляр `TransformerEncoder`), после чего вместе с выходными данными французского эмбедингового слоя поступают на декодер, являющийся экземпляром `TransformerDecoder`. Декодер выдает вектор, представляющий следующий шаг в переводе, а выходной слой `softmax` преобразует этот вектор в набор вероятностей — по одной для каждого слова из французского словаря, — определяющих следующий токен. Во время обучения параметр `mask_zero=True`, передаваемый эмбединговому слою французского языка, ограничивает модель предсказаниями на основе токенов, предшествующих прогнозируемому. Другими словами, при наличии набора французских токенов, пронумерованных от 0 до n , модель обучается прогнозировать, каким будет токен $n + 1$, не заглядывая в $n + 1$ в обучающем тексте.

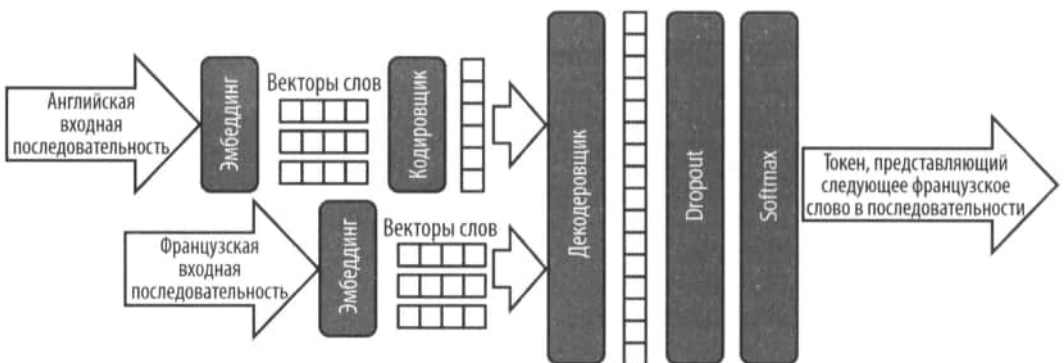


Рис. 13.10. Архитектура NMT на основе трансформера

Теперь вызовите `fit` для обучения модели и используйте обратный вызов `EarlyStopping` для завершения обучения, если точность валидации не улучшается в течение трех последовательных эпох:

```
from tensorflow.keras.callbacks import EarlyStopping

callback = EarlyStopping(monitor='val_accuracy', patience=3,
                        restore_best_weights=True)
hist = model.fit(inputs, outputs, epochs=50, validation_split=0.2,
                callbacks=[callback])
```

Обучение обычно занимает от двух до трех минут на эпоху на CPU. Если у вас нет GPU, а обучение идет слишком медленно, я рекомендую запустить код в Google Colab (<https://oreil.ly/J1AgM>). (Обязательно зайдите в **Notebook settings** (Настройки блокнота) в меню **Edit** (Правка) и выберите GPU в качестве аппаратного ускорителя.) Когда обучение будет завершено, постройте графики точности обучения и валидации в каждой эпохе и проследите, как последняя неуклонно растет, пока не выровняется:

```
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
sns.set()

acc = hist.history['accuracy']
val = hist.history['val_accuracy']
epochs = range(1, len(acc) + 1)

plt.plot(epochs, acc, '-', label='Точность обучения')
plt.plot(epochs, val, ':', label='Точность валидации')
plt.title('Точность обучения и валидации')
plt.xlabel('Эпоха')
plt.ylabel('Точность')
plt.legend(loc='lower right')
plt.plot()
```

Результат должен выглядеть примерно так, как показано на рис. 13.11.

На графике видно, что по истечении 14 эпох модель с точностью около 75% перевела английские образцы в валидационных данных на французский язык. Это не очень надежная мера точности, поскольку она буквально сравнивает каждое слово в прогнозируемом тексте с каждым словом в целевом тексте и не учитывает тот факт, что пропущенный или неправильно поставленный артикль, например *le* (эквивалентный *the*), не обязательно означает плохой перевод. Точность моделей NMT обычно измеряется с помощью баллов *дублера двуязычной оценки* (bilingual evaluation understudy, BLEU; <https://oreil.ly/hdxar>). Показатели BLEU легко вычисляются

после завершения обучения с помощью таких пакетов, как NLTK, но во время обучения точность валидации является разумной метрикой, позволяющей определить, когда следует прекратить обучение.

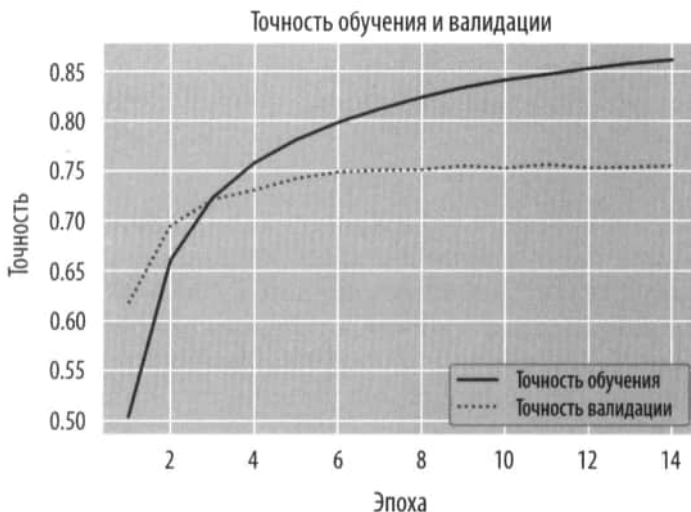


Рис. 13.11. Графики точности обучения и валидации модели перевода текста

Может ли модель действительно переводить текст с английского языка на французский? Используйте следующий код, чтобы определить функцию, принимающую английскую фразу и возвращающую французскую фразу. Затем вызовите ее на 10 фразах, использованных для проверки модели во время обучения, и убедитесь в этом сами. Обратите внимание, что один вызов `translate_text` приводит к нескольким обращениям к модели. Например, чтобы перевести "hello world", `translate_text` вызывает модель с входными данными "hello world" и "[start]". Предполагая, что модель прогнозирует, что следующим словом будет *salut*, `translate_text` снова обращается к ней с входными данными "hello world" и "[start] salut". Этот цикл повторяется до тех пор, пока следующим словом, предсказанным моделью, не будет "[end]", означающее конец перевода.

```
def translate_text(text, model, en_tokenizer, fr_tokenizer,
                  fr_index_lookup, sequence_len):
    input_sequence = en_tokenizer.texts_to_sequences([text])
    padded_input_sequence = pad_sequences(input_sequence, maxlen=sequence_len,
                                         padding='post')

    decoded_text = '[start]'

    for i in range(sequence_len):
        target_sequence = fr_tokenizer.texts_to_sequences([decoded_text])
        padded_target_sequence = pad_sequences(
            target_sequence, maxlen=sequence_len, padding='post')[: , :-1]
```

```

prediction = model([padded_input_sequence, padded_target_sequence])

idx = np.argmax(prediction[0, i, :]) - 1
token = fr_index_lookup[idx]
decoded_text += ' ' + token

if token == '[end]':
    break

return decoded_text[8:-6] # Удалить токены [start] и [end]

```

```

fr_vocab = fr_tokenizer.word_index
fr_index_lookup = dict(zip(range(len(fr_vocab)), fr_vocab))
texts = en[40000:40010].values

```

```

for text in texts:
    translated = translate_text(text, model, en_tokenizer, fr_tokenizer,
                                fr_index_lookup, sequence_len)
    print(f'{text} => {translated}')

```

Ниже приведен результат:

```

its fall now => cest desormais tombe
im losing => je suis en train de perdre
it was quite funny => ce fut assez amusant
thats not unusual => ce nest pas inhabituel
i think ill do that => je pense que je ferai ca
tom looks different => tom a lair different
its worth a try => ca vaut le coup dessayer
fortune smiled on him => la chance lui souri
lets hit the road => cassonsnous
i love winning => jadore gagner

```

Если вы не говорите по-французски, воспользуйтесь Google Translate, чтобы перевести некоторые французские фразы на английский. Например, согласно Google, "la chance lui souri" переводится как "Luck smiled on him" ("Удача улыбнулась ему"), а "ce nest pas inhabituel" — как "it's not unusual" ("это не необычно"). Модель не идеальна, но и не плоха. Словарный запас, который вы использовали, невелик, поэтому нельзя ввести любую фразу и ожидать, что модель ее переведет. Но простые фразы, в которых используются слова из обучающего текста, переводятся достаточно хорошо.

В завершение воспользуйтесь функцией `translate_text`, чтобы посмотреть, как модель переводит "Hello world" на французский:

```

translate_text('Hello world', model, en_tokenizer, fr_tokenizer,
               fr_index_lookup, sequence_len)

```

Я не учил французский со школы, но даже я знаю, что "Salut le monde" — это подходящий перевод "Hello world".

Использование предварительно обученных моделей для перевода текста

Инженеры Microsoft, Google и Facebook располагают ресурсами для сбора миллионов примеров перевода текста и оборудованием для обучения на них сложных трансформерных моделей, но мы с вами — нет. Хорошая новость заключается в том, что это не мешает нам разрабатывать программное обеспечение, переводящее текст с одного языка на другой. Компания Hugging Face опубликовала несколько предварительно обученных моделей трансформеров, которые отлично справляются с переводом текста. Использовать эти модели в Python очень просто.

Вот пример, переводящий текст на английском языке на французский:

```
from transformers import pipeline
translator = pipeline('translation_en_to_fr')
translation = translator('Programming is fun!') [0]['translation_text']
print(translation)
```

Этот же синтаксис можно использовать для перевода с английского на немецкий и с английского на румынский. Просто замените `translation_en_to_fr` на `translation_en_to_de` или `translation_en_to_ro` при создании конвейера.

Для других языков используется более подробный синтаксис для загрузки трансформера и соответствующего токенизатора. Следующий пример переводит голландский язык на английский:

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM

# Инициализация токенизатора
tokenizer = AutoTokenizer.from_pretrained('Helsinki-NLP/opus-mt-nl-en')

# Инициализация модели
model = AutoModelForSeq2SeqLM.from_pretrained('Helsinki-NLP/opus-mt-nl-en')

# Токенизация входного текста
text = 'Hallo vrienden, hoe gaat het vandaag?'
tokenized_text = tokenizer.prepare_seq2seq_batch([text], return_tensors='pt')

# Выполнение перевода и декодирование выходных данных
translation = model.generate(**tokenized_text)
translated_text = tokenizer.batch_decode(
    translation, skip_special_tokens=True)[0]
print(translated_text)
```

На сайте организации (<https://oreil.ly/MSuEz>) вы найдете исчерпывающий список переводчиков и токенизаторов Hugging Face. Их сотни, и они охватывают десятки языков.

Двунаправленное представление кодировщика на основе трансформеров (BERT)

Внедрение трансформеров в 2017 году заложило основу для еще одной знаковой инновации в NLP-пространстве: *двунаправленное представление кодировщика на основе трансформеров* (bidirectional encoder representations from transformers, BERT). Представленный исследователями Google в 2018 году в работе под названием "BERT: предварительное обучение глубоких двунаправленных трансформеров для понимания языка" ("BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding", <https://oreil.ly/2fGXv>), BERT значительно продвинул технологию, предоставив предварительно обученные трансформеры, которые можно точно настроить для различных задач NLP.

Модель BERT была обучена пониманию языка путем тренировки на более чем 2,5 млрд слов из статей Википедии и 800 млн слов из Google Books. Обучение заняло четыре дня на 64 тензорных процессорах (TPU). Тонкая настройка осуществляется путем дальнейшего обучения предварительно обученной модели с использованием образцов, специфичных для конкретной задачи, и снижением скорости обучения (рис. 13.12). Например, очень просто настроить BERT для выполнения анализа настроения и превзойти по точности модели мешка слов. Ценность BERT заключается в том, что она обладает врожденным пониманием языков, на которых была обучена, и может быть усовершенствована для выполнения специфических задач.

Помимо того, что модель была обучена на огромном количестве образцов, ключом к способности BERT понимать человеческие языки является инновация, известная как *моделирование языка по маске* (masked language modeling, MLM; <https://oreil.ly/5OrrT>). Суть MLM заключается в том, что модель имеет больше шансов предсказать, какое слово должно заполнить пробел во фразе "Every good ____ does fine", чем предсказать следующее слово во фразе "Every good ____". Ответом может быть *boy*, как во фразе "Every good boy does fine", или *turn*, как во фразе "Every good turn deserves another". Однонаправленные модели изучают текст слева или справа и пытаются предсказать, каким должно быть следующее слово. MLM, в свою очередь, использует текст слева и справа для принятия решений. Именно поэтому BERT является "двунаправленным" трансформером.

При предварительном обучении BERT-моделей определенный процент словесных токенов в каждой последовательности — обычно 15% — случайным образом удаляется или "маскируется", чтобы модель могла научиться предсказывать их по окружающим словам. Кроме того, BERT-модели обычно предварительно обучаются прогнозированию следующего предложения, что делает их более эффективными в некоторых задачах NLP, таких как ответы на вопросы.

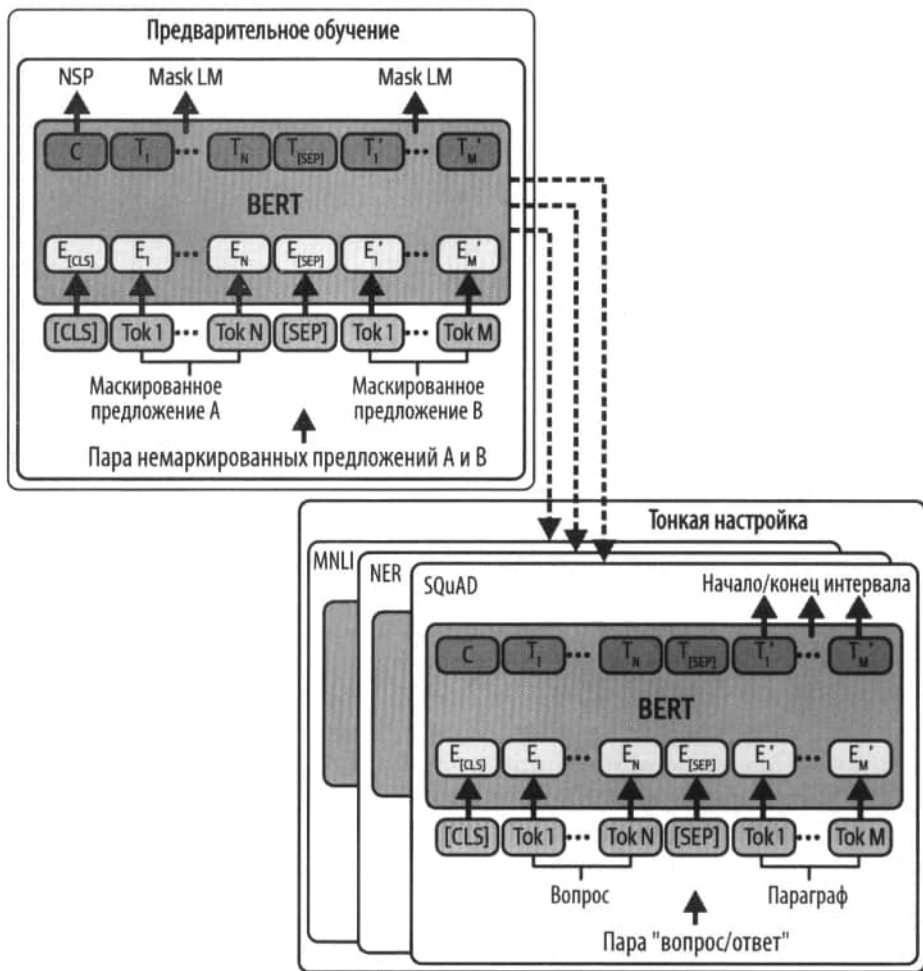


Рис. 13.12. Двухнаправленное представление кодировщика на основе трансформеров (BERT)

BERT называют "швейцарским армейским ножом" NLP. Google использует ее для улучшения результатов поиска и предсказания текста, который вы вводите в Gmail или Google Doc. Были опубликованы десятки вариантов, включая DistilBERT (<https://oreil.ly/3u60q>), который сохраняет 97% точности оригинальной модели, но при этом весит на 40% меньше и работает на 60% быстрее. Также доступны варианты BERT, уже настроенные для решения конкретных задач, таких как ответы на вопросы. Подобные модели можно доработать с использованием наборов данных, специфичных для конкретной области, или же их можно использовать как есть.

Построение системы ответов на вопросы на основе BERT

Пакет трансформеров Hugging Face содержит несколько предварительно обученных BERT-моделей, уже настроенных для решения конкретных задач. Одним из

примеров является модель minilm-uncased-squad2 (<https://oreil.ly/eBU3a>), которая была обучена на наборе данных SQuAD 2.0 (<https://oreil.ly/2Cw0C>) в Стэнфорде для ответов на вопросы путем извлечения текста из документов. Для того чтобы понять, на что способны модели, подобные этой, давайте используем их для создания простой системы ответов на вопросы.

Сначала немного контекста. Когда вы задаете Google вопрос, подобный тому, что показан на рис. 13.13, он запрашивает базу данных, содержащую миллиарды веб-страниц, чтобы найти те, которые могут содержать ответ. Затем с помощью модели NLP на основе BERT он извлекает ответы из найденных страниц и ранжирует их по степени достоверности.

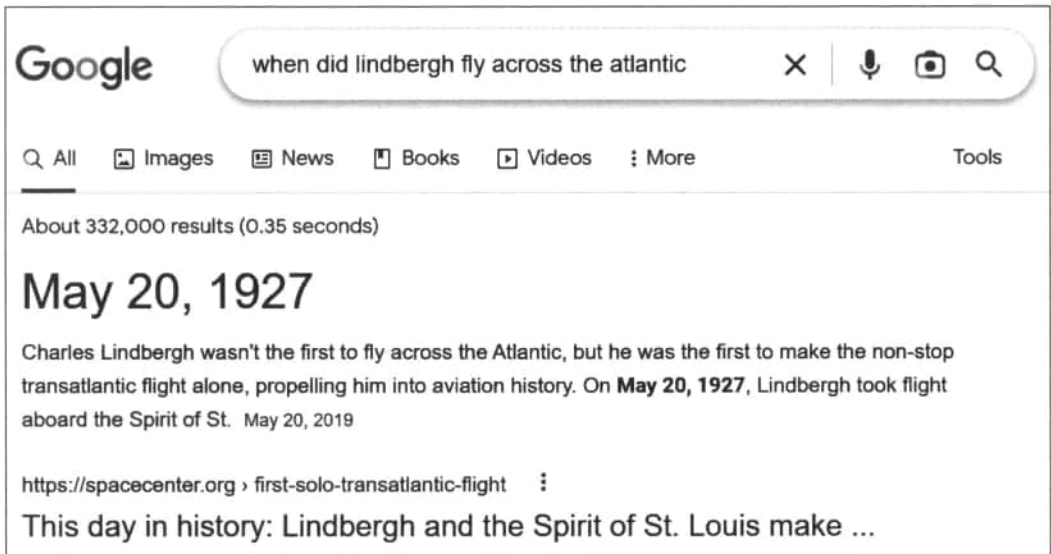


Рис. 13.13. Ответ на запрос в Google¹

Давайте загрузим предварительно обученную модель BERT, уже настроенную отвечать на вопросы, и используем ее для извлечения ответов из отрывков текста этой книги. Модель, которую будем использовать, — это версия модели MiniLM, представленная в статье 2020 года под названием "MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-trained Transformers" ("MiniLM: глубокая дистилляция самовнимания для независимого от задач сжатия предварительно обученных трансформеров", <https://oreil.ly/37m84>). Эта версия была доработана на SQuAD 2.0, содержащем более 100 000 сгенерированных людьми вопросов в паре со взятыми из статей Википедии ответами, а также 50 000 вопросов, на которые нет ответов. Архитектура MiniLM позволяет применять понимание прочитанного, полученное на одном наборе данных, к другим наборам данных практически без переобучения.

¹ When did Lindbergh fly across the atlantic. — Когда Линдберг перелетел через Атлантику. — Прим. ред.



Мой учитель истории в восьмом классе, мистер Эйрд, как-то летом жил в одной комнате с Чарльзом Линдбергом в начале 1920-х годов. Очевидно, самый известный в мире авиатор был в колледже чем-то вроде сорвиголо-вы, и я никогда не забуду, что сказал о нем мистер Эйрд. "В 1927 году, ко-гда я узнал, что Чарльз в одиночку перелетел через Атлантику, я не уди-вился. Я никогда не встречал человека, который относился бы к собствен-ной жизни с меньшим уважением, чем Чарльз Линдберг" ☺.

Начните с создания нового блокнота Jupyter и использования следующих операторов для загрузки предварительно обученной модели MiniLM из хаба Hugging Face (<https://oreil.ly/sZaah>) и токенизатора для токенизации поступающего в модель текста. (BERT использует специальный формат токенизации под названием WordPiece (<https://oreil.ly/h8rZ4>), немного отличающийся от того, который используют класс Tokenizer и слой TextVectorization в Keras. К счастью, у Hugging Face есть решение и для этого.) Затем составьте из них конвейер. При первом запуске этого кода вы заметите кратковременную задержку, пока загружаются предвари-тельно обученные веса. После этого веса будут кешироваться, и загрузка будет бы-строй:

```
from transformers import AutoTokenizer, TFAutoModelForQuestionAnswering, pipeline
```

```
id = 'deepset/minilm-uncased-squad2'
tokenizer = AutoTokenizer.from_pretrained(id)
model = TFAutoModelForQuestionAnswering.from_pretrained(id, from_pt=True)
pipe = pipeline('question-answering', model=model, tokenizer=tokenizer)
```

Hugging Face хранит веса для этой конкретной модели в формате PyTorch. Параметр `from_pt=True` преобразует веса в формат TensorFlow. Преобразование весов нейронной сети из одного формата в другой не является тривиальной задачей, но библиотека Hugging Face сводит ее к простому параметру функции.

Теперь используйте конвейер для ответа на вопрос, извлекая текст из абзаца:

```
question = 'What does NLP stand for?'
```

```
context = 'Natural Language Processing, or NLP, encompasses a variety of ' \
          'activities, including text classification, keyword and topic ' \
          'extraction, text summarization, and language translation. The ' \
          'accuracy of NLP models has improved in recent years for a variety ' \
          'of reasons, not the least of which are newer and better ways of ' \
          'converting words and sentences into dense vector representations ' \
          'that incorporate context, and a relatively new neural network ' \
          'architecture called the transformer that can zero in on the most ' \
          'meaningful words and even differentiate between multiple meanings ' \
          'of the same word.'
```

```
pipe(question=question, context=context)
```

Точен ли ответ? Человек мог бы легко прочитать этот абзац и дать такой же ответ, но тот факт, что это может сделать модель глубокого обучения, указывает на то, что модель демонстрирует некоторый уровень понимания прочитанного. Обратите внимание, что результат содержит ответ на вопрос, а также оценку достоверности и начальный и конечный индексы ответа в абзаце:

```
{'score': 0.9793193340301514,
 'start': 0,
 'end': 27,
 'answer': 'Natural Language Processing'}
```

Теперь попробуйте повторить это с другим вопросом и в другом контексте:

```
question = 'When was TensorFlow released?'
```

```
context = 'Machine learning isn\'t hard when you have a properly ' \
          'engineered dataset to work with. The reason it\'s not ' \
          'hard is libraries such as Scikit-Learn and ML.NET, which ' \
          'reduce complex learning algorithms to a few lines of code. ' \
          'Deep learning isn\'t difficult, either, thanks to libraries ' \
          'such as the Microsoft Cognitive Toolkit (CNTK), Theano, and ' \
          'PyTorch. But the library that most of the world has settled ' \
          'on for building neural networks is TensorFlow, an open source ' \
          'framework created by Google that was released under the ' \
          'Apache License 2.0 in 2015.'
```

```
pipe(question=question, context=context)['answer']
```

На этот раз на выходе получается ответ, предоставленный моделью, а не содержащим ответ словарем. И снова: подходит ли этот ответ?

Повторите этот процесс с другим вопросом и контекстом, из которого нужно извлечь ответ:

```
question = 'Is Keras part of TensorFlow?'
```

```
context = 'The learning curve for TensorFlow is rather steep. ' \
          'Another library, named Keras, provides a simplified ' \
          'Python interface to TensorFlow and has emerged as the ' \
          'Scikit of deep learning. Keras is all about neural networks. ' \
          'It began life as a standalone project in 2015 but was ' \
          'integrated into TensorFlow in 2019. Any code that you write ' \
          'using TensorFlow\'s built-in Keras module ultimately executes ' \
          'in (and is optimized for) TensorFlow. Even Google recommends ' \
          'using the Keras API.'
```

```
pipe(question=question, context=context)['answer']
```


Проведите завершающий тест, используя тот же контекст, что и раньше, но другой вопрос:

```
question = 'Is it better to use Keras or TensorFlow to build neural networks?'
pipe(question=question, context=context)['answer']
```

Заданные здесь вопросы были отобраны вручную, чтобы подчеркнуть возможности модели. Нетрудно придумать вопросы, на которые модель не сможет ответить. Тем не менее вы доказали принцип, согласно которому предварительно обученная и отлаженная на SQuAD 2.0 модель BERT может отвечать на простые вопросы из представленных ей отрывков текста.

Сложные системы ответа на вопросы используют архитектуру искателя и читателя (retriever-reader), в которой искатель ищет в хранилище данных релевантные документы — те, которые могут содержать ответ на вопрос, а читатель извлекает ответы из документов. В качестве считывающего устройства часто выступает экземпляр BERT, подобный представленному ранее. Искатель может быть одним из искателей из библиотеки с открытым исходным кодом Haystack (<https://oreil.ly/fopHE>), опубликованной специализирующейся на решениях в области NLP немецкой компанией Deepset (<https://oreil.ly/OIzma>). Искатели Haystack взаимодействуют с широким спектром хранилищ документов, включая хранилища Elasticsearch (<https://oreil.ly/CHDKf>), отличающиеся высокой масштабируемостью. Если вы хотите узнать больше или создать собственную систему искателей-читателей, рекомендую прочитать главу 7 книги "Обработка естественного языка с помощью трансформеров" Льюиса Танстолла, Леандро фон Верры и Томаса Вольфа (Tunstall L., von Werra L., Wolf T. Natural Language Processing with Transformers. — O'Reilly, 2022; <https://oreil.ly/tjvsQ>).

Тонкая настройка BERT для выполнения анализа настроений

Современный анализ настроения можно реализовать путем тонкой настройки предварительно обученных моделей BERT на наборах данных для анализа настроения. Давайте настроим BERT и посмотрим, сможем ли мы создать модель анализа настроения, которая будет более точной, чем представленная ранее в этой главе модель мешка слов. Если ваш компьютер не оснащен GPU, я *настоятельно* рекомендую запустить этот пример в Google Colab. Даже на GPU его выполнение может занять около часа.

Если вы запускаете этот код локально, убедитесь, что установлен пакет Datasets (<https://oreil.ly/sYdlg>) от Hugging Face. Затем создайте новый блокнот Jupyter. Если вы используете Colab, создайте новый блокнот и выполните следующие команды в первой ячейке, чтобы установить необходимые пакеты в среду Colab:

```
!pip install transformers
!pip install datasets
```

Затем с помощью следующих команд загрузите набор данных IMDB из пакета Datasets. Это альтернатива загрузке из CSV-файла. Поскольку мы используем модели Hugging Face, можно загрузить данные и из Hugging Face. Кроме того, если используется Colab, это избавит вас от необходимости загружать CSV-файл в среду Colab. Обратите внимание, что в первый раз загрузка набора данных может занять несколько минут:

```
from datasets import load_dataset
imdb = load_dataset('imdb')
imdb
```

Возвращаемое `load_dataset` значение — это словарь, содержащий три набора данных Hugging Face. Вот результат выполнения последнего оператора:

```
DatasetDict({
  train: Dataset({
    features: ['text', 'label'],
    num_rows: 25000
  })
  test: Dataset({
    features: ['text', 'label'],
    num_rows: 25000
  })
  unsupervised: Dataset({
    features: ['text', 'label'],
    num_rows: 50000
  })
})
```

`imdb['train']` содержит 25 000 образцов для обучения, а `imdb['test']` — 25 000 образцов для тестирования. Отзывы о фильмах хранятся в столбце `text` каждого набора данных. Метки хранятся в столбце `label`.

Далее следует токенизация входных данных с помощью токенизатора BERT WordPiece:

```
from transformers import AutoTokenizer
```

```
tokenizer = AutoTokenizer.from_pretrained('distilbert-base-uncased')
```

```
def tokenize(samples):
    return tokenizer(samples['text'], truncation=True)
```

```
tokenized_imdb = imdb.map(tokenize, batched=True)
```

Теперь, когда отзывы токенизированы, их нужно преобразовать в наборы данных TensorFlow (<https://oreil.ly/R35hn>) с помощью метода `Dataset.to_tf_dataset` (<https://oreil.ly/fcK3H>) от Hugging Face. Передаваемая в метод функция сборки ди-

намически разбивает последовательности, чтобы все они были одинаковой длины. Можно также указать токенизатору, что он будет выполнять вставку, но выполненная таким образом вставка является статической и требует больше памяти:

```
from transformers import DataCollatorWithPadding
```

```
data_collator = DataCollatorWithPadding(tokenizer=tokenizer, return_tensors='tf')
```

```
train_data = tokenized_imdb['train'].to_tf_dataset(
    columns=['attention_mask', 'input_ids', 'label'],
    shuffle=True, batch_size=16, collate_fn=data_collator
)
```

```
validation_data = tokenized_imdb['test'].to_tf_dataset(
    columns=['attention_mask', 'input_ids', 'label'],
    shuffle=False, batch_size=16, collate_fn=data_collator
)
```

Теперь всё готово к тонкой настройке. Вызовите как обычно функцию `fit` для модели, но установите скорость обучения оптимизатора (множитель, используемый для корректировки весов и смещений в процессе обратного распространения) равной 0,00002, что в разы меньше скорости обучения по умолчанию в Adam, равной 0,001:

```
from tensorflow.keras.optimizers import Adam
from transformers import TFAutoModelForSequenceClassification
```

```
model = TFAutoModelForSequenceClassification.from_pretrained(
    'distilbert-base-uncased', num_labels=2)
model.compile(Adam(learning_rate=2e-5), metrics=['accuracy'])
hist = model.fit(train_data, validation_data=validation_data, epochs=3)
```

Постройте графики точности обучения и валидации, чтобы увидеть, где последняя достигает максимума:

```
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
sns.set()
```

```
acc = hist.history['accuracy']
val = hist.history['val_accuracy']
epochs = range(1, len(acc) + 1)
```

```
plt.plot(epochs, acc, '-', label='Точность обучения')
plt.plot(epochs, val, ':', label='Точность валидации')
```

```
plt.title('Точность обучения и валидации')
plt.xlabel('Эпоха')
plt.ylabel('Точность')
plt.legend(loc='lower right')
plt.plot()
```

Вот что получилось у меня (рис. 13.14).

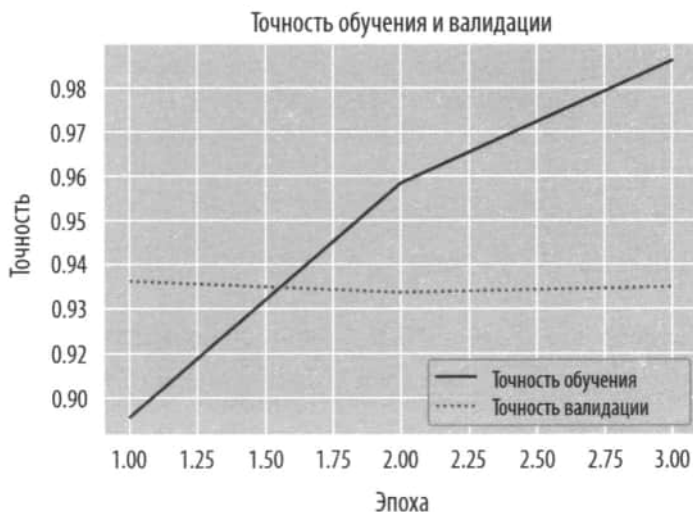


Рис. 13.14. Графики точности обучения и валидации модели анализа настроения после настройки BERT

Если повезет, точность валидации составит около 93% — на несколько пунктов лучше, чем у эквивалентной модели "мешок слов". Только представьте, чего можно добиться, если обучить модель на более чем 25 000 отзывов. Одна из предварительно обученных моделей анализа настроения Hugging Face — модель `twitter-roberta-base` (<https://oreil.ly/uplef>) — была обучена на 58 млн твитов. Неудивительно, что она прекрасно справляется с оценкой текста на предмет настроения.

В завершение определите функцию `analyze_text`, возвращающую оценку настроения, и используйте ее для оценки положительного отзыва на предмет настроения. Модель возвращает объект, обернутый в тензор, содержащий ненормированные оценки настроения (одну для отрицательного и одну для положительного), но вы можете использовать функцию `softmax` (<https://oreil.ly/LY91Q>) от TensorFlow для нормализации их до значений от 0,0 до 1,0:

```
import tensorflow as tf
```

```
def analyze_text(text, tokenizer, model):
    tokenized_text = tokenizer(text, padding=True, truncation=True,
                               return_tensors='tf')
```

```
prediction = model(tokenized_text)
return tf.nn.softmax(prediction[0]).numpy()[0][1]
```

```
analyze_text('Great food and excellent service!', tokenizer, model)
```

Попробуйте еще раз с отрицательным отзывом:

```
analyze_text('The long lines and poor customer service really turned me off.',
            tokenizer, model)
```

Тонкая настройка — удовольствие не из дешевых, но она не так дорога, как обучение сложного трансформера с нуля. Тот факт, что можно обучить модель анализа настроений с такой точностью примерно за час работы на GPU, является заслугой мощности предварительно обученных BERT-моделей и создавших их инженеров Google.

Резюме

Обработка естественного языка, или NLP, — это область глубокого обучения, включающая классификацию текстов, ответы на вопросы, перевод текстов и другие задачи, требующие от компьютеров обработки текстовых данных. Ключевым элементом каждой модели NLP является эмбединговый слой, представляющий слова в виде массивов чисел с плавающей точкой, которые моделируют отношения между словами. Например, векторы слов *excellent* и *amazing* в эмбединговом пространстве расположены близко друг к другу, в то время как векторы слов *butterfly* и *basketball* находятся далеко друг от друга, поскольку эти слова не имеют семантической связи. Эмбединги слов изучаются в процессе обучения модели.

Поступающий в эмбединговый слой текст сначала должен быть токенизирован и превращен в последовательности одинаковой длины. Большую часть работы выполняет класс `Tokenizer` из Keras. Вместо того чтобы отдельно заниматься токенизацией и упорядочиванием текста, можно включить в модель слой `TextVectorization`, который будет выполнять токенизацию и вставку автоматически.

Один из способов классификации текста — использование традиционного плотного слоя для классификации векторов, полученных из эмбедингового слоя. Альтернативный вариант — применение сверточных слоев или рекуррентных слоев для извлечения информации о положении слова из эмбединговых векторов и классификации выходных данных этих слоев.

Использование глубокого обучения для перевода текста на другие языки известно как нейромашинный перевод, или NMT. До недавнего времени для NMT использовались модели кодировщиков-декодировщиков на основе LSTM. Сегодня эти модели в значительной степени уступили место трансформерам, использующим нейронное внимание для выделения наиболее значимых слов в фразе и моделирования контекста слова. Трансформер знает, что слово *train* имеет разное значение в фразах "meet me at the train station" и "it's time to train a model", и учитывает порядок слов в своих расчетах.

BERT — это сложная умеющая понимать язык модель-трансформер, которую инженеры Google обучили на миллиардах слов. Ее можно тонко настраивать для решения конкретных задач, таких как ответы на вопросы и анализ настроения, и несколько ее версий были опубликованы для всеобщего использования. Иногда эти модели можно применять без изменений. В других случаях их можно доработать и адаптировать к конкретным задачам. Поскольку тонкая настройка требует на порядки меньше данных и вычислительных мощностей, чем обучение BERT с нуля, предварительно обученные (и предварительно настроенные) модели BERT оказались благом для NLP.

Сложные модели NLP обучаются на миллионах слов или фраз, что требует значительных инвестиций в сбор данных и оборудование для обучения. Такие компании, как Hugging Face, публикуют предварительно обученные модели, которые можно использовать в своем коде. Это растущая тенденция в области искусственного интеллекта — публикация моделей, уже обученных для решения общих задач. Вы все равно будете создавать модели для решения специфичных задач, но многие задачи, особенно связанные с NLP, не привязаны к конкретной области.

Загрузка предварительно обученных моделей не единственный способ использовать сложный искусственный интеллект для решения бизнес-задач. Такие компании, как Microsoft и Amazon, сами обучают модели глубокого обучения и предоставляют к ним открытый доступ через REST API. Microsoft называет свой набор сервисов Azure Cognitive Services. Вы узнаете о них в *главе 14*.

Azure Cognitive Services

В моем детстве в 1960-х годах я боготворил астронавтов "Аполлона". Выходящие на стартовую площадку и отправляющиеся в космос на огнедышащих ракетах, они были моими супергероями. Но те, кому я действительно хотел подражать — те, кем я хотел стать, — были инженерами в Mission Control (Центре управления полетами). Сидящие перед своими ЭЛТ-экранами в белых рубашках и черных галстуках, переговаривающиеся с астронавтами и готовые действовать при первых признаках неполадок, они были воплощением крутизны. Они использовали компьютеры меньшей мощности, чем современные смартфоны, для отправки людей на Луну — научное достижение, которое не превзойдено и по сей день.

Благодаря глубокому обучению компьютеры сегодня могут совершать такие волшебные подвиги, о которых инженеры в Mission Control могли только мечтать. Они могут распознавать объекты на изображениях, переводить текст и речь на другие языки, идентифицировать людей в видеопотоке, превращать искусство в слова, а слова — в искусство и многое другое. Но современные модели глубокого обучения слишком сложны и дорогостоящи, чтобы их мог создать обычный инженер или разработчик программного обеспечения. По имеющимся данным, Microsoft потратила сотни тысяч долларов на обучение модели ResNet, которая победила в 2015 году в конкурсе ImageNet Large Scale Visual Recognition Challenge. Создание этой модели потребовало большого опыта, огромного количества времени GPU и миллионов изображений.

Сегодня в сфере искусственного интеллекта наблюдается положительный тренд — искусственный интеллект *как сервис*. Microsoft, Amazon, Google и другие технологические гиганты нанимают профессиональных специалистов по обработке данных, создающих сложные модели глубокого обучения. Компании обучают модели за свой счет и предоставляют их всем желающим с помощью REST API (<https://oreil.ly/OGfQf>). Если вы умеете программировать код для отправки HTTP-запроса через Интернет, вы можете использовать эти API для внедрения искусственного интеллекта в свои приложения, не отрываясь от основной работы для получения степени доктора наук в области глубокого обучения.

Microsoft называет свой набор сервисов для работы с искусственным интеллектом Azure Cognitive Services (<https://oreil.ly/NhIQs>). Amazon использует название AWS AI Services (<https://oreil.ly/dmLGH>). Оба сервиса предлагают богатый набор API, обслуживаемых моделями глубокого обучения на внутреннем уровне — моделями,

которые постоянно совершенствуются, чтобы со временем стать еще умнее. Нужно сделать подписи к фотографиям, загруженным на сайт? Сервис Computer Vision (<https://oreil.ly/K7C1P>) от Microsoft может это выполнить, как и сервис Rekognition (<https://oreil.ly/NGDCv>) от Amazon. А как насчет создания программы для чтения с экрана с реалистичным человеческим голосом, чтобы помочь слабослышащим людям? С этим справится Amazon Polly (<https://oreil.ly/H4Zfe>), а также Speech (<https://oreil.ly/D5M1S>) от Azure Cognitive Services и API преобразования текста в речь (<https://oreil.ly/8OzPW>) Google. Это лишь несколько примеров того, какие действия могут выполнять когнитивные сервисы, причем зачастую благодаря всего нескольким строкам кода.

Вы видели работу Azure Cognitive Services в *главе 12*, когда использовали сервис Custom Vision (<https://oreil.ly/NAW9S>) для обучения пользовательской модели обнаружения объектов. Это один из нескольких сервисов, входящих в семейство Cognitive Services. В этой главе представлены другие и показано, как их использовать и как создавать с их помощью решения. Особое внимание к Azure Cognitive Services не означает, что они лучше своих аналогов от Amazon и Google. Просто я лучше знаком с ними, поскольку тесно сотрудничаю с Microsoft уже более двух десятилетий. Как только вы научитесь вызывать API-интерфейсы Azure Cognitive Services, вам не составит труда применить эти знания к когнитивным сервисам других разработчиков.

Готовы творить волшебство? Давайте начнем.

Знакомство с Azure Cognitive Services

Набор сервисов, из которых состоит Azure Cognitive Services, время от времени меняется по мере добавления новых сервисов, устаревания других или их перехода в прочие линейки продуктов Microsoft. На рис. 14.1 показаны сервисы, которые предлагаются в настоящее время, и они разделены на четыре категории: зрение, язык, речь и принятие решений.

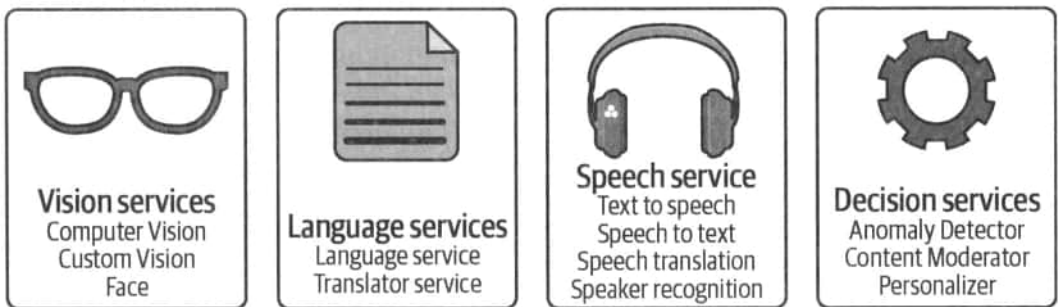


Рис. 14.1. Azure Cognitive Services

Сервисы зрения позволяют использовать глубокие нейронные сети для решения проблем компьютерного зрения. Сервис Custom Vision позволяет создавать пользо-

вательские модели классификации изображений и обнаружения объектов. Сервис Face (<https://oreil.ly/TSV6H>), используемый компанией Uber для проверки личности своих водителей (<https://oreil.ly/SZja1>), поддерживает самые современные системы распознавания лиц. Сервис Computer Vision предоставляет богатый API, содержащий множество способов анализа изображений и извлечения из них информации. Одно из применений этого сервиса — создание подписей к фотографиям и генерация характеризующих их содержание ключевых слов. На рис. 14.2 показано разработанную мною веб-приложение под названием Intellipix. Оно создает подписи к загруженным пользователями изображениям и сохраняет описывающие их ключевые слова в базе данных, чтобы пользователи могли легко найти все изображения, содержащие, например, замки или фотографии с водой на переднем или заднем плане.

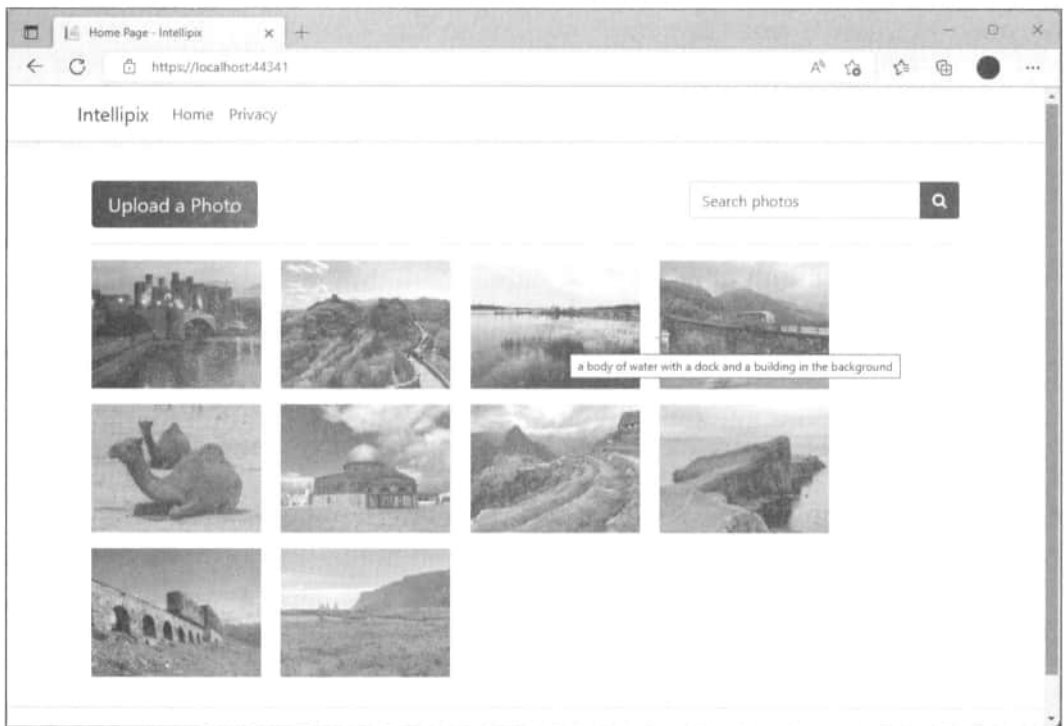


Рис. 14.2. Приложение Intellipix с демонстрацией сгенерированной компьютером надписи к изображению

Языковая категория предлагает сервисы, основанные на моделях глубокого обучения, обученных выполнять обработку естественного языка (NLP). Сервис Language (<https://oreil.ly/vOcSJ>) предоставляет API для анализа настроений, распознавания именованных объектов, ответов на вопросы, извлечения ключевых фраз, понимания языка и многого другого. Среди множества применений этого сервиса — создание чат-ботов, осмысленно отвечающих на запросы о продуктах и услугах компании. Есть также сервис Translator (<https://oreil.ly/bHAv2>), который переводит

текст с более чем 100 языков и диалектов. Компания Volkswagen использует его для перевода экранных инструкций в автомобилях (<https://oreil.ly/JWXRf>) для соответствия языку руководства пользователя и для обеспечения качества документации, выпускаемой на более чем 40 языках.



Сервис Language — это единый сервис, который включает в себя функции нескольких более ранних сервисов, в том числе сервиса Text Analytics, сервиса Language Understanding и QnA Maker. Microsoft будет продолжать поддерживать старые сервисы в течение какого-то времени, но новые разработки должны ориентироваться на сервис Language.

Сервис Speech (<https://oreil.ly/rJLPI>) предоставляет API для преобразования текста в речь (<https://oreil.ly/4U9jJ>) и речи в текст (<https://oreil.ly/ngZ2v>). Компания KPMG использует последний для расшифровки записанных звонков (<https://oreil.ly/GUfGr>) и утверждает, что это позволило ее клиентам сэкономить миллионы долларов на соблюдении нормативных требований. Airbus использует его для создания голосовых приложений для обучения пилотов. Сервис Speech также включает API распознавания говорящего (<https://oreil.ly/OLVbn>), который позволяет идентифицировать голос человека, и API перевода речи (<https://oreil.ly/q3CXc>), способный переводить речь на десятки других языков в режиме реального времени.

Сервисы принятия решений включают Anomaly Detector (<https://oreil.ly/OYwFc>), выявляющий аномалии в транслируемых или записанных потоках данных и способный объединять данные из сотен разрозненных источников, таких как датчики температуры и манометры, и моделировать взаимосвязи между ними (см. главу 6 для ознакомления с обнаружением аномалий). Airbus использует Anomaly Detector для анализа напряжений и деформаций в самолетах (<https://oreil.ly/Uci2C>), а Siemens — для проверки медицинских приборов на наличие дефектов (<https://oreil.ly/kdT5N>) на завершающем этапе производства. Сервис Content Moderator (<https://oreil.ly/Tzmyj>) применяет искусственный интеллект, дополняемый человеческим участием, чтобы отмечать оскорбительное содержание изображений и видео, нецензурные тексты и другой неприемлемый (и потенциально клеветнический) контент. И последний, но не менее важный сервис — Personalizer (<https://oreil.ly/hn797>), который можно назвать рекомендательной системой на стероидах, предоставляющей конечным пользователям персонализированный контент и опыт.

Ключи и конечные точки

Azure Cognitive Services предоставляет REST API, которые вызываются путем передачи HTTPS-запросов через Интернет. API-интерфейсы не зависят от языка: к ним можно обращаться из любого языка программирования, умеющего передавать запросы. Это Python, Java, C#, C, C++, JavaScript, Swift, Go и практически все другие современные языки программирования. Вызовы также можно осуществлять с

помощью таких инструментов, как Postman (<https://oreil.ly/LoaJw>) и команда `curl` в Linux.

Большинство сервисов Azure Cognitive Services до определенного момента бесплатны, но не совсем. Например, вы можете отправить 5000 образцов текста в месяц в сервис Language для анализа настроений без каких-либо затрат. Однако при превышении этого объема Azure необходимо знать, с кого взимать плату за подписку Azure. Следовательно, перед работой с Azure Cognitive Service вам необходимы:

- ◆ конечная точка сервиса (эндпойнт) — URL-адрес, на который направляются вызовы;
- ◆ ключ подписки на сервис или другие средства проверки подлинности вызовов.

Оба параметра можно получить на Azure Portal (<https://oreil.ly/r5FA9>). В качестве примера предположим, что вы хотите выполнить анализ настроений для коллекции твитов. Сначала вы открываете портал Azure Portal в браузере и авторизуетесь под своей учетной записью Microsoft. Затем вы создаете ресурс Cognitive Services Language и указываете, с кого следует взимать плату за подписку Azure, регион Azure, в котором должен быть расположен сервис, и ценовой уровень, как показано на рис. 14.3. Бесплатный уровень обычно доступен, если он еще не существует для того же сервиса в рамках той же подписки.

The screenshot shows the 'Review + create' step in the Azure Portal for creating a new Cognitive Services Language resource. The interface includes tabs for 'Basics', 'Network', 'Identity', 'Tags', and 'Review + create'. Below the tabs, there is a descriptive paragraph about the service. The 'Project Details' section contains two dropdown menus: 'Subscription' (set to 'Microsoft Azure') and 'Resource group' (set to 'cognitive-services-rg'). The 'Instance Details' section contains three dropdown menus: 'Region' (set to 'South Central US'), 'Name' (set to 'wintellect-language'), and 'Pricing tier' (set to 'S (1K Calls per minute)'). A 'View full pricing details' link is located at the bottom left.

Basics Network Identity Tags Review + create

Unlock insights from unstructured text using advanced natural language processing. Use sentiment analysis to find out what customers think of your brand. Find topic-relevant phrases using key phrase extraction and identify the language of the text with language detection. Detect and categorize entities in your text with named entity recognition.

Learn more

Project Details

Subscription * ⓘ Microsoft Azure ▼

Resource group * ⓘ cognitive-services-rg ▼

Create new

Instance Details

Region ⓘ South Central US ▼

Name * ⓘ wintellect-language ✓

Pricing tier * ⓘ S (1K Calls per minute) ▼

View full pricing details

Рис. 14.3. Создание ресурса Language на портале Azure Portal

После создания ресурса Language вы открываете его в Azure Portal и щелкаете на разделе **Keys and Endpoint** в меню в левой части страницы. Оттуда можно получить ключ подписки и конечную точку для сервиса (рис. 14.4). Ключ — это набор букв и цифр, который однозначно идентифицирует подписку Azure. Обращайтесь с ним осторожно, поскольку его утечка может стоить вам денег. Лучшей практикой считается периодическая ротация ключа (замена его на новый) на случай, если он попадет в чужие руки. Именно поэтому на портале есть кнопка **Regenerate Key** (Пересоздать ключ). Почему портал предоставляет два ключа? Для того чтобы у вас был действующий ключ, который можно использовать после регенерации другого. Если бы у вас был только один ключ и вам пришлось регенерировать его, то затем обязательно понадобилось бы заниматься модификацией всех приложений, которые его используют, и наступил бы "мертвый период", в течение которого совершаемые этими приложениями вызовы не выполнялись бы.

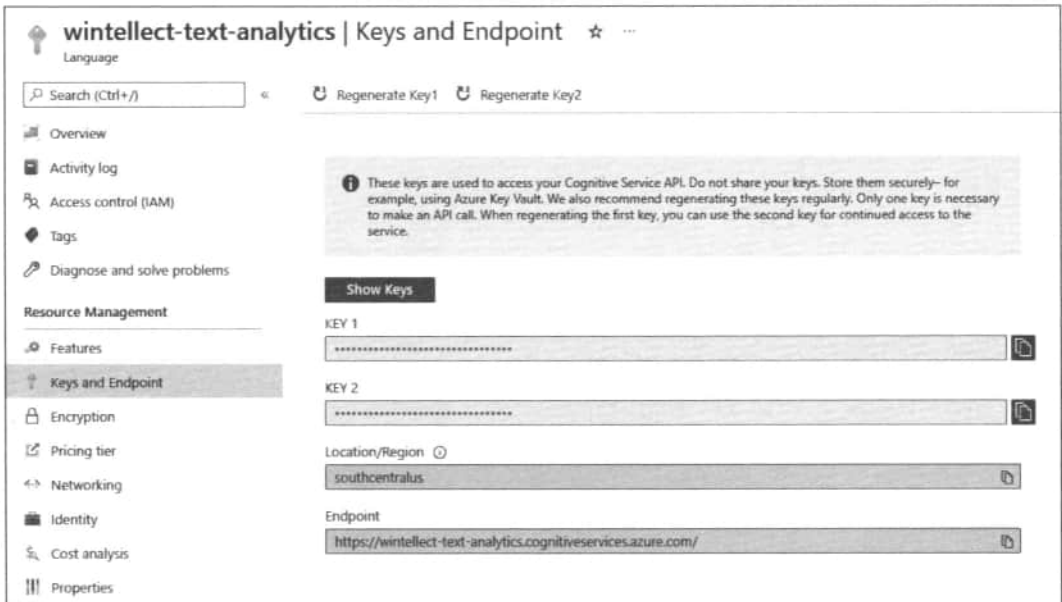


Рис. 14.4. Получение ключа и конечной точки для ресурса Azure Cognitive Services

Ключ подписки в этом примере — это ключ подписки на один сервис, поскольку он работает только с созданным ресурсом Language. Если вы создадите другой ресурс Cognitive Services, например для сервиса Computer Vision, вы получите для него отдельный ключ. Для того чтобы избежать управления множеством ключей для различных сервисов, Azure Cognitive Services поддерживает мультисервисные ключи, работающие с целым рядом сервисов. Для того чтобы получить мультисервисный ключ, просто создайте ресурс Cognitive Services, а не ресурс конкретного сервиса, и используйте ключ и конечную точку, которые предоставляет портал.

Ключи подписки не единственный способ аутентификации обращений к сервисам Azure Cognitive Services. Большинство сервисов поддерживают аутентификацию

Azure Active Directory (AAD), которая повышает безопасность за счет сочетания идентификационных данных AAD с контролем доступа на основе ролей (role-bases access control, RBAC). Аутентификация AAD особенно важна для приложений, которые используют API-интерфейсы Cognitive Services и сами размещены в Azure. Дополнительные сведения см. в статье "Аутентификация запросов к Azure Cognitive" ("Services Authenticate Requests to Azure Cognitive Services", <https://oreil.ly/ofCuI>).

Обращения к API-интерфейсам Azure Cognitive Services

Когда у вас есть ключ подписки и конечная точка, вы готовы к работе. Вот фрагмент кода на Python, использующий сервис Language для оценки настроения фразы "Programming is fun, but the hours are long" ("Программирование — это весело, но отнимает много времени"). KEY — это ключ подписки ресурса Language, а ENDPOINT — соответствующая конечная точка. Оператор `import` импортирует пакет Python Requests (<https://oreil.ly/1pTT8>), упрощающий работу с протоколом запросов и ответов HTTP:

```
import requests
```

```
input = { 'documents': [
    { 'id': '1000', 'text': 'Programming is fun, but the hours are long'} ]
}
```

```
headers = {
    'Ocp-Apim-Subscription-Key': KEY,
    'Content-type': 'application/json'
}
```

```
uri = ENDPOINT + 'text/analytics/v3.0/sentiment'
response = requests.post(uri, headers=headers, json=input)
results = response.json()
```

```
for result in results['documents']:
    print(result['confidenceScores'])
```

Результат получился следующий:

```
{'positive': 0.94, 'neutral': 0.05, 'negative': 0.01}
```

Сервис возвращает три оценки: одну для позитивного настроения, одну для нейтрального и одну для негативного. Обратите внимание, что за один вызов API можно отправить несколько образцов текста, поскольку элемент *документа* в передаваемом вызову словаре представляет собой список Python.

Из кода это может быть неочевидно, но входные данные должны быть закодированы в JSON и отправлены в теле запроса. Выходные данные также возвращаются в виде JSON. Пакет Requests упрощает кодирование и декодирование JSON.

`requests.post` кодирует входные данные и передает POST-запрос (<https://oreil.ly/1lqYK>) на указанную конечную точку, а `response.json` преобразует возвращаемый в ответе JSON в словарь Python.

Для того чтобы упростить работу и оградить вас от изменений в базовых API по мере их развития, Microsoft предлагает бесплатные наборы средств разработки программного обеспечения (Software Development Kit, SDK) для большинства когнитивных сервисов. Пакет для Python под названием `Azure-ai-textanalytics` (<https://oreil.ly/Begdi>) представляет собой SDK для анализа настроений и других видов текстовой аналитики и официально известен как клиентская библиотека `Azure Cognitive Services Text Analytics for Python` (<https://oreil.ly/QC8Cr>). Его можно установить, как и любой другой пакет Python, например командой `pip install`. Вот предыдущий пример, переписанный для использования SDK:

```
from azure.core.credentials import AzureKeyCredential
from azure.ai.textanalytics import TextAnalyticsClient

client = TextAnalyticsClient(ENDPOINT, AzureKeyCredential(KEY))
input = [{ 'id': '1000', 'text': 'Programming is fun, but the hours are long' }]
response = client.analyze_sentiment(input)

for result in response:
    print(result.confidence_scores)
```

Результат получается точно таким же, но JSON скрыт, и вам не нужно знать, какую волшебную строку добавлять в конечную точку, потому что `analyze_sentiment` (<https://oreil.ly/PW0wN>) делает это за вас. Этот метод принадлежит классу `TextAnalyticsClient` (<https://oreil.ly/ICRTV>) SDK, и это один из нескольких доступных для анализа текста методов.

Еще одно преимущество использования SDK — более надежная обработка ошибок. Вызовы могут иногда давать сбои, и хорошо написанное приложение должным образом реагирует на такие сбои. При неудачном вызове `Azure Cognitive Services` код в SDK выбрасывает исключение, которое вы можете перехватить в своем коде. Следующий пример реагирует на сбои, выводя содержащееся в объекте исключения сообщение об ошибке:

```
from azure.core.credentials import AzureKeyCredential
from azure.ai.textanalytics import TextAnalyticsClient
from azure.core.exceptions import AzureError

try:
    client = TextAnalyticsClient(ENDPOINT, AzureKeyCredential(KEY))
    input = [{ 'id': '1000',
                'text': 'Programming is fun, but the hours are long' }]
    response = client.analyze_sentiment(input)
```

```

    for result in response:
        print(result.confidence_scores)

```

```

except AzureError as e:
    print(e.message)

```

`AzureError` определен в `Azure-core` (<https://oreil.ly/z0NAS>), которая является библиотекой, общей для всех Python SDK от Azure Cognitive Services. Это родительский класс для других классов исключений, соответствующих конкретным ошибкам. Например, если передать недействительный ключ подписки (возможно, потому что соответствующая подписка больше не активна или платеж просрочен), возникнет исключение `ClientAuthenticationError`. Можно включить столько обработчиков исключений, сколько необходимо для реагирования на определенные типы ошибок. В предыдущем примере в качестве обработчика используется `AzureError`.

Cognitive Services SDK доступны для различных языков программирования, в том числе для Python, Java и C#. Вот тот же пример, написанный на C# с использованием клиентской библиотеки Azure Cognitive Services Text Analytics для .NET (<https://oreil.ly/3PCW0>), которая поставляется в виде пакета NuGet под названием `Azure.AI.TextAnalytics` (<https://oreil.ly/mXRL1>):

```

using Azure;
using Azure.AI.TextAnalytics;
using System;

try
{
    var client = new TextAnalyticsClient(
        new Uri(ENDPOINT),
        new AzureKeyCredential(KEY)
    );

    var response =
        client.AnalyzeSentiment("Programming is fun, but the hours are long");
    var sentiment = response.Value.ConfidenceScores.Positive;
    Console.WriteLine(sentiment);
}

catch (Exception ex)
{
    Console.WriteLine(ex.Message);
}

```

Результат, полученный с помощью этого кода, равен 0,94, что совпадает с оценкой положительного настроения, полученной с помощью примеров на Python. Разный язык, разный SDK, но результат один и тот же, и все это без особых усилий. В этом и заключается суть Azure Cognitive Services.

Контейнеры Azure Cognitive Services

Cognitive Services значительно упрощают процесс внедрения искусственного интеллекта в разрабатываемые приложения и снижают квалификационный барьер для этого. Но у них есть и недостатки.

- ◆ Поскольку Azure Cognitive Services работают в облаке, использующим их приложениям требуется подключение к Интернету.
- ◆ Передаваемые через Интернет вызовы имеют более высокие задержки, чем вызовы, выполняемые локально.
- ◆ API-интерфейсы Azure Cognitive Services со временем развиваются, и иногда в них вносятся изменения, из-за чего код, который вчера работал отлично, завтра может повести себя иначе или стать неработоспособным.
- ◆ Модели глубокого обучения в бэкенде постоянно дорабатываются и совершенствуются, в результате чего оценка настроения для "programming is fun, but the hours are long" может быть 0,94 сегодня и 0,85 завтра.

Изменения в API редко бывают неожиданными. Обычно Microsoft предупреждает своих клиентов за несколько месяцев до внесения изменений в API, и во многих случаях можно указать версию сервиса или API, которую вы хотите использовать, и застраховать себя от будущих изменений. Для небольших компаний эти проблемы могут не иметь большого значения, но для предприятий, ценящих стабильность и надежность превыше всего, они могут стать серьезным препятствием.

Microsoft решила эти проблемы, сделав большинство сервисов Azure Cognitive Services доступными в контейнерах Docker. Контейнерная служба может работать как в локальной сети, так и в облаке, и в ней фиксируются версии API и модели, на которых они основаны. Никаких изменений не произойдет, пока вы не обновите образ контейнера или не замените его более новой версией. Узнать больше и посмотреть актуальный список доступных в контейнерной форме сервисов можно в статье "Что такое контейнеры Azure Cognitive Services?" ("What Are Azure Cognitive Services Containers?", <https://oreil.ly/pPPFv>). Контейнерные службы также являются решением для политик безопасности и конфиденциальности, требующих, чтобы данные оставались в организации. Microsoft не хранит передаваемые в Cognitive Services данные, однако они покидают домен вашей компании.

Контейнерные службы — это *не способ* обойти биллинговый отдел и использовать Azure Cognitive Services бесплатно. Из документации:

"Контейнеры Cognitive Services должны предоставлять информацию об учетных данных для целей оплаты. Если не включить в список разрешений различные сетевые каналы, на которые опираются контейнеры Cognitive Services, контейнер не сможет работать".

Контейнеры отправляют зашифрованную информацию об использовании в Microsoft через порт 443, и Microsoft использует эту информацию для оплаты подписки Azure. Следовательно, контейнеры должны быть подключены к Интернету, даже если они размещены в локальной сети компании.

Есть одно исключение. Если вы хотите использовать Azure Cognitive Services в приложениях, которые невозможно подключить к Интернету по техническим причинам или в соответствии с нормативными требованиями, контейнеры могут работать без подключения к глобальной сети, и они не будут передавать информацию в Microsoft. Однако использовать их может не каждый. Вы должны подать заявку в Microsoft для получения разрешения, и одним из требований является наличие у вашей организации корпоративного соглашения (Enterprise Agreement, EA) с Microsoft. Дополнительные сведения и обновленный список сервисов Azure Cognitive Services, доступных для использования в изолированных сценариях, см. в разделе "Использование контейнеров Docker в изолированных средах" ("Use Docker Containers in Disconnected Environments", <https://oreil.ly/5pbPw>).

Сервис Computer Vision

К этому моменту вы, вероятно, уже готовы прекратить рассуждения об Azure Cognitive Services и начать писать код. Я тоже. Давайте начнем, пожалуй, с самого многофункционального когнитивного сервиса. Сервис Computer Vision — это один из трех сервисов технического зрения в Azure Cognitive Services. Он предоставляет набор API-интерфейсов, поддерживающих различные задачи, среди которых:

- ◆ создание подписей к изображениям;
- ◆ генерация тегов, описывающих содержимое изображения;
- ◆ идентификация объектов (и их границ) на изображениях;
- ◆ обнаружение подозрительного или неуместного содержимого на изображениях;
- ◆ извлечение текста из изображений;
- ◆ обнаружение лиц на изображениях (подмножество сервиса Face);
- ◆ генерация "умных миниатюр" с помощью искусственного интеллекта для определения основного объекта фотографии и создания миниатюрной версии изображения, ориентированной на этот объект;
- ◆ выполнение пространственного анализа в видеопотоках.

Пространственный анализ — последнее дополнение к сервису Computer Vision. Среди прочего, он позволяет отслеживать людей в видеопотоках, измерять расстояния между ними в реальном времени и определять, кто носит маску (а кто нет), как показано на рис. 14.5. В настоящее время он находится в стадии публичного предварительного ознакомления, и вы можете прочитать о нем в статье "Что такое пространственный анализ?" ("What Is Spatial Analysis?", <https://oreil.ly/Y98w6>).

Лучший способ познакомиться с сервисом Computer Vision — вызвать несколько его API. Сначала установите Azure Cognitive Services Computer Vision SDK for Python (<https://oreil.ly/JYNIA>). (Он находится в пакете Python под названием `Azure-cognitiveservices-vision-computervision` (<https://oreil.ly/e4voR>).) Затем загрузите ZIP-файл (<https://oreil.ly/lq08y>) с несколькими примерами изображений и скопируйте их в подкаталог Data, где размещены ваши блокноты Jupyter. С помощью портала

Azure Portal создайте ресурс Computer Vision и получите ключ и конечную точку. Затем создайте новый блокнот и выполните следующий код в первой ячейке, заменив KEY ключом, а ENDPOINT — конечной точкой:

```
from azure.cognitiveservices.vision.computervision import ComputerVisionClient
from msrest.authentication import CognitiveServicesCredentials
```

```
client = ComputerVisionClient(ENDPOINT, CognitiveServicesCredentials(KEY))
```



Рис. 14.5. Использование пространственного анализа для проверки социального дистанцирования и ношения масок (изображения использованы с разрешения Microsoft)

Класс `ComputerVisionClient` реализует несколько методов (<https://oreil.ly/ipHCn>), которые можно использовать для вызова API Computer Vision. Большинство этих методов имеют две версии: одна принимает URL-адрес изображения, а другая — фактическое изображение. Например, метод `describe_image` (<https://oreil.ly/eHIOG>) принимает URL-адрес изображения, а метод `describe_image_in_stream` (<https://oreil.ly/RiYLK>) принимает содержащий изображение поток. В моих примерах используются методы `in_stream`, но вы можете легко модифицировать их, чтобы передавать не изображения, а их URL.

Теперь загрузите одно из изображений-образцов (рис. 14.6), скопированных из ZIP-файла, и используйте `describe_image_in_stream` для создания подписи:

```
%matplotlib inline
import matplotlib.pyplot as plt

image = plt.imread('Data/dubai.jpg')
fig, ax = plt.subplots(figsize=(12, 8), subplot_kw={'xticks': [], 'yticks': []})
ax.imshow(image)

with open('Data/dubai.jpg', mode='rb') as image:
    result = client.describe_image_in_stream(image)

    for caption in result.captions:
        print(f'{caption.text} ({caption.confidence:.1%})')
```

Убедитесь, что вывод выглядит следующим образом:

A man riding a sand dune (53.8%)¹

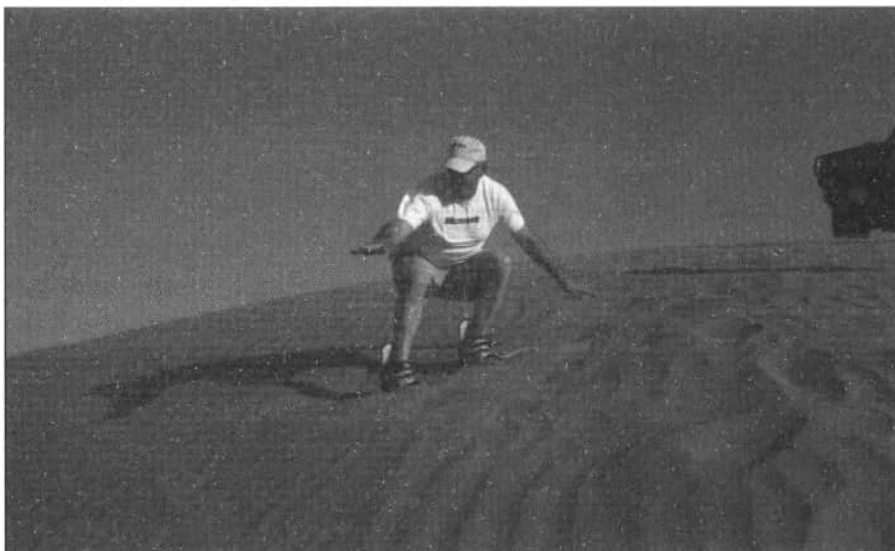


Рис. 14.6. Изображение-образец

`describe_image_in_stream` возвращает ноль или более подписей, каждая из которых имеет оценку, отражающую уверенность сервиса компьютерного зрения в том, что подпись является точной. В данном примере возвращается только одна подпись с достоверностью 53,8%.

Помимо создания подписей к изображениям, сервис Computer Vision может генерировать список ключевых слов ("тегов"), описывающих содержание изображения. Один из вариантов использования таких тегов — подготовить базу данных изображений с возможностью поиска. Используйте метод `tag_image_in_stream` (<https://oreil.ly/oJU1X>), чтобы присвоить теги приведенному в предыдущем примере изображению с подписью:

```
with open('Data/dubai.jpg', mode='rb') as image:
    result = client.tag_image_in_stream(image)

    for tag in result.tags:
        print(f'{tag.name} ({tag.confidence:.1%})')
```

Вот теги и присвоенные им уровни уверенности:

```
dune (99.5%)
sky (99.2%)
outdoor (98.7%)
```

¹ Человек, катающийся по песчаной дюне (53,8%). — Прим. ред.

clothing (98.2%)
 desert (98.1%)
 sand (97.9%)
 aeolian landform (96.9%)
 person (96.1%)
 singing sand (95.8%)
 erg (94.0%)
 sahara (93.6%)
 nature (93.4%)
 footwear (90.9%)
 landscape (88.0%)
 sand dune (83.5%)
 ground (77.5%)

Сервис Computer Vision также может обнаруживать объекты на изображениях. Следующие операторы загружают изображение и показывают все обнаруженные объекты вместе с ограничительными рамками и оценками уверенности:

```
from matplotlib.patches import Rectangle
```

```
def annotate_object(name, confidence, bbox, min_confidence=0.5):
    if (confidence > min_confidence):
        x, y, w, h = bbox.x, bbox.y, bbox.w, bbox.h
        rect = Rectangle((x, y), w, h, color='red', fill=False, lw=2)
        ax.add_patch(rect)
        text = f'{name} ({confidence:.1%})'
        ax.text(x + (w / 2), y, text, color='white',
                backgroundcolor='red', ha='center', va='bottom',
                fontweight='bold', bbox=dict(color='red'))

image = plt.imread('Data/xian.jpg')
fig, ax = plt.subplots(figsize=(12, 8), subplot_kw={'xticks': [], 'yticks': []})
ax.imshow(image)

with open('Data/xian.jpg', mode='rb') as image:
    result = client.detect_objects_in_stream(image)
```

```
for object in result.objects:
```

```
    annotate_object(object.object_property, object.confidence, object.rectangle)
```

`detect_objects_in_stream` (<https://oreil.ly/bwtVI>) обнаружил людей и велосипеды на фотографии, а также несколько других предметов (рис. 14.7).

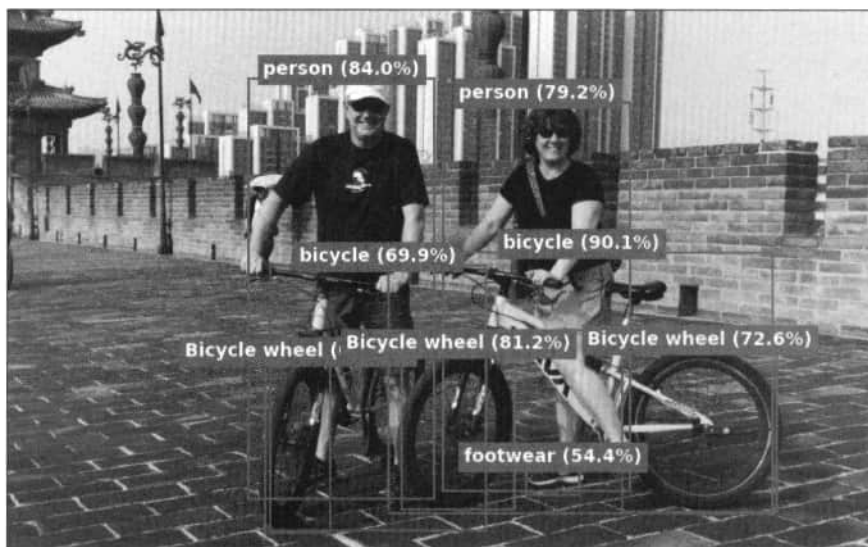


Рис. 14.7. Результат обнаружения людей и предметов на фотографии с помощью Computer Vision

Может ли сервис Computer Vision распознавать лица на фотографиях? Конечно, может. Кроме того, он может предоставить информацию о возрасте и поле. В следующем примере лица на фотографии помечены метками, обозначающими возраст и пол. В классе `ComputerVisionClient` нет специального метода для обнаружения лиц, но вы можете вызвать метод общего назначения `analyze_image_in_stream` (<https://oreil.ly/OoC7K>) с параметром `visual_features`, запрашивающим информацию о лицах:

```
from azure.cognitiveservices.vision.computervision.models \
import VisualFeatureTypes

def annotate_face(face):
    x, y = face.face_rectangle.left, face.face_rectangle.top
    w, h = face.face_rectangle.width, face.face_rectangle.height
    rect = Rectangle((x, y), w, h, color='red', fill=False, lw=2)
    ax.add_patch(rect)
    text = f'{{face.gender}} ({{face.age}})'
    ax.text(x + (w / 2), y, text, color='white', backgroundcolor='red',
            ha='center', va='bottom', fontweight='bold',
            bbox=dict(color='red'))

image = plt.imread('Data/amsterdam.jpg')
fig, ax = plt.subplots(figsize=(12, 8), subplot_kw={'xticks': [], 'yticks': []})
ax.imshow(image)
```

```

with open('Data/amsterdam.jpg', mode='rb') as image:
    result = client.analyze_image_in_stream(
        image, visual_features=[VisualFeatureTypes.faces])
    for face in result.faces:
        annotate_face(face)

```

Результат представлен на рис. 14.8.



Рис. 14.8. Результат распознавания лиц на фотографии с помощью Computer Vision



Сервис Face может распознавать лица и определять возраст, пол, эмоции, характерные черты лица и многое другое. В нем также есть методы для сравнения изображений лиц, идентификации лиц и создания систем распознавания лиц. Его API является основой API лицевых сервисов Computer Vision. Однако в результате инициативы Microsoft "Responsible AI" ("Ответственный искусственный интеллект", <https://oreil.ly/3TN8x>) сервис Face больше не доступен для широкого круга пользователей. Дополнительные сведения о службе Face и о том, как подать заявку на доступ к ней, см. в разделе "Что такое сервис Azure Face?" ("What is the Azure Face Service?", <https://oreil.ly/kZcSu>).

Предположим, вы являетесь владельцем принимающего загружаемые фотографии веб-сайта и хотите отклонять фотографии, содержащие неприемлемое содержимое. Вызванная следующим образом функция `analyze_image_in_stream` оценивает фотографию на степень взрослости (содержит ли она наготу?), приличия (содержит ли она голую кожу?) и насилия (содержит ли она кровь и насилие?) по шкале от 0,0 до 1,0.

Вот как она реагирует на фотографию молодой девушки, прыгающей с утеса в купальнике на Гавайях (рис. 14.9):

```
image = plt.imread('Data/maui.jpg')  
fig, ax = plt.subplots(figsize=(12, 8), subplot_kw={'xticks': [], 'yticks': []})  
ax.imshow(image)
```

```
with open('Data/maui.jpg', mode='rb') as image:  
    result = client.analyze_image_in_stream(image,  
        visual_features=[VisualFeatureTypes.adult])  
  
    print(f'Adulthood: {result.adult.adult_score}')  
    print(f'Raciness: {result.adult.racy_score}')  
    print(f'Goriness: {result.adult.gore_score}')  
    print(f'Is adult: {result.adult.is_adult_content}')  
    print(f'Is racy: {result.adult.is_racy_content}')  
    print(f'Is gory: {result.adult.is_gory_content}')
```

Ниже приведен результат:

Adulthood: 0.02214685082435608

Raciness: 0.4205135107040405

Goriness: 0.0016634463099762797

Is adult: False

Is racy: False

Is gory: False



Рис. 14.9. Фотография для оценивания

Бикини на фотографии получило среднюю оценку приличия, но она ниже порога 0,5, необходимого для того, чтобы `is_gacy_content` оказался `True`. Основываясь на других полученных оценках, сервис компьютерного зрения считает, что фотография не является ни "взрослой", ни жестокой.

Еще одна возможность, предоставляемая сервисом Computer Vision разработчикам приложений, — использование искусственного интеллекта для извлечения текста из фотографий. Это идеально подходит для оцифровки печатных документов. Вот пример:

```
def draw_box(bbox):
    vals = bbox.split(',')
    x = int(vals[0])
    y = int(vals[1])
    w = int(vals[2])
    h = int(vals[3])
    rect = Rectangle((x, y), w, h, color='red', fill=False, lw=2)
    ax.add_patch(rect)

image = plt.imread('Data/1040-es.jpg')
fig, ax = plt.subplots(figsize=(12, 8), subplot_kw={'xticks': [], 'yticks': []})
ax.imshow(image)

with open('Data/1040-es.jpg', mode='rb') as image:
    result = client.recognize_printed_text_in_stream(image)

    for region in result.regions:
        for line in region.lines:
            text = ' '.join([word.text for word in line.words])
            draw_box(line.bounding_box)
            print(text)
```

На рис. 14.10 представлен результат за вычетом всех строк текста, выведенных оператором `print`.

Метод `recognize_printed_text_in_stream` (<https://oreil.ly/2KhMK>) распознает только печатный текст. Связанный с ним метод `read_in_stream` (<https://oreil.ly/GNFXO>) распознает и рукописный текст. Давайте посмотрим, как он работает на том же отсканированном документе:

```
import time
from azure.cognitiveservices.vision.computervision.models \
import OperationStatusCodes

def draw_box(bbox):
    x, y = bbox[0], bbox[1]
```



```

w = bbox[4] - x
h = bbox[5] - y
rect = Rectangle((x, y), w, h, color='red', fill=False, lw=2)
ax.add_patch(rect)

image = plt.imread('Data/1040-es.jpg')
fig, ax = plt.subplots(figsize=(12, 8), subplot_kw={'xticks': [], 'yticks': []})
ax.imshow(image)

with open('Data/1040-es.jpg', mode='rb') as image:
    response = client.read_in_stream(image, raw=True)

    location = response.headers["Operation-Location"]
    opid = location[len(location) - 36:]
    results = client.get_read_result(opid)

    while results.status == OperationStatusCodes.running:
        results = client.get_read_result(opid)
        time.sleep(1)

    if results.status == OperationStatusCodes.succeeded:
        for result in results.analyze_result.read_results:
            for line in result.lines:
                draw_box(line.bounding_box)
                print(line.text)

```

1040-ES Department of the Treasury Internal Revenue Service		2022 Estimated Tax		Payment Voucher 4 OMB No. 1545-0074	
File only if you are making a payment of estimated tax by check or money order. Mail this voucher with your check or money order payable to "United States Treasury." Write your social security number and "2022 Form 1040-ES" on your check or money order. Do not send cash. Enclose, but do not staple or attach, your payment with this voucher.				Calendar year—Due Jan. 17, 2023 Amount of estimated tax you are paying by check or money order \$ 100	
Print or type	Your first name and middle initial Jeff		Your last name Proise		Your social security number 111-22-0000
	If joint payment, complete for spouse				
	Spouse's first name and middle initial		Spouse's last name		Spouse's social security number
	Address (number, street, and apt. no.) 1313 Mockingbird Lane				
	City, town, or post office. If you have a foreign address, also complete spaces below Oak Ridge			State TN	ZIP code 37830
Foreign country name		Foreign province/county		Foreign postal code	
For Privacy Act and Paperwork Reduction Act Notice, see instructions.					Form 1040-ES (2022)

Рис. 14.10. Результат оцифровки печатного документа без учета рукописного текста

Этот пример немного сложнее, потому что `read_in_stream` возвращается до завершения вызова. Поэтому мы выполняем цикл до завершения вызова, а затем получаем результаты. Вызов `sleep` в каждой итерации цикла `while` позволяет пользовательскому интерфейсу оставаться реагирующим в ожидании завершения вызова. Результаты стоят того, чтобы подождать (рис. 4.11).

1040-ES Department of the Treasury Internal Revenue Service **2022 Estimated Tax** **Payment Voucher** **4** CMB No. 1545-0074

File only if you are making a payment of estimated tax by check or money order. Mail this voucher with your check or money order payable to "United States Treasury." Write your social security number and "2022 Form 1040-ES" on your check or money order. Do not send cash. Enclose, but do not staple or attach, your payment with this voucher.

Calendar year—Due Jan. 17, 2023
Amount of estimated tax you are paying by check or money order **\$100**

Your first name and middle initial **Jeff** Your last name **Prosisie** Your social security number **111-22-0000**

If joint payment, complete for spouse
Spouse's first name and middle initial _____ Spouse's last name _____ Spouse's social security number _____

Address (number, street, and apt. no.) **1313 Mockingbird Lane**
City, town, or post office. If you have a foreign address, also complete spaces below. **Oak Ridge** State **TN** ZIP code **37830**

Foreign country name _____ Foreign province/county _____ Foreign postal code _____

For Privacy Act and Paperwork Reduction Act Notice, see instructions. **Form 1040-ES (2022)**

Рис. 14.11. Результат оцифровки печатного документа с учетом рукописного текста

Возможно, вы предпочтете извлечь из документа только рукописный текст. Вы можете сделать и это, поскольку для каждой обнаруженной строки текста метод `read_in_stream` включает атрибут `style`, равный "handwriting" для рукописного текста. Для того чтобы проверить это, замените две последние строки в предыдущем примере следующими строками:

```
if (line.appearance.style.name == 'handwriting'):
    draw_box(line.bounding_box)
    print(line.text)
```

На этот раз выделен только рукописный текст (рис. 4.12).

`read_in_stream` отлично справляется с преобразованием рукописного текста в строки Python:

```
$ 100
Jeff
Prosisie
111-22-0000
1313 Mockingbird Lane
Oak Ridge
TN
37830
```

Form 1040-ES Department of the Treasury Internal Revenue Service	2022 Estimated Tax		Payment Voucher 4 OMB No. 1545-0074
File only if you are making a payment of estimated tax by check or money order. Mail this voucher with your check or money order payable to "United States Treasury." Write your social security number and "2022 Form 1040-ES" on your check or money order. Do not send cash. Enclose, but do not staple or attach, your payment with this voucher.		Calendar year—Due Jan. 17, 2023 Amount of estimated tax you are paying by check or money order. \$100	
Print or type	Your first name and middle initial Jeff	Your last name Prosize	Your social security number 11-22-0000
	If joint payment, complete for spouse		
	Spouse's first name and middle initial	Spouse's last name	Spouse's social security number
	Address (number, street, and apt. no.) 1313 Mockingbird Lane		
	City, town, or post office. If you have a foreign address, also complete spaces below. Oak Ridge		State TN
Foreign country name		Foreign province/county	
		Foreign postal code	
For Privacy Act and Paperwork Reduction Act Notice, see instructions.			Form 1040-ES (2022)

Рис. 14.12. Результат оцифровки только рукописного текста

И последнее замечание по поводу сервиса Computer Vision: он не принимает изображения размером более 4 Мбайт. Все, что больше, приводит к исключению `ComputerVisionErrorResponse`. Эти исключения (или исключения `AzureError`, которые находятся выше в пищевой цепочке) можно отлавливать и аккуратно обрабатывать, а можно проверять размер изображения перед отправкой и уменьшать его при необходимости.

Сервис Language

В сервисе Computer Vision используются модели глубокого обучения, аналогичные тем, о которых рассказывалось в главах 10–12. Сервисы Language и Translator используют модели, подобные тем, что описаны в главе 13. Последние два сервиса воплощают в себе NLP: анализ настроения, нейронный машинный перевод, ответы на вопросы и многое другое. Класс `TextAnalyticsClient` в Python text analytics SDK (<https://oreil.ly/gkD9c>) предоставляет удобный интерфейс для многих функций сервиса Language. Вы уже видели, как он используется для анализа настроений. Вот еще один пример, в котором анализ настроения применяется к нескольким образцам текста с одним обращением к сервису Language:

```
from azure.core.credentials import AzureKeyCredential
from azure.ai.textanalytics import TextAnalyticsClient
```

```
client = TextAnalyticsClient(ENDPOINT, AzureKeyCredential(KEY))
```

```
input = [
    { 'id': '1000', 'text': 'Programming is fun, but the hours are long' },
    { 'id': '1001', 'text': 'Great food and excellent service' },
```

```
{ 'id': '1002', 'text': 'The product worked as advertised but is ' \
    'overpriced' },
{ 'id': '1003', 'text': 'Moving to the cloud was the best decision ' \
    'we ever made' },
{ 'id': '1004', 'text': 'Programming is so fun I\'d do it for free. ' \
    'Don\'t tell my boss!' }
]
response = client.analyze_sentiment(input)
```

```
for result in response:
    text = ''.join([x.text for x in result.sentences])
    print(f'{text} => {result.confidence_scores.positive}')
```

Вот результат:

```
Programming is fun, but the hours are long => 0.94
Great food and excellent service => 1.0
The product worked as advertised but is overpriced => 0.0
Moving to the cloud was the best decision we ever made => 1.0
Programming is so fun I'd do it for free. Don't tell my boss! => 1.0
```

Допустим, вы пишете приложение, собирающее твиты с упоминанием вашей компании и анализирующее их на предмет настроения. Задача заключается в том, чтобы в случае негативного настроения предупредить отдел маркетинга. Быстрее и эффективнее сделать один вызов Azure Cognitive Services и проанализировать 100 твитов, чем делать 100 вызовов, анализируя по одному твиту за раз.

Анализ настроения — одна из нескольких операций, поддерживаемых классом `TextAnalyticsClient`. Другой операцией является распознавание именованных сущностей. Предположим, вы создаете систему, которая сортирует и расставляет приоритеты для заявок, поступающих в службу поддержки вашей компании. Метод `recognize_entities` (<https://oreil.ly/UtuZx>) извлекает из входного текста такие сущности, как люди, места, организации, даты и время, а также количества. Он также показывает типы сущностей:

```
documents = [
    'My printer isn\'t working. Can someone from IT come to my office ' \
    'and have a look?'
]

results = client.recognize_entities(documents)
```

```
for result in results:
    for entity in result.entities:
        print(f'{entity.text} ({entity.category})')
```

Результаты данного примера выглядят следующим образом:

```
printer (Product)
IT (Skill)
office (Location)
```

Связанный с ним метод `recognize_pii_entities` (<https://oreil.ly/nOGsr>) извлекает из вводимого текста персонально идентифицируемые данные (personal identifiable information, PII), такие как информация о банковском счете, номера социального страхования и кредитных карт, а метод `extract_key_phrases` (<https://oreil.ly/Au69j>) извлекает ключевые фразы:

```
documents = [
    'Natural Language Processing, or NLP, encompasses a variety of ' \
    'activities including text classification, keyword extraction,' \
    'named-entity recognition, question answering, and language '\
    'translation.'
]
```

```
results = client.extract_key_phrases(documents)
```

```
for result in results:
    for phrase in result.key_phrases:
        print(phrase)
```

Ниже приведен результат:

```
Natural Language Processing
language translation
text classification
keyword extraction
question answering
NLP
variety
activities
recognition
```

Этот пример довольно простой, поскольку текст очень короткий, но вы можете передать сотни больших документов и использовать результаты для получения слепка содержания каждого документа или группировки документов, содержащих похожие ключевые слова.

`TextAnalyticsClient` обеспечивает обертку для API текстовой аналитики. Другие SDK для Python открывают дополнительные возможности сервиса Language. Например, SDK для ответов на вопросы (<https://oreil.ly/h85Bc>) предоставляет API для ответов на вопросы из руководств, FAQ, записей в блогах и других предоставленных вами документов. Для получения дополнительной информации и просмотра этого аспекта службы Language в действии обратитесь к статье "Клиентская биб-

библиотека Azure Cognitive Language Services Question Answering Client Library for Python" ("Azure Cognitive Language Service Question Answering client library for Python", <https://oreil.ly/J6kqg>).

Сервис Translator

Сервис Translator использует самый современный нейронный машинный перевод для перевода текста с десятков языков. Он также может распознавать рукописные тексты. Предположим, имеется задача — перевести на английский вопросы, написанные на других языках и заданные через веб-сайт вашей компании. Сначала нужно определить, является ли исходный язык английским. Если нет, то нужно перевести его, чтобы можно было ответить на запрос клиента.

Следующий код анализирует образец текста и показывает, на каком языке он написан. В настоящее время Microsoft не предлагает Python SDK для службы Translator, но вы можете использовать пакет Requests в Python для упрощения вызовов. Для того чтобы изучить эту тему, создайте ресурс Translator в Azure Portal и получите ключ подписки (тот, что помечен как Text Translation), конечную точку и регион. Затем выполните следующий код, заменив KEY и ENDPOINT ключом и конечной точкой, а REGION — выбранным вами регионом Azure (например, southcentralus):

```
import requests

input = [{ 'text': 'Quand votre nouveau livre sera-t-il disponible?' }]

headers = {
    'Ocp-Apim-Subscription-Key': KEY,
    'Ocp-Apim-Subscription-Region': REGION,
    'Content-type': 'application/json'
}

uri = ENDPOINT + 'detect?api-version=3.0&to=en'
response = requests.post(uri, headers=headers, json=input)
results = response.json()

print(results[0]['language'])

На выходе получается fr — французский. Теперь, когда определено, что исходный
язык — это не английский, можно перевести его следующим образом:
uri = ENDPOINT + 'translate?api-version=3.0&from=fr&to=en'
response = requests.post(uri, headers=headers, json=input)
results = response.json()

print(results[0]['translations'][0]['text'])
```

Текст переведен следующим образом:

When will your new book be available?²

Можно опустить `from=fr` в URL-адресе и позволить сервису Translator определить язык самостоятельно. Также можно определить язык и перевести текст одним запросом. Если вызвать конечную точку `translate` без параметра `from`, возвращаемое значение будет включать элемент `detectedLanguage`, который идентифицирует язык исходного текста:

```
{'detectedLanguage': {'language': 'fr', 'score': 1.0}, 'translations': [{'text': 'When will your new book be available?', 'to': 'en'}]}
```

Если передать список, содержащий несколько текстов, сервис Translator переведет их все за один вызов. Он также поддерживает *транслитерацию* — перевод текста на другие алфавиты. Для того чтобы убедиться в этом, переведите с помощью Google Translate вопрос "When will your new book be available" на тайский или хинди. Затем вставьте текст на тайском или хинди поверх французского текста в предыдущем примере и запустите его снова. Не забудьте также изменить `from=fr` на `from=th` или `from=hi` или вообще убрать параметр `from`.



Когда вы создаете ресурс Translator, у вас есть выбор: создать глобальный ресурс или привязанный к определенному региону Azure. В предыдущих примерах предполагалось, что вы создали его как региональный ресурс. Если вы создали глобальный ресурс Translator, можете установить для заголовка `Ocp-Apim-Subscription-Region` значение `global` или вообще отказаться от него.

Когда вы получали конечную точку для службы Translator, заметили ли вы, что Azure Portal предлагает две конечные точки — одну для Text Translation и другую для Document Translation? Это связано с тем, что служба Translator имеет второй API для перевода целых документов, включая PDF-файлы. Для этого есть даже Python SDK (<https://oreil.ly/uuVmJ>). Единственная загвоздка заключается в том, что документы сначала должны быть загружены в хранилище `blob-объектов`³ Azure. Примеры, демонстрирующие API перевода документов в действии, см. в статье "Клиентская библиотека перевода документов Azure для Python" ("Azure Document Translation Client Library for Python", <https://oreil.ly/RFTbT>). API является асинхронным и может обрабатывать партии документов за один вызов, поэтому он идеально подходит не только для перевода отдельных документов, но и для перевода больших массивов документации.

Сервис Speech

Одна из самых сложных задач для моделей глубокого обучения — обработка человеческой речи. В Azure Cognitive Services есть сервис Speech (<https://oreil.ly/NHBcG>),

² Когда выйдет твоя новая книга? — Прим. ред.

³ Binary Large Object — двоичный большой объект. — Прим. ред.

преобразующий текст в речь, речь в текст и многое другое. Он даже способен создавать субтитры к записанным видео и транслируемым в реальном времени видеопотокам (<https://oreil.ly/1x9gU>), отфильтровывая при этом ненормативную лексику. SDK на языке Python (<https://oreil.ly/Heaxx>) упрощает написание кода и позволяет легко внедрять речь в приложения.

Для изучения установите пакет `Azure-cognitiveservices-speech` (<https://oreil.ly/4cwJo>), содержащий Python Speech SDK. С помощью портала Azure Portal создайте ресурс Cognitive Services Speech и запишите ключ подписки и регион обслуживания. Затем создайте блокнот Jupyter и выполните следующий код в первой ячейке, заменив KEY ключом подписки, а REGION — выбранным вами регионом:

```
from azure.cognitiveservices import speech
```

```
speech_config = speech.SpeechConfig(KEY, REGION)
speech_config.speech_recognition_language = 'en-US'
```

Теперь выполните следующие операторы и, когда потребуется, произнесите слова в микрофон. Этот пример создает объект класса `SpeechRecognizer` (<https://oreil.ly/ZUKOC>) и использует его метод `recognize_once_async` (<https://oreil.ly/NY681>) для преобразования до 30 секунд живого звука с микрофона на вашем компьютере в текст. Обратите внимание, что текст появляется только после того, как вы закончите говорить:

```
recognizer = speech.SpeechRecognizer(speech_config)
```

```
print('Speak into your microphone')
result = recognizer.recognize_once_async().get()
```

```
if result.reason == speech.ResultReason.RecognizedSpeech:
    print(result.text)
```

Проще некуда! А как насчет преобразования текста в речь? Вот пример, использующий класс `SpeechSynthesizer` (<https://oreil.ly/3qC4H>) SDK для озвучивания предложения. Синтезированный голос принадлежит англоговорящей Дженни (`en-US-JennyNeural`), и это один из более чем 300 нейронных голосов (<https://oreil.ly/mXyka>), которые вы можете выбрать:

```
speech_config.speech_synthesis_voice_name = 'en-US-JennyNeural'
synthesizer = speech.SpeechSynthesizer(speech_config)
synthesizer.speak_text_async('When will your new book be published?').get()
```

Все "дикторы" являются многоязычными. Если вы попросите французского диктора, например `fr-FR-CelesteNeural`, синтезировать английское предложение, в его вокализации будет присутствовать французский акцент.

Вы можете объединить объект `TranslationRecognizer` (<https://oreil.ly/R38mY>) с объектом `SpeechSynthesizer`, чтобы переводить речь в реальном времени. В следующем

примере на вход подается английская речь, а воспроизводится она на французском языке с использованием голоса носителя французского языка:

```
speech_config.speech_synthesis_voice_name = 'fr-FR-YvetteNeural'
synthesizer = speech.SpeechSynthesizer(speech_config)
```

```
translation_config = speech.translation.SpeechTranslationConfig(KEY, REGION)
translation_config.speech_recognition_language = 'en-US'
translation_config.add_target_language('fr')
```

```
recognizer = speech.translation.TranslationRecognizer(translation_config)
```

```
print('Speak into your microphone')
result = recognizer.recognize_once_async().get()
```

```
if result.reason == speech.ResultReason.TranslatedSpeech:
    text = result.translations['fr']
    synthesizer.speak_text_async(text).get()
```

Эти примеры используют стандартный микрофон вашего компьютера для ввода голоса и стандартные динамики для вывода. Вы можете указать другие источники ввода и вывода, передав объект `AudioConfig` (<https://oreil.ly/tj43k>) методам, создающим объекты — `SpeechRecognizer`, `SpeechSynthesizer` и `TranslationRecognizer`. Среди опций, которые можно задать, — использование файла или потока, а не микрофона в качестве источника входного сигнала.

Собираем все вместе: Contoso Travel

Представьте, что вы — веб-разработчик, а ваш клиент — компания Contoso Travel. Для того чтобы мотивировать своих клиентов оставаться на связи, туристическое агентство хочет, чтобы его веб-сайт включал переводящий вывески и знаки сервис. Идея заключается в том, чтобы путешествующий по далеким странам клиент мог сфотографировать вывеску, загрузить ее на сайт и увидеть перевод на выбранный им язык. Никакого набора текста, никаких форм для заполнения — просто "Вот фотография, скажите мне, что на ней написано".

Еще несколько лет назад такой сайт был бы немыслим для большинства малых предприятий. Сегодня это под силу любому, кто может написать несколько строк кода. Давайте воспользуемся фреймворком Flask (<https://oreil.ly/WqqwN>) от Python для создания сайта, который сделает вашего клиента счастливым. Затем используем Azure Cognitive Services для извлечения и перевода текста, и в итоге получим продукт, показанный на рис. 14.13.

Для начала убедитесь, что установлены все необходимые пакеты, включая Flask, Requests и Azure-cognitiveservices-vision-computervision. Затем создайте каталог

проекта в выбранном вами месте на жестком диске. Назовите его Contoso или как-нибудь еще. Затем загрузите ZIP-файл (<https://oreil.ly/KmFZD>) со стартовым набором для веб-сайта и скопируйте его содержимое в каталог проекта. Уделите время изучению скопированных файлов. Эти файлы представляют собой веб-сайт, написанный на Python и Flask. Они включают в себя следующее:

app.py

Содержит написанный на Python код, который управляет сайтом.

templates/index.html

Содержит главную страницу сайта.

static/main.css

Содержит CSS для оформления главной страницы.

static/banner.jpg

Содержит баннер сайта.

static/placeholder.jpg

Содержит изображение-заглушку для еще не загруженных фотографий.

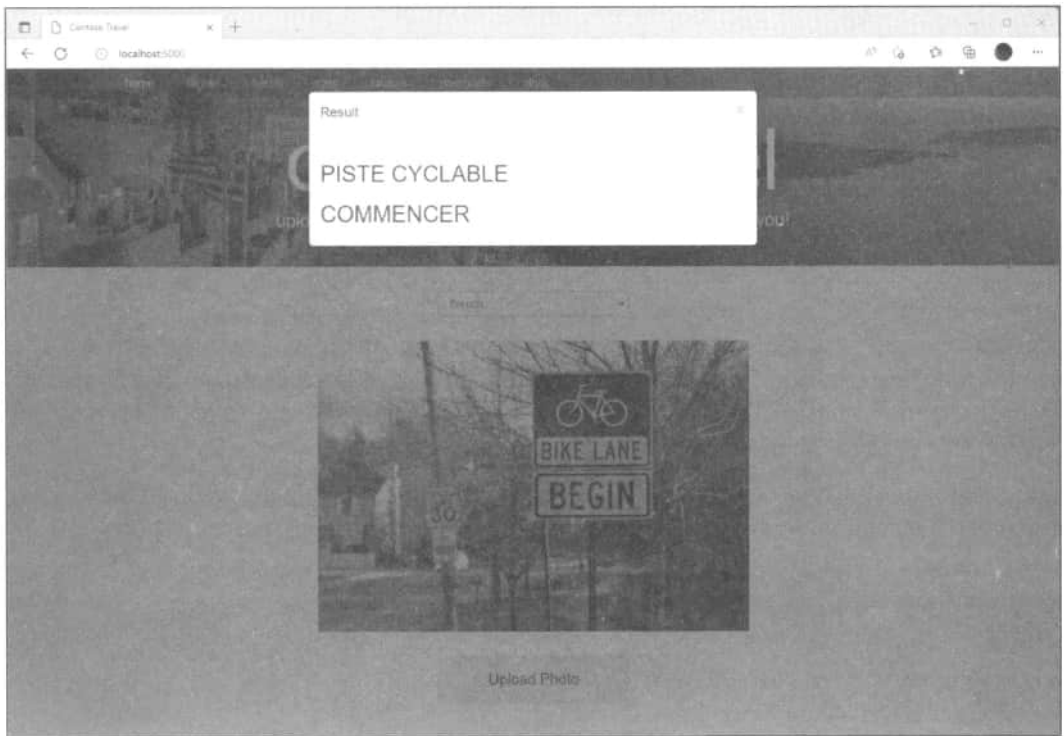


Рис. 14.13. Веб-сайт Contoso Travel переводит дорожный знак

Сайт содержит одну страницу *index.html*, отображаемую при переходе пользователя на сайт. В *app.py* находится отображающий страницу код, который включает

логику загрузки фотографий. Файл `app.py` надо изменить, чтобы извлечь и перевести текст из загружаемых пользователями фотографий.



Вам знакомо слово "Contoso"? Это название компании, используемое в бесчисленных примерах и руководствах Microsoft. Есть причина такой популярности. Если вы пишете материалы для Microsoft, нельзя придумывать названия компаний, потому что для Microsoft это может привести к судебным разбирательствам. Contoso — одно из нескольких вымышленных названий компаний (<https://oreil.ly/7PszJ>), которые Microsoft запатентовала в течение многих лет. Среди других, которые могут быть вам знакомы, — Northwind Traders, Fabrikam и AdventureWorks.

Откройте командную строку или окно терминала и перейдите по `cd` в каталог проекта. Если вы работаете в Windows, используйте следующую команду для создания переменной окружения `FLASK_ENV`, указывающей Flask на запуск в режиме разработки:

```
set FLASK_ENV=development
```

Если вы работаете под управлением Linux или macOS, используйте эту команду:

```
export FLASK_ENV=development
```

Запуск Flask в режиме разработки полезен при создании сайта, поскольку Flask автоматически перезагружает все файлы, изменившиеся во время работы сайта. Если вы переведете Flask в режим производства и измените содержимое HTML-файла или другого актива, придется перезапустить Flask, чтобы изменения появились в браузере.

Теперь выполните следующую команду, чтобы запустить Flask:

```
flask run
```

Откройте браузер и перейдите на сайт <http://localhost:5000>. Когда появится главная страница Contoso Travel, нажмите кнопку **Upload Photo** (Загрузить фотографию) и выберите файл JPG или PNG на своем компьютере. Убедитесь, что фотография загружается без ошибок. На данном этапе обработка фотографии не выполняется, поэтому единственным визуальным признаком успешной загрузки является появление фотографии на веб-странице.

Далее воспользуйтесь порталом Azure Portal, чтобы создать ресурс Computer Vision и ресурс Translator, если это еще не сделано. Откройте `app.py` в своем любимом редакторе кода и найдите следующие операторы в верхней части файла:

```
# Определение переменных Cognitive Services
```

```
vision_key = 'VISION_KEY'
```

```
vision_endpoint = 'VISION_ENDPOINT'
```

```
translator_key = 'TRANSLATOR_KEY'
```

```
translator_endpoint = 'TRANSLATOR_ENDPOINT'
```

```
translator_region = 'TRANSLATOR_REGION'
```

Замените `VISION_KEY` и `TRANSLATOR_KEY` ключами подписки сервисов, `VISION_ENDPOINT` и `TRANSLATOR_ENDPOINT` — конечными точками, а `TRANSLATOR_REGION` — выбранным для

сервиса Translator регионом. Для Translator используйте конечную точку с надписью **Text Translation** (Перевод текста). Не забудьте оставить нетронутыми метки, разделяющие строки.

Теперь добавьте в `app.py` следующую функцию для извлечения текста из фотографии, поместив ее сразу после комментария "Function that extracts text from images" (Функция, извлекающая текст из изображений) в нижней части файла:

```
def extract_text(endpoint, key, image):
    try:
        client = ComputerVisionClient(endpoint, CognitiveServicesCredentials(key))
        result = client.recognize_printed_text_in_stream(image)
        lines = []

        for region in result.regions:
            for line in region.lines:
                text = ' '.join([word.text for word in line.words])
                lines.append(text)

        if len(lines) == 0:
            lines.append('Photo contains no text to translate')
        return lines

    except ComputerVisionErrorResponseException as e:
        return ['Error calling the Computer Vision service: ' + e.message]

    except Exception as e:
        return ['Error calling the Computer Vision service']
```

Затем добавьте следующую функцию для перевода текста. Поместите ее после комментария "Function that translates text into a specified language" ("Функция, переводящая текст на указанный язык") в нижней части файла `app.py`:

```
def translate_text(endpoint, region, key, lines, language):
    try:
        headers = {
            'Ocp-Apim-Subscription-Key': key,
            'Ocp-Apim-Subscription-Region': region,
            'Content-type': 'application/json'
        }

        input = []

        for line in lines:
            input.append({ "text": line })
```

```

uri = endpoint + 'translate?api-version=3.0&to=' + language
response = requests.post(uri, headers=headers, json=input)
response.raise_for_status() # Выбросить исключение, если запрос неуспешен
results = response.json()

translated_lines = []

for result in results:
    for translated_line in result["translations"]:
        translated_lines.append(translated_line["text"])

return translated_lines

except requests.exceptions.HTTPError as e:
    return ['Error calling the Translator service: ' + e.strerror]

except Exception as e:
    return ['Error calling the Translator service']

```

Найдите функцию с именем `index` в файле `app.py`. Ее вызов происходит при запросе домашней страницы или загрузке фотографии. Под комментарием "Use the Computer Vision service to extract text from the image" ("Используйте службу компьютерного зрения для извлечения текста из изображения") добавьте следующую строку кода для вызова `extract_text` с фотографией, которую только что загрузил пользователь:

```
lines = extract_text(vision_endpoint, vision_key, image)
```

Под комментарием "Use the Translator service to translate text extracted from the image" ("Используйте службу Translator для перевода текста, извлеченного из изображения") добавьте строку кода, чтобы перевести полученный с помощью `extract_text` текст на указанный язык:

```
translated_lines = translate_text(translator_endpoint, translator_region,
                                  translator_key, lines, language)
```

Добавьте следующие утверждения сразу после комментария "Flash the translated text" ("Отметить переведенный текст"):

```
for translated_line in translated_lines:
    flash(translated_line)
```

Эти утверждения используют поддержку Flask для отображения строк в `translated_lines` в модальном диалоге.

Завершите работу, сохранив изменения в `app.py`. Вернитесь в браузер, в котором запущен сайт, и обновите страницу (если вы закрыли ее, откройте новый экземпляр браузера и перейдите по адресу <http://localhost:5000>). Выберите язык из выпадаю-

шего списка в верхней части страницы. Затем загрузите фотографию с дорожным знаком или любое другое изображение с текстом.

Удалось ли извлечь и перевести текст? Приложение не идеально, но в большинстве случаев оно должно работать как надо. Если вы не удовлетворены, попробуйте заменить вызовы `recognize_printed_text_in_stream` на вызовы `read_in_stream`. Последний более агрессивен в поиске текста на фотографиях. В процессе работы можно также использовать службу Translator для перевода сообщений об ошибках на выбранный пользователем язык или службу Speech для озвучивания переведенного текста. С помощью Azure Cognitive Services все ограничивается только вашим изображением.

Резюме

Сервисы Azure Cognitive Services снижают планку для внедрения сложных моделей глубокого обучения в создаваемые приложения путем предоставления REST API, к которым может обратиться любое имеющее подключение к Интернету приложение. Контейнерные версии большинства сервисов позволяют размещать их локально или на территории предприятия, а также предоставляют возможность работы в изолированных сценариях. Одним из преимуществ контейнеров является возможность привязки к конкретной версии используемых моделей или API, которые к ним обращаются, так что изменения в Azure Cognitive Services не повлияют на ваши приложения.

Спектр предлагаемых Azure Cognitive Services сервисов включает в себя сервисы технического зрения для извлечения информации из изображений и видеопотоков, языковые сервисы для перевода текста на другие языки и выполнения прочих задач NLP, а также речевые сервисы для внедрения речи в ваши приложения. Для большинства служб доступны бесплатные SDK. Они упрощают написание кода и реализованы для множества популярных языков программирования, включая Python, Java и C#.

Машинное обучение и глубокое обучение делают мир лучше с каждой моделью. Скоро они станут частью набора необходимых навыков каждого инженера и разработчика программного обеспечения. Готовы ли вы справиться с этой задачей? Я надеюсь, что эта книга даст вам уверенность, чтобы сказать "да", и инструменты, чтобы сделать это. Уже слишком поздно следить за миссиями "Аполлона" в Центре управления полетами, но другие бесчисленные рубежи ждут своих исследователей.

Предметный указатель

A

Activation function 207
Adam, класс, оптимизатор 217
AgglomerativeClustering, класс 46
AlexNet 247
AlphaGo 34
Amazon Polly 393
Anchor 324

- ◊ boxes 324

ArcFace 310
Artificial intelligence (AI). См. Искусственный интеллект
AudioConfig, класс 418
Average pooling 249
AWS AI Services 392
Azure AI Vision 246
Azure Cognitive Services 337, 392
Azure Multivariate Anomaly Detector 178

B

Bag of words 107
Bidirectional encoder representations from transformers (BERT) 381
Bilingual evaluation understudy (BLEU) 377

C

California housing dataset, набор данных 71
Callback, класс 242
Caret 201
CascadeClassifier, класс 294
COCO, набор данных 326
Computer Vision Service 393, 402
ComputerVisionClient, класс 403, 406
Confusion matrix. См. Матрица ошибок
ConfusionMatrixDisplay, класс 88
Conv1D, класс 251
Conv2D, класс 250
Conv3D, класс 251
Convolution kernel 247

Convolutional neural network (CNN) 206, 247

- ◊ региональная 321

CountVectorizer, класс 108, 111, 182, 195, 350
CSVLogger, класс 243
Custom Vision service 337

D

Data view 195
DataOperationsCatalog, класс 195
DBSCAN:

- ◊ класс 46
- ◊ кластеризация 46

DecisionTreeClassifier, класс 63, 82
DecisionTreeRegressor, класс 63, 68
Deep learning (DL). См. Обучение глубокое
Deep neural network 206
Dense, класс 216
Digits, набор данных 166, 167
Docker 180, 187
Dropout. См. Отсев
Dropout, класс 239

E

EarlyStopping, класс 242
Embedding, класс 353

F

F1-мера 85
Face:

- ◊ embedding 310
- ◊ verification 310

Flask 180, 185, 418

G

Generative adversarial network (GAN) 206
GitHub 55
GlorotUniform, класс, инициализатор 218

Google Colab 377
 Gradient-boosted decision trees (GBDT) 67
 Gradient-boosting machines (GBM) 48, 67
 GradientBoostingClassifier, класс 68, 82
 GradientBoostingRegressor, класс 68
 Graphics processing unit (GPU) 35, 205
 GridSearchCV, класс 136
 GRU, класс 363

H

H2O 193
 H5, формат 240
 HashingVectorizer, класс 108, 111
 Hugging Face 364

I

IDataView, интерфейс 195
 ImageDataGenerator, класс 272
 ImageNet, набор данных 260
 Instance segmentation 324
 Intersection-over-union (IoU) 321
 Iris, набор данных 52
 Isomap, класс 165

J

Java-ML 201

K

Keras 213, 215

- ◊ API для нейронных сетей:
 - последовательный 215
 - функциональный 215
- ◊ decode_predictions, метод 262
- ◊ get_weights, метод 240
- ◊ pad_sequences, функция 352
- ◊ preprocess_input, функция 262
- ◊ set_weights, метод 240
- ◊ инициализаторы 218
- ◊ обратные вызовы 241
- ◊ оптимизаторы 217

 KerasNLP, пакет 371
 Kernel trick 69
 k-means clustering 36
 KMeans, класс 38
 k-nearest neighbors 49
 KNeighborsClassifier, класс 51, 54, 82
 KNeighborsRegressor, класс 51
 Kubernetes 187

L

Labeled Faces in the Wild (LFW), набор данных 33, 143, 154, 235, 302
 Language Service 412
 Lasso, класс 59
 LearningRateScheduler, класс 243
 Limited-memory Broyden — Fletcher — Goldfarb — Shanno (L-BFGS) 83
 LinearRegression, класс 59
 LinearSVC, классификатор 148
 LoadColumnAttribute, класс 196
 Log loss 226
 LogisticRegression, класс 83, 113, 182, 195
 LogisticRegressionCV, класс 84
 Long Short-Term Memory (LSTM) 363
 LSTM, класс 363

M

Machine learning. См. Обучение машинное
 Mask R-CNN 324
 Masked language modeling (MLM) 381
 Matplotlib 37, 163
 Max pooling 249
 MaxPooling2D, класс 250
 Mean absolute error (MAE) 217
 Mean average precision (mAP) 344
 Mean squared error (MSE) 58, 74, 217
 Microsoft Cognitive Toolkit (CNTK) 215
 Mini-batch gradient descent (MBGD) 213
 MinMaxScaler, класс 138
 ML.NET 193, 194

- ◊ книга рецептов 194
- ◊ репозиторий GitHub с примерами 194

 MLContext, класс 195

- ◊ Model, свойство 197

 MLPClassifier, класс 208
 MLPRegressor, класс 208
 MNIST, набор данных 251
 MobileNetV2, класс 261
 ModelCheckpoint, класс 243
 ModelOperationsCatalog, класс:

- ◊ CreatePredictionEngine, метод 195
- ◊ Load, метод 197
- ◊ Save, метод 197

 Mplot3D 164
 Multi-head attention (МНА) 369
 MultiHeadAttention, класс 371

Multilayer perceptron 206
 MultinomialNB, класс 118, 120
 Multitask cascaded convolutional neural networks (MTCNN) 296

N

Naive Bayes 115
 Natural language processing (NLP) 206, 349
 Natural Language Toolkit (NLTK) 110
 Neural machine translation (NMT) 366
 Non-maximum suppression (NMS) 321
 NumPy 254

- ◊ `tensor.dot`, функция 254

 NuSVC, классификатор 148
n-грамма 109

O

One-hot encoding 89
 OneHotEncoder, класс 90
 one-vs-rest 101
 Open Neural Network Exchange (ONNX) 181, 189, 325
 OpenCV 294
 Optical Recognition of Handwritten Digits, набор данных 102, 164
 Output network (O-Net) 296

P

Pandas 180
 Perceptron, класс 208
 PolynomialFeatures, класс 59
 Pooling layer 247
 Precision. См. Точность
 PredictionEngineBase<Tsrc,TDst>, класс:

- ◊ `Predict`, метод 195

 Principal component analysis (PCA) 60, 151
 Proposal network (P-Net) 296
 Python Imaging Library (PIL) 301
 Python Librosa 283
 PyTorch 215

R

Radial basis function (RBF) 131
 RadiusNeighborsClassifier, класс 52, 55
 RadiusNeighborsRegressor, класс 51
 RandomForestClassifier, класс 65, 82
 RandomForestRegressor, класс 65

Recall. См. Полнота
 Rectified linear unit (ReLU) 208
 Recurrent neural network (RNN) 206
 Refine network (R-Net) 296
 Region of interest (ROI) 323
 Region proposal network (RPN) 323
 Region-based/region CNN (R-CNN) 321
 Rekognition 393
 ReLU 220
 ResNet-152 247
 ResNet50V2 262
 REST API 180, 185, 392
 Ridge, класс 59
 RocCurveDisplay, класс 87
 Rules-based machine translation (RBMT) 366

S

SavedModel 240
 Scaper 282
 Scikit-Learn 36, 180
 Scree plot 157
 SdcaLogisticRegressionBinaryTrainer, класс 195
 Seaborn 60
 Segmentation mask 324
 Sensitivity. См. Чувствительность
 Sequential, класс 216

- ◊ `add`, функция 216

 Similarity matrix 124
 Specificity. См. Специфичность
 Speech 393
 Speech Service 416
 SpeechRecognizer, класс 417
 SpeechSynthesizer, класс 417
 SQuAD 2.0, набор данных 383
 StandardScaler, класс 138, 141
 StandardTrainersCatalog, класс:

- ◊ `SdcaLogisticRegression`, метод 195

 Stochastic gradient descent (SGD) 213
 StopCallback, класс 242
 Stop-words, библиотека 110
 Supervised learning 47, 168
 Support vector classifier (SVC) 128
 Support vector machines (SVM) 48, 69, 128, 168
 Support vector regressor (SVR) 128
 SVC, класс 70, 128
 SVR, класс 70, 128

T

t-Distributed stochastic neighbor embedding
(*t*-SNE) 60, 165
Tensor processing unit (TPU) 205
TensorBoard:
 ◇ инструмент 243
 ◇ класс 243
TensorFlow 213, 215
TensorFlow Lite 281
TextAnalyticsClient, класс 412, 413
TextCatalog, класс:
 ◇ FeaturizeText, метод 195
TextFeaturizingEstimator, класс 195
Text-to-Speech AI 393
TfidfVectorizer, класс 108
Theano 215
Tidymodels 201
Titanic, набор данных 91
Tokenizer, класс 350, 351
Transfer learning 193, 265
Transformers, пакет 365

TranslationRecognizer, класс 417
Translator Service 415
TSNE, класс 165

U

UrbanSound8K, набор данных 282
User-defined function (UDF) 197

V

VGGFace 310
 ◇ класс 308

X

Xlwtings, библиотека 197

Y

You Only Look Once (YOLO) 331

Z

Zeroes, класс, инициализатор 218

A

Алгоритм:
 ◇ CART 63
 ◇ *k* ближайших соседей 49
 ◇ Виолы — Джонса 292
 ◇ обратного распространения ошибки 211
 ◇ обучения 29
 ◇ ограниченной памяти Бройдена — Флетчера — Голдфарба — Шанно 83
 ◇ подавления немаксимумов 321
 ◇ селективного поиска 321
Анализ:
 ◇ главных компонент 60, 151
 ◇ настроения 32, 107, 365
 ◇ пространственный 402
Аугментация данных 267, 272

B

Бустинг градиентный 48, 66

B

Валидация перекрестная 72
 ◇ *k*-кратная 73

Вектор:

- ◇ опорный 128
- ◇ слов 349

Векторизация 107

Верификация лица 310

Вероятность 31

Вес 207

Внимание многоголовое 369

Выброс 59

Вызов обратный 241

Выпрямленные линейные единицы 208

Выравнивание ROI 324

Г

Главная компонента набора данных 152

Глубина нейронной сети 219

Градиент исчезающий 220, 363

Граница классификационная 48

График осыпи 157

Д

Дерево:

- ◇ бинарное 61
- ◇ двоичное 61
- ◇ решений 61
- с градиентным бустингом 67

Диаграмма осыпная 157
 Диаграммы парные 60
 Дисперсия единичная 141
 Дублер двуязычной оценки 377

З

Значение категориальное 89
 Зрение компьютерное 246

И

Идентификация лиц 290
 Изображение интегральное 293
 Инерция 38
 Искусственный интеллект 33, 206

К

Классификатор:
 ◊ бинарный 90
 ◊ каскадный 292
 ◊ с закрытым набором 317
 ◊ с открытым набором 318
 Классификация:
 ◊ бинарная 90, 225
 ◊ изображений 32
 ◊ многоклассовая 100
 ◊ мультиклассовая 100
 ◊ текста 32, 354
 Кластеризация 36
 ◊ DBSCAN 46
 ◊ агломеративная 46
 ◊ по методу k средних 36
 ◊ пространственная для приложений с шумом на основе плотности 46
 Кодирование:
 ◊ горячее 89
 ◊ меток 43
 ◊ позиционное 371
 Контейнер 187
 ◊ образ 187
 Коэффициент:
 ◊ детерминации 73
 ◊ Джини 63
 ◊ обучения 68
 Кривая:
 ◊ рабочей характеристики приемника 86
 ◊ сигмоидная 82
 Кросс-валидация 72
 Кросс-энтропия бинарная 226

Л

Ландшафт потерь 212
 Лемматизация 110
 Лес случайный 48, 65

М

Маска сегментации 324
 Матрица:
 ◊ ковариационная 154
 ◊ ошибок 87, 148
 ◊ подобия 124
 Машины:
 ◊ градиентно-бустинговые 67
 ◊ опорных векторов 128
 – одноклассовые 168
 Метод:
 ◊ `analyze_image_in_stream` 406
 ◊ `corr` 76
 ◊ `describe_image` 403
 ◊ `describe_image_in_stream` 403, 404
 ◊ `extract_key_phrases` 414
 ◊ `fit` 55, 73
 ◊ `fit_on_texts` 350
 ◊ `from_estimator` 88
 ◊ `from_predictions` 88
 ◊ `inverse_transform` 111, 154
 ◊ `LoadFromTextFile` 195
 ◊ `predict` 54, 84, 185, 219
 ◊ `predict_proba` 84, 185
 ◊ `read_in_stream` 409
 ◊ `recognize_entities` 413
 ◊ `recognize_once_async` 417
 ◊ `recognize_pii_entities` 414
 ◊ `recognize_printed_text_in_stream` 409
 ◊ `score` 54, 70, 84
 ◊ `sequences_to_texts` 351
 ◊ `summary` 222
 ◊ `tag_image_in_stream` 404
 ◊ `texts_to_sequences` 350
 ◊ локтя 38
 ◊ опорных векторов 48, 69, 128
 ◊ соседей по радиусу 51
 Мешок слов 107, 361
 Моделирование:
 ◊ аддитивное 67
 ◊ ансамблевое 66
 ◊ регрессионное 57
 ◊ языка по маске 381

Модель:

- ◊ анализа настроения 365
- ◊ классификации изображений 364
- ◊ классификационная 47
 - бинарная 47, 81
 - двоичная 81
 - многоклассовая 47, 81
 - многометочная 81
 - мультиклассовая 81
- ◊ обнаружения объектов 364
- ◊ обучения:
 - контролируемого 35
 - неконтролируемого 35
- ◊ регрессионная 47

Модуль:

- ◊ NimbusML 193
- ◊ pickle 181

Мультиколлинеарность 59**Н****Набор данных:**

- ◊ California housing dataset 71
- ◊ COCO 326
- ◊ Digits 166, 167
- ◊ ImageNet 260
- ◊ Iris 52
- ◊ Labeled Faces in the Wild (LFW) 33, 143 154, 235, 302
- ◊ MNIST 251
- ◊ Optical Recognition of Handwritten Digits 102, 164
- ◊ SQuAD 2.0 383
- ◊ Titanic 91
- ◊ UrbanSound8K 282
- ◊ для прогнозирования отказов подшипников 172
- ◊ кинокритических IMDB 111, 358
- ◊ морские черепахи 338
- ◊ о фильмах 124
- ◊ по раку молочной железы 138, 162
- ◊ примеры 52
- ◊ со звуками тропических лесов 282
- ◊ спамерских писем 119, 355
- ◊ транзакций по кредитным картам 96, 168

Наивный Байес 115, 225**Настройка гиперпараметров 127, 135****Нейрон 206****Нормализация:**

- ◊ Z-score 141
- ◊ данных 60
- ◊ пакетная 261

О**Область интереса 323****Обнаружение:**

- ◊ аномалий 167
 - многомерных 178
- ◊ лиц 290, 291

Обработка естественного языка 206, 349**Образ контейнера 187****Обучение 29**

- ◊ ансамблевое 66
- ◊ без учителя 35
- ◊ глубокое 34, 205, 206
- ◊ инкрементное 241
- ◊ контролируемое 47, 168
 - модели 47
- ◊ машинное 29, 33
- ◊ непрерывное 241
- ◊ с учителем 35, 168
- ◊ трансферное 193, 247, 264, 265

Окклюзия 342**Оптимизатор 136, 211****Отсев 238****Оценка объектности 324****Оценщик 142****Ошибка:**

- ◊ восстановления 168
- ◊ среднеквадратическая 58, 74, 217
- ◊ средняя абсолютная 217

П**Пакет:**

- ◊ Microsoft.ML.OnnxRuntime 190
- ◊ NuGet 190
- ◊ Onnxruntime 189
- ◊ Skl2onnx 189

Память долгая краткосрочная 363**Перевод:**

- ◊ машинный на основе правил 366
- ◊ нейромашинный 366

Переобучение 64, 279**Перцептрон многослойный 206****Пни деревьев решений 67****Поиск селективный 321****Показатель пересечения и объединения 321**

- Полнота 85
- Потеря:
- ◊ кросс-энтропийная 226
 - ◊ логарифмическая 226
- Правильность 84
- Представление данных 195
- Произведение:
- ◊ скалярное 131
 - ◊ тензорное точечное (скалярное) 254
- Производная частная 59
- Процессор:
- ◊ графический 35, 205
 - ◊ тензорный 205
- Пулинг:
- ◊ ROI 323
 - ◊ глобальный 280
 - ◊ максимальный 249
 - ◊ средний 249
- ## Р
- Рамка якорная 324
- Распознавание лиц 290, 302
- Расстояние:
- ◊ евклидово 51
 - ◊ манхэттенское 51
 - ◊ Минковского 51
- Регион интереса 323
- Регрессия:
- ◊ линейная 48, 57
 - множественная 59
 - простая 59
 - ◊ логистическая 82, 225
 - мультиномиальная 101
 - ◊ по методу наименьших квадратов 58
- Регуляризация 59
- Реестр контейнера 188
- ## С
- Самовнимание 369
- Сглаживание:
- ◊ аддитивное 117
 - ◊ Лапласа 117
- Сегментация экземпляров 324
- Сеть:
- ◊ выходная 296
 - ◊ генеративная состязательная 206
 - ◊ нейронная:
 - глубина 219
 - глубокая 206
 - для бинарной классификации 226, 229
 - для регрессии 222
 - обучение 211
 - рекуррентная 206, 362
 - сверточная 206, 247
 - многозадачная каскадная 296
 - предварительно обученная 260
 - ширина слоя 219
 - ◊ предложений 296
 - областей 323
 - ◊ усовершенствований 296
- Система:
- ◊ обучения с подкреплением 34
 - ◊ рекомендательная 122
 - ◊ рекомендаций:
 - коллаборативная 122
 - контентная 122
 - основанная на популярности 122
 - ◊ экспертная 33
- Словарь 108, 350
- Слой:
- ◊ Conv1D 361
 - ◊ Conv2D 219
 - ◊ Dropout 219
 - ◊ MaxPooling1D 361
 - ◊ RepeatVector 367
 - ◊ TimeDistributed 367
 - ◊ TokenAndPositionEmbedding 371
 - ◊ TransformerDecoder 371
 - ◊ TransformerEncoder 371
 - ◊ классификационный 250
 - ◊ объединяющий 247, 249
 - ◊ остаточный 261
 - ◊ плотный 250
 - ◊ полносвязный 207
 - ◊ пулинговый 247, 249
 - ◊ рекуррентный 362
 - ◊ сверточный 247
 - ◊ скрытый 206
 - ◊ субдискретизации 247
 - ◊ ширина 219
 - ◊ эмбединговый 349, 353
- Смещение 207
- Спам-фильтр 35
- Спектрограмма 281
- Специфичность 88
- Спуск стохастический градиентный 213
- Стандартизация 141
- Стемминг 110

Столбец:

- ◊ категориальный 89
- ◊ меток 30
- ◊ признаков 30

Стохастическое вложение соседей с t -распределением 60, 165

Сходство косинусное 123

Т

Тензор 215

Теорема:

- ◊ аппроксимации 208
- ◊ Байеса 115
- ◊ Ковера 129

Токен 350

Точность 85

- ◊ средняя 344

Трансформер 349

Трюк ядерный 69, 131

У

Узел 206

- ◊ листовой 62

Ф

Файл:

- ◊ bearings.csv 172
- ◊ creditcard.csv 96, 168, 228
- ◊ customers.csv 39
- ◊ Dockerfile 188
- ◊ en-fr.txt 372
- ◊ ham-spam.csv 119
- ◊ movies.csv 124
- ◊ reviews.csv 112
- ◊ sentiment.pkl 185
- ◊ taxi-fares.csv 221
- ◊ titanic.csv 91
- ◊ vggface.h5 308, 313
- ◊ изображений белых медведей 256
- ◊ с изображениями лиц 312

Формат:

- ◊ H5 240
- ◊ SavedModel 240

Функция:

- ◊ binary_crossentropy 226
- ◊ confusion_matrix 88
- ◊ cosine_similarity 123, 311
- ◊ cross_val_score 73
- ◊ f1_score 85
- ◊ fit 54

- ◊ load_iris 52
- ◊ logit 82
- ◊ make_blobs 37
- ◊ make_pipeline 142, 182
- ◊ MobileNetV2 284
- ◊ pairplot 60
- ◊ precision_score 85
- ◊ recall_score 85
- ◊ roc_auc_score 87
- ◊ scatter 37
- ◊ softmax 101
- ◊ train_test_split 53, 70, 218
- ◊ активации 207
 - relu 216
 - sigmoid 216, 225, 233
 - softmax 216, 233
 - tanh 216
- ◊ логистическая 82
- ◊ определяемая пользователем 197
- ◊ потеря:
 - categorical_crossentropy 235
 - sparse_categorical_crossentropy 233

Ч

Чувствительность 88

Ш

Ширина слоев нейронной сети 219

Шум 159

Э

Эмбединг:

- ◊ лица 310
- ◊ позиционный 371
- ◊ слов 349

Энтропия перекрестная 226

Эпоха 217

Эстиматор 142

Я

Ядро 248

- ◊ RBF 131
- ◊ линейное 131
- ◊ полиномиальное 131
- ◊ сверточное 247
- ◊ сигмоидное 131
- ◊ Собеля нижнее 248

Якорь 324

Джеф Просиз

Прикладное машинное обучение и искусственный интеллект для инженеров

Перевод с английского И. Донченко

ТОО "АЛИСТ"
010000, Республика Казахстан,
г. Астана, пр. Сарыарка, д. 17, ВП 30

Подписано в печать 14.05.24.
Формат 70×100¹/₁₆. Печать офсетная. Усл. печ. л. 34,83.
Тираж 1200 экз. Заказ № 9436.

Отпечатано с готового оригинал-макета
ООО "Принт-М", 142300, РФ, М.О., г. Чехов, ул. Полиграфистов, д. 1

Прикладное машинное обучение и искусственный интеллект для инженеров

В то время как многие руководства по искусственному интеллекту (ИИ) представляют собой скорее учебники по математике, в этой книге математики практически нет. Вместо этого автор помогает инженерам и разработчикам программного обеспечения интуитивно понять и использовать ИИ для решения технических и бизнес-задач. Эта книга научит вас практическим навыкам, необходимым для внедрения ИИ и машинного обучения в вашей компании.

В книге приводятся примеры и иллюстрации из курсов по искусственному интеллекту и машинному обучению, которые автор преподает в компаниях и исследовательских институтах по всему миру. Здесь нет пустых слов и страшных уравнений — только полезная информация для инженеров и разработчиков программного обеспечения, дополненная практическими примерами.

Эта книга поможет вам:

- узнать, что такое машинное обучение и глубокое обучение;
- понять, как работают популярные алгоритмы машинного обучения и когда их следует применять;
- построить модели машинного обучения на языке Python с помощью библиотеки Scikit-Learn, а также создать нейронные сети, используя библиотеки Keras и TensorFlow;
- обучать и оценивать регрессионные модели, а также модели бинарной и многоклассовой классификации;
- создавать модели распознавания лиц и обнаружения объектов;
- строить языковые модели, отвечающие на естественно-языковые вопросы и переводящие текст на другие языки;
- использовать набор облачных API Cognitive Services для внедрения ИИ в создаваемые вами приложения.

«Если вы хотите понять, как на самом деле работают искусственный интеллект и машинное обучение, как развивались и развиваются эти технологии, прочитайте эту книгу».

— **Тодд Файн**,
директор по стратегии, Atmosera

«Чтение этой книги вызовет у вас желание пойти и что-нибудь построить».

— **Дэг Тернур**,
специалист по облачной платформе Azure, Microsoft

Джеф Просиз — инженер, чья страсть — знакомить других инженеров и разработчиков программного обеспечения с чудесами искусственного интеллекта и машинного обучения. Соучредитель компании Wintellec, он написал девять книг и сотни статей, обучил тысячи разработчиков в Microsoft и выступал на крупнейших мировых конференциях. До этого Джеф занимался лазерными системами и исследованиями в области термоядерной энергетики в Национальной лаборатории Оук-Ридж и Ливерморской национальной лаборатории имени Лоуренса. После продажи своей фирмы в 2021 г. занимает должность директора по обучению в компании Atmosera, где он помогает клиентам внедрять искусственный интеллект в их продукты.

ISBN 978-601-09-5051-1



9 786010 950511